

Andreas Krall (Ed.)

LNCSE 2826

Software and Compilers for Embedded Systems

7th International Workshop, SCOPES 2003
Vienna, Austria, September 2003
Proceedings



Springer

Lecture Notes in Computer Science

2826

Edited by G. Goos, J. Hartmanis, and J. van Leeuwen

Springer

Berlin

Heidelberg

New York

Hong Kong

London

Milan

Paris

Tokyo

Andreas Krall (Ed.)

Software and Compilers for Embedded Systems

7th International Workshop, SCOPES 2003
Vienna, Austria, September 24-26, 2003
Proceedings



Springer

Series Editors

Gerhard Goos, Karlsruhe University, Germany
Juris Hartmanis, Cornell University, NY, USA
Jan van Leeuwen, Utrecht University, The Netherlands

Volume Editor

Andreas Krall
Technische Universität Wien
Institut für Computersprachen
Argentinierstr. 8, 1040 Wien, Austria
E-mail: andi@complang.tuwien.ac.at

Cataloging-in-Publication Data applied for

A catalog record for this book is available from the Library of Congress

Bibliographic information published by Die Deutsche Bibliothek
Die Deutsche Bibliothek lists this publication in the Deutsche Nationalbibliographie;
detailed bibliographic data is available in the Internet at <<http://dnb.ddb.de>>.

CR Subject Classification (1998): D.3, D.4, D.2, D.1, C.3, C.2

ISSN 0302-9743

ISBN 3-540-20145-9 Springer-Verlag Berlin Heidelberg New York

This work is subject to copyright. All rights are reserved, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, re-use of illustrations, recitation, broadcasting, reproduction on microfilms or in any other way, and storage in data banks. Duplication of this publication or parts thereof is permitted only under the provisions of the German Copyright Law of September 9, 1965, in its current version, and permission for use must always be obtained from Springer-Verlag. Violations are liable for prosecution under the German Copyright Law.

Springer-Verlag Berlin Heidelberg New York
a member of BertelsmannSpringer Science+Business Media GmbH

<http://www.springer.de>

© Springer-Verlag Berlin Heidelberg 2003
Printed in Germany

Typesetting: Camera-ready by author, data conversion by Christian Grosche, Hamburg
Printed on acid-free paper SPIN: 10953648 06/3142 5 4 3 2 1 0

Preface

This volume contains the proceedings of the 7th International Workshop on Software and Compilers for Embedded Systems, SCOPES 2003, held in Vienna, Austria, September 24–26, 2003. Initially, the workshop was referred to as the International Workshop on Code Generation for Embedded Systems. The first workshop took place in 1994 in Schloss Dagstuhl, Germany. From its beginnings, the intention of the organizers was to create an atmosphere in which the researchers could participate actively in dynamic discussions and profit from the assembly of international experts in the field.

It was at the fourth workshop, in St. Goar, Germany, in 1999, that the spectrum of topics of interest for the workshop was extended, and not only code generation, but also software and compilers for embedded systems, were considered.

The change in fields of interest led to a change of name, and this is when the present name was used for the first time. Since then, SCOPES has been held again in St. Goar, Germany, in 2001; Berlin, Germany, in 2002; and this year, 2003, in Vienna, Austria.

In response to the call for papers, 43 very strong papers from all over the world were submitted. The program committee selected 26 papers for presentation at SCOPES 2003. All submitted papers were reviewed by at least three experts in order to ensure the quality of the work presented at the workshop.

The papers were divided for presentation into the following categories: code size reduction, code selection, loop optimizations, automatic retargeting, system design, register allocation, offset assignment, analysis and profiling, and memory and cache optimizations.

In addition to the selected contributions, an invited talk was given by Jim Dehnert from Transmeta Corporation. An abstract of this talk is also included in this volume.

I would like to thank all the authors who submitted papers for consideration, since none of this would have been possible without their research efforts. I would like to gratefully acknowledge the support of our sponsor, Atair Software. I thank the program committee and all the referees for carefully reviewing the submitted papers. Finally, I thank Nerina Bermudo and Ulrich Hirnschrott for dealing with the local organization, compiling the proceedings, and maintaining the web site.

Organization

SCOPES 2003 was organized by the Institut für Computersprachen, Technische Universität Wien and CD-Lab Compilation Techniques for Embedded Processors in cooperation with EDAA, sponsored by Atair Software.

Committee

General Chair:

Andreas Krall (Technische Universität Wien, Austria)

Program Committee:

Uwe Assmann (Linköping University, Sweden)
Shuvra S. Bhattacharyya (University of Maryland, USA)
Christine Eisenbeis (INRIA, France)
Antonio González (Universitat Politècnica de Catalunya
and Intel Labs, Spain)
David Gregg (Trinity College Dublin, Ireland)
Rajiv Gupta (University of Arizona, USA)
Seongsoo Hong (Seoul National University, Korea)
Nigel Horspool (University of Victoria, Canada)
Masaharu Imai (Osaka University, Japan)
Ahmed Jerraya (IMAG, France)
Rainer Leupers (RWTH Aachen, Germany)
Annie Liu (SUNY Stony Brook, USA)
Peter Marwedel (Universität Dortmund, Germany)
SangLyul Min (Seoul National University, Korea)
Frank Mueller (North Carolina State University, USA)
Tatsuo Nakajima (Waseda University, Japan)
Alex Nicolau (Univ. California at Irvine, USA)
Yunheung Paek (Seoul National University, Korea)
Hans van Someren (ACE, The Netherlands)
Hiroyuki Tomiyama (Nagoya University, Japan)
Sreeranga P. Rajan (Fujitsu Labs, USA)
Bernard Wess (Technische Universität Wien, Austria)
David Whalley (Florida State University, USA)
Reinhard Wilhelm (Saarland University, Germany)

Local Organization:

Nerina Bermudo (Technische Universität Wien, Austria)
Ulrich Hirschrott (Technische Universität Wien, Austria)

Referees

Alex Aletà
 Sid Ahmed Ali Touati
 Cédric Bastoul
 Marcel Beemster
 Christoph Berg
 Nerina Bermudo
 Ramon Canal
 Bruce Childers
 Junghoon Cho
 Josep M. Codina
 Albert Cohen
 Bjoern Decker
 Heiko Falk
 Nico Fritz
 Liam Fitzpatrick
 Enric Gilbert
 Rajiv Gupta
 Sang-il Han

Michael Hind
 Manuel Hohenauer
 Nagisa Ishiura
 Martien de Jong
 Dae-hwan Kim
 Saehwa Kim
 Toru Kisuki
 Jens Knoop
 Shinsuke Kobayashi
 Arvind Krishaswamy
 Marc Langenbach
 ChokSheak Lau
 Jaesoo Lee
 Bengu Li
 Markus Lorenz
 Jadeep Marathe
 Christopher Milner
 Bryan Olivier

Santosh Pande
 Jiyong Park
 Ruben van Royen
 Jesus Sanchez
 Jun Sato
 Kiran Seth
 Viera Sipkova
 Sriraman Tallam
 Stephan Thesing
 François Thomasset
 Xavier Vera
 Jens Wagner
 Oliver Wahlen
 Lars Wehmeyer
 Sebastian Winkel
 Kwangkeun Yi
 Thomas Zeitlhofer

Table of Contents

Invited Talk

The Transmeta Crusoe: VLIW Embedded in CISC	1
<i>James C. Dehnert</i>	

Code Size Reduction

Limited Address Range Architecture for Reducing Code Size in Embedded Processors	2
<i>Qin Zhao, Bart Mesman, and Henk Corporaal</i>	
Predicated Instructions for Code Compaction	17
<i>Warren Cheung, William Evans, and Jeremy Moses</i>	
Code Generation for a Dual Instruction Set Processor Based on Selective Code Transformation	33
<i>Sheayun Lee, Jaejin Lee, Sang Lyul Min, Jason Hiser, and Jack W. Davidson</i>	

Code Selection

Code Instruction Selection Based on SSA-Graphs	49
<i>Erik Eckstein, Oliver König, and Bernhard Scholz</i>	
A Code Selection Method for SIMD Processors with PACK Instructions .	66
<i>Hiroaki Tanaka, Shinsuke Kobayashi, Yoshinori Takeuchi, Keishi Sakanushi, and Masaharu Imai</i>	
Reconstructing Control Flow from Predicated Assembly Code	81
<i>Björn Decker and Daniel Kästner</i>	

Loop Optimizations

Control Flow Analysis for Recursion Removal	101
<i>Stefaan Himpe, Francky Catthor, and Geert Deconinck</i>	
An Unfolding-Based Loop Optimization Technique	117
<i>Litong Song, Krishna Kavi, and Ron Cytron</i>	
Tailoring Software Pipelining for Effective Exploitation of Zero Overhead Loop Buffer	133
<i>Gang-Ryung Uh</i>	

Automatic Retargeting

Case Studies on Automatic Extraction of Target-Specific Architectural Parameters in Complex Code Generation	151
<i>Yunheung Paek, Minwook Ahn, and Soonho Lee</i>	

Extraction of Efficient Instruction Schedulers from Cycle-True Processor Models	167
<i>Oliver Wahlen, Manuel Hohenauer, Gunnar Braun, Rainer Leupers, Gerd Ascheid, Heinrich Meyr, and Xiaoning Nie</i>	

System Design

A Framework for the Design and Validation of Efficient Fail-Safe Fault-Tolerant Programs	182
<i>Arshad Jhumka, Neeraj Suri, and Martin Hiller</i>	

A Case Study on a Component-Based System and Its Configuration	198
<i>Hiroo Ishikawa and Tatsuo Nakajima</i>	

Composable Code Generation for Model-Based Development	211
<i>Kirk Schloegel, David Oglesby, Eric Engstrom, and Devesh Bhatt</i>	

Code Generation for Packet Header Intrusion Analysis on the IXP1200 Network Processor	226
<i>Ioannis Charitakis, Dionisios Pnevmatikatos, Evangelos Markatos, and Kostas Anagnostakis</i>	

Register Allocation

Retargetable Graph-Coloring Register Allocation for Irregular Architectures	240
<i>Johan Runeson and Sven-Olof Nyström</i>	

Fine-Grain Register Allocation Based on a Global Spill Costs Analysis...	255
<i>Dae-Hwan Kim and Hyuk-Jae Lee</i>	

Offset Assignment

Unified Instruction Reordering and Algebraic Transformations for Minimum Cost Offset Assignment	270
<i>Sarvani V.V.N.S. and R. Gorvindarajan</i>	

Improving Offset Assignment through Simultaneous Variable Coalescing .	285
<i>Desiree Ottoni, Guilherme Ottoni, Guido Araujo, and Rainer Leupers</i>	

Analysis and Profiling

Transformation of Meta-information by Abstract Co-interpretation	298
<i>Raimund Kirner and Peter Puschner</i>	

Performance Analysis for Identification of (Sub-)Task-Level Parallelism in Java	313
<i>Richard Stahl, Robert Paško, Luc Rijnders, Diederik Verkest, Serge Vernalde, Rudy Lauwereins, and Francky Catthoor</i>	
Towards Superinstructions for Java Interpreters	329
<i>Kevin Casey, David Gregg, M. Anton Ertl, and Andrew Nisbet</i>	
Memory and Cache Optimizations	
Data Partitioning for DSP Software Synthesis	344
<i>Ming-Yung Ko and Shuvra S. Bhattacharyya</i>	
Efficient Variable Allocation for Dual Memory Banks of DSPs	359
<i>Viera Sipkova</i>	
Cache Behavior Modeling of Codes with Data-Dependent Conditionals ..	373
<i>Diego Andrade, Basilio B. Fraguera, and Ramón Doallo</i>	
FICO: A Fast Instruction Cache Optimizer	388
<i>Marco Garatti</i>	
Author Index	403

The Transmeta Crusoe: VLIW Embedded in CISC

James C. Dehnert

Transmeta Corporation
dehnert@transmeta.com

Abstract. Transmeta's Crusoe microprocessor is a full, system-level implementation of the x86 architecture, comprising a native VLIW microprocessor with an embedded software layer, the Code Morphing Software (CMS). CMS combines an interpreter, dynamic binary translator, optimizer, and run-time system. By moving infrequently-used or complex functionality to CMS, Crusoe achieves a much simpler hardware implementation than would otherwise be possible, while improving flexibility. This makes it attractive for such applications as low-power laptops and notebooks or embedded systems such as printers, while still allowing the use of widely available standard software and development tools.

In its general structure, CMS resembles other binary translation systems described in the literature, but it is unique in several respects. It must robustly handle the full range of x86 workloads, with the performance of a hardware-only implementation as well as full system-level x86 compatibility. This exposes issues that have received little or no attention in the binary translation literature, such as exceptions and interrupts, I/O, DMA, and self-modifying code.

This talk will begin with an overview of the Crusoe system and some observations concerning its unique characteristics. We will then discuss several challenges raised by the issues above, and present some of the techniques developed in Crusoe and CMS to meet those challenges, especially the Crusoe paradigm of aggressive speculation, recovery to a consistent x86 state using unique hardware commit-and-rollback support, and adaptive retranslation when exceptions occur too often to be handled efficiently by interpretation. Finally, we will discuss novel techniques used to test and debug the software embedded in this unusual microprocessor.

Limited Address Range Architecture for Reducing Code Size in Embedded Processors

Quin Zhao¹, Bart Mesman^{1,2}, and Henk Corporaal^{1,3}

¹ Eindhoven University of Technology, Department of Electrical Engineering
P.O. Box 513, NL-5600 MB Eindhoven, The Netherlands

² Philips Research Laboratories, Eindhoven

Prof. Holstlaan 4, NL-5656 AA Eindhoven, The Netherlands

³ IMEC, Kapeldreef 75, B-3001 Leuven, Belgium

Abstract. In embedded systems a processor core must be designed with low power consumption, low cost and small silicon area in mind since program code often resides in on-chip ROM. To obtain small code size, not only the amount of instruction-level parallelism can be restricted by instruction sets, but also the encoding cost can be reduced by restricting the access to register files. However, communication among register files has to be supported by hardware, e.g. buses and wires, and compilers. In this paper, we propose a new type of architecture by limiting the encoding range to a subset of registers in a register file on the one hand, and keeping the overlap among different ranges on the other hand in order to support communication between all the functional units. We also propose the annotated conflict graph approach for modeling the range constraints in this architecture, which can be applied in combination with any scheduler. However, to overcome the phase coupling between address range assignment and scheduling in code generation, in this paper the address range constraints are transformed and integrated with the existing timing, resource and register file constraints. Constraint analysis techniques [9] are adapted to prune the search spaces based on those constraints. Results show that we can reduce code size up to 24.58% by applying our technique.

1 Introduction

An increasingly common architecture for embedded systems is to integrate processor core, program ROM/RAM, application specific integrated circuit (ASIC) and peripherals on a single chip. The processor core can be a microprocessor, a digital signal processor (DSP) or an application specific instruction-set processor (ASIP). Since program code resides in on-chip ROM, code size is translated into silicon area and cost directly, which has to be kept small in order to keep the power consumption low and limit the design cost.

To obtain small code size in DSPs and ASIPs, those processors often employ a lot of irregularities in the architecture, e.g. heterogeneous register sets, a small number of very specialized registers, very specialized functional units, restricted connectivity, limited addressing, and highly irregular datapaths. The

use of conventional code generation techniques and compilers often produce very inefficient code [12] for these architectures. Therefore, in order to meet the given constraints with respect to execution time, code size and energy consumption, often the critical paths of programs are written in assembly code by hand. This heavily reduces the portability and maintainability of the generated code. Due to the increasing complexity in digital signal processing, high-level compilation is desirable.

Conventional general-purpose VLIW architectures usually employ a simple central register file for local storage. While this is convenient for compilation, the use of a central register file contributes to large code size, high power consumption and long execution delay. New designs usually partition the architecture into clusters and each cluster contains one small local register file [3] [6]. The code size is reduced with a large amount. However, the communication of values among different clusters has to be supported by extra hardware, including buses, and probably separate copy operations have to be inserted [5] [7]. The latter complicates the design of a compiler and is not coherent with the small code size purpose.

In this paper, we propose a new type of architecture, the so-called limited address range (LAR) architecture for the purpose of reducing code size in embedded processors. Since encoding operands is costly in instructions, which is directly related to the number of registers, we restrict the encoding range to a subset in a register file. Instead of treating the subsets as independent register files, we allow overlap among different ranges. Communication between functional units can be put into the commonly addressable registers. This will reduce the communication cost significantly. However, it will introduce a new phase called address range assignment. So, although the LAR architecture can be used with any scheduler, the additional phase coupling may yield inferior results. Therefore we also consider in this paper an approach that integrates the address range constraints with timing, resource and register file constraints. Efficient search space pruning techniques are used to prevent decisions that inevitably lead to violations with those constraints.

The rest of the paper is organized as follows. Section 2 discusses the related work. Section 3 gives the definition of a conflict graph and an annotated conflict graph. In Section 4 an example architecture is depicted to show the reduction in code size. Section 5 discusses the conflict graph construction for conventional register allocation first. Then it is modified to an annotated conflict graph for an LAR architecture and is applied to the conventional graph coloring. In Section 6 a worst-case conflict graph containing not only potential lifetime conflicts, but also assignment decisions, is introduced to include all the possible constraints before scheduling. Section 7 gives the problem definition and the global approach of the code generation for the LAR architecture. The experimental results are given in Section 8. Finally, Section 9 concludes the work and discusses the future topic.

2 Related Work

Although a lot of work has been done on reducing the code size in VLIW architectures by partitioning the data path into clusters with local registers [3] [6], little work has been done on using global registers for the communication among clusters. MAJC, a scalable microprocessor from SUN [8], is one of them. In MAJC, each functional unit is self-contained and has a local register file, local wiring, local control (e.g. instruction decode logic) and state information. Functional units share global registers in a processor unit. Local registers specific to a functional unit are not accessible to other functional units. The global register file size is variable from 32 to 512 and implementation specific.

The MAJC architecture does not require much modification for the conventional compiler, since the number of registers in the global register file is normally enough for transferring values among functional units. In addition, the functional units and general purpose register files in the MAJC architecture are data type agnostic. This provides more registers for applications that involve dedicated data type processing and significantly improves the performance. Further, it provides the compiler with the flexibility to allocate any type of data to any register. However, the general-purpose processor is not very applicable for System-on-Chips (SoCs) because of large area and large power consumption.

In Cydra 5 [10], the context register matrix, which is a matrix of certain amount of registers, is provided to dynamically allocate iterations of loops at run-time. Each iteration is allocated in an iteration frame. Since the number of registers in the context register matrix is finite, the iteration frames for past iterations must be deallocated at the same rate as the new frames. This is necessary for loop variants. However, for loop invariants, which are only used but never computed, this will cause them to be overwritten unless they are copied in each iteration. To avoid those copies, General Purpose Register file (GPR) is provided with global registers to all the iterations. In a single cycle, it can be read by any number of functional unit input ports, but can be written only by one output port of the functional units.

3 Definitions

Conflict graphs and graph coloring algorithms are used frequently for register allocation in embedded processors. In this section, we give the related preliminaries.

Definition 1: A *conflict graph* (CG) $CG = (V^c, E^c)$ is an undirected graph, where

- V^c is the set of vertices representing values to be allocated in a register file, and
- $E^c \subset V^c \times V^c$ is a set of edges. There is an edge $(u, v) \in E^c$ if there is a conflict between u and v .

Vertex coloring of a graph consists of assigning a color to every vertex such that no two adjacent vertices have the same color. *Exact coloring* refers to the coloring using the minimum number of colors and this minimum number of colors is called the *chromatic number*.

We like to express in this graph model, the limitations on the colors available for each vertex. Therefore we introduce the annotated conflict graph.

Definition 2: An *Annotated Conflict Graph (ACG)* is an undirected graph represented by a tuple $(V^c, E^c, c, \mathcal{Z})$, where

- V^c is the set of vertices,
- $E^c \subset V^c \times V^c$ is a set of edges denoting conflicts,
- $\mathcal{Z}$ is a set of colors, and
- the mapping $c(v) \rightarrow 2^{\mathcal{Z}}$ defines the “label set” for each $v \in V^c$.

4 Limited Address Range Architecture

In this section, we generalize the idea of reducing the code size by limiting the encoding address and propose the so-called limited address range (LAR) architecture. In this architecture, the number of registers encoded in the instruction set for a functional unit is reduced to a subset of registers. The subsets can be different for different functional units or clusters. For multiple address ranges, we allow certain overlap among the subsets. The advantage of this approach is that communication among functional units or clusters which has to be performed on buses explicitly are transferred to the communication on the overlapping registers implicitly. As a consequence, code size is reduced not only by reducing the encoding ranges, but also by avoiding extra move operations.

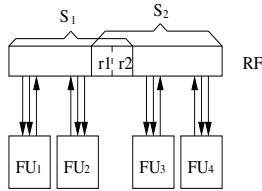


Fig. 1. Limited address range architecture

Figure 1 illustrates an example of the LAR architecture. In this figure, register file RF has 14 registers and is grouped into two subsets S_1 and S_2 , each containing 8 registers. Functional units FU_1 and FU_2 read and write values within range S_1 , and functional units FU_3 and FU_4 read and write values within range S_2 . At this moment, we assume that values have already been assigned to different ranges, and range assignment, which is a similar concept to cluster partitioning, has been performed. Later in Section 6, this assumption is released. In many clustered architectures, there is only one functional unit per type in a cluster.

Therefore assigning operations to functional units [2] is trivial after partitioning. Otherwise, operation assignment still has to be performed within one range or cluster. Range S_1 and range S_2 share registers $r1$ and $r2$. Values produced by FU_1 or FU_2 and consumed by FU_3 or FU_4 are stored in $r1$ or $r2$, and vice versa. No extra hardware is needed and no extra move operations have to be inserted in the compiler unless the number of overlapping registers is not large enough for the communication. Suppose that an opcode is encoded with 5 bits. An instruction with three operands will cost 17 bits for the central register file architecture. Alternatively, it will cost 14 bits for the LAR architecture. Thus the code size saving of one instruction is 17.65%, which will cause a reduction in the total code size and power consumption.

The concept of the LAR architecture is not limited to address ranges in one central register file. It can also be extended to multiple register files. It is in fact coherent with the multiple distributed register file architectures with limited connectivities. In these architectures, for the sake of saving the cost of connectivity, a functional unit is connected to a few register files instead of all the register files. In this situation, the set of register files connected to one functional unit can be viewed as a address range in LAR. In this paper we always assume that the limited overlapping registers is sufficient for transferring data among functional units and no insertion of move operations is needed. This assumption implies that every pair of functional units should have overlap. In practice, this can be relieved by making a few registers accessible by all the functional units.

5 Register Binding after Scheduling for LAR Architectures

Graph coloring is frequently used for register allocation in general-purpose and embedded processors. The general approach is to analyze the lifetimes of values in an application and assign values having a lifetime conflict with different colors, thus into different registers.

Using the exact graph coloring algorithm in [4], one color is assigned to a node in the conflict graph respecting to the lifetime conflicts. There is no restriction on which color to use until all the colors are used up for the maximum clique. When a value is limited to certain address range, it implies that it can only be assigned with limited colors in the conflict graph. Therefore the original conflict graph has to be adapted to these constraints.

The example in Figure 2 shows this limitation. A conflict graph is constructed in Figure 2 (b) for the DFG in Figure 2 (a) according to the lifetime analysis. By coloring this graph, we obtain that maximally three colors (registers) are needed for allocating all the values. We know, for example, value a cannot be put into the same register as value b , but there is no limitation on which concrete register value a can reside. All the possible allocation results are shown in Figure 2 (c), (d), (e), (f), (g), (h). Assume that an LAR architecture is employed with three registers; further assume that values a and c can only reside in registers 1 or 2 (as a result of the given schedule and functional unit assignment), and values b and d can only reside in registers 2 or 3. For each value we collect all

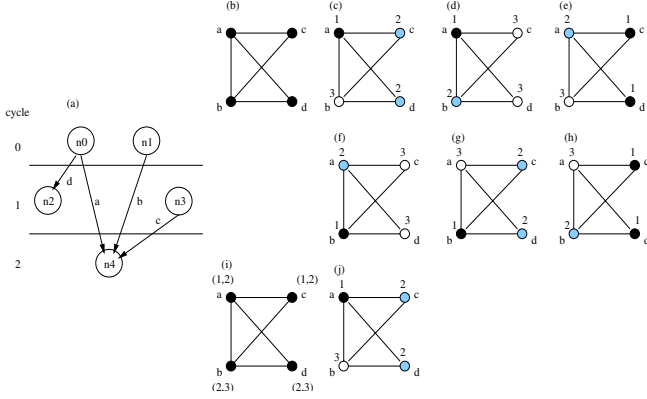


Fig. 2. Data flow graph, conflict graph, and annotated conflict graph

the labels of the possible registers in which it can reside and assign a label set to the corresponding node. Therefore, label set $(1, 2)$ is associated with node a and c , and label set $(2, 3)$ is associated with node b and d . The annotated conflict graph (ACG) is constructed as in Figure 2 (i). For this simple example, we can see immediately that only one allocation result is possible and it is given in Figure 2 (j).

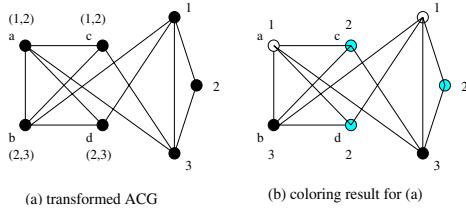


Fig. 3. Coloring for transformed ACG

The ACG cannot be applied to graph coloring algorithms directly, since coloring algorithms focus only on the conflicts and cannot recognize the labels. But the ACG can be transformed by modeling all the label sets explicitly as conflicts in the conflict graph. The transformation is performed in the following way: a set of dummy nodes $R = \{r_i, i = 1, \dots, k\}$, where k is the number of registers in the register file, is included in the ACG. Each one represents an existing register. Each pair of the dummy nodes have a conflict. This is obvious, since the registers are physically independent and need different colors. The label set of node u in the ACG is denoted as l_u . The conflict c_{u,r_i} between node u and a dummy node r_i is

$$c_{u,r_i} = \begin{cases} 1 & \text{if } r_i \notin l_u \\ 0 & \text{if } r_i \in l_u \end{cases}$$

The transformed ACG is depicted in 3 (a) for the ACG in Figure 2 (i). Notice that value a has a conflict with register 3 since the label set for node a , i.e. $\{1, 2\}$ does not include 3. After coloring the transformed ACG, we obtain the register allocation result in Figure 3 (b), which is the same as in Figure 2 (j).

6 Integrated Scheduling and Register Binding for LAR Architectures

In the previous section we showed how to do register binding for LAR architectures *after* scheduling. A conflict graph is constructed according to the lifetime analysis between each pair of values in the DFG, as the one depicted in Figure 2 (b). However, this approach ignores the obvious phase coupling between scheduling on the one hand, and address assignment (and register binding) on the other hand. Ignoring this phase coupling may yield inferior results. Therefore we also introduce an integrated approach in this section.

Note that in this integrated approach lifetimes are not fixed prior to register binding. In order to express in the conflict graph both real conflicts and potential conflicts, in [1] three different relations are classified.

- Strong conflict: values u and v have strong conflict if their lifetimes overlap for sure. There is overlap between u and v iff the production of value v is before the consumption of value u **and** the production of value u is before the consumption of value v .
- No conflict: values u and v have no conflict if their lifetimes can never overlap. There is no overlap between u and v iff the consumption of value v is before the production of value u **or** the consumption of value u is before the production of value v .
- Weak conflict: values u and v have weak conflict if neither of the above conditions holds. It means that u and v may potentially have conflict, and this depends on the scheduling decisions.

The worst-case conflict graph (WCCG), which is used to capture all the potential conflicts in the worst case, is constructed containing all the weak and strong conflicts. The WCCG contains all the freedom for register allocation since it includes all the possible lifetime conflicts before a scheduling decision is made. The best-case conflict graph (BCCG) is constructed only for strong conflicts. By coloring the WCCG, the requirement for registers in the worst case is obtained without noticing the register file constraints. It can be used further for identifying the bottlenecks in lifetime conflicts to be solved later on. The BCCG is constructed to reason the feasibility regarding to the capacity of the register file $C(RF)$. It implies that there is no more freedom for register allocation with the amount of registers available.

An example of WCCG is depicted in Figure 4 (b) for the example DFG in Figure 4 (a). In contrast to the DFG in Figure 2 (a), this DFG is not yet scheduled. Weak conflicts are drawn as dashed edges, and strong conflicts are drawn as solid edges. Since the DFG is not scheduled yet, we conclude that for example, values a and e have weak conflict from the lifetime analysis. This is because if node $n4$ is schedule together with node $n0$ at the cycle zero, then a and e have a conflict. While if node $n4$ is scheduled at the same cycle or later than node $n2$, then this conflict will not exist.

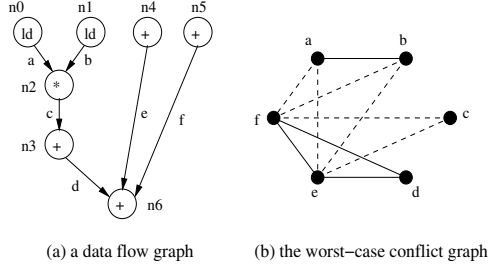


Fig. 4. Data flow graph and worst-case conflict graph

Similarly, when we apply this idea to the LAR architectures, the assignment of values to different address ranges may have some effect on their lifetimes, even cause lifetime conflicts. Therefore it may affect the result of register binding and the scheduling decisions greatly. If assignment decisions are made totally before register binding and scheduling, it might be that these decisions cause congestion in one subset of the register file, and finally cause an infeasible register binding. On the other hand, if register binding is performed beforehand, the address range constraints are easily ignored, as the example in Figure 2 suggests. In order to keep the search space freedom and postpone the assignment decisions, we establish a similar concept as the weak conflict in lifetime analysis and classify the assignment conflicts between a value u labeled with l_u and a register r_i (node r_i in the transformed ACG) as follows:

- Strong conflict: value u and register r_i have strong conflict if value u can never reside in the address range where register r_i belongs.
- No conflict: value u and register r_i have no conflict if value u can always reside in the address range where register r_i belongs.
- Weak conflict: value u and register r_i have weak conflict if value u can be assigned to other ranges besides the address range where register r_i belongs.

For the example DFG in Figure 4 (a), assume that there are four registers $r1$, $r2$, $r3$, $r4$ available. Registers $r1$, $r2$, $r3$ form the address range I, registers $r2$, $r3$, $r4$ form the address range II. Also assume that values produced by *add* operations are stored in range I, and values produced by *mul* operations are stored in range II. Load operation can store the result loaded from memory to either range I or range II. The final annotated worst-case conflict graph (AWCCG) is illustrated

in Figure 5. Notice that node f has strong conflict with node $r4$ since value f can only be allocated in address range I, while node a has weak conflict with node $r1$, $r2$, $r3$, or $r4$, since *load* operation can reside the value to either of the ranges and the assignment decision has to be made later on. Similarly, the annotated best-case conflict graph (ABCCG) contains all the strong conflicts.

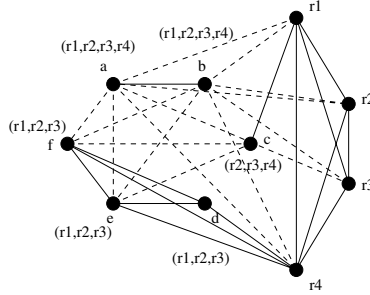


Fig. 5. Annotated worst-case conflict graph

The AWCCG is constructed containing all the weak and strong conflicts. Although in principle there are two different kinds of weak conflicts, namely weak conflicts from lifetime analysis and weak conflicts from address range analysis, we do not distinguish them explicitly in our approach. The reason is that our search space is always regulated by the constraint analysis techniques [9] [11], which have the ability to deal with all the integrated timing and resource constraints. Any decision regarding to solving the weak conflicts in value lifetimes or address range assignment will be reflected in the whole search space.

7 Problem Statement and Approach

In this section, we define the scheduling and register binding problem for LAR architecture. We decompose the problem and construct a block diagram of the global approach. Our problem can be defined as follows.

Problem Definition: Given a data flow graph, resource constraints, timing constraints, a register file RF with its capacity $C(RF)$, this register file can be subdivided into address ranges and each functional unit in the data path can store the value in one or more address ranges, find an assignment of values to registers and a schedule such that all the timing, resource, capacity and address range constraints are satisfied.

The global approach is based on the previous work of [1] with some additions, and is depicted in Figure 6. It is decomposed into several steps since decisions affect the search space in both scheduling domain and the register allocation domain. The central part, *constraint analysis*, generates additional precedence constraints that are implied by the combination of all the timing and resource

constraints. The additional precedences refine the start times of operations and prevent decisions leading to infeasibility.

The worst case or upper bound ub is computed by constructing the AWCCG for all the values in RF . It corresponds to the requirement of registers in the worst case when scheduling is roughly performed without noticing the register file constraints. If it is larger than $C(RF)$, some potential conflicts have to be solved either by serializing lifetimes or by address range assignment. Lower bound lb is computed for ABCCG to deduce the feasibility of a schedule with the register file capacity constraint and address range constraints. Upper bound and lower bound give a general overview of the RF ability for a certain application, while the detailed register allocation has to be worked out.

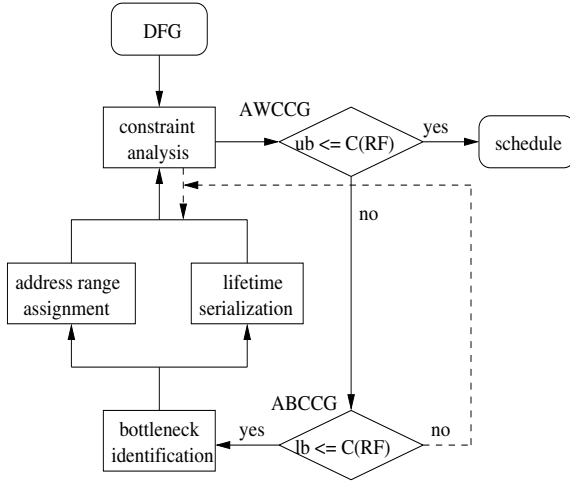


Fig. 6. Global approach

The *bottleneck identification* is performed to identify the weak conflict which are potentially easier to solve. This is done by calculating the saturation number of node v , i.e. the number of different colors in the neighborhood, and the degree number, i.e. the number of neighbors of node v . Several heuristics are provided in [1] for bottleneck identification. The bottleneck identified here can be the potential lifetime overlap, which can be solved by serializing their lifetimes if feasibility is maintained. It can also be the uncertainty of limited address range utilization, which can be solved by making an assignment decision. Note that if the schedule is fixed, the identification and reduction of bottleneck is still useful. However, the opportunity for doing so is restricted because there are less conflicts in the conflict graph.

Lifetime serialization is performed by adding sequential edges to force one lifetime proceeds another one. For example, let value u be produced by operation P^u and consumed by C^u , and let value v is produced by operation P^v and consumed by C^v . Serializing u and v can be done in two ways: $C^u \rightarrow P^v$ or

$C^v \rightarrow P^u$. Often the constraint analysis is able to exclude one of the possibilities. If not, it has to make a decision based on sacrificing the least possible schedule freedom. If constraint analysis detects infeasibility anyway, it will discard the serialization and wait for another pair to be chosen.

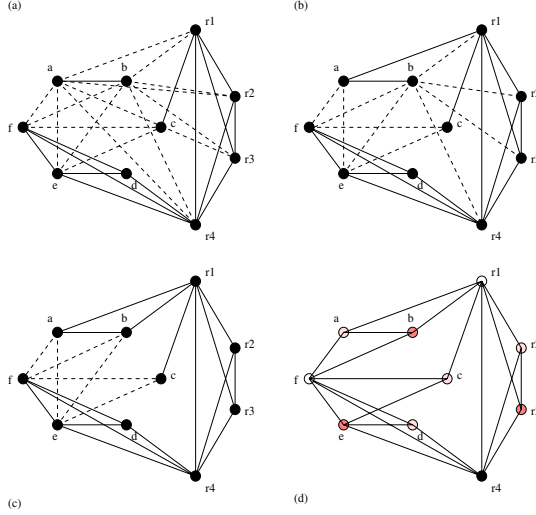


Fig. 7. Solving the weak conflicts to a fixed schedule

Address range assignment is made by assigning a value to certain address range. As a result, some weak conflicts will disappear while others will become strong conflict. In general, the weak conflicts are greatly reduced. The more weak conflicts being discarded, the more efficient the approach is. However, strong conflicts are also created, which will potentially cause infeasibility. Heuristics are used to take care that on the one hand, large amount of weak conflicts can be discarded; on the other hand, the assignment decisions will not create strong conflicts such that register allocation will only focus on a few registers.

The result of lifetime serialization or address range assignment reduces the freedom of the search space and constraint analysis will immediately reason the feasibility. This is performed iteratively until a feasible solution is found.

Note that the construction of the annotated conflict graph can be applied to any existing scheduling algorithm. The reason here that it is integrated with scheduling is because of the convenience for solving the phase coupling. The complexity of the construction of the AWCCG is $O(e + c)^2$, where e is the number of values assigned to the register file, and c is the number of registers. We employed Coudert’s coloring algorithm [4] for coloring the conflict graph, which is fast and exact for 1-perfect graphs, and slow but exact for others. Constraint analysis techniques have been reviewed thoroughly in [11] and [9]. Experimental results in section 8 will quantify the run time with or without coupling the range assignment phase.

For the AWCCG in Figure 5, the bottleneck identification will first select the edge between node a and node $r4$ because the saturation number of a and e are both 4, and the degree number for them are 7 and 8 respectively. Solving this weak conflict implies that value a will be assigned into address range II. This will cause the strong conflict between a and $r1$ as depicted in Figure 7 (b). Next, node b and $r4$ are identified as bottleneck. Similarly, this causes the assignment of value b into range II. Subsequently the AWCCG is reduced to Figure 7 (c). The next bottleneck that is identified is the edge between node e and a . The serialization of lifetimes of e and a will cause f to have strong conflict with a , b and c , as well as e with c . This will fix the schedule finally and the register binding result with respect to the range constraints and register file constraint is given in Figure 7 (d).

8 Experimental Result

In this section, we present the experimental results obtained after implementing the proposed method within FACTS environment [11]. All experiments are run on the machine with a Pentium IV processor running at 1.5GHz. We performed the experiments on several benchmarks from DSP algorithm kernels, e.g. AR filter, fifth-order digital elliptical wave filter, fast discrete cosine transform (FDCT), Loeffler algorithm and Chen algorithm that performs 8-point 1-dimensional inverse discrete cosine transform (IDCT). For each benchmark, fu denotes the number of functional units per type and l is the latency in clock cycles.

Table 1. Central register file vs. two-range LAR

$DFG_{fu,l}$	encoding			run time		assn after seri			seri with assn		
	central	LAR	%	central	LAR	$ S $	$ S_o $	$T(s)$	$ S $	$ S_o $	$T(s)$
$ar_filter_{1,18}$	392	308	78.57	0.08	0.09	5	2	0.07	5	2	0.09
$wdelf_{1,27}$	476	398	83.61	0.04	0.12	6	2	0.07	6	2	0.12
$fdct_{2,20}$	714	588	82.35	0.91	4.95	9	5	<i>inf</i>	9	5	4.95
						9	7	0.88	9	7	<i>inf</i>
$fdct_{4,11}$	714	588	82.35	0.32	0.36	9	6	<i>inf</i>	9	6	0.35
						9	7	0.26	9	7	0.37
$loef_{2,15}$	952	952	100	1.10	1.43	12	7	0.95	12	7	<i>no</i>
						12	8	<i>inf</i>	12	8	1.43
						12	9	0.2	12	9	<i>no</i>
$loef_{4,11}$	952	952	100	0.46	0.89	12	9	<i>inf</i>	12	9	0.89
$chen_{2,15}$	680	560	82.35	0.22	2.78	8	4	<i>inf</i>	8	4	1.28
						8	5	0.24	8	4	4.28
						9	3	0.22	9	3	<i>no</i>
$chen_{4,8}$	680	560	82.35	0.18	0.24	9	4	0.19	9	4	0.24
						9	5	0.16	9	5	0.24

In Table 1, the register requirements, the run time and the encoding costs are compared between a central register file architecture and a two-range LAR architecture. We assume three-operand instructions, with two source operands and one destination operand. In addition, we assume that the opcode is encoded with 5 bits. In this table, columns 2 to 4 report the code size for the central register file architecture and the LAR architecture and shows the encoding reduction in percentage. Columns 5 and 6 compare the average run time for the central and the LAR architecture with phase coupling approach. Code generation results without phase coupling, i.e. range assignment is performed after lifetime serialization, and with phase coupling, i.e. range assignment is performed together with lifetime serialization are compared in terms of the total number of registers ($|S|$), the number of overlapping registers ($|S_o|$) and run time ($T(s)$). “no” refers to no solution can be found in given time and “inf” means that the code generation cannot find a feasible solution.

As we can see, for most of the benchmarks, the code size is reduced for the LAR architecture. The average run time for the LAR architecture is about 1 to 5 times of the central register file architecture. In general, the run time for the phase coupling approach is longer than that without phase coupling. This is understandable, since phase coupling approach works on a bigger conflict graph. The phase coupling approach for the LAR architecture is quite sensitive to the number of overlapping registers to be chosen. For example, for *loef_{2,15}*, by only increasing or decreasing the number of $|S_o|$ by one, the code generation cannot find a solution within a given time. This is because the parameters given here are nearly all the the pareto points for the architecture. If those parameters are released, the less chance that the code generation will not find the solution. In *loef_{4,11}* and the first example of *chen_{2,15}*, although phase coupling approach can find a satisfying solution, the sequential approach in which range assignment is performed after serialization is not capable to find a feasible one. This indicates that phase coupling approach is more promising since one decision in a phase will always be taken into account in the whole search space, while the sequential approach will observe this effect afterwards.

For some benchmarks, e.g. *loef_{2,15}*, the LAR architecture doesn’t help reducing the code size. The reason is that since there are a lot of swapping of values between the two ranges, the amount of operations write to and read from different address ranges is large. Therefore the number of commonly addressable registers has to be kept large enough. This can be improved by further partitioning each range into smaller overlapping ranges. Table 2 shows the results for a four-range LAR architecture.

In this table, columns 2 to 4 report the code size for the central register file architecture and the LAR architecture and the encoding reduction in percentage. Columns 5 and 6 compare the average run time for the central register file architecture and the LAR architecture with phase coupling approach. Since the two ranges are subdivided further into four ranges, columns 8 and 9 report the total registers and the number of overlapping registers between the two ranges *R1* and *R2* in the first division, and columns 10 and 11 report the number of overlapping registers within range *R1* and *R2* respectively in the second division.

Table 2. Central register file vs. four-range LAR

$DFG_{fu,l}$	encoding			run time		LAR					
	central	LAR	%	central	LAR	$ S $	$ S_o $	$ S_1 $	$ S_2 $	after	with
$fdct_{2,20}$	714	588	82.35	0.87	0.95	9	3	5	5	0.87	no
	714	564	78.99			9	3	4	5	0.88	0.71
	714	564	78.99			9	3	4	4	inf	1.09
$fdct_{4,11}$	714	588	82.35	0.30	0.44	9	2	4	4	0.30	0.44
$loef_{2,15}$	952	784	82.35	0.94	1.79	12	4	6	6	inf	1.86
	952	763	80.15			12	3	5	4	0.59	1.71
	952	724	76.05			12	2	4	4	0.30	no
$chen_{2,15}$	680	536	78.82	0.15	1.95	11	3	5	4	0.14	3.54
	680	542	79.71			10	3	4	5	0.13	0.36
	680	518	76.18			10	3	4	4	0.17	no

Columns 12 and 13 compare the run time without and with phase coupling. From this comparison we can see that the encoding reduction is increased to 23.95% maximally, although the total number of registers $|S|$ may also be increased. With the increase of the number of divisions further, the reduction is less obvious while the architecture becomes more complicated.

9 Conclusions and Discussions

In this paper, we introduced a new type of architecture and encoding style for reducing the code size of embedded processors. We also proposed the corresponding compilation technique for register binding and scheduling for this architecture. The advantage is that by reducing addresses to a certain range of a register file, only necessary encoding bits are used in the instruction sets. In addition, we allow certain overlap between different ranges. Therefore swapping values between ranges are retained in the overlapping addresses and no extra hardware as well as no extra move operations are necessary. In order to solve the range assignment problem, we introduced the annotated conflict graph. The annotation contains the range constraints in the LAR architecture. Since assignment decisions will cause additional phase coupling problem in code generation and that is not desirable, we postpone the assignment decisions and introduce the similar concept as weak conflicts for modeling the uncertainty of the scheduling decisions, which can be combined with other timing, resource and register file constraints and is applicable to constraint analysis techniques.

One disadvantage of this approach is that the number of weak conflicts becomes large for large benchmarks with complicated ranges, which may give a heavy burden on the coloring of the worst-case conflict graph that is frequently used in register allocation. Another disadvantage is that new instructions have to be introduced. The code size reduction is limited since the number of overlapping registers always has to be kept large enough. Once this number is not enough, extra move operations are still needed. In this paper, we assume that

source and destination operands always access the same range. In fact, architectures can be designed with more flexibility, such as source and destination operands access different overlapping ranges. Although our approach solved the phase coupling problem of address range assignment, we still assume that the operation assignment, i.e. the assignment of operations to functional units [2], is performed separately. In practice, it is desirable to combine the operation assignment with this approach. In the future work, we would also like to exploit more versatile LAR architectures.

References

1. C. Alba Pinto, B. Mesman, and K. van Eijk. Register files constraint satisfaction during scheduling of DSP code. In *Proceedings of the XII Symposium on Integrated Circuits and Systems Design*, pages 74–77, Los Alamitos, CA, USA, October 1999. IEEE Computer Society Press.
2. M. Bekooij, B. Mesman, and J. van Meerbergen, and J.A.G. Jess. Constraint analysis for operation assignment in FACTS. In *Proceedings of the 11st ProRISC/IEEE Benelux Workshop on Circuits, Systems and Signal Processing*, pages 229–236, Utrecht, The Netherlands, November 2000. STW Technology Found.
3. A. Capitanio, N. Dutt, and A. Nicolau. Partitioned register files for VLIWs: A preliminary analysis of tradeoffs. *SIGMICRO-Newsletter*, 23(1-2):292–300, december 1992.
4. O. Coudert. Exact coloring for real-life graphs is easy. In *Proceedings of the 34th Design Automation Conference*, pages 121–126, New York, NY, USA, 1997. ACM.
5. M. Fernandes, J. Llosa, and N. Topham. Partitioned schedules for clustered VLIW architecture. In *Proceedings of the First Merged International Parallel Processing Symposium and Symposium on Parallel and Distributed Processors*, pages 130–134, Los Alamitos, CA, USA, 1999. IEEE Computer Society Press.
6. J. Janssen and H. Corporaal. Partitioned register file for TTAs. In *Annual International Symposium in Microarchitecture*, pages 303–312, Los Alamitos, CA, USA, November 1995. IEEE Computer Society Press.
7. R. Leupers. Instruction scheduling for clustered VLIW DSPs. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, pages 291–300, Los Alamitos, CA, USA, October 2000. IEEE Computer Society Press.
8. MAJC architecture tutorial, <http://www.sun.com/processors/majc>.
9. B. Mesman, A. Timmer, J. van Meerbergen, and J. Jess. Constraint analysis for DSP code generation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 18(1):44–57, January 1999.
10. B. Rau, D. Yen, W. Yen, and R. Towle. The cydra 5 departmental supercomputer: Design philosophies, decisions and trade-offs. *Computer*, 22(1):12–35, January 1989.
11. C. van Eijk, B. Mesman, C. Alba Pinto, Q. Zhao, M. Bekooij, J. van Meerbergen, and J. Jess. Constraint analysis for code generation: Basic techniques and applications in FACTS. *ACM Transactions on Design Automation of Electronic Systems*, 5(4):774–793, October 2000.
12. V. Zivojnovic, J. M. Velarde, and C. Schlager. Dspstone: A DSP-oriented benchmarking methodology. In *Proceedings of International Conference on Signal Processing Applications and Technology*, pages 715–720, Waltham, MA, USA, October 1994. DSP Associates.

Predicated Instructions for Code Compaction^{*}

Warren Cheung, William Evans, and Jeremy Moses

Department of Computer Science
University of British Columbia, Vancouver, B.C. V6T 1Z4
{wcheung,will,jmoses}@cs.ubc.ca

Abstract. Procedural abstraction, the replacement of several identical code sequences with calls to a single representative function, is a powerful tool in producing compact executables. We explore how predicated instructions can be used to allow procedural abstraction of non-identical basic blocks. A predicated instruction is one that the processor executes if a condition (specified in the opcode) is true, otherwise the instruction has no effect. Architectures such as the ARM provide predicated versions of most of their instructions. By using predicated instructions within a representative function and setting the appropriate flags prior to the call, a single function can serve to represent several different code sequences. To find representative functions, we group sequences that have a short common supersequence and use this supersequence as a representative. We report results on the use of predication for procedural abstraction on the ARM and also indicate the potential compaction benefit of allowing more predication conditions.

1 Introduction

Decreasing program size is becoming increasingly important as users are expecting more functionality from devices that cannot afford more memory. The limitations on memory size may be the result of economic forces or power consumption budgets and are especially restrictive in embedded system applications and hand-held devices. In these cases, programmers seeking to provide more functionality must design their applications to minimize memory usage by carefully choosing algorithms and data structures that consume as little memory as possible during execution. In many cases, however, the application code itself – the program’s set of instructions – consumes a large fraction of the memory needed by the application. The programmer then must rely on the compiler to produce space-efficient code, or carefully tune assembly code for the processor.

Code compaction is an attempt to tune code automatically in order to make it occupy less space while maintaining all of its original functionality. Link-time assembly-code compaction has a slight advantage over compile-time space optimization methods in that it can “see” compaction opportunities that a compiler

^{*} Supported in part by the Natural Sciences and Engineering Research Council of Canada under grant NSERC-238828-01 and the National Science Foundation under grant CCR-0073394.

working at the single function, class, or object file level might miss. It has the disadvantage that it does not have the compiler’s knowledge of the high-level semantics of the code.

The basic techniques of post link-time code compaction are the typical compile-time space optimizations (which, nevertheless, may be more effective post link-time): common subexpression elimination, dead code elimination (code may become dead via constant propagation between functions), code factoring (moving replicated code fragments to a position that pre- or post-dominates their original locations), expression simplification, and various peephole optimizations. In addition, code compaction uses a technique that is not a typical compiler optimization: the replacement of replicated code by branches to a single representative code fragment or *procedural abstraction* (also known as *procedure exlining* [13]).

We focus on procedural abstraction and explore the benefit of using predicated instructions to permit non-identical fragments to share a single abstracted representative. Prior to the branch to the representative, we insert an instruction that sets a flag to identify the fragment that is making the call. The representative uses this flag to execute the correct sequence of instructions for that fragment by predicating some of its instructions. If the fragments sharing a representative contain a long common subsequence of instructions, the representative will contain only one copy of this subsequence (unconditionally predicated), and we will achieve some amount of compaction.

Section 2 describes predicated instructions. Section 3 reviews common abstraction mechanisms and describes predicated procedural abstraction. In Section 4, we describe how we choose groups of code fragments to abstract and how we form the representative function from these groups using sequence comparison techniques. Section 5 introduces the features of the ARM processor that we exploit in our compaction system. The results of the compaction appear in Section 6 and we conclude with related work in Section 7.

2 Traditional Use of Predicated Instructions

A predicated (or guarded) instruction specifies a boolean condition as well as an operation. The operation is performed only if the condition is true. Using predicated machine instructions rather than branches can result in impressive speedups on architectures that exploit instruction level parallelism [8]. Rather than a costly branch missprediction, a processor that schedules a predicated instruction with a false condition incurs only a modest penalty (the resources to begin execution of and then discard the predicated instruction). In addition, predicated instructions express control dependence as data dependence. That is, the execution of an instruction that depended on the outcome of a branch now depends on the value of a flag. This provides a compiler/processor more opportunities to recognize independent instructions and schedule them in parallel.

Our goal is to use predicated instructions to reduce code size. One way to do this is to perform *if-conversion* to remove branches. If-conversion predicates

instructions whose execution was previously governed by explicit branches, thus translating control dependence into data dependence [1]. For example, in Figure 1 if-conversion converts three basic blocks and their related control flow into one block. This process results in the elimination of two branch instructions. August mentions this code reduction as a fortunate side effect of if-conversion, even though it is not if-conversion’s primary purpose [2].

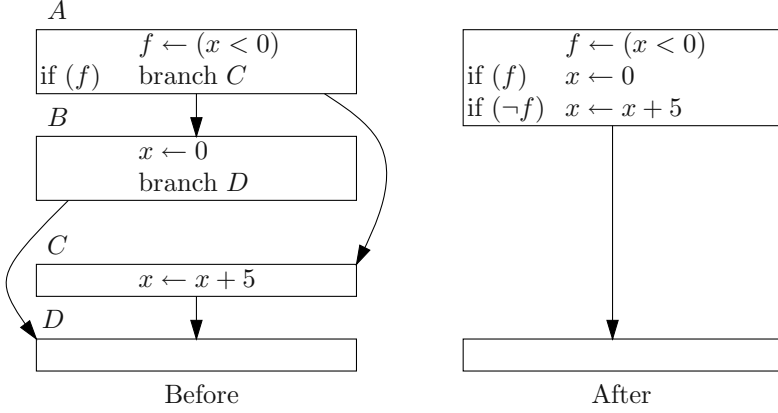


Fig. 1. Performing if-conversion can decrease the number of instructions in a program

Compilers targeted for architectures that support predicated execution currently reap these benefits. We explore a different use of predicated instructions for code compaction: using predication to increase the applicability of procedural abstraction to non-identical basic blocks.

3 Non-identical Block Abstraction

Suppose that basic block B occurs k times in a program. We can replace each occurrence by a function call to a representative function that is simply a copy of B with a return instruction at its end. This is called *identical code abstraction*. By doing this, we eliminate $k|B|$ and add $k + |B| + 1$ instructions (assuming a single instruction to return and one for each call). This reduces the overall number of instructions if $k|B| > k + |B| + 1$ or, equivalently (for $k \geq 2$), if $|B| > \frac{k+1}{k-1}$. Thus abstracting a two-instruction basic block that occurs only four times results in a decrease in program size.

This type of basic block abstraction requires the abstracted blocks to be identical. Sometimes it is possible for sets of non-identical basic blocks to be abstracted. If the sequences of operator in several blocks are identical (or they can be re-ordered to be identical) but some operands differ, then we may be able to construct a single representative function for all. The representative function

performs the sequence of operations on a “canonical” set of registers. We then replace each block with a sequence of instructions that moves values into the appropriate canonical registers, calls the function, and, on return, restores the values to their original locations. This register renaming process permits values to be passed to the representative function and realizes a form of *parameterized procedural abstraction*. (In general, parameterized procedural abstraction also allows values to be passed to the representative function via the stack.)

Even if two operand sequences are not identical, we may still be able to abstract them partially. The most common example of this is *cross jumping* where one sequence is a suffix of the other and only this common suffix is abstracted.

We shall now describe a new mechanism, which can augment these existing techniques, to abstract non-identical basic blocks, using predicated instructions to handle differences among the blocks. The general idea is as follows: Given a set $S = \{B_1, B_2, \dots, B_k\}$ of k “similar” basic blocks (where each block B_i is a sequence of instructions), we form a representative function F (again, a sequence of instructions) that contains the instruction sequence of each basic block as a subsequence. Some of the instructions in F are predicated so that setting certain flags prior to calling F causes B_i ’s sequence of instructions to be executed during F ’s execution. We then replace each block B_i by the appropriate flag setting instructions followed by a call to the representative function F . Instruction predication within the representative function selects the subsequence of instructions that match the original block. Figure 2 shows an example of three different blocks abstracted as one function using predicated instructions. (Figure 4 shows a less abstract example.)

4 Selecting and Replacing Similar Basic Blocks

In order to perform procedural abstraction that takes advantage of instruction predication, we must identify sets of similar basic blocks and form a representative function to replace them. We discuss two methods to accomplish this task. Both are greedy heuristics and both are based on the notion of a *shortest common supersequence*.

A sequence of instructions A is a *supersequence* of a sequence B (and B is a *subsequence* of A) if B can be obtained by removing zero or more instructions from A . A is a *common supersequence* (*subsequence*) of $B_1, B_2, \dots, B_k$ if A is a supersequence (subsequence) of every B_i .

A common supersequence F of blocks $B_1, B_2, \dots, B_k$ can be used as their representative function by predicated the instructions that occur in F so that an instruction is executed when, and only when, F is called by the blocks that need that instruction. To determine the instructions’ predication conditions, we mark, for each B_i , the subsequence of instructions in F that form B_i . Each instruction receives marks from some subset of the k blocks. The predication condition on an instruction is one that evaluates to *true* when, and only when, F is called by any of the B_i ’s that gave the instruction a mark (Figure 2).

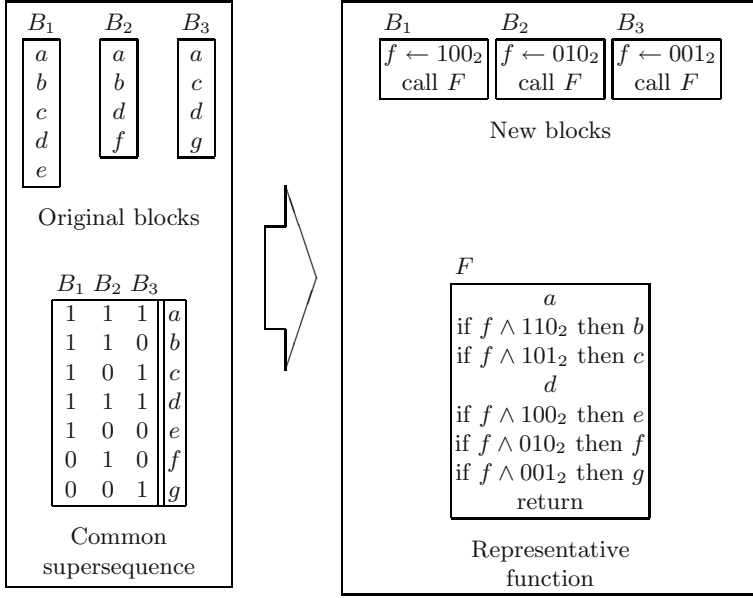


Fig. 2. An example of how instruction predication permits the abstraction of three non-identical basic blocks. We indicate, by 0 and 1, which blocks require which instructions in the common supersequence

There are $2^k - 1$ non-empty subsets of $\{B_1, B_2, \dots, B_k\}$ that may share an instruction in F . Thus, in some situations, we may be forced to express $2^k - 1$ predication conditions on instructions. This implies that each predicable instruction has at least k bits devoted to a condition code. In other words, if the architecture has only k predication flags, we may only be able to abstract groups of k non-identical basic blocks.

Choosing the shortest common supersequence (SCS) as the representative results in the elimination of the largest number of instructions. In performing the abstraction, we remove $|B_1| + |B_2| + \dots + |B_k|$ instructions, add $|F|$ instructions, and add $k - 1$ call and k flag setting instructions (assuming we can set the appropriate flags in one instruction). We only need $k - 1$ call instructions since one of the blocks, B_1 , will fall-through to the representative. We also add one return instruction to F (predicated so that B_1 doesn't execute it). Thus the decrease in code size is $|B_1| + |B_2| + \dots + |B_k| - (|F| + 2k)$. This can be quite large, especially if many blocks share long common instruction sequences.

Our first method, *GreedyGroup*, ranks each subset of at most k basic blocks by its *benefit*: the number of instructions that would be eliminated by abstracting the set using the SCS representative. We then abstract sets, in order of decreasing benefit, until no more sets are beneficial.

Finding the SCS of the set of k sequences $B_1, B_2, \dots, B_k$ is, in general, an NP-hard problem. We use a dynamic programming algorithm for the problem that runs in time $O(k|B_1||B_2| \cdots |B_k|)$ [6]. Since typical block sizes are small (five instructions on average in our benchmarks), this is not as impractical as it might first appear. We avoid calculating the SCS for some sets of basic blocks by recognizing that they contain pairs of blocks so dissimilar that the SCS of the group could not be profitable. Even with this optimization, finding good sets of $k = 6$ blocks is not practical at this point. The problem is that even though blocks are typically small, there are many of them. Let n be the number of basic blocks. Examining all $\Theta(n)$ subsets of size six is too time consuming.¹ However, since the number of dissimilar blocks we can hope to abstract is constrained by the available predication flags, we may only need to consider small values of k . We return to this issue when we discuss our experimental results in Section 5.

Our second method, *GreedyPair*, ranks each pair of basic blocks by the number of instructions eliminated in abstracting them using their SCS representative. If B_1 and B_2 are the highest ranked pair, we create the SCS A for B_1 and B_2 , remove B_1 and B_2 from the set of blocks, and add A as a new block. We then remove all pairs that involve B_1 or B_2 from the ranking, add new pairs that pair A with every remaining block, and repeat by again finding the highest ranked pair. The sequence A may later be merged with another block, which itself may be a pairwise merge of original blocks. The algorithm ends when no more pairs can be abstracted to decrease the number of instructions. In order to avoid huge clusters of basic blocks having a single representative, we can limit the allowed pairs to only those whose resulting representative would represent at most k original basic blocks.

The decrease in the number of instructions when we abstract two “blocks” A and B depends on whether they are original basic blocks or pairwise merged sequences. In either case, we eliminate the *longest common subsequence* (LCS) of A and B . If A and B are original blocks, we must add a call and flag setting instruction for B , a flag setting instruction for A (A needs no call because it will fall-through to the representative), and a return instruction (predicated to prevent A executing it). That is, we add four instructions for a total decrease of $|\text{LCS}(A, B)| - 4$ instructions. If A (or B) is already a pairwise merged sequence, then we don’t pay a two-instruction overhead for A (or B). Each of the original basic blocks represented by A (or B) has already paid for two additional instructions, and these instructions are enough to allow every original block in A (or B) to set flags and call a representative (or fall-through and pay for the return). Thus, if one of A or B is a pairwise merged sequence and the other is an original basic block, the instruction decrease is $|\text{LCS}(A, B)| - 2$. If both A and B are pairwise merged sequences, the instruction decrease is $|\text{LCS}(A, B)|$.

¹ On average for our benchmarks, GreedyGroup takes approximately 270, 340, and 2500 seconds for group size $k = 2, 3$, and 4 (respectively) on a 700MHz workstation.

GreedyPair considers only $\Theta(n^2)$ pairs of basic blocks rather than the $\Theta(n^k)$ subsets of k basic blocks that GreedyGroup considers. In practice, GreedyPair runs quickly² but eliminates fewer instructions than GreedyGroup.

In the following two sections, we explore how well the GreedyGroup and GreedyPair approaches to predicated procedural abstraction work in practice. We describe the implementation of a compaction system that performs procedural abstraction of ARM executables using ARM’s predicated instruction capabilities. We report on the compaction that can be achieved using this system on the current ARM architecture, and indicate the potential of additional predication flags on these compaction results.

5 ARM Conditional Execution

The ARM processor permits the conditional execution of virtually any³ instruction based on the status of certain flags (bits) in the Current Processor Status Register (CPSR) [12]. The flags that play a role in instruction predication are labelled N , Z , C , and V . Every opcode has a condition field that determines under what flag conditions the instruction executes (Figure 3). A condition is simply a boolean function of the four flags, and the ARM provides 15 such functions as conditions (out of a total of $2^{2^4} = 65536$ possible conditions on these four flags).

Normally, flags are set after compare-type instructions to detect exceptional conditions or to help with control flow decisions. They can, however, be set directly, even in the processor’s user mode with a Move to Status Register instruction (MSR). This is the instruction we use to set the appropriate flags before a call to an abstracted block’s representative. It allows us to create any setting of the four flags that we like. After it is executed, the flags serve as a label denoting which of the blocks we are calling from. An example of abstracting two basic blocks from one of our benchmarks is shown in Figure 4.

In general, it is not a trivial task to choose $NZCV$ -flag settings for each basic block and ARM predication conditions on the instructions in the representative function F . To indicate the complications, we return to the example shown in Figure 2 of abstracting three different blocks B_1 , B_2 , and B_3 . That example is reproduced in Figure 5 using ARM’s N , Z , C , and V flags, and ARM’s predication conditions.

It isn’t perhaps immediately apparent why we chose the flag settings $\{0000_2, 0001_2, 0100_2\}$ for the basic blocks in Figure 5. The choice allows us to select, for any subset of the k basic blocks, an ARM predication condition that is true if and only if the representative is called by a basic block in that subset. We may not need a predication condition for *all* subsets. For instance, in the example, no instruction in F requires a condition that is *true* if and only if F is called by

² For unbounded k (the slowest case), most benchmarks take less than three seconds to process on a 700MHz workstation, and the slowest, `djpeg`, takes less than 30 seconds.

³ Instructions that are not conditionally executable are Breakpoint (BKPT) and Branch and Link with Exchange to Thumb (BLX).

Mnemonic extension	Execution condition
EQ	Z
NE	$\overline{Z}$
CS/HS	C
CC/LO	$\overline{C}$
MI	N
PL	$\overline{N}$
VS	V
VC	$\overline{V}$
HI	$C \wedge \overline{Z}$
LS	$\overline{C} \vee Z$
GE	$N = V$
LT	$N \neq V$
GT	$(N = V) \wedge \overline{Z}$
LE	$(N \neq V) \vee Z$
blank/AL	1

Fig. 3. Mnemonics added to ARM instructions and the conditions they indicate that must be true for the instruction to execute [12]

B_2 or B_3 . However, as the example shows, for $k = 3$ such a robust set of flag settings exists.

With four predication flags and 15 predication conditions, one would expect that a robust set of four flag settings exists. Its existence, however, depends not only on having k predication flags and $2^k - 1$ conditions, but also on the ability of these conditions to select all (except the empty) subsets of a set of four flag settings. In other words, we need to be able to choose a set S of four settings for the N , Z , C , and V flags so that no matter what non-empty subset of S we choose, one of the 15 predication conditions evaluates to *true* for the settings in the subset and *false* for the other settings in S . This corresponds to our ability to predicate an instruction in the representative function so that it is executed when, and only when, the function is called from any of the basic blocks that need it. For example, for the set $S = \{0011_2, 0101_2, 0110_2, 1001_2\}$, the LT condition selects the subset $\{0011_2, 0101_2\}$ of S . Unfortunately, for this particular choice of S , no condition selects the subset $\{0101_2\}$ of S . It turns out that no set S of size four allows the selection of all of its 15 non-empty subsets using ARM’s conditions. However, several sets allow ARM’s conditions to select 13 of the 15 non-empty subsets.

It is rarely the case that a single representative function contains instructions that need predication for every subset of its represented basic blocks. Therefore, we report results for abstracting upto $k = 4$ blocks using GreedyGroup, with the hope that this can be achieved even with the limited predication conditions available. We also report results for abstracting arbitrarily large groups of blocks

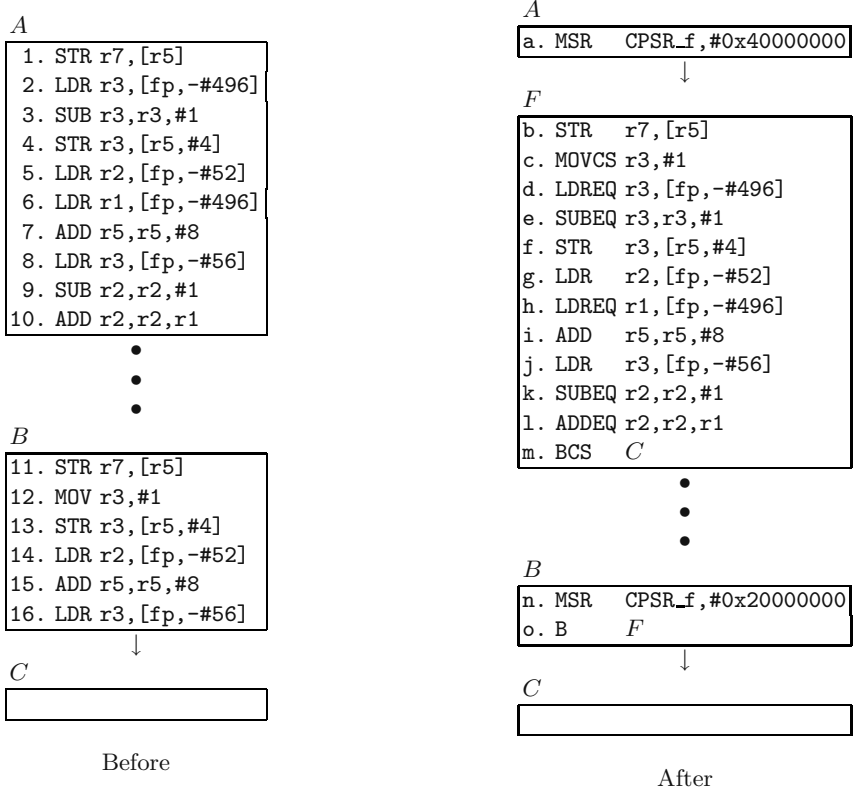


Fig. 4. Abstraction of two blocks from an ARM executable. The predicated branch instruction (instruction *m*) permits block *A* to fall-through (↓) to *F* and continue after *F* without branching to *C*

using GreedyPair, to indicate what might be achieved by increasing the number of predication flags and conditions.

6 Experimental Results

We first evaluate the compaction that can be obtained using predication for procedural abstraction. We calculate how many instructions can be eliminated by allowing the abstraction of 2 to 4 non-identical basic blocks using the Greedy-Group method, and 2 to 8 using the GreedyPair method. We also consider the unlimited GreedyPair method as an example of what can be achieved using predication if an architecture has as many predication flags and conditions as desired.

We apply the two methods to a selection of programs from the MediaBench benchmark suite (available at <http://www.cs.ucla.edu/~leec/mediabench>)

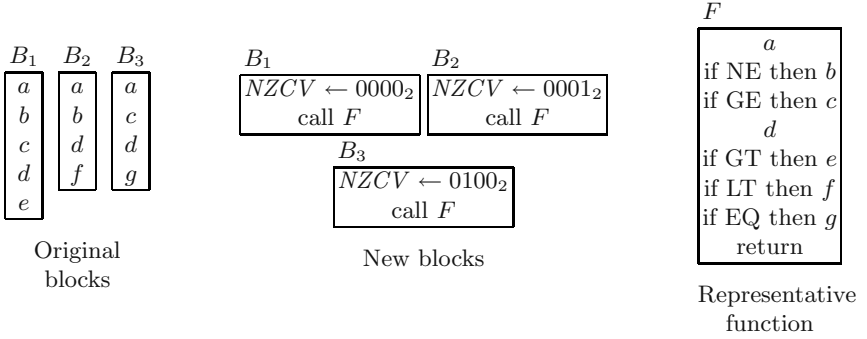


Fig. 5. Abstraction of three dissimilar basic blocks using ARM’s predication flags and conditions

and the results appear in Figures 6 and 7. The programs are first compiled using `gcc -O2` (version 2.95.3) and `armcc` (from the ARM Developer Suite version 1.2). In both cases, we statically link the binaries with any required library code, which is required by our binary-rewriting tool. Both compilers produce ARM machine code that is intended to be simulated, but by different simulators (the two provide different semi-hosting environments). Thus the machine code created by `armcc` does not contain the same low-level code (e.g. the same `printf` code) as that created by `gcc`. Also `armcc` is tailored for the ARM and produces much more concise code.

The ARM-executable version of each program is then read by our binary-rewriting system, which breaks the program into its basic blocks. Our system is based on *PLTO* [11], modified to handle the ARM instruction set. *PLTO* is a link-time optimizer that reads an executable, constructs its control-flow graph, and performs several optimizations before outputting a modified executable. In our system, we perform none of *PLTO*’s optimizations except dead code removal.⁴ This gives us a base instruction count. All percentages reported in Figures 6 and 7 are relative to this base.

After breaking a program into its basic blocks, we eliminate some blocks from consideration for abstraction. Since most branch instructions are unique (having different targets), we place them in their own, separate basic block, which effectively removes them from consideration. We (conservatively) remove from the set of candidates for abstraction all basic blocks that read the program counter (PC). This is to avoid complications in our current implementation, but is not fundamentally necessary. We expect that by removing this restriction, our results will improve substantially since in the ARM, which stores data intermixed with code, reading from a PC-relative address to access data is common. These blocks are eliminated prior to any code abstraction.

⁴ Post link-time optimization often recognizes dead code, particularly in libraries, that a traditional space-optimizing compiler misses.

We then perform identical block abstraction. Abstraction uses the return address register (called the link register on the ARM), so blocks that read or write this register are eliminated from consideration. The percentage of instructions removed by identical block abstraction appears as the bottom bar in each stacked bar graph (Figures 6 and 7). Identical code abstraction has the advantage of being able to abstract arbitrarily many blocks into one procedure. Predicated abstraction, on the other hand, can only abstract a limited number of non-identical basic blocks because each block in the group must be identified by a unique setting of the predication flags. Any block that reads or writes **CPSR** flags (e.g. by performing a comparison) cannot be abstracted using predication (we do not save and restore flag settings) and so we remove it, at this step, from consideration.

The final step is to perform either the GreedyGroup or GreedyPair method to select sets of blocks for predicated abstraction and calculate the number of eliminated instructions. The results for both methods for various values of k appear in Figures 6 and 7 (based on Tables 1 and 2). Notice that GreedyGroup and GreedyPair eliminate the same instructions when $k = 2$, since they are the same algorithm in this case.

The results are encouraging. On average, the predication method using the GreedyGroup method with a group size bounded by $k = 3$ improves on identical code abstraction by 28% for **gcc** produced executables and 37% for **armcc** executables.

In addition to calculating the number of instructions saved, we have also tested the impact of predicated procedural abstraction on the execution time and the number of executed instructions. We obtained execution times using a NetWinder 2100 with a StrongARM SA-110 processor. We compared the execution times of the original, uncompact **gcc**-produced executables with those obtained after predicated procedural abstraction. The differences in execution times were negligible. In all cases, execution time increased by less than 6% with an average increase of 0.7%. We obtained instruction counts by modifying the ARM simulator included in **gdb** version 5.2. In all cases, the number of instructions considered for execution increased by less than 4% with an average increase of 1.5%.

7 Related Work

The general area of program compression is quite broad, encompassing techniques that require decompression before execution (so-called “wire-format” techniques); decompression on-the-fly; interpretation; and, as in this work, no decompression. An additional dimension is the choice of program representation to compress. Compressing high-level source code or abstract syntax trees typically results in very compact program representations, partly because source code provides concise abstractions for common constructs, but also because it obeys a grammatical structure. The downside is that decompression and compilation must precede execution. More low-level representations, i.e. virtual machine

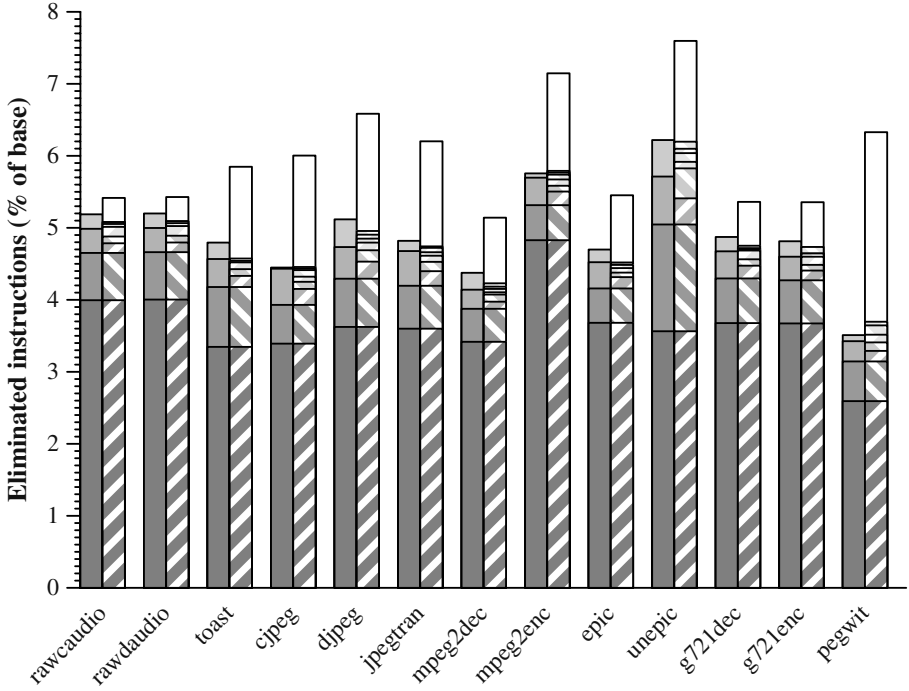


Fig. 6. Results for compaction of `gcc` executables. The left stacked bar graph in each pair depicts GreedyGroup results while the right (striped) stacked bar graph depicts GreedyPair results. The bottom bar in each stack shows the percentage of base instructions eliminated by identical block abstraction. The higher bars in the stack show the additional percentage of base instructions eliminated by using predication to abstract sets of k non-identical blocks for increasing values of k ($k = 2, 3, 4$ for GreedyGroup and $k = 2, 3, 4, 5, 6, 7, 8, \infty$ for GreedyPair)

codes or bytecodes, often can be decompressed and executed, or directly interpreted in their compressed form while requiring little or no additional memory – a substantial benefit for execution on limited memory devices. At the extreme (for software based methods) is compression of executable machine code to a form that is still executable, a technique often called “program compaction”.

Early work on program compaction treated the program as a sequence of instructions and used suffix trees to find repeated code fragments for procedural abstraction or cross jumping [5]. This resulted in the elimination of, on average, 7% of PDP-11 instructions from their sample programs. Cooper and McIntosh used the same suffix tree approach but allowed mismatches in register names, using register renaming (over the entire live range of a register) to make similar blocks equivalent [3]. Despite the additional opportunities for abstraction that register renaming allowed, they achieved an average RISC code size decrease of 5%. Part of the explanation for this is the difficulty in compacting RISC code

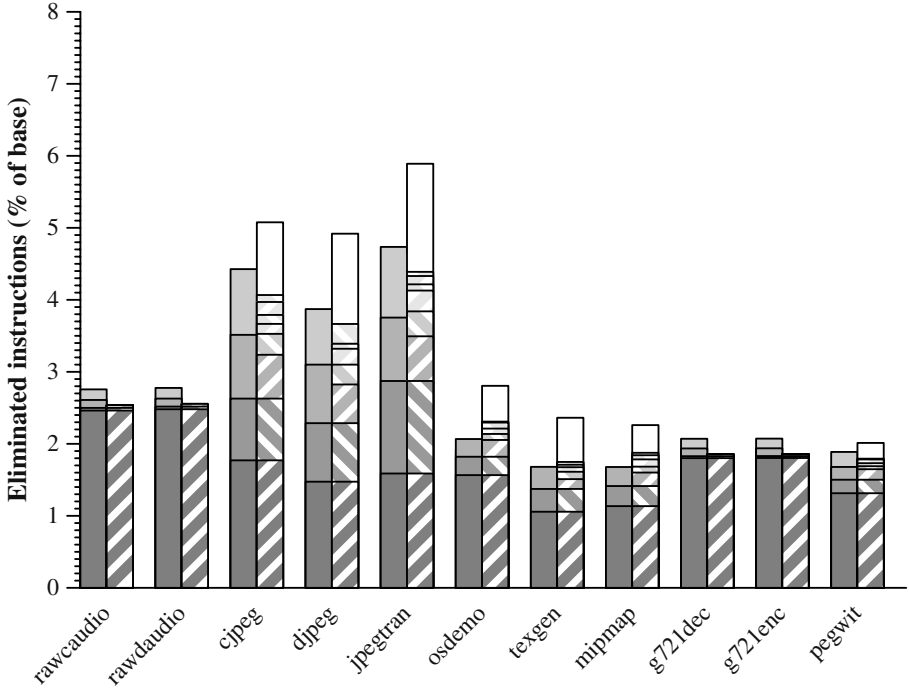


Fig. 7. Results for compaction of `armcc` executables

using procedural abstraction. Debray, et al. used basic block *fingerprints*, a hash of the operator sequence of a basic block, rather than suffix trees to identify repeated code [4]. The procedural abstraction part of their work, which also used register renaming but on a per-block rather than live-range basis, resulted in a code size decrease of 8% using the Alpha instruction set. Overall, their system decreased code size by 30%.

To increase the number of candidates for procedural abstraction, these latter methods essentially redid the register allocation originally done by the compiler. Runeson hypothesized that register allocation obscures potential matches and suggested performing procedural abstraction before register allocation [10]. His results are impressive: a 21% decrease in code size on average, however, the measurements are made on intermediate code size and don't include additional instructions, such as register spills, that might result.

A more general approach to creating opportunities for procedural abstraction is to permit the abstracted procedures to take parameters. Marks described such a scheme for IBM System/370 code [9]. His results are also impressive: a typical savings of 15%. Zastre also investigated parametrized procedural abstraction, for the SPARC, reporting average decreases of 2.7% [14].

A variation on basic block abstraction, due to Liao, et al., is a technique based on the *external pointer macro* model of compression [7]. They create a

sequence of instructions, the *dictionary*, and replace the program by a sequence of instructions and **calls** into the dictionary, the *skeleton*. A **call** into the dictionary causes a sequence of instructions to be executed that ends with a **return** to the skeleton. Any point in the dictionary may be a call site, which permits the abstraction of fragments smaller than entire basic blocks. Typically, one imagines that the **return** is an explicit instruction in the dictionary. Liao, et al. point out that a **call** instruction that specifies not only the call site but also the number of instructions to execute starting at that call site, removes the need for an explicit **return**. This means that any consecutive sequence of instructions in the dictionary can be called. They report reducing the total number of instructions by, on average, 12% in the explicit **return** model and 16% in the generalized **call** model.

Unlike this previous work on procedural abstraction, we consider (for abstraction) code fragments that differ not only in their operands, but also in their sequence of operators. The tool we use is predicated execution, passing a flag into the abstracted function to select the correct code sequence to execute. This is a different version of parameterized procedural abstraction. It permits the selection of *control flow* within the abstracted procedure on the basis of a passed parameter.

8 Conclusions and Future Work

This work represents the first effort to use predicated execution to improve procedural abstraction. We describe how shortest common supersequences can be used to create a small representative functions for groups of non-identical code fragments. Preliminary results, though modest, are encouraging. Predication improves on identical code abstraction by about 28% (for `gcc`) or 37% (for `armcc`) on average, and this is when permitting groups of only $k = 3$ blocks.

We restricted our evaluation to abstraction of entire basic blocks. This was in order to focus on the improvement to basic block abstraction that predication permits. Predication could be used to improve whole region abstraction, or to improve abstraction in conjunction with techniques, such as register renaming or instruction re-ordering, that attempt to create more similar blocks. The advantage that predication has is the ability to create a single representative block with, essentially, multiple execution paths through it. No other abstraction technique has this ability.

The obvious next step is to consider larger group sizes and code fragments other than single basic blocks. The two challenges with this step are designing efficient algorithms to discover these larger groups, and insuring that enough predication flags and predication conditions are available to predicate the instructions within the representative functions.

References

1. J.R. Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. Conversion of control dependence to data dependence. In *Proceedings of the 10th ACM Symposium on Principles of Programming Languages*, pages 177–189, 1983.
2. David Isaac August. *Systematic Compilation for Predicated Execution*. Ph.D thesis, University of Illinois at Urbana-Champaign, 2000.
3. K.D. Cooper and N. McIntosh. Enhanced code compression for embedded RISC processors. In *ACM Conference on Programming Language Design and Implementation*, pages 139–149, May 1999.
4. S.K. Debray, W. Evans, R. Muth, and B. de Sutter. Compiler techniques for code compaction. *ACM Transactions on Programming Languages and Systems*, 22(2):378–415, March 2000.
5. C. Fraser, E. Myers, and A. Wendt. Analyzing and compressing assembly code. In *Proc. of the ACM SIGPLAN Symposium on Compiler Construction*, volume 19, pages 117–121, June 1984.
6. Stephen Y. Itoga. The string merging problem. *BIT*, 21(1):20–30, 1981.
7. S. Liao, S. Devadas, and Kurt Keutzer. Code density optimization for embedded DSP processors using data compression techniques. In *Proc. Conf. on Advanced Research in VLSI*, pages 393–399, 1995.
8. Scott A. Mahlke, Richard E. Hank, James E. McCormick, David I. August, and Wen-mei W. Hwu. A comparison of full and partial predicated execution support for ILP processors. In *Proceedings of the 22nd Annual International Symposium on Computer Architecture*, pages 138–149, June 1995.
9. Brian Marks. Compilation to compact code. *IBM Journal of Research and Development*, 24(6):684–691, November 1980.
10. Johan Runeson. Code compression through procedural abstraction before register allocation. Master’s thesis, Computing Science Department, Uppsala University, March 2000.
11. Benjamin Schwarz, Saumya Debray, Gregory Andrews, and Matthew Legendre. PLTO: A link-time optimizer for the Intel IA-32 architecture. In *Proc. 2001 Workshop on Binary Rewriting (WBT-2001)*, September 2001.
12. David Seal, editor. *ARM Architecture Reference Manual*. Addison-Wesley, second edition, 2001.
13. F. Vahid. Procedure exlining: A transformation for improved system and behavioral synthesis. In *International Symposium on System Synthesis*, pages 84–89, September 1995.
14. M.J. Zastre. Compacting object code via parameterized procedural abstraction. Master’s thesis, Dept. of Computing Science, University of Victoria, 1995.

Table 1. Results for compaction of gcc executables. “base” is the number of instructions after dead code elimination. “identical” is the number of instructions removed by identical code abstraction. The remaining columns show the number of instructions removed by predicated abstraction of groups of upto k blocks, using either the GreedyGroup or GreedyPair method. For $k = 2$, the methods are the same. Numbers in brackets are percentages of the number of base instructions

Group	Program	base	identical	$k = 2$	$k = 3$		$k = 4$		$k = 5$	$k = 6$	$k = 7$	$k = 8$	$k = \infty$
					G-Group	G-Pair	G-Group	G-Pair	G-Group	G-Pair	G-Group	G-Pair	G-Group
adpcm	rawadpcm	7460	298 [3.99]	347 [4.65]	372 [4.99]	357 [4.79]	387 [5.19]	364 [4.88]	374 [5.01]	377 [5.05]	379 [5.08]	379 [5.08]	404 [5.42]
	rawaudio	7443	298 [4.00]	347 [4.66]	372 [5.00]	357 [4.80]	387 [5.20]	364 [4.89]	374 [5.02]	377 [5.07]	379 [5.09]	379 [5.09]	404 [5.43]
	toast	14911	499 [3.35]	623 [4.18]	681 [4.57]	646 [4.33]	715 [4.80]	660 [4.43]	674 [4.52]	677 [4.54]	677 [4.54]	682 [4.57]	872 [5.85]
	cjpeg	25268	857 [3.39]	993 [3.93]	1120 [4.43]	1049 [4.15]	1124 [4.45]	1074 [4.25]	1092 [4.32]	1115 [4.41]	1120 [4.43]	1126 [4.46]	1517 [6.00]
jpeg	djpeg	29114	1055 [3.62]	1250 [4.29]	1378 [4.73]	1319 [4.53]	1490 [5.12]	1365 [4.69]	1396 [4.79]	1412 [4.85]	1428 [4.90]	1443 [4.96]	1917 [6.58]
	jpegtran	24671	888 [3.60]	1035 [4.20]	1154 [4.73]	1085 [4.40]	1180 [4.82]	1117 [4.53]	1138 [4.61]	1150 [4.66]	1165 [4.72]	1170 [4.74]	1530 [6.20]
	mpeg2	19198	656 [3.42]	744 [3.88]	795 [4.14]	763 [3.97]	840 [4.38]	782 [4.07]	788 [4.10]	797 [4.15]	803 [4.18]	812 [4.23]	987 [5.14]
	mpeg2decode	15920	1379 [4.83]	1518 [5.31]	1627 [5.70]	1572 [5.50]	1644 [5.76]	1595 [5.58]	1620 [5.67]	1639 [5.74]	1647 [5.77]	1654 [5.79]	2041 [7.15]
epic	unepic	15920	586 [3.68]	662 [4.16]	720 [4.52]	687 [4.32]	748 [4.70]	697 [4.38]	707 [4.44]	714 [4.48]	719 [4.52]	719 [4.52]	868 [5.45]
	unepic	13217	471 [3.56]	667 [5.05]	755 [5.71]	715 [5.41]	822 [6.22]	770 [5.83]	782 [5.92]	798 [6.04]	806 [6.10]	819 [6.20]	1004 [7.60]
	decode	9029	332 [3.68]	388 [4.30]	422 [4.67]	404 [4.47]	440 [4.87]	412 [4.56]	423 [4.68]	424 [4.70]	426 [4.72]	429 [4.75]	484 [5.36]
	encode	8850	325 [3.67]	378 [4.27]	407 [4.60]	390 [4.41]	426 [4.81]	397 [4.49]	407 [4.60]	411 [4.64]	411 [4.64]	419 [4.73]	474 [5.36]
pegwit	pegwit	17777	461 [2.59]	559 [3.14]	609 [3.43]	585 [3.29]	624 [3.51]	606 [3.41]	625 [3.52]	648 [3.65]	657 [3.70]	656 [3.69]	1125 [6.33]

Table 2. Results for compaction of armcc executables

Group	Program	base	identical	$k = 2$	$k = 3$		$k = 4$		$k = 5$	$k = 6$	$k = 7$	$k = 8$	$k = \infty$
					G-Group	G-Pair	G-Group	G-Pair	G-Group	G-Pair	G-Group	G-Pair	G-Group
adpcm	rawadpcm	2721	67 [2.46]	68 [2.50]	71 [2.61]	69 [2.54]	75 [2.76]	69 [2.54]	69 [2.54]	69 [2.54]	69 [2.54]	69 [2.54]	69 [2.54]
	rawaudio	2701	67 [2.48]	68 [2.52]	71 [2.63]	69 [2.55]	75 [2.78]	69 [2.55]	69 [2.55]	69 [2.55]	69 [2.55]	69 [2.55]	69 [2.55]
	cjpeg	7229	128 [1.77]	190 [2.63]	255 [3.51]	234 [3.24]	320 [4.43]	255 [3.53]	265 [3.67]	274 [3.79]	287 [3.97]	294 [4.07]	367 [5.08]
	djpeg	7258	107 [1.47]	166 [2.29]	225 [3.10]	205 [2.82]	281 [3.87]	225 [3.10]	241 [3.32]	246 [3.39]	246 [3.39]	266 [3.66]	357 [4.92]
mesa	jpegtran	6927	110 [1.59]	199 [2.87]	260 [3.75]	242 [3.49]	328 [4.74]	266 [3.84]	286 [4.13]	292 [4.22]	300 [4.33]	304 [4.39]	408 [5.89]
	osdenn	22740	356 [1.57]	414 [1.82]	470 [2.07]	467 [2.05]	436 [1.92]	486 [2.14]	503 [2.21]	522 [2.30]	514 [2.26]	525 [2.31]	638 [2.81]
	texgen	20532	217 [1.06]	282 [1.37]	345 [1.68]	310 [1.51]	308 [1.50]	331 [1.61]	344 [1.68]	351 [1.71]	359 [1.75]	358 [1.74]	485 [2.36]
	mipmap	20795	236 [1.13]	294 [1.41]	348 [1.88]	333 [1.60]	314 [1.51]	350 [1.68]	371 [1.78]	383 [1.84]	390 [1.88]	390 [1.88]	470 [2.26]
g721	decode	3719	67 [1.80]	68 [1.83]	72 [1.94]	69 [1.86]	77 [2.07]	69 [1.86]	69 [1.86]	69 [1.86]	69 [1.86]	69 [1.86]	69 [1.86]
	encode	3719	67 [1.80]	68 [1.83]	72 [1.94]	69 [1.86]	77 [2.07]	69 [1.86]	69 [1.86]	69 [1.86]	69 [1.86]	69 [1.86]	69 [1.86]
pegwit	pegwit	9589	126 [1.31]	144 [1.50]	161 [1.68]	158 [1.65]	181 [1.89]	162 [1.69]	166 [1.73]	171 [1.78]	171 [1.78]	172 [1.79]	193 [2.01]

Code Generation for a Dual Instruction Set Processor Based on Selective Code Transformation^{*}

Sheayun Lee¹, Jaejin Lee¹, Sang Lyul Min¹,
Jason Hiser², and Jack W. Davidson²

¹ School of Computer Science and Engineering
Seoul National University, Seoul 151-742, Korea
sylee@archi.snu.ac.kr, jlee@cse.snu.ac.kr, symin@dandelion.snu.ac.kr

² Department of Computer Science, University of Virginia
Charlottesville, VA 22903, U.S.A.
jdh8d@cs.virginia.edu, jwd@virginia.edu

Abstract. Code size is an important design constraint in cost-sensitive embedded systems, since the amount of available memory is often limited. This constraint motivates dual instruction set processors, which support a reduced instruction set with a smaller instruction length in addition to a normal instruction set. This dual instruction set provides an effective mechanism for code size reduction. However, the code size reduction comes at the price of degraded performance because a program compiled into the reduced instruction set executes a larger number of instructions than the same program compiled into the normal instruction set. Motivated by this observation, we propose a technique that can be used to enable a flexible tradeoff between the code size and execution time of a program by using the two instruction sets selectively for different parts of a program. Our proposed approach determines the instruction set to be used for each basic block using a path-based profitability analysis, so that the execution time of the resulting program is minimized while the code size constraint is satisfied. The results from our experiments verify that the tradeoff relationship exists between a program's code size and execution time, and further indicate that the proposed technique can effectively exploit this tradeoff to improve performance within the given code size budget.

1 Introduction

Embedded systems are often characterized by stringent code size constraint of application programs, due to a limited amount of available memory. One promising approach for code size reduction is to use a *dual instruction set* processor [1]

^{*} This work was supported in part by the Ministry of Education under the Brain Korea 21 Project in 2003, and by the Ministry of Science and Technology under the National Research Laboratory program. The ICT at Seoul National University provided research facilities for this study.

that supports both a full (normal) instruction set and a reduced (compressed) instruction set. Examples of such processors include ARM with the 16-bit Thumb instruction set [2], MIPS 32/16-bit TinyRISC [3], and ARC Tangent [4] processors. By generating code using instructions in the reduced instruction set, we can achieve significant reduction in the code size because those instructions have a smaller bit width than the instructions in the full instruction set. However, a program compiled in the reduced instruction set typically runs slower than its full instruction set counterpart because the program in the full instruction set executes fewer instructions, since a single full instruction can perform more operations than a single reduced instruction. Another contributing factor to difference in performance between a full instruction set program and a reduced instruction set program is that the reduced instruction set can typically access only a subset of the registers accessible to the full instruction set. The greater number of registers accessible to the full instruction set allows more program variables to be allocated to registers (thereby reducing the overall number of memory references) and it offers more opportunities for the application of other code optimizations.

Intuitively, we can use the full instruction set for code sections that are frequently executed and thus critical for the performance of the entire program while using the reduced instruction set for the rest of the program to keep the code size as small as possible. Based on this observation, we propose a compiler technique that can be used to balance the tradeoff between the code size and execution time of a program when the mix of both instruction sets is used in generating its code. Given an application program and a constraint on its code size, our technique determines the instruction set to be used for each basic block in the program, in such a way that the execution time of the whole program is minimized while the code size does not exceed the given upper bound.

Our technique consists of three steps. First, we compile the whole program into the reduced instruction set. Then, we gather profile information running this code to determine which basic blocks are executed frequently. Second, based on the profile information, we determine the set of blocks to be converted into the full instruction set, in such a way that the resulting program gives the maximum reduction of the execution time while satisfying the code size constraint. The decision is based on a profitability analysis that accurately estimates the cost and benefit of transforming the blocks on each subpath in the program. Finally, we actually convert the selected blocks into the full instruction set and generate a mixed instruction set program as a result.

To show the validity and effectiveness of the proposed approach, we implemented our technique in the Zephyr compiler infrastructure [5] targeting the ARM/Thumb dual instruction set processor, and performed experiments on a set of benchmark programs. The results show that the tradeoff exists between the code size and execution time of a program, and that our proposed approach can effectively exploit the tradeoff.

The rest of the paper is organized as follows. In the next section, we summarize existing code generation approaches for dual instruction set processors.

Section 3 gives a brief overview of the dual instruction set processors and the issues in compiling code for them. In Section 4, we detail our proposed compiler technique, along with a formal description of our problem. Then, we describe the implementation of our technique for the ARM/Thumb processor and give the experimental results in Section 5. Section 6 concludes the paper with possible extensions to the proposed technique.

2 Related Work

Halambi et al. [1] developed a method to reduce the code size significantly by generating mixed instruction set code. In their approach, a given program is first compiled into generic instructions, and then translated into different instruction sets. Specifically, the technique groups consecutive instructions that can be translated into the reduced instructions, and decides whether to actually translate them based on the estimation of the size of the resulting code. Largely, the technique ignores the execution time of the resulting program, since their main objective is to minimize the code size by generating mixed instruction set code. Another approach for generating mixed instruction set code has been proposed by Krishnaswamy and Gupta [6]. They propose four different types of heuristics for a function-level coarse-grained approach, with emphasis on enhancing the instruction cache performance in terms of execution time and energy consumption. In addition, they provide a fine-grained approach, where they first compile the whole program into the reduced instruction set, and then identify and replace patterns of instruction sequences that are better executed by the full instructions. The effectiveness of this fine-grained approach is shown to be only comparable to the coarse-grained approach, since the full instruction set is applied to the instruction sequence that matches certain predefined patterns without estimating its impacts.

Our approach is distinguished from the previous approaches mentioned above, in that we determine the instruction set assignment on a per-basic-block basis, instead of substituting part of instruction sequences. This requires a control-flow analysis to determine the program points where the processor's execution mode should be switched. Therefore, our approach handles the mode switches by identifying the control flow edges where mode transitions should be triggered, and inserting appropriate instructions on each of them. Furthermore, the technique presented in this paper incorporates a detailed profitability analysis based on the concept of acyclic subpaths, which accurately estimates the cost and benefit of using different instruction sets for different parts of a given program. This technique enables a flexible tradeoff between code size and execution time by trying to improve the execution time as much as possible, while satisfying the constraint imposed on the total size of the program.

3 Dual Instruction Set Processors

A number of recent embedded microprocessors support a dual instruction set, where a reduced instruction set (Thumb for example) is provided in addition to a full instruction set. The main purpose of providing this reduced instruction set is to reduce code size by offering smaller instructions, usually half the length of full instructions. For example, it is reportedly known that a program compiled into Thumb instructions is on average 30 % smaller than the same program compiled into ARM instructions [7]. This code size reduction is achieved at the cost of increased execution time because multiple reduced instructions need to be executed to perform an operation of a full instruction in general.

The dual instruction set processors provide a mechanism to change its execution mode at run-time, so that some parts of a program can be written in one mode (e.g., reduced instruction set) and the rest in the other mode (e.g., full instruction set). Specifically, at a given time instance, the processor executes instructions in either of the two modes that can be dynamically altered. The mode transitions are typically triggered by executing a special instruction or sequence of instructions. For example, the ARM/Thumb dual instruction set processor provides a special form of branch instruction, called *branch and exchange* (**bx**), which takes a register operand for the branch target address. When a **bx** instruction is executed, the processor transfers the flow of control to the address specified by the register operand. The target address is aligned to a 16-bit boundary for Thumb and 32-bit boundary for ARM. The least significant bit of the register operand, which is not used as part of the branch target address, is used to indicate the mode in which the processor will execute after executing the **bx** instruction. Note that a single mode transition possibly requires the execution of multiple instructions, since the value of the register operand should be set with the target address before the **bx** instruction is executed.

The ARM Developer Suite (ADS) compiler currently supports the mixed use of the two instruction sets at the module-level (i.e., file-level) by means of command-line options. This is called ARM/Thumb interworking [8].

When ARM/Thumb interworking is enabled, the compiler/linker generates code in such a way that mode transitions are properly handled by the **bx** instruction on each call and return for the functions in modules of different instruction set modes. Using this interworking mechanism, the user can compile the time-critical functions in ARM mode for faster execution while generating the rest of the program in Thumb mode for smaller code size.

In general, however, the degree of freedom in the module-level interworking is insufficient to fully exploit the tradeoff between code size and execution time because mode switching is limited to the function boundary. For example, suppose a given program has a tight loop that executes many times and contains two alternative execution paths, one being executed most of the time and the other seldom executed (e.g., one path representing the common case while the other handling exceptions). In such cases, a more desirable approach is using a finer-grained method than the module level because only frequently executed sections of the code can be compiled into the full instruction set in order to

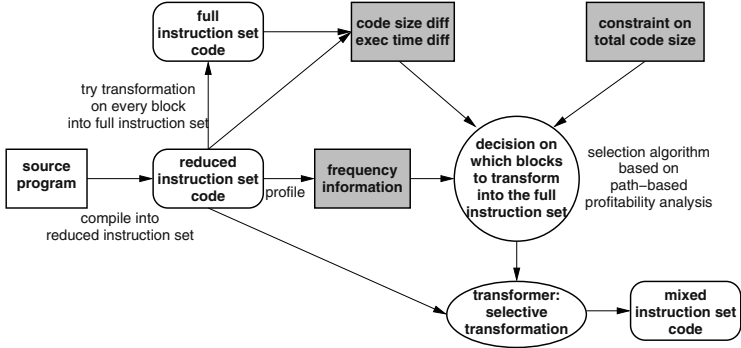


Fig. 1. Overview of our mixed mode code generation

improve performance while maintaining a small code size. Note that such an approach requires special attention to the overhead arising from the mode switch instructions. Specifically, insertion of a large number of mode switch instructions can substantially increase the code size. Moreover, frequent execution of the mode switch instructions degrades the overall performance of the program by incurring large execution time overhead. Therefore, a fine-grained approach requires a detailed analysis of the impacts on the code size and execution time by the mixed use of the two different instruction sets. In the next section, we propose a technique that provides a solution to this problem by determining the type of the instruction set to be used on a per-basic-block basis. The technique is based on an accurate path-based analysis of the code size and execution time, taking the mode switching overhead into account.

4 Compilation for a Dual Instruction Set Processor

This section details the proposed technique for compiling a given program for a processor that supports a dual instruction set. Section 4.1 outlines our proposed approach for code generation using a dual instruction set, based on selective code transformation from the reduced instruction set into the full instruction set. We give a formal description of the problem of selective code transformation in Section 4.2. Finally, Section 4.3 describes our path-based profitability analysis and a selection algorithm that determines the type of instructions to be used for each basic block.

4.1 Our Approach

As explained earlier in Section 1, our proposed technique first compiles the whole program into the reduced instruction set and then selectively transforms a set of basic blocks into the full instruction set. We take this approach because one or more reduced instructions can be combined and translated into a full instruction,

while in general the transformation in the other direction cannot be mechanically done.

The procedure of selective code transformation is illustrated in Figure 1. In order to determine the set of basic blocks to be transformed into the full instruction set in code generation, we need information about the code size and execution time of each basic block compiled into both the reduced instruction set and the full instruction set. The code size of each basic block can be estimated in a straightforward manner because it can be statically determined by examining the instruction sequence in the block. On the other hand, we assume that the execution time of each basic block can be estimated using a simple model for the given processor architecture. We obtain the code size and execution time information of each basic block in the full instruction set by performing the transformation on the block without generating code.

In addition to the code size and execution time of each block, the selection algorithm requires information about their execution frequency. This frequency information is obtained by profiling the given program that is compiled into the reduced instruction set. The profile information for each block combined with the code size and the execution time differences is the input to the basic block selection algorithm. Based on its results, the selected blocks are transformed into the full instruction set and the final mixed instruction set code is generated.

4.2 Problem Description

A program can be represented by a control flow graph $P = \langle V, E \rangle$, where V is the set of basic blocks and E is the set of edges which represent the control flow in the program. When the program has a total of n basic blocks,

$$V = \{v_i \mid i = 1, 2, \dots, n\}, \quad (1)$$

$$E = \{e_{ij} = \langle v_i, v_j \rangle \mid \text{there is a control flow from } v_i \text{ to } v_j\}. \quad (2)$$

We define a set of functions to denote the code size and the execution time of a basic block when compiled into the two different instruction sets. Let $s_F(v)$ and $s_R(v)$ denote the code size of a block v compiled into the full and the reduced instruction sets, respectively. Similarly, we denote by $t_F(v)$ and $t_R(v)$ the execution time of block v compiled into the full and the reduced instruction sets, respectively.

The problem is to determine the type of the instruction set to be used for each basic block, so that the execution time of the resulting mixed instruction set program is minimized while maintaining the total code size under a given upper bound. That is, we try to find a mode assignment $f : V \rightarrow \{\alpha, \beta\}$ that minimizes the execution time of the whole program and satisfies the code size constraint, where α and β denote the full and the reduced instruction sets, respectively. This assignment partitions the set of basic blocks into two disjoint subsets $F = \{v \mid f(v) = \alpha\}$ and $R = \{v \mid f(v) = \beta\}$.

Assume that each basic block has already been assigned the type of instruction set to be compiled into. Then the total code size can be computed by

summing the size of each block and adding the code size increase s^* due to the mode switch instructions:

$$S = \sum_{v \in F} s_F(v) + \sum_{v \in R} s_R(v) + s^* . \quad (3)$$

In order to compute the reduction of execution time achieved by transforming the blocks in F into the full instruction set, we first sum the difference in execution time for each basic block v multiplied by its dynamic execution count $c_V(v)$, and then subtract the execution time overhead t^* due to executing mode switch instructions:

$$\Delta_t = \sum_{v \in F} c_V(v) \times (t_R(v) - t_F(v)) - t^* . \quad (4)$$

Because the code size overhead s^* results from the mode switch instructions inserted during the transformation, we need to identify the program points where mode switch instructions are inserted in order to estimate this overhead. A mode transition occurs when the control flows from a block in F to a block in R and vice versa. Thus, a mode switch instruction can be inserted on an edge of the control flow graph. The set of control flow edges along which a mode switch occurs is given as follows:

$$E^* = \{e_{ij} \in E \mid (v_i \in F \wedge v_j \in R) \vee (v_i \in R \wedge v_j \in F)\} . \quad (5)$$

Now the code size overhead s^* is given by a straightforward equation:¹

$$s^* = o_s \times |E^*| , \quad (6)$$

where o_s denotes the total size of instructions required for a single mode switch.² On the other hand, the execution time overhead t^* is computed by,

$$t^* = o_t \times \sum_{e \in E^*} c_E(e) , \quad (7)$$

where o_t denotes the execution time overhead incurred by the instructions that switch the execution mode once, and $c_E(e)$ gives the dynamic execution count of the edge e .

In summary, the selective code transformation problem can be formulated as the following constrained optimization problem: Given a program $P = \langle V, E \rangle$, find an assignment $f : V \rightarrow \{\alpha, \beta\}$ (i.e., a partition of V into F and R), such that it maximizes

$$\Delta_t = \sum_{v \in F} c_V(v) \times (t_R(v) - t_F(v)) - o_t \times \sum_{e \in E^*} c_E(e) , \quad (8)$$

¹ Since mode switch instructions can be shared among the control flow edges in E^* that fall to the same basic block, we can be more accurate by defining $V^* = \{v_j \mid \exists i, e_{ij} = \langle v_i, v_j \rangle \in E^*\}$ and letting $s^* = o_s \times |V^*|$.

² In general, the overhead of mode switch instructions from the reduced instruction set to the full instruction set is different from that of the mode switch instructions in the other way. For simplicity, we use a single constant o_s without loss of generality. The same applies to the execution time overhead constant o_t .

and satisfying

$$S = \sum_{v \in F} s_F(v) + \sum_{v \in R} s_R(v) + o_s \times |E^*| \leq U_s, \quad (9)$$

where U_s gives the upper bound on the maximum code size for the whole program.

4.3 Path-Based Selective Code Transformation

One might be tempted to solve the problem using 0/1-ILP (integer linear programming). Indeed, the problem might well be mapped to a 0/1-ILP formulation if Equations 8 and 9 do not include the overhead due to mode switches. However, if we consider the mode switching overhead, the problem cannot be solved by an ILP technique because the instruction set assignment of a basic block affects the assignment of its adjacent blocks. One straightforward but impractical solution is to estimate the objective function (Equation 8) and the constraint (Equation 9) for all possible combinations of the mode assignments. This approach causes combinatorial explosion, since it would require evaluation of 2^n different combinations of mode assignments, when the number of blocks in the program is n . Therefore, we need an approximation method that can be efficiently used to assess the impact of our transformation on the code size and execution time.

For an approximation method, we define a cost-benefit model based on *intraprocedural acyclic subpaths* [9]. They are the maximal subpaths within a function that do not traverse any back edge of a loop. Since the acyclic subpaths capture the set of basic blocks executed together, this cost-benefit model can be effectively used for selection of basic blocks to be transformed.

We define the *cost* of transforming a control flow path to be the increase in the code size when the basic blocks on the path are transformed from the reduced instruction set into the full instruction set. Of course, we take into account the insertion of mode switch instructions during the transformation. On the other hand, the *benefit* of transforming a path is defined as the reduction in execution time achieved by transforming the basic blocks on the path taking into account the execution time overhead due to mode switches. Based on this cost-benefit model, we select the blocks to be transformed as follows. First, we enumerate all the acyclic subpaths in the given program, and estimate the cost and benefit associated with each of them. Then we apply a simple heuristic that iteratively selects the subpath to be transformed by giving priority to the one with the maximum benefit per unit cost (i.e., $\frac{\text{benefit}}{\text{cost}}$).

In order to compute the cost of transforming a path, we first sum the code size differences of all blocks being transformed. Then we estimate the mode switch overhead caused by the transformation of the path and add it to the sum. Note that transforming the blocks on the path not only causes insertion of new mode switch instructions, but also possibly results in removal of certain mode switch instructions that were previously needed. Specifically, when a block is transformed into the full instruction set, we should remove the mode switch

instructions that were previously inserted on the edges connecting the block with other blocks that are already in the full instruction set.

To account for the removal of mode switch instructions as well as the insertion of newly introduced mode switch instructions, we define $E^M(p)$ to be the set of edges where mode switch instructions are newly introduced, and $E^m(p)$ to be the set of edges where existing mode switch instructions are removed. In addition, let $V(p)$ denote the set of all the basic blocks on a path p . Then, the set of blocks to be transformed on the path p is given by $V(p) \cap R$, which contains only those blocks on p that have not yet been transformed. Then the cost $c(p)$ of transforming p can be computed as follows:

$$c(p) = \sum_{v \in V(p) \cap R} (s_F(v) - s_R(v)) + o_s \times (|E^M(p)| - |E^m(p)|) . \quad (10)$$

On the other hand, the benefit $b(p)$ of transforming a path can be computed by first summing the execution time difference for each block multiplied by its execution frequency, and then subtracting the mode switch overhead. That is, the benefit associated with transformation of a path p is given by

$$\begin{aligned} b(p) = & \sum_{v \in V(p) \cap R} (c_V(v) \times (t_R(v) - t_F(v))) \\ & - o_t \times \left(\sum_{e \in E^M(p)} c_E(e) - \sum_{e \in E^m(p)} c_E(e) \right) . \end{aligned} \quad (11)$$

Now we define a reward function $r(p)$ for each subpath p to be the ratio of $b(p)$ to $c(p)$. That is, the reward function for path p is given by

$$r(p) = b(p)/c(p) , \quad (12)$$

which indicates the expected amount of execution time reduction for the unit increase in the code size.

Based on this cost-benefit model, we apply a simple greedy heuristic as follows. First we set the code size budget equal to the difference between the upper bound of the code size and the total code size of the program compiled entirely into the reduced instruction set. We begin by enumerating all the intraprocedural acyclic subpaths for the program, and compute $c(p)$, $b(p)$, and $r(p)$ for each subpath p . Then the selection algorithm iteratively chooses a path with the maximum reward function value, among those subpaths whose transformation cost is less than or equal to the remaining code size budget.³ When the selection process goes on, the code size budget is adjusted accordingly, as well as the set of candidate subpaths. We repeat this process until no more transformations can be done because one or more of the following conditions are met:

³ A subpath with a negative cost is always given a higher priority than others provided that the benefit associated with that path is nonnegative, because transforming such a subpath will decrease the code size while improving the execution time of the whole program.

V : set of all basic blocks in the flow graph
 U_s : upper bound on the total code size
 S_R : code size of the program when compiled entirely
 into the reduced instruction set

 $B \leftarrow U_s - S_R$
 $R \leftarrow V$
 $F \leftarrow \phi$
 $P \leftarrow \{\text{intraprocedural acyclic subpaths}\}$

do {
 for each $p \in P$, calculate $r(p) = b(p)/c(p)$
 select $p \in P$ with maximum $r(p)$ with $c(p) \leq B$
 $B \leftarrow B - c(p)$
 $F \leftarrow F \cup V(p)$
 $R \leftarrow R - V(p)$
 $P \leftarrow P - \{p \mid V(p) \cap R = \phi\}$
} while ($B \geq \min_{p \in P} \{c(p)\} \wedge \max_{p \in P} \{b(p)\} \geq 0 \wedge R \neq \phi$)

Fig. 2. Path-based algorithm for selection of blocks to be transformed. After the algorithm is finished, F will contain the blocks to be transformed from the reduced instruction set into the full instruction set

1. Selection of any of the remaining subpaths would violate the code size limit,
2. No further reduction of the execution time is possible, or
3. All the blocks in the program have already been transformed.

Note that, when a subpath is selected and its blocks are determined to be transformed, the cost and benefit of other subpaths may change because (1) the transformed blocks may be shared between the selected subpath and other paths, and (2) introducing or removing mode switch instructions possibly affects the cost and benefit of other subpaths. Therefore, we adjust the cost and benefit of each subpath in each iteration of our process.

Figure 2 illustrates our greedy selection algorithm. Although the resulting instruction set assignment is not guaranteed to be optimal, the algorithm gives an adequate solution to our problem, since (1) the acyclic subpaths well reflect the dynamic behavior of the program, and (2) the greedy nature of the algorithm favors a subpath that results in the maximum reduction of execution time in each iteration.

5 Implementation and Results

In this section, we describe our implementation of the proposed technique targeted for the ARM/Thumb dual instruction set processor, and present the results from our experiments to show the validity and effectiveness of the proposed approach.

5.1 Implementation

We implemented the algorithms described in the previous section in the Zephyr compiler infrastructure that features an optimizer called VPO (very portable optimizer) [10]. The transformation is enabled by a machine-independent intermediate representation called RTL (register transfer lists), combined with the instruction selection mechanism of VPO based on peephole optimization [11]. Since the Thumb instruction set is a proper subset of the ARM instruction set, one or more RTL statements for the Thumb architecture can be translated into one ARM assembly instruction with a simple modification to the VPO's instruction selection algorithm.

In addition to modifying the code generation interfaces, we implemented a profiler by porting part of EASE (Environment for Architecture Study and Experimentation) [12] that is a program instrumentation framework based on VPO. The profiler inserts a minimal set of instrumentation code sufficient for gathering the execution count of each basic block in the program. In addition to the block execution counts given by the profiler, the proposed approach requires the execution count of each control flow edge, which cannot be generally extracted from the basic block profile [9]. Therefore, we derive the edge frequencies using an approximation algorithm explained in [13].

To determine the set of blocks to be transformed, we implemented the path-based profitability analysis⁴ and the selection algorithm. The algorithm requires information about the differences of code size and execution time for each block. The code size is estimated directly from the instruction sequence in each block, compiled into ARM and Thumb instructions, respectively. On the other hand, the execution time of each basic block is approximated by the instruction count for that block. The simple instruction set structure and pipeline organization of our target processor allows us to roughly estimate the execution time in this way, especially because we only need the relative difference in the execution time of each block compiled into the two different instruction sets. Nonetheless, an accurate analysis of execution time based on a detailed hardware model would enhance the precision of the analysis.

After the analysis, each basic block is annotated in its RTL representation with the instruction set assignment obtained during the analysis. Finally, the VPO-based code transformer applies a two-phase instruction selection on the annotated RTL representation of the program to generate a mixed instruction set target program. That is, we translate the RTL statements belonging to the blocks to be in Thumb mode in the first phase, and then the second phase generates ARM code for the blocks to which ARM instruction set is assigned.

⁴ Note that, for large programs, the number of acyclic subpaths can be considerable. Therefore, to maintain the problem complexity, we applied heuristic pruning of subpaths based on thresholds on the edge frequencies and the path lengths.

Table 1. Benchmark programs used in the experiments. The first and the second column show the name of each program and the benchmark suite from which the program is taken, respectively, while the last column gives a brief description of the program

Name	Source	Description
crc	MiBench	32-bit CRC checksum computation
sha	MiBench	secure hash algorithm (160-bit)
adpcm.rawaudio	MiBench	adaptive differential pulse code modulation
adpcm.rawdaucio	MiBench	speech encoding and decoding
G.721.encode	MediaBench	CCITT G.721 voice
G.721.decode	MediaBench	compression and decompression

5.2 Evaluation Environments

We evaluate our proposed approach using a set of benchmark programs. Table 1 summarizes the programs used in our experiments. They are taken from MiBench [14] and MediaBench [15] benchmark suites that are collections of application programs commonly used in embedded systems. These applications are executed on an evaluation board with an Intel XScale core-based PXA250 processor and 64 MB of main memory, running a port of ARM Linux [16]. We measure the execution time of programs by calling the `gettimeofday()` system call.

5.3 Results

Figure 3 summarizes the results from our experiments. We build six different versions of executables. Executables T and A are obtained by compiling the whole program into Thumb instructions and into ARM instructions, respectively. Executable A' is generated by transforming all the basic blocks in T into ARM instructions. This is used for the purpose of execution time comparison with the mixed instruction set code, since it gives the baseline result for the performance of the program when a subset of blocks are transformed from Thumb instructions into ARM instructions. We generate four different versions of mixed instruction set code, with different code size limits. We set the code size limits so that the code size budget for each mixed instruction set code is 20 %, 40 %, 60 %, and 80 % of the size difference between code A' and T .

In the figure, the code size of each program is normalized to that of T , while the execution time is normalized to that of A' . We observe that a program compiled into Thumb instructions (T) is significantly smaller than the same program compiled into ARM instructions (A). This difference in size is dependent on the characteristics of each program, which ranges from 28 % to 42 % for the set of our benchmark programs. In addition, as we increase the code size limit, the execution time of the resulting mixed instruction set code decreases until the point where the execution time is close to the execution time of A' . Note

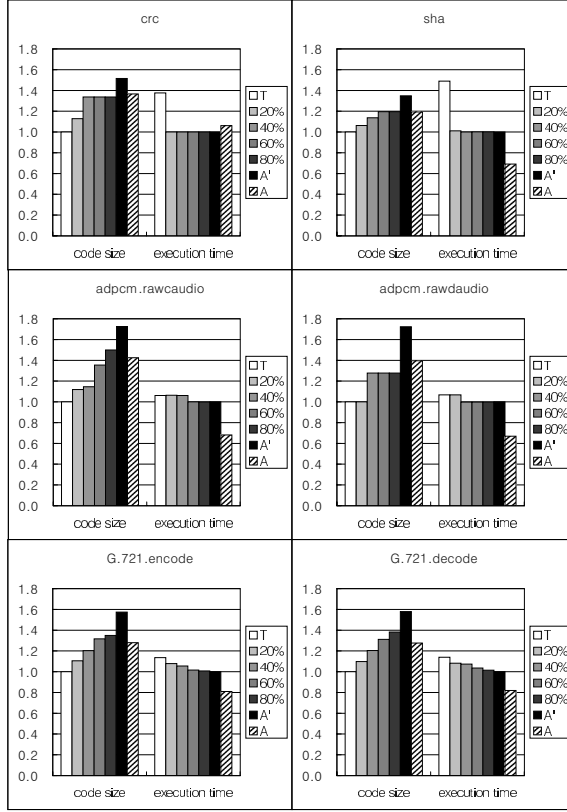


Fig. 3. Code size and execution time comparison for benchmark programs

that the execution time reduction achieved by the selective code transformation varies from one program to another. We also observe that the code size of A' is often larger than that of A . After careful examination of the resulting code for the programs, we discovered that this results from the fact that a larger number of instructions are generated when the program is first compiled into the Thumb instruction set and later transformed into the ARM instruction set, mainly due to the difference in register allocation. That is, since the transformation uses only those registers initially allocated to the program variables in Thumb mode, the transformed code possibly has a larger number of instructions for memory loads and stores that are not found in the pure ARM version.

For *crc*, the proposed technique reduces the execution time remarkably even for a slight increase in the code size. This can be explained by the fact that the program spends most of its execution time on a tight loop that accounts for only a small portion of the whole program size. Therefore, after the execution

time reaches a certain point, no further reduction in execution time is possible although we increase the code size limit. Interestingly, the mixed instruction set versions of `crc` outperform the program compiled entirely in ARM instructions (A). This seemingly anomalous behavior results from the number of registers saved and restored in a function prologue and epilogue. Specifically, in a function repeatedly called inside the tight loop, the mixed instruction set version allocates fewer registers to program variables than the ARM version does, resulting in a smaller execution time for saving and restoring the callee-saved registers.

The result for `sha` is similar to the case of `crc`, in that the execution time can be significantly reduced by using a small amount of additional code space. One notable difference is that, for `sha`, the execution time of A is substantially smaller than that of A' . This performance gap results from the high register pressure in a frequently executed section of the program. Since code transformation from Thumb to ARM uses only those registers that are visible from the Thumb instruction set, A' requires extra memory access time for the local variables that cannot be register allocated, whereas A possibly has those variables allocated to registers. Nonetheless, the mixed instruction set programs run approximately 32 % faster than T , while their code size is roughly 11 % smaller than that of A .

In the case of `adpcm.rawaudio` and `adpcm.rawdaudio`, the execution time reduction by the selective code transformation is marginal. For instance, in `adpcm.rawaudio`, the execution time reduction is only 6 % from T to A' . For these two programs, the transformation does not dramatically decrease the number of instructions executed because the frequently executed instructions are mostly memory loads and stores and each of them is expressed as one instruction both in the ARM and in the Thumb instruction sets.

The results for `G.721.encode` and `G.721.decode` clearly illustrate the trade-off between code size and execution time. As we increase the code size limit, the size of the mixed instruction set code increases accordingly while the execution time is gradually reduced. Compared with the other applications, these two programs have a large number of frequently executed basic blocks in different locations. Therefore, a large degree of freedom is given to the algorithm for selecting blocks to be transformed, which in turn results in more flexible tradeoff between code size and execution time. However, a noticeable performance gap still remains between A' and A . This results from the fact that A' has fewer registers available than A when a basic block is transformed from the Thumb instruction set to the ARM instruction set. This suggests that a post-pass register allocation algorithm combined with the selective code transformation would further improve the execution time of mixed instruction set code. The register allocation for mixed instruction set code introduces a challenging problem: If a variable accessed by a basic block b_F in the full instruction set and another block b_R that is in the reduced instruction set, and if it is allocated to a register in b_F that is not visible from the reduced instruction set, additional instructions should be inserted to move the variable to the register that is visible to the reduced instruction set before it is accessed by b_R .

6 Conclusions and Future Work

We have presented an approach to enable a flexible tradeoff between code size and execution time by generating mixed instruction set code for a dual instruction set processor. The proposed technique generates a mixed instruction set code in a way that the execution time of the resulting program is minimized, while the code size is maintained under a given upper bound. Our approach first compiles the whole program into the reduced instruction set, and then selectively transforms a subset of basic blocks into the full instruction set using profile information. To determine the set of blocks to be transformed, we use a path-based profitability analysis technique combined with a heuristic selection algorithm.

The proposed technique has been implemented for the ARM/Thumb dual instruction set processor, whose validity and effectiveness has been demonstrated by experiments on a set of benchmark programs. The results show that the tradeoff between the code size and execution time can be effectively exploited by using the two different types of instruction sets selectively for different parts of a given program. Our approach provides a strong base in exploiting this tradeoff in order to enhance the performance of the program as much as possible within a given code size budget.

Our future research will focus on developing an efficient register allocation algorithm for dual instruction set processors. An effective register allocation algorithm is expected to boost the applicability of the proposed approach by improving the performance of mixed instruction set code.

References

1. A. Halambi, A. Shrivastava, P. Biswas, N. Dutt, and A. Nicolau. An efficient compiler technique for code size reduction using reduced bit-width ISAs. In *Proceedings of the DATE (Design, Automation and Test in Europe)*, Paris, France, March 2002.
2. S. Furber. *ARM System Architecture*. Addison-Wesley, 1996. ISBN 0-201-40352-8.
3. K. Kissel. MIPS16: High-density MIPS for the embedded market. Technical report, Silicon Graphics MIPS Group, 1997.
4. ARC Cores (<http://www.arc.com>). *The ARCTangent-A5 Processor*.
5. A. Appel, J. Davidson, and N. Ramsey. The zephyr compiler infrastructure. <http://www.cs.virginia.edu/zephyr>, 1998.
6. A. Krishnaswamy and R. Gupta. Profile guided selection of ARM and Thumb instructions. In *Proceedings of LCTES/SCOPES*, Berlin, Germany, June 2002.
7. L. Goudge and S. Segars. Thumb: Reducing the cost of 32-bit RISC performance in portable and consumer applications. In *Proceedings of COMPCON*, 1996.
8. ARM Limited. *ARM Developer Suite Developer Guide*.
9. T. Ball and J.R. Larus. Efficient path profiling. In *Proceedings of the 29th Annual IEEE/ACM Symposium on Microarchitecture*, pages 46–57, Paris, France, 1996.
10. M.E. Benitez and J.W. Davidson. Target-specific global code improvement: Principles and applications. Technical Report CS-94-42, Department of Computer Science, University of Virginia, April 1994.

11. J.W. Davidson and C.W. Fraser. Code selection through object code optimization. *ACM Transactions on Programming Languages and Systems*, 6(4):505–526, October 1984.
12. J.W. Davidson and D.B. Whalley. A design environment for addressing architecture and compiler interactions. *Microprocessors and Microsystems*, 15(9):459–472, November 1991.
13. A. Tamches and B.P. Miller. Dynamic kernel code optimization. In *Proceedings of the 3rd Workshop on Binary Translation*, Barcelona, Spain, June 2001.
14. M.R. Guthaus, J.S. Ringenberg, D. Ernst, T.M. Austin, T. Mudge, and R.B. Brown. MiBench: A free, commercially representative embedded benchmark suite. In *Proceedings of the IEEE 4th Annual Workshop on Workload Characterization*, Austin, TX, December 2001.
15. C. Lee, M. Potkonjak, and W.H. Mangione-Smith. MediaBench: A tool for evaluating and synthesizing multimedia and communications systems. In *Proceedings of the 30th Annual International Symposium on Microarchitecture*, pages 330–335, December 1997.
16. ARM Linux Project. <http://www.arm.linux.org.uk>.

Code Instruction Selection Based on SSA-Graphs

Erik Eckstein¹, Oliver König¹, and Bernhard Scholz²

¹ ATAIR Software GmbH, Vienna, Austria
{eckstein,koenig}@atair.co.at

² Institute of Computer Languages, Vienna University of Technology, Austria
scholz@complang.tuwien.ac.at

Abstract. Instruction selection for embedded processors is a challenging problem. Embedded system architectures feature highly irregular instruction sets and complex data paths. Traditional code generation techniques have difficulties to fully utilize the features of such architectures and typically result in inefficient code.

In this paper we describe an instruction selection technique that uses static single assignment graphs (SSA-graphs) as underlying data structure for selection. Patterns defined as graph grammar guide the instruction selection to find (nearly) optimal results. We present an approach which maps the pattern matching problem to a *partitioned boolean quadratic optimization problem* (PBQP). A linear PBQP solver computes optimal solutions for almost all nodes of a SSA-graph.

We have implemented our approach in a production DSP compiler. Our experiments show that our approach achieves significant better results compared to classical tree matching.

1 Introduction

Highly specialized processors such as digital signal processors (DSP) or micro controller systems feature irregularities in their instruction sets. Therefore code generation for these processors is still a research topic and is not satisfying solved so far.

In a traditional compiler framework code generation is decomposed in several sub-problems. The main building blocks of a code generator are *instruction selection*, *instruction scheduling*, and *register allocation*. First, a front end of a compiler translates the source program into an *intermediate representation*. After performing high-level optimizations, the instruction selector translates the intermediate representation into target code. Instruction scheduling reorders the target code to keep register pressure low and to utilize pipelining and parallel units of the target architecture. Register allocation assigns hardware registers to pseudo registers. Beside these three building blocks, most compilers for embedded systems also perform additional optimizations to utilize target dependent hardware features, e.g. addressing modes [3].

Tree pattern matching is a widely used technique for instruction selection [1]. Usually the unit of translation is a statement which is represented as a data flow tree (DFT). A set of rules is used to match the DFT. The matcher selects those rules such that the sum of all applied rule costs is a minimum. An algorithm for tree pattern matching has two phases: labeling and reducing. In the labeling phase minimal costs are calculated for each node and each non-terminal. This is done by checking each non-terminal combination in a bottom-up walk of the tree. In the reduction phase the tree is traversed top-down and the rules with minimal costs are selected. The tree matching algorithm employs *dynamic programming* firstly introduced by BEG [8] and BURG [6]. The dynamic programming approach is performed in linear time. Though this technique is fast, it does not consider the computational flow of a function.

DAG matching is an extension to tree matching. Instead of trees, directed acyclic graphs are considered. DAG matching is a NP-complete problem. A proof for NP completeness of matching DAGs is given by [11]. In the work of Ertl [4] an approach is presented, which modifies the tree pattern matcher algorithm so that it can be used on DAGs. A checker proves whether the DAG matching algorithm yields optimal results for a specific grammar. This approach differs from our approach in some points: First, the algorithm does code duplication. Second, it is not possible to perform the algorithm on a graph containing cycles, because it still relies on the bottom-up and top-down phases of the tree pattern matcher. DAG matching was also mapped to the binate covering problem [10]. However, DAG matching still does not consider the computational flow of functions.

Beside the dynamic programming method, there are a number of specialized approaches for code generation with pattern matching. Leupers introduced code selection for SIMD instruction generation, based on integer linear programming [9].

This paper presents a new technique for instruction selection of code generators. In contrast to previous approaches the computational flow of a whole function is taken into account. For representing the computational flow the SSA-graph is used which combines data flow trees (DFT) and def-use relations of a function. An ambiguous grammar describes possible derivations of the SSA-graphs. Production rules have cost terms and code templates. Cost terms are used to find the derivation with minimal overall costs. Unlike conventional approaches, parsing SSA-graphs is more difficult since cycles are allowed in the graphs.

Parsing generic graphs is NP-complete since even parsing DAGs is NP-complete [11]. To get a handle on the problem, we map the instruction selection problem for SSA-graphs to *partitioned boolean quadratic problem* (PBQP). The basic concept of our SSA-graph matching algorithm is shown in Figure 1. First, the SSA-graph with its ambiguous grammar is mapped to PBQP. Second, the PBQP solver computes the grammar derivation with minimal costs. Third, based on the grammar derivation code is produced.

Note that the PBQP solver consists of two phases: In the first phase the graph is reduced until a trivial solution remains. In the second phase the solution is

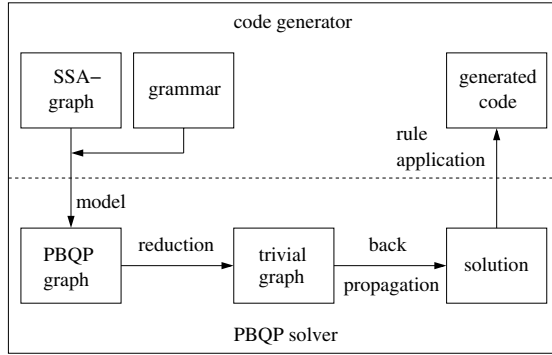


Fig. 1. Instruction selection

back-propagated. The two phases of the PBQP solver are very similar to the two phases of the dynamic programming algorithm of tree pattern matchers. In fact, if the PBQP-graph is a tree, the two algorithms are almost identical. The significant difference is that a tree pattern matcher decides between non-terminals whereas the PBQP solver decides between rules.

Though the PBQP is NP-complete our PBQP solver [12,3] computes a solution in linear time. For a negligible number of SSA-graph nodes (see Section 5), no optimal solution can be computed and heuristics are applied. Consequently, the PBQP solution is nearly optimal.

Our approach goes beyond existing work by considering the computational flow of a function. Based on SSA-graphs we can produce better code quality in comparison to conventional techniques that only consider statements or sequences of statements. Experimental results show that we achieve significantly better results compared with classical tree pattern matching methods.

Our paper is organized as follows. In Section 2 we motivate our approach. A running example is shown. In Section 3 we map the instruction selection problem to the Partitioned Boolean Quadratic Problem (PBQP). In Section 4 we give a brief overview of the PBQP algorithm [12] and some specific extensions for the algorithm. In Section 5 we show some experimental results of a production compiler and in Section 6 we draw our conclusion.

2 Motivation

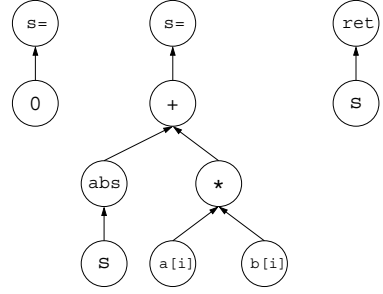
Consider the example in Figure 2 that shows a typical DSP code. The elements of two vectors **a** and **b** are multiplied and the absolute value of the last iteration is added. The example stresses the usage of accumulator variable **s** that occurs in three statements. Note that the loop control code is abstracted in pseudo code.

Let us assume that the computations for Variable **s** are performed in fixed point arithmetic on a DSP processor. In contrast to standard processors, DSP

```

int f(short *a, short *b)
{
(1)   int s = 0;
      loop(i) {
(2)     s = abs(s) + a[i] * b[i];
      }
(3)   return s;
}

```

Fig. 2. Example source code**Fig. 3.** Data flow trees of example

processors have multiplication units that perform a multiplication by shifting the result by one bit to the left. This multiplication idiosyncrasy was specifically designed for DSP algorithms. However, for compilers it is difficult to exploit this shift. Without knowing the context of the computation an additional shift operation is needed to re-adjust the multiplication result.

For obtaining faster code, computations inside the loop should be performed with a shifted result by one bit to the left. Otherwise an additional shift-operation would be introduced inside the loop and would worsen runtime. Since the return statement requires an un-shifted value, a shift operations has to be inserted prior to the return statement outside of the loop.

To express architectural computation properties (e.g. shifted or un-shifted) we use a graph grammar consisting of terminals, non-terminals, productions and a start-symbol. Terminals represent specific nodes such as a plus operation, etc. Non-terminals describe sub-graphs and the productions describe how non-terminals are derived and at which costs. Note that graph grammars are ambiguous in most cases since several semantically correct code selections exist. The objective of code selection is to find a grammar derivation for the graph with minimal costs. For generic graphs this problem is NP-complete since even for directed acyclic graphs it is already NP-complete [11]. Only for trees optimal and efficient algorithms exist [5].

- | | |
|---|---|
| (1) $\text{reg} \rightarrow \text{const}(0) [], 1, r=0$ | (8) $\text{reg} \rightarrow \text{load}[\text{ptr}], 5, r=*ptr$ |
| (2) $\text{sreg} \rightarrow \text{const}(0) [], 1, r=0$ | (9) $\text{top} \rightarrow \text{ret}[\text{reg}], 1, ret$ |
| (3) $\text{reg} \rightarrow +[\text{reg}, \text{reg}], 3, r=r+r$ | (10) $\text{reg} \rightarrow \text{sreg}, 1, r=r>>1$ |
| (4) $\text{sreg} \rightarrow +[\text{sreg}, \text{sreg}], 3, r=r+r$ | (11) $\text{sreg} \rightarrow \text{reg}, 1, r=r<<1$ |
| (5) $\text{reg} \rightarrow \text{abs}[\text{reg}], 2, r=\text{abs}(r)$ | (12) $\text{reg} \rightarrow \text{s}[], 0$ |
| (6) $\text{sreg} \rightarrow \text{abs}[\text{sreg}], 2, r=\text{abs}(r)$ | (13) $\text{top} \rightarrow \text{s}=[\text{reg}], 0$ |
| (7) $\text{sreg} \rightarrow *[\text{reg}, \text{reg}], 4, r=r*r$ | |

Fig. 4. Production rules

For our running example the production rules are given in Figure 4. A production has a left-hand side and a right-hand side, i.e. $nt \rightarrow pattern, cost, code$.

On the left-hand side a non-terminal specifies the result of the computation. On the right-hand side there is a *pattern* that consists of terminals and non-terminals. In addition the matching *cost* and the *code* template are given (separated by commas). Note that the code templates are only shown for better understanding of the rules, but they do not influence the matching algorithm.

In our grammar the shift property of the multiplication is represented by two non-terminals: **reg** and **sreg**. Nonterminal **reg** represents an un-shifted value whereas **sreg** represents a value which is shifted left by one bit. For example the multiplication rule requires two un-shifted input values and produces a shifted value. This is reflected in Rule 7. Plus operations and absolute value operations can be performed with un-shifted values (Rules 3 and 5) or shifted values (Rules 4 and 6). The constant 0 can either be loaded as shifted or un-shifted value (Rules 1 and 2). The memory load is represented by Rule 8 and can only produce an un-shifted value. Return statements require un-shifted values to preserve program semantics (Rule 9). Rules 10 and 11 are chain-rules that convert a shifted value to an un-shifted value and vice versa.

In the example three statements contain the accumulator variable **s**. Figure 3 shows the DFTs of the three statements which are processed by a typical tree pattern matcher. Two additional rules are required to match the DFTs: a rule to match variable uses (Rule 12) and a rule to match variable definitions (Rule 13). But these rules can only exist for a single non-terminal (either **reg** or **sreg**). Otherwise occurrences of a variable in various places would be interpreted differently. This means that with a tree pattern matcher the non-terminals for variables must be selected before matching.

To overcome the limitations of a tree pattern matcher we extend the scope of the matcher to SSA-graphs [7]. The base for SSA-graphs is *static single assignment* form [2]. The essential idea behind SSA is that each use has only a single definition. If there are multiple definitions for a use in the non-SSA form, in the SSA form a ϕ -term is inserted. Figure 5 shows the SSA form of our example program. It contains a ϕ -term for **s** at the loop head where the definition of the initialization and the definition of the computation of the last iteration are merged.

A SSA-graph describes the flow of computation for a whole function. Basically, the data structure combines the data flow trees (DFT) with def-use relations. For our running example the SSA-graph is shown in Figure 6. The nodes in the graph represent computations. The outgoing edges from a computation indicate data dependencies to other nodes which use the computation. Note that SSA-graphs do not contain explicit nodes for variable uses (**s**) and variable definitions (**s=**). In contrast to classical approaches which use DAG and tree representations of the computations, cycles are possible in the SSA-graphs.

For our running example we have several node types in the SSA-graph. E.g., plus operations(**+**), absolute value operations(**abs**), multiplications(*****), element access(**a[i]**), ϕ -nodes, constant nodes, and a return node(**ret**) for the return


```

int f(short *a, short *b)
{
(1)   int s1 = 0;
      loop(i) {
(2)     s2 =  $\phi(s_1, s_3)$ 
          s3 = abs(s2) + a[i] * b[i];
(3)   }
      return s2;
}

```

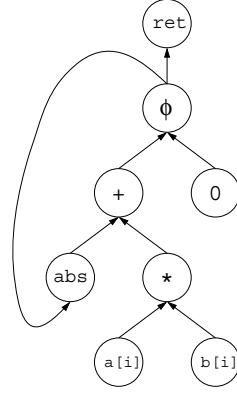


Fig. 5. SSA-form of running example **Fig. 6.** SSA-graph of running example

statement. The incoming edges specify the inputs of the computation. For example, the multiplication node has two incoming edges. One edge is from the operand `a[i]` the other edge is from the operand `b[i]`. In the SSA-graph all nodes, except the return node, pass their result on to other nodes.

The grammars used for matching SSA-graphs are similar to grammars used by tree pattern matchers [1]. As the SSA-graph does not contain explicit nodes for variable uses and variable definitions, no rules are required to match such nodes. Instead a grammar for SSA-graph matching must contain rules for matching ϕ -terms. In the example grammar the Rules (12) and (13) are no longer needed. Instead we need following rules to match the ϕ -term nodes.

- (14) $\text{reg} \rightarrow \phi(\text{reg}, \dots, \text{reg}), 0$
(15) $\text{sreg} \rightarrow \phi(\text{sreg}, \dots, \text{sreg}), 0$

In contrast to matching rules of other nodes, ϕ -nodes do not emit any code. They only need to match non-terminals of the same type. Rules 14 and 15 handle shifted and un-shifted values respectively for ϕ -nodes. As these rules do not generate any code, they do not have a code template.

Because matching directed graphs is NP-complete, the code selection for SSA-graphs is not a simple problem anymore. However, the optimization of the computation flow of a whole function is superior to classical approaches where only statements or sequences of statements are optimized.

3 Matching Problem

In this section we describe the mapping of the SSA-graph matching problem to PBQP. In the first step of the mapping, the grammar is transformed to normal form [1]. A grammar is in normal form if there are only production rules which are either base or chain rules. A base rule has the form $nt_0 \rightarrow P[nt_1, \dots, nt_n]$ where

nt_i are non-terminals and P is a terminal symbol. A chain rule is given by $nt_1 \rightarrow nt_2$ where on the left-hand side and on the right-hand side of the production are non-terminals. Production rules, which are neither chain rules nor base rules, can be decomposed into base and chain rules. For example rule $\text{reg} \rightarrow +[\text{reg}, *[\text{reg}, \text{reg}]], 2$ is neither a base nor a chain rule. By introducing a new non-terminal nt we can decompose the rule in $\text{reg} \rightarrow +[\text{reg}, nt], 2$ and $nt \rightarrow *[\text{reg}, \text{reg}], 0$.

For solving the matching problem we employ PBQP solver introduced in [12,3]. The PBQP problem is defined as follows,

$$\min f = \left[\sum_{1 \leq i < j \leq n} \mathbf{x}_i \cdot C_{ij} \cdot \mathbf{x}_j^T \right] + \left[\sum_{1 \leq i \leq n} \mathbf{c}_i \cdot \mathbf{x}_i^T \right] \quad (1)$$

$$\text{subject to: } \forall i \in 1 \dots n : \mathbf{x}_i \cdot \mathbf{1}^T = 1 \quad (2)$$

where $\mathbf{x}_i$ are boolean vectors for which only one element is set to one and n is the number of vectors. C_{ij} are cost matrices, and $\mathbf{c}_i$ is a cost vector. The aim of the optimization problem is to minimize costs. For each boolean vector $\mathbf{x}_i$ an element has to be chosen, such that the objective function f becomes a minimum.

In [12] a graph-theoretical representation of the problem is introduced. The PBQP-graph is a graph whose nodes represent boolean vectors (i.e. $\mathbf{x}_i$ for node i) and whose edges represent cost matrices that are unequal of the zero matrix. For each node i in the PBQP-graph there is a decision which element of $\mathbf{x}_i$ is set to one. Different decisions for a vector $\mathbf{x}_i$ contribute to different costs of the objective function. Note that the number of the elements of the boolean vector $\mathbf{x}_i$ can vary.

The main idea of the mapping is that the PBQP-graph is equivalent to the SSA-graph. For each node in the SSA-graph there are several base rule options. The number of these alternatives determines the size of the boolean vector for this node. The cost vector of this node is derived from the base rule costs of the node. Edges in the PBQP-graph express cost dependencies between two nodes. In our mapping an element in a cost matrix has several meanings: (1) It reflects the transition from one rule to another one, i.e. from the result non-terminal of one rule to an operand non-terminal of another rule $nt_1 \rightarrow nt_2$. (2) The non-terminal does not change, i.e. $nt \rightarrow nt$. These cost elements are zero. (3) Some transitions are not allowed in the grammar and we assign infinite costs to those elements in the matrix.

The mapping from the SSA-graph matching problem is done in three steps: (1) construct the PBQP-graph based on SSA-graph, (2) determine cost-vectors of nodes, and (3) determine cost-matrices of edges.

As already mentioned the PBQP-graph is equivalent to the SSA-graph. Nodes in the SSA-graph are nodes in the PBQP-graph and vice versa. Similarly, edges of the PBQP-graph are edges in the SSA-graph. In our approach the computational flow is represented as a graph where nodes represent functions and the incoming edges of the node give the input of the function. However, graphs do not define an

order for incoming edges which is required for the correctness of our approach. To overcome this problem, we define a mapping function $opnum(e)$ that determines the index of the operand of the edge e in the expression tree.

In the last two steps of the construction cost vectors and matrices for the PBQP are computed. If the compiler should produce fast code, the cost model has to consider dynamic execution weights for moving code from heavily executed portions of the function to rarely executed portions. A weight function w yields the execution weights for nodes and edges in the SSA-graph. The weight function for nodes yields the dynamic execution count of the basic block where the operation of the node is executed. For edges the weight function yields the dynamic execution count of the basic block where the code of chain-rules is inserted. This might be either the basic block of the predecessor or successor node of the edge. The weight function may also yield ∞ for edges where no chain-rule code is applicable. E.g., this is the case for edges between two ϕ -terms.

For our example we assume that the loop is executed 10 times. This yields a weight value of 10 for all nodes inside the loop, i.e. all nodes except 0 and **ret**. Hence all edges, except the adjacent edges of 0 and **ret** have a weight value of 10. The nodes 0 and **ret** and their adjacent edges have a weight value of 1.

For each node we enumerate all applicable rules. The cost vector for the node is the vector of rule costs scaled by the weight function.

Let $R_i = \{r_1^i, \dots, r_n^i\}$ be the set of matching rules for node i . For our example all matching rules for its nodes are listed in Figure 7. As we can see that for some nodes we have only one alternative which maps to a boolean decision vector with only one element. For others we have two alternatives. Therefore, the size for their boolean decision vectors is two.

Definition 1. Let $\text{cost}(r)$ be the cost of rule r . Then, cost vectors are given as follows

$$c_i = (\text{cost}(r_1^i), \dots, \text{cost}(r_n^i)) * w(i)$$

where all rule costs are weighted by weight function w .

For our example the cost vectors of the matching rules are given in Figure 8. For nodes inside the loop the cost elements are multiplied by 10 since we assume that the loop is executed 10 times. For nodes outside the loop the weight function yields one.

$$\begin{aligned} R_{\text{ret}} &= \{ \text{top} \rightarrow \text{ret}[\text{reg}] \} \\ R_0 &= \{ \text{reg} \rightarrow \text{const}(0) [], \text{sreg} \rightarrow \text{const}(0) [] \} \\ R_{\star} &= \{ \text{reg} \rightarrow +[\text{reg}, \text{reg}], \text{sreg} \rightarrow +[\text{sreg}, \text{sreg}] \} \\ R_{\text{abs}} &= \{ \text{reg} \rightarrow \text{abs}[\text{reg}], \text{sreg} \rightarrow \text{abs}[\text{sreg}] \} \\ R_{\star} &= \{ \text{sreg} \rightarrow *[\text{reg}, \text{reg}] \} \\ R_{\text{a}[i]} &= R_{\text{b}[i]} = \{ \text{reg} \rightarrow \text{load}[\text{ptr}] \} \\ R_{\phi} &= \{ \text{reg} \rightarrow \phi[\text{reg}, \text{reg}], \text{sreg} \rightarrow \phi[\text{sreg}, \text{sreg}] \} \end{aligned}$$

Fig. 7. Matching rule sets of running example

The last step in the PBQP definition is to determine the transition cost matrices for all edges in the graph. For convenience we define a function *chaincost*, which is used to get chain costs between two rules rather than between two non-terminals.

Definition 2. Let $r = nt_0^r \rightarrow P[nt_1^r, \dots, nt_n^r]$ and $s = nt_0^s \rightarrow Q[nt_1^s, \dots, nt_m^s]$ be base rules. Then, $\text{chaincost}(r, s, i) = c$, where c are minimal costs of all chain rule derivations from nt_0^r to nt_i^s . If there is no chain rule derivation from nt_0^r to nt_i^s , then $c = \infty$.

Function *chaincost*(r, s, i) yields the chain costs between the result non-terminal of rule r and the i^{th} source non-terminal of rule s . For chain-costs between two identical non-terminals we have zero costs. If there are costs between two different non-terminals it depends whether a derivation with chain-rule exists. If there exists at least one derivation, the chaining costs are determined by the derivation with minimal costs. If no derivation exists, the transition is prohibited and the chaining costs are ∞ .

Based on function *chaincost* cost matrices of edges in the PBQP-graph are computed. For each edge in the graph a cost matrix is determined. The elements of a cost matrix are given as follows,

$$C_{\langle p, s \rangle}(i, j) = \text{chaincost}(r_j^p, r_i^s, \text{opnum}(\langle p, s \rangle)) * w(\langle p, s \rangle)$$

where $\langle p, s \rangle$ is the edge, (i, j) is the row and column of the matrix, and r_j^p and r_i^s are the rules of node p and s .

A matrix of an edge contains the costs of a transition between the non-terminals of two adjacent rules. The matrix element c_{ij} defines the costs of applying chain rules from the result non-terminal of the predecessor rule r_i and the source non-terminal of the successor rule r_j . The selection of the source non-terminal in the successor rule pattern is determined by the *opnum* function for the edge.

For our example the cost matrices are given in Figure 9. The matrix $C_{\langle \text{abs}, + \rangle}$ contains a zero diagonal, the remaining elements are 10. Both the **abs** and **+** nodes have two rules, where the first rules only contain **reg** non-terminals and the second rules only contain **sreg** non-terminals. The transition costs between the first rule of **abs** and first rule of **+** are the chain rule costs of deriving **reg** from **reg**. Obviously this is zero. The same holds for the transition costs between the second rules. All other transitions need a chain rule from **reg** to **sreg** or vice versa. The rule costs for these chain rules are one, which is weighted by 10 (the execution count of the loop).

4 PBQP Solver

A PBQP Solver was already introduced in [12]. The solver works in two phases. In the first phase reduction rules are applied to nodes with degree one and two (ReduceI and ReduceII reductions). ReduceI reduction eliminates a node

$$\begin{aligned}
c_{\text{ret}} &= (1) \\
c_0 &= (1, 1) \\
c_+ &= (30, 30) \\
c_{\text{abs}} &= (20, 20) \\
c_* &= (40) \\
c_{\text{a}[i]} = c_{\text{b}[i]} &= (50) \\
c_\phi &= (0, 0)
\end{aligned}$$

Fig. 8. Cost vectors of example

$$\begin{aligned}
C_{<\text{a}[i],*>} &= C_{<\text{b}[i],*>} = (0) \\
C_{<*,*>} &= (10, 0) \\
C_{<0,\phi>} &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \\
C_{<\text{abs},*>} &= C_{<+,\phi>} = C_{<\phi,\text{abs}>} = \begin{pmatrix} 0 & 10 \\ 10 & 0 \end{pmatrix}
\end{aligned}$$

Fig. 9. Transition costs of example

i of degree one. The node's cost vector c_i and the adjacent cost matrix C_{ij} are transferred to the cost vector c_j of the adjacent node j . ReduceII reduction eliminates a node i of degree two. The node's cost vector c_i and the two adjacent cost matrices C_{ij} and C_{ik} are transferred to the cost matrix of the edge between the adjacent nodes j and k .

These reductions do not destroy the optimality of the PBQP. If the reduction with ReduceI and ReduceII is not possible, i.e. at some point of the reduction process there are only nodes with degree three or higher in the graph, a heuristic must be applied (ReduceN reduction). The heuristic selects the local minimum for the chosen node and eliminates the node. The reduction process is performed until a trivial solution remains, i.e. nodes with degree zero are left. Then the solution of the remaining nodes is determined. In the second phase, the graph is re-constructed in reverse order of the reduction phase and the solution is back-propagated.

In addition to the solver presented in [12] we perform simplification reductions: (1) elimination of nodes which have only one cost vector element and (2) elimination of independent edges. Both steps reduce the degree of nodes in the graph and have a positive impact for the obtaining a (nearly) optimal solution.

The first simplification step removes nodes which have only one element in boolean decision vector. This situation occurs if there is only one rule applicable for a node in the SSA-graph. Since there is no alternative for such a node, the node can be removed from the graph. The contribution of such a node collapses to a constant in the objective function and the node does not influence the global minimum. This process is equivalent to splitting a node into separate nodes for each adjacent edge, which are then reduced by ReduceI reductions (see Figure 10).

In our example all nodes, which have only one matching rule, can be eliminated by simplification. These nodes are **ret**, *****, **a[i]** and **b[i]**. With the first simplification step the cost vectors of ϕ -nodes and **+** change to the following values:

$$\begin{aligned}
c_+ &= (40, 30) \\
c_\phi &= (0, 1)
\end{aligned}$$

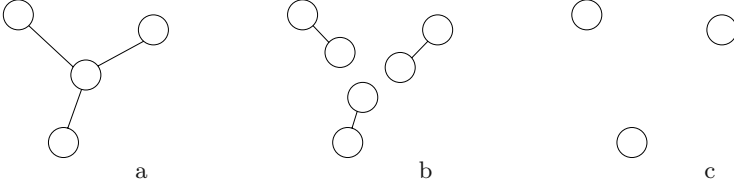


Fig. 10. Elimination of a node with a single rule (a). The node is split (b), the split nodes can be reduced with ReduceI (c)

The second simplification step eliminates edges with independent transition costs. Independent transition costs are costs which do not result in a decision dependence between the two adjacent nodes, i.e. the rule selection of one adjacent node does not depend on the rule selection of the other adjacent node. A simple example for independent transition costs is a zero matrix. In general all matrices which can be made to a zero matrix by subtracting a column vector and a row vector are independent.

Lemma 1. *Let C be a matrix and $\mathbf{u}$ and $\mathbf{v}$ be vectors. The matrix C is independent iff*

$$C = \begin{pmatrix} u_1 + v_1 & \dots & u_1 + v_m \\ \vdots & \ddots & \vdots \\ u_n + v_1 & \dots & u_n + v_m \end{pmatrix}.$$

An independent edge is eliminated by adding $\mathbf{u}$ to the predecessor cost vector and adding $\mathbf{v}$ to the successor cost vector.

Figure 11 shows the reduction sequence of the example graph. The `*`, `a[i]`, `b[i]` and `ret` nodes are already eliminated by simplification, because only a single rule can be matched on these nodes. The remaining graph contains one node with degree one, i.e. node 0. In the first step it is eliminated by ReduceI reduction. This increments the cost vector of the ϕ -node to (1, 2). Three nodes with degree 2 remain (ϕ , $+$ and `abs`). One of them - in this example the `abs` node - is eliminated by applying ReduceII reduction. The resulting edge of the reduction has a cost matrix of

$$C_{<\phi,+>} = \begin{pmatrix} 20 & 30 \\ 30 & 20 \end{pmatrix}$$

It is combined with the existing edge between ϕ and $+$ which results in

$$C_{<\phi,+>} = \begin{pmatrix} 20 & 40 \\ 40 & 20 \end{pmatrix}$$

In the last step the ϕ -node can be eliminated with ReduceI reduction which results in a cost vector of (61, 52) for the remaining node $+$. It has degree zero and the second rule (`sreg` $\rightarrow$ `+[sreg,sreg]`) can be selected, because the second

vector element (which is 52) is the element with minimal costs. Because no ReduceN reduction had to be applied for the example graph, the solution of this PBQP is optimal.

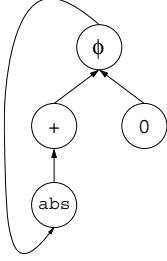
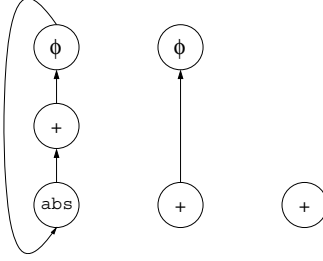


Fig. 11. Reduction sequence of running example



```

(1)  $\_f$ :
(2)   r0 = 0;
(3)   loop {
(4)     r1 = *ptr1
(5)     r2 = *ptr2
(6)     r3 = r1 * r2
(7)     r0 = abs(r0)
(8)     r0 = r0 + r3
(9)   }
(10)  r0 = r0 >> 1
(11)  ret

```

Fig. 12. Resulting code

After reduction only nodes with degree zero remain and the rules can be selected by finding the index of the minimum vector element. The rules of all other nodes can be selected by reconstructing the PBQP graph in reverse order of reductions. In each reconstruction step one node is re-inserted into the graph and the rule of this node is selected. Selecting the rule is done by choosing the rule with minimal costs for the node. This can be done, because the rules of all adjacent nodes are already known.

The back-propagation process for our example graph reconstructs the ϕ -node. The second rule is selected for this node ($\text{sreg} \rightarrow \phi[\text{sreg}, \text{sreg}]$). Then the **abs** and **0** nodes are re-inserted, with a rule selection of $\text{sreg} \rightarrow \text{abs}[\text{sreg}]$ and $\text{sreg} \rightarrow \text{const}(0)[]$ respectively. The nodes **ret**, *****, **a[i]** and **b[i]** need not be reconstructed, because the first (and only) rule has already been selected in the simplification phase for these nodes.

The solution of the PBQP yields the rule selections for the SSA-graph nodes. The code can be generated by applying the code generation actions of the selected rules. As the SSA-graph does not contain any control flow information, the places where the code is generated must be derived from the input program. So the code for a specific node is generated in the basic block which contains the operation of the node. The order of code generation within a basic block is also defined by the statement order and operator order in the input program.

Figure 12 shows the resulting code after register allocation (for clarity the loop control code and addressing code is not shown in this figure). As we can see in the generated code, inside the loop the addition operation and the **abs** function is performed with a shifted value. Prior to the return statement the value of variable **s** is converted to an un-shifted value.

5 Experimental Results

We have integrated the SSA-graph pattern matcher within the CC77050 C-Compiler for the NEC μ PD77050 DSP family. The μ PD77050 is a low-power DSP for mobile multimedia applications that has VLIW features. Seven functional units (two MAC, two ALUs, two load/store, one system unit) can execute up to four instructions in parallel. The register set consists of eight 40 bit general purpose registers and eight 32 bit pointer registers.

The grammar contains 724 rules and 23 non-terminals. The non-terminals select between address registers or general purpose registers. For the general purpose registers there are separate non-terminals for sign-extended values and non-sign-extended values and there are various non-terminals which place a smaller value at different locations inside a 40 bit register.

We have conducted experiments with a number of DSP benchmarks. The first group of benchmarks contains three complete DSP applications: AAC (advanced audio coder), MPEG, and GSM (gsm half rate). All three benchmarks are real-world applications that contain some large PBQP graphs. The second group of benchmarks are DSP-related algorithms of small size. These kind of benchmarks allow the detailed analysis of the algorithm for typical loop kernels of DSP applications. All benchmarks are compiled “out-of-the-box”, i.e. the benchmark source codes are not rewritten and tuned for the CC77050 compiler.

In Table 1 the number of the graphs (graphs num.) and the sizes of the graphs are given. In the “num.” columns the accumulated values over the whole benchmark is shown and in the “max.” columns the maximum value over all graphs is given. The total number of cost vector elements in the graph and the maximum number of cost vector elements for each node is shown in the last two columns. The number of cost vector elements is the number of matching rules of a node. These numbers depend on the used grammar. With our test grammar a maximum of 62 rules per node occurs in the graphs.

An important question when using a PBQP solver arises regarding the quality of the solution. It highly depends on the density of the PBQP graphs. If a graph can be reduced with ReduceI and ReduceII rules, the solution is optimal. Figure 13 shows the distribution of reductions. 31% of nodes can be eliminated by simplification, because they are trivial, i.e. only a single rule can match these nodes. Another important observation is that only a small fraction (less than 1%) of all nodes are ReduceN nodes. Therefore the solutions obtained from the PBQP solver are near optimal. The distribution of nodes in Figure 13 also shows the structure of the PBQP-graph: The fraction of degree zero nodes (R0) indicates the number of independent sub graphs in the SSA-graphs, i.e. a third of the nodes form own sub-graphs. ReduceI nodes (RI) are nodes which are part of a tree, whereas ReduceII (RII) and ReduceN (RN) nodes are part of a more complex subgraph. In addition, 37% of all edges can be eliminated by simplification, because they contain independent transition costs.

An effective way to improve the solution is to recursively enumerate the first ReduceN nodes in a graph. In many graphs only few ReduceN nodes exist and by moderate enumeration an optimal solution can be achieved. We have

performed our benchmarks in three different configurations: (1) reducing all ReduceN nodes with heuristics (H), (2) enumerate the first 100 permutations before applying heuristics (E 100) and (3) enumerate the first two million permutations (E 2M) before applying heuristics. The third configuration can yield the optimal solution in almost all cases. It is used to compare the other configurations against the optimum. Table 2 shows the percentages of optimally solved graphs and optimally reduced nodes in each configuration. The left columns (gropt) show the percentage of optimally solved graphs in each benchmark, the right columns (rnopt) show the percentage of ReduceN nodes, which are reduced by enumeration and do not destroy the optimality of the solution. A value of 100% is also given if there are no ReduceN nodes in a benchmark. In the first configuration (H) no enumeration was applied therefore all ReduceN nodes are reduced with the heuristics (0% in the H/rnopt column or 100% if there are no ReduceN nodes in a benchmark). Even without enumeration most of the graphs (H/gropt) can be solved optimally. The results of the second configuration (E 100) show that with a small number of permutations almost all graphs (E 100/gropt) and a majority of ReduceN nodes (E 100/rnopt) can be solved optimal.

For the performance evaluation we compare the SSA-graph matcher with a conventional tree pattern matcher, using the same grammar. For the tree-pattern matcher we had to make a pre-assignment of non-terminals to local variable definitions and uses. We assigned the most reasonable non-terminals to local variables, e.g. a pointer non-terminal to pointer variables, a register low-part non-terminal to 16 bit integer variables, etc. This is how a typical tree pattern matching would generate code. The performance improvements for all three configurations is shown in Figure 14. The configuration which enumerates 100 permutations gives a (marginal) improvement in just one benchmark(AAC). And the near optimal configuration does not improve the result anymore. This indicates that the heuristic for reducing ReduceN nodes is sufficient for this problem. The performance improvement for the small benchmarks is higher than for the large applications, because the applications contain much control code beside the numerical loop kernels.

The compile time overhead for the three DSP applications is shown in Table 3 (the compile time overhead for the small DSP algorithms is negligible and therefore not shown). The table compares the total compile time of two compilers, the first with SSA-graph matching, the second with tree pattern matching. The table contains the compile time overhead of the SSA-graph matching compiler to the tree matching compiler in percent for all three configurations. The overhead of the first two configurations (H and E 100) is equivalent. This means that it is feasible to allow a small number of permutations for ReduceN nodes.

6 Summary and Conclusion

For irregular architectures such as digital signal processors, code generators contribute significantly to the performance of a compiler. With traditional tree pattern matchers only separate data flow trees of a function can be matched, which

Table 1. Problem size

Benchmark	Graphs	Nodes		Edges		vec. elements	
	num.	num.	max.	num.	max.	num.	max.
mp3	60	37197	8491	40321	8854	556819	62
gsm	129	71376	24175	76884	26154	1138903	62
aac	71	25875	13093	26886	13523	405220	62
iirc	1	263	263	271	271	4877	62
iirbiqc	4	986	493	1002	501	17760	62
matmult	2	640	320	656	328	12182	62
vadd	2	244	122	242	121	4390	33
vdot	2	268	134	268	134	4812	62
vmin	2	306	153	304	152	5652	33
vmult	2	276	138	274	137	4976	62
vnorm	2	252	126	252	126	4590	62
sum/max	277	137683	24175	147360	26154	2160181	62

Table 2. Optimal graph and node reductions in percent

Benchmark	H		E 100		E 2M	
	gropt	rnopt	gropt	rnopt	gropt	rnopt
mp3	83.33	0.00	98.33	54.76	98.33	73.81
gsm	93.02	0.00	99.22	82.35	100.00	100.00
aac	91.55	0.00	98.59	75.00	100.00	100.00
iirc	0.00	0.00	100.00	100.00	100.00	100.00
iirbiqc	50.00	0.00	100.00	100.00	100.00	100.00
matmult	100.00	100.00	100.00	100.00	100.00	100.00
vadd	100.00	100.00	100.00	100.00	100.00	100.00
vdot	100.00	100.00	100.00	100.00	100.00	100.00
vmin	100.00	100.00	100.00	100.00	100.00	100.00
vmult	100.00	100.00	100.00	100.00	100.00	100.00
vnorm	100.00	100.00	100.00	100.00	100.00	100.00

Table 3. Compile time overhead in percent

Benchmark	H	E 100	E 2M
mp3	14	14	4252
gsm	6	6	7
aac	3	3	349

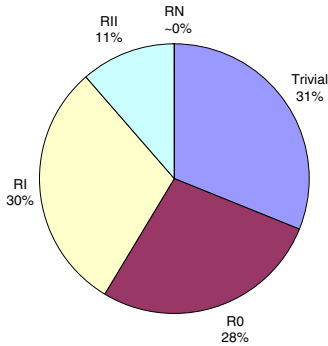


Fig. 13. Reduction statistics

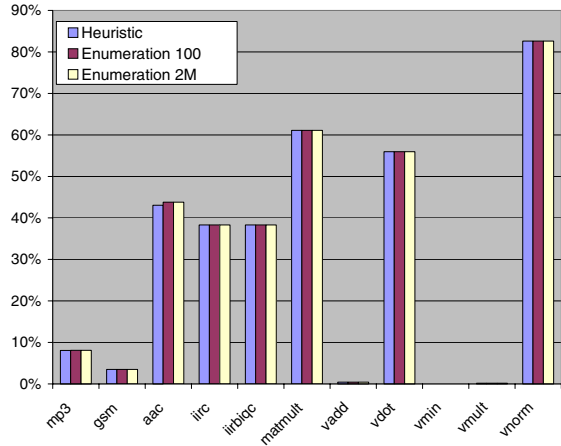


Fig. 14. Performance improvement

has a negative impact for the quality of the code. Only if the whole computational flow of a function is taken into account, the matcher is able to generate optimal code.

Matching SSA-graphs is NP-complete. For solving the matching problem we employ the partitioned boolean quadratic problem (PBQP) for which an effective and efficient solver [12] exists. The solver features linear runtime and only for few nodes in the SSA-graph heuristics needs to be applied. As shown in our experiments the PBQP solver has proven to be an excellent vehicle for graph matching. For a small fraction of the SSA-graphs a heuristic has to be applied.

Our experiments have shown that the performance gain of a SSA-graph matcher compared to a tree pattern matcher is significant (up to 82%) in comparison to classical tree matching methods. These results were obtained without modifying the grammar. Though the overhead of the PBQP solver is higher than tree matching methods, the compile time overhead is in acceptable bounds.

References

1. S. Biswas A. Balachandran, and D.M. Dhamdhere. Efficient retargetable code generation using bottom-up tree pattern matching. *Computer Languages*, 15(3):127–140, 1990.
2. R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck. An efficient method of computing static single assignment form. In ACM, editor, *POPL’89. Proceedings of the sixteenth annual ACM symposium on Principles of programming languages, January 11–13, 1989, Austin, TX*, pages 25–35, New York, NY, USA, 1989. ACM Press.
3. E. Eckstein and B. Scholz. Address mode selection. In *Proceedings of the International Symposium of Code Generation and Optimization (CGO 2003)*, San Francisco, March 2003. IEEE/ACM.

4. M. Anton Ertl. Optimal code selection in DAGs. In *Principles of Programming Languages (POPL'99)*, 1999.
5. C. Fraser, R. Henry, and T. Proebsting. BURG – fast optimal instruction selection and tree parsing. *ACM SIGPLAN Notices*, 27(4):68–76, April 1992.
6. Christopher W. Fraser and David R. Hanson. A code generation interface for ANSI c. *Software - Practice and Experience*, 21(9):963–988, 1991.
7. Michael P. Gerlek, Eric Stoltz, and Michael Wolfe. Beyond induction variables: Detecting and classifying sequences using a demand-driven SSA form. *ACM Transactions on Programming Languages and Systems*, 17(1):85–122, January 1995.
8. Rudolf Landwehr Helmut Emmelmann, Friedrich-Wilhelm Schröer. Beg - a generator for efficient back ends. *SIGPLAN'99 Conference on Programming Language Design and Implementation*, pages 227–237, 1989.
9. Rainer Leupers. Code generation for embedded processors. In *ISSS*, pages 173–179, 2000.
10. S. Liao, S. Devadas, K. Keutzer, and S. Tjiang. Instruction selection using binate covering for code size optimization. In *International Conference on Computer Aided Design*, pages 393–401, Los Alamitos, Ca., USA, November 1995. IEEE Computer Society Press.
11. Todd A. Proebsting. Least-cost instruction selection in dags is np-complete. <http://research.microsoft.com/~toddpro/papers/proof.htm>.
12. B. Scholz and E. Eckstein. Register allocation for irregular architecture. In *Proceedings of Languages, Compilers, and Tools for Embedded Systems (LCTES 2002) and Software and Compilers for Embedded Systems (SCOPES 2002)*, Berlin, June 2002. ACM.

A Code Selection Method for SIMD Processors with PACK Instructions

Hiroaki Tanaka, Shinsuke Kobayashi, Yoshinori Takeuchi,
Keishi Sakanushi, and Masaharu Imai

Graduate School of Information Science and Technology
Osaka University
{h-tanaka,kobayasi,takeuchi,sakanushi,imai}@ist.osaka-u.ac.jp

Abstract. This paper proposes a code selection method for SIMD instructions considering PACK instructions. The proposed method is based on a code selection method using Integer Linear Programming. The proposed method selects SIMD instructions efficiently, because it considers data transfer between registers. Data transfers are represented as nodes of PACK instructions. In the proposed method, nodes for data transfers are added to DAGs representing basic blocks. The nodes are covered by covering rules for PACK instructions. Code selection problems are formulated into Integer Linear Programming. Experimental results show that the proposed method reduced code size by 10% and execution cycles by 20 % or more, comparing to the method without PACK instructions.

1 Introduction

Systems for real-time multimedia applications such as image processing, speech processing and so on strongly need high cost-performance and low power processing. DSPs (Digital Signal Processor) are customized to execute multimedia applications efficiently to realize such multimedia systems. Moreover, DSPs can reduce power consumption comparing to general purpose processors such as Pentium. In multimedia applications, a large quantity of data whose bit length is shorter than 32 bits is processed by using same operations. Therefore, many DSPs adopt SIMD (Single Instruction Multiple Data) instructions to achieve high performance processing [1][2][3]. SIMD instructions perform operations using two source registers, and each register includes multiple data. When a SIMD instruction is executed, same operations are executed at the same time.

Currently, there used two major approaches to utilize SIMD instructions. One is assembly code approach, and the other is Compiler-Known-Functions approach. In assembly code approach, designers write assembly code considering SIMD instructions. In Compiler-Known-Function approach, compilers translate Compiler-Known-Functions to SIMD instructions directly. Therefore, designers have to consider data flow of programs. Using these approaches, designers can specify SIMD instructions correctly. These approaches, however, decrease portability of source code because programs depend on a specific processor. This is a disadvantage of embedded system design since design productivity is largely

reduced. In the embedded system design, time-to-market issue is also strongly important. Hence, the compiler approach that generates SIMD instructions from machine independent source code written in high level language such as C language is expected in order to make time-to-market short.

The concept of SIMD was appeared in the field of super computing. SIMD machines consist of multiple processing elements and a control processor. The control processor makes processing elements perform same instructions using different data. SIMD instructions are introduced in the platform of multimedia such as general purpose processors, DSPs and so on. While it is easy to implement SIMD instructions to processors, it is difficult to handle SIMD instructions in compiler. The compiler for general purpose processors with multimedia extensions, SIMD Within a Register C compiler is proposed [4]. In [4], in order to handle SIMD data type, C language is extended to SIMD Within a Register C language. Introducing SIMD data type representation, the compiler analyzes source programs based on the data type, and generates SIMD instructions. Language extension approach is effective to utilize SIMD instructions, however, the approach decreases portability. Leupers proposes a code selection method for media processors with SIMD instructions [6]. In this method, candidates to apply for SIMD instructions are extracted by analyzing data flow, and which to be executed by SIMD instructions is determined by formulating the problem into ILP (Integer Linear Programming) and solving it. However, exploitations of SIMD instructions are often missed, since this method does not consider data transfer. Therefore, data transfer should be considered in compilers for high exploitation of SIMD instructions. A method to extract SIMD parallelism is also proposed [5]. In this method, a basic block is represented in three-address form, and operations executed by a SIMD instruction are represented in a set of statements of three-address code. Using def-use chain, candidates of SIMD instructions are computed and SIMD instructions are decided heuristically so that the cost of packing and unpacking may become as low as possible. This method improves performance of generated code, however, comparing to [6], this method does not consider instructions which is peculiar to DSPs. Moreover, retargetability is not discussed.

In this paper, a code selection method considering data transfer is proposed. The proposed method is extension of the method [6] mentioned above to include data transfer operations such as MOVE, PACK and so on. In the proposed method, nodes for data transfers are inserted to DAGs representing process of programs, where the nodes annotate how each data move. Moreover, ILP formulations for PACK instructions are introduced by extending the Leupers's method. The problem can be solved by using ILP solver. Consequently, the compiler generates assembly code including SIMD instructions and PACK instructions. The advantage of the proposed method is that the SIMD instruction utilization is higher than that of the Leupers's method because of PACK instructions. As a result, performance and code size are improved at the same time. Moreover, retargetability is considered in this method, hence, the method is applied to retargetable compilers.

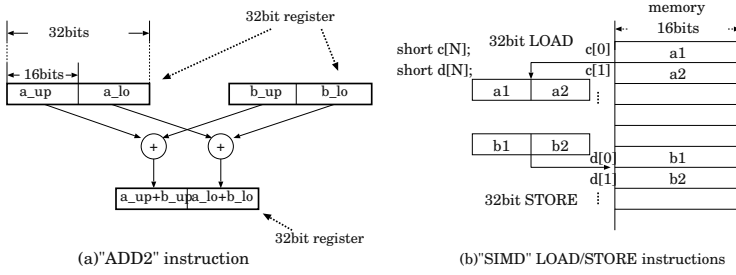


Fig. 1. Examples of SIMD instructions

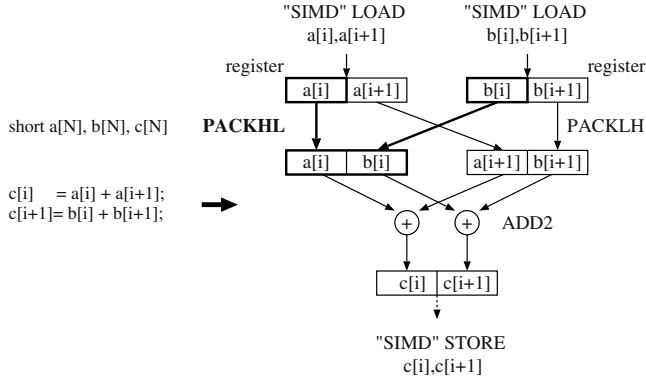


Fig. 2. An example of PACK instructions

The rest of this paper is organized as follows: Section 2 describes SIMD instructions. Section 3 introduces a code selection method using tree parsing and dynamic programming [7]. Section 4 explains the Leupers's method [6]. Section 5 describes the proposed method. Section 6 shows experimental results. Section 7 concludes this paper and shows our future work.

2 SIMD Instructions

In SIMD instructions, a value in a register consists of several values. Fig. 1(a) shows a SIMD instruction that performs two additions on upper and lower parts of registers. LOAD/STORE instructions are also regarded as SIMD instructions. Fig. 1(b) shows an example of SIMD LOAD/STORE instructions. Usually, processors with SIMD instructions also have PACK instructions. PACK instructions transfer several values from a couple of registers into a register. PACK instructions are useful to execute SIMD instructions effectively because PACK instructions produce packed data type.

Fig. 2 shows an example of PACK instructions. In Fig.2, $a[i]$ and $a[i+1]$ are loaded by a LOAD instruction as well as $b[i]$ and $b[i+1]$. Since source values of

additions are not located regularly, a SIMD instruction is not applied right after loading. However, replacing values by PACK instructions, SIMD instructions can be applied and the program is executed efficiently.

3 Code Selection

Code selection is usually implemented by using tree pattern matching and dynamic programming [7].

Let us assume a DAG $G = (V, E)$ representing a given basic block. Here $v \in V$ represents an IR level operation such as arithmetic, logical, load and store. $e \in E$ represents data dependency. A DAG is divided at its CSE(Common Sub Expression) into DFT(Data Flow Tree). Consequently, a set of DFTs is got for a basic block.

In tree pattern matching and dynamic programming technique, an instruction set is modeled as a tree grammar. A tree grammar consists of a set of terminals, a set of nonterminals, a set of rules, a start symbol and a cost function for rules. Terminals represent operators in a DFT. Nonterminals represent hardware resources which can store data such as registers and memories. A cost function determines a cost for each instruction, which is usually execution cycle of the instruction corresponding to the rule. Rules are used to represent behavior of instructions. For example, an ADD instruction which performs addition of two register contents, and stores the result to a register is represented as follows.

$$reg \rightarrow PLUS(reg, reg)$$

Code selection for a DFT is carried out by deriving the DFT which has minimal cost. In order to derive a tree which has minimal cost, dynamic programming is used. In a bottom-up traversal, all nodes v in the DFT are labeled with a set of triples (n, p, c) , where n is a nonterminal, p is a rule, and c is the cost for subtree which root is v . This means that node v can be reduced to nonterminal n by applying rule p at cost c .

4 SIMD Instruction Formulation

In this chapter, formulation and solution of reference [6] are summarized.

4.1 Rules for SIMD Instructions

A set of DFTs mentioned in section 3 is considered. The flow of this method is as follows; first, a set of rules is computed at each node in DFTs by pattern matching. Then, a rule is selected from the set under condition that cost is minimum. For the sake of simplicity, we discuss the case of two data placed in a register. However, it is easy to extend this method to the case of three or more data placed in a register.

When a N -bit processor with SIMD instructions performs an operation on $\frac{N}{2}$ -bit data, there are three options to execute the operation.

- Execute an instruction that performs on N -bit data
- Execute a SIMD instruction, where the operations perform on $\frac{N}{2}$ -bit data in the upper part of register
- Execute a SIMD instruction, where the operations perform on $\frac{N}{2}$ -bit data in the lower part of register

In the tree grammar, it is necessary to distinguish full registers as well as upper and lower subregisters. To represent the operation on upper and lower parts of a register, additional nonterminals *reg_hi* and *reg_lo* are introduced. Using *reg_hi* and *reg_lo*, three operations mentioned above can be represented.

- Arithmetic and logical operations
For example, 32-bit addition and upper and lower parts of SIMD addition are represented as follows.

$$\begin{aligned} reg &\rightarrow PLUS(reg, reg) \\ reg_hi &\rightarrow PLUS(reg_hi, reg_hi) \quad reg_lo \rightarrow PLUS(reg_lo, reg_lo) \end{aligned}$$

Other operations can be represented similarly to the example of addition.

- Loads and stores
Similar to arithmetic and logical operations, 16-bit load operations are represented as follows.

$$\begin{aligned} reg &\rightarrow LOAD_SHORT(addr) \\ reg_hi &\rightarrow LOAD_SHORT(addr) \quad reg_lo \rightarrow LOAD_SHORT(addr) \end{aligned}$$

16-bit store operations are represented as follows.

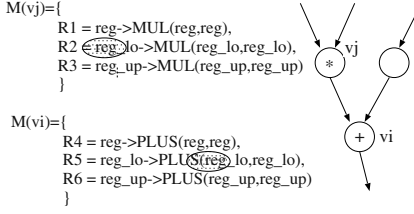
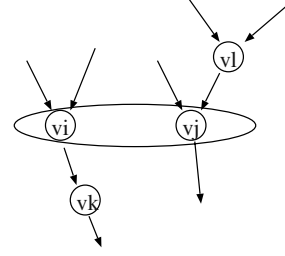
$$\begin{aligned} S &\rightarrow STORE_SHORT(reg, addr) \\ S &\rightarrow STORE_SHORT(reg_hi, addr) \\ S &\rightarrow STORE_SHORT(reg_lo, addr) \end{aligned}$$

- Common sub expressions
The definition and the use of CSEs are respectively represented as follows.

$$\begin{aligned} S &\rightarrow DEF_SHORT_CSE(reg) \\ S &\rightarrow DEF_SHORT_CSE(reg_hi) \\ S &\rightarrow DEF_SHORT_CSE(reg_lo) \\ \\ reg &\rightarrow USE_SHORT_CSE \\ reg_hi &\rightarrow USE_SHORT_CSE \\ reg_lo &\rightarrow USE_SHORT_CSE \end{aligned}$$

4.2 Constraints on Selection of Rules

In matching phase, a set of rules is annotated at each node. In the next phase, a rule is selected from the set, while the selection of rule have to be done under constraints as follows.

**Fig. 3.** Consistency of nonterminals**Fig. 4.** Schedulability

- Selection of a single rule
For each node v_i , exactly one rule has to be selected.
- Consistency of nonterminals
Let v_j and v_k be children of v_i in a DFT. Here, a nonterminal which is left hand side of a rule is called target nonterminal. Each target nonterminal of the rule selected for v_j and v_k corresponded to argument of the rule selected for v_i has to be consist.
Fig. 3 shows an example of consistency of nonterminals. If R_2 is selected for v_i , R_5 has to be selected for v_j .
- Common sub expressions
Nonterminal of the rule selected for definition of CSE v_i and nonterminal of the rule selected for its use v_j must be identical.
- Node pairing
When v_i is executed by a SIMD instruction, another node v_j which is executed by an identical SIMD instruction must be existed.
- Schedulability
When we determine which nodes are executed by SIMD instructions, data dependency between each pair should be considered. As shown in Fig. 4, if v_i and v_j are executed by an identical SIMD instruction, v_k and v_l cannot be executed at the same time.

4.3 ILP Formulation

Let $V = \{v_1, \dots, v_n\}$ be the set of DFG nodes, and let $M(v_i) = \{R_{i_1}, R_{i_2}, \dots, R_{i_k}, \dots\}$ be a set of all rules matching v_i . Boolean solution variables x_{ii_k} is defined as follows:

$$x_{ii_k} = \begin{cases} 1, & \text{if } R_{i_k} \text{ is selected for } v_i \\ 0, & \text{other} \end{cases} \quad (1)$$

variables x_{ii_k} denotes which rule is selected for v_i from $M(v_i)$ after ILP is solved.

Let a pair of nodes (v_i, v_j) denote a SIMD pair if it holds below conditions.

- v_i and v_j can be executed in parallel. Namely, there is no path from v_i to v_j or from v_j to v_i in DFG.
- v_i and v_j represent same operation.

- $M(v_i)$ contains a rule with target nonterminal *reg_hi*, and $M(v_j)$ contains a rule with target nonterminal *reg_lo*.
- If v_i and v_j are LOAD or STORE, which work on memory address p_i and p_j , then $p_i - p_j$ is equal to the number of bytes occupied by the 16-bit value.

Boolean auxiliary variables y_{ij} is defined as follows:

$$y_{ij} = \begin{cases} 1, & \text{if } v_i \text{ and } v_j \text{ are executed by an identical SIMD instruction} \\ 0, & \text{other} \end{cases} \quad (2)$$

where variable y_{ij} denotes nodes that are executed by an identical SIMD instruction, and the result of the operation on v_i is stored to upper part of a destination register, the result of the operation on v_j is stored to lower part of a destination register.

Constraints described above are represent as follows.

- Selection of a single rule

Since only one x_{ii_k} becomes 1 each v_i , this constraint represents as follows.

$$\forall v_i : \sum_{R_{i_k} \in M(v_i)} x_{ii_k} = 1 \quad (3)$$

- Consistency of target nonterminals

Assuming that $R_{i_k} \in M(v_i)$, $R_{i_k} = n_1 \rightarrow t(n_2, n_3)$ for a terminal t and nonterminals n_1, n_2, n_3 , and v_{i_l} and v_{i_r} be the left and right child of v_i . Let $M^N(v) \subseteq M(v)$ denote the subset of rules matching v that have N as the target nonterminal. If $R_{i_k} = n_1 \rightarrow t(n_2, n_3)$ is selected for v_i , then the rule chosen for v_l and v_r must have the target nonterminals n_2 and n_3 . This constraint is represented as follows.

$$\forall v_i : \forall R_{i_k} \in M(v_i) : x_{ii_k} \leq \sum_{R_{l_k} \in M^{n_2}(v_l)} x_{ll_k} \quad (4)$$

$$\forall v_i : \forall R_{i_k} \in M(v_i) : x_{ii_k} \leq \sum_{R_{r_k} \in M^{n_3}(v_r)} x_{rr_k} \quad (5)$$

- Common subexpressions

Definitions of 16-bit CSEs and uses of 16-bit CSEs have been defined as follows.

$$\begin{aligned} R_1 &= S \rightarrow DEF_SHORT_CSE(reg) \\ R_2 &= S \rightarrow DEF_SHORT_CSE(reg_hi) \\ R_3 &= S \rightarrow DEF_SHORT_CSE(reg_lo) \end{aligned}$$

$$\begin{aligned} R_4 &= reg \rightarrow USE_SHORT_CSE \\ R_5 &= reg_hi \rightarrow USE_SHORT_CSE \\ R_6 &= reg_lo \rightarrow USE_SHORT_CSE \end{aligned}$$

Therefore, if v_i is definition of CSE and v_j is use of CSE, it is clear that $M(v_i) = \{R_1, R_2, R_3\}$ and $M(v_j) = \{R_4, R_5, R_6\}$. This constraint is represented as follows.

$$\forall v_i, v_j : x_{i1} = x_{j4}, \quad x_{i2} = x_{j5}, \quad x_{i3} = x_{j6} \quad (6)$$

– Node pairing

Let P denote the set of SIMD pairs. If $R_{i_k} \in M^{hi}(v_i)$ is selected for v_i , there must be v_j and $R_{j_k} \in M^{lo}(v_j)$ which holds $(v_i, v_j) \in P$. This condition is represented as follows.

$$\forall v_i : \sum_{R_{i_k} \in M^{hi}(v_i)} x_{ii_k} = \sum_{j:(v_i, v_j) \in P} y_{ij} \quad (7)$$

$$\forall v_i : \sum_{R_{i_k} \in M^{lo}(v_i)} x_{ii_k} = \sum_{j:(v_i, v_j) \in P} y_{ji} \quad (8)$$

– Schedulability

Let $X(v)$ denote a set of nodes that must be executed before v , and let $Y(v)$ denote a set of nodes that must be executed after v . If $(v_i, v_j) \in P$, then a set Z_{ij} defined below have to be empty.

$$Z_{ij} = P \cap (X(v_i) \times Y(v_j) \cup X(v_j) \times Y(v_i)) \quad (9)$$

This constraint is represented as follows.

$$\forall (v_i, v_j) \in P : \forall (v_p, v_q) \in Z_{ij} : y_{ij} + y_{pq} \leq 1 \quad (10)$$

– Objective function

The optimization goal is to make the maximum use of SIMD instructions. Since target nonterminals of the rules for SIMD instructions are *reg_hi* or *reg_lo*, the objective function is represented as follows.

$$f = \sum_{v_i \in V} \left(\sum_{R_{i_k} \in M^{hi}(v_i) \cup M^{lo}(v_i)} x_{ii_k} \right) \quad (11)$$

5 SIMD Instruction Formulation with PACK Instructions

In this section, the proposed method is explained. The proposed method is extended from the Leupers's method [6]. Data transfers for SIMD instructions are considered in instruction selection of compiler. The following subsections explain the proposed method in detail.

5.1 IR and Rules for Data Packing and Moving

To represent data transfers on DFTs, nodes that represent data transfer operations are introduced. Since candidates of data transfers appear between operations, nodes for data transfers are inserted between all operations. Fig. 5 shows nodes insertion for data transfers. DT1, DT2, and DT3 are added to the DFT. Moreover, rules of data transfer are also introduced. When a processor executes a PACK instruction, there are three conditions according to the locations where data exist.

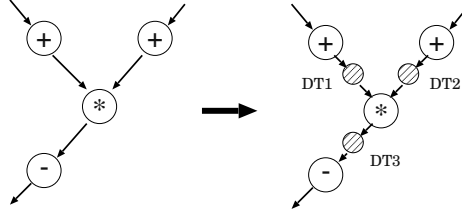


Fig. 5. Nodes insertion for data transfers

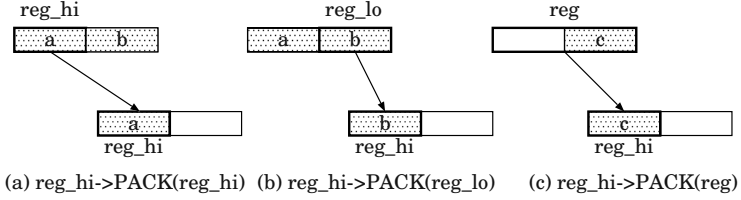


Fig. 6. Rules of PACK instructions

- Two values are located in a register. The value that would be packed is in the upper part of the register.
- Two values are located in a register. The value that would be packed is in the lower part of the register.
- A value is located in a register

These three conditions are shown in Fig.6. Fig. 6(a) shows a data transfer from upper part of a source register to upper part of a destination register. To represent PACK instructions, terminal *PACK* is used. Fig. 6(a) represents the rule $reg_hi \rightarrow PACK(reg_hi)$. Similarly, Fig. 6(b) represents the rule $reg_hi \rightarrow PACK(reg_lo)$. Fig. 6(c) shows a data transfer from source register occupied by a value to upper part of a destination register. Fig. 6(c) represents the rule $reg_hi \rightarrow PACK(reg)$. Data transfer to the lower part of destination register is represented as same as the case of data transfer to the upper part mentioned above. These conditions for PACK instructions are formulated as additional rules shown below.

$$\begin{array}{ll}
 reg_lo \rightarrow PACK(reg_lo) & reg_hi \rightarrow PACK(reg_lo) \\
 reg_lo \rightarrow PACK(reg_hi) & reg_hi \rightarrow PACK(reg_hi) \\
 reg_lo \rightarrow PACK(reg) & reg_hi \rightarrow PACK(reg)
 \end{array}$$

where a PACK instruction consists of two rules : one has reg_hi as a target nonterminal, and the other has reg_lo as a target nonterminal.

For example, consider four PACK instructions shown in Fig. 7, which are PACK instructions of TMS320C62x [1]. Using the rules introduced above, PACK instructions are represented. PACKH2 consists of two data transfers, one is from upper part of source register to upper part of destination register, and the other is from upper part of source register to lower part of destination register. Former

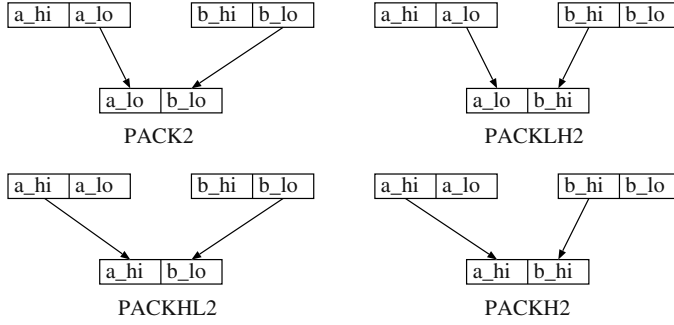


Fig. 7. Examples of PACK instructions

data flow is represented by $reg_hi \rightarrow PACK(reg_hi)$, and latter is represented by $reg_lo \rightarrow PACK(reg_hi)$, therefore, PACKH2 instruction can be represented by a pair of rules, $reg_hi \rightarrow PACK(reg_hi)$ and $reg_lo \rightarrow PACK(reg_hi)$.

Moreover, the rule for UNPACK which is an instruction that moves a value located upper or lower parts of a register into a register is adopted. Those rules are represented as follows.

$$reg \rightarrow UNPACK(reg_lo) \quad reg \rightarrow UNPACK(reg_hi)$$

In addition, rules which indicate no operation called “NOMOVE” are introduced.

$$\begin{aligned} reg &\rightarrow NOMOVE(reg) \\ reg_lo &\rightarrow NOMOVE(reg_lo) \quad reg_hi \rightarrow NOMOVE(reg_hi) \end{aligned}$$

These rules are selected when it is not necessary to move data.

PACK and *UNPACK* have some costs since actual instructions are executed if they are selected. However, *NOMOVE* has no cost since that is corresponded to no actual instruction.

5.2 Constraints on Selection of Rules

In order to introduce DFT nodes and rules, the following constraints have to be considered.

- Node pairing for PACK

PACK, *UNPACK* and *NOMOVE* match DFT nodes for data transfers. Those rules must be selected under constraints shown below.

- If *PACK* is selected for v_i , another node v_j that is selected as *PACK* must exist, and they execute an identical *PACK* instruction.
- If *UNPACK* is selected for v_i , there is no node executed with v_i .
- If *NOMOVE* is selected for v_i , even if a target nonterminal is reg_hi or reg_lo , v_i is not paired with other nodes because behavior of *NOMOVE*

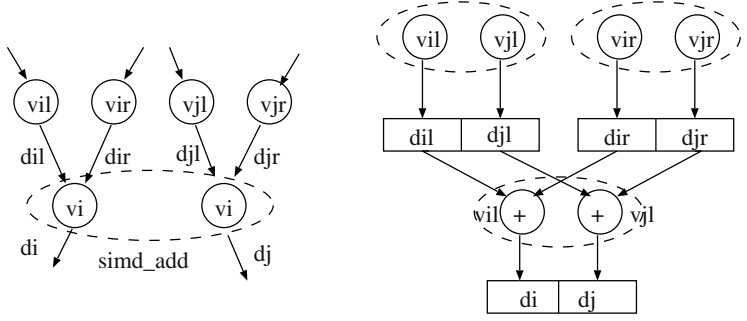


Fig. 8. Packed data

does not depend on other part of a register. However, when SIMD instructions are executed successively, the nodes for data transfers between SIMD instructions must be selected as *NOMOVE* and must be paired them.

– Packed data

When a SIMD instruction is executed, left arguments have to be packed in an identical register, and right arguments also have to be packed in the source register. Fig. 8 shows an example of packed data. Each result of v_{i_l} and v_{j_l} must be packed in an identical register to perform v_i and v_j as a SIMD instruction as well as v_{i_r} and v_{j_r} .

5.3 ILP Formulation

In this section, ILP formulation for PACK instructions is explained.

– Node pairing for PACK

Boolean auxiliary variables a_{ij} and b_{ij} are defined as follows:

$$a_{ij} = \begin{cases} 1, & v_i \text{ and } v_j \text{ are executed an identical } PACK \text{ instruction} \\ 0, & \text{other} \end{cases}$$

$$b_{ij} = \begin{cases} 1, & v_i \text{ and } v_j \text{ are stayed in an identical register} \\ 0, & \text{other} \end{cases}$$

Let V_{MOVE} denote a set of nodes for data transfers, and let $M_{OP}^N(v)$ denote a subset of rules in $M^N(v)$ that have *OP* as the terminal *OP*. This constraint is represented as follows.

$$\forall v_i \in V_{MOVE} : \sum_{R_{i_k} \in M_{PACK}^{hi}(v_i)} x_{ii_k} = \sum_{j:(v_i, v_j) \in P} a_{ij} \quad (12)$$

$$\forall v_i \in V_{MOVE} : \sum_{R_{i_k} \in M_{PACK}^{lo}(v_i)} x_{ii_k} = \sum_{j:(v_j, v_i) \in P} a_{ji} \quad (13)$$

$$\forall v_i \in V_{MOVE} : \sum_{R_{i_k} \in M_{NOMOVE}^{hi}(v_i)} x_{ii_k} \geq \sum_{j:(v_i, v_j) \in P} b_{ij} \quad (14)$$

$$\forall v_i \in V_{MOVE} : \sum_{R_{i_k} \in M_{NOMOVE}^{lo}(v_i)} x_{ii_k} \geq \sum_{j:(v_j, v_i) \in P} b_{ji} \quad (15)$$

Following constraint is needed from the definition of y_{ij} , a_{ij} , and b_{ij} .

$$\forall y_{ij} \in V_{MOVE} : y_{ij} = a_{ij} + b_{ij} \quad (16)$$

– Packed data

Let v_{i_l} and v_{i_r} be left and right children of v_i in DFT, v_{j_l} and v_{j_r} be left and right children of v_j . In order to execute a SIMD instruction for v_i and v_j , results of v_{i_l} and v_{j_l} must be packed in a register as well as v_{i_r} and v_{j_r} . When v_{i_l} and v_{j_l} are executed by an identical SIMD instruction, the results of v_{i_l} and v_{j_l} are stored to a register. Therefore, to execute a SIMD instruction for v_i and v_j , v_{i_l} and v_{j_l} , and v_{i_r} and v_{j_r} must be executed by a SIMD instruction. y_{ij} denotes that SIMD instructions is executed for v_i and v_j . This constraint is represented as follows.

$$\forall (v_i, v_j) \in P, v_i \in V : y_{ij} \leq y_{i_l j_l} \quad (17)$$

$$\forall (v_i, v_j) \in P, v_i \in V : y_{ij} \leq y_{i_r j_r} \quad (18)$$

– Objective function

The optimization goal is to minimize code size. Consider variables x_{ij} and y_{ij} for arithmetic, logical operation and load/store, y_{ij} corresponds to a SIMD instruction, and x_{ij} for the rule which has *reg* as a target nonterminal corresponds to an instruction. On the other hand, if variables x_{ij} , a_{ij} and b_{ij} represent data transfer operations, a_{ij} corresponds to a PACK instruction, x_{ij} for *UNPACK* corresponds to a data transfer operation, and x_{ij} , b_{ij} for *NOMOVE* corresponds to no instruction. Let P_{MOVE} denote a set of pairs of nodes for data transfer, and code size can be represented as follows.

$$\begin{aligned} f = & \sum_{v_i \in V - V_{MOVE}} \sum_{R_{i_k} \in M^{reg}(v_i)} x_{ii_k} + \sum_{(v_i, v_j) \in P - P_{MOVE}} y_{ij} \\ & + \sum_{v_i \in V_{MOVE}} \sum_{R_{i_k} \in M_{UNPACK}^{reg}(v_i)} x_{ii_k} + \sum_{(v_i, v_j) \in P_{MOVE}} a_{ij} \end{aligned} \quad (19)$$

6 Experimental Results

The proposed formulation was implemented by using CoSy compiler development environment [10] on RedHat Linux 8.0. For evaluation, a DLX based processor that had DLX instruction set without floating point arithmetic operation, but had SIMD instructions, such as ADD2, MULT2, and several PACK instructions was used. ADD2 instruction performs two additions on 16-bit values,

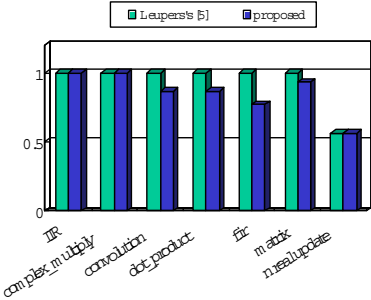


Fig. 9. The ratio of generated code size

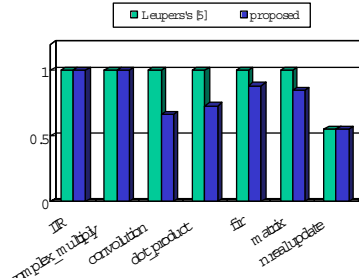


Fig. 10. The ratio of execution cycles

Table 1. Generated code size and execution cycles

program	unrolling factor	no SIMD optimization		Leupers's method		proposal method	
		code size	execution cycles	code size	execution cycles	code size	execution cycles
iir_biquad_N_section	0	132	420	132	420	132	420
complex_multiply	0	126	562	126	562	126	562
convolution	3	62	784	62	784	54	514
dot_product	1	57	162	57	162	44	118
FIR	3	88	828	88	828	67	730
matrix	3	137	5268	137	5268	127	4458
n_real_update	3	95	1162	53	634	53	634

Table 2. The number of DFT nodes, variables, and constraints in ILP and CPU time

program	Leupers's method				proposed method			
	# of nodes	# of variables	# of constraints	CPU time [sec]	# of nodes	# of variables	# of constraints	CPU time [sec]
iir_biquad_N_section	40	189	190	0.11	69	2304	7974	0.99
complex_multiply	16	62	69	0.09	30	776	1789	0.18
convolution	34	149	174	0.09	60	2062	7504	1.99
dot_product	18	67	88	0.08	32	704	1522	0.18
FIR	48	305	627	0.17	81	3660	20097	5679.00
matrix	34	149	174	0.12	60	2062	7504	3.79
n_real_update	28	129	137	0.12	51	2166	4557	22.72

MULT2 instruction which two multiplications on 16-bit values, and a variety of PACK instructions are PACKL, PACKLH, PACKHL and PACKHH. To compare the quality of generated code, three compilers were used: (1) a compiler generated by the compiler generator of ASIP meister [11], (2) a compiler applied the Leupers's method based on (1)'s compiler, and (3) a compiler applied the proposed method based on (1)'s compiler. Programs for evaluation that consists of iir_biquad_one_section, complex_multiply, convolution, dot_product, fir, matrix and n_real_updates were selected from DSPstone benchmark [9]. Original codes such as convolution, dot_product, fir, matrix, n_real_updates were unrolled to easily extract parallel executions.

Table 1 shows generated code sizes and the number of execution cycles of each program compiled by each compiler. Fig. 9 shows ratios of code sizes generated by (2) and (3) to generated by (1) respectively, and Fig. 10 shows the ratio of execution cycles of generated code. Table 2 shows the number of nodes of DFT, the number of variables and constraints in ILP and CPU time.

In Figs. 9 and 10, the Leupers's method was effective in only `n_real_updates`. However, the proposed method reduced code size and execution cycles in `convolution`, `dot_product`, `FIR`, `matrix`, and `n_real_updates`. The Leupers's method can select SIMD instructions the case where a sequence of instructions consists of SIMD instructions only because the Leupers's method does not consider data transfer. However, such conditions are not often filled. On the other hand, the proposed method inserts data transfer instructions when SIMD instructions can be applied by moving values, or unpacked data. Actually, in convolution, the proposed method selected a PACK instruction to adapt the location of values for SIMD multiplication instruction and select it. Experimental results show that Leupers's method reduces code size and execution cycles in only one program. This is because base processor used in this experiment does not have instructions peculiar to digital signal processors while Leupers's method includes such instructions that take values in upper and lower parts of registers. For example, MULTH instruction that TIC6201 has takes 16-bits values in upper parts of registers from source registers, and stores 32-bits values to destination register. In Leupers's method, the possibility of exploiting SIMD instructions is increased because instructions such as MULTH can take values produced by SIMD instructions. In this experiment, DLX based processor has been used for simple implementation. Applying the proposed method to real DSPs and comparing to Leupers's method is future work.

In Table 2, comparing the Leupers's method and the proposed method, the proposed method takes much more time to solve ILP. This is because the proposed method has wider solution space than the Leupers's method. Therefore, the proposed method spends much time to get an optimum solution. However, the proposed method can select SIMD instructions effectively. The code size of the proposed method is smaller than that of the Leupers's method, and execution cycles of the proposed method is smaller than that of the Leupers's method.

7 Summary

In this paper, a code selection method for SIMD instructions considering data transfer has proposed. In the proposed method, nodes for data transfers are added to DAGs, and rules for data transfer are introduced. Similar to the Leupers's method, code selection problem is formulated into ILP, and the problem is solved by using ILP solver. Experimental results show that the proposed method can generate more efficient codes than the Leupers's method, which uses data transfer instructions to exploit SIMD instructions. Our future work includes developing heuristics whose compilation time is faster than the time using ILP, and retargeting technique for our compiler generator.

Acknowledgment

We would like to thank Mr. Kentaro Mita and all member of the VLSI system design laboratory at Osaka University. We also would like to thank ACE Associated Compiler Experts bv. for providing the compiler development kit CoSy. We also would like to thank Japan Novel Corp.

References

1. Texas Instruments, TMS320C6000 CPU and Instruction Set Reference Guide, 2000.
2. Philips Semiconductors, PNX 1300 Series Databook, 2002.
3. MIPS Technology, MIPS64 Architecture For Programmers Volume II: The MIPS64 Instruction Set, 2001.
4. "SWARC: SIMD Within a Register C."
<http://www.ece.purdue.edu/~hankd/SWAR/Sccl.html>.
5. S. Larsen, "Exploiting Superword Level Parallelism with Multimedia Instruction Sets," ACM SIGPLAN Notices, Vol. 35, No. 5, pp. 145–156, 2000.
6. R. Leupers, "Code Optimization Techniques for Embedded Processors," Kluwer Academic Publishers, 2000.
7. A.V. Aho, M. Ganapathi, and S.W.K. Tijang, "Code Generation Using Tree Matching and Dynamic Programming," ACM Trans. on Programming Languages and Systems Vol. 11, No. 4, pp. 491–516, 1989.
8. J.L. Hennessy and D.A. Patterson, "Computer Architecture – A Quantitative Approach," Morgan Kaufmann Publishers Inc., 1990.
9. V. Zivojnovic, J. Martinez, C. Schlger, and H. Meyr, "DSPstone: A DSP-Oriented Benchmarking Methodology," Proc. of International Conference on Signal Processing Applications and Technology, 1994.
10. ACE Associated Compiler Experts, <http://www.ace.nl/>.
11. S. Kobayashi, K. Mita, Y. Takeuchi, and M. Imai, "A Compiler Generation Method for HW/SW Codesign Based on Configurable Processors," IEICE Trans. on Fundamentals of Electronics, Communications and Computer Sciences, Vol. E85-A, No. 12, pp. 2586–2595, Dec. 2002.

Reconstructing Control Flow from Predicated Assembly Code

Björn Decker¹ and Daniel Kästner²

¹ Saarland University
bdecker@cs.uni-sb.de

² AbsInt GmbH
kaestner@absint.com

Abstract. Predicated instructions are a feature more and more common in contemporary instruction set architectures. Machine instructions are only executed if an individual guard register associated with the instruction evaluates to true. This enhances execution efficiency, but comes at a price: the control flow of a program is not explicit any more. Instead instructions from the same basic block may belong to different execution paths if they are subject to disjoint guard predicates. Postpass tools processing machine code with the purpose of program analyses or optimizations require the control flow graph of the input program to be known. The effectiveness of postpass analyses and optimizations strongly depends on the precision of the control flow reconstruction. If traditional reconstruction techniques are applied for processors with predicated instructions, their precision is seriously deteriorated. In this paper a generic algorithm is presented that can precisely reconstruct control flow from predicated assembly code. The algorithm is incorporated in the PROPAN system that enables high-quality machine-dependent postpass optimizers to be generated from a concise hardware specification. The control flow reconstruction algorithm is machine-independent, and automatically derives the required hardware-specific knowledge from the machine specification. Experimental results obtained for the Philips TriMedia TM1000 processor show that the precision of the reconstructed control flow is significantly higher than with reconstruction algorithms that do not specifically take predicated instructions into account.

1 Introduction

Many of today's microprocessors use instruction-level parallelism to achieve high performance. They typically have multiple execution units and provide multiple issue slots (EPIC, VLIW) or deep pipelining (superscalar architectures). However, since the amount of parallelism inherent in programs tends to be small [1], it is a problem to keep the available execution units busy. For architectures with static instruction-level parallelism this problem is especially virulent, since if not enough parallelism is available the issue slots of the long instruction words are filled with nops. For embedded processors this means a waste of program memory and energy.

Guarded (predicated) execution [2,3,4] has been implemented in many different microprocessors such as the TriMedia TM1000, the ADSP-2106X SHARC processor, and the Intel IA-64 architecture [5,6,7]. It provides an additional boolean register to indicate whether the instruction is executed or not. This register is called the *guard* or the *guard register* of the instruction. A guard register having the value **true** forces the processor to execute the corresponding instruction. If the value of the guard is **false** the operation typically is dismissed without having any effect. An example is shown in Fig. 1. The original program consists of three basic blocks; if predicated execution is exploited only one basic block remains. If supported by the target architecture, i_2 and i_4 resp. i_3 and i_5 can be allocated in the same VLIW instruction.

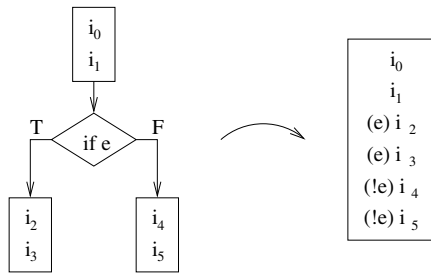


Fig. 1. Guarded code

Predicated execution can significantly improve code density since it allows to fill issue slots of the same instruction with micro-operations from different control paths. Moreover it enhances performance since it allows conditional branches to be removed from the program. Conditional branches can degrade the performance since they interrupt the sequential instruction stream. Mispredicted branches can introduce bubbles in the pipeline and may degrade cache performance if code sequences are prefetched but have to be discarded again. Thus, predicated execution can enhance performance for architectures with static parallelism (EPIC, VLIW), and for superscalar pipelined architectures.

Embedded processors are used in a variety of application fields: healthcare technology, telecommunication, automotive and avionics, multimedia applications, etc. Common characteristics of many applications is that high computation performance has to be obtained at low cost and low power consumption. The incorporation of application-specific functionality has the additional consequence that the architectural design of these microprocessors often is highly irregular. In the area of the classical general-purpose processors, compiler technology has reached a high level of maturity. However, for irregular architectures, the code quality achieved by traditional high-level language compilers is often far from satisfactory [8,9]. Generating efficient code for irregular architectures requires highly optimizing techniques that have to be aware of specific hardware features of the target processor.

The PROPAN system [10,11,12,13] has been developed as a retargetable framework for high-quality code optimizations and machine-dependent program analyses at assembly level. From a concise hardware specification a machine-sensitive postpass optimizer is generated that especially addresses irregular hardware architectures. The generated optimizer reads assembly programs and performs efficiency-increasing program transformations.

A precondition for the code transformations performed by PROPAN-generated optimizers is that the control flow graph of the input program is known. In the presence of guarded code, whether an instruction is executed or not depends on the contents of the guard register. Code sequences that compute guard values look just like 'normal' computations — with the exception that the end result is stored in a guard register and this influences the control flow of the program. Thus, an important part of control flow reconstruction from guarded code is detecting the operations that determine the control flow. Moreover, in order to recognize that some operations are executed under mutually exclusive conditions, relations between the contents of guard registers have to be computed. Determining relations between register contents requires simulating the effect of operations on the machine state, i. e. evaluating the instruction semantics. In cases where an exact evaluation is not statically possible conservative approximations have to be available. Thus, a symbolic evaluation is required that is generic to ensure retargetability and that is very precise to enable accurate control flow reconstruction. How this can be achieved is described in this paper.

The article is structured as follows: Sec. 2 gives an overview of related work in the area of control flow reconstruction with the focus on predicated code. Sec. 3 addresses the PROPAN framework. The guarded code semantics which is at the base of our work is presented in Sec. 4; Sec. 5 gives an overview of the control flow reconstruction problem and the approach chosen in PROPAN. Our algorithm to compute the control flow graph is detailed in Sec. 6. The experimental results are presented in Sec. 7, and Sec. 8 concludes.

2 Related Work

Reconstructing control flow for predicated code has not been an issue in most previous approaches. The Executable Editing Library EEL reconstructs control flow graphs from binary code to support editing programs without knowledge of the original source code [14]. Based on a simple high-level machine description EEL can be retargeted to new architectures. The reconstructed control flow graphs are reported to be very precise for some machines, e. g. the SPARC architecture. However, [15] reports that the system is not sufficiently generic to deal with complex architectures and compiler techniques. Reconstructing control flow from predicated instructions is not supported.

`exec2cr1` [15] uses a bottom-up approach for reconstructing the basic control flow graph which solves some problems specific to control flow reconstruction from executables. The targets of control flow operations are computed precisely for most indirections occurring in typical DSP programs. The reconstructed

control flows graphs are used for static analyses of worst case execution times of binary programs. There is no support for reconstructing control flow from predicated instructions.

`asm2c` is an assembly to C translator for the SPARC architecture [16]. The translation requires a CFG which is computed using extended register copy propagation and program slicing techniques. Extended register copy propagation was first used in the `dcc` decompiler [17] which was developed to recover C code from executable files for the Intel 80286. In contrast to `EEL` and `exec2crl`, `asm2c` and `dcc` are not retargetable by specification of a high-level machine-description; the problem of reconstructing control flow from predicated code is not considered. [16] and [17] do not contain any information about the precision of the reconstructed control flow graphs.

An algorithm for reconstructing control flow from guarded (predicated) code, called *reverse if-conversion*, is presented in [18] as a part of a code generation framework. In this framework first a local part of the control flow is if-converted (see Sec. 4) in order to enlarge the scope of the scheduling process. Then the resulting guarded code is scheduled. Subsequently, the reverse if-conversion re-translates the obtained guarded code segment back into a control flow graph which offers precise control flow information to the final analysis and optimization steps. During the if-conversion performed in the early stages of the code generation process all operations which are responsible for control flow joins and forks are marked. The reverse if-condition is depending on those markings to detect operations which directly alter the 'control flow' of the program. Relying on the presence of such markings is contradictory to the retargetability principle of PROPAN, since this would severely restrict the set of supported assembly languages. Thus, we have to explicitly compute all reconstruction information from the assembly source.

3 The PROPAN Framework

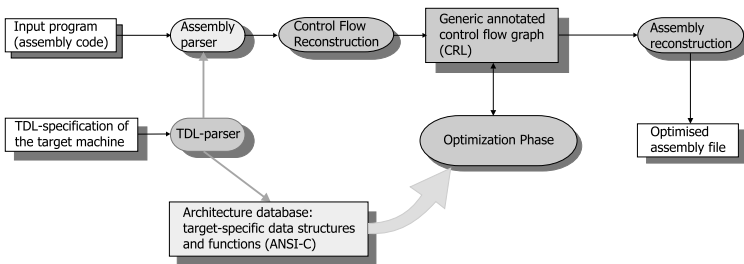


Fig. 2. The PROPAN System

The PROPAN system [10,11,12] has been developed as a retargetable framework for high-quality code optimizations and machine-dependent program analyzes at assembly level. An overview of PROPAN is shown in Fig. 2. The input

of PROPAN consists of a TDL-description of the target machine and of the assembly programs that are to be analyzed or optimized. The TDL specification is processed once for each target architecture; from the TDL description a parser for the specified assembly language and the architecture database are generated. The architecture database consists of a set of ANSI-C files where data structures representing all specified information about the target architecture and functions to initialize, access and manipulate them are defined. The core system is composed of generic and generated program parts. Generic program parts are independent from the target architecture and can be used for different processors without any modification. Hardware-specific information is retrieved in a standardized way from the architecture 'database'. For each target architecture, the generic core system is linked with the generated files yielding a dedicated hardware-sensitive postpass optimizer.

The GECORE-module (GENeric COntrol flow REconstruction) of PROPAN performing the reconstruction of control flow graphs from assembly programs is liable to the same requirements as the PROPAN core system itself. Its core has to be generic while the required target-specific information is retrieved from the architecture database. The first part of the GECORE-module is a generic control flow reconstruction algorithm that reconstructs control flow from assembly programs [13]. Input is a sequence of assembly instructions. Using the architecture description, branch operations are detected and a control flow graph of the input program is determined. In that part guarded execution is not taken into account. The second part is subject of this paper: here an explicit representation of control flow information coded in guard registers is computed.

The optimizations modules of PROPAN are based on integer linear programming and allow a phase-coupled modeling of instruction scheduling, register assignment and resource allocation taking precisely into account the hardware characteristics of the target architecture. By using ILP-based approximations, the calculation time can be drastically reduced while obtaining a solution quality that is superior to conventional graph-based approaches [11,19]. The optimizations are not restricted to basic block level; instead a novel superblock concept allows to extend the optimization scope across basic block and loop boundaries. The superblock mechanism also allows to combine the ILP-based high-quality optimizations with fast graph-based heuristics. This way, ILP optimizations can be restricted to frequently used code sequences like inner loops, providing for computation times that are acceptable for practical use [12].

4 Guarded Code Semantics

If-conversion [20,2,3,21,4] is a compiler algorithm that removes conditional branches from programs by converting programs with conditional branches into *guarded code*. Guarded code contains less branches since the conditions under which an instruction is executed are represented by its guard register. Thus, if-conversion transforms *explicit control flow* via branch and jump operations into *implicit control flow* based on the information of the guard registers.

Given a previously if-converted piece of code, the implicit control flow has to be reconstructed from the guarded code before other analyses or optimizations are performed. Otherwise the implicit control flow information would be lost and the precision of the control flow graph would be degraded which could severely reduce the effectiveness of postpass analyses and optimization techniques. As an example consider the following two predicated instructions:

```
(r3) r5 = load (r9)
(!r3) r7 = r8 + r5
```

If the information that the instructions are guarded by disjoint control flow predicates (**r3** and **!r3**) was not available to a data dependency analysis, a data dependency between both instructions would be reported. This would prevent any reordering or parallelization of both instructions although this would be perfectly feasible.

Our approach for control flow reconstruction is based on the static semantics inference mechanism of [4], which will be summarized in the remainder of this section. The semantics of a guard is a logical formula consisting of branch conditions represented by predicate variables. In these formulas the operators $\wedge$, $\vee$ and $\neg$ are allowed. There also exist constants, such as **true** and **false**. An operation is executed if and only if its guard's semantics is **true**.

A piece of guarded code is a sequence of guarded operations $\langle \text{guard} \rangle ? \langle \text{operation} \rangle$. A statement $C \vdash S$ denotes that, from a piece of guarded code C , a set S of semantics of all occurring guards is deducible. For the reduction of the guard semantics three inference rules exist: *taut*, *fork*, *join*. This analysis requires all guard registers to be initialized to **false** and to be assigned at most once on each control flow path.

$$\begin{aligned}
 & [\text{taut}] \vdash \{g_0 = \mathbf{true}\} \\
 & [\text{fork}] \frac{C \vdash S \cup \{g_1 = l_1\}}{C; g_1 ? g_2 := l_2 \vdash S \cup \{g_1 = l_1\} \cup \{g_2 = (l_1 \wedge l_2)\}} \\
 & [\text{join}] \frac{C \vdash S \cup \{g_1 = l_1\} \cup \{g_2 = l_2\}}{C; g_1 ? g_2 := l_3 \vdash S \cup \{g_1 = l_1\} \cup \{g_2 = ((l_1 \wedge l_3) \vee l_2)\}}
 \end{aligned}$$

The first rule (*taut*) specifies that g_0 always evaluates to **true**; it is used, e.g., as guard of the entry block.

For forks of the control flow a second inference rule (called *fork*) is introduced. From a given code segment $C; g_1 ? g_2 := l_2$ it can be deduced that $S \cup \{g_1 = l_1\} \cup \{g_2 = (l_1 \wedge l_2)\}$ holds if the statement $C \vdash S \cup \{g_1 = l_1\}$ is deducible. Let $S \cup \{g_1 = l_1\}$ be the semantical information of the guard registers obtained by analyzing the operation sequence C . Then, for the sequence $C; g_1 ? g_2 := l_2$ the set of guard semantics $S \cup \{g_1 = l_1\} \cup \{g_2 = (l_1 \wedge l_2)\}$ can be derived. Intuitively formulated, if the semantical information derived for C contains a binding of g_1 to l_1 , then from the guarded statement $g_1 ? g_2 := l_2$ the additional information

that g_2 is bound to $l_1 \wedge l_2$ can be deduced. Since the assignment of l_2 to g_2 is only executed if l_1 is **true**, the effective condition associated with g_2 is $l_1 \wedge l_2$.

The third rule (called *join*) is applied at joins of control flow. In contrast to the *fork* rule the semantical value of g_2 , l_2 , is already known. l_2 represents all values of g_2 reaching the current instruction on control flow paths $\pi_0, \dots, \pi_x$. The semantical value of g_2 on path π_{x+1} (containing the operation $g_1 ? g_2 := l_3$) is $l_1 \wedge l_3$. The semantical value of g_2 after the current instruction is the disjunction of its semantical values reaching the instruction $g_1 ? g_2 := l_3$ on paths $\pi_0, \dots, \pi_x$ or π_{x+1} .

5 Control Flow Reconstruction

The control flow reconstruction module GECORE of the PROPAN system works in two phases. In the first phase control flow reconstruction is done without taking predicated instructions into account. The input of this phase is a generic representation of the assembly instructions of the input program which is provided by the assembly parser generated from the TDL description. An extended program slicing algorithm is used that can deal with unstructured control flow instructions typical for assembly programs. The data structure used for representing control flow is the interprocedural control flow graph (ICFG) [22] which completely represents the control flow of programs. It consists of two components:

1. The *call graph* (CG) describes relationships between procedures of the program. Its nodes represent procedures, its edges represent procedure calls.
2. The *basic block graph* (BBG) describes the intraprocedural control flow of each procedure. Its nodes are the *basic blocks* of the program. A *basic block* is a sequence of instructions that are executed under the same control conditions, i. e., if the first instruction of the block is executed, the others are executed as well. The edges of the BBG represent jumps and fall-through edges³.

Details about this phase can be found in [13].

After the explicit control flow has been reconstructed in the first phase, the second phase deals with the implicit control flow represented by instruction predicates. In the ideal case the reconstructed ICFG represents the control flow precisely. Whenever this is not possible, a safe approximation has to be computed. Another important requirement is that the reconstruction algorithms are generic, i. e. that they can be used for any target architecture without modification. All information about the architecture should be retrieved from the TDL description. From these requirements, several problems arise that have to be addressed when recovering implicit control flow information from guarded code:

1. Each operation possibly affects control flow.

³ Fall-through edges point to successors that are reached by sequential execution of the instructions instead of following a branch.

2. The contents of registers cannot always be statically determined at every instruction. Thus, a symbolic representation of register contents is necessary. In this representation, also the semantical relations to other registers have to be established.
3. In the presence of frequent memory accesses statically determining register contents becomes even more difficult. Enabling the reconstruction algorithm to identify the contents of memory cells requires a precise memory analysis to be incorporated. Since a precise control flow graph is not yet available during the reconstruction process, dedicated analysis approaches are required.

6 The Reconstruction Algorithm

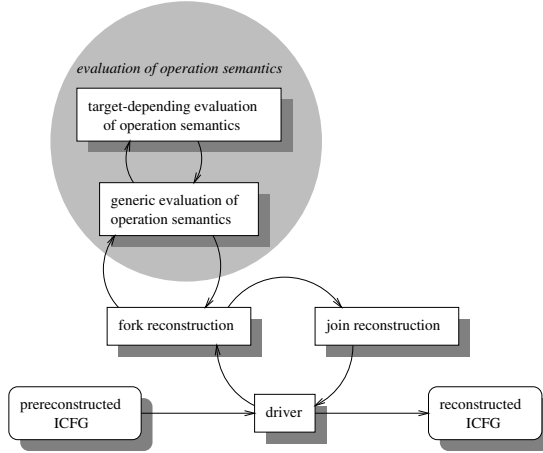


Fig. 3. Recovering implicit control flow

Recovering control flow from guarded code is performed by refining the pre-reconstructed CFG (see Fig. 3). The reconstruction algorithm is applied to each basic block of the pre-reconstructed ICFG. It incorporates two subtasks:

1. For each basic block an equivalent micro-block structure is built which represents implicit forks in the control flow (see Sec. 6.3). During the reconstruction of forks the semantics of assembly operations has to be evaluated (see Sec. 6.2) as part of the value analysis performed.
2. In the second subtask, the micro-block structure is refined to represent control flow joins (see Sec. 6.4); the result is the refined basic block graph where implicit control flow has been made explicit.

Finally, the input basic block is replaced by the computed basic block graph.

6.1 Definitions

An *instruction* is defined as a set of microoperations whose execution is started simultaneously. This definition is mainly used in the context of VLIW architectures. However, a processor not exhibiting instruction-level parallelism can be seen as a special case of a VLIW architecture with each instruction containing only one microoperation.

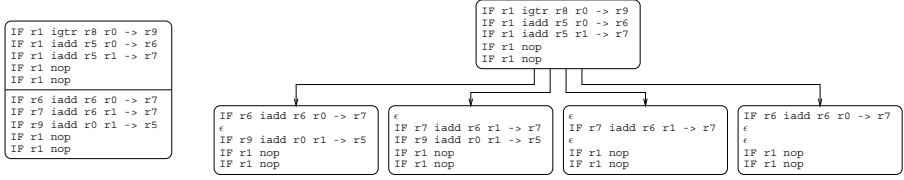


Fig. 4. A procedure and its instruction occurrence graph

We will conceptually distinguish between a microoperation (in the following called operation) and the instantiation of a microoperation in the input program. We will use *operation* to denote the operation type provided by the processor, and use the term *operation instance* to refer to an occurrence of a microoperation with concrete arguments in the input program. To give an example, an operation instance of operation **add** could be **add r1,r2,r3**. The same terminology is canonically applied to instructions.

While reconstructing control flow from guarded code, it can become necessary to duplicate operations or replace them by **nop** if a basic block is decomposed into different control flow paths. For this purpose we use the notion of *operation variation* resp. *instruction variation*. Let o be an operation and $\tilde{o}$ an instance of o . Then a *variation* $\hat{o}$ of $\tilde{o}$ is an instance of o that has exactly the same operands as $\tilde{o}$ or is the empty operation ϵ . The empty operation ϵ is equivalent to an unconditionally executed **nop**. For a processor with k instruction slots, a variation $\hat{i}$ of an instruction i is represented by a $k+1$ -tuple $(a, \hat{o}_1, \dots, \hat{o}_k)$ where $\hat{o}_i$ are variations of operations contained in the instruction instance with address a .

An *execution sequence* π of a procedure is a possible sequence of instruction variations containing only the operations that are executed at run-time, i. e. for which the guard register evaluates to **true**. The occurrence of the variation $\hat{o}$ of an operation o in the execution sequence π is called an *operation occurrence* of the operation o .

The example in Fig.4 shows a block consisting of two TriMedia TM1000 instructions on the left. Paths through the graph on the right are exactly the feasible execution paths through the block on the left. One instruction, shown as a box, consists of five microoperations that are executed simultaneously. The nodes of the graph on the right hand side are feasible instruction variations of the two instructions on the left. Edges represent their ordering. A guard is

interpreted as **true** if the least significant bit is set. Each execution path can contain instructions guarded either by $r6$ or by $r7$, but not both. In the second and third operation of the first instruction they are set to values that cannot be **true** at the same time. The contents of $r5$ is unknown, but adding it to $r0$ (hardwired to $0x0$) always results in a different truth-value (least significant bit) than adding it to $r1$ (hardwired to $0x1$). Thus, in the second instruction operations guarded by $r6$ and those guarded by $r7$ are never executed at the same time. Therefore, feasible instruction variations of the second instruction contain operations that are guarded by either $r6$ (first and fourth instruction variation) or $r7$ (second and third instruction variation). Without information about the contents of $r8$ we are not able to exactly evaluate the greater-than-comparison in the first operation of the first instruction. Therefore, we assume $r9$ to evaluate to either **true** (first and second instruction variation of the second instruction) or **false** (third and fourth instruction variation).

Since during static analyses register contents are not necessarily known at every point of execution, symbolic values have to be introduced. The set of *concrete values* V contains natural numbers, strings and floating point values; *symbolic values* are contained in V' (see Eq. 6.1). Additionally, we have to keep track of the development of register contents over time. Therefore, we introduce the term *register instance* to denote the value of some register at a given point in time. A register instance is a register tagged with a timestamp of a point in time when a value is assigned to the register. We allow register instances to be written only once. Let $\mathcal{R}_I$ be the set of register instances defined in the TDL specification. Then, the set of symbolic values V' is defined as follows:

$$V' = \left\{ \begin{array}{l} \top, \mathbf{true}, \mathbf{false}, \mathit{ref}(r), \mathit{not}(v_x), \\ \mathit{and}(v_x, v_y), \mathit{or}(v_x, v_y) \end{array} \middle| \begin{array}{l} r \in \mathcal{R}_I, \\ v_x, v_y \in V' \cup V \end{array} \right\}$$

While evaluating operation semantics it is not guaranteed that each condition of an if-statement can be properly evaluated. These if-conditions can consist of comparisons or logical computations. However, we require all if-conditions ($\mathcal{C}_I$) to be interpreted either as **true** or **false**. Therefore, whenever an if-condition is reached that cannot be evaluated it is necessary to make assumptions on the truth-value of the condition. In order to face this problem the concept of *meta-environments* is introduced.

Definition 1 (Environment). Let $\mathcal{R}_I$ denote the set of instances of all registers specified in the TDL-description of the target processor and let $\mathcal{C}_I$ be the set of if-condition instances. Furthermore let V be the set of concrete values and V' the set of symbolic values. A symbolic environment $\sigma_{V \cup V'}$ is a triple $(\mathit{map}, \mathit{act}, \mathit{force})$. The function $\mathit{map} : \mathcal{R}_I \cup \mathcal{C}_I \rightarrow V \cup V'$ maps register instances and if-condition instances to (concrete or symbolic) values. The function $\mathit{act} : \mathcal{R} \rightarrow \mathcal{R}_I$ maps a generic register to its active instance. The function force is used to force a register to evaluate to a certain truth-value.

A *meta-environment* is a set of environments; in each environment every occurring condition can be evaluated to **true** resp. **false** during semantics eval-

uation. For each combination of occurring conditions, a dedicated environment has to be contained in a meta-environment.

During the reconstruction of control flow from guarded code, for each basic block in the input ICFG increasingly refined versions of the *micro-block graph* are computed which explicitly represents the implicit control flow of the basic block. Before defining the micro-block graph some additional definitions have to be given.

Definition 2 (Instruction Occurrence Graph). *Let a basic block B of the control flow graph of a procedure p be given. The instruction occurrence graph of B is a minimal directed graph $G_I = (N_I, E_I, N_A, N_\Omega)$ with node labels. For each instruction occurrence i' of each instruction i in B which belongs to an execution sequence of p , there is a node $n_{i'} \in N_I$ that is marked by i' . Edges (n', m') exist in E_I if and only if n' and m' are subsequent instruction occurrences of the same execution sequence. N_A is the set of occurrences of the entry instruction of B and N_Ω is the set of occurrences of the exit instruction of B .*

Definition 3 (Micro-Block). *A micro-block of an instruction occurrence graph is a path of maximal length which has no joins except possibly at the beginning and no forks except possibly at the end.*

Definition 4 (Micro-Block Graph). *The micro-block graph $G_M = (N_M, E_M, m_A, m_\Omega)$ of an instruction occurrence graph $G_I = (N_I, E_I, N_A, N_\Omega)$ is formed from G_I by combining each micro-block into a node. Edges of G_I leading to the first node of a micro-block lead to the node of that micro-block in G_M . Edges of G_I leaving the last node of a micro-block, lead out of the node of that micro-block in G_M . m_A denotes the (possibly empty) entry micro-block that has an edge to each micro-block containing an entry node. m_Ω denotes the set of micro-blocks containing the exit nodes.*

During the process of building the micro-block graph, all executions of the basic block are simulated such that all feasible execution paths are covered. Let π be the path in the partially reconstructed micro-block graph from the entry node to the leaf micro-block b . The meta-environment of b in the partially reconstructed micro-block graph represents the contents of registers after the execution of all instruction variations on the path in the micro-block graph from the entry node to the leaf node b .

Within the scope of the reconstruction of guarded code, a *safe approximation* of the micro-block graph is the micro-block graph of a safe approximation of the instruction occurrence graph. An approximation of the instruction occurrence graph IOG^0 is *safe* if it contains at least all paths of the IOG^0 .

Definition 5 (Fitting Instruction). *Let i be an instruction and $\Sigma_{V \cup V'}$ be a meta-environment. i_F is the fitting instruction of i and $\Sigma_{V \cup V'}$ if for all operations o_F contained in i_F and the corresponding operations o of i holds that*

- $o_F = \epsilon \iff$ the guard register of o evaluates to **false** within all environments of $\Sigma_{V \cup V'}$ or
- $o_F = o \iff$ the guard register of o evaluates to **true** within all environments of $\Sigma_{V \cup V'}$.

A fitting operation is a single operation for which one of the conditions above holds.

The existence of a fitting instruction is not guaranteed. The guard register of an operation could evaluate to **true** as well as **false** within the meta-environment of a micro-block. Assume the meta-environment $\{\{r3 \mapsto \top\}\}$. A fitting operation does not exist for the following operation because the guard register, $r3$, cannot be uniformly evaluated to **true** or **false**: **IF** $r3$ **add** $r0$ $r1 \rightarrow r4$. For **IF** $r1$ **add** $r0$ $r1 \rightarrow r4$, the fitting operation is the operation itself since $r1$ evaluates to **true**.

6.2 Instruction Semantics Evaluation

The operation semantics is defined in the instruction set section of the TDL specification. TDL provides its own *register transfer language RTL*, which is statement-oriented in order to generate cycle-accurate instruction-set simulators requiring a precise specification of what happens in which cycle. It is described in detail in [23,12]; a formal approach defining the operation semantics using derivation rules is presented in [24].

The symbolic evaluation of instruction semantics must be aware of the definition of truth values by the processor modeled. For instance, the TriMedia TM1000 interprets register contents as **true** or **false** depending on the least significant bit. In the ADSP-2106X SHARC on the other hand, a register evaluating to **false** must have all bits set to 0. For different interpretations slightly different derivation rules have to be defined. Within the scope of this paper we model memory locations as unknown values since no value analysis for memory cells and no alias analysis is performed. Incorporating these analysis in the control flow reconstruction process is a goal of future work.

Our approach is based on an extended constant propagation analysis supporting symbolic values. The relevant program state comprises the contents of all registers and is represented by meta-environments. During the reconstruction of implicit control flow for a basic block increasingly refined versions of the micro-block graph are computed. The micro-block graph is built bottom-up. Whenever the analysis determines that an instruction occurrence has to be arranged within a specific micro-block the meta-environment of that block is updated by evaluating the instruction semantics. This simulates multiple executions: the instruction occurrence is "executed" within each binding of registers represented by the meta-environment. In order to properly evaluate if- and while-statements the corresponding condition is always required to evaluate to **true** or **false**. To ensure this, appropriate environments are added to the current meta-environment. In order to reduce the number of environments in

a meta-environment, environments which are indistinguishable with respect to the truth value of all registers are replaced by a single representative. Detailed information about the symbolic evaluation can be found in [24].

6.3 Fork Reconstruction

While building up the micro-block graph, its leaf blocks are called *visible*. The *env* function is used to retrieve the meta-environment from a micro-block; the *instr* function is used to access the set of instructions of a micro-block.

Starting point of the reconstruction is a basic block of the precomputed CFG and a micro-block graph containing only one empty micro-block. The empty micro-block contains no instructions, has no successors and is associated with an environment that maps all registers to $\top$, i. e. that does not force any register to evaluate to a special value. First, we successively arrange the instructions of the input block into the visible blocks of the micro-block graph. For this purpose we compute the fitting instruction for each instruction in every visible block.

The fitting operation of an operation and a meta-environment is computed as follows: In the case the operation is not guarded or the guard register evaluates to **true**, the fitting operation is o . If the operation is guarded but the guard register evaluates to **false**, the operation cannot change the environment; the fitting operation is ϵ . The result is undefined if it cannot be uniformly determined whether the guard register evaluates to **true** or **false** within the meta-environment Σ' .

If a fitting instruction exists we add it to this block and update the meta-environments using semantics evaluation. In case the fitting instruction does not exist for a certain block we introduce two empty successor blocks with the same meta-environment as the block. In one block the guard register preventing the existence of the fitting instruction is forced to evaluate to **true**, in the other to **false**. Then, these blocks are considered for arranging the instruction instead of their parent block. Once all visible blocks are processed for an instruction, the subsequent instruction is arranged. Using this technique we separate different control flow paths from each other.

An example of an input block containing TriMedia TM1000 assembly instructions is given in Fig.5. Instructions are referred to as i_0 , i_1 and i_2 . The micro-block graph obtained from reconstructing forks of the input block given in Fig.5 is illustrated in Fig.6.

We refer to the instruction variations in Fig. 5 by i'_0 for the instruction variation of i_0 and i'_1 , i'_2 for the instruction variations of i_1 and i_2 in block b_1 . Block b_2 contains instruction variations i''_1 and i''_2 . Instruction i_0 can be arranged without problems in block b_0 resulting in i'_0 because all operations are guarded by $r1$ which is hardwired to $0x1$. Since the contents of $r8$ is unknown ($\top$) at the beginning of the analysis, we cannot compute the exact value of $r9$. We split the environment into a meta-environment containing two environments: one where $r9$ is **true** and one where it is **false**. Within the environments of that meta-environment we are able to evaluate the less-or-equal-comparison of the second operation: $r6$ is **true** in the environment where $r9$ evaluates to **false** and it evaluates to **false** for the environment containing $r9$ to be **true**.

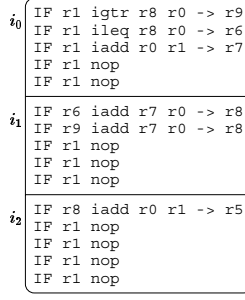


Fig. 5. An input block containing TriMedia TM1000 assembly instructions

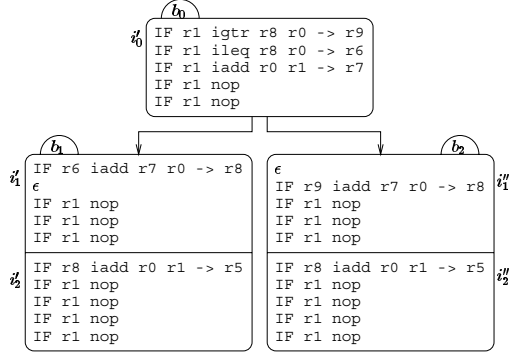


Fig. 6. Micro-block graph after reconstructing forks of the block in Fig. 5

In both environments of the meta-environment $n7$ is set to $0x1$. Next we try to arrange i_1 into b_0 , but within b_0 no fitting instruction for i_1 exists. In the meta-environment associated with b_0 , $r9$ can possibly evaluate to **true** as well as to **false**. Thus, successor blocks (b_1 and b_2) are introduced. b_1 is associated with the meta-environment containing only those environments with $r9$ evaluating to **true**. The meta-environment of b_2 contains only environments where $r9$ is **false**. This implies that $r6$ is **false** in b_1 resp. **true** in b_2 . In both blocks (b_1 , b_2) a fitting instruction for i_1 exists: i_1' in b_1 , i_1'' in b_2 . Since within b_1 $r9$ evaluates to **true** and $r6$ to **false** i_1' contains the operation guarded by $r9$ but the operation guarded by $r6$ is replaced by ϵ ; i_1'' is handled likewise. These instruction variations set $r8$ to $r7$ which is $0x1$ in the meta-environments of both blocks. Therefore, instruction variations i_2' and i_2'' both contain the operation guarded by $r8$.

1

6.4 Join Reconstruction

From the first phase of the reconstruction, an approximated micro-block graph in form of a tree is obtained. This graph explicitly represents all forks in the control flow of an input block. It is a safe approximation of the micro-block graph (a proof is given in [24]). Different control flow paths are separated from each other

but joins of control flow are not represented yet. Presuming no additional paths are introduced, reconstructing joins is equivalent to computing a smaller solution of the approximated micro-block graph, i.e. the resulting micro-block graph is more precise.

We recognize joins of control flow by identifying equal instruction occurrence sequences at the end of paths through the micro-block graph. Assume two equal subpaths starting with instruction i . Then, these paths can be combined into one single subpath starting with i and representing a join of control flow.

Join detection is initiated at the lowest address of instructions in the input basic block. We look for pairs of instruction occurrences at address a that are roots of equivalent subgraphs in the micro-block graph. In the case such a pair is found at address a the join is reconstructed by modifying the micro-block graph in such a way that the common subgraphs are shared. If no pair of equivalent instruction occurrences can be found at address a the subsequent address $a + 1$ is inspected.

Considering two instruction occurrences as equivalent requires introducing the notion of *similar* operations. Operations are similar either if they are equal or if one of them is ϵ and the other is **nop**. Similar operations have the same effect on environments. Then, two single instructions can be denoted equivalent if they are similar themselves, i.e. all operations contained are similar, and for each immediate successor instruction of the first instruction an equivalent successor instruction of the second can be found and vice versa.

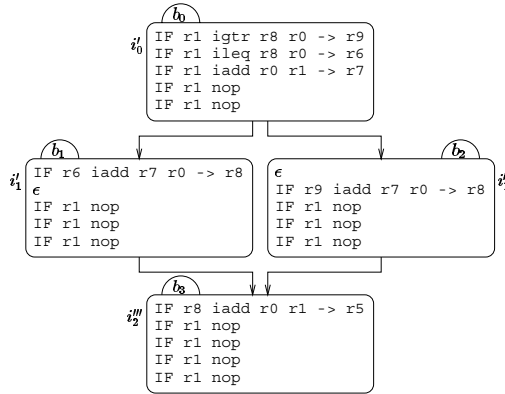


Fig. 7. Micro-block graph after reconstructing joins of the block in Fig.5

The micro-block graph for the input block in Fig.5 after reconstructing also joins of the control flow is shown in Fig.6. For reconstructing joins we successively inspect instruction occurrences at the same address of the micro-block graph obtained from fork reconstruction. For i'_0 there is no other instruction occurrence to compare with. Instructions i'_1 and i''_1 are not equivalent since they themselves are not similar. Instruction occurrences i'_2 and i''_2 (in Fig.6) can be combined

into the single instruction occurrence at the bottom of Fig.7. They are similar and since they do not have successors they are considered as equivalent.

7 Experimental Results

The algorithm for recovering control flow from guarded code has been evaluated using the TriMedia TM1000 [5]. The TriMedia TM1000 is a multimedia VLIW processor providing several hardware characteristics that make control flow reconstruction difficult: it exhibits significant instruction level parallelism, implements procedure calls and returns by jump instructions and uses predicated execution for all machine operations.

Our input programs comprise the DSPSTONE benchmark [8] and some additional typical digital signal programming applications. The experiments have been executed on an AMD Athlon 1400 processor with 512 MByte RAM running Linux; the assembly files have been generated by the Philips `tmcc` compiler [5] at highest optimization level.

Fig. 8 shows the statistics of the control flow reconstruction. Column $\#I$ gives the number of assembly instructions for each input program. Columns $\#B_{ex}$ shows the number of blocks after the reconstruction of explicit control flow [13]; column $\#B_{em}$ shows the number of blocks after reconstructing the implicit control flow. Before implicit control flow reconstruction there is only one path through every block. Hence, the number of paths through the blocks is equal to the number of blocks. During the guard-sensitive reconstruction these blocks are split and additional paths are introduced (see Fig.4); the number of these additional paths through the reconstructed blocks are shown in column $\#P$. Edges representing explicit control flow are not taken into account in the figures of column $\#P$. Column $\#P/\#B_{ex}$ shows the number of intra-block paths after reconstructing implicit control flow divided by the number of intra-block paths before implicit reconstruction. The execution time of the reconstruction in milliseconds is presented in column t . It shows the time for reconstructing the implicit control flow only and does not include the time needed for building the initial CFG.

The numbers of paths shown in the fifth column give a hint on how much precision is gained by reconstructing implicit control flow from guarded code. To give an example, for `whet` the control flow graph obtained by reconstructing explicit control flow contains 46 blocks. After reconstructing implicit control flow from predicated instructions the control flow graph contains 62 additional blocks. While after reconstructing explicit control flow exactly one path is visible through each basic block, the reconstruction of implicit control flow makes 22 additional "intra-block" paths visible. The number of "intra-block" paths is lower than the number of blocks after recovering implicit control flow because in situations where joins are reconstructed (see Fig.7) more additional blocks are introduced than additional intra-block paths become visible. For example the micro-block graph in Fig.7 contains 3 additional blocks but only 1 additional intra-block path compared to its input block. An illustration of the number of

file name	#I	#B _{ex}	#B _{em}	#P	#P/#B _{ex}	t [msec]
biquad_N_sections	56	6	12	8	1.33	27
biquad_one_section	28	7	7	7	1	12
c_fir	250	54	144	87	1.61	599
c_firfxd	168	23	50	32	1.39	249
c_vecsum	681	120	309	184	1.53	2,069
complex_multiply	10	4	4	4	1	2
complex_update	8	4	4	4	1	2
convolution	49	7	16	10	1.43	34
dot_product	25	7	7	7	1	11
fft	436	52	108	71	1.37	507
fir	56	11	23	15	1.36	41
fir2dim	193	15	50	27	1.8	830
iir1	27	3	6	4	1.33	26
iir2	27	9	15	11	1.22	26
lms	65	10	28	16	1.6	51
mat1x3	40	6	9	7	1.17	25
matrix1	202	42	226	134	3.19	781
matrix2	238	45	196	116	2.58	452
n_complex_updates	57	14	29	19	1.36	37
n_real_updates	70	12	27	17	1.42	44
puzzle	392	82	396	226	2.76	1,035
real_update	24	7	7	7	1	9
vec_mpy1	58	3	6	4	1.33	86
vec_mpy2	26	8	11	9	1.13	19
whet	648	46	108	68	1.48	1,618

Fig. 8. Statistics of control flow reconstruction

intra-block paths before and after the reconstruction of implicit control flow for each input program is given in Fig. 9.

Since our approach works at basic block level, we have to assume that at the entry of each basic block register contents are unknown. Values read from memory also have to be considered as unknown. Thus, we may overestimate the number of possible control flow paths. However, this overestimation does not reduce the enhanced precision of analyses and optimizations gained by making implicit control flow paths explicitly visible. Nevertheless, if the control flow graph contains infeasible control flow paths the computation time of algorithms working with the control flow graph may increase, and their scope may be reduced. The control flow graph resulting after the reconstruction of implicit control flow provides a safe basis for global value analyses and memory analysis. Such analyses can be used to remove infeasible control flow paths from the reconstructed control flow graph; incorporating them into our framework is subject of future work.

paths the computation time of algorithms working with the control flow graph may increase, and their scope may be reduced. The control flow graph resulting after the reconstruction of implicit control flow provides a safe basis for global value analyses and memory alias analyses. Incorporating suitable analyses to remove spurious control flow paths into the GECORE module is subject of our future work. Another goal is to apply the reconstruction to other processors featuring predicated execution like the Intel IA64-architecture [7].

References

1. B. Rau and J. Fisher, "Instruction-Level Parallel Processing: History, Overview, and Perspective," *The Journal of Supercomputing*, vol. 7, pp. 9–50, 1993.
2. J. Park and M. Schlansker, "On Predicated Execution," Tech. Rep. HPL-91-58, Hewlett-Packard Laboratories, Palo Alto CA, May 1991.
3. J. Dehnert and R. Towle, "Compiling for the Cydra 5," *The Journal of Supercomputing*, vol. 1/2, pp. 181–228, May 1993.
4. P. Hu, "Static Analysis for Guarded Code," in *Languages, Compilers, and Run-Time Systems for Scalable Computers*, pp. 44–56, 2000.
5. Philips Electronics North America Corporation, *TriMedia TM1000 Preliminary Data Book*, 1997.
6. Analog Devices, *ADSP-2106x SHARC User's Manual*, 1995.
7. Intel, *IA-64 Architecture Software Developer's Manual, Volume 1: IA-64 Application Architecture, Revision 1.1*, July 2000.
8. V. Zivojnovic, J. Velarde, C. Schläger, and H. Meyr, "DSPSTONE: A DSP-Oriented Benchmarking Methodology," in *Proceedings of the International Conference on Signal Processing Applications and Technology*, 1994.
9. R. Leupers, *Retargetable Code Generation for Digital Signal Processors*. Kluwer Academic Publishers, 1997.
10. D. Kästner and M. Langenbach, "Code Optimization by Integer Linear Programming," in *Proceedings of the 8th International Conference on Compiler Construction CC'99* (S. Jähnichen, ed.), pp. 122–136, Springer LNCS 1575, Mar. 1999.
11. D. Kästner, "PROPAN: A Retargetable System for Postpass Optimisations and Analyses," *Proceedings of the ACM SIGPLAN Workshop on Languages, Compilers and Tools for Embedded Systems*, June 2000.
12. D. Kästner, *Retargetable Code Optimisation by Integer Linear Programming*. PhD thesis, Saarland University, 2000.
13. D. Kästner and S. Wilhelm, "Generic Control Flow Reconstruction from Assembly Code," *Proceedings of the ACM SIGPLAN Jointed Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES 2002) and Software and Compilers for Embedded Systems (SCOPES'02)*, June 2002.
14. J. Larus and E. Schnarr, "EEL: Machine-Independent Executable Editing," in *SIGPLAN Conference on Programming Language Design and Implementation*, pp. 291–300, 1995.
15. H. Theiling, "Extracting Safe and Precise Control Flow from Binaries," in *7th International Conference on Real-Time Computing Systems and Applications*, July 2000.
16. C. Cifuentes, D. Simon, and A. Fraboulet, "Assembly to High-Level Language Translation," pp. 228–237, Aug. 1998.

17. C. Cifuentes, “Interprocedural Data Flow Decompilation,” Tech. Rep. 4(2), June 1996.
18. N.J. Warter, S.A. Mahlke, W.-M.W. Hwu, and B.R. Rau, “Reverse If-Conversion,” *ACM SIGPLAN Notices*, vol. 28, no. 6, pp. 290–299, 1993.
19. D. Kästner, “ILP-based Approximations for Retargetable Code Optimization,” *Proceedings of the 5th International Conference on Optimization: Techniques and Applications (ICOTA 2001)*, 2001.
20. J. Allen, K. Kennedy, C. Porterfield, and J. Warren, “Conversion of control dependence to data dependence,” in *Conference record of the 10th ACM Symposium on Principles of Programming Languages (POPL)*, pp. 177–189, 1983.
21. J. Hoogerbrugge and L. Augusteijn, “Instruction Scheduling for TriMedia,” 1999.
22. F. Martin, *Generation of Program Analyzers*. PhD thesis, Saarland University, 1999.
23. D. Kästner, “TDL: A Hardware and Assembly Description Language,” Tech. Rep. TDL1.4, Transferbereich 14, Saarland University, 2000.
24. B. Decker, “Generic Reconstruction of Control Flow for Guarded Code from Assembly,” Master’s thesis, Saarland University, 2002.

Control Flow Analysis for Recursion Removal

Stefaan Himpe¹, Francky Catthoor², and Geert Deconinck¹

¹ Katholieke Universiteit Leuven

Kasteelpark Arenberg 10, 3001 Leuven

{Stefaan.Himpe,Geert.Deconinck}@esat.kuleuven.ac.be

² IMEC, Kapeldreef 75, 3001 Leuven

Catthoor@imec.be

Abstract. In this paper a new method for removing recursion from algorithms is demonstrated. The method for removing recursion is based on algebraic manipulations of a mathematical model of the control flow. The method is not intended to solve all possible recursion removal problems, but instead can be seen as one tool in a larger tool box of program transformations. Our method can handle certain types of recursion that are not easily handled by existing methods, but it may be overkill for certain types of recursion where existing methods can be applied, like tail-recursion. The motivation for a new method is discussed and it is illustrated on an MPEG4 visual texture decoding algorithm.

1 Introduction

Recursion allows for elegant specification of certain types of algorithms. In the context of optimizing compilers for embedded systems, however, recursion is known to often cause overhead in terms of function calls and stack frames. Our first concern is *not* to remove all this overhead by removing the recursion. Instead we intend to remove recursion to enable other (parallelizing) transformations that actually remove overhead. In this paper we will demonstrate a new method for removing recursion from applications on a quality-of-service scalable MPEG4[1] visual texture decoding algorithm.

Consider the code presented in Figure 1. This code is a small part from a prototype implementation of a real-life MPEG21 related application[2]. The algorithm implements an n -level recursive quadtree decomposition and decoding of a rectangular image. Profiling reveals that over 50% of the visual texture decoder's execution time is spent in the recursive Decode function. (The exact numbers depend on the compiler and compiler flags being used.) Clearly this is a function which would benefit from optimization. One approach to reduce execution time is to parallelize the code, and to map it to multiple functional units or even processors. The Decode function is called many times while decoding an MPEG 4 texture, each time with different values for its arguments. The workload of the Decode function varies exponentially with the value of n . We may try to parallelize inside the DecodePixel function to be able to reduce the execution time, but in this specific example this appears to be difficult, due to the complex algorithm with many data dependencies. Another approach could be

to run multiple DecodePixel functions in parallel instead, but the entanglement of the calculation with the recursive control flow makes this more complicated than needed.

The exact execution order of the DecodePixel and Check functions needs to be preserved to ensure correctness in the presence of side-effects. Unrolling the recursion one level seems an option to enable parallelization of the code, but it becomes awkward when the number of processors on the target platform is not a multiple of 2. Especially the code size can increase dramatically because of the unrolling and handling of border cases. If we can serialize the recursion to a regular loop, compiler optimizations like unrolling and software pipelining can be applied very flexibly.

In addition to the reasons above which stem from our background in embedded system design, recursion is also known to cause resource consumption due to creation of stack frames at run-time which hold the variables that might be needed after a specific invocation of the recursive function ends. If the recursion could be removed entirely, this storage overhead can be removed as well. The recursion removal we propose can indeed remove this storage overhead, but will introduce extra computations. In the past, it has been shown that having more computations as opposed to using more memory can still have positive consequences for energy efficiency[3]. A trade-off between memory cost and amount of calculations that must be evaluated by the system designer results.

In this paper, we show how we systematically remove the recursion from the MPEG4 VTC decoder algorithm, and arrive at an equivalent iterative solution. This will be done in such a way that we do not have to look inside the implementations of the DecodePixel and Check functions, even if they contain certain side-effects. This is especially useful if the implementations of DecodePixel and Check are too complex (or would take too much design time) to fully analyze. Our on-going work indicates that this method can be generalized and applied to other recursive algorithms.

Algorithm MPEG4 visual texture de- coding	Example 2-level quadtree decomposition
<pre>Decode(int n, int x, int y) { if (n==0) DecodePixel(x,y); else { int k; --n; k = 1<<n; Decode(n, x, y); if (n==4) Check(); Decode(n, x+k, y); if (n==4) Check(); Decode(n, x, y+k); if (n==4) Check(); Decode(n, x+k, y+k);if (n==4) Check(); } }</pre>	

Fig. 1. MPEG4 visual texture algorithm

2 Related Work

Recursion removal is a well studied subject, and many results are available both in theory and practice. One remarkable result is that any recursive function that ends in a finite amount of time, can be implemented in an iterative way by insertion of a stack data structure[4]. Given such a result, why would anyone look for more methods? Well, often several iterative versions of recursive functions can be found which behave differently with respect to space and time complexity as compared to the recursive function, and which are better than the result of the general recursion removal by stack insertion.

Another well-known result is that tail-recursion is equivalent to a loop[4] and in some cases, non-tail recursion can be rearranged to become tail-recursion[5]. Extensions also exist for removal of mutual recursion. One approach for removal of mutual recursion is by inlining some functions in other functions, resulting in direct recursion which can then further be optimized using normal recursion removal techniques, as reported in [6,7].

Many results are based on recognizing patterns in the source code which can be proved, using a myriad of techniques, to be equivalent to certain iterative patterns. These patterns are known as recursion schemas[5,6].

A lot of results in recursion removal were found in the context of optimizing compilers for functional programming languages. In 1962 already, John McCarthy implemented tail recursion removal in his LISP compiler[8]. Later John Backus worked on formal transformations on his FP programming language[9,10]. In such languages, however, some assumptions can be made which are not necessarily true in imperative languages[11]. One typical assumption is the lack of side-effects as a result of modifying state, since pure functional programming languages have no concept of state[12].

Recent work[13] has concentrated on removing recursion in quite general situations, and providing measures for the effects in terms of time and space complexity. Their basic idea is to identify a so-called generalized function increment, and then use a loop to calculate the result using the increment. The method is very general and can guarantee improvements in both time and space. In the context of our work, which is related to program transformations to enable parallelization for multi-processor embedded platforms, their method has a potential draw-back: the method itself introduces explicit loop iteration dependencies. Indeed, before the next iteration can be evaluated, the previous one needs to be completed.

Our on-going work addresses some forms of non-linear recursion, which can contain code with side-effects, and systematically finds an iterative version that has no explicit iteration dependencies caused by the method. Such a transformed version is expected to be beneficial in the context of task level parallelization of algorithms. Future work will include further formalization and generalization of the method to better understand the conditions that need to be fulfilled and the assumptions that need to be made in order to yield useful results.

3 Terminology

We start by establishing some terminology.

Definition 1. A call graph is a directed graph represented by a tuple (V, E) . V is a set of functions defined in the implementation of the program under consideration. E is a set of edges $(v_1, v_2) \in V \times V$. An edge (v_1, v_2) is contained in E if function v_1 could directly call function v_2 during the execution of the program, as specified in the implementation.

Definition 2. Two or more different functions f_j are mutually recursive if their call graph contains a loop involving all of the functions f_j . A function f is said to be simple recursive if its call graph (V, E) has an edge $(f, f) \in E$, and f is not mutually recursive to another function $g \neq f$.

In this paper we will restrict our attention to simple recursive functions. A necessary condition for a function to terminate in a finite amount of computing steps, is the existence of at least one branch in the function definition that does not contain a call to the function itself. This leads to the following definitions.

Definition 3. A base case of a simple recursive function f is a branch in the definition of f that does not contain a call to function f .

A recursive case of a simple recursive function f is a branch in the definition of f that contains a call to function f .

Definition 4. A parameter or variable is called a control flow parameter with respect to a function if it contributes to determining the control flow inside this function.

Definition 5. A simple recursive function f is said to be linear recursive if each of the recursive cases calls the function f only once. If at least one recursive case calls the function f multiple times, the recursion is said to be non-linear.

Definition 6. A call-value graph is a directed graph represented by a tuple (V, E) . V is a set of functions with a specific value for their control flow parameters. E is a set of edges $(v_1, v_2) \in V \times V$. There is an edge (v_1, v_2) in the call-value graph if a function v_1 with specific control flow parameter values could directly call function v_2 with specific control flow parameter values as specified in the implementation.

Definition 7. A function is tree recursive if it is simple recursive and its call-value graph has no loops.

Note that a function with a cyclic call-value graph does not necessarily lead to infinite recursion, because edges occur if a certain function could be called from another function. It does not mean that this particular function will be called.

Definition 8. A basic block is a maximal sequence of instructions that can be entered only at the first of them and exited only from the last of them. For our purposes, a function call is not considered a branch, except where the function calls itself recursively.

Definition 9. A side-effect is a computational effect caused by expression evaluation that persists after the evaluation is completed.

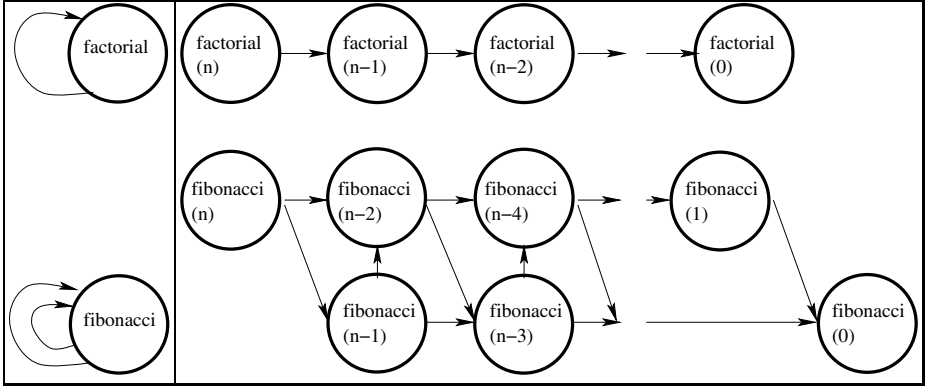


Fig. 2. Call graph (left) vs. call-value graph (right)

In the decoder example depicted in figure 1 the branch executed when $n = 0$ is a base case, the branch executed when $n \neq 0$ is a recursive case. As the recursive case calls the function Decode multiple times, this is an example of non-linear recursion (in this case: tree recursion).

4 Two-Step Method

We now proceed to show how we can systematically remove the recursion from the visual texture decoding algorithm shown in figure 1. Profiling revealed that the base case calculations (DecodePixel) dominate with respect to execution time. We first separate the recursion from the calculations. This causes storage overhead, but already gives opportunities for parallelization. In the second step, the storage overhead is removed, without imposing restrictions for parallelization, at the expense of more calculations.

4.1 Step 1: Separate Recursive Control Flow from Base-Case Calculations

1. Generate appropriately instrumented version of recursive function which
 - (a) records the sequence of basic blocks that are activated in the recursive cases, if they contain helper function calls that need to be preserved, as well as the function arguments that are passed to these helper functions

Algorithm 1 Separating recursion from calculation in the visual texture decoder

```

int x_v[upperbnd], y_v[upperbnd];
bool eq_n_4[upperbnd]; int cnt = -1;
// instrumented version of recursive Decode function
void DecodePhase1(int n, int x, int y) {
    if (n==0) {
        ++cnt; x_v[cnt] = x; y_v[cnt] = y;
    }
    else {
        int k; --n; k = 1<<n;
        DecodePhase1(n, x, y);    if (n==4) eq_n_4[cnt]=true;
        DecodePhase1(n, x+k, y);  if (n==4) eq_n_4[cnt]=true;
        DecodePhase1(n, x, y+k);  if (n==4) eq_n_4[cnt]=true;
        DecodePhase1(n, x+k, y+k);if (n==4) eq_n_4[cnt]=true;
    }
}
void Decode (int n, int x, int y) {
    DecodePhase1(n, 0, 0);
    // synthesized main loop
    for (int i=0; i<=cnt; i++) {
        DecodePixel(x+x_v[i],y+y_v[i]);
        if (eq_n_4[i])
            Check();
    }
}

```

(b) records values of arguments in the base cases if they are needed in helper function calls

2. Synthesize new iterative loop that uses the recorded results to perform the actual calculations

This first separation step is a platform architecture and application independent enabling step. Separating recursion from base case calculation in itself will not necessarily result in an improved implementation. There are advantages, however, to having a recursive control flow which no longer contains base case calculations. The recursive part will no longer be dominant with respect to execution time. The synthesized main loop will be dominant with respect to execution time. Profiling reveals that the main loop indeed takes about 7 times more execution time than the recursive information collection. Between different iterations of the loop no dependencies exist other than those that are potentially hidden inside the base case calculations themselves. The separation step does not impose any restrictions on further parallelization. Hence if memory cost is considered less important than execution time, this first step may already be advantageous.

The results of applying the first step on the Decode algorithm are depicted in Algorithm 1. Note that the extra code between each of the recursive function calls is identical. Therefore, only one eq_n_4 basic block activation counter is needed, as opposed to four different ones. While this is certainly a special case, it is by no means required to apply any of the steps in this paper.

4.2 Step 2: Replace Recorded Data with a Function Generating this Data

Part 1: Modeling Argument Flow. We replace the data recording with a function that generates this data, and which needs less storage space than the

equivalent look-up table. We will show how we replace the data that was recorded during the recursive traversal with equations that generate the same data in the same order. Preserving the ordering in time of specific function activations is important because only then one can ignore potential side-effects inside the *DecodePixel* function or inside the *Check* function. This avoids having to analyze the implementations of *DecodePixel* or *Check*. This is important because such analysis can be quite complex and/or time-consuming.

The result will be that the storage overhead that was introduced by the instrumentation is removed, as well as the storage needed for stack-frames. Some execution time will be removed because the number of function calls decreases, but execution time will be added in order to evaluate the equations. Clearly there is no guarantee that both execution time and memory cost will be reduced at the same time. Instead a trade-off between the different cost factors can result, and a system designer will need to evaluate it. We need to solve two types of subproblems.

The first subproblem is the problem of finding the number of iterations needed in the main loop for a given value of the control flow parameter n . In the decoder example it is easily found by inspection: each call to *DecodePhase1* ($n, _, _$) results in 4 calls to *DecodePhase1* ($n - 1, _, _$). The number of iterations for a given value of n , $I(n)$, results from solving a recurrence equation $I(n) = 4 \cdot I(n - 1)$ where $I(0) = 1$ with result

$$I(n) = 4^n = 2^{2n} = 1 \ll (n \ll 1). \quad (1)$$

(The $\ll$ denotes the bit-wise shift left operator.)

The second subproblem is the one of replacing recorded data with an equation producing that data. We first derive an equation that describes how the arguments passed to the *DecodePixel* function are being produced through the recursive control flow. Define the operator $\oplus$ as follows.

Definition Operator $\oplus$: Given two tuples of matrices $x = (x_0, x_1, \dots, x_{N-1})$, $x_i \in \mathbb{R}^{2 \times 1}$ and $y = (y_0, y_1, \dots, y_{M-1})$, $y_i \in \mathbb{R}^{2 \times 1}$. Then

$$\begin{aligned} x \oplus y &= (x_0 + y_0, x_0 + y_1, \dots, x_0 + y_{M-1}, \\ &\quad x_1 + y_0, x_1 + y_1, \dots, x_1 + y_{M-1}, \\ &\quad \dots, \\ &\quad x_{N-1} + y_0, x_{N-1} + y_1 \dots, x_{N-1} + y_{M-1}). \end{aligned} \quad (2)$$

If the arguments passed to the first invocation of the recursive function are given by control flow parameter n and extra arguments $\begin{bmatrix} x \\ y \end{bmatrix}$, then the arguments passed to the *DecodePixel* base case calculation are described by:

For $n = 0$, the arguments are given by

$$A_0 = \begin{bmatrix} x \\ y \end{bmatrix}.$$

For $n \in \mathbb{N}$, $n \geq 1$, the arguments are given by

$$A_n = \left[\bigoplus_{i=0}^{n-1} \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 2^{n-i-1} \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 2^{n-i-1} \end{bmatrix}, \begin{bmatrix} 2^{n-i-1} \\ 2^{n-i-1} \end{bmatrix} \right) \right] \oplus \begin{bmatrix} x \\ y \end{bmatrix}. \quad (3)$$

Our intention is to construct a formula to calculate the j -th member of sequence A_n , A_{nj} , from the elements of A_1 without having to calculate and store any intermediate sequences A_i , $i < n$. See figure 3 for an illustration. The arrows on the figure indicate dependencies between sequence members. They are only shown between A_1 and A_2 for reasons of clarity. Similar dependencies exist between A_i and A_{i+1} , $i = 1 \dots n-1$.

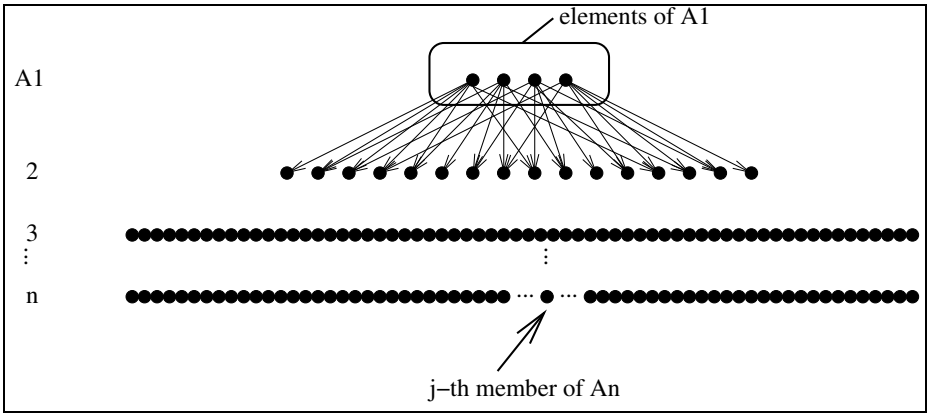


Fig. 3. Sequences A_i , $i = 1 \dots n$ and calculating the j -th member of A_n from the elements of A_1

To start, we derive some properties of the $\oplus$ operator as defined in Eq. (2).

Given a sequence $a = (a_0, a_1, \dots, a_{A-1})$ and a second sequence $b = (b_0, b_1, \dots, b_{B-1})$ with length A and B respectively. Then

$$\begin{aligned} & (a_0, a_1, \dots, a_{A-1}) \oplus (b_0, b_1, \dots, b_{B-1}) \\ &= (a_0 + b_0, \dots, a_{A-1} + b_{B-1}) \\ &= (r_0, r_1, \dots, r_{A \cdot B - 1}). \end{aligned}$$

It is clear that $a_u + b_v$ results in element $r_{B \cdot u + v}$ of the resulting sequence. What is more interesting, however, is the inverse problem: what elements from a and b need to be summed to result in a value r_q ? In what follows, “div” denotes integer division: $a, b \in \mathbb{N}, b \neq 0 : a \text{ div } b = \lfloor \frac{a}{b} \rfloor$ and “mod” means taking the remainder after integer division: $a, b \in \mathbb{N}, b \neq 0 : a \text{ mod } b = a - (a \text{ div } b) \cdot b$. From number theory we know that

$$r_q = a_q \text{ div } B + b_{q \text{ mod } B}. \quad (4)$$

We can use these formulas to derive relations about the sequences A_n . For the purpose of the example in this paper, we are mainly interested in relations that describe properties of sequences formed by combining together n sequences with fixed length L , as in equation 3.

Before presenting a general formula we will examine a special case first. Consider the sequence of numbers $a = (a_0, a_1, \dots, a_{L-1})$, $b = (b_0, b_1, \dots, b_{L-1})$, $c = (c_0, c_1, \dots, c_{L-1})$, each with length L . We are interested in relating the elements in the sequence $(a \oplus b) \oplus c$ to the elements of a , b and c .

$$(a \oplus b) \oplus c = \alpha \oplus c = r$$

$$\begin{aligned} r &= \alpha \oplus c \\ (r_0, r_1, \dots, r_{L^3-1}) &= (\alpha_0, \alpha_1, \dots, \alpha_{L^2-1}) \oplus (c_0, c_1, \dots, c_{L-1}) \\ &= (\alpha_0 + c_0, \dots, \alpha_{L^2-1} + c_{L-1}) \end{aligned}$$

We know from equation 4 that

$$r_j = \alpha_{j \operatorname{div} L} + c_{j \operatorname{mod} L}. \quad (5)$$

We also know that

$$\begin{aligned} \alpha &= a \oplus b \\ (\alpha_0, \alpha_1, \dots, \alpha_{L^2-1}) &= (a_0, a_1, \dots, a_{L-1}) \oplus (b_0, b_1, \dots, b_{L-1}) \end{aligned}$$

Using equation 4 it follows that

$$\alpha_p = a_{p \operatorname{div} L} + b_{p \operatorname{mod} L}. \quad (6)$$

Combining equations 5 and 6 by setting $p = j \operatorname{div} L$, we get

$$r_j = a_{(j \operatorname{div} L) \operatorname{div} L} + b_{(j \operatorname{div} L) \operatorname{mod} L} + c_{j \operatorname{mod} L}. \quad (7)$$

Equation 7 describes how to calculate the j -th element of sequence r directly from the given sequences a , b and c .

In general for n sequences $s_0, s_1, \dots, s_{n-1}$ with length L , we get a system of equations similar to equations 5 and 6:

$$\begin{aligned} r_j &= \alpha_{j \operatorname{div} L} + s_{(n-1)_{j \operatorname{mod} L}} \\ \alpha_{j_2} &= \beta_{j_2 \operatorname{div} L} + s_{(n-2)_{j_2 \operatorname{mod} L}} \\ &\vdots \\ \mu_{j_{n-1}} &= \nu_{j_{n-1} \operatorname{div} L} + s_{1_{j_{n-1} \operatorname{mod} L}} \\ \nu_{j_n} &= s_{0_{j_n \operatorname{div} L}} + s_{1_{j_n \operatorname{mod} L}} \end{aligned} \quad (8)$$

The system 8 can be solved using back-substitution to get a general formula that relates the j -th component of the resulting sequence r to the appropriate

elements of each of the input sequences $s_0, s_1, \dots, s_{n-1}$. We now use from number theory that for $a, b, c \in \mathbb{N}$ and $b, c \neq 0$: $(j \operatorname{div} a) \operatorname{div} b = j \operatorname{div} (a \cdot b)$ to arrive at

$$\begin{aligned} r_j = & s_{0_{j \operatorname{div} L^{n-1}}} + s_{1_{(j \operatorname{div} L^{n-2}) \bmod L}} + \\ & \dots + s_{i_{(j \operatorname{div} L^{n-i-1}) \bmod L}} + \dots + \\ & s_{(n-2)_{(j \operatorname{div} L) \bmod L}} + s_{(n-1)_j \bmod L} . \end{aligned} \quad (9)$$

An interesting special case arises when L is a power of 2. Then the “div” and “mod” operations are implemented efficiently using bit-wise right shifting (denoted by operator $\gg$) and bit masking (denoted by operators $\&$ and $\mid$) respectively:

$$\begin{aligned} j \operatorname{div} 2^x &= j \gg x \\ j \bmod 2^x &= j \text{ and } (2^x - 1) = x \text{ least significant bits of } j \\ j \cdot 2^x &= j \ll x \\ j + 1 &= j \text{ or } 1, \text{ if } j \text{ even.} \end{aligned}$$

We now come back to the problem we try to solve. We apply equation 9 to equation 3, which models passing of arguments in the decoder algorithm. We know that $L = 4 = 2^2$. When the algorithm is called with parameter value n , n sequences are combined to form the sequence that describes how the function arguments are transformed when they arrive in the base case:

$$\begin{aligned} s_i &= (s_{i_0}, s_{i_1}, s_{i_2}, s_{i_3}) \\ &= \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 2^{n-i-1} \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 2^{n-i-1} \end{bmatrix}, \begin{bmatrix} 2^{n-i-1} \\ 2^{n-i-1} \end{bmatrix} \right) . \end{aligned}$$

We want to evaluate the expression

$$r = \bigoplus_{i=0}^{n-1} s_i .$$

Remember that the sequence r is the sequence of arguments that are recorded in the base case. Using the equations 9, we can find the j -th component of r . Because this is a closed formula, no recursion is needed anymore to implement the algorithm. In addition, no iteration dependencies exist between the calculation of some r_j and another r_k , $j \neq k$. It suffices to synthesize a loop that calculates the successive r_j values, and then uses them as arguments in calls to the base case calculation.

$$\begin{aligned} r_j = & s_{0_{(j \operatorname{div} 2^{2(n-1)})}} + s_{1_{(j \operatorname{div} 2^{2(n-2)}) \bmod 2^2}} + \\ & \dots + s_{i_{(j \operatorname{div} 2^{2(n-i-1)}) \bmod 2^2}} + \dots + \\ & s_{n-1_{j \bmod 2^2}} . \end{aligned} \quad (10)$$

We can further optimize the implementation of this formula using application specific knowledge. Each of the index expressions of the form $(j \operatorname{div} 2^{(x-i)}) \bmod 2^2$ yields a number from the set $\{0, 1, 2, 3\}$, by construction. Looking at the model equation 3, we see that whenever such expression yields a value of 0 or 2, the corresponding element in each of the s_{i_j} matrices for the x-argument is zero; when it is 1 or 3, the corresponding element in the s_{i_j} matrices is 2^{n-i-1} . This can be summarized as follows:

$$s_{i_j} = \begin{bmatrix} 2^{n-i-1} \cdot (j \bmod 2) \\ 2^{n-i-1} \cdot (j \operatorname{div} 2) \end{bmatrix}.$$

Therefore we can rewrite equation 10:

$$\begin{aligned} r_j = & \begin{bmatrix} 2^{n-1} \cdot ((j \operatorname{div} 2^{2(n-1)}) \bmod 2) \\ 2^{n-1} \cdot ((j \operatorname{div} 2^{2(n-1)}) \operatorname{div} 2) \end{bmatrix} + \\ & \begin{bmatrix} 2^{n-2} \cdot (((j \operatorname{div} 2^{2(n-2)}) \bmod 4) \bmod 2) \\ 2^{n-2} \cdot (((j \operatorname{div} 2^{2(n-2)}) \bmod 4) \operatorname{div} 2) \end{bmatrix} + \\ & \dots + \begin{bmatrix} 2^0 \cdot ((j \bmod 4) \bmod 2) \\ 2^0 \cdot ((j \bmod 4) \operatorname{div} 2) \end{bmatrix}. \end{aligned}$$

Using the facts that for $a, b, c \in \mathbb{N}, b, c \neq 0 : (a \operatorname{div} b) \operatorname{div} c = (a \operatorname{div} (b \cdot c))$, $(a \bmod (b \cdot c)) \bmod c = a \bmod c$ and $(a \bmod (b \cdot c)) \operatorname{div} b = (a \operatorname{div} b) \bmod c$:

$$r_j = \begin{bmatrix} \sum_{i=0}^{n-1} 2^i \cdot ((j \operatorname{div} 2^{2^i}) \bmod 2) \\ \sum_{i=0}^{n-1} 2^i \cdot ((j \operatorname{div} 2^{2^i+1}) \bmod 2) \end{bmatrix}.$$

For this example, the resulting formulas can be implemented efficiently using bit-shifting ($\ll, \gg$) and bit-masking (and, or) operations.

Part 2: Handling Code in between the Recursive Calls. As can be seen in figure 1 the recursive case contains extra checking functionality. Up to now we have ignored these statements. Now we can reinsert them. The conditions cause an extra difficulty: the Check function should not be activated in every iteration. The Check function must be triggered at the right moments in time because of its potential side-effects and their potential interaction with side-effects of the base case calculations.

In the recursive decoding algorithm example we use throughout this paper, we can reason as follows. As soon as a call to the Decode function happens with $n = 5$, (possibly as a result of an ongoing recursion) this³ results in 4 calls to the Decode function with $n = 4$, each separated by the amount of calls generated by a call to Decode with $n = 4$. From the reasoning presented in the calculation of the iteration upper bound (equation 1), we know this amount of

³ In practice, the maximum value of $n = 4$. But this information is usually unknown while transforming.

calls to be $4^4 = 256$. If the recursive Decode function was originally called with an argument $n > 4$, however, the iteration indexes at which $n > 4$ will also be multiples of 256. Testing if the iteration index is a multiple of 256 as a condition to execute the Check function is not enough. It now suffices to note (by the same reasoning) that the iteration indexes for which $n \geq 5$ will be multiples of 1024, because they will be separated by exactly $4^n, n \geq 5$ calls to the function Decode where argument $n \leq 4$. So the condition under which the Check function must be executed is given by

$$eq_n_4(i) = (((i \bmod 256) == 0) \&\& ((i \bmod 1024) != 0)) .$$

Again efficient implementation in this example is possible using bit-masking operators.

This step would have caused more difficulties if the Check function took parameters that depend on the values of the arguments being transformed by the recursion. It would mean that the function call to Check has to be executed at a specific moment during the calculation of r_j , in order to use the correct intermediate argument values. In the decoder example of this paper, the Check function call can be performed after completing the calculation of r_j , because the argument transforming functions have no side-effects beyond themselves, and because Check needs no intermediate argument values. Although not explained in this paper, we have already found solutions to handle such a more complex case.

Putting all results together, we come to Algorithm 2.

Algorithm 2 Final code after recursion removal from the visual texture decoder

```
// helper functions
void calc_offsets(int n, int j, int x, int y, int *x_ofs, int *y_ofs){
    int c; int tx=0,ty=0;
    // calculate the  $r_j$  values
    for (c=0;c<n;c++) {
        int ti=c<<1; int tip1=ti|1;
        tx=tx+(((j>>ti)&1)<<c); ty=ty+(((j>>tip1)&1)<<c);
    }
    *x_ofs = x+tx; *y_ofs = y+ty;
}

int eq_n_4(int i){ return (((i&255)&&(i&1023))); }
// iterative algorithm
Decode(int n, int x, int y) {
    // number of iterations needed
    int cnt = 1<<(n<<1);
    int i,x_ofs,y_ofs;
    for (i=0; i<cnt; i++) {
        calc_offsets(n,i,x,y,&x_ofs,&y_ofs);
        DecodePixel(x_ofs,y_ofs);
        if (eq_n_4(i))
            Check();
    }
}
```

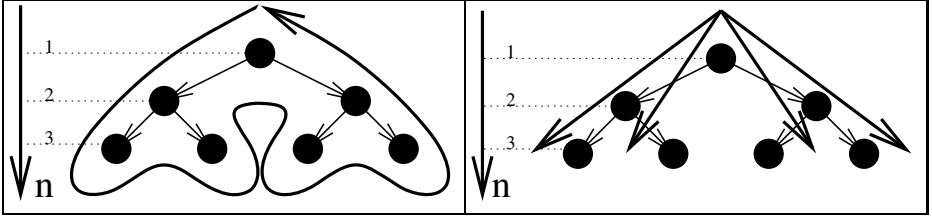


Fig. 4. Calculations in the recursive (left) vs. iterative (right) solution

5 Comparison Recursive and Iterative Version

This section will look at the differences between the way the recursive algorithm and the iterative algorithm works. It will become clear that the iterative algorithm uses less memory at the expense of performing more calculations. We will touch upon how we plan to make a systematic trade-off between memory usage and amount of calculations by introducing partial memoization.

5.1 Recursive Version

The recursive version of the algorithm performs $\frac{B^n - 1}{B - 1} - 1$ argument transforming steps, and the same amount of recursive function calls. B is the amount of branches when going one level deeper in the tree, n is the depth of the tree.

In figure 4, $B = 2$, $n = 3$, resulting in 6 argument transformation steps. In the MPEG4 VTC decoder example $B = 4$, n varies between 0 and 4, resulting in 0, 4, 20, 84, 340 transforming steps respectively (and the same amount of function calls). The implementation of recursion using stack-frames causes implicit memoization on the argument transformation calculations. Indeed, on back-tracking the transformed arguments are still available in the stack-frame.

One argument transformation step in the recursive version of the VTC decoder costs approximately one function call, and one or two additions and sometimes a bit-shift operation.

The memory required for holding stack-frames in the recursive version equals $S \cdot d$ where S is the size of the stack-frame, and d the maximal depth of the call tree. For the VTC decoder this depth is at most 5. The stack-frame size S is 48 bytes.

5.2 Iterative Version

The iterative version of the algorithm redoes many argument transformation calculations many times making it inefficient in terms of number of calculations. The number of argument transforming steps is $(n - 1) \cdot (B^{n-1})$, where B is the amount of branches when going one level deeper in the call tree, n is the depth of the tree.

In figure 4, $B = 2$, $n = 3$, resulting in 8 argument transformation steps. In the MPEG4 VTC decoder $B = 4$, n varies between 0 and 4, resulting in 0, 4, 32, 192, 1024 argument transforming steps respectively.

One argument transformation step in the iterative version of the VTC decoder costs approximately 5 bit-shifts, 3 bit-masks and 2 additions.

Because the recursion is removed, no time or memory is required for setting up and keeping stack-frames.

5.3 Conclusions

The net result in the VTC decoder, after profiling, appears to be that the unparallelized iterative version runs about as fast as the recursive function, but uses less memory. This will typically result in a more energy-efficient implementation [3]. In [14] it is explained how the recursion removal described here enables further task-level transformations of the VTC code to get systematic Pareto-optimal energy consumption versus timing budget trade-offs.

The amount of calculations that needs to be done in the iterative version of the algorithm increases a lot compared to the recursive version. The reason for this is that the recursive version implicitly provides memoization of the calculations which transform the arguments, whereas in the iterative version each of these calculations is redone every time (see Figure 4).

We are currently investigating the possibility of reintroducing (partial) memoization in the iterative version of the algorithm. This should allow for getting systematic trade-offs between memory consumption and amount of calculations, while keeping opportunities for parallelization. It is clear that the transformed arguments which will be stored in the memoization table are those which are needed most often, i.e. the ones that are produced near the top of the call tree. Memoization will be useful only if looking up the values from memory costs less (e.g. in terms of energy) than recalculating them every time. Part of our future work will be to measure and quantify these effects, as well as the effect on the resulting energy consumption.

6 Discussion and Future Work

After step one is completed, the recursion is separated from the calculations which, by assumption, dominate the execution time. While this can already enable parallelization of the iterative loop, it is not a good solution in terms of memory requirements.

After step two, the recursion is completely removed, and we have an equivalent iterative algorithm. Because of the exact equivalence in terms of order of activation and argument values to the functions, we can safely ignore many side-effects which might be caused inside the base case calculations or inside additional code in between the recursive functions.

The only side-effects we assume not to happen are those side-effects that influence the call value graph. An example of such a side-effect in our example decoder algorithm would be that the base case calculation somehow finds out

the address of one or more arguments and changes their value. Because of such side-effect the iterative and recursive algorithm might no longer be equivalent. This assumption seems rather reasonable in practice, however.

Parallelization which can happen after separation or removal of the recursion will have to look at any side-effects inside the function calls, as they can cause iteration dependencies which are not visible in the VTC algorithm.

Currently better formalization and extensions of this method are being developed to explore what exactly the assumptions are that need to be fulfilled to remove recursion using a method similar to one presented here, and to be able to integrate the techniques with a parser front-end. We are looking into partial memoization of the iterative version of the algorithm as a way to trade-off memory cost and amount of calculations. Also measures are being developed to estimate the effects on parallelization and evaluate trade-offs in cost factors such as execution time, code size and energy. Finally, we plan to perform a series of experiments using a multiprocessor platform simulator to accurately measure the effects of the transformations on the cost factors execution time, energy consumption, and code size.

References

1. JTC1/SC29/WG11/N4668, I.: Overview of the mpeg-4 standard (2002) Ed.: Rob Koenen.
2. JTC1/SC29/WG11/N5231, I.: Mpeg 21 overview v.5 (2002) Eds.: Jan Bormans and Keith Hill.
3. Catthoor, F., Wuytack, S., Greef, E.D., Balasa, F., Nachtergaele, L., Vandecappelle, A., de Greef, E., Wuytack, S.: Custom Memory Management Methodology: Exploration of Memory Organisation for Embedded Multimedia System Design. 1st edn. Kluwer Academic Publishers (1998)
4. Abelson, H., Sussman, G.J., Sussman, J.: Structure and Interpretation of Computer Programs. 2nd edn. MIT Press, ISBN 0-26201-153-0 (1996) text also online at <http://mitpress.mit.edu/sicp>.
5. Bauer, F.L., Wössner, H.: Algorithmic Language and Program Development. Texts and Monographs in Computer Science. Springer Verlag, ISBN 0-387-11148-4 (1982)
6. Partsch, H.A.: Specification and Transformation of Programs: A Formal Approach to Software Development. Texts and monographs in Computer Science. Springer Verlag, ISBN 0-38752-356-1 (1990)
7. Kaser, O., Pawagi, S., Ramakrishnan, C.R.: On the conversion of indirect to direct recursion. ACM Letters on Programming Languages and Systems (LOPLAS) **2** (1993) 151–164
8. McCarthy, J., Abrahams, P.W., Edwards, D.J., Hart, T.P., Levin, M.I.: LISP 1.5 Programmer's Manual. MIT Press, Cambridge, Massachusetts (1962)
9. Backus, J.: Can programming be liberated from the von neumann style? A functional style and its algebra of programs. Communications of the ACM **21** (1978) 613–641 ISSN: 0001-0782.
10. Backus, J.: From function level semantics to program transformation and optimization. In: Proceedings of the international joint conference on theory and practice of software development (TAPSOFT). Volume 1: Colloquium on trees in algebra and programming (CAAP'85). Berlin, Germany (1985) 60–91

11. Collard, J.F.: Reasoning about Program Transformations: Imperative programming and flow of data. Springer Verlag ISBN 0-387-95391-4 (2003)
12. Sabry, A.: What is a purely functional language? *The Journal of Functional Programming* **8** (1998) 1–22
13. Liu, Y.A., Stoller, S.D.: From recursion to iteration: What are the optimizations? In: *Proceedings of the 2000 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-Based Program Manipulation (PEPM 2000)*. Volume 34 of *SIGPLAN Notices.*, Boston, Massachusetts, USA, ACM Press, ISBN 1-58113-201-8 (1999) 73–82
14. Ma, Z., Wong, C., Himpe, S., Delfosse, E., Catthoor, F., Deconinck, G.: Task concurrency analysis and exploration of visual texture decoder on a heterogeneous platform. In: *Proceedings of the 2003 IEEE WORKSHOP ON SIGNAL PROCESSING SYSTEMS (SiPS 2003)*, Seoul, South Korea (2003)

An Unfolding-Based Loop Optimization Technique

Litong Song¹, Krishna Kavi¹, and Ron Cytron²

¹ Department of Computer Science
University of North Texas, Denton, Texas, 76203, USA
{slt,kavi}@cs.unt.edu

² Department of Computer Science and Engineering
Washington University, St. Louis, MO 63130, USA
{cytron}@cs.wustl.edu

Abstract. Loops in programs are the source of many optimizations for improving program performance, particularly on modern high-performance architectures as well as vector and multithreaded systems. Techniques such as loop invariant code motion, loop unrolling and loop peeling have demonstrated their utility in compiler optimizations. However, many of these techniques can only be used in very limited cases when the loops are “well-structured” and easy to analyze. For instance, loop invariant code motion works only when invariant code is inside loops; loop unrolling and loop peeling work effectively when the array references are either constants or affine functions of index variable. It is our contention that there are many opportunities overlooked by limiting the optimizations to well structured loops. In many cases, even “badly-structured” loops may be transformed into well structured loops. As a case in point, we show how some loop-dependent code can be transformed into loop-invariant code by transforming the loops. Our technique described in this paper relies on unfolding the loop for several initial iterations such that more opportunities may be exposed for many other existing compiler optimization techniques such as loop invariant code motion, loop peeling, loop unrolling, and so on.

1 Introduction

Loops in programs are the source of many optimizations for improving program performance, particularly on modern high-performance architectures as well as vector and multithreaded systems. Techniques such as loop invariant code motion, loop peeling and loop unrolling have demonstrated their utility among in compiler optimizations. However, many of these techniques can only be used in very limited cases when the loops are “well-structured” and easy to analyze. For instance, loop invariant code motion works only when invariant code is inside loops; loop unrolling and loop peeling work effectively when the loop indices and array references are either constant or affine functions. Let us first give a brief review on a few common loop optimization techniques such as loop invariant code motion, loop unrolling and loop peeling, and discuss the limitations of these techniques.

1.1 Reviews of a Few Loop Optimization Techniques

Loop invariant code motion is a well-known loop transformation technique. When a computation in a loop does not change during the dynamic execution of the loop, we can hoist this computation out of the loop to improve execution time performance. For instance, the evaluation of expression $a \times 100$ is loop invariant in Fig. 1(a); Fig. 1(b) shows a more efficient version of the loop where the loop invariant code has been removed from the loop.

<pre>for (i = 1; i <= 100; i++) { x = a × 100; y = y + i; }</pre> (a) A source loop <pre>t = a × 100; for (i = 1; i <= 100; i++) { x = t; y = y + i; }</pre> (b) The resulting code

Fig. 1. An example for loop invariant code motion

Modern computer systems exploit both instruction level parallelism (ILP) and thread (or task) level parallelism (TLP). Superscalar and VLIW systems rely on ILP while multi-threaded and multiprocessor systems rely on TLP. In order to fully benefit from ILP or TLP, compilers must perform complex analyses to identify and schedule code for the architecture. Typically compilers focus on loops for finding parallelism in programs [26], [27]. Sometimes it is necessary to rewrite (or reformat) loops such that loop iterations become independent of each other, permitting parallelism. Loop peeling is one such technique [3], [15], [21]. When a loop is peeled, a small number of early iterations are removed from the loop body and executed separately. The main purpose of this technique is for removing dependencies created by the early iterations on the remaining iterations, thereby enabling parallelization.

The loop in Fig. 2(a) is not parallelizable because of a flow dependence between iteration $i = 1$ and iterations $i = 2 \dots n$. Peeling the first iteration makes the remaining iterations fully parallel, as shown in Fig. 2(b). Using vector notation, the loop in Fig. 2(b) can be rewritten as: $a(2:n) = a(1) + b(2:n)$. That is to say, $n - 1$ assignments in $n - 1$ iterations of the loop can be executed in parallel.

<pre>for (i = 1; i <= n; i++) { a[i] = a[1] + b[i]; }</pre> (a) A source loop <pre>if (1 <= n) { a[1] = a[1] + b[1]; }</pre> <pre>for (i = 2; i <= n; i++) { a[i] = a[1] + b[i]; }</pre> (b) The resulting code after peeling first iteration
--

Fig. 2. The first example for loop peeling

The loop in Fig. 3(a) is not parallelizable because variable *wrap* is neither a constant nor a linear function of inductive and index variable i . Peeling off the first iteration allows the rest of loop to be vectorizable, as shown in Fig. 3(b). The loop in Fig. 3(b) can be rewritten as: $a(2:n) = b(2:n) + b(1:n-1)$.

Loop unrolling is a technique, which replicates the body of a loop a number of times called the unrolling factor u and iterates by step u instead of step 1. It is a fundamental technique for generating efficient instructions required to exploit ILP and TLP. Loop unrolling can improve the performance by (i) reducing loop overhead; (ii) increasing instruction level parallelism; (iii) improving register, data cache, or TLB locality. Fig. 4 shows an example of loop unrolling. Loop overhead is cut in a second because one additional iteration is performed before the test and branch at the end of the loop. Instruction parallelism is increased because the first and second assignments can be executed on pipeline. If array elements are assigned to registers, register locality will improve because $a[i]$ is used twice in the loop body, reducing the number of loads per iteration.

<pre> for (i = 1; i <= n; i++) { a[i] = b[i] + b[wrap]; wrap = i; } (a) A source loop if (1 <= n) { a[1] = b[1] + b[wrap]; wrap = i; } for (i = 2; i <= n; i++) { a[i] = b[i] + b[i-1]; } (b) The resulting code after peeling first iteration </pre>

Fig. 3. The second example for loop peeling

<pre> for (i = 2; i <= n; i++) { a[i] = a[i-2] + b[i]; } (a) A source loop for (i = 2; i <= n-1; i = i+2) { a[i] = a[i-2] + b[i]; a[i+1] = a[i-1] + b[i+1]; } if (mod(n-2, 2) == 1) { a[n] = a[n-2] + b[n]; } (b) The resulting code after loop unrolling </pre>

Fig. 4. An example of loop unrolling

1.2 Issues

As we mentioned previously, loop invariant code motion, loop peeling and loop unrolling are all very practical and important compiler optimization techniques for today's architectures. Nevertheless, these techniques are only suitable for well-structured loops, which are relatively easy to analyze. For loop invariant code motion, it works only when there are clearly and easily identifiable invariant code inside loops; for loop unrolling and loop peeling, they usually work when subscripts of array references are constants or affine functions. In many practical programs, loops are not well-structured; but in some cases, these loops may be quasi well-structured ones. That is to say, they may be converted into well-structured. For instance, in the loop of Fig. 5(a), there is only one invariant expression $b \times c$. If we unfold the loop twice, however, we can get the resulting code in Fig. 5(b), which is much more efficient than the source loop. This is because: (i) variables x and y become invariant variables in the resulting loop, so that assignments $x = y + a$ and $y = b \times c$ can be removed from the remaining loop; (ii) expression $x \times y$ and $x > d$ are invariant expressions in the remaining loop so they can be hoisted outside the remaining loop, which can actually be done by the conventional loop invariant code motion; (iii) because expression $x > d$ is in-

variant during the dynamic execution of the remaining loop, it will improve the branch predication and significantly decrease branch misses of the conditional contained in the remaining loop. This example shows that an effective transformation of badly structured loops is possible and desirable.

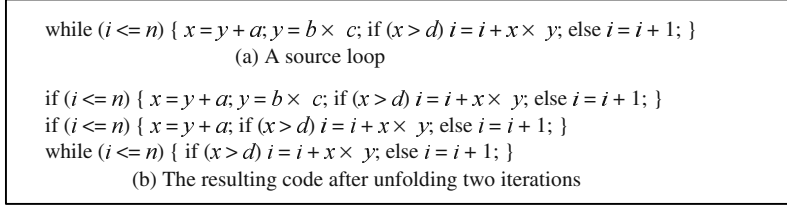


Fig. 5. Loop quasi-invariant code motion

For the loop in Fig. 6(a), in the two assignments $a[i] = b[i] + b[j]$ and $c[i] = c[j] \times b[i]$, j and $wrap$ are not constants or affine functions of index variable i , so we have no way to directly parallelize any of them, and we can not even unroll the loop since we do not know what is going on for loop-carried dependences. If peeling or unfolding the loop for two iterations, however, the remaining loop in Fig. 6(b) is very suitable for parallelization and loop unrolling. Statement $a[i] = b[i] + b[i-2]$ can be parallelized to be $a[3:n] = b[3:n] + b[1:n-2]$, and statement $c[i] = c[i-2] \times b[i]$ can be unrolled to be $c[i] = c[i-2] \times b[i]$; $c[i+1] = c[i-1] \times b[i+1]$; such that the two statements can be executed in parallel since there is no loop-carried dependence among them. Thus, some pre-optimizations or transformations based on loop unfolding may be very useful and lead to the application of conventional compiler optimization techniques.

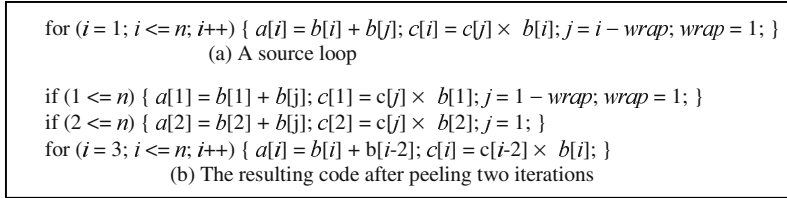


Fig. 6. An example for loop peeling and loop unrolling

In this paper we present a technique that is based on loop dependence analysis, so that traditional optimization techniques can benefit from it. In particular, our goal is to find a general and systematic way for pre-optimizations of using loop unfolding to remove anti-dependences as much as possible.

2 Preliminaries

This section provides the background necessary for the rest of the paper, including a simple language we will use to describe our loop optimization technique and the well-known static single assignment (SSA) form.

2.1 DO-Language

For the purpose of describing our technique, we first introduce a simple imperative language, shown in Fig. 7; the semantics is similar to C. For the sake of simplifying the presentation, we assume a call-by-value semantics for function parameters, assume freedom of side effects, and we treat all functions as primitive operations.

<i>Sts</i>	::= <i>St</i> <i>St</i> ; <i>Sts</i>
<i>St</i>	::= <i>Ass</i> <i>Cond</i> <i>Loop</i> <i>Call</i>
<i>Ass</i>	::= <i>Var</i> = <i>Exp</i>
<i>Cond</i>	::= if (<i>Exp</i>) { <i>Sts</i> } else { <i>Sts</i> }
<i>Loop</i>	::= for (<i>Var</i> = <i>Exp</i> ; <i>Exp</i> ; <i>Var</i> = <i>Var</i> + <i>Exp</i>) { <i>Sts</i> } while (<i>Exp</i>) do { <i>Sts</i> }
<i>Call</i>	::= <i>f</i> (<i>Exp</i> *)
<i>Exp</i>	::= <i>Var</i> <i>Const</i> <i>Op</i> (<i>Exp</i> *) <i>Call</i>
<i>Op</i>	::= + - × / > < <= >= = !

Fig. 7. The syntax of the DO-language

2.2 Static Single Assignment

Variables inside a loop may be modified for multiple times. In order to perform dependency analyses, it is necessary to distinguish the modifications. Here, we make use of the well-known static single assignment (SSA) [10] for this purpose. SSA form is a program representation in which every variable is assigned only once, and every use of the variable is defined by that assignment. Most compilers use SSA representations for performing optimizations. Here we use the term to refer to variables as ϕ -variables assigned by ϕ -function. An efficient algorithm that converts a program into SSA form with linear time complexity (in term of the size of the original program) was presented in [9].

$\begin{array}{l} x = \dots; \dots = x; \\ x = \dots; \dots = x; \end{array} \quad \Rightarrow \quad \begin{array}{l} x_1 = \dots; \dots = x_1; \\ x_2 = \dots; \dots = x_2; \end{array}$
(a) straight-line code and its SSA form
$\text{if } (test) \{ x = \dots; \} \text{ else } \{ x = \dots; \} \quad \Rightarrow \quad \text{if } (test) \{ x_1 = \dots; \} \text{ else } \{ x_2 = \dots; \} x_3 = \phi(x_1, x_2);$
(b) conditional and its SSA form

Fig. 8. SSA form transformation

3 Quasi-invariant and Quasi-index Variables

The invariant variables of a loop are those variables whose values are invariant in all the iterations of the loop. The index variable of a loop is a variable whose values in successive iterations form an arithmetic progression. Index variables are often used in array subscripts. Here, we present four notions:

- **Quasi-invariant variable.** A variable that is not invariant inside a loop but will become invariant after a small number of iterations of the loop.
- **Quasi-index variable.** A variable that is not an index variable but will become equal to an affine function of the index variable after a small number of iterations of the loop.
- **Unfolding factor of quasi-invariant variable.** If a quasi invariant variable becomes invariant after at least n iterations of a loop, n is referred to as the unfolding factor of the variable.
- **Unfolding factor of quasi-index variable.** If a quasi index variable becomes an affine function of the index variable after at least n iterations of a loop, n is referred to as the unfolding factor of the variable.

For instance, in Fig. 5, x and y are quasi invariant variables, and their unfolding factors are 2 and 1, respectively; in Fig. 6, $wrap$ is a quasi invariant variable but j is a quasi index variable, and their unfolding factors are 1 and 2, respectively. Now, we face two issues: (i) identifying quasi invariant and quasi index variables; (ii) calculating the unfolding factors of these variables.

4 Variable Dependences

Compiler usually relies on both control and data dependence analyses for performing optimizations [5], [27]. These dependencies relate to those among statements. In our case, we only rely on dependencies among variables. We recognize two forms of data dependences: true data dependence, anti-data dependence, and two forms of control dependences: true control dependence, anti-control dependence.

- **True data dependence.** The first statement stores into variable x that is later read by the second statement:

$$S_1: x = \dots; \quad S_2: y = \dots x;$$
We say y has a true data dependence to x , and denote the dependence as $y \delta_d x$.
 - **Anti-data dependence.** The first statement reads x into which the second statement later stores:

$$S_1: y = \dots x; \quad S_2: x = \dots;$$
We say y has an anti-data dependence to x , and denote the dependence as $y \delta_d^- x$.
 - **True control dependence.** The first statement stores into variable x that is later read by the test of second statement (conditional):

$$S_1: x = \dots; \quad S_2: \text{if } (\dots x) y = \dots; \text{ else } y = \dots;$$
 - **Anti-control dependence.** The test of first statement (conditional) reads x into which the second statement later stores:

$$S_1: \text{if } (\dots x) y = \dots \text{ else } y = \dots; \quad S_2: x = \dots;$$
We say y has an anti-control dependence to x and denote it as $y \delta_c^- x$.
- According to the definitions above, the variable dependences in Fig. 5(a) and Fig. 6(a) should be: $x \delta_d^- y$, $x \delta_c i$, $i \delta_d i$, $j \delta_d wrap$, $j \delta_d i$, $i \delta_d i$
- Note that we only discuss the dependences between scalar variables here.

5 An Extension of Control Dependences

In Sect. 4 we presented two general notions for control dependences. In this section, we present special cases of conditionals to elaborate on control dependences. Variable assignments inside conditionals can be distinguished into two cases:

- *A variable is assigned inside both then-part and else-part of a conditional:*

for ($i = 1; i \leq n; i++$) { if ($test$) { $x_1 = e_1$; } else { $x_2 = e_2$; } $x_3 = \phi(x_1, x_2)$; }

The assignment to a quasi invariant variable can be removed after the variable becomes invariant, and the symbolic value (an affine function) of a quasi index variable might be substituted for references to the variable after it is equal to an affine function. Whether $x_1 = e_1$ or $x_2 = e_2$ can be removed or not, is dependent on not only e_1 or e_2 but also $test$. If $test$ is variant then neither $x_1 = e_1$ nor $x_2 = e_2$ can be removed even if e_1 or e_2 may be invariant. Otherwise, x_3 might be assigned to an incorrect value. By contrast, if $test$ is invariant then either $x_1 = e_1$ or $x_2 = e_2$ can be removed as long as e_1 or e_2 is invariant. This is because the selection of the value of x_3 is invariant inside the remaining loop.

- *A variable is assigned both inside one branch of a conditional and outside the conditional:*

for ($i = 1; i \leq n; i++$) { $x_1 = e_1$; if ($test$) { $x_2 = e_2$; } $x_3 = \phi(x_1, x_2)$; }

Similar to case 1, both x_1 and x_2 are control dependent on $test$. In addition, we distinguish between two cases as below:

- *There exist references to x_1 .*

Because the value of $test$ is unknown, $x_1 = e_1$ can not be removed even if the $test$ is invariant. Note that x_1 , x_2 and x_3 will be renamed to be a same name in resulting program, which will be described in Sect. 8. Accordingly, $x_2 = e_2$ can not be removed either. If x_1 and x_2 are ϕ -variables, their operands can not be removed either, and thus a recursive processing is needed to determine which assignments can not be removed from the resulting loop. Assuming that we use γ to denote the closure of this kind of variables, and σ to denote the variables already handled, γ will be defined as follows:

$$\gamma(x) = \begin{cases} \sigma & \text{if } x \in \sigma \\ \sigma \cup \{x\} & \text{if } x \notin \sigma \wedge x \notin \phi\text{-variables} \\ \gamma(x_1)_{\sigma \cup \{x\}} \cup \gamma(x_2)_{\sigma \cup \{x\}} & \text{if } x \notin \sigma \wedge x = \phi(x_1, x_2) \end{cases}$$

- *There exists no reference to x_1 .*

Because $x_1 = e_1$ is outside the conditional, $x_2 = e_2$ can be removed only when assignment $x_1 = e_1$ is removed (otherwise x_3 will be always equal to x_1). The special dependence between x_1 and x_2 is actually an *ad hoc* true control dependence, which is still denoted by $x_2 \delta_c x_1$.

After the analysis of control dependences, we need to collect all the related dependences introduced by ϕ -functions. A ϕ -function is temporarily introduced only for static analysis and it will be removed in resulting programs, so any control dependence introduced by a ϕ -variable is actually a dependence introduced by the operand variables of the ϕ -variable. This is a recursive process and a closure should be computed. Assuming that there exists a control dependence denoted as $x_1 \delta_c x_2$, function ϕ is used to

denote the closure, and σ is used to denote the dependences already handled, function ϕ can be defined as follows:

$$\phi(x)_\sigma = \begin{cases} \sigma & \text{if } x \delta_c y \wedge (x \delta_c y) \in \sigma \\ \sigma \cup \{(x \delta_c y)\} & \text{if } x \delta_c y \wedge (x \delta_c y) \notin \sigma \wedge y \notin \phi\text{-variables} \\ \phi(x, y_1)_{\sigma \cup \{(x \delta_c y)\}} \cup \phi(x, y_2)_{\sigma \cup \{(x \delta_c y)\}} & \text{if } x \delta_c y \wedge (x \delta_c y) \notin \sigma \wedge y = \phi(y_1, y_2) \end{cases}$$

For instance, suppose we have the following program segment inside a loop:

$x_1 = 1$; if ($i > j$) { if ($k > 5$) { $x_2 = 2$; } else { $x_3 = 3$; } $x_4 = \phi(x_2, x_3)$; } $x_5 = \phi(x_1, x_4)$;

We can compute the following dependences: $x_1 \delta_c i, x_1 \delta_c j, x_2 \delta_c i, x_2 \delta_c j, x_2 \delta_c k, x_2 \delta_c x_1, x_3 \delta_c i, x_3 \delta_c j, x_3 \delta_c k, x_3 \delta_c x_1, x_4 \delta_c i, x_4 \delta_c j, x_4 \delta_c x_1$

6 Dependence Relation Graph

Based on the two types of data dependences and two types of control dependences, we can construct a directed graph called dependence relation graph.

Definition 1 (Dependence Relation Graph). The dependence relation graph (DRG) of a loop is a directed graph (V, E) , where

$V = \{ x \mid x \text{ is a variable modified inside the loop} \};$

$E = \{ \text{a directed real thin line from } x \text{ to } y \mid y \delta_d x \} \cup \{ \text{a directed real bold line from } x \text{ to } y \mid y \delta_c x \} \cup \{ \text{a directed dotted thin line from } x \text{ to } y \mid y \delta_d^- x \} \cup \{ \text{a directed dotted bold line from } x \text{ to } y \mid y \delta_c^- x \}$

```
for (i = 1; i <= n; i++) {
  a[i] = p[x] + q[y+k];
  if (odd(i)) { w = i - 1; b[i] = b[w] + c[z]; } else { w = i; k = d; b[i] = b[w] + c[z]; }
  t = j + z; z = 2; x = y; y = i + 1;
}
```

(a) A source loop

```
for [ x1 = φ(x0, x2); t1 = φ(t0, t2); z1 = φ(z0, z2); y1 = φ(y0, y2); k1 = φ(k0, k3); w1 = φ(w0, w4); ]
(i = 1; i <= n; i++) {
  a[i] = p[x1] + q[y1+k1];
  if (odd(t1)) { w2 = i - 1; b[i] = b[w2] + c[z1]; } else { w3 = i; k2 = d; b[i] = b[w3] + c[z1]; }
  w4 = φ(w2, w3); k3 = φ(k1, k2); t2 = j + z1; z2 = 2; x2 = y1; y2 = i + 1;
}
```

(b) The corresponding SSA form

Fig. 9. An example for SSA form conversion

For instance, assuming that we have a program segment shown in Fig. 9, the DRG for this program is shown in Fig. 10. Here, the semantics of loop for [Sts] ($Var = Exp$; Exp ; $Var = Var + Exp$) { Sts } means that statements in [Sts] will be executed before the evaluation of loop test. Note that this intermediate form is only used for static analysis and it will be converted back to original form after optimization.

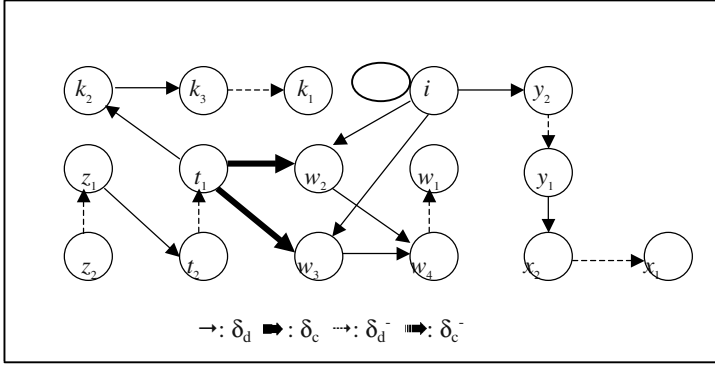


Fig. 10. The DRG of the loop in Fig. 9

7 Identifying Quasi-invariant/index Variables and Computing their Unfolding Factors

In Sect. 3 we defined quasi invariant variables, quasi index variables and their unfolding factors. Using dependence relation graphs, we can identify quasi invariant variables and quasi index variables, and efficiently compute their unfolding factors.

7.1 Quasi-invariant Variables and Unfolding Factors

- **Quasi-invariant variable.** For any vertex on the DRG of a loop, if among all the paths ending in this vertex, there is no path that contains a vertex that is a vertex on a strongly connected path, then the variable corresponding to the vertex is a quasi invariant variable.
- **Unfolding factor of quasi-invariant variable.** For any quasi invariant variable x on a DRG, the unfolding factor of x is equal to $\max \{ n \mid n = \text{the number of dependence } \delta_d^- \text{ edges (represented by directed thin dotted line) and dependence } \delta_c^- \text{ edges (represented by directed bold dotted line) on a path ending in } x \}$.

For instance, in Fig. 10, t_1 , t_2 , z_1 , z_2 , k_1 , k_2 and k_3 are all quasi invariant variables, but the other variables are not because each of them is on a path which contains a strongly connected graph. Because there is a path ending in quasi invariant variable t_1 and this path contains two (maximum) directed thin dotted lines, the unfolding factor of t_1 is 2. In the same way, the unfolding factors of quasi invariant variables t_2 , z_1 , z_2 , k_1 , k_2 and k_3 are 1, 1, 0, 3, 2 and 2, respectively.

7.2 Quasi-index Variables and Unfolding Factors

For any variable assigned inside a loop, it must be either a quasi invariant variable or a variant variable. We can further distinguish three types of variant variables: (i) index

variables; (ii) quasi index variables; (iii) others. Identification of index variables has been studied by many others, thus we assume here that index variables have been identified. Our goal is to identify quasi index variables. Within a loop, if the test of a conditional is variant, then all variables assigned inside the branches of the conditional are not quasi index variables, since any reference to a quasi index variable can be replaced by an affine function of index variable after a small number of loop iterations.

- **Quasi-index variable.** *For any variant variable (non-invariant variable and non-quasi-invariant variable) x on the DRG of a loop, if any path ending in the vertex of x contains, only vertexes of index, quasi index or quasi invariant variables, and contains neither δ_c^- dependence edges nor δ_c^+ dependence edges that starts from a vertex of variant variable, then x is a quasi index variable.*
- **Unfolding factor of quasi-index variable.** *For any quasi index variable x on a DRG, the unfolding factor of x is equal to $\max\{n \mid n = \text{the number of } \delta_a^- \text{ edges (represented by directed thin dotted line) and } \delta_c^- \text{ edges (represented by directed bold dotted line) on a path that ends in } x \text{ and contains no strongly connected graph.}\}$.*

For instance, in Fig. 10, $y_1, y_2, x_1, x_2, w_1, w_2, w_3$ and w_4 are quasi index variables, and their unfolding factors are 1, 0, 2, 1, 3, 2, 2 and 2, respectively.

8 Algorithms of Evaluating Quasi-invariant/index Variables and Unfolding Factors

In this section, we present efficient algorithms for identifying quasi invariant/index variables and computing their unfolding factors. The main work of this paper is divided into two phases: 1. Quasi invariance/index analysis that includes (i) detecting dependences among variables and (ii) identifying quasi invariant/index variables and computing their unfolding factors; 2. Loop unfolding. We already discussed how to detect dependences among variables. Based on the dependences, we present two efficient algorithms to identify quasi invariant/index variables and to compute their unfolding factors. **Alg. 1** is based on the well-known algorithm presented by Warshall [24]. The time complexities of Warshall algorithm is $O(n^3)$ in the worst case, where n is the number of the variables modified inside a given loop. Assume that there are n variables $x_1 \dots x_n$ modified inside a given loop, and five Boolean $n \times n$ matrices $\Phi_{\delta_a^+}, \Phi_{\delta_a^-}, \Phi_{\delta_c^+}, \Phi_{\delta_c^-}$ indicating $\delta_a^+, \delta_a^-, \delta_c^+, \delta_c^-$ dependence relations among these variables, respectively. $\Phi = \Phi_{\delta_a^+} \vee \Phi_{\delta_a^-} \vee \Phi_{\delta_c^+} \vee \Phi_{\delta_c^-}$. Here, for any two variables x_i and x_j , we have:

$$\Phi_{\delta_a^+}(i, j) = \begin{cases} 1, & \text{if } x_i \delta_a^+ x_j \\ 0, & \text{otherwise} \end{cases} \quad \Phi_{\delta_c^+}(i, j) = \begin{cases} 1, & \text{if } x_i \delta_c^+ x_j \\ 0, & \text{otherwise} \end{cases}$$

$$\Phi_{\delta_a^-}(i, j) = \begin{cases} 1, & \text{if } x_i \delta_a^- x_j \\ 0, & \text{otherwise} \end{cases} \quad \Phi_{\delta_c^-}(i, j) = \begin{cases} 1, & \text{if } x_i \delta_c^- x_j \\ 0, & \text{otherwise} \end{cases}$$

Moreover, suppose Ix denotes the set of index variables, Qiv denotes the set of quasi invariant variables and Qix denotes the set of quasi index variables.

Alg. 1 (*identifying quasi invariant vs index variables*)

Input: Φ, Ix

Output: Qiv, Qix

Begin

for ($i = 1; i \leq n; i++$)

for ($j = 1; j \leq n; j++$)

if ($\Phi(j, i)$) for ($k = 1; k \leq n; k++$) { $\Phi(j, k) = \Phi(j, k) \vee \Phi(i, k);$ }

$Qiv = \{x_i \mid \forall i (1 \leq i \leq n) \forall j (1 \leq j \leq n) \bullet (\Phi(i, j) \rightarrow \neg \Phi(j, i))\};$

$Qix = \{x_i \mid \forall i (1 \leq i \leq n) \bullet (x_i \notin Qiv \wedge \forall j (1 \leq j \leq n) \bullet (\Phi(i, j) \rightarrow \neg \Phi(j, i) \vee (\Phi(j, j) \wedge x_j \in Ix)))\};$

End

The worst case time complexity of **Alg. 1** is $O(n^3)$. Note that Ix is a subset of set Qix . While computing the unfolding factors of quasi invariant/index variables, we can exploit the well-known algorithm of Floyd[13] for computing the shortest distance between a pair of vertexes. Because the main focus of computing unfolding factors is anti-dependences, we suppose the length of each anti-dependence edge to be 1 and that of each true dependence edge to be 0. Floyd's algorithm was originally used to compute the shortest path between a pair of vertexes on a directed graph, but we need to compute the longest path here. If a directed graph does not contain any strongly connected subgraphs, then essentially there will be no difference between computing shortest and longest paths between a pair of vertexes when using Floyd's algorithm. If we delete all the edges starting from or ending in index variable, then all the paths ending in a quasi index variable should not contain any strongly connected graph. In addition to the variables used in **Alg. 1**, we utilize two additional integer $n \times n$ matrices $\wp_{IV}$ and $\wp_{IX}$ defined as: $\wp_{IV} = \wp_{IX} = \Phi_{\delta_d} \vee \Phi_{\delta_c}$. $\omega(x)$ indicates the unfolding factor of variable x . Alg. 2 is a variation of Floyd's algorithm, its worst-case time complexity is $O(n^3)$.

Alg. 2 (*computing the unfolding factors of quasi invariant vs index variables*)

Input: $\Phi, Ix, Qiv, Qix, \wp_{IV}, \wp_{IX}$

Output: ω

Begin

//Computing the unfolding factors of quasi invariant variables.

for any $x_i \in Qiv$

for any $x_j \in Qiv$

for any $x_k \in Qiv$

if ($\Phi(j, i) \wedge \Phi(i, k) \wedge \Phi(j, k)$)

if ($(\wp_{IV}(j, k) < \wp_{IV}(j, i) + \wp_{IV}(i, k))$ $\wp_{IV}(j, k) = \wp_{IV}(j, i) + \wp_{IV}(i, k);$

for any $x_i \in Qiv$ $\omega(x_i) = \max\{\wp_{IV}(j, i) \mid x_j \in Qiv\};$

//Computing the unfolding factors of quasi index variables.

for any $x_i \in Ix$

for any $x_j \in Ix$

$\wp_{IX}(i, j) = \Phi(i, j) = 0;$

for any $x_i \in Qix$

for any $x_j \in Qix$ for any $x_k \in Qix$
 if $(\Phi(j, i) \wedge \Phi(i, k) \wedge \Phi(j, k))$
 if $(\wp_{Iv}(i, k) < \wp_{Iv}(j, i) + \wp_{Iv}(i, k))$ $\wp_{Iv}(j, k) = \wp_{Iv}(j, i) + \wp_{Iv}(i, k)$;
 for any $x_j \in Qix$
 $\omega(x_j) = \max\{\wp_{Iv}(j, i) \mid x_j \in Qix\}$;
 End

9 Loop Unfolding

After identifying the set of quasi invariant/index variables and figuring out their unfolding factors by using Alg. 1 and Alg. 2, all that remains now is to select the maximum unfolding factors as the number of iterations that should be unfolded. Because source programs have been converted into SSA form for the purpose of static analysis, it is necessary to convert the SSA form back into original source forms. The main issue to deal with is the removal of all ϕ -functions. For any ϕ -variable x (say defined as $x_3 = \phi(x_1, x_2)$), each reference to x_3 is actually a reference to x_1 or x_2 . To preserve the correctness of semantics, we must use a same name for x_1 , x_2 and x_3 such that each reference to x_3 will actually be a reference to x_1 or x_2 . The following two cases must be considered.

- *Either x_1 or x_2 is a ϕ -variable.* We recursively rename until no new ϕ -variable is encountered.
- *x_3 is an operand of another ϕ -variable.* Suppose x_3 is an operand of another ϕ -variable (e.g., $y = \phi(z, x_3)$), y , z and x_3 should also be renamed using the same name. The process continues recursively until no new ϕ -variable is encountered.

Assuming that function α is used to compute the set of variables that should be renamed by a same name, and σ denotes the set of variables already handled, α is defined as below:

$$\begin{aligned}
 \alpha(x)_\sigma &= \begin{cases} \sigma & \text{if } x \in \sigma \\ \sigma \cup \{x\} & \text{if } x \text{ is not a } \phi\text{-variable} \\ \alpha(y)_{\sigma \cup \{y\}} \cup \alpha(z)_{\sigma \cup \{z\}} \cup \beta(x)_{\sigma \cup \{x\}} & \text{if } x = \phi(y, z) \end{cases} \\
 \beta(x)_\sigma &= \begin{cases} \sigma & \text{if } x \in \sigma \text{ or } x \text{ is not an argument of a } \phi\text{-function} \\ \alpha(z)_{\sigma \cup \{z\}} \cup \beta(y)_{\sigma \cup \{y\}} & \text{if } y = \phi(x, z) \end{cases}
 \end{aligned}$$

For instance, in Fig. 9 there are two ϕ -function assignments: $w_1 = \phi(w_0, w_4)$ and $w_4 = \phi(w_2, w_3)$. All the variables in the set $\alpha(w_1) = \alpha(w_4) = \{w_1, w_0, w_2, w_3\}$ should be renamed by the same name (e.g., w). Similarly, the variables in each of the sets $\{x_1, x_0, x_2\}$, $\{y_1, y_0, y_2\}$, $\{z_1, z_0, z_2\}$, $\{t_1, t_0, t_2\}$, $\{k_1, k_0, k_2, k_3\}$ should be renamed with same names, respectively. After renaming variables, we can unfold loops. The unfolded code of Fig. 9 is shown in Fig. 11. After unfolding a loop, the assignment to each quasi invariant variable can be eliminated since the variable becomes invariant inside the remaining loop. In the remaining loop, each quasi index variable is substituted for a linear expression of index variable. Thus any reference to a quasi index variable can be replaced by the corresponding linear expression of index variable. For instance, x and y are equal to $i - 1$ and i , and w in then-part and else-part are equal to $i - 1$ and i , respectively. In the remaining loop of Fig. 11, $a[i] = p[i-1] + q[i+k]$, $b[i] = b[i] + c[2]$

can be vectorized as $a[3:n] = p[2:n-1] + q[3+k:n+k]$, $b[3:n] = b[3:n] + c[2]$, respectively.

```

if (1 <= n) {
  a[1] = p[x] + q[y+k];
  if (odd(t)) { w = 0; b[1] = b[0] + c[z]; } else { w = 1; k = d; b[1] = b[1] + c[z]; }
  t = j + z; z = 2; x = y; y = 2;
}
if (2 <= n) {
  a[2] = p[x] + q[2+k];
  if (odd(t)) { w = 1; b[2] = b[1] + c[2]; } else { w = 2; k = d; b[2] = b[2] + c[2]; }
  t = j + 2; x = y; y = 3;
}
if (3 <= n) {
  a[3] = p[2] + q[3+k];
  if (odd(t)) { w = 2; b[3] = b[2] + c[2]; } else { w = 3; k = d; b[3] = b[3] + c[2]; }
  t = j + 2; x = y; y = 4;
}
for (i = 4; i <= n; i++) {
  a[i] = p[i-1] + q[i+k];
  if (odd(t)) { w = i - 1; b[i] = b[i-1] + c[2]; } else { w = i; b[i] = b[i] + c[2]; }
  x = y; y = i + 1;
}

```

Fig. 11. The unfolded code of Fig. 9

10 Related Work

As three code optimization techniques, loop invariant code motion, loop unrolling and loop peeling have widely been studied and used by compilers. A comprehensive survey of these and other source level optimization can be found in [4]. A more recent survey of many state of the art optimization techniques for high performance architectures can be found in [2], [19].

Loop invariant code motion was originally mentioned in [1]. The notion of quasi invariant grew out of our work on partial evaluation [21]. Loop quasi invariant code motion is an extension of loop invariant code motion, which hoists invariant code to outside of loops by unfolding loops for a small number of iterations. A recently developed transformation is partial redundancy elimination (PRE), which is a global optimization technique, generalizing the removal of common sub-expressions and loop-invariant computations. Initial implementation of PRE failed to completely remove the redundancies [20, 23]. More recent PRE algorithms based on control flow restructuring [6, 24] can achieve a complete PRE and are capable of eliminating loop quasi invariant code. However, these techniques have exponential (worst-case) time complexity as well as code size explosion resulting from replication of the code. Our techniques statically determine a finite fixed point of computations induced by assignments, loops and conditionals and tries to compute the optimal unfolding factors to get maximal code motion and parallelization; and our algorithm has a polynomial time complexity.

Loop peeling was originally mentioned in [15], and automatic loop peeling techniques were discussed in [16]. August [3] showed how loop peeling can be applied in practice, and elucidated how this optimization alone may not increase program performance, but may expose opportunities for other optimization leading to performance improvements. August [3] used only heuristic loop peeling techniques. We feel that when applied to new and innovative architectures such as the SDF [14] (Scheduled Dataflow architecture, a decoupled memory/executing, multithreaded architecture using non-blocking threads), our pre-optimization approach may prove to be of significant importance. The benefits of loop unrolling have been studied for various architectures [11]. It is a fundamental technique for generating the long instruction sequences required by VLIW machines [12]. A key issue in applying loop peeling and loop unrolling is the number of iterations that must be peeled off or replicated from the loop body. Current techniques use heuristic or ad hoc techniques that are based on loop-carried dependence analysis.

Many optimization techniques can be formalized conveniently using static single assignments, including the elimination of partial redundancies [16], constant propagation [7,17], and code motion [10]. We followed the same approach to express our loop optimization technique.

11 Conclusion and Future Work

In this paper, we presented a loop oriented optimization technique based on dependence analysis. In particular, our technique detects anti-dependencies among variables involved in loop, and then tries to remove some anti-dependencies as much as possible by unfolding loops for a small number of iterations. After the removal of quasi invariant variables and the substitution of quasi index variables for linear functions of index variables, there will be only inductive variables inside loops, and thus loops will be relatively clean and easy to analyze and expose more opportunities for other optimization leading to performance improvements. Exploiting this technique, we can extend conventional loop invariant code motion to loop quasi invariant code motion, which is capable of moving not only invariant code but also quasi invariant code. Loop quasi invariant code motion is well-suited as a supporting transformation in compilers, partial evaluators, and other program transformers. Moreover, removing loop-independent dependences may make static analysis based on loop-carried dependence easier, which will be very beneficial to many other optimizations leading to performance improvements such as loop unrolling, loop peeling and so on. Our technique has the potential to increase the accuracy of program analyses and to expose newer program optimizations (e.g., branch predication, for extracting instruction-level parallelism from programs.), which are of central importance to many compilers and program transformations. The algorithms presented in this paper uses the infrastructure already present in many compilers, such as dependence graphs and static single assignments. Thus they do not require fundamental changes to existing systems. The application of this technique to our ongoing compiler for the multithreaded architecture SDF, and larger practical programs is hoped to reveal the significance of the work presented here. To the

best of our knowledge, this is the first attempt of systematically making use of loop-independent dependences among variables to unfold loops for optimization.

References

1. Aho A.V., Sethi R., and Ullman J. D., "Compilers: Principles, Techniques, and Tools", Addison-Wesley, Reading, Mass, 1986.
2. Allen R., and Kennedy K., "Optimization Compilers for Modern Architectures", Morgan Kaufmann Publishers, 2002.
3. August D.I., "Hyperblock performance optimizations for ILP processors", M.S. thesis, Department of Electrical and Computer Engineering, University of Illinois, Urbana, IL, 1996.
4. Bacon D.F., and Graham S.L., "Compiler transformations for high-performance computing", ACM Computing Surveys, December 1994, Vol. 26, No. 4, pp.345-420.
5. Banerjee, U., "An introduction to a formal theory of dependence analysis", Journal of Supercomput. Vol. 2, No.2, 1988, pp.133-149.
6. Bodik R., Gupta R., and Soffa M L., "Complete removal of redundant expressions", Prod. ACM Conf. On Programming Language Design and Implementation, pp.1-14, ACM Press, 1998.
7. Buljonkov M.A., and Kochetov D.V., "Practical aspects of specialization of Algol-like programs", eds. Dancy O., Glueck R., and Thiemann P., "Partial Evaluation", Proceedings. LNCS, Vol. 1110, pp.17-32, Springer-Verlag, 1996.
8. Cocke J., and Schwartz J.T., "Programming languages and their compilers (preliminary notes)", 2nd Courant Institute of Mathematical Science, New York University, New York.
9. Cytron R., and Ferrante J., "Efficiently computing static single assignment form and the control dependence graph", ACM TOPLAS, October, 1991, Vol. 13, No. 4, pp.451-490.
10. Cytron R., Lowry A., and Zadeck F.K., "Code motion of control structures in high-level languages", Conference Record of the 13th ACM Symposium on Principle of Programming Languages, pp. 70-85, ACM Press, 1986
11. Dongarra J., and Hind A.R., "Unrolling loops in Fortran", Softw. Pract. Exper., Vol. 9, No. 3, pp. 219-226, 1979.
12. Ellis J.R., "Building: A Compiler for VLIW Architecture", ACM Doctoral Dissertation Award. MIT Press, Cambridge, Mass, 1986.
13. Floyd R.W., "Algorithm 97: Shortest path", Communications of the ACM, 1962, Vol. 5, No. 6, pp.345.
14. Kavi K. M., Giorgi R. and Arul J., "Scheduled Dataflow: Execution paradigm, architecture and performance evaluation", IEEE Transactions on Computer, Vol. 50, No. 8, pp. 834-846, Aug. 2001.
15. Lin D.C., "Compiler support for predicated execution in superscalar processors", M.S. thesis, Department of Electrical and Computer Engineering, University of Illinois, Urbana, IL, 1992.
16. Mahlke S.A., "Exploiting instruction level parallelism in the presence of conditional branches", Ph.D. thesis, Department of Electrical and Computer Engineering, University of Illinois, Urbana, IL, 1995.
17. Metzger R., and Stroud S., "Interprocedural constant propagation: An empirical study", ACM Letters on Programming Languages and Systems, Vol. 2, No.1, pp. 213-232, 1993.
18. Padua D.A., and Wolfe M.J., "Advanced compiler optimizations for supercomputers", Communications of the ACM, December 1986, Vol. 29, No. 12, pp. 1184-1201.

19. Pande S., and Agrawal D. P., (Eds.) "Compiler Optimizations for Scalable Parallel Systems", LNCS 1808, Springer, 1998.
20. Rosen B.K., Wegman M.N., and Zadeck F.K., "Global value numbers and redundant computations", Conference Record of the 15th ACM Symposium on Principles of Programming Languages, ACM Press, 1988, pp. 12-27.
21. Song L., "Studies on Termination Methods of Partial Evaluation", Ph.D. thesis, Department of Computer Science, Waseda University, Tokyo, Japan, 2001.
22. Steffen B., "Property oriented expansion", Symposium on Static Analysis, LNCS 1145, pp. 22-41, Springer-Verlag, 1996.
23. Steffen B., Knoop J., and Rüthing O., "The value flow graph: A program representation for optimal program transformations", ed. Jones N. D., ESOP'90, LNCS 432, pp. 389-405, Springer-Verlag, 1990.
24. Warshall S., "A theorem on Boolean matrices", Journal of the ACM, January 1962, Vol. 9, No. 1, pp. 11-12.
25. Wolfe, M.J., "Optimizing supercompilers for supercomputers", Research Monographs in Parallel and Distributed Computing, MIT Press, Cambridge, Mass.
26. Wolfe, M.J., "High performance compilers for parallel computing", Addison-Wesley Publishing Company, Inc., 1996.
27. Zima H., and Chapman B., "Supercompiler for parallel and vector computers", Frontier, Series, ACM Press, 1990.

Tailoring Software Pipelining for Effective Exploitation of Zero Overhead Loop Buffer^{*}

Gang-Ryung Uh

Computer Science
Boise State University
uh@cs.boisestate.edu

Abstract. A Zero Overhead Loop Buffer (ZOLB) is an architectural feature that is commonly found in DSPs (Digital Signal Processors). This buffer can be viewed as a compiler (or program) managed cache that can hold a limited number of instructions, which will be executed a specified number of times without incurring any loop overhead. Preliminary versions of the research, which exploit a ZOLB, report significant improvement in execution time with a minimal code size increase [UH99,UH00]. This paper extends the previous compiler efforts to further exploit a ZOLB by employing a new software pipelining methodology. The proposed techniques choose complex instructions, which capitalize on instruction level parallelism across loop iteration boundaries. Unlike the traditional pipelining techniques, the proposed pipelining strategy is tightly coupled with instruction selection so that it can perform register renaming and/or proactively generate additional instruction(s) on the fly to discover more loop parallelism on the ZOLB. This framework reports additional significant improvements in execution time with modest code size increases for various signal processing applications on the DSP16000.

1 Introduction

The common features of signal processing applications are intense numerical computations with hard real-time constraints [CAL93]. Digital signal processors (DSPs) are special purpose embedded processors requiring low power and limited memory to support such applications [LEE88,LEE89]. Thus, DSPs typically support heterogeneous multiple register files, that are placed in an arbitrary datapath to reduce the processor size and irregular instruction sets optimized for instruction length [ALL85].

However, the irregularities, which are present in both microarchitectures and instruction sets of DSPs, make compiler code generation extremely difficult and challenging [ARA95,ARA195,DES93,LIA96]. Unless applying target specific compiler transformations that are specially tailored and tuned for a given DSP, applications written in a high level language are typically translated

^{*} David Whalley, Teresa Cole, and the anonymous reviewers provided helpful suggestions that improved the quality of the paper. This research was supported by NSF EPSCoR Start-up Augmentation Funding.

into a sequence of DSP instructions that run significantly slower than those of hand-crafted DSP instructions.

The main objective of this paper is to design and implement an effective instruction selection framework that chooses high quality instructions, which capitalize on instruction level parallelism across loop iteration boundaries. For that purpose, the author integrated software pipelining into the instruction selection framework. Unlike the conventional software pipelining techniques, the proposed strategy proactively attempts to direct register renaming and/or to select instructions on the fly to expose more parallelism for a given loop to the final code generator. Thus, the ZOLB (Zero Overhead Loop Buffer) on DSP16000 can be further exploited [LU97,UH99,UH00].

1.1 Zero Overhead Loop Buffer as an Architectural Features of DSPs

Target signal processing applications require extensive arithmetic computations. For example, consider the following calculations typically found in communication and image processing.

$$\begin{aligned}
 FIR: \quad y_k &= \sum_{n=0}^N b_n x_{k-n} \\
 FFT: \quad y_k &= \sum_{j=0}^{N-1} w^{jk} x_j, \text{ where } w = e^{\frac{-2i\pi}{N}} \\
 2D-DCT: \quad F(u,v) &= \frac{1}{N^2} \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} f(m,n) \cos\left[\frac{(2m+1)u\pi}{2N}\right] \cos\left[\frac{(2n+1)v\pi}{2N}\right]
 \end{aligned}$$

The main computational engines (or kernels) of these algorithms can be easily programmed into tight small loops, and a large percentage of the execution time will be spent the innermost loops. Without any software or hardware support, the execution of kernels for the aforementioned algorithms would incur significant loop overhead, and stringent industry real-time constraints may be difficult to meet with current microprocessor fabrication technology.

A Zero Overhead Loop Buffer (ZOLB) is an architectural feature commonly found in DSP processors to reduce loop overhead. A ZOLB is a buffer that can contain a fixed number of instructions to be executed a specified number of times under program control. The ZOLBs currently available in TI, ADI, and DSP processors are discussed in [LAP96]. This buffer can be used to increase the speed of applications with no increase in code size and often with reduced power consumption. Depending on the implementation of the DSP architecture, some instructions may be fetched faster from a ZOLB than from conventional instruction memory. In addition, the same memory bus used to fetch instructions can sometimes be used to access data when certain registers are de-referenced. Thus, memory bus contention can be reduced when instructions are fetched from

a ZOLB. Due to addressing complications, transfers of control instructions are not typically allowed in such buffers. Therefore, a compiler or assembly writer attempts to execute many of the innermost loops of programs from this buffer. A ZOLB can be viewed as a compiler (software) controlled cache since special instructions are used to load instructions into it.

1.2 Compiler-Based Support to Exploit a ZOLB

A HLL (High Level Language) compiler for a DSP should play the following two major roles to lead the processor into satisfactory performance. First, the compiler should relieve application programmers from the burden of developing code in assembly language, which is both time-consuming and error-prone. Thus, a compiler can help meet fast time-to-market and reliability requirements for DSPs. Second, the compiler should generate high quality code to satisfy tight real-time performance constraints with a minimal code size increase.

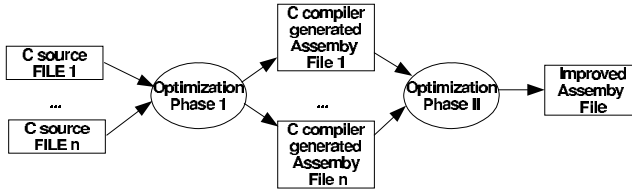


Fig. 1. Overview of the Compilation Process for the DSP16000

In order to allow the DSP16000 C compiler to perform these two roles, we have designed and implemented compiler optimization strategies for exploiting the ZOLB that is available on the DSP16000 architecture. The implementation has proven the effectiveness of the techniques for wireless communication applications [UH99,UH00]. The author believe that these strategies have the potential for being readily adopted by compiler writers for DSP processors since they rely on the use of traditional compiler improving transformations and data flow analysis techniques.

Figure 1 presents an overview of the compilation process that we used to generate and improve code for this architecture. Code is generated using a GNU C compiler retargeted to the DSP16000. Conventional improving transformations in this C compiler are applied and assembly files are generated. Finally, the generated code is then processed by the assembly optimizer that the author helped to develop. This optimizer performs a number of improving transformations including those that exploit the ZOLB on this architecture.

There are advantages of attempting to exploit a ZOLB using this approach. First, the exact number of instructions in a loop will be known after code generation, which will ensure that the maximum number of instructions that can be

contained in the ZOLB is not exceeded. While performing these transformations after code generation sometimes resulted in more complicated algorithms, the optimizer was able to apply transformations more frequently since it did not have to rely on conservative heuristics concerning the ratio of intermediate operations to machine instructions. Second, inter-procedural analysis and transformations also proved to be valuable in exploiting a ZOLB [YHO99,UH99,UH00].

Even though the effectiveness of the previous compiler optimization technologies has been proven significant, the postpass optimizer in Figure 1 does not yet produce code using DSP16000 complex instructions (F1/F1E class) yet. In order to overcome such a code generation limitation, the author developed an integrated instruction selection strategy specially tailored for the loop kernels loaded in the DSP16000 ZOLB.

This paper is organized as follows. In section 2, detailed architectural features of DSP16000 ZOLB and instructions to manipulate the buffer are explained. As a highlight of this paper, section 3 explains a new instruction selection framework. The proposed techniques actively discover complex instructions that capitalize on multiple effects across loop iteration boundaries for loop kernels on the DSP16000 ZOLB. In section 4, benchmark performance results are presented. Finally, the conclusion of the paper is in section 5.

2 Using the DSP16000 ZOLB

The target architecture for which the author generated code was the DSP16000 developed at Lucent Technologies. Two special instructions, the *do* and the *redo*, are used to control the ZOLB on the DSP16000 [LU97]. Figure 2(a)(i) shows the assembly syntax for using the *do* instruction, which specifies that the n instructions enclosed between the curly braces are to be executed k times. The actual encoding of the *do* instruction includes a value of n , which can range from 1 to 31, indicating the number of instructions following the *do* instruction that are to be placed in the ZOLB. The value k is also included in the encoding of the *do* instruction and represents the number of iterations associated with an innermost loop placed in the ZOLB. When k is a compile-time constant less than 128, it may be specified as an immediate value. Otherwise, the value of zero is encoded and the number of times the instructions in the ZOLB will be executed is obtained from the *loop* register. The first iteration results in the instructions enclosed between the curly braces being fetched from the memory system, executed, and loaded into the ZOLB. The remaining $k-1$ iterations are executed from the ZOLB. The *redo* instruction shown in Figure 2(a)(ii) is similar to the *do* instruction, except that the current contents of the ZOLB are executed k times. Figure 2(b) depicts some of the hardware used for a ZOLB, which includes a 31 instruction buffer, a *loop* register is initially assigned the number of iterations and is implicitly decremented on each iteration, and a *cstate* register containing the number of instructions in the loop and the pointer to the current instruction to execute. Performance benefits are achieved whenever the number of iterations executed is greater than one.

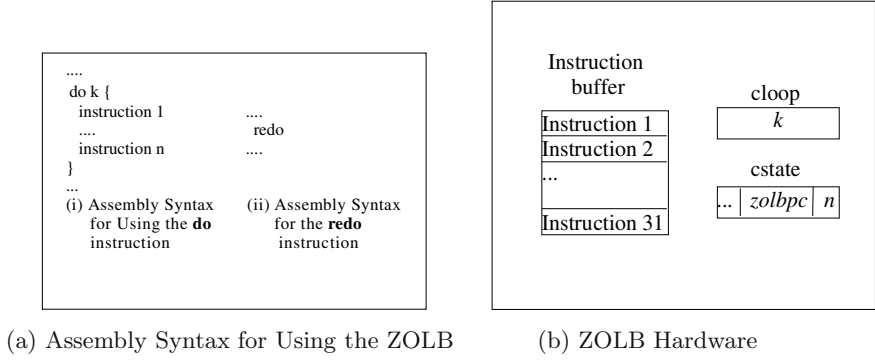


Fig. 2. DSP16000 Zero Overhead Loop Support

Figure 3 shows a simple example of exploiting the ZOLB on the DSP16000. Figure 3(a) contains the source code for a simple loop. Figure 3(b) depicts the corresponding code for the DSP16000 without placing instructions in the ZOLB. The effects of these instructions are also shown in this figure. The array in Figure 3(a) and the arrays in the other examples in the paper are of type **short**. Thus, the post-increment causes *r0* to be incremented by 2. Many DSP architectures use an instruction set that is highly specialized (unorthogonal) for known DSP applications. The DSP16000 is no exception and its instruction set has many complex features, which include separation of address (*r0-r7*) and accumulator (*a0-a7*) registers, post-increments of address registers, and implicit sets of condition codes from accumulator operations. Figure 3(b) also shows that the loop variable is set to a negative value before the loop and is incremented on each loop iteration. This strategy allows an implicit comparison to zero with the increment to avoid performing a separate comparison instruction. Figure 3(c) shows the equivalent code after placing the loop in the ZOLB. The branch in the loop is deleted since the loop will be executed the desired number of iterations. After applying basic induction variable elimination and dead assignment elimination, the increment and initialization of *a1* are removed. Thus, the loop overhead has been eliminated.

3 Instruction Selection Sensitive Software Pipelining to further Exploit ZOLBs Available on DSPs

3.1 DSP16000 Pipeline and Instruction Set Features

Many signal processing algorithms comprise the following three operations - (1) read coefficient and input from streaming data, (2) multiply, and (3) accumulate the result. In a typical microarchitecture, each operation will be positioned at the pipeline with the following time-stamps:

<pre> for (i = 0; i < 1000; i++) a[i] = 0; (a) Source Code of a Simple Loop </pre>		
<pre> r0 = a # r[0] = ADDR(_a) a2 = 0 # a[2] = 0; a1 = -9999 # a[1] = -9999 L5: *r0++ = a2 # M[r[0]] = a[2]; r[0] = r[0] + 2 a1 = a1 + 1 # a[1] = a[1] + 1; IC = a[1] + 1 ? 0; if le goto L5 # PC = IC ≤ 0 ? L5 : PC; </pre>	<pre> cloop = 10000 r0 = _a do cloop { *r0++ = a2 } </pre>	
(b) DSP16000 Assembly and Corresponding RTLs without using the ZOLB	(c) After using the ZOLB	

Fig. 3. Example of Using the ZOLB on the DSP16000

1	READ C1,D1		
2	READ C2,D2	MULT P1,C1,D1	
3	READ C3,D3	MULT P1,C2,D2	ACCUMULATE P1
...	...	...	ACCUMULATE P1

Once the pipeline is fully loaded for the operations, the operations performed at the time slot 3 will be the common signal. In order to achieve higher code density and performance, DSP16000 provides F1/F1E class instructions to capture common signals typically observed from communication oriented applications [AIK88,MAL81]. As one illustration for F1/F1E instructions, DSP16000 provides the following instruction to support the example control signal:

$a0 = a0 + p0 \quad p0 = xh * yh \quad p1 = xl * yl \quad y = *r0++ \quad x = *pt0++$

where a represents a 40 bit accumulator, p represents a 32 bit product register, r represents an address register for data memory and pt represents an address register for coefficient memory. x and y registers¹ are used to hold 32 bit values from the addresses pointed by $r0$ and $pt0$. Note that all the effects in the above instruction are compressed into a single 16 bit word. Therefore, the permissible order of operations is very limited and the register usage is restricted to only a few (at the most four) different registers² in each file. In addition to that, the above example must have all 5 effects to be legal. Therefore, conventional compiler ILP scheduling algorithms, which do not correctly model the nature of the instruction set design and encoding restrictions, do not perform well on DSPs, i.e. ADSP-21xx and TMS320C54x, that have similar encoding restrictions. The detailed explanation is forthcoming in the following section.

¹ xl/yl denotes the low byte of x/y register and xh/yh denotes the high byte of x/y register.

² Note that operands in certain fields are required to be either even or odd number registers.

3.2 Compiler Mission and Experience with Related Works

The mission of the compiler to exploit DSP16000 pipeline can be defined as follows,

For a given loop written in a naive programming style for the time-stationary pipeline DSP, group as many instructions as possible from different loop iterations such that those grouped instructions can be combined into more powerful and fewer instructions.

In order to exploit the DSP16000 complex instructions that comprise multiple effects, the author has tried to adapt iterative modulo scheduling algorithm to implement software pipelining for the DSP16000 [ERIC99,HUF93,RAU94]. Software pipelining is an aggressive compiler optimization technique for restructuring a loop so that each iteration in the pipelined loop is made from instructions scheduled from different iterations of the original loop [LAM88]. Thus, rescheduled loops can better exploit (or saturate) resources provided by ILP (Instruction Level Parallelism) architectures, such as VLIW or Superscalar. The main reason for choosing the modulo scheduling among many other alternatives [VIK95] is that the effectiveness of the scheduling algorithm has been widely validated by several industry compilers targeted for high performance microarchitectures [ERIC99,EIC97].

However, the author learned the hard way that the reported modulo scheduling algorithms are not well suited for DSP16000. First, the core of reported iterative modulo scheduling algorithms [ERIC99,HUF93,RAU94] fundamentally lies in how to model the near optimal cyclic scheduling for the slack³ created from different latencies of loop kernel instructions. Therefore, in the absence of sufficient scheduling slack, the effectiveness of the reported algorithms are seriously limited. For instance, TMS320C6X has a multiply instruction on the M unit that requires two cycle latencies and has a load instruction on the D unit that requires five cycle latencies. The lifetime of a value on TMS320C6X is the distance in the schedule between the placement of the operation that defines the value and the operation that uses the value. Operations with latency > 1 are in-flight until they complete execution. The TMS320C6X architecture allows multiple in-flight operations to have pending writes to the same register. Therefore, the adapted modulo scheduling algorithm for TMS320C6X [ERIC99] is basically to achieve near optimal scheduling for the slack produced from load and multiply latencies by exploiting the in-flight pipeline feature. Unfortunately, the DSP16000 has very few variation on instruction latency and virtually all instructions can be considered one cycle latency (including the load instruction). Thus, there is not enough slack to enjoy any of the aforementioned iterative modulo slack scheduling techniques for loops on the DSP16000 ZOLB.

As one concrete illustration, consider the loop kernel *iir_32*, shown in Figure 4, on the DSP16000 ZOLB and its data dependency graph represented as

³ When an operation is placed into a partial schedule, it will in general have an earliest start time (Estart) and a latest start time (Lstart), due to predecessors and successor instructions that have already placed. The difference between these two bounds is denoted as the operation's slack.

```

do 50 {
/* inst 1 */  xh = *(r0 + j)
/* inst 2 */  yh = *r3++
/* inst 3 */  r4 = j
/* inst 4 */  p0 = xh*yh p1 = xl*yl
/* inst 5 */  a2 = a2+p0
/* inst 6 */  j = r4+1
}

```

Fig. 4. iir_32 Loop Kernel Code

	Start	Inst-1	Inst-2	Inst-3	Inst-4	Inst-5	Inst-6	End
Start	X	(0,0)	(0,0)	(0,0)	(0,0)	(0,0)	(0,0)	(0,0)
Inst-1	X	X	X	X	(0,1)	X	X	(0,0)
Inst-2	X	X	X	X	(0,1)	X	X	(0,0)
Inst-3	X	X	X	X	X	X	(0,1)	(0,0)
Inst-4	X	(1,0)	(1,0)	X	X	(0,1)	X	(0,0)
Inst-5	X	X	X	X	(1,0)	X	X	(0,0)
Inst-6	X	(1,1)	X	(1,1)	X	X	X	(0,0)
End	X	X	X	X	X	X	X	X

Fig. 5. Adjacency Matrix for the iir_32 loop body

the Adjacency Matrix, which shown in Figure 5. First, a typical iterative modulo scheduling algorithm computes an initial loop initiation interval by taking $\text{MAX}(\text{RecII}, \text{ResII})$. The RecII is the smallest loop initiation interval that can meet all the deadlines imposed by all data dependence recurrences (circuits) in the Adjacency Matrix. The ResII is the smallest initiation interval that can meet the total resource requirements of all operations in a given loop. For the example loop, the RecII is computed to be 3, that makes $\text{MinDist}[i,j] \leq 0$ with the Floyd's shortest path algorithm on the Adjacency Matrix shown in Figure 6. For the same example loop, the initial ResII is 2 since there are four AGU instructions (inst-1, inst-2, inst-3, and inst-6) with two AGU functional units. Thus, the initial loop initiation interval is 3. Second, the iterative modulo scheduling algorithm attempts to find the loop schedule starting when the loop initiation interval = 3. At this stage, the algorithm computes the slack (possible issue slots that satisfies all data dependency constraints) for each instruction and prioritizes the instructions so that the most critical instruction can be scheduled first in the slack without violating resource constraints. In the absence of DSP16000 instruction encoding restriction resource constraint, the Richard Huff's slack scheduling [HUF93] produces the partial schedule as shown in Figure 7.

	Start	Inst-1	Inst-2	Inst-3	Inst-4	Inst-5	Inst-6	End
Start	X	0	0	0	1	2	1	2
Inst-1	X	-2	-2	X	1	2	X	2
Inst-2	X	-2	-2	X	1	2	X	2
Inst-3	X	-1	-3	-1	0	1	1	1
Inst-4	X	-3	-3	X	-2	1	X	1
Inst-5	X	-6	-6	X	-3	-2	X	0
Inst-6	X	-2	-4	-2	-1	0	-1	0
End	X	X	X	X	X	X	X	X

Fig. 6. $\text{MinDist}[i][j]$ Matrix, where $\text{Start} \leq i, j \leq \text{Stop}$

Operation	Slack		Issue Time
	Estart	Lstart	
Inst-1	0	1	0
Inst-2	0	1	0
Inst-3	0	1	1
Inst-4	1	2	1
Inst-5	2	3	1
Inst-6	2	2	2

Fig. 7. Slack and Issue time for each iir_32 kernel instruction in Figure 4

As easily can be observed from Figure 7, there is not enough slack (at the most two issue slots) for each instruction to be manipulated with loop initiation interval = 3. The more serious problem is that, once instruction encoding is considered as an additional resource constraint, there is virtually no legal schedule to achieve non-trivial loop initiation interval (< 6) for any possible issue slot placement decision. In Figure 7, (inst-1,inst-2) and (inst-3,inst-4,inst-5) are legal in a partial schedule in the absence of DSP16000 instruction encoding restriction. However, there exists no legal DSP16000 instruction that encodes either of these two sets of operations in a partial schedule. Furthermore, there are often cases where there exist a DSP16000 complex instruction, which accounts for (inst-i,inst-j,inst-k), but there is no legal encoding to capture any proper subset of (inst-i,inst-j,inst-k). Considering the iterative modulo scheduling attempts to construct the partial schedule by placing one instruction at a time, the desired schedule is not achievable by any known pipelining techniques.

In order to address these two major problems (lack of slack and instruction encoding) that prevent conventional modulo scheduling algorithms to exploit DSP16000 complex instructions, the author designed a new software pipelining framework which is independent of the reported algorithms [RAU94,VIK95]. The uniqueness of the proposed method lies in the fact that the desired ILP optimization to exploit DSP16000 complex instructions occurs within the instruction selection framework. This allows the instruction selection to proactively perform register renaming and to introduce additional instruction(s) on the fly to transform a potential set of parallel operations (or effects), which discovered during software pipelining, into a legal DSP16000 complex instruction. The author believes that this is the only viable solution to exploit high quality DSP16000 instructions, which capitalize on instruction level parallelism across loop iteration boundaries. Thus, the ZOLB (Zero Overhead Loop Buffer) on DSP16000 can be further exploited [LU97,UH99,UH00].

3.3 Strategy to Further Exploit ZOLB by Improving Instruction Level Parallelism

In order not to interfere with other existing code improving optimizations, the extended instruction selection for loops on DSP16000 ZOLB is performed after all the loop optimizations have been attempted, which include basic induction variable elimination, extraction of basic induction variable assignments, and local/global list scheduling [UH00]. By doing this way, the author can safely guarantee the proposed optimization can improve the resource utilization across loop iteration boundaries. Note that the proposed optimization may occur before other loop optimizations to discover more parallelism and, as a result, more loops can be placed on the ZOLB. However, due to the potential code size explosion of the new scheme, the author strictly limits the techniques to the loops placed on the DSP16000 ZOLB in this paper. The overall schemes of the proposed instruction selection strategy are as follows.

- **Task 1:** Partition a given loop body into n instruction groups, $G_1, G_2, \dots, G_n$, such that instructions in G_i can be potentially scheduled in G_k , where $k = (i + 1), (i + 2), \dots, n$.
- **Task 2:** Restructure the loop such that its body consists of n instruction groups, where each group is selected from a different loop iteration.
- **Task 3:** Perform instruction selection among n groups in the restructured loop body such that selected instructions can be combined into fewer instructions. If necessary, perform register renaming or/and proactively introduce extra instruction(s) to reform a potential set of parallel operations to a legal DSP16000 encoding.

Task 1 - Partition a Candidate Loop Body:

In the DSP16000 instruction set, there are about 50 different F1/F1E instruction templates, where each template allows only a certain combination of register usage. Two DSP16000 instructions I_i and I_{i+1} are defined potentially pipelineable only when either one of the following conditions can be met:

- a) – There exists a F1/F1E class instruction template that holds (may be more) both effects I_i and I_{i+1} in parallel [LU97]. This implies that I_i and I_{i+1} can be potentially combined into a single complex instruction.
- b) – I_{i+1} does not depend on the result of I_i .

The first condition models the fact that the DSP16000 instructions has a single cycle latency: this property allows that any instruction I_{i+1} at the j -th iteration can be overlapped with I_i at the $(j + 1)$ -th iteration as far as there potentially exists some legal encoding. The second condition accounts for the placement of instruction(s) from different loop iterations when instructions are allowed to be overlapped.

```

/* initialization */
VOID partition() {

    create a new group G1;
    add an instruction I1 to G1;
    i = 2;
    j = 1;

    FOR each instruction Ii in the loop DO {
        IF (Ii-1 and Ii are potentially pipelineable) {
            j = j + 1;
            create a new group Gj;
            Tag Gj with F1/F1E instruction template if exists;
        }
        add an instruction Ii to Gj;
        i = i + 1;
    }
}

```

Fig. 8. Partition Algorithm

Instruction	DSP Code Fragment	Groups
	do 92 {	
I ₁	y=r0++ x=*pt0++	G ₁ =(I ₁)
I ₂	p0=xh*yh p1=xl*yl	G ₂ =(I ₂)
I ₃	a0=*r2	G ₃ =(I ₃)
I ₄	a0=a0+p1	G ₃ =(I ₃ ,I ₄)
I ₅	*r2++=a0	G ₃ =(I ₃ ,I ₄ ,I ₅)
I ₆	a0=*r2	G ₃ =(I ₃ ,I ₄ ,I ₅ ,I ₆)
I ₇	a0=a0+p0	G ₃ =(I ₃ ,I ₄ ,I ₅ ,I ₆ ,I ₇)
I ₈	*r2++=a0	G ₃ =(I ₃ ,I ₄ ,I ₅ ,I ₆ ,I ₇ ,I ₈)
	}	

Fig. 9. Example of Partitioning for *fir* Loop Kernel Code

For each loop on the ZOLB, the actual partitioning is performed by the algorithm described in Figure 8. The partitioning algorithm has a couple of drawbacks. First, the placement of the first instruction on each group affects the final schedule. Second, the DSP16000 F1/F1E instruction template search based on two instructions is sub-optimal since the better template may be found by considering more than two instructions. The author will definitely refine the partitioning algorithm in the future. However, the partition scheme shown in Figure 8 can be justified to a certain degree. For the first drawback, a typical communication kernel is prepared to exploit load-multiply-accumulate instructions so that there may not be much room for the local scheduling due to its inherent data dependences. For the second drawback, however the partitioning algorithm shown in Figure 8 is driven by considering only two instruction, the subsequent instruction selection algorithm described in Task 3 can find a certain level of full parallelism in a transitive manner.

As an illustration of the algorithm, consider the DSP16000 assembly code fragment shown in Figure 9, which is produced by the C compiler when translating the *fir* kernel. The loop body is partitioned into three different instruction groups (G_1, G_2, G_3) by the above partition algorithm.

Task 2 - Restructure the Loop:

Assuming that given partitioned instruction groups ($G_1, G_2, \dots, G_n$) are maximally pipelined, the restructured loop body will consist of ($G_n, G_{n-1}, \dots, G_1$), as shown in Figure 10, where group G_{n-i} is selected from $(i+1)$ -th iteration of the original loop for $i = 1, \dots, (n-1)$. Note that the restructured loop body does not contain any immediate data dependency since each instruction group is only overlapped with adjacent instruction group from the subsequent loop iteration. Also, note that this overlapping with the adjacent instruction group is only performed in the presence of potential DSP16000 complex instruction template. In short, the optimizer will only look for a valid schedule among the adjacent instruction groups, where each group has an attribute of the desired DSP16000 complex instruction encoding template. Based on the encoding template, the optimizer may later direct register renaming or/and may proactively introduce extra instructions to meet the associated DSP16000 encoding restriction. Since the directed scheduling will be only performed among inter instruction groups, the postpass optimizer can project the actual code layout for the restructured loop as follows:

1. Loop Prologue: ($G_1, G_2, \dots, G_{n-1}, G_1, G_2, \dots, G_{n-2}, \dots, G_1$)
2. Software-pipelined Loop Body: ($G_n, G_{n-1}, \dots, G_2, G_1$)
3. Loop Epilogue: ($G_n, G_{n-1}, G_n, \dots, G_{n-k}, \dots, G_n$)

Consider the example loop kernel shown in Figure 9 as an illustration for restructuring. As a result of Task 1, the loop kernel is partitioned into three instruction groups (G_1, G_2, G_3). Thus, the maximally pipelined loop will be projected by the postpass optimizer as follows (the actual projection is illustrated in Figure 11).

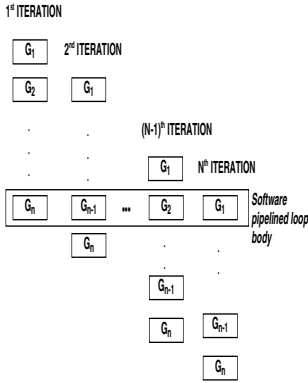


Fig. 10. Software Pipelined Loop

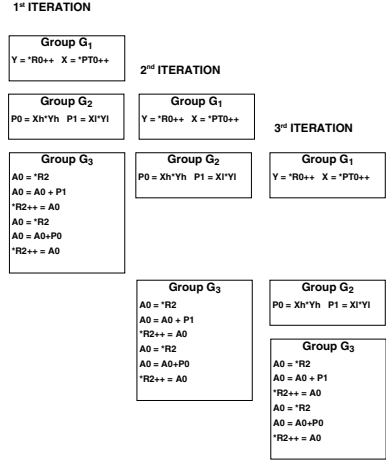


Fig. 11. Restructured Loop for an Example Kernel shown in Figure 9

1. Loop Prologue: (G_1, G_2, G_1)
2. Software-pipelined Loop Body: (G_3, G_2, G_1)
3. Loop Epilogue: (G_3, G_2, G_3)

Task 3 - Instruction Selection among Instruction Groups Restructure the Loop:

When the partitioned instruction groups ($G_1, G_2, \dots, G_n$) are fully pipelined, the restructured loop body will consist of ($G_n, G_{n-1}, \dots, G_1$), where group G_{n-i} is selected from $(i+1)$ -th iteration of the original loop, where $i = 1, \dots, (n-1)$. Based on the projection of the maximally pipelined loop as shown in Figure 12, the algorithm described in Figure 13 starts finding a partial schedule. If the given projection is not achievable, then it combines two adjacent instruction groups into one starting from the first instruction group and reiterate the described steps of Task 2 and Task 3. In an iterative manner, possible overlapping opportunities can be exhausted.

The instruction selection algorithm in Figure 13 exploits the fact that the reformed loop body does not contain any immediate data dependency between adjacent instruction groups. Thus, the instructions in G_i can be safely scheduled with those of G_{i+1} . The other caveat of the instruction selection algorithm is that the instruction combining will be placed only when (1) there exists a legal DSP16000 complex instruction template and (2) the combined operations can satisfy the register encoding restrictions. Thus, register renaming can be applied in a demand driven manner within the instruction selection algorithm. Furthermore, the combined effects are tagged with the complex instruction template so that the final scheduling may insert additional effect(s) to make the overlapped

```

/* Prologue of Software Pipelining */
y = r0++ x=pt0++ // from G1 at the 1st ITERATION
p0=xh*yh p1=xl*yl // from G2 at the 1st ITERATION
y=r0++ x=pt0++ // from G1 at the 2nd ITERATION

/* Adjust number of loop iterations (peeled two times) */
do 90 {
  /* Software-pipelined loop body */
  a0=r2 // from G3 at the 1st ITERATION
  a0=a0+p1 // from G3 at the 1st ITERATION
  *r2++=a0 // from G3 at the 1st ITERATION
  a0=r2 // from G3 at the 1st ITERATION
  a0=a0+p0 // from G3 at the 1st ITERATION
  *r2++=a0 // from G3 at the 1st ITERATION
  p0=xh*yh p1=xl*yl // from G2 at the 2nd ITERATION
  y=r0++ x=pt0++ // from G1 at the 3rd ITERATION
}

/* Epilogue of Software Pipelining */
a0 =*r2 // from G3 at the 2nd ITERATION
a0=a0+p1 // from G3 at the 2nd ITERATION
*r2++ =a0 // from G3 at the 2nd ITERATION
a0=*r2 // from G3 at the 2nd ITERATION
a0=a0+p0 // from G3 at the 2nd ITERATION
*r2++ = a0 // from G3 at the 2nd ITERATION
p0=xh*yh p1=xl*yl // from G2 at the 3rd ITERATION
a0=*r2 // from G3 at the 3rd ITERATION
a0=a0+p1 // from G3 at the 3rd ITERATION
*r2++ =a0 // from G3 at the 3rd ITERATION
a0=*r2 // from G3 at the 3rd ITERATION
a0=a0+p0 // from G3 at the 3rd ITERATION
*r2++ = a0 // from G3 at the 3rd ITERATION

```

Fig. 12. DSP16000 Zero Overhead Loop Support

```

/* MAIN */
DO {
  change = FALSE;
  FOR each instruction group  $G_i$  DO {
     $I_F$  = first instruction group  $G_i$ ;
    IF ( $I_F$  == NULL)
      CONTINUE;
    IF (change == FALSE)
      change = Combine_Insts( $I_F, G_{i+1}$ );
    ELSE
      (void) Combine_Insts( $I_F, G_{i+1}$ );
    /* Advance to the next instruction group */
     $i = i + 1$ ;
  } /* end of FOR */
} WHILE (change == TRUE)

/* COMBINE INSTRUCTIONS */
BOOLEAN Combine_Insts( $I_M, G_N$ )
{
  IF ( $G_N$  == NULL)
    /*  $G_N$  is empty */
    return FALSE;
   $L$  = last instruction number in  $G_N$ ;
  FOR each instruction  $I_L$  in  $G_N$  DO {
    IF (there exists F1/F1E instruction template that
        accounts for effects  $I_{ML}$ )
    {
      IF ( $I_{LM}$  satisfies the register encoding restrictions)
      {
        replace  $I_L$  with  $I_{ML}$ ;
        remove  $I_M$  from  $G_{N+1}$ ;
        return TRUE;
      }
      ELSE IF (there exist available register(s) that
          can make  $I_{LM}$  satisfy the register
          encoding restrictions)
      {
        perform register renaming;
        tag the  $I_{ML}$  with the F1/F1E instruction template;
        remove  $I_M$  from  $G_{N+1}$ ;
        return TRUE;
      }
    }
    ELSE IF (there exists no dependency from  $I_M$  and  $I_L$ )
       $L = L - 1$ ;
    ELSE
      BREAK;
  } /* END FOR */

  Merge  $G_{N+1}$  and  $G_N$  into one instruction group;
  RETURN FALSE;
} /* END COMBINE_INSTS */

```

Fig. 13. Instruction Selection Algorithm

effects meet the format of the tagged complex instruction template. A similar technique should also be applied for the loop prologue and epilogue to reduce the overhead of restructured loop.

As an illustration of the instruction selection algorithm described in Figure 13, consider the same *fir* loop kernel shown in Figure 9. Once the restructuring techniques described in Task 2 are applied to the example loop, the maximally pipelined loop will be projected as shown in Figure 11. The instruction selection algorithm will be directed based on such projection for the instruction groups (G_1, G_2, G_3) as follows.

1. The routine `Combine_insts()` is invoked with instruction "`y = *r0++ x = *pt0++`" and instruction group G_2 as arguments.
2. The routine `Combine_insts()` returns TRUE with the following side-effects:
 - (a) Instruction "`p0 = xh*yh p1 = xl*y1`" in G_2 is replaced with "`p0 = xh*yh p1 = xl*y1 y = *r0++ x = *pt0++`"
 - (b) Instruction "`y = r0++ x=*pt0++`" is removed from G_1
3. The routine `Combine_insts()` is invoked with instruction "`p0=xh*yh p1=xl*y1 y=*r0++ x=*pt0++`" and instruction group G_3 as arguments.
4. The routine `Combine_insts()` returns TRUE with the following side-effects assuming that "`*r2++=a0`" and "`y=*r0++ x=*pt0++`" do not interfere each other in memory:
 - (a) Instruction "`a0=a0+p1`" in G_3 is replaced with "`a0=a0+p1 p0=xh*yh p1=xl*y1 y=*r0++ x=*pt0++`"
 - (b) Instruction "`p0=xh*yh p1=xl*y1 y=*r0++ x=*pt0++`" is removed from G_2
5. The algorithm terminates when no further changes occur.

The restructured loop body after the application of the instruction selection algorithm is shown in Figure 14. Note that the restructured loop body only requires six instructions compared to the original loop body that requires eight instructions. Thus, ZOLB can be further exploited by the new instruction selection sensitive software pipelining techniques described in this paper.

```

/* Adjust number of loop iterations (peeled two times) */
do 90 {
  /* Software-pipelined loop body */
  a0=*r2
  a0=a0+p1
  *r2++=a0
  a0=*r2
  a0=a0+p0  p0=xh*yh  p1=xl*y1  y=*r0++ x=*pt0++
  *r2++=a0
}

```

Fig. 14. Restructured Loop Body after Applying Task 3

4 Results

Table 1 explains the benchmarks and applications used to evaluate the impact of using the proposed instruction selection algorithm on the DSP16000 ZOLB. All of these test programs are either DSP benchmarks used in industry or typical DSP applications. Many DSP benchmarks represent kernels of programs where most of the cycles occur. Such kernels in DSP applications have been historically optimized in assembly code by hand to ensure high performance. Thus, many established DSP industrial benchmarks are small since they were traditionally hand coded.

Table 1 contrasts the results for a set of optimizations reported from [UH00], which exploits the DSP16000 ZOLB, and the same set of optimization with the

Table 1. Test Programs

Program	Description
add8	Add two 8-bit images
convolution	Convolution code
copy8	Copy one 8-bit image to another
fft	128 point complex FFT
fir	Finite Impulse Response filter
fir_no_red_ld	FIR filter with redundant load elimination
fire	FIRE encoder
iir	IIR filtering
inverse8	Invert an 8-bit image
jpegdct	JPEG Discrete Cosine Transformation
scale8	Scale an 8-bit image
sumabsdifs	Sum of abs diffs of two images
vec_mpy	Simple vector multiply

additional instruction selection techniques described in this paper. Execution measurements were obtained by accessing a cycle count from a DSP16000 simulator. The previous paper [UH00] reports that 31.79% improvement in execution time by applying the set of optimizations exploiting a ZOLB. The optimization techniques described in this paper alone made the significant improvement (9.24% additional improvement on average) in execution time. Code size measurements were gathered by obtaining diagnostic information provided by the linker.

Table 2. Impact on Execution Time

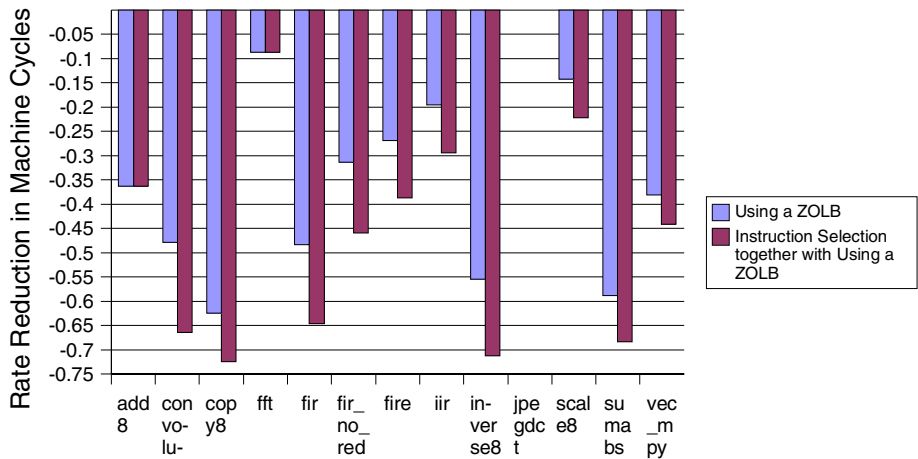
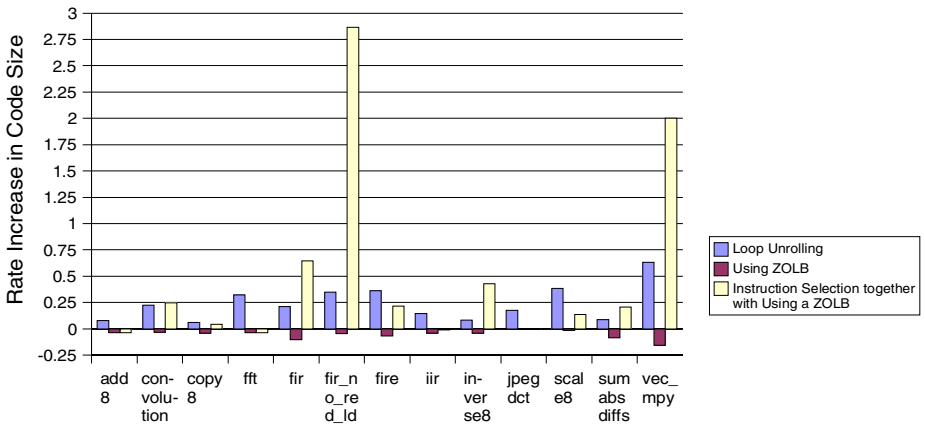


Table 3 explains the impact on the code size increase from the proposed techniques. On average, the additional optimization incurs about 51.53% code size increase compared to that of the previous reported techniques [UH00]. The increase is due to the extra code fragments for loop prologues and epilogues. The worst case complexity in space of the proposed algorithm for a loop prologue and epilogue is $O(n^2)$, where n is the number of instructions in a given loop. The author experienced significant code size increase on the benchmarks *fir_no_red_ld* and *vec_mpy* since the original loop bodies are partitioned into too many small instruction groups. Thus, the algorithms results in excessive loop peeling for the loop prologue and epilogue. Nevertheless, note that the author has not applied any other additional optimizations to reduce the code size for loop prologues and epilogues. Furthermore, the author did not control the algorithm to minimize the code growth by limiting the frequency of loop peeling. Without counting these two extreme benchmark cases, the code size increases are about the same as those of the loop unrolling by a factor of two. Thus, the average performance benefits of using the new techniques are impressive, particularly when code size is important.

Table 3. Impact on Code Size



5 Conclusions

Programmability and re-configurability are important market requirements for DSPs along with performance. A DSP should be easily programmed to customize various feature sets. In addition, the DSP should be rapidly reconfigured to support frequently varying industry standards. In order to meet these two requirements, DSPs commonly support HLL compilers.

However, the irregularities, which are present in both microarchitectures and instruction sets of DSPs, make compiler code generation extremely difficult

and challenging [ARA95,ARA195,DES93,LIA96]. Unless applying target specific compiler transformations that are specially tailored and tuned for a given DSP, applications written in a high level language are typically translated into the sequence of DSP instructions that run significantly slower than those of hand-crafted DSP instructions.

In order to make the DSP16000 C compiler exploit DSP16000 complex (F1/F1E) instructions, the author has explored conventional iterative modulo scheduling techniques. However, due to its structural limitations, reported algorithms fail to exploit complex instructions supported by extremely unorthogonal instruction set DSPs. In order to circumvent such limitations, the author designed a new software pipelining model that effectively improves the DSP code generation quality of the DSP16000 C compiler.

The proposed method chooses high quality (F1/F1E class) instructions, which capitalize on instruction level parallelism across loop iteration boundaries. Unlike the traditional pipelining techniques, the proposed strategy is tightly coupled with instruction selection that can perform register renaming in a demand driven way and proactively insert additional instruction(s) on the fly to achieve more loop parallelism on the DSP16000 ZOLB. These techniques report additional significant improvements in execution time with modest code size increases for various signal processing applications. Thus, the ZOLB of DSP16000 can be further exploited with moderate code size increases since the selected complex (F1/F1E) instructions captures effects of instructions across the multiple iterations of the original loop.

References

- AIK88. A. Aiken, A. Nicolau: A Development Environment for Horizontal Microcode. *IEEE Transaction on Software Engineering*, **14** (1988) 584-594
- ALL85. F.E. Allen, J. Cocke: Computing Architecture for Digital Signal Processing. *Proceedings of the IEEE International Conference on Acoustics, Speech, Signal*, **75(5)** (May 1985) 852-873
- ARA95. G. Araujo, S. Malik: Optimal code generation for embedded memory non-homogeneous register architectures. *Proceedings of the IEEE International Symposium on System Synthesis*, (September 1995), 36-41
- ARA195. G. Araujo, S. Devadas, K. Keutzer, S. Liao, S. Malik, A. Sudarsanam, S. Tjiang, A. Wang: Challenges in code generation for embedded processors. P. Marwedel and G. Gossens, editors, *Code Generation for Embedded Processors*, Kluwer Academic Publishers (1995)
- CAL93. J.P. Calvez: *Embedded real-time systems*, Wiley Series in Software Engineering Practice (1993)
- DES93. D. Desmet, D. Genin: ASSYNT: efficient assembly code generation for digital signal processors. *Proceedings of the IEEE International Conference Acoustics, Speech, Signal*, Minneapolis (April 1993)
- EIC97. A.E. Eichenberger: *Modulo Scheduling, Machine Representations, and Register-Sensitive Algorithms*. Ph.D Thesis, University of Michigan (1997)
- ERIC99. E. Stotzer, E. Leiss: Modulo Scheduling for the TMS320C6X VLIW DSP Architecture. *ACM SIGPLAN Workshop on Languages, Compilers, and Tools for Embedded Systems* (May 5, 1999) 28-34

- HUF93. R.A. Huff: Lifetime-Sensitive Modulo Scheduling. Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation, (June 1993) 258–267
- LAM88. M. LAM: Software Pipelining: An Effective Scheduling Technique for VLIW Machines. Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation, (June 1988) 318–328
- LAP96. P. Lapsley, J. Bier, E. Lee: DSP Processor Fundamentals - Architecture and Features. IEEE Press (1996)
- LEE88. E.A. Lee: Programmable DSP Architectures: Part I. IEEE ASSP Magazine (January 1988) 4–19
- LEE89. E.A. Lee: Programmable DSP Architecture: Part II. IEEE ASSP Magazine (January 1989) 4–19
- LIA96. S.Y. Liao: Code generation and optimization for embedded digital signal processors. Ph.D Thesis, Massachusetts Institute of Technology (June 1996)
- LSI02. LSI Logic: ZSP500 Digital Signal Processor Core Architecture (2002)
- LU97. Lucent Technologies: DSP16000 Digital Signal Processor Core Instruction Set Manual (1997)
- MAL81. S. Mallet, D. Landskov, B.D. Shriver, P.W. Mallett: Some Experiments in Local Microcode Compaction for Horizontal Machines. IEEE Transaction on Computers, **30(7)** (1981) 460–477
- RAP97. R. Leupers, P. Marwedel: Time-Constrained Code Compaction for DSPs. IEEE Transactions on VLSI Systems, **5(1)** (1997)
- RAU94. B.R. Rau: Iterative Modulo Scheduling: An Algorithm For Software Pipelining Loops. Proceedings of the 27th Annual International Symposium on Microarchitecture (November 1994) 63–74
- UH99. G.R. Uh, Y. Wang, D. Whalley, S. Jinturkar, S. Burns, V. Cao: Effective Exploitation of a Zero Overhead Loop Buffer. Proceedings of the ACM SIGPLAN 1999 Workshop on Languages, Compilers, and Tools for Embedded System (1999) 10–19
- UH00. G.R. Uh, Y. Wang, D. Whalley, S. Jinturkar, C. Burns, V. Cao: Techniques for Effective Exploitation of a Zero Overhead Loop Buffer. Proceedings of the 9th International Conference on Compiler Construction (March 2000)
- VIK95. V.H. Allan, R.B. Jones, R.M. Lee, S.J. Allan: Software Pipelining. ACM Computing Surveys, **27(3)** (September 1995)
- YHO99. Y. Wang: Interprocedural Optimizations for Embedded Systems. MS project, Florida State University (April 1999)

Case Studies on Automatic Extraction of Target-Specific Architectural Parameters in Complex Code Generation

Yunheung Paek, Minwook Ahn, and Soonho Lee

School of Electrical Engineering
Seoul National University, Korea*

Abstract. To cope with the highly complex and irregular embedded processor architectures, we employ the two traditionally-known most aggressive and computationally expensive code generation methods. One is *integrated code generation* where two main subproblems of code generation, instruction selection and register allocation, are simultaneously solved. The other is *directed acyclic graph (DAG) covering*, not tree covering, for code generation. In principle, unifying these two expensive methods may increase compilation time prohibitively. However often in practice, we have observed that the overall time can be manageably short without degrading the code quality by adding a few heuristics that fully capitalize on specific characteristics of target processor models.

1 Introduction

As compared to traditional general-purpose processors (GPPs), embedded processors usually require special hardware structures with irregular data paths and heterogeneous register architectures. They also require exceptionally high quality with small code sizes and fast execution times subject to strict real-time constraints. Thus, generating optimal code for embedded processors is extremely complex and demands expensive algorithms. However, applications for embedded processor are executed over a long time, and thus longer code generation time is generally accepted, which gives much flexibility to compilers [4].

All these unique qualifications for compilers targeting embedded processors galvanize many studies of the past decade to develop more aggressive code generation techniques than those developed for conventional GPPs. For instance, conventional code generation divides the work into several phases with manageable sub-works, which are then sequentially ordered. Although phase ordering reduces compilation time drastically, it often fails to generate optimal code for processors with irregular architectures. To remedy this problem, several researchers developed *integrated code generation* algorithms [2,5,7] where all or part of the code generation phases are performed simultaneously. For another

* This work is supported in part by KRF contract D00263, ETRI and a seed grant for a new faculty member from Seoul National University.

instance, *tree covering* with dynamic programming has been the norm in conventional code generation, even though the dataflow in a source program naturally comes in a DAG form. Tree covering has been a favorite since it uses linear-time algorithms. However, tree covering requires splitting original DAGs into trees, which not only causes extra loads/stores but also, more importantly for embedded processors, precludes opportunities to find complex instruction patterns that lie across several split trees. To alleviate this problem, more aggressive code generation algorithms were developed based on *DAG covering* rather than tree covering [3,8,10]. Unfortunately, these aggressive algorithms could be too limited in some cases to produce the best codes for embedded processors. Although this limitation could be circumvented by taking even more aggressive techniques that unify integrated code generation with DAG covering, few researchers took this aggressive approaches due mainly to its tremendously increased computational overhead. However, we have observed that the overhead can be often amortized with help of *target-specific* heuristics which take full advantage of specific characteristics of target architectures.

The first principle of our target-specific compiler is not trading off code quality for compilation time. Our compiler development process is to (1) implement a basic code generation framework where instruction selection and register allocation are simultaneously performed on input DAGs without any attempt to approximate the optimal code quality with heuristics, then (2) gradually improve the compilation speed by adding target-specific heuristics carefully chosen from the architecture information to avoid or at least minimize the degradation of the code quality. Every time the compilation speed improved, the code quality was measured in comparison with previous compilers. Although this work requires more extensive, in-depth research before drawing a final conclusion, the empirical results obtained from our early implementation for two embedded processors show that relatively simple heuristics may be effective all across target architectures although each target also has its own demands for unique heuristics specifically tailored for its architecture.

2 Motivation

In heterogeneous register architectures, the relation between registers and instructions is tightly coupled. When we select an instruction, somehow we should also determine operand registers. Thus, integrated code generation techniques that cleverly combine these closely related code generation phases have proved to be very effective for embedded processors. One such example is the SPAM compiler [2]. Its code generator (*TWIF*) can generate highly optimized codes for a commercial embedded processor in most cases. Leupers and Marwedel [7] also presented an integrated code generation algorithm further extended from *TWIF* to deal with *parallel instructions*. However, our work differs from these two works in that their code generation algorithms are based on tree parsing. Therefore, the time complexity of these algorithms is practically linear; thereby eliminating the need for aggressive heuristics to minimize compilation time.

Researchers also investigated DAG parsing since it usually produces better quality code than tree parsing. Liao, et al. [8] developed a linear-time code generation algorithm based on DAG parsing. Their algorithm, however, comes with certain restrictions on target architectures; all ALU operations (θ) should be of the form: $acc \leftarrow acc \ \theta \ mem$, where acc represents an accumulator and mem a memory location. Likewise, Ertl [3] used DAG covering for code generation with certain restrictions on instruction selection grammars (or equivalently, target machine models). Although he showed typical regular architectures such as MIPS and SPARC are *DAG-optimal* (i.e., optimal code can be found with his algorithm), irregular architectures like those commonly found in embedded processors are not DAG-optimal.

Like us, Chess [10] has a code generator based on an *architecture description language* (ADL), not the tree grammars which have been used by many other conventional compilers. Their ADL called *nML* is used to describe not only instruction sets but also the underlying hardware data paths including pipelines. In addition, their instruction selection phase is integrated, though weakly, with the register allocation phase by using a technique called *late binding*. Although they also uses DAG covering, their covering algorithm uses predetermined heuristics based on branch-and-bound methods. We deem that these heuristics are not truly target-specific since they are rather more dependent on search and pruning strategies than specific features of the target machine.

To cap, the code generation problem is intrinsically intractable, so any realistic compilers inevitably need techniques to reduce compilation time. This is true embedded system compilers where fairly long compilation times are tolerable. Without such techniques compilation time may increase exponentially as the code size increases. As discussed above, the strategies chosen to reduce compilation time have been different from one compiler to another. Some compilers simplify or decompose a code generation problem into small sub-problems with manageable complexities. Others impose certain restrictions or apply predetermined heuristics in order to prune the enormous search space. As in compilers based on tree parsing, code generation problem is sometimes overly simplified so that the chances to obtain optimal codes are completely stymied even for relatively regular architectures. Also, predetermined, non-target-specific heuristics may behave properly for some targets while they do the opposite for others; that is, if they are too aggressively designed for a certain family of architectures, they may act adversely on other families of architectures. On the other hand, if a heuristic is too conservative, it may not reduce compilation time significantly.

Based on these observations, we attempt to avoid imposing any predetermined heuristics or restrictions on the target machine models. We rather customize heuristics for each individual target machine; that is, we apply different heuristics to each machine only after they are proved to be effective. As our compiler accumulates more knowledge about architectural features of various target machines and more effective heuristics for individual architectures, it can selectively apply a set of custom-fit heuristics even for new processors. Besides, considering the fact that architectural variations of embedded processors are

much wider than those of GPPs, this might be possibly the best strategy to meet the stringent performance requirement, yet completing compilation within a reasonable time. One challenging issue here is how we let the compiler automatically characterize a given target machine before applying heuristics. Our compiler currently identifies several *architectural parameters* from the given ADL description for accurate selection of heuristics.

3 Basic Code Generation

In this section, we present our basic code generation framework where, temporarily ignoring compilation time, we focus only on finding optimal code for a given basic block. Each block is given as a DAG $G = (V, E)$ where V represents operations or temporary storages, and edges in E the data flow. Our basic code generation process contains four subtasks: matching, covering, scheduling, and register allocation (for more details, see [9]). To find optimal codes, we exhaustively searches given DAGs. Matching finds all possible instruction patterns matched to G . Among the matching patterns, covering selects a set of instruction patterns that covers G . Then, scheduling chooses only valid sequences of instructions from the covering patterns. Finally, register allocation determines the instruction sequence which has a minimal cost based on our cost equation.

3.1 Target Processor Modeling with XR²

Our ADL, called XR² characterizes target processors by specifying storage and instruction patterns. Storage consists of registers, memory and *register classes* [6]. A register class is defined by a set of registers that can appear as operands at the same position of instructions. For example, in a multiply instruction MPYA reg_i, reg_j, reg_k , TI320c54x restricts reg_i to be register T or accumulator A, reg_j to be A, and reg_k to be A or B. For these operands, we define three register classes: {AT} for reg_i , {A} for reg_j and {AB} for reg_k . We define 10 different register classes for TI320c54x to handle all operand types.

We often call processors with many register classes heterogeneous, since the types of operands can be different from one instruction to another. Meanwhile, processors like ARM9 or SPARC have a few classes of general-purpose registers (*e. g.* ARM9 has only one register class). These processors are called homogeneous.

XR² describes an instruction as a *register transfer list* (RTL), similar to [1]. Each RTL is a list of register transfers (RTs), which can be executed simultaneously: $instruction \equiv RTL \equiv \{RT_1, RT_2, RT_3\}$. Each RT is a single-cycle operation corresponding to a single expression of the form: $lvalue = rvalue$, where the $rvalue$ is an expression with 1 or 2 operands and $lvalue$ is a location to store the result of $rvalue$. Operands can be either registers, register classes or memory.

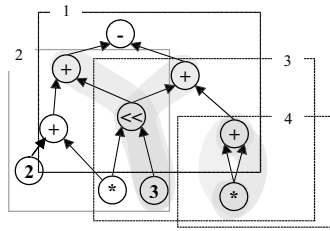
3.2 Matching

After we model target processors with a set of RTLs, we transform every RT into a tree form. The matching phase, taking codes represented in DAGs, matches these RT trees against subgraphs of the DAGs. Matching one tree against another is a time-consuming, tedious process since it requires many repetitive tree walks. In order to minimize this overhead, we translate each RT tree into a string, called a *signature*, where each element corresponds to a node in the tree enumerated in a breadth-first search order. To eliminate ambiguity due to not specifying tree leaves, we assume that a signature should be of a complete binary tree form. Then, signatures include “_” representing an empty node.

All signatures are then transformed into a hash table in order to implement fast matching between DAGs and RT trees. To hash the table, we use the root node value of an RT tree as a key. This implies that any RTs with the same root node value will collide in the table. To resolve a collision, all RTs hashing to the same hash entry are chained together with unique integer values each representing the shape of the corresponding RT tree and the types of its nodes. Figure 1 demonstrates an example of the pattern matching process. The table (b) shows the signatures generated from RT trees. Since RTs are matched against all subgraphs of the input DAG, every node in the DAG should have its own signature representing the subgraph rooted at that node. However, the signature does not always have to represent the whole subgraph particularly when the graph is too large. This is mainly because RT trees are relatively small, usually with depth of less than three, compared to input DAGs. This fact enables us to apply a target-specific heuristic that limits the depth of the subgraph in DAGs to the maximum depth of target architecture’s RTs. Table (a) in Figure 1 shows the signatures for the subgraphs of depth three, each enclosed in a rectangular window displayed in the DAG diagram, since the maximum depth of RT trees is three (Table (b) in Figure 1). Notice here that subgraphs of DAG, unlike RT trees, may have shared nodes as a result of common subexpression elimination. Therefore, shared nodes may appear multiple times in DAG signatures.

3.3 DAG Covering

Our DAG covering algorithm enumerates every possible cover by exhaustively searching the DAG. In the matching phase, we annotate every node n in DAGs with a *candidate set* of RTs that matched with subgraphs rooted at n . Then, our algorithm exhaustively traverses annotated DAGs in DFS, expanding *search trees* with the matched RTs in candidate sets. To illustrate this algorithm, consider the example in Figure 2, where the annotated DAG G is shown in (a) and the DAG search space, called the *search tree*, is shown in (b). The numbers in candidate sets stand for indexes of RTs in Figure 1 (b). For example, the number 3 in $\{3\}$ represents **sub** in the RT table. As soon as an RT r is selected at node n in DAG G , we first mark the visit tag for r in order to avoid selecting r again when, assuming n is a shared node, we reach n from another parent; thereby, avoiding generating code for a CSE again.



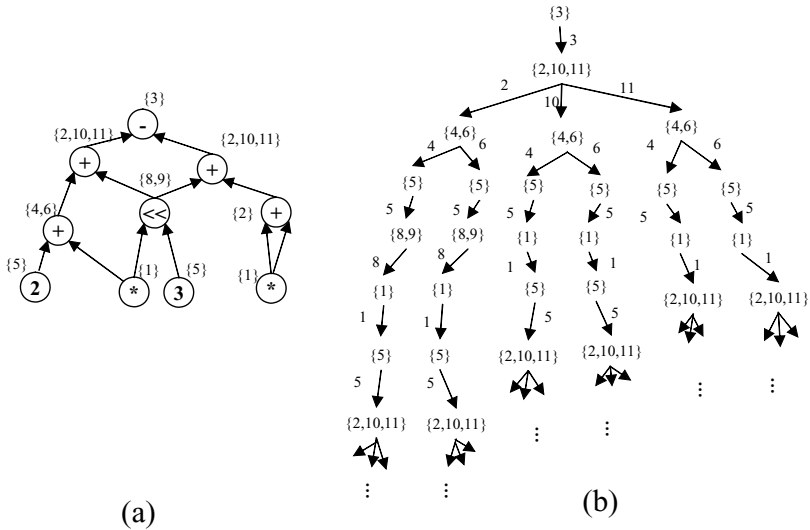
Window	Signature
1	- + + + << << +
2	+ + << i * * i
3	+ << + * i * *
4	+ * * _ _ _ _

(a)

No.	RT	Signature
1	mul	* r r
2	add	+ r r
3	sub	- r r
4	load	M
5	loadi	i
6	negate	~
7	mac	+ r * _ _ r r + * r r r _ _
8	shift	<< r r
9	shift-imm	<< r i
10	add-shift	+ r << _ _ r r + << r r r _ _
11	add-shift-imm	+ r << _ _ r i + << r r i _ _

(b)

Fig. 1. The matching process: The symbol *i* represents an integer constant, and the symbol *r* a wildcard variable that can match any symbol.



(a)

(b)

Fig. 2. The annotated DAG *G* for Figure 1 and the search tree built for *G*

3.4 RTL Scheduling and Register Allocation

Once we find a DAG cover at the leaf node of search tree, we perform two remaining tasks: RTL scheduling and register allocation. These tasks compute the minimal cost instructions for the selected DAG cover. First, each RTL is synthesized from one or more RTs along the path from the root to the leaf node in the search tree. Through this process, we may find *composite* instructions such as `mac` and `add-shift`. Then, RTL scheduling constructs a valid sequence of RTLs by mapping the RTLs in the cover to time slots $\{1, \dots, n\}$ without violating data dependence constraints. Finally, register allocation assigns registers to the operands of scheduled RTLs.

During register allocation, we consider the cost of scheduled RTL sequences to handle heterogeneous architectures. Heterogeneous architectures usually have special registers dedicated to certain instructions. If an instruction reads a source operand from a dedicated register that can only be used with special instructions, an extra move will be required to transfer a value in dedicated register to an appropriate source register. The cost of this transfer can be known only after register allocation is complete. As a result, the cost of the scheduled RTL sequence includes not only the total sum of the cost $c(r_i)$ of each RTL but also the additional costs for those extra moves $c(m_i)$ as well as spills $c(s_i)$:

$$\text{cost}(\text{code}_{\text{RTL}}) = \sum c(r_i) + \sum c(m_i) + \sum c(s_i),$$

For a given DAG, there will be multiple DAG covers and multiple valid instruction sequences. Thus, minimal cost schedule is determined only after traversing all the paths in a search tree.

4 Target-Specifically Speeding-Up Code Generation

In the previous section, we discussed how we generate the best quality codes by exhaustive search. Here we discuss our strategies to reduce compilation time. Our basic strategies that so far have been made are largely divided into two-fold:

- reduce the number of covers by slashing the DAG search space.
- reduce the running time of RTL scheduling and register allocation.

The heuristics we introduce in this section may loose opportunities to find the best quality codes, if they are applied without target-specific consideration. We, however, cautiously apply the heuristics when our target architectures allow zero or very small degradation of code quality.

4.1 Reducing the DAG Search Space

To reduce the search space, we consider a few target specific heuristics. Figure 3 (a) demonstrates one such case where a single `MAC` RT covers a subgraph whose nodes are also individually covered by a pair of smaller `ADD` and `MUL` RTs. In this case, the compiler usually has two choices for DAG covering between a large *clustered* RT and a combination of small RTs. Considering the formula for $\text{cost}(\text{code}_{\text{RTL}})$ introduced in the previous section, the decision should be made

based on the individual costs of its three components which are usually target dependent. To explain our heuristics, we assume an imaginary architecture with following condition:

$$c(r_{\text{MAC}}) < c(r_{\text{ADD}}) + c(r_{\text{MUL}})$$

This is true in many architectures, since MAC instruction is often implemented with single cycle in hardware. We also assume the move costs of the imaginary architecture are:

$$c(m_{\text{MAC}}) = c(m_{\text{ADD+MUL}})$$

Even if the processor is heterogeneous, most ALU instructions often operate on the same functional units using the registers in the same class. Thus, the above condition may be true for many architectures. We finally assume the spill costs of the imaginary architecture are:

$$c(s_{\text{MAC}}) \leq c(s_{\text{ADD+MUL}})$$

Since heterogeneous architectures tend to require dedicated registers for different instructions, they often use more registers and increase register spill. Meanwhile, homogenous architectures are likely to use the same registers used in MAC for ADD and MUL. Thus, spill cost may be the same for both cases (ADD and MUL may use fewer registers if situation allows maximum register reuse).

On our imaginary architecture, we can apply a target-specific heuristic called *clustering* for the case of Figure 3 (a). Clustering prefers one clustered RT to several small inclusive RTs in DAG covering and produces better code on our imaginary architecture. Clustering would eliminate a half of the search space below the clustered node.

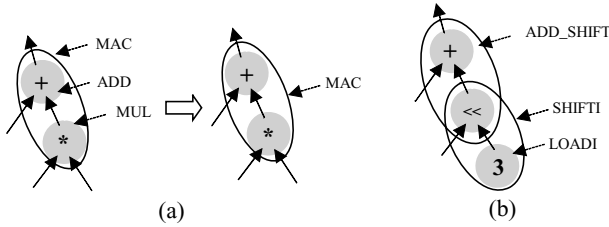


Fig. 3. *Clustering* examples of DAG covering: *shifti/loadi* are operations with immediate operands

Figure 3 (b) shows another case where a pair of clustered *intersecting* RTs cover a subgraph whose nodes are also individually covered by three small RTs. Although this case looks similar to the earlier one, they are in fact a bit different due to the intersection of the clustered RTs. In this case, the compiler has the choice of which RTs should be clustered: one choice with **ADD_SHIFT** and **LOADI**, or the other with **ADD** and **SHIFTI**. Although we also have the third choice without clustering (i.e., **ADD+SHIFT+LOADI**), this may be safely excluded for most machines as discussed with Figure 3 (a). In many architectures, we have

$$c(r_{\text{ADD_SHIFT}}) + c(r_{\text{LOADI}}) = c(r_{\text{ADD}}) + c(r_{\text{SHIFTI}})$$

Likewise, for homogeneous and most heterogeneous machines, we normally have

$$c(m_{\text{ADD_SHIFT+LOADI}}) = c(m_{\text{ADD+SHIFTI}}).$$

If the machine is homogeneous, the third condition is true:

$$c(s_{\text{ADD_SHIFT+LOADI}}) = c(s_{\text{ADD+SHIFTI}}).$$

If the machine is heterogeneous, this condition may vary on the types of instructions. These observations indicate that for homogeneous machines, the choice seems determined by the first condition. If the machine has no special hardware to satisfy the first condition (*i.e.* one clustered RT takes more than one cycle), the composite of fewer cycle clusters are better. Otherwise, either choice results in the same quality of code. For heterogeneous machine, however, the choice should be more determined by the third condition since many heterogeneous machines have CISC-style architectures meeting the first condition. In any situation, we can eliminate at least one choice, with possibly one more, out of the three; hence, slashing one third or two thirds of the search space below the cluster nodes. In all, for intersecting RTs, clustering should be selectively used depending on the target architecture.

Figure 4 depicts choices for a shared node in DAG, which is introduced from CSE. In this case, the compiler has two choices depending on whether clustering is applied or not. If clustering is used, the effect would be logically tantamount to splitting the original DAG at the shared node as shown in Figure 4 (a). Otherwise, three small RTs will be chosen as shown in Figure 4 (b). From these figures, we can realize that for many architectures, we have

$$c(r_{\text{MAC+MAC}}) < 2c(r_{\text{ADD}}) + c(r_{\text{MUL}}).$$

Similarly to earlier cases, we have

$$c(m_{\text{MAC+MAC}}) = c(m_{\text{ADD+MUL+ADD}}).$$

Due to the same reason for Figure 3 (a), we have for homogeneous architectures

$$c(s_{\text{MAC+MAC}}) \geq c(s_{\text{ADD+MUL+ADD}}),$$

and for heterogeneous architectures

$$c(s_{\text{MAC+MAC}}) \leq c(s_{\text{ADD+MUL+ADD}}).$$

In all, this case is similar to Figure 3 (a). Therefore, when determining whether clustering should be used, the compiler can examine the same architectural parameters as the case for Figure 3 (a). If it could find the solution deterministically at this point, then it could reduce the search space by a quarter below the CSE node.

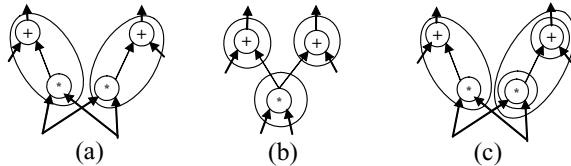


Fig. 4. Effects of different choices for a CSE node

Even when the compiler fails to locally determine whether clustering is useful for this case, it still has another chance to partially reduce the search space with *partial clustering*. When the compiler visits this subgraph through the search

tree paths, it has two possible search tree paths for the two left nodes: choosing one `mac` RT or a pair of `add` and `mul` RTs. If it chooses the former shown in Figure 4 (c), then as we reasoned in Figure 3 (a), choosing another `mac` RT for the right nodes should be obviously better than choosing a pair of RTs. Originally without this heuristic, we could have four search paths (i.e., 2×2) spanning from this subgraph. Thus, this partial clustering still would help us to reduce the search space by half below the CSE node.

4.2 Speeding up Register Allocation

Since register allocation is invoked for every valid RTL sequence, accelerating register allocation will lead to the reduction of the overall compilation time. Since register allocation for GPPs were already well addressed in the past literature, we focused more on register allocation for heterogeneous register architectures. When we design heuristics specifically for these architectures, we have considered the following aspects:

1. Since each instruction is tightly coupled with registers, we have few choices in registers once a certain RTL is selected earlier.
2. Generally, only a small number of registers are available to each instruction.
3. Many embedded processors have fast on-chip software-controllable memory.
4. Since the register allocator is repeatedly invoked, its speed may become more important than its effectiveness.

Based on these considerations, we have implemented a localized register allocation algorithm where we simply find a physical register for a variable based on how it is defined and used in the RTL code (See [9] for detailed algorithm). In our algorithm, for each unallocated input variable, we identify its register classes in the RTLs that define (*lhs*) or use (*rhs*). If there is a free register that exists in both classes, then that register is allocated. Otherwise, we need a move instruction to copy the result to an appropriate register that can be later used as source register. If registers in the desired class are all occupied, one of them are forced to spill. Since registers allocated to shared nodes are likely to spill due to their long life time, we first look up spill candidates from those registers. This will help reduce overall spill cost. The registers assigned to shared nodes in DAGs are not immediately available for reuse even after one of their parents use the value, since other parents still need to use them. Thus, the life time of those registers may be prolonged, increasing register spills. To tackle this issue, we utilize the second and third considerations stated above. Since a heterogeneous architecture usually has only a few registers allocatable to each RTL, even a highly sophisticated algorithm that allocates registers globally throughout the whole DAG might suffer from frequent spills anyhow. Also if the architecture has on-chip memory with latency of one cycle, the spill cost could be almost ignorable. With these consideration, we try to speed up the register allocation by *localizing* the original problem. This has been achieved by splitting the life span of a register referenced in a shared node. Although this localization heuristic does not always produce the best code, it does for many real cases as in this example.

5 Case Studies

To validate the effectiveness of our approach, we choose two extreme types of embedded processors. One is fairly regular and homogeneous, and the other is highly irregular and heterogeneous.

5.1 ARM9

Although ARM is a renowned manufacturer of embedded processors for low power applications, their processors have more or less GPP-style structures with relatively regular and homogeneous RISC architectures. As one of their processor series, ARM9 has one ALU and sixteen general-purpose registers. Due to its homogeneous architecture, many of traditional techniques developed for GPPs should be effective for ARM9 as well. Thus, conventional wisdom would hold here that our code generation approach might be too expensive for ARM9. Our recent experiment, however, reveals that this regular and homogeneous structure in fact provides us with much wider chance to apply powerful heuristics, which helps us reduce the compilation time dramatically. In this sense, we consider our approach is adaptive as compared to most of previous approaches.

On top of this, these target specific heuristics can be automatically extracted by testing several architectural parameters from the XR² architecture description; thereby, imposing no extra burden on the user to manually tailor the code generator for the target machine. As stated in Section 4.1, some of these parameters include the instruction cycle counts, the register classes and the amount of resource needed for each instruction. As for ARM9, the cycle counts vary on the types of instructions. Even it can be different for the same operation but with different source operand types. Another feature of ARM9 is that executing a composite instruction is better in terms of time and code size since it always takes no more cycles than sequentially executing multiple simple operations and the size of all instructions is the same 32 bit.

These architectural features of ARM9 make the best candidate to apply the clustering technique to the cases like those in Figures 3 (a). More formally, we can define the conditions for clustering as follows: *Let $r_0, \dots, r_n$ be the RTs matched against the subgraph g , and r_0 the largest RT to which all other RTs $r_j, j \neq 0$, are included. Then, these n RTs can be clustered into r_0 if (1) r_0 does not intersect with any other RTs except $r_1, \dots, r_n$, and (2) g does not have a shared node except at the root.* The second condition enforces g to be in a tree form. This enforcement is necessary since it otherwise would mean that there is an RT r' matched against some other subgraph next to g which needs the input from one of RTs $r_j, j \neq 0$. It in turns means that r_j cannot be clustered into r_0 because it must be used to emit an instruction that produces the input for r' . Another architectural factor that contribute curtailing the DAG search time for ARM9 is its RISC-like instruction set; that is, there are only a few RTs that can match each DAG node. We have also found that register allocation for ARM9 is fairly simple and fast because it has homogeneous register files. Therefore, we did not make a special effort to reduce γ in the formula for the compilation time. As been expected

from the observations on ARM9, we have found in our experiments that our exhaustive search time has been substantially reduced by clustering.

5.2 TI320c54x

TI320c54x is a digital signal processor (DSP), which like many other DSPs has a CISC-style structure with an irregular and heterogeneous architecture. Thus, its instructions are tightly bound to its special registers. It has not only an ALU for data processing but also a special functional unit, called an AGU, for address generation.

It has a special hardware to efficiently support various complex composite instructions. So, in the case of TI320c54x, the first condition in Section 4.1 always favors one clustered large RT to many small RTs for DAG covering in terms of time and code size. Also, our analysis reveals that the move cost usually stays the same regardless of clustering. This is mainly because each instruction has a unique requirement for registers as its source and destination. A clustered instruction, say **MAC** in Figure 5 (a), usually uses the same functional unit as its subinstructions, say **MUL** and **ADD**, thereby being bound to the same register class. In the example, **MAC** reads the register in the **T** class (with additional memory operand **M**) and writes one in the **AB** class. Likewise, **MUL** also reads the one in the **T** class and **ADD** writes one in the **AB** class. Consequently, the move effect between the subgraphs neighboring this subgraph would be the same regardless the choice of RTs for this graph. The spill cost, as the third condition, also favors clustering for TI320c54x due to the same reason. For instance in Figure 5 (a), running **MUL** and **ADD** RTs does not save one register as compared to running one **MAC** RT since **MUL** and **ADD** require different registers and hence cannot reuse one of their registers between them. In the case of Figure 5 (b), clustering even reduces register pressure because **ADD_SHIFT** needs one register from the **AB** register class while **ADD** needs two registers from the same class. By checking all three architectural conditions, the compiler can find that the same clustering heuristic useful for ARM9 is useful for TI320c54x as well.

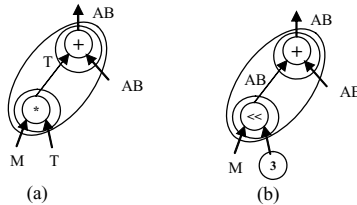


Fig. 5. Clustering in TI320c54x

Another heuristic in addition to clustering is *separate matching* of ALU and AGU instructions on the DAG. In DSPs, simple arithmetic operations like **ADD**/**SUB** can use the AGU and the ALU interchangeably. This in fact is a main cause of the explosive growth in the search space for TI320c54x, as reported in

Section 5.3, since the candidate set for a DAG node with simple operation types may include RTs using both the units. But we have found that the TI's heterogeneous structure helps us here again to reduce the search space. In TI320c54x, AGU instructions use register classes different from those ALU instructions use. Thus, if an ALU instruction lies next to an AGU instruction in the code with data dependence between them, then an extra move should be inserted to transfer the data. This fact gives us an insight to use a heuristic scheme that in the DAG, if a node is matched to an ALU/AGU RT, its parent node(s) should be also matched to an RT using the same ALU/AGU in order to avoid extra moves. Although this separate matching scheme may increase register pressure, this would not be the case for TI320c54x since the machine allows a single cycle on-chip memory access so using as operands memory instead of registers does not degrade the performance. Another architectural factor supporting this scheme is that the instructions with the same operator type supported from both the units have the same cost; therefore, even if an ALU instruction is used for typical AGU operations or vice versa, the overall instruction cost should be the same.

5.3 Experimental Results

Our basic code generation framework augmented with the target-specific heuristics has been implemented in our compiler infrastructure, called *Soargen*. The goal of *Soargen* is to provide a retargetable compiler that can generate a good quality code for various types of application specific instruction-set processors. Figure 6 shows the overall organization of *Soargen*. Taking as input the architecture description in XR² and a C program, it transforms them into the intermediate representation (IR) in a DAG form. By matching these two in *Soargen* IR, the final target code is generated as described in Section 3. The code is executed either directly on the target machine or on our retargetable simulator that can also be built from the same architecture description in XR².

To test our code generator, we selected basic blocks from the MediaBench and DSPstone benchmark programs. The programs were compiled on an 1GHz Pentium III with 256MB RAM and executed on the two target processors in Section 5. When we chose basic blocks, we had two criteria : the complexity of instruction patterns and the block size. Some blocks were chosen to evaluate our code generator's ability to find complex composite instruction patterns from DAGs since they have operations that can be translated to such complex instructions on the target machines. Along with small blocks, some other larger blocks were also chosen to see the growth rate of compilation time according to the increase in code size.

In our experiments, each basic block was translated into a DAG form and given to the code generator without any serious optimizations. The performance was measured for each code in two aspects: code quality and compilation time. Table 1 shows how much the heuristics improved the code quality and compilation time on ARM9. For this, two versions of code were generated: one with the basic code generation scheme and the other with the same scheme plus the additional heuristics described in Section 5.1. If the compiler could not generate the

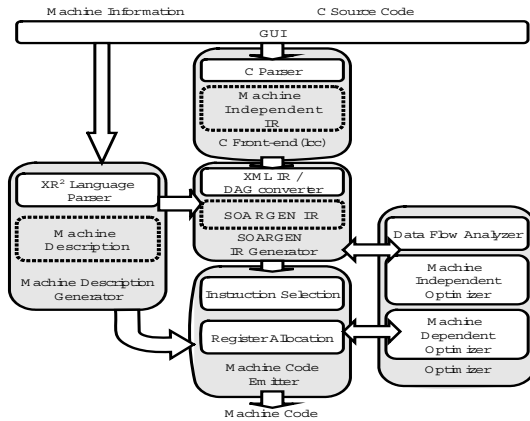


Fig. 6. Retargetable compilation in Soargen

code within that period of time, then we stopped it to emit the best code from those ever being selected. The time ∞ in the table denotes that the compilation time for the code exceeded the 30 minutes limit.

Table 1. Performance comparison between the basic code generation algorithm with/without heuristics on ARM9

code	tried DAG covers		compile time (sec)		execution time (cycles)		code size (words)	
	basic	improved	basic	improved	basic	improved	basic	improved
convolution	21952	14	209	0.1	65	65	11	11
fir	115112	84	∞	0.66	132	114	18	16
lms block 1	129802	84	∞	0.63	124	110	17	15
lms block 2	100128	14	∞	0.41	111	98	14	13
lms block 3	18816	6	342	0.01	68	68	11	11
n_complex_updates	61234	16	∞	0.55	466	328	67	47
adpcm init	96002	5	∞	0.13	224	149	32	22
idctrow 1	97664	6	∞	0.15	241	169	38	26
idctrow 2	10231	18200	∞	∞	1452	964	274	172

Not surprisingly, the table shows that for most benchmark programs, our compiler, with no heuristic schemes, consumes more than 30 minutes to traverse an extremely large amount of search space. In all programs, heuristics reduce the compilation time significantly. However, for `idctrow2`, both schemes could not find the optimal code within 30 minutes mainly because the program has very large basic blocks with about 30 C statements. Even though we have ∞ for `idctrow2` with heuristics, the increase in the number of tried DAG covers proves the effectiveness of heuristics because this indicates that they reduced the size of each DAG cover and were able to try more covers within the same time limit. Notice that the execution time and size of the code generated by the basic scheme are worse than the code generated by the scheme with heuristics. This is because without heuristics the compiler could not try all DAG covers, thereby ending up with suboptimal codes.

Table 2 shows our performance results on TI320c54x. Similarly to ARM9, the heuristic schemes were effective for every code on TI320c54x although as discussed in Section 5.2, we needed more heuristics for this irregular machine. Therefore, most of our earlier analysis made on the performance results for ARM9 are also valid on those for TI320c54x. One noticeable difference here, however, is that the numbers of DAG covers tried are generally larger than those for ARM9. These are mainly due to the TI’s CISC-like instruction set. This reason actually contributed the increase of the search space and consequently the explosion of compilation time. The table shows that unlike on ARM9 the basic scheme could not try all DAG covers for `convolution` and `lms_block_3` which are the smallest of all the programs. Even with heuristics, we could not complete compilation within the time limit for three programs.

Table 2. Performance comparison between the basic code generation algorithm with/without heuristics on TI320c54x

code	tried DAG covers		compile time (sec)		execution time (cycles)		code size (words)	
	basic	improved	basic	improved	basic	improved	basic	improved
convolution	174500	432	∞	2	46	29	28	15
fir	215100	224	∞	2	74	63	39	21
lms block 1	159700	192	∞	12	67	46	40	20
lms block 2	147300	128	∞	0.8	57	28	34	14
lms block 3	224200	80	∞	1	49	37	28	16
n complex updates	126700	78400	∞	∞	204	145	131	75
adpcm init	140900	16	∞	0.2	141	35	73	35
idctrow 1	67400	79600	∞	∞	152	66	92	36
idctrow 2	16700	19600	∞	∞	603	450	462	270

6 Conclusion

The purpose of this work is to examine how effectively we can use target specific information for our code generation. Due to the increasing complexities of modern embedded processors, many compiler writers are struggling ever harder to develop powerful, yet also practically fast, code generation techniques for these processors. The best approach for this would be to manually tailor the code generator into the target processor with target-specific architectural information. In our approach, we attempt to automate this tedious and time-consuming process by designing our compiler to extract such information from the architecture description in the XR² language.

For this, we have identified case by case several architectural parameters that the compiler should check from the architecture description. Each case has been carefully examined to see how certain heuristics are affected by target specific information when they try to reduce the huge search space for DAG covering. Also we described a register allocation algorithm that uses heuristics capitalizing on certain architectural features of embedded processors. Although this work requires further research to complete, we present empirical evidence opening the

door to some possibilities that target-specific information may be automatically extracted to develop various heuristic schemes that ease this notoriously complex code generation problem for embedded processors.

References

1. A. Appel, J. Davidson, and N. Ramsey. The Zephyr Compiler Infrastructure. Technical report, University of Virginia, 1998.
2. G. Araujo and S. Malik. Code Generation for Fixed-point DSPs. *ACM Transactions on Design Automation of Electronic Systems*, 3(2):136–161, April 1998.
3. M. Ertl. Optimal code selection on dags. In *POPL*, 1999.
4. P. Faraboschi, G. Desoli, and J. Fisher. The latest word in digital and media processing. In *IEEE Signal Processing Magazine*, March 1998.
5. S. Hanono and S. Devadas. Instruction Selection, Resource Allocation and Scheduling in the AVIV Retargetable Code Generator. In *Design Automation Conference*, June 1998.
6. S. Jung and Y. Paek. The Very Portable Optimizer for Digital Signal Processors. In *International Conference on Compilers, Architectures and Synthesis for Embedded Systems*, pages 84–92, Nov. 2001.
7. R. Leupers and P. Marwedel. Instruction Selection for Embedded DSPs with Complex Instructions. In *European Design Automation Conference*, Sep. 1996.
8. S. Liao, K. Keutzer, and S. Tjiang. A New Viewpoint on Code Generation for Directed Acyclic Graphs. *ACM Transactions on Design Automation of Electronic Systems*, 3(1):51–75, 1998.
9. Y. Paek, S. Oh, S. Jung, and D. Park. Towards Simultaneous Instruction Selection and Register Allocation in DAGs for Embedded Processors. Technical report, Seoul National University, 2003.
10. J. Van Praet, D. Lanner, W. Geurts, and G. Goossens. Processor Modeling and Code Selection for Retargetable Compilation. *ACM Transactions on Design Automation of Electronic Systems*, 6(3):277–307, July 2001.

Extraction of Efficient Instruction Schedulers from Cycle-True Processor Models^{*}

Oliver Wahlen¹, Manuel Hohenauer¹, Gunnar Braun², Rainer Leupers¹,
Gerd Ascheid¹, Heinrich Meyr¹, and Xiaoning Nie³

¹ Integrated Signal Processing Systems
Aachen University of Technology, Aachen, Germany
`lisa@iss.rwth-aachen.de`

² CoWare Inc., Aachen, Germany

³ Infineon Technologies, Munich, Germany

Abstract. This paper proposes a technique for extracting an instruction scheduler from a LISA processor description. The generated tool reads unscheduled, sequential assembly code from a C compiler. It schedules the instructions using an efficient backtracking scheduling algorithm that allows automated delay slot filling and utilization of instruction level parallelism. For an industrial network processor and a multimedia VLIW architecture the quality of the generated assembly code is compared to that of compilers with handwritten scheduler specifications.

1 Introduction

Compared to ASICs, DSPs, μ Cs or general purpose processors, so-called application specific instruction set processors (ASIPs) provide a surpassing tradeoff of computational performance and flexibility on the one hand and power consumption on the other. Therefore, these processors that are designed to execute specific tasks very efficiently can be found in many embedded systems of today's mobile or automotive applications.

Unfortunately, compared to assembling systems with standard processors, a design containing ASIPs is of much higher development complexity. The additional design and verification time for a custom processor is one reason for the increasing industry demand on design automation. Beside the hardware model, software tools like assembler, linker, and simulator have to be written, too. If hardware and software are available, profiling results are acquired that usually lead to architecture modifications making the processor more efficient. To ensure consistency between hardware and software models, all software development tools need to be modified, too. The duration of these so-called exploration cycle determines the overall design time of the processor.

The algorithm to be performed by the ASIP is usually specified by algorithm designers in a high level language like C. If there is no C compiler available, the time for converting the C code to assembly is also part of the architecture

^{*} This work has been partially supported by CoWare Inc.

exploration loop. It was shown in [22] that the overall design time can be significantly reduced by introducing a C compiler into the exploration loop. Besides a drastic reduction in implementation and verification time, the availability of a C compiler also increases the system reusability for similar applications.

An architecture exploration loop that includes a C compiler can only be beneficial if there is a high degree of design automation. The LISA processor design platform (LPDP) [1], which is commercially available from CoWare Inc. [5], satisfies this demand by automatically generating most software tools and a hardware model (VHDL or SystemC) from a single LISA processor architecture description. Ongoing work deals with generation of C compilers from LISA models.

This paper proposes a technique for generating a so-called mixedBT backtracking instruction scheduler from a LISA processor description. Section 2 gives an overview of work related to compiler generation from processor descriptions and other scheduling techniques. The mixedBT scheduler is implemented in a post-pass tool called *lpacker* that reads the output of a CoSy [3] based C compiler. The complete tool chain including *lpacker* is subsumed in section 3. Section 4 outlines the LISA processor description language and explains how compiler related information can be extracted. The algorithm of the mixedBT scheduler is presented in detail in section 5. The quality of the scheduler generator is evaluated in section 6 by comparing the output of compilers with handwritten scheduler specifications to code generators that utilize our automatically generated post-pass scheduler. The paper concludes with an outlook in section 7.

2 Related Work

A detailed overview of work related to compiler generation from processor architecture description languages (ADLs) or compiler specifications is given in [16]. An environment that is mainly useful for VLIW architectures is **ISDL** [9]. It hierarchically describes the processor and lists invalid instruction combinations in a *constraints section*. This list becomes very lengthy and complex for DSP architectures like the Motorola 56k. Therefore ISDL is mainly useful for “orthogonal” processors.

Trimaran [21] is capable of retargeting a compiler for a very restricted class of VLIW architectures called *HPL-PD*. The tool input is a manual specification of processor resources (functional units), instruction latencies, etc. The focus of the project is more on compiler technology than on supporting a broad range of architectures.

From a **FlexWare2** [18] description, an extension of the CoSy [3] environment can be retargeted producing good code quality. Unfortunately, for the generation of the other software tools FlexWare2 requires separate descriptions. This redundancy introduces a consistency/verification problem.

The concept for scheduler generation in **EXPRESSION** [8] and **PEAS-III** [12] is quite similar to our approach: Both environments extract structural information from the processor description that allows the tracing of instructions through the pipeline. Instructions are automatically classified by their temporal I/O behavior and their resource allocation. Based on this information, a

scheduler can be generated. In PEAS-III, all functional units that are used to model the behavior of instructions are taken from a predefined set called flexible hardware model database (FHT). **EXPRESSION** does not have this restriction. Unfortunately, no results related to the quality of the scheduler generated from **EXPRESSION** have been published.

MIMOLA [15] traces the interconnects of functional units to detect resource conflicts and I/O behavior of instructions. For non-pipelined architectures, it is possible to generate a compiler (called **MSSQ**) which also includes an instruction scheduler. The abstraction level of **MIMOLA** descriptions is very low, which slows down the architecture exploration cycle.

The **CHESS** [14] code generator is based on an extended form of the **nML** [7] **ADL**. Similar to the **MSSQ** compiler, the scheduler uses the instruction coding to determine which instructions can be scheduled in parallel. In contrast to **MSSQ**, the **CHESS** compiler can be used to generate code for pipelined architectures. This is achieved by manually adding latency information (e.g. number of delay slots) of the instructions. **CHESS** is primarily useful for retargeting compilers for DSPs.

The Mescal group as part of the Gigascale Research Center recently proposed a so-called operation state machine **OSM** [19] based modeling framework. **OSM** separates the processor into two interacting layers: an operation and timing layer and a hardware layer that describes the micro-architecture. A StrongARM and a PowerPC-750 simulator have been generated. It is stated that the provided information can also be used to generate compilers.

The novel mixedBT scheduler that is retargeted from a **LISA** description is an improvement of the **operBT**/**listBT** backtracking schedulers [2]. In contrast to a list scheduler, these schedulers are able to automatically fill branch delay slots by analyzing negative edge weights in the data dependence graph.

Another alternative to implement delay slot filling is in a peephole optimization phase just before emitting the assembly code. Unfortunately most of these techniques are processor specific and not easily retargetable. Other approaches utilize integer linear programming (ILP) for assembly code optimization (e.g. [6]). ILP is retargetable and for basic blocks or simple loop structures it leads to good code quality. Since ILP is an NP-complete problem the performance of the code generator is low, though.

3 System Overview

A system overview of the retargetable code generator and the related software tools is depicted in figure 1.

The **CoSy** [3] compiler development system is used to generate a C compiler. This generated **lisacc** compiler parses the C code, applies typical high level optimizations, utilizes a tree pattern matcher for code selection and conducts global register allocation. The output of **lisacc** is unscheduled sequential assembly code. This means that each assembly instruction contains an instruction class identifier and information about the resources (e.g. registers, memory) that are read or written. From this input, the **lpacker** tool creates a directed

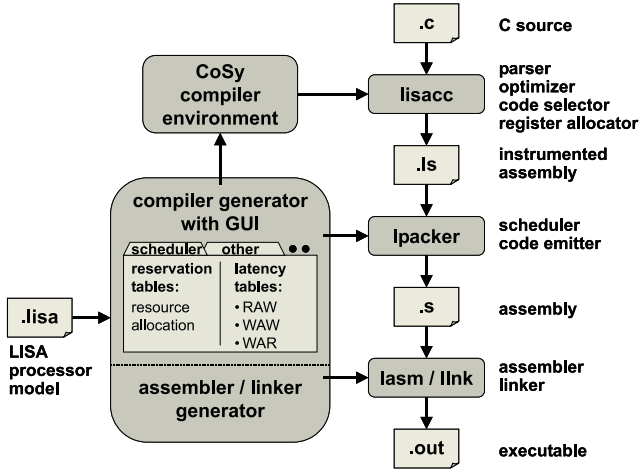


Fig. 1. The code generator tool chain

acyclic graph (DAG) as depicted in figs. 5 and 7. This dependence DAG is fed into the mixedBT scheduler which is implemented in the lpacker tool. The result is LISA model compliant assembly code which is read by the LISA generated assembler/linker.

The scheduler generation from a LISA processor model is completely automated. In addition, all compiler related analysis results are visualized in a graphical user interface (GUI) and can optionally be overridden by the user. The benefit of this opportunity was demonstrated in [22]: It is possible to start the processor design with a very simple LISA model that mainly describes the instruction set but no temporal behavior (i.e. the pipeline is not modeled). The compiler specification can be used to model instruction latencies, register file sizes, etc. Thus, the impact of major architectural changes can quickly be profiled through the compiler. This methodology can significantly speed up the architecture exploration phase. Another reason for the GUI is the opportunity to override analysis results that are too conservative. This might occur if the architecture contains unrecognized hardware to hide instruction latencies.

4 Extracting Scheduling Information from a LISA Processor Description

For a given set of instructions, a scheduler decides which instructions are issued on the processor in which cycle. For instruction level parallelism (ILP) architectures, this does not only mean that the scheduler decides on the sequence in which instructions are executed, but it also arranges instructions to be executed in parallel.

As described in [13], the freedom of scheduling is limited by two major constraints: structural hazards and data hazards. Structural hazards result from instructions that utilize exclusive processor resources. If two instructions require the same resource, these two instructions are mutually exclusive. A typical example for structural hazards is the number of issue slots available on a processor architecture: It is never possible to issue more instructions in a cycle than the number of available slots.

Data hazards result from the temporal I/O behavior of instructions. They can be subdivided into read after write (RAW), write after write (WAW), and write after read (WAR) hazards. An example for a RAW dependency would be a multiplication that takes two cycles to finish computation on a processor without interlocking hardware followed by a second instruction that has to consume the result of the multiplication. In this case the multiplication has a RAW dependence of two cycles onto the second instruction, which means that the second instruction must be issued two or more cycles after the multiplication.

The extraction of structural hazards from a LISA architecture description has been explained in earlier work [23], which explains how to associate a reservation table with each instruction of the LISA processor description. The resources used in the table have no direct correspondence with the processor hardware and are therefore called *virtual* resources. Based on the reservation table technique, the scheduler can decide which instructions are allowed to be issued in the same clock cycle.

The automatic extraction of the RAW, WAW, and WAR data flow hazards from a LISA processor description is presented in this paper. This allows the generation of a complete instruction scheduler from a LISA model.

4.1 Structure of a LISA Description

LISA has been used to describe a wide variety of architectures including ARM7, C62x, C54x, MIPS32 4K, and to develop ASIPs like ICORE2 [22]. The following section outlines the structure of a LISA description as far as required to understand the scheduler analysis technique. An in-depth explanation of LISA and the related software tools is given in [1].

A LISA processor description consists of two parts: The LISA operation tree and a resource specification. The operation tree is a hierarchical specification of instruction coding, syntax, and behavior. The resource specification describes memories, caches, processor registers, signals, and pipelines. An example for a single LISA operation is given in figure 2. The name of this operation is `register_alu_instr` and it is located in the ID stage (instruction decode) of the pipeline `pipe`. The `DECLARE` section lists the sons of `register_alu_instr` in the operation tree. `ADD` and `SUB` are names of other LISA operations that have their own binary coding, syntax, and behavior. As one can see the respective sections are referenced by the group declarator `Opcode` in the `CODING` and `SYNTAX` section. The `BEHAVIOR` section indicates that elements of the `GP_Regs` array resource are read, and the contents are written into pipeline registers. This means that the general purpose register file is read in the instruction decode stage.

```

OPERATION register_alu_instr IN pipe.ID
{
  DECLARE {
    GROUP Opcode = { ADD || SUB };
    GROUP Rs1, Rs2, Rd = { gp_reg };
  }

  CODING { Opcode Rs2 Rs1 Rd Ob0[10] }
  SYNTAX { Opcode ~" " Rd ~" " Rs1 ~" " Rs2 }

  BEHAVIOR {
    PIPELINE_REGISTER(pipe,ID/EX).src1 = GP_Regs[Rs1];
    PIPELINE_REGISTER(pipe,ID/EX).src2 = GP_Regs[Rs2];
    PIPELINE_REGISTER(pipe,ID/EX).dst = Rd;
  }

  ACTIVATION { Opcode }
}

```

Fig. 2. Example for a LISA operation

The ACTIVATION section describes the subsequent control flow of the instruction “through” the processor instruction pipeline. The LISA operation referenced by the group Opcode (i.e. either ADD or SUB) is eventually located in a subsequent pipeline stage, which means that it will be activated in a subsequent cycle. Thus, the ACTIVATION sections create a chain of operations as depicted in figure 3.

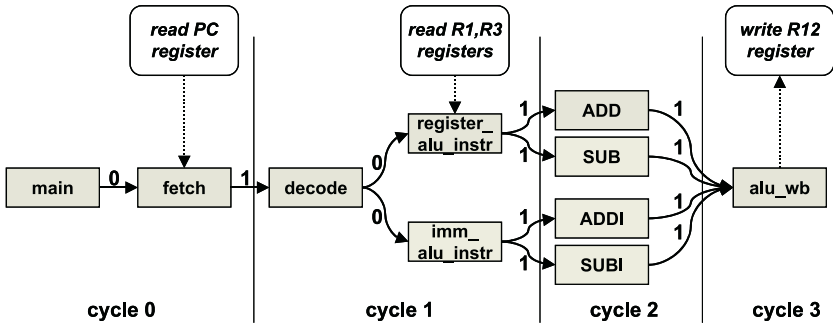


Fig. 3. Activation chains of LISA operations

4.2 Extracting Instruction Latencies

Based on the LISA activation chains, it can be analyzed when an instruction accesses processor resources: The starting point of all activation chains is the

main operation which has a special meaning. It is executed in every control step of the simulator and activates the operation(s) in the first pipeline stage (fetch) which in turn activate(s) operations of subsequent pipeline stages. For each instruction, the declared **GROUPs** are resolved (i.e. **Opcode** is assigned either **ADD** or **SUB**). This means that based on the activation chain, the execution clock cycle for each LISA operation can be determined. Furthermore it can be analyzed whether the C code in the **BEHAVIOR** section of the operations accesses processor resources of the LISA model. The analysis of activation chains differs from the trace technique used in other design environments (e.g. **EXPRESSION** [10]): Traces include information about which functional units are used by an instruction in a specific cycle, requiring modeling of functional units and their interconnects. In the LISA language functional units and interconnects are not modeled explicitly which significantly speeds up the architecture exploration phase.

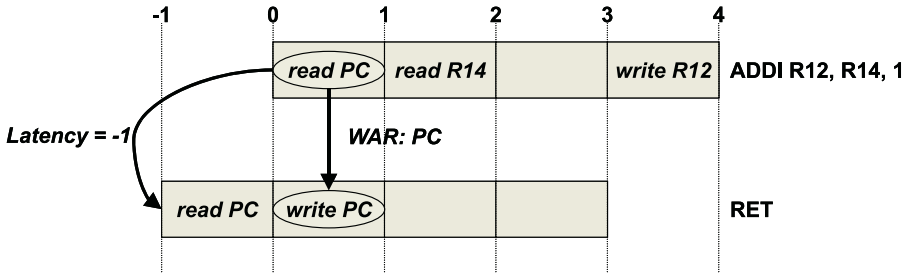


Fig. 4. Latency analysis of two activation chains

The access direction (read or write) and the resource names are organized in an instruction specific vector. Starting from cycle 0, each vector component represents a cycle that is required to execute the instruction. The vectors of two example assembly instructions are depicted in figure 4.

To schedule a sequence of instructions a DAG like the one depicted in figure 5 is constructed.

Each edge weight of the DAG represents a RAW, WAW, or WAR dependency between a pair of instructions. If there is more than one latency between two instructions (e.g. the second instruction reads and writes a register that is written by the first instruction), the maximum latency is taken.

If a second instruction I_2 reads a register resource R written by the first instruction I_1 , the RAW latency is calculated by the following formula: $RAW = last_write_cycle(I_1, R) - first_read_cycle(I_2, R) + 1$. The $last_write_cycle$ function iterates through the vector of I_1 and returns the greatest component index that indicates a write to R . Similarly, the $first_read_cycle$ function returns the first component index of I_2 that contains a read of R . The inherent resource latency is taken into account by the last addend: Since it takes one cycle to read

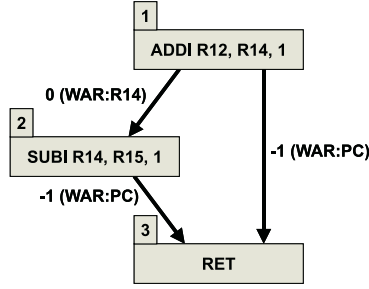


Fig. 5. Instruction dependency for which listBT gives suboptimal results

a value from a register that has been written to it, an addition of 1 is required. If two subsequent instructions write to R , the WAW latency is computed by $WAW = last_write_cycle(I_1, R) - last_write_cycle(I_2, R) + 1$. Here, the addition of 1 is needed because it is not possible that two instructions write a resource at the same time. If the second instruction writes R and the first instruction reads R the WAR latency is computed by: $WAR = last_read_cycle(I_1, R) - first_write_cycle(I_2, R)$.

An example for a WAR latency is depicted in figure 4. The `ADDI R12,R14,1` instruction reads the program counter (PC) in its cycle 0. In its cycle 1, it reads a source operand from register R14, and in cycle 3, it writes a result back to register R12. It is followed by a `RET` instruction that reads the PC in its cycle 0 and writes it in its cycle 1. This leads to a WAR latency $WAR : PC = 0 - 1 = -1$. Consequently, the `RET` instruction must be scheduled -1 or more cycles behind the `ADDI R12,R14,1`. The negative latency can be interpreted as an opportunity to fill the delay slot of the `RET` instruction: For the scheduler, it is possible to issue the `RET` one cycle before the `ADDI R12,R14,1`. This means that the activation chains can be used to automatically generate schedulers capable of delay slot filling. The CPU time required for analyzing the latencies in the scheduler generator is negligible.

5 Scheduling Algorithms

5.1 List Scheduler

Unfortunately, a conventional list scheduler [20] is not capable of filling delay slots. A list scheduler operates on a dependence DAG representing a basic block as depicted in figure 5. It selects one or more of the nodes that have no predecessor (the so-called ready set) to be scheduled into a cycle determined by a `current_cycle` variable. The scheduled nodes are removed from the DAG, the `current_cycle` is potentially incremented and the loop starts again. In the example of figure 5, `current_cycle` is initialized to 0 and the list scheduler would schedule instruction 1 which is the only ready node (i.e. it has no predecessor)

into that cycle. The node is removed from the DAG and instruction 2 becomes ready. If we assume that the underlying architecture has only a single issue slot, it is not possible to schedule any other instruction into `current_cycle` (which is still 0). Consequently, `current_cycle` is incremented. Since no latency constraint is violated, instruction 2 is scheduled into cycle 1. After another scheduling loop, instruction 3 is scheduled into cycle 2. Unfortunately, this `RET` instruction has a delay slot so the list scheduler has to append a `NOP` as the last instruction of the basic block.

A better schedule would be 1-3-2, which means that the delay slot of the `RET` is filled with one of the preceding instructions. To create this schedule, the scheduler must be able to revoke decisions on instructions being scheduled into certain cycles.

5.2 Backtracking Schedulers

An example of such a so-called backtracking scheduler is given in [2]. The paper presents two different backtracking scheduler techniques: The *operBT* scheduler and the *listBT* scheduler. Both schedulers assign priorities to the nodes of the dependence DAG. In contrast to all other schedulers, the *operBT* scheduler does not maintain a ready list. It utilizes a list of nodes not yet scheduled that is sorted by node priority. It takes the highest priority node from this list and schedules it using one of the following three scheduling modes:

- schedule an operation without unscheduling (normal)
- unschedule lower priority operations and schedule into `current_cycle` (displace)
- unschedule high priority operations to avoid invalid schedules and schedule an instruction into a so-called `force_cycle` (force)

The *operBT* scheduler has the drawback of being relatively slow due to a lot of unscheduling operations. To overcome this drawback, the *operBT* scheduler was extended to the *listBT* scheduler. This scheduler tries to combine the advantage of the conventional list scheduler (fast) with the advantage of the *operBT* scheduler (better schedule). The *listBT* scheduler does maintain a ready list. This means, only nodes that are ready can be scheduled. Unfortunately, the delay slot filling of the *listBT* scheduler does not work for all cases. Figure 5 is an example of a dependence DAG for which the *listBT* scheduler creates the following schedule after 11 scheduling loops iterations: (0) `ADDI R12,R14,1`; (1) `NOP`; (2) `RET`; (3) `MULI R14,R15,1`. The reason for the `NOP` is that in one of the schedule loop iterations the scheduler tries to schedule `MULI R14,R15,1` instead of the higher prioritized `RET`. This leads to a correct but suboptimal schedule.

5.3 mixedBT Scheduler

The *mixedBT* scheduler that we retarget from a LISA description is an improved approach of combining the advantages of the conventional list scheduler and the backtracking scheduler. The basic idea is to reduce the number of computational

intense instruction unscheduling operations by maintaining a ready list while still being able to switch to the better quality priority scheduling when applicable. To support both modes, a ready list and a list of nodes not yet scheduled are maintained. The pseudo code of the scheduling algorithm is depicted in figure 6.

```

initialize(node_priorities, unsched_list, ready_list);
loop(until_all_insns_scheduled)
    get_next_current_insn_to_be_scheduled(unsched_list, ready_list);
    for_current_insn_compute(early_cycle, late_cycle);
    force_cycle = max(attempted_cycle_of_current_insn+1, early_cycle);
    unforcefull_scheduled = false;
    loop(current_cycle ranging from early_cycle to late_cycle)
        try_schedule_current_insn_into(current_cycle);
        if(success)
            // this is normal scheduling
            update_lists(unsched_list, ready_list);
            unforcefull_scheduled = true;
            break;
        elseif((current_cycle >= force_cycle) and
            current_insn_has_higher_priority_than_conflicting_in(current_cycle))
            // this is displace scheduling
            unschedule_each_conflict(current_cycle);
            schedule_current_insn_into(current_cycle);
            update_lists(unsched_list, ready_list);
            unforcefull_scheduled = true;
            break;
        end if
    end loop
    if(unforcefull_scheduled == false)
        // this is force scheduling
        unschedule_each_conflict(current_cycle);
        schedule_current_insn_into(force_cycle);
        attempted_cycle_of_current_insn=force_cycle;
        update_lists(unsched_list, ready_list);
    end if
end loop

```

Fig. 6. Pseudocode for mixedBT scheduler

The initial priority of the DAG leaf nodes is equivalent to the cycles these instructions require to finish their computation. For all other nodes, the edge weights of any path from that node to any leaf node are accumulated. The priority of the leaf node is added to each sum. The maximum sum of all possible paths is the node priority. The `get_next_current_insn_to_be_scheduled` function decides from which list to take the next node that is to be scheduled. It takes the highest priority node from the list of nodes not yet scheduled if the priority is higher than any node priority in the ready list. Otherwise, the highest priority node from the ready list is scheduled next.

If there are only positive data dependencies, the ready nodes always have the highest priorities. For nodes that have zero latency, the function selects the father node. The operBT scheduler would potentially select the son here. This would most probably lead to an unscheduling of this node later on. If nodes are connected by a negative latency, the son has a higher priority. It will be scheduled first even if it is not ready. This speeds up the filling of delay slots.

There are still constellations of the data dependence DAG that can lead to sub-

optimal schedules for all scheduler types. An example is depicted in figure 7. Both instruction 1 and instruction 3 are ready and have the same priority. If

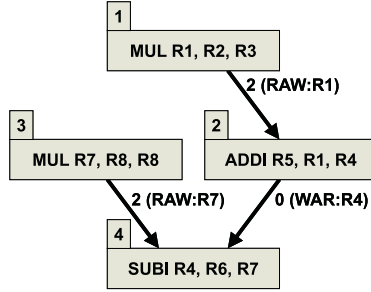


Fig. 7. Instruction dependency for which list scheduler can give suboptimal results

instruction 1 is chosen to be scheduled first the resulting schedule for all schedulers will be: 1-3-2-4. But if instruction 3 is chosen to be scheduled first this leads to 3-1-NOP-2-4. For this reason, the maximum path length to a DAG leaf is additionally stored for each node. If two nodes have equal priority, the node with the longer path length is chosen.

6 Results

To evaluate the quality of *lpacker*, we utilized several compilers to generate assembly code from C kernels for the PP32 network processing unit (NPU) from Infineon Technologies AG. Using the LISA assembler, linker, and simulator, performance and size of the generated code have been determined. The PP32 is the successor architecture of the PP16 [17]. It provides a multithreaded RISC core extended with special purpose instructions for bit manipulation and I/O control. The evaluated kernels were:

- frag:** IPv4 packet fragmentation for constant size (228 lines C code)
- tos:** extraction of the type of service field from an IPv4 header (29 lines C code)
- hwacc:** access to control registers/bits (30 lines C code)
- route:** IPv4 routing routines (258 lines C code)
- reed:** reed solomon encoder/decoder (749 lines C code)
- md5:** MD5 message-digest algorithm (612 lines C code)
- crc:** cyclic redundancy check (CRC) calculation (230 lines C code)

The results of a CoSy based C compiler for the NPU was taken as a reference (labeled *CoSy*) in figures 8 and 9. Here the native CoSy list scheduler has been used. We also retargeted the *lcc* [4] compiler for the PP32. The *lcc* backend does not contain a scheduler. Instead we used *lpacker* as a list scheduler to achieve

the *lcc+lpacker(list)* results. The elements labeled *CoSy+lpacker(list)* represent a CoSy based C compiler that generates unscheduled, sequential assembly code processed by lpacker working as a list scheduler. The same CoSy compiler was used for *CoSy+lpacker(mixedBT)* with lpacker running as a mixedBT scheduler.

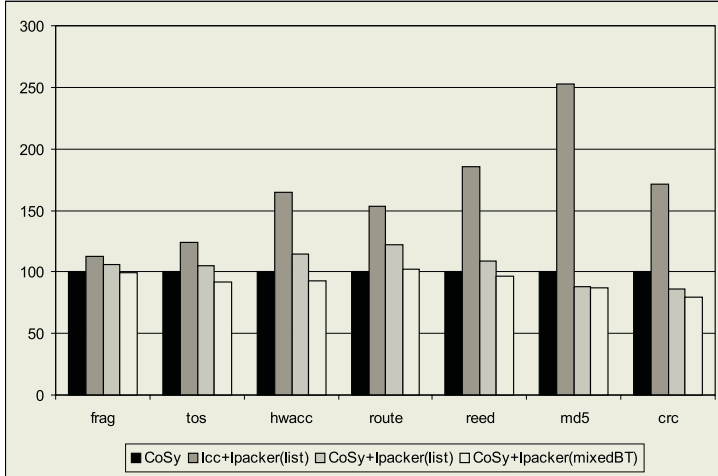


Fig. 8. Execution Cycles relative to CoSy compiler with handcrafted scheduler specification

Figure 8 demonstrates that the code quality of CoSy based compilers is much better than the one of the *lcc+lpacker(list)* code generator. The reason is the absence of most high level optimizations in the lcc compiler. The lcc data demonstrate that compared to the other optimizations of the CoSy environment the utilization of an instruction based backtracking scheduler leads to significant performance improvements. Since the CoSy compilers perform function inlining and heuristic loop unrolling the lcc generated code size can be smaller, though. The suboptimal code quality of the *CoSy+lpacker(list)* combination results from the lack of a multiplication unit in the PP32: In the CoSy compiler with handcrafted scheduler description, multiplications are matched with a very dense handwritten assembly routine. If the instructions of the multiplication algorithm are list scheduled by lpacker (*list-mode*), delay slots are filled with NOPs. If lpacker is used as a mixedBT scheduler (*CoSy+lpacker(mixedBT)*), there is an average improvement of 7.4% in cycle count and 5.2% in code size compared to the CoSy compiler with handcrafted scheduler specification and native list scheduler. One reason for the improvement is lpacker’s ability to efficiently fill

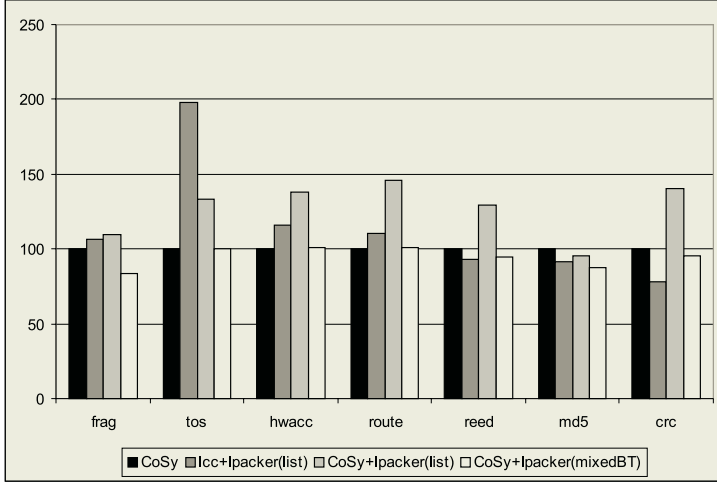


Fig. 9. Code size relative to CoSy compiler with handcrafted scheduler specification

delay slots¹. Another reason is the ability of lpacker to schedule on instruction basis. If no additional effort in IR lowering is spent, the scheduler that is part of the CoSy environment schedules blocks of assembly instructions: Each block is associated with a grammar rule of the tree pattern matcher. If for example the beginning of a function (i.e. the prologue) is matched by a sequence of assembly statements, this sequence is fixed and cannot be scheduled with the rest of the function body. These fixed blocks of assembly code can lead to suboptimal schedules.

The number of execution cycles of the code generated by the mixedBT scheduler is equal to the one generated by the operBT scheduler. The code generated by the listBT scheduler takes about 7% more cycles to run. For all benchmarked kernels the CPU time of all schedulers is below two seconds (Linux PC with Athlon XP 2000+ CPU). The mixedBT scheduler is about 20% slower than the listBT scheduler and two times faster than the operBT scheduler.

To demonstrate the applicability of lpacker for instruction level parallelism (ILP) architectures, a similar benchmarking was conducted for the ST200 VLIW processor [11]. The ST200 is a configurable VLIW core for media-oriented applications developed jointly by HP Labs and STMicroelectronics. The ST200 does not have branch delay slots. Nevertheless, the instruction based scheduling of lpacker leads to an average improvement compared to a CoSy compiler with handwritten scheduler specification of 3.4% for the cycle count and 7.3% for the code size.

¹ According to ACE, the upcoming version of CoSy will have better support for delay slot filling.

7 Conclusions and Outlook

It was demonstrated how an efficient scheduler can automatically be retargeted from a hierarchical processor architecture description. The generated mixedBT backtracking scheduler improves existing backtracking scheduling algorithms with respect to the quality of the generated assembly code and scheduler execution time. To evaluate the quality of the scheduler, the algorithms were integrated into the tool chain of the LISA processor design platform. Compared to a CoSy compiler with handcrafted scheduler specification, it was demonstrated that a mixedBT scheduler based CoSy compiler can improve the average execution time of the generated code by 3.4% to 7.4% dependent on the processor architecture. The average code size is improved by 5.2% to 7.3%. Improvements result from efficiently filling branch delay slots and scheduling instructions instead of fixed instruction blocks associated with tree grammar rules.

Future work will deal with the integration of the scheduling techniques into the CoSy environment and the evaluation of the generated schedulers for further processor architectures modeled in LISA. In addition, we are investigating retargeting techniques for further code generation phases, e.g. code selection.

References

1. A. Hoffmann, H. Meyr, and R. Leupers. *Architecture Exploration for Embedded Processors With Lisa*. Kluwer Academic Publishers, Jan. 2003. ISBN 1-4020-7338-0.
2. S.G. Abraham, W. Meleis, and I.D. Baev. Efficient backtracking instruction schedulers. In *IEEE PACT*, pages 301–308, May 2000.
3. ACE – Associated Computer Experts bv. *The COSY Compiler Development System*, <http://www.ace.nl>.
4. C. Fraser and D. Hanson. *A Retargetable C Compiler: Design and Implementation*. Benjamin/Cummings Publishing Co., 1994.
5. CoWare Inc. <http://www.coware.com>.
6. D. Kästner. *Retargetable Postpass Optimisation by Integer Linear Programming*. Verlag Pirrot, Oct. 2000. ISBN 3-9307-1455-8.
7. A. Fauth, J. Van Praet, and M. Freericks. Describing Instruction Set Processors Using nML. In *Proc. of the European Design and Test Conference (ED & TC)*, Mar. 1995.
8. Peter Grun, Ashok Halambi, Nikil D. Dutt, and Alexandru Nicolau. RTGEN: An Algorithm for Automatic Generation of Reservation Tables from Architectural Descriptions. In *Proc. of the Int. Symposium on System Synthesis (ISSS)*, pages 44–50, 1999.
9. G. Hadjiyiannis, S. Hanono, and S. Devadas. ISDL: An Instruction Set Description Language for Retargetability. In *Proc. of the Design Automation Conference (DAC)*, Jun. 1997.
10. A. Halambi, P. Grun, V. Ganesh, A. Khare, N. Dutt, and A. Nicolau. EXPRESSION: A Language for Architecture Exploration through Compiler/Simulator Retargetability. In *Proc. of the Conference on Design, Automation & Test in Europe (DATE)*, Mar. 1999.

11. F. Homewood and P. Faraboschi. ST200: A VLIW Architecture for Media-Oriented Applications. In *Microprocessor Forum*, Oct. 2000.
12. M. Itoh, S. Higaki, J. Sato, A. Shiomi, Y. Takeuchi A. Kitajima, and M. Imai. PEAS-III: An ASIP Design Environment. In *Proc. of the Int. Conf. on Computer Design (ICCD)*, Sep. 2000.
13. J. Hennessy and D. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann Publishers Inc., 1996. Second Edition.
14. D. Lanner, J. Van Praet, A. Kifli, K. Schoofs, W. Geurts, F. Thoen, and G. Goossens. Chess: Retargetable Code Generation for Embedded DSP Processors. In P. Marwedel and G. Goossens, editors, *Code Generation for Embedded Processors*. Kluwer Academic Publishers, 1995.
15. R. Leupers and P. Marwedel. Retargetable Code Generation based on Structural Processor Descriptions. *Design Automation for Embedded Systems*, 3(1):1–36, Jan. 1998. Kluwer Academic Publishers.
16. R. Leupers and P. Marwedel. *Retargetable Compiler Technology for Embedded Systems*. Kluwer Academic Publishers, Boston, Oct. 2001. ISBN 0-7923-7578-5.
17. X. Nie, L. Gazsi, F. Engel, and G. Fettweis. A new network processor architecture for high-speed communications. In *Proc. of the IEEE Workshop on Signal Processing Systems (SIPS)*, pages 548–557, Oct. 1999.
18. P. Paulin. Towards Application-Specific Architecture Platforms: Embedded Systems Design Automation Technologies. In *Proc. of the EuroMicro*, Apr. 2000.
19. W. Qin and S. Malik. Flexible and formal modeling of microprocessors with application to retargetable simulation. In *Proc. of the Conference on Design, Automation & Test in Europe (DATE)*, Mar. 2003.
20. R. Allen and K. Kennedy. *Optimizing Compilers for Modern Architectures*. Morgan Kaufmann Publishers Inc., Oct. 2001. ISBN 1-5586-0286-0.
21. Trimaran. *An Infrastructure for Research in Instruction-Level Parallelism*, <http://www.trimaran.com>.
22. O. Wahlen, T. Glöckler, A. Nohl, A. Hoffmann, R. Leupers, and H. Meyr. Application Specific Compiler/Architecture Codesign: A Case Study. In *Proc. of the Joint Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES) and Software and Compilers for Embedded Systems (SCOPE)*, Jun. 2002.
23. O. Wahlen, M. Hohenauer, R. Leupers, and H. Meyr. Instruction scheduler generation for retargetable compilation. In *IEEE Design & Test of Computers*, Jan. 2003.

A Framework for the Design and Validation of Efficient Fail-Safe Fault-Tolerant Programs

Arshad Jhumka^{1*}, Neeraj Suri¹, and Martin Hiller²

¹ Department of Computer Science

TU - Darmstadt, Germany

{arshad,suri}@informatik.tu-darmstadt.de

² Department of Electronics and Software

Volvo Technology Corporation

Göteborg, Sweden

`martin.hiller@volvo.com`

Abstract. We present a framework that facilitates synthesis and validation of fail-safe fault-tolerant programs. Starting from a fault-intolerant program, with safety specification SS , that satisfies its specification in the absence of faults, we present an approach that automatically transforms it into a fail-safe fault-tolerant program, through the addition of a class of detectors termed as *SS-globally consistent detectors*. Further, we make use of the SS-global consistency property of the detectors to generate pertinent test cases for testing the fail-safe fault-tolerant program, or for fault injection purposes. The properties of the resulting fail-safe fault-tolerant program are that (i) it has minimal detection latency, and (ii) perfect error detection. The application area of our framework is in the domain of distributed embedded applications.

Keywords: Detectors, software synthesis, fault tolerance, fail-safe, test cases.

1 Introduction

Safety-critical applications need to satisfy stringent dependability requirements in their provision of services. To reduce the complexity of designing such applications, Arora and Kulkarni [2] proposed a transformational approach, whereby an initially fault-intolerant program is systematically transformed into a fault-tolerant one. The main step involved in designing the fault-tolerant program is composing the corresponding fault-intolerant program with components that (i) detect and/or (ii) correct errors that arise as a result of faults, depending on the level of fault-tolerance to be achieved. The class of programs that achieves the first goal is termed *detectors* while the class of programs that achieves the second goal is called *correctors* [3].

In this paper we restrict our attention to designing *fail-safe* fault-tolerant programs. Intuitively this means that it is acceptable that the fail-safe fault-tolerant program “halts” when faults occur, as long as it always remains in a

* Contact author: Arshad Jhumka (`arshad@informatik.tu-darmstadt.de`).

“safe” state. This type of fault-tolerance is often used in (nuclear) power plants or train control systems where safety (avoidance of catastrophic events) is more important than continuous provision of service. Thus, a fail-safe fault-tolerant program has to satisfy at least its safety specification¹ in presence of faults. Arora and Kulkarni, in [3], showed that fail-safe fault-tolerance can be achieved by merely employing detectors, i.e., a fail-safe fault-tolerant program can be obtained by composing the corresponding fault-intolerant program with detectors only. In safety-critical systems, detection may be the only option [10], and once faults have been detected, a back-up system takes over.

Detectors can be regarded as an abstraction of many different existing fault-tolerance mechanisms. For example, a common way to achieve fault-tolerance is to replicate a critical task and schedule it on different processors. The outputs of these tasks are brought together in a voter which outputs a consistent value. The voter contains a comparator which is an instance of a detector. However, use of replication is often computationally expensive. Another (maybe more obvious) example of a detector is error detecting codes. Other error handling mechanisms like acceptance tests, self checks or executable assertions can also be formulated as detectors in the sense of Arora and Kulkarni [3]. Hence, reasoning at the level of detectors makes an approach applicable to many different practical settings.

However, the design of *efficient* fail-safe fault-tolerant programs is problematic, since design of efficient detectors is difficult, as observed in [10]. For design of fail-safe fault-tolerant programs, programmers tend to program defensively to ensure that the safety specification is not violated, i.e., they use very “restrictive” detectors. In formal terms, this means that those detectors are not *accurate*. When detectors are not accurate, the efficiency of the system (for example, response times, QoS etc) may decrease. For example, given a certain program with some valid inputs where these are flagged as erroneous by an inaccurate detector, there may not be any response at all, since the system may have “halted”. Inaccurate detectors may still preserve safety, however there may be an associated decrease in performance.

Similarly, detectors may fail to detect certain erroneous situations, i.e., the detectors are not restrictive enough. In formal terms, this means that the detectors are not *complete*. Hence, for design of *efficient* fail-safe fault-tolerant systems, detectors need to be both accurate, and complete, i.e., detectors need to be *perfect*² (detectors are complete and accurate). However, there is a dearth of frameworks or guidelines pertaining to the design of efficient detectors (or efficient fail-safe fault-tolerant programs).

In this paper, our approach is to transform an initially fault-intolerant program with safety specification SS that satisfies its specification in the absence of faults, but violates it in the presence of faults³, into an efficient fail-safe fault-tolerant program, through the addition of a class of detectors, termed as *SS-globally consistent detectors*. We will later show that composing a given fault-

¹ We will explain this term in Section 2.

² Classical measures for efficiency of detectors are detection coverage, and latency.

³ We will refer to this program as a fault-intolerant program.

intolerant program with a SS-globally consistent detectors results in a fail-safe fault-tolerant program that has *minimal* detection latency, and *perfect detection*⁴. Once the fail-safe fault-tolerant program has been obtained, it needs to be validated, through testing or fault injection. Both approaches can be computationally expensive, since they require generation of test cases. For this case, we exploit the SS-global consistency property of detectors to efficiently and automatically generate test cases.

Thus, our contributions are the following:

1. We introduce a class of detectors called *globally consistent detectors* which are instances of perfect detectors, and show, by means of examples, how fail-safe programs are obtained.
2. We explain how the SS-global consistency property can be exploited for systematic and automatic generation of test cases for validation (i.e., testing, or fault injection)

Throughout the paper, we will use examples to illustrate the different concepts involved in our approach. Our framework allows automatic synthesis of a fail-safe fault-tolerant program, as well as automatic generation of test cases for its validation. To the best of our knowledge, this framework is novel, since no other work has addressed the design of fail-safe fault-tolerant programs with perfect detection, and minimal detection latency in a systematic manner.

The paper is structured as follows: Section 2 presents the models (system, faults) used in the paper. Section 3 explains the role of detectors in the provisioning of fail-safe fault tolerance. Section 4 explains the concept of adding fail-safe fault tolerance to a fault-intolerant program. Section 5 introduces a class of detectors, called *SS-globally consistent detectors*, for which design is polynomial in the state space of the fault intolerant program. We further show how pertinent test cases for validating the fail-safe fault-tolerant program can automatically be generated from knowledge of the SS-globally consistent detectors in Section 6. In Section 7, we perform fault injection experiments to ascertain the viability of concept of SS-global consistency. We summarize the paper in Section 8.

2 Preliminaries

In this section, we will present the basic notations and terminologies that will underpin our presentation.

2.1 Program

A program P consists of a set of variables V_P , and a set of actions A_P , both partitioned among n processes $p_1 \dots p_n$. Each variable in V_P stores a value from an associated predefined non-empty, but finite, domain, and each action in P updates the value of one or more variables in V_P . A given value association

⁴ Hence, SS-globally consistent detectors are instances of perfect detectors.

with variables in V_P is called a *state* of P , and the set of all such possible value associations defines the *state space* of P . There also exists a subset of the state space of P that we refer to as the *set of initial states* of P . We assume actions of P to be deterministic, however execution of actions of P is non-deterministic. An *event* is said to occur when a program action executes. A given event can be *good* or *bad*, depending on whether or not it violates the safety specification. An action defines a set of transitions, and the set of actions defines the complete transition system of the program.

Two processes p_r and p_w of P communicate as follows: there exists a set of “shared” variables V_s between p_r and p_w . In such cases, for each variable in V_s , p_r is the reader of that variable, and p_w the writer, i.e., if p_w (the writer) updates the variable, then p_r (reader) reads it. This defines the information flow between two processes, and V_s is the interface between p_i and p_j .

There exists a set of special variables, denoted by V_o , that are shared by some processes (that write to the variables), and the environment that reads them. These special variables are commonly referred to as the *output variables*. There exists also a special set of variables, denoted by V_i , where each of the variables is written to by the environment, and read by a process in P . Such variables are known as *input variables*. Input and output variables represent the interface of the program P with its environment. Such program model is suitable for embedded applications, for which our framework is targeted.

Also, we assume programs to contain *critical actions* and *non-critical actions*. Critical actions are those that need to be monitored with detectors, while the non-critical actions do not [2]. Examples of critical actions for embedded systems are those actions that control progress, i.e., those actions that provide the output value, or commit some value to the environment.

A detector⁵ D in program P monitoring an action A of P is a boolean expression over the state space of P . Specifically, when D evaluates to “True” in a given state, that state is considered a *valid* state of P , and it also means that execution of action A can safely take place. In Section 3, we will explain in more details the role of detectors in ensuring fail-safe fault-tolerance.

2.2 Specification

A specification of a program consists of two parts, namely (i) a safety specification, and (ii) a liveness specification [1]. Given our focus on fail-safe fault-tolerance, we will explain safety specifications only. The liveness specification is needed so as to rule out any trivial program, such as one that does nothing, which always satisfies the safety specification. We also assume the specification to be *fusion-closed*. Informally, fusion-closure of a specification guarantees that the entire history of a given execution of the program “is available” in the current state, such that it is possible to determine if the next action to be executed is “desirable”. It has been observed [2] that low level specifications, such as C programs, are fusion-closed. In general, a specification that is not fusion-closed

⁵ A detector in our context will be an executable assertion.

can be converted into a fusion-closed specification by the addition of history variables, so the fusion-closure requirement is not a hindrance.

Informally, a safety specification of a program states that “something bad never happens”. Specifically, it rules out certain *sequences of events* that should never happen during execution of P . However, the fusion-closure property of the specification allows identification of a *set of events* (rather than set of sequences of events) that should not occur in any given execution of the program. Therefore, we take the safety specification of a program to specify the set of events that should not occur in any execution of the program, i.e., it specifies the set of bad events. Also, fusion-closure of a specification guarantees the existence of detectors (detection predicates) [2].

2.3 Faults

In this paper, we focus on the set of fault models that can potentially be tolerated, i.e., we do not consider faults that directly violate the safety specification of the program. For example, if the safety specification constrains the output variables of a program, as is often the case in embedded applications, then we disallow the faults to directly modify the output variables of the program that could result directly in a safety specification violation. However, faults can arbitrarily alter the state of the program in such a way that subsequent execution of program actions can lead to violation of the safety specification. Thus, safety is violated due to execution of a certain program action, such that the corresponding event is ruled out by the safety specification.

3 Detectors and their Role in Constructing Fail-Safe Fault Tolerant-Programs

We adopt the view of Arora and Kulkarni [3] that a fault-tolerant program is the composition of a fault-intolerant program with fault-tolerance components. Using the same system model as in this paper, Arora and Kulkarni proved that a class of program components called *detectors* are necessary and sufficient to establish fail-safe fault-tolerance.

Recall that the safety specification of a program P specifies a set of events that should not occur during any execution of P , i.e., the set contains bad events. Intuitively, to avoid violating a safety specification requires to keep track of the current program execution (history) and take precautions so that none of the events which are disallowed by the safety specification (bad events) occurs. From our restrictions of the fault model (faults do not directly violate safety), we know that these bad events occur when program actions are executed. Thus, a detector monitoring a given action in the program works in such a way that the action is never executed whenever its execution will result in the occurrence of a bad event. Overall, a detector allows execution of a corresponding program action only if its execution is “safe” (not a bad event). Also, it was shown in [6] that a bad event cannot occur without the occurrence of faults. This means that if no

fault occurs, then only good events are observed from the program. Detectors can also prevent potentially bad events from occurring [6], i.e., they prevent events that can potentially lead the program to violate its safety specification from occurring. Such potentially bad events may also be considered as bad events. Thus, the safety specification can be extended to also rule out those potentially bad events.

However, designing detectors has its inherent complexities [7,8]. In subsequent sections, we will explain how detectors can be designed that will transform a fault-intolerant program into a fail-safe fault-tolerant one.

At this point, we provide an example to illustrate some of the concepts we have presented:

```

Program P1
  var w init 1, c1 init 1 : int // process a
  var x init 5, y init 1, z init 10, c2 init 1 : int // process b

process a:
  c1 = 1 → w := read(); c1 := c1 + 1; // value between 15 and 25
  c1 = 2 ∧ x ≤ 15 → w := w + 5; c1 := 1; // loop
  c1 = 2 ∧ x > 15 → w := w - 15; c1 := 1; // loop

process b:
  c2 = 1 → x := read(); c2 := c2 + 1; // value between 0 and 20
  c2 = 2 → y := w; c2 := c2 + 1;
  c2 = 3 → z := y + x; c2 := c2 + 1;
  c2 = 4 → output(z); c2 := 1; // loop

F (faults):
  true → x := random [10...45]
  true → w := random [10...50]

```

Fig. 1. An example program to illustrate the different concepts

In the example in Fig. 1, the program *a* is written in the UNITY logic [4]. Variables c_1 and c_2 are two program counters, for process *a* and *b* respectively. For example, in process P_1 , the first statement says that when the program counter c_1 is 1, then variable w is assigned a sensor value, and the program counter is incremented. In process *b*, when $c_2 = 4$, an actuator value is sent through $output(z)$. The faults indicate for example that, at any time, the value of variable x can be randomly changed to one within $[10 \dots 45]$. Note that we do not consider faults affecting variable z , as per our fault model.

Processes *a* and *b* communicate as follows: variable w is written to by process *a* and read by process *b*. This defines the information flow between the two processes. An example of a safety specification for program P_1 is $10 \leq z \leq 50$. This means that whenever the value of variable z is outside of the given

range, a safety specification violation occurs. Since a fault cannot cause variable z to take values outside of the permissible range, the action that updates z (i.e., $z := y + x$) should be monitored by a detector to avoid occurrence of bad events that will lead to safety specification violation. For example, starting from a program state $P1s = (w = 15, x = 15, y = 60, z = 10)$ (we exclude the counters), executing the action $z := y + x$, will lead to a program state $P1e = (w = 15, x = 15, y = 60, z = 75)$, which violates the safety specification. Executing the program action starting from state $P1s$ to state $P1e$ is a bad event. Thus, a detector that monitors whether the sum of values of variables x and y (i.e., $x + y$) is within 10 and 50 is needed. If the sum is outside of the range, then the detector flags an error, and the program can possibly halt.

We will use this example as a running example to explain how our framework works. In the next section, we explain what it means to transform a fault-intolerant program into a fail-safe fault-tolerant one.

4 The Transformation Problem

We now state the problem of transforming a fault-intolerant program p into a fail-safe fault-tolerant version p' for a given safety specification SS and fault model F [9,6].

When deriving p' from p , only fault tolerance should be added, i.e., p' should not satisfy SS in new ways in the absence of faults. Specifically, there are two conditions to be satisfied in the transformation problem:

- If there exists an event e in p' that did not occur in p to satisfy SS , then event e cannot be used by p' to satisfy SS , since this means that there are other ways p' can satisfy SS in the absence of faults. Thus, the set of events occurring in p' should be a subset of the set of events occurred in p .
- Also, if there exists a state s reachable by p' in the absence of faults that is not reached by p in the absence of faults, then this means that p' can satisfy SS differently from p in the absence of faults, and such a state s should not be reached by p' in the absence of faults. Thus, in the presence of faults, the set of states reachable by p' should be a subset of the set of states reachable by p , and in the absence of faults, the sets of reachable states are equal.
- In the presence of faults, p' satisfies SS .

Overall, the first two conditions state that in the absence of faults, the fault-intolerant program p is “equivalent”⁶ to the fail-safe fault-tolerant program p' . Also, in presence of faults, p' satisfies its safety specification, while p does not.

In [6], we showed that composing critical actions of a program with a class of detectors, called *perfect* detectors, is sufficient to solve the transformation problem. In the next section, we will define the concept of SS-globally consistent detectors, and explain that they are instances of perfect detectors. Thus,

⁶ The two programs exhibit the same behavior in the absence of faults, i.e., are behavior-equivalent.

composing a fault-intolerant program with SS-globally consistent detectors will result in a program that will always satisfy its safety specification in the presence of faults, i.e., it is fail-safe fault-tolerant.

5 Adding Globally Consistent Detectors to a Program

In this section, we will explain the concept of globally consistent detectors. We then explain that a class of globally consistent detectors, called *SS-globally consistent detectors* are instances of perfect detectors.

5.1 Consistent Detectors and Globally Consistent Detectors

Before explaining the concept of globally consistent detectors, we will first explain the concept of *consistent detectors*. Recall that a detector d monitors the safe execution of a program action A , such that no bad event actually occurs upon execution of A . The detector d defines a set of states from which execution of A is safe, i.e., execution of A from any state defined by d will not give rise to a bad event.

A detector d_i monitoring a program action A_i is said to be *consistent* with a detector d_j monitoring program action A_j if and only if no sequence of events (thus good events since they have not been ruled out by d_i) starting from execution of A_i will cause execution of A_j to violate the safety specification. In other words, if A_i executes safely, followed by a sequence of good events, such that A_j is executing, the execution of A_j is safe. For example, see process a of Fig. 2.

Program $P1'$
 var x, y init 1, $c1$ init 1 : int // process a
process a:
 $c1 = 1 \wedge (15 \leq read() \leq 25) \rightarrow x := read(); c1 := c1 + 1; //$ value of x
 between 15 and 25
 $c1 = 2 \wedge (25 \leq x + 10 \leq 35) \rightarrow y := x + 10; c1 := c1 + 1; //$ loop
 $c1 = 3 \rightarrow output(y); c1 := 1; //$ loop
 F (faults):
 true $\rightarrow x := random [10 \dots 45]$

Fig. 2. An example to show the concept of consistent detectors

In process a , the detector $d_i, (15 \leq read() \leq 25)$, monitors action $A_i, x := read()$, while detector $d_j, (25 \leq x + 10 \leq 35)$, monitors action $A_j, y := x + 10$. If A_i executes, then it means a good event has occurred (i.e., it satisfies d_i). If no fault happens, then A_j will execute as well. Thus, d_i and d_j are consistent. In other words, if x can take a value between 15 and 25, then adding 10 (to obtain value for y) will cause y to take value between 25 and 35, hence consistency of

detectors. If a set of n detectors is incorporated in a program, and each detector is consistent with the safety specification, then the set of detectors is said to be *SS-globally consistent*.

5.2 Design of SS-Globally Consistent Detectors

In this section, we introduce a class of detectors, called SS-globally consistent detectors, for which the design complexity is polynomial in the size of the fault-intolerant program, and we will argue that this class of detectors is an instance of perfect detectors, i.e., they are complete (detect all errors that will cause violation of the safety specification of the program) and accurate (no false detection).

The design of *SS-globally consistent detectors*⁷, is tractable for a class of programs known as *bounded programs*. The main property of bounded programs is that the length of event sequences before the program outputs a value is bounded, i.e., there are no infinite loops within a process, nor is there some infinite communication between processes, and also that variables take values from finite domains. An example of bounded programs is embedded applications.

A set of SS-globally consistent detectors for a given program p with safety specification SS has the property that each detector in the set is consistent with SS . Recall that SS can effectively be a detector that monitors the critical action of the program. Overall, it means that if a detector d_i monitoring program action A_i is consistent with the safety specification of the program, then no sequence of (good) events, starting from execution of A_i , will violate the safety specification. Hence, d_i is accurate. Also, for any bad event ruled out by SS , there will also be a corresponding event ruled out by d_i . Hence, d_i is complete. Therefore, a globally consistent detector is indeed a perfect detector (accurate and complete).

At this point, we explain how SS-globally consistent detectors can be designed. Since each detector d_i is consistent with the safety specification of the program, we exploit this relationship, and start with the safety specification to automatically generate the SS-globally consistent detectors.

Since SS defines a detector that monitors the critical action of the program, we perform a backward propagation procedure, starting from the critical action of the program, against the flow of information. We illustrate this using a series of examples, see Fig 3–Fig. 5.

Then, for $10 \leq x + y \leq 50$ (safety specification) to be satisfied, and given that variable y is assigned the value of variable w , then the detector that monitors the program action $y := w$ should ensure that $10 \leq w + x \leq 50$. In such case, it is easy to verify that detector $(10 \leq w + x \leq 50)$ is consistent with safety specification $(10 \leq y + x \leq 50)$, see Fig 3.

Likewise, the detector monitoring program action $x := read()$ should ensure that $10 \leq read() + w \leq 50$, such that when variable x is assigned value from $read()$, then this does not violate the detector $10 \leq w + x \leq 50$ monitoring program action $y := w$, see Fig 4.

⁷ Whenever it is obvious from the text, we will use the term globally consistent detectors to mean SS-globally consistent detectors.

```

Program P1
  var w init 1, c1 init 1 : int // process a
  var x init 5, y init 1, z init 10, c2 init 1 : int // process b

process a:
  c1 = 1 → w := read(); c1 := c1 + 1; // value between 15 and 25
  c1 = 2 ∧ x ≤ 15 → w := w + 5; c1 := 1; // loop
  c1 = 2 ∧ x > 15 → w := w - 15; c1 := 1; // loop

process b:
  c2 = 1 → x := read(); c2 := c2 + 1; // value between 0 and 20
  c2 = 2 ∧ (10 ≤ w + x ≤ 50) → y := w; c2 := c2 + 1;
  c2 = 3 ∧ (10 ≤ y + x ≤ 50) → z := y + x; c2 := c2 + 1;
  c2 = 4 → output(z); c2 := 1; // loop

F (faults):
  true → x := random [10...45]
  true → w := random [1...50]

```

Fig. 3. An Example to show generation of SS-globally consistent detectors. Observe that the critical action $z := y + x$ is monitored by a detector defining SS .

```

Program P1
  var w init 1, c1 init 1 : int // process a
  var x init 5, y init 1, z init 10, c2 init 1 : int // process b

process a:
  c1 = 1 → w := read(); c1 := c1 + 1; // value between 15 and 25
  c1 = 2 ∧ x ≤ 15 → w := w + 5; c1 := 1; // loop
  c1 = 2 ∧ x > 15 → w := w - 15; c1 := 1; // loop

process b:
  c2 = 1 ∧ (10 ≤ w + read() ≤ 50) → x := read(); c2 := c2 + 1; // value
between 0 and 20
  c2 = 2 ∧ (10 ≤ w + x ≤ 50) → y := w; c2 := c2 + 1;
  c2 = 3 ∧ (10 ≤ y + x ≤ 50) → z := y + x; c2 := c2 + 1;
  c2 = 4 → output(z); c2 := 1; // loop

F (faults):
  true → x := random [10...45]
  true → w := random [1...50]

```

Fig. 4. An Example to show generation of SS-globally consistent detectors

As for process a , the information flow is from process a to process b , through the shared variable w . The value of variable w is used to update the value of variable y of process b . Thus, if the detector for program action $y := w$ is to be satisfied, then the value of variable w in process a should be cognizant of the fact that variable w can be updated in two different ways. The detector monitoring the *if-then* action of process a is as follows: $(10 \leq w + x - 15 \leq 50) \vee (10 \leq w + x + 5 \leq 50)$, which is equivalent to $(25 \leq w + x \leq 65) \vee (5 \leq w + x \leq 45)$. The program is shown in Fig. 5.

```

Program P1
  var w init 1, c1 init 1 : int // process a
  var x init 5, y init 1, z init 10, c2 init 1 : int // process b

process a:
  c1 = 1  $\wedge$   $((25 \leq x + w \leq 65) \vee (5 \leq x + w \leq 45)) \rightarrow w := \text{read}()$ ;
  c1 := c1 + 1; // value between 15 and 25
  c1 = 2  $\wedge$   $x \leq 15 \wedge (5 \leq x + w \leq 45) \rightarrow w := w + 5; c1 := 1$ ; // loop
  c1 = 2  $\wedge$   $x > 15 \wedge (25 \leq x + w \leq 65) \rightarrow w := w - 15; c1 := 1$ ; // loop

process b:
  c2 = 1  $\wedge$   $(10 \leq w + \text{read}() \leq 50) \rightarrow x := \text{read}(); c2 := c2 + 1$ ; // value
  between 0 and 20
  c2 = 2  $\wedge$   $(10 \leq w + x \leq 50) \rightarrow y := w; c2 := c2 + 1$ ;
  c2 = 3  $\wedge$   $(10 \leq y + x \leq 50) \rightarrow z := y + x; c2 := c2 + 1$ ;
  c2 = 4  $\rightarrow \text{output}(z); c2 := 1$ ; // loop

F (faults):
  true  $\rightarrow x := \text{random}[10 \dots 45]$ 
  true  $\rightarrow w := \text{random}[1 \dots 50]$ 

```

Fig. 5. The final program with SS-globally consistent detectors

Depending on the fault model, some of those detectors may be excluded. For example, if faults were not to affect variables x and y say, then the detector monitoring program action $z := x + y$ will not be needed, i.e., $10 \leq x + y \leq 50$.

As can be deduced, the complexity of the procedure is polynomial in the size of the program (i.e., polynomial in the state space of the fault-intolerant program). As we have explained earlier, SS-globally consistent detectors are instances of perfect detectors, i.e., they detect errors if and only if they lead to violation of the safety specification. Thus, whenever a fault occurs, and given the fact that the detectors are perfect implies that the corresponding error will be detected earlier, i.e., it has a *lower* latency, than if the error is to be detected by the detector guarding the critical action. Specifically, given our fault model and a set D of perfect detectors for program p (resulting in fail-safe fault-tolerant p') with safety specification SS , then a detector $d_i \in D$ exists such that when-

ever a bad event is about to occur, d_i will flag the problem, i.e., p' has minimal detection latency (i.e., “0-step” – no bad event happens).

Overall, in this section, we have argued that SS-globally consistent detectors are perfect detectors, and we have illustrated, by means of examples, how these are generated. We also argued that when SS-globally consistent detectors are incorporated into a program, the program has a better detection latency.

6 Automatic Generation of Test Cases for Testing/Fault Injection Using Perfect Detectors

In this section, we explain how the use of perfect detectors (i.e., SS-globally consistent detectors) help in the automatic generation of test cases for testing the fail-safe fault-tolerant program, or for fault-injection purposes.

For test case generation, we use the perfect detectors to partition the state (input) space. In [5], the authors argued that for partition testing to be efficient, there is a need to group within one given partition all inputs that will cause the system to fail. The availability of SS-globally consistent detectors, i.e., perfect detectors means that those detectors will partition the input space “perfectly”, i.e., one can group into one partition all inputs that will cause the program to fail. For example, the safety specification of our example program is $10 \leq z \leq 50$. If we want to test the program using test cases that will cause the program to fail (e.g., for fault-injection), we should choose test cases from the space $((x + y < 10) \vee (x + y > 50))$ just before executing the critical action.

So, when the resulting fail-safe fault-tolerant program has to be validated (e.g., testing or fault-injection), whenever execution reaches one of the detectors, an appropriate test case can be automatically generated, and the behavior of the program observed. For example, consider Fig. 5. If the action $y := w$ in process b is about to be executed when the program is running, the detector for this action is $(10 \leq w + x \leq 50)$. So, to generate a test case for fault-injection, we need to invert the detector condition, i.e., $(w + x < 10) \vee (w + x > 50)$, and choose an appropriate value of w that will satisfy the inverted condition.

Similarly, for testing, i.e., testing the fail-safe fault-tolerant program, if the system designer wants to perform unit testing, i.e, testing of each process, the detectors can again be used to help automatically generate the required test cases (assuming all the required stubs are available). For example, considering Fig. 5 again, unit testing of process b will “force” *read()* to generate a value that will violate the detector monitoring this action. For integration testing, i.e., testing communication between processes, the detectors again help in automatically generating test cases. For example, from Fig. 5, for testing the interface between processes a and b , we reuse the detector $(10 \leq w + x \leq 50)$ to generate the relevant test cases.

7 Fault Injection Experiments to Ascertain SS-Global Consistency

We have explained that SS-globally consistent detectors are perfect detectors, i.e., they detect errors if and only if they will lead to violation of the safety specification. We have also shown how to automatically generate the perfect detectors, by means of an example. We also explained how, by exploiting the information obtained from the perfect detectors, test cases for fault-injection or testing can be automatically generated. In this section, we present the results from an experiment to ascertain the viability of the concept of SS-globally consistent detectors.

The target software is an aircraft arresting system, used on short runways, see Fig. 6. It consists of 6 modules. We focus on module *V-REG*. *V-REG* uses the signals *SetValue* and *IsValue* to control *OutValue*, the output value to the pressure valve. The value of the *OutValue* signal is calculated by evaluating a function on the difference between the *SetValue* and the *IsValue* signals. The reason for choosing module *V-REG* is that it is medium-sized, and SS-globally consistent detectors could be easily generated for the module.

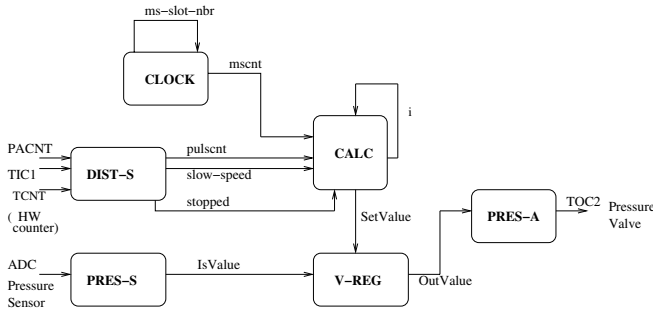


Fig. 6. Software Architecture of the Target System

To ensure that SS-globally consistent detectors are indeed perfect detectors, we compare them against detectors obtained directly from the specification of the system. The detectors obtained from the specification monitored signals *SetValue* (EA_1) and *IsValue* (EA_2), while the SS-globally consistent detector (EA_3) obtained monitored both signals at the same time. EA_4 is the safety specification of the program, monitoring the critical action of the program. EA_3 is the SS-globally consistent detector generated by our approach. Observe that we had obtained a set of SS-globally consistent detectors, however, since we are assuming that errors only get into the system via the input signals of module *V-REG*, we only need to monitor the input signals. If faults could corrupt the value of program variables, we would have included all of the SS-globally consistent detectors in module *V-REG*. We determine SS-global consistency by

determining the consistency value of each detector with the safety specification. If the consistency value of all detectors is 1, then all the detectors are SS-globally consistent.

7.1 Fault Injection in V-REG Module

When performing the fault injection experiments, sometimes errors were injected after an aircraft has been arrested. We therefore use the term errors to denote whenever errors are injected before an aircraft has been arrested. The errors injected were bit-flips in the input variables of the module, at different given time instance.

First, we want to ascertain that EA_1 and EA_2 are not SS-globally consistent. Thus, we try to ascertain the fact that there are cases where EA_4 detects an error whereas EA_1 and EA_2 do not, or vice versa.

From the fault injection experiments, we calculated the consistency of a given EA, EA_i , with respect to the safety specification (EA_4) by calculating (i) the number of concurrent error detection by EA_4 and EA_i *conc-det*, and (ii) number of error detection by EA_4 , *ss-det*. The consistency values in Table 1 is then calculated as follows: $(1 - \text{abs}(\text{ss-det} - \text{conc-det})/(\text{ss-det}))$. For example, there were 3840 error injections into *SetValue*, and of these, EA_1 detected 1932, giving a detection coverage of 0.50313 for EA_1 . Also, there were 2051 corruptions of the system state, leading to violations of the safety specification. Of these, 1561 errors were detected by EA_1 . Using the consistency equation above gives a value of 0.76109 for consistency. The same is repeated for the consistency value of EA_2 and EA_3 . The consistency value of EA_4 is NA since we assume it to be 1 (by default).

Table 1. Consistency Values of detectors for errors injected in *SetValue*

Metrics	EA_1	EA_2	EA_3	EA_4
Consistency	0.76109	0.44369	1	NA

For data in Table 2, errors injected in *IsValue*, the following values were obtained:

Table 2. Consistency Values of detectors for errors injected in *IsValue*

Metrics	EA_1	EA_2	EA_3	EA_4
Consistency	0.082714	0.90441	0.99951	NA

We also note that the consistency value of EA_3 from Table 2 is less than 1. From closer inspection, we found that this mismatch is due to the fact that

sometimes error is detected after the aircraft has been arrested, and when the system is performing some reset action. So, this slight mismatch can be safely ignored, since we did not consider the case when the system is actually resetting. The overall consistency value of each detector is summarized in Table 3.

Overall, we have found that the detector generated by our approach is indeed a perfect detector, since it has consistency value of almost 1 (it is consistent with the safety specification of the system). However, the specification-based detectors EA_1 and EA_2 sometimes allow errors to go undetected, and violate the safety specification, which can have disastrous consequences for safety critical systems, or are inaccurate leading to performance degradation. Also, these detectors also seem to detect errors even though those errors are “harmless” (will not lead to violation of safety specification). These observations corroborate those made by Leveson et. al [10], i.e., specification-based detectors are likely to be inaccurate and/or incomplete. Hence, our approach can be seen as a first step in addressing the problem of generating perfect detectors.

Table 3. Consistency values of detectors for errors injected in V-REG module inputs

Metrics	EA_1	EA_2	EA_3	EA_4
Consistency	0.42457	0.67224	0.99975	NA

8 Discussion and Conclusions

In this paper, we have presented an approach for designing efficient fail-safe fault-tolerant program, i.e., programs with perfect error detection and optimal detection latency. We have explained, through examples, how such detectors (perfect detectors) can be generated. We also explained how, by using the perfect detectors, test cases for validating the fail-safe fault-tolerant program can be automatically obtained. The complexity of our method is polynomial in the size of the program (state space) [6]. Our approach is novel, and to the best of our knowledge, there is no work that has addressed the design of perfect detectors for software. Our work also solves some of the problems posed in [10], and the observations made when running the fault-injection experiments corroborate all the findings presented in [10].

We have shown that SS-globally consistent detectors are instances of perfect detectors, and we have presented an experimental analysis of how SS-global consistency can be verified. Our approach works for a class of programs, known as bounded programs, of which embedded applications are instances.

In this work, we have looked at continuous signals. As future work, we will look at including discrete signals in our framework. An initial possible approach is to partition the space of the continuous signal into disjoint sets of continuous values, where each set can represent one discrete value.

A final note on our approach: note that our approach is not based on inverting the code of the program, rather it makes use of the computation itself to generate detectors. Thus, the problem with having non-invertible functions, such as hash functions, do not apply, and the way our approach deals with such situations is to include the function call inside the detector, e.g., $0 \leq x + F(y) \leq 25$.

References

1. Bowen Alpern and Fred B. Schneider. Defining liveness. *Information Processing Letters*, 21:181–185, 1985.
2. Anish Arora and Sandeep S. Kulkarni. Component based design of multitolerant systems. *IEEE Transactions on Software Engineering*, 24(1):63–78, January 1998.
3. Anish Arora and Sandeep S. Kulkarni. Detectors and correctors: A theory of fault-tolerance components. In *Proceedings of the 18th IEEE International Conference on Distributed Computing Systems (ICDCS98)*, May 1998.
4. K. Mani Chandy and Jayadev Misra. *Parallel Program Design: A Foundation*. Addison-Wesley, Reading, MA, Reading, Mass., 1988.
5. B. Jeng and E.J. Weyuker. Analyzing partition testing strategies. *IEEE Transactions on Software Engineering*, July 1991.
6. A. Jhumka, F. Gärtner, C. Fetzer, and N. Suri. On systematic design of fast, and perfect detectors. Technical report, Ecole Polytechnique Federale de Lausanne (EPFL), School of Computer and Communication Sciences, Technical Report 200263, September 2002.
7. A. Jhumka, M. Hiller, V. Claesson, and N. Suri. *On Systematic Design of Globally Consistent Executable Assertions in Embedded Software*. In *Proceedings LCTES/SCOPEs*, pages 74–83, 2002.
8. S. Kulkarni and A. Ebnenasir. “Complexity of Adding Fail-Safe Fault Tolerance”. In *Proceedings International Conference on Distributed Computing Systems*, 2002.
9. Sandeep S. Kulkarni and Anish Arora. Automating the addition of fault-tolerance. In Mathai Joseph, editor, *Formal Techniques in Real-Time and Fault-Tolerant Systems, 6th International Symposium (FTRTFT 2000) Proceedings*, number 1926 in Lecture Notes in Computer Science, pages 82–93, Pune, India, September 2000. Springer-Verlag.
10. N. Leveson, S.S. Cha, J.C. Knight, and T.J. Shimeall. *The Use of Self-Checks and Voting in Software Error Detection: An Empirical Study*. *IEEE Transactions on Software Engineering*, 16(4):432–443, 1990.

A Case Study on a Component-Based System and Its Configuration

Hiroo Ishikawa and Tatsuo Nakajima

Department of Information and Computer Science
Waseda University
3-4-1 Okubo, Shinjuku, Tokyo, 169-8555, Japan
{ishikawa,tatsuo}@dcl.info.waseda.ac.jp

Abstract. Ubiquitous computing proliferates complexity and heterogeneity of software. Component software provides better productivity and configurability by assembling software from several components. The purpose of this paper is to investigate system configurations on a component-based system and the side effects of the configurations. We have implemented a component-based Java virtual machine named Earl Gray, by modifying an existing Java virtual machine. The case study revealed several problems to use the current component framework when configuring software. We report three experiments of those problems and present a future direction to solve the problem.

1 Introduction

We can find many consumer electronic devices that contain computers. In ubiquitous computing environments, computers are embedded in various objects and environments, such as furniture, dishes, and cloths[11]. These embedded computers are networked and cooperated to provide various services satisfying a user's requirements. For example, a computer embedded into a chair cooperates with an air conditioning system in the same room and adjusts the room's temperature according to a person's preference or posture.

The vision of ubiquitous computing promises calm computing and integrated spaces between real world and virtual world. Consequently, system software such as operating systems and middleware are required to be more heterogeneous and complex, because various types of networked embedded systems will be available to build the integrated spaces.

Embedded systems are required various kinds of configurations according to requirements and resource constraints. Despite increasing the number of configurations, the current embedded systems are evolved by modifying their source code directly. This causes the reduction of maintainability and configurability of the systems.

For building such heterogeneous and complex software, a component-based approach provides better productivity and configurability by assembling software from several components. A component software allows embedded systems to be modified systematically. A component software can be modified by replacing the

component in it. So we don't manage the entire software, but each component of software. This advantage of component software encourages reusability and configurability of embedded systems.

This paper presents Earl Gray that is a component-based Java virtual machine. Earl Gray has been implemented by modifying an existing virtual machine. Since component-based design makes the system structure clearer than before, the system becomes more configurable. For instance, we can replace several components according to a platform's characteristics.

We also present three configurations on Earl Gray. Our experimental results show several problems to use the current component frameworks when configuring software. We report three case studies that show the problems and we present a direction to solve the problem.

The remainder of this paper is organized as follows. Section 2 describes the design and implementation of Earl Gray. In Section 3, we compare the performance of Earl Gray and that of the original JVM. In Section 4, three experiments are presented. They are (1) replacing the thread scheduler and associated components, (2) replacing the bytecode verifier component with one on a remote machine, and (3) extending Earl Gray to be able to handle the scoped memory functionality. The result and problems of each case are also described. In Section 5, we present related work, and we conclude the paper in Section 6.

2 A Component-Based Java Virtual Machine: Earl Gray

We have developed a component-based Java virtual machine, named Earl Gray, by modifying the existing JVM, Wonka[15], and in a component description language, Knit[7]. On the process of decomposition of the system, we had several decision about component granularity, and component interface definitions. Despite describing in components, the performance of Earl Gray is changed little from Wonka. This section presents the off-the-shelf virtual machine and composition tool, and then describes the design and implementation of Earl Gray.

2.1 Off-the-Shelf System and Tool

Wonka Virtual Machine: Wonka is an open source virtual machine and supports the Java Virtual Machine specification provided by Sun Microsystems, Java 1.2 APIs with AWT, and several I/O devices such as RS232C ports.

Knit Component Description Language: We have adopted Knit to describe the components of Earl Gray. Knit is a component description language developed by the Flux research group at the University of Utah for describing components in OSKit[4].

A component in Knit consists of a set of typed input ports and output ports. The advantage of this model is that a connection between two components is explicitly described outside the components. Each port bundles some interfaces, and the interfaces are implemented by a set of functions written in C. The input

ports of a component specify the services that the component requires, while the output ports specify the services that the component will provide. An interface type consists of a set of methods, named constants, and the other interface types. A component in Knit is a black box component. The internal implementation of a component is hidden from clients.

There are two types of components in Knit as shown in Fig. 1 and 2. An atomic component is the smallest unit to compose programs, while a compound component consists of atomic components and/or other compound components. A system is structured by composing these two types of components.

```

bundletype Collector_T = {
    gc_collect,
    gc_create,
    ...
}

unit Collector = {
    imports [ heap : Memory_T ];
    exports [ gc : Collector_T ];
    depends { exports needs imports; };
    files { "src/heap/collector.c" }
}

```

Fig. 1. An example of an atomic component. **bundletype** defines an interface of a component in which function names in C are described. The **depends** block indicates dependencies between interfaces in **imports** and that in **exports**. The **files** block indicates an implementation of the component

A components in Knit is a compile-time component. Components are statically combined into one executable binary after the compilation. Unlike CORBA and COM, component binding at run-time is not supported by Knit. The advantage of Knit is to keep the system small without communication overhead among components, discovery and binding mechanism. The compilation of Knit is executed in the following way: (1) Knit compiler checks syntax and dependencies between ports. (2) The compiler creates a rename table according to the link description in the compound components. For example, a function name **gc_create** is renamed to **Collector_gc_create**. (3) It compiles each component to a binary file by using **gcc**. (4) It renames entries in the symbol table in each object file according to the rename table created in phase (2). This is because Knit allows more than one components to be implemented the same interface. The compiler distinguishes components with the same interface by referring the renaming table. (5) The **ld** linker program links all object files into one executable program. The implementation of an atomic component in Knit is written in C or assembly languages. The atomic component consists of more than one C and/or assembly source files.

```

unit RuntimeMemoryArea = {
  imports [
    thread : Thread_T,
    exception : Exception_T,
    ...
  ];
  exports [
    gc : Collector_T,
    method : Method_T,
    ...
  ];
  link {
    [ method ] <- MethodArea <- [ thread, malloc, ... ];
    [ gc ] <- Heap <- [ exception, thread, malloc, ... ];
    [ malloc ] <- Malloc <- [];
  }
}

```

Fig. 2. A compound component example. A compound component includes the link block that explicitly connects atomic component and other compound components. MethodArea and Heap component is connected with thread interface from outside of Collector component. Malloc is an internal component of RuntimeMemoryArea component and it connects to Heap and MethodArea components

2.2 Overall Architecture

Earl Gray is designed based on the original architecture as much as possible, because component software allows source code to explicitly reflect the image of the architecture. For instance, in Knit, connections among components are described in compound components. Those descriptions realize a clear architectural view in the source code.

Earl Gray consists of three large components, *Kernel*, *Middleware*, and *VM*, as shown in Fig.3. The kernel component provides low level services such as thread management and memory management. The middleware component provides common services for the VM component, such as string operations and network or serial port drivers. The VM component contains basic functionalities to implement Java virtual machine such as class loader, a runtime memory area, an execution engine, and native interfaces bridging to JavaAPI[10].

2.3 Component Granularity

We have implemented the functionality contained in each file as an atomic component. Wonka is a well-structured Java virtual machine. Each source file of Wonka usually contains one functionality. Since the component contains one functionality, each atomic component is usually small.

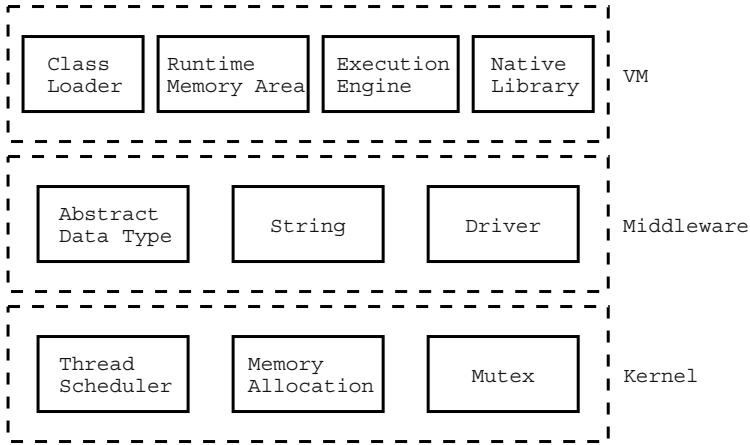


Fig. 3. Earl Gray Architecture

The granularity of compound components varies depending on their functionalities. For example, the native library component is the largest component of Earl Gray, because it includes many components implementing Java API. On the other hand, the Class Loader component contains only four atomic components.

2.4 Component Interface

A component interface is a definition of an end point which other components connect to and communicate with. Port is an instance of the component interface. The number of links among ports depends on how many ports each component provides. The ports are classified into two types, input ports and output ports. Components are explicitly composed by connecting an input port and an output port by a connector. This approach makes the system architecture clearer than the original source code. For example, it is difficult to understand the relationship among the functions without examining all source code files of usual C programs. However, it is much easier to understand the relationship among components by examining component description files.

In our design, an atomic component offers only one interface to make an atomic component as simple as possible in order to clearly separate the roles of atomic components and compound components. If a component needs to offer two interfaces, we decompose the component into two atomic components, and create a compound component from the two atomic components. For example, **Runtime Memory Area** component consists of two atomic components, the heap component and the method area component. The heap and method area components provide **Collector_T** and **MethodArea_T** interfaces respectively.

2.5 Implementation

Earl Gray is a component-based Java virtual machine that is built by modifying the Wonka virtual machine, which is developed for embedded systems. Earl Gray supports Java Virtual Machine Specification provided by Sun Microsystems, Java 1.2 APIs, and several I/O devices such as RS232C ports. It does not support JIT (just-in-time) compilation. The current version of Earl Gray runs on Linux for the Intel x86 family processor.

In the current implementation, the kernel component contains 16 atomic components and 1 compound component when the default scheduler is selected. The middleware component contains 25 atomic components and 3 compound components. Lastly, the VM component contains 108 atomic components and 8 compound components. All the atomic components are described in Knit and implemented in C.

3 Evaluation

This section compares Earl Gray with Wonka which is the original JVM of Earl Gray in terms of program size and performance. Despite using Knit, Earl Gray is as almost same size and performance as Wonka. Each JVM is compiled by gcc version 2.95 with `-O6` option without any debugging options, and doesn't include JIT compiler nor AWT support.

3.1 Program Size

The size of each JVM without symbols is almost same (Table 1). The component descriptions are dealt with in order to check the connections among components and rename the symbol tables. Thus, the descriptions are not compiled into the binary file. Earl Gray is 128byte bigger than Wonka. This is because Knit generates additional files in order to initialize and finalize the program.

Table 1. Size comparison between Earl Gray and Wonka

Program	Size (byte)
Earl Gray	567496
Wonka	567368

3.2 Performance

In order to measure the performance of Earl Gray, we have executed the Richards and DeltaBlue benchmarks[14] on Earl Gray and Wonka. The Richards is a

Table 2. Performance Evaluation (Execution Time)

Benchmarks	Wonka	Earl Gray
richards_gibbons	198ms	198ms
richards_gibbons_final	195ms	195ms
richards_gibbons_no_switch	231ms	231ms
richards_deutsch_no_acc	322ms	321ms
richards_deutsch_acc_final	700ms	697ms
richards_deutsch_acc_virtual	700ms	700ms
richards_deutsch_acc_interface	755ms	753ms
DeltaBlue	87ms	88ms

medium-sized language benchmark that simulates the task dispatcher in the kernel of an operating system. The DeltaBlue is a constraint solver benchmark.

Table 2 shows the results of the benchmarks on Earl Gray and Wonka. All benchmarks were measured on a 1.2GHz Pentium 3 with 1024MB of RAM running Linux version 2.4.20. Earl Gray was compiled with gcc version 2.95.4 at optimization level - O6. The results were reported by using the benchmark programs themselves. Therefore, they does not include any JVM initializations.

The performance of Earl Gray is as almost same as that of Wonka. Each result is the average of 100 times benchmarking. There are a few differences between Earl Gray and Wonka. This is because locations of functions in an executable file compiled by Knit are different from the one in the original executable file compiled by just gcc. In the Knit version, two set of functions in two atomic components respectively are placed closely in the executable file, if the components are compounded into one.

4 Case Studies on Component-Based Configuration

This case study shows three configurations by replacing or adding components and the side effect of the configurations. In each case, we found problems of a component-based system. Although component interfaces indicate inter-component dependencies, there are other inter-component dependencies that component interfaces can not indicate explicitly. The case studies described in this section show the implicit inter-component dependencies appeared when configuring a system. We have examined the following three cases:

1. **Replacing Thread Scheduler.** Replacing the default scheduler with a scheduler provided by a host operating system.
2. **Modifying Bytecode Verifier.** Replacing the default bytecode verifier with a bytecode verifier executed in a remote machine.
3. **Adding a Real-time Feature.** Adding scoped memory, that is one of the features described in the Real-time Specification for Java[1], to Earl Gray.

4.1 Scenario

In ubiquitous computing environments, a system is expected to be automatically modified and extended, because the software has to adapt to the current situation[2]. This case study is examined based on this kind of situation. In case 1 and 3, appropriate components are downloaded and activated according to the requirements. In case 2, due to the memory constraints, the system connects to a component on a remote machine instead of downloading it.

4.2 Using a Platform Functionality

This experiment aims to change a system to use alternative functionalities provided by a platform, instead of ones included originally. This change is realized by replacing components.

This experiment changes a scheduler component and investigates the effect of the change to the entire virtual machine. Because the thread scheduler is one of the core mechanisms of the Java virtual machine, the effect of the replacement must be examined.

Implementation: We replace the original thread scheduler with a scheduler that maps a thread in the virtual machine to a thread provided by the Linux kernel directly. The original thread scheduler's implementation includes a thread dispatcher mechanism and the threads are multiplexed on a single Linux thread. This replacement takes a scheduler mechanism away from Earl Gray, and the Linux kernel schedules the threads.

When implementing a new scheduler component, monitor and mutex components in the kernel component are also replaced because the kernel component includes monitor and mutex which are needed to synchronize threads.

As a result of direct mapping to the scheduler provided by the host operating system, the number of components in the kernel component was decreased. The components in the kernel component originally consist of 17 core components and 4 sub components. 8 components in 17 core components are used only inside of the kernel component. The 8 components contain mechanisms for thread management such as interrupt handling, timer, generating random number, and so on. The direct mapping implementation does not need these actual implementations. The remaining 9 components are still used when the new scheduler component is selected.

Since the kernel component is completely separated from other components, the new implementation does not affect other components in terms of explicit dependencies among components.

Implicit Dependency on Scheduling Policy: When a scheduler component is replaced, we found that the system was stopped unexpectedly. A race condition occurs in the function to uncompress a zip file where *push* and *pop* functions are invoked (Fig. 4). The functions were not considered that thread switch timing is different due to a different scheduling policy.

The original implementation assumes that the scheduler is not preemptive. Therefore, the queue structure in the uncompress component does not need to be protected from concurrent accesses while accessing it. However, Linux kernel threads are preemptive, thus we need to use mutex variables to protect the queue. Moreover, adding critical sections requires the initialization of the mutex variables, and this requires to modify the initialization component.

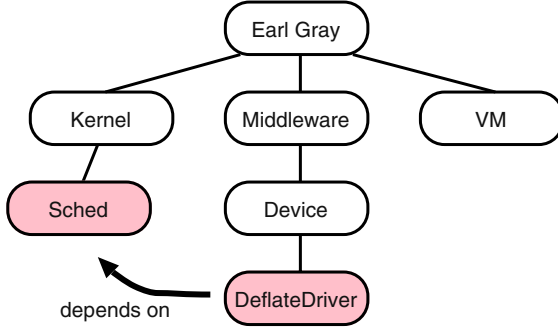


Fig. 4. Implicit dependency on the thread scheduler

4.3 Using a Remotely Available Resource

The second experiment changes a system to use components on a remote machine, instead of ones on the local machine. We investigated the difference between a local component and a remote component, and the effect of such a replacement. A bytecode verifier may invoke exceptions when it detects an invalid bytecode sequence. In the case of a remote bytecode verifier, the exceptions have to be invoked not only by invalid bytecode sequence, but also by network errors. The remote bytecode verifier requires a virtual machine to manage the exceptions raised by network errors in addition to the default exceptions.

Implementation: The remote bytecode verifier consists of two components, a stub component and a remote verifier component. Figure 5 depicts the verifier setting. The VM component requires a component providing the service with the `Verifier_T` interface. `Verifier` (local or stub) components provide the service with `Verifier_T` interface.

The stub component provides the same interface as the local bytecode verifier component. Therefore, the default verifier can be replaced by the remote bytecode verifier without modifying the other codes in the virtual machine.

The remote bytecode verifier communicates with the stub component by using the remote procedure call (RPC). We have adopted ORBit[13], which is one of the CORBA[12] implementations, as an RPC mechanism.

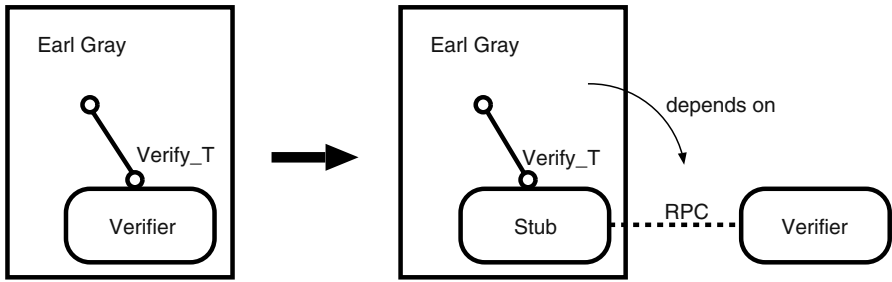


Fig. 5. (a)The verifier component is locally composed in Earl Gray. (b)The behavior of Earl Gray depends upon the network condition between the stub and the verifier component.

Implicit Component Behavior: Since the verifier component is located on a remote machine, we have to consider the effect of the network connection between Earl Gray and the verifier component. The original verifier component is located on the local machine and composed with in Earl Gray statically, thus it returns the result immediately after finishing verification and the behavior of the verifier component is defined as verifying bytecode sequences. In the case of using the remote bytecode verifier component, however, it is unsure whether the result of verification is returned immediately after the verification. The behavior of the remote bytecode verifier component is not only defined as verifying bytecode sequences, but also the condition of the network connection.

In the case of this implementation, the virtual machine never expects that the remote verifier definitely returns errors. Instead, the virtual machine assumes that the verifier returns a result whenever it is invoked. In other words, components that invoke the bytecode verifier depend on whether a local or a remote bytecode verifier is used.

The `Verify_T` interface includes a function that creates `java.lang.verifyError`, which is thrown when the verifier detects the inconsistency of bytecode. Although network errors can occur in the case of the remote bytecode verifier, the interface does not include any functions that handle network errors. Thus, the system does not detect any network errors caused by the remote bytecode verifier.

4.4 Extending the System

The aim in the third experiment is to investigate the effect of a change when adding a new component. A component is a unit of deployment[8]. Thus, it will not be difficult to extend the virtual machine by adding a new component.

Implementation: The scoped memory feature which is one of the features described in Real-time Specification for Java[1], is implemented. The scoped

memory enables an application to deallocate memory area explicitly when a program exits from the current scope. For example, if a method allocates a local (within the method) instance in the scoped memory area, the scoped memory feature makes sure that the instance is deallocated when the method is returned. In other words, instances in the scoped memory area are never collected by the garbage collector, instead, applications need to manage memory allocation and deallocation explicitly.

The scoped memory feature is realized by two components. One is a scoped memory allocation component. This component has own memory area in order to allocate the scoped objects, while the default allocation mechanism instantiates objects on the heap and registers them to the garbage collector. The other component consists of several native interface components which are bridges between Java real-time APIs and the virtual machine.

Adding a New Functionality: The implementation of the scoped memory API requires the thread structure to be extended in order to include a pointer to a scoped memory area. Because the specification defines that a scoped memory area is created in a thread and destroyed when the thread is terminated.

Fortunately, the extension of the thread data structure did not affect the other components. However, the modification of a data structure might affect the implementation of the other components because the memory layout is changed if the data structure is modified. This causes a chain of the modifications of components.

Consequently, this case study shows that we have to still be careful to extend a component-based system with additional components. Because there is a chain of implicit dependencies among components.

4.5 Discussions

According to the above experiments, there are implicit dependencies among components even though components are well-separated. The experiments show how implicit dependencies are caused according to the behavior of components. Since component interfaces cannot represent the component behavior, another mechanism is required to specify the behavior. The last case study shows that architecture design is very important for evolving a component-based system.

The following sentences summarize the implicit dependencies described in those experiments.

1. A critical section in the decompressor component depends on the original scheduler, thus a race condition occurs with a new scheduler component. This dependency is completely implicit. That is to say, the dependency is not appeared until the system is built and runs.
2. The behavior of components that invokes bytecode verifiers depends on whether the bytecode verifier runs on local or remote. A stub component allows us to replace from a local one to a remote one in a simple way. However, the networked components have to be taken into account of response delay and exceptions due to the network faults.

3. The scoped memory manager component depends on several other components. A programmer needs to understand the internal of the components, and the side effects of it at the same time.

The result of the case study indicates the existence of behavioral dependencies among components, despite a component is generally defined as a unit of independent deployment. The number of software components will be increased much more, and the constraints for deploying components will become more rigid in ubiquitous computing environments. The behavioral dependencies have to be considered to build a component-based system in a correct way.

Component behavior must be taken into account when building component-based systems, since the behavior causes the inter-component dependency that may prevent the component-based system from configuring in a flexible way. Currently, we are designing a new component framework to allow us to specify implicit dependencies among components by representing the behavior of components explicitly like IOA[5] and CORAL[9].

5 Related Work

Jupiter is a modular and extensible JVM developed from scratch[3]. It focuses on scalability issues of the JVM for high-performance computing. The principle of design and implementation of modules make interfaces small and simple such that UNIX shells build complex command pipelines out of discrete programs. That principle facilitates to modification of JVM functionality. This principle is similar to our component design. Jupiter, however, doesn't address the dependency issues.

Knit has been adopted for building the current version of OSKit[4]. The components of OSKit are well-modularized. Moreover, design patterns are partially adopted for flexibility. Reid et al. mentioned that Knit declarations for OSKit components revealed many properties and interactions among the components that a programmer would not have been able to learn from the documentation alone[7]. This is the same as our observation that a component-based system contributes comprehensibility. OSKit, however, doesn't address the dependency issues except interface dependency processed by Knit.

Kon and Campbell[6] proposed the inter-component dependency management by the human readable descriptions and event propagation mechanisms based on CORBA. Hardware and software requirements are described in a file (e.g. machine type, native OS, minimum RAM size, CPU speed/share, file system, and window manager) with human readable descriptions. And inter-component dependency is managed by the event propagation mechanisms with (un)hook and (un)registerClient methods. However, these methods don't take into account of any component behaviors.

6 Conclusion

This paper has described a component-based Java virtual machine, Earl Gray, and has investigated on component dependencies through three case studies.

The component description explicitly draws the connection among architectural components. Thus, it increases configurability of software.

The case studies have presented dependencies among components. The current component description ensures the consistency between interfaces, but doesn't maintain the behavior of components. In the first case, the problem was caused by the timing of thread scheduling. In the second case, the problem was caused by network errors. In the third case, the problem was caused because adding a new functionality requires to understand various components.

Currently, we are designing a new component framework that can specify the behavior of components, and we will use the component framework to build middleware infrastructures for ubiquitous computing.

References

1. Gregory Bollella, James Gosling, Benjamin Brosgol, Peter Dibble, Steve Furr, and Mark Turnbull. *The Real-Time Specification for Java*. Addison-Wesley, 2000.
2. Anind K. Dey, Gregory D. Abowd, and Daniel Salber. A Conceptual Framework and a Toolkit for Supporting the Rapid Prototyping of Context-Aware Applications. *HUMAN-COMPUTER INTERACTION*, vol.16, pp.99-166, Lawrence Erlbaum Associates, 2001.
3. Patrick Doyle and Tarek S. Abdelrahman. A Modular and Extensible JVM Infrastructure. In *proceedings of the 2nd Java Virtual Machine Research and Technology Symposium 2002 (JVM'02)*, August 2002.
4. Bryan Ford, Godmar Back, Greg Benson, Jay Lepreau, Albert Lin, and Olin Shivers. The Flux OSKit: A Substrate for Kernel and Language Research. In *proceedings of the 16th ACM Symposium on Operating Systems Principles*, October 1997.
5. Stephen J. Garland, Nancy A. Lynch, and Mandana Vaziri. IOA: A Language for Specifying, Programming, and Validating Distributed Systems. MIT Laboratory for Computer Science, October 2001.
6. Fabio Kon and Roy H. Campbell. Dependence Management in Component-Based Distributed Systems. *IEEE Concurrency*, 8(1):26-36, January-March 2002.
7. Alastair Reid, Matthew Flatt, Leigh Stoller, Jay Lepreau, and Eric Eide. Knit: Component Composition for Systems Software. In *proceedings of the Fourth Symposium on Operating Systems Design and Implementation (OSDI 2000)*, October 2000.
8. Clemens Szyperski, Dominik Gruntz, and Stephan Murer. *Component Software: Beyond Object-Oriented Programming*, 2nd ed. Addison-Wesley, 2002.
9. Vugranam C. Sreedhar. ACOEL on CORAL: A Component Requirement and Abstraction Language. In *OOPSLA workshop on Specification of Component-Based Systems*, October 2001.
10. Bill Venners. *Inside The Java 2 Virtual Machine*. MacGraw Hill, 2000.
11. Mark Weiser. The Computer for the 21st Century. *Scientific American*, 265(30), pp.94-104, 1991.
12. CORBA. <http://www.corba.org>.
13. ORBit. <http://orbit-resource.sourceforge.net>.
14. Mario Wolczko. Benchmarking Java with the Richards benchmark. http://research.sun.com/people/mario/java_benchmarking/richards/richards.html.
15. Wonka - The Embedded VM from ACUNIA. <http://wonka.acunia.com>.

Composable Code Generation for Model-Based Development¹

Kirk Schloegel, David Oglesby, Eric Engstrom, and Devesh Bhatt

Honeywell International

3660 Technology Drive, Minneapolis, MN 55418

{Kirk.Schloegel,David.Oglesby,Eric.Engstrom,Devesh.Bhatt}@honeywell.com

Abstract. Many engineering and application domains, including distributed real-time and embedded (DRE) systems, are increasingly employing a graphical model-based development approach. However, the full potential of this approach has not yet been realized due to the complexity of automatically generating non-standard types of code. In this paper, we present a new framework for generating code that is referred to as *composable code generation*. Under this framework, code generators are not written as monolithic programs that are separate from their corresponding graphical models as has been the practice in the past. Instead, code generators are composed of modular entity-specific generation routines that are attached directly to modeling entities, their meta-data, or to collections of modeling entities. Code is built up by traversing the model, querying each entity that is encountered for a specific type of code generation routine and then executing each accessed routine. We describe this framework in detail and provide experimental results from a DRE application domain.

1 Introduction

Many engineering and application domains, including distributed real-time and embedded (DRE) systems, are increasingly employing a graphical model-based development approach. With the recent availability of graphical modeling notations, methodologies, and tools for domain specific modeling, a model-based development approach has the promise of providing a seamless progression of a system model through the different phases of development. These phases include, for example, preliminary design, analysis and simulation, automated code generation, testing, and system integration. However, some core technologies still need to be developed in order to realize this promise. A key enabling technology is a method for robust and complete generation of code, configuration files, inputs to analysis tools, documentation, and other textual artifacts from graphical models.

A number of advances have been made in the area of model-based code generation in the last decade. The result is that it is now commonplace to write a generator for a

¹ This material is based upon work supported by the United States Air Force under Contract No. F33615-00-C-1705. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the United States Air Force.

single type of code given models that conform to a single modeling notation. A number of academic and commercial off-the-shelf tools provide this capability. For example, Rational Rose® can automatically generate structural code (e.g., header files and object shells) for a number of object-oriented languages such as Ada, C++, and Java [11]. Matlab® and Simulink® support the automatic generation of behavioral code in C as well as the automatic generation of inputs for a model simulation engine [9]. GME [7] and PTOLEMY II [8] provide similar capabilities.

Yet current approaches for the automatic generation of code from graphical models are still limited in their usability, flexibility, and customizability for three reasons.

(i) *User-customizable code generators are separate from models.* While a number of tools provide inherent code generation capabilities, these generate only to a limited number of high-level programming languages (e.g., C++) and possibly some types of documentation (e.g., html pages). Furthermore, these are not user customizable. Few model-based development tools support the creation of custom domain-specific code generators beyond providing a set of interfaces that can be used to query information about the model. Under this framework, custom code generators are external to and separate from the model-based development tool. A key disadvantage is that because code generation logic is highly dependent upon the syntactic and semantic structure of the modeling notation, such separation increases the complexity of writing, maintaining, and verifying code generators.

(ii) *There is little support for cross-notation code generation.* Model-based development tools typical utilize a small number of tightly coupled modeling notations. Each modeling notation is naturally amenable to the generation of certain types of code, while less so to others. For example, UML class diagrams (i.e., models that conform to the UML class diagram modeling notation) specify software structure. Therefore, these provide good foundations for generation of object-oriented structural code (i.e., the declarations and shells of classes and methods). Since it is more difficult to model data and control flow with class diagrams, however, behavioral (e.g., method body) code is typically not generated from such models. Conversely, data/control flow diagrams support behavioral code generation well but not structural code generation.

As the demand for more complete code generation has grown, some work has focused on developing cross-notation code generation capabilities. As a simple example, UML class diagrams and data/control flow diagrams could be used cooperatively to describe both the structure and behavior of a software system. Class diagrams can specify software structure, while data/control flow diagrams can specify method body behavior. In this approach, each instance of a concrete operation in UML has a special *implementation* property whose value points to a data/control flow model. Code generation is driven by the class diagram tool. However, this tool hands off control to the data/control flow tool for generation of method body code. This cross-notation approach leads to more complete code generation. Essentially, in this example, two aspects of a system are modeled, software structure and behavior. In the general case, a number of diverse modeling notations could be used to specify different aspects of a complex, multi-model system design.

We refer to modeling entity properties (such as the *implementation* property of UML operations proposed above), whose values are other modeling entities from different notations as *cross-notation linkages* [12]. Cross-notation linkages essentially provide the glue that binds diverse modeling notations together. They are similar in function to relationship arcs that are found in virtually all graphical modeling notations. That is, they specify a relationship that exists between multiple (usually two) modeling entities. The obvious difference is that there can be no visible arcs between the origin and destination entities since these never exist in the same model or view. Another difference is that since modeling notations are syntactically and semantically oblivious to concepts outside of their domains, there are no mechanisms for defining the syntax or semantics of cross-notation linkages from within any of the participant notations. For example, the data/control flow notation is oblivious to the concept of a class, while the class diagram notation is oblivious to the concept of a data flow arc. Therefore, there is not a syntax that specifies how and when classes can be legally linked to data flow arcs. Similarly, there are no semantics to indicate what such a linkage means within either notation.

We would argue that cross-notation obliviousness is a good software engineering practice that should be maintained. However, a lack of syntactic and semantic specification creates challenges for cross-notation code generation. Syntax ensures that cross-notation linkages are sound (e.g., by supporting type checking), while semantics are vital for increased automation (e.g., more complete code generation).

(iii) *There is little support for extensibility or semantic composition.* A number of model-based development tool suites have addressed some of the issues concerning linking together diverse graphical modeling notations in support of cross-notation code generation for specific application domains [6][7][11]. These have seen successes in generating code for specific procedural and/or object-oriented programming languages. However, the few modeling notations that are supported by such tool suites are linked together internally (i.e., a predefined set of cross-notation linkages are hard-coded in the tool infrastructure). They provide no means for developers to customize cross-notation linkages in support of new code generation applications. Also, these tool suites specialize in generating standard programming languages, but provide little support for generating other types of textual artifacts. The result is that the burden is on the code generator developer to extract and integrate concepts from different modeling notations. The problem becomes even more difficult when code generation requires not only multiple modeling notations, but also multiple tools (i.e., the required modeling notations are implemented in different tool suites), as integration of competing tool suites can be problematic [3].

1.1 Our Contributions

In this paper, we present a new framework for generating code and other artifacts, referred to as *composable code generation*, and describe how it addresses the above issues. Under this framework, code generators are not written as monolithic programs that are separate from their corresponding models as has been the practice in the past.

Instead, code generators are composed of modular entity-specific generation routines that are attached directly to modeling entities, their meta-data¹, or to collections of modeling entities. Code generation is accomplished by traversing the entities of the model, querying each modeling entity that is encountered for a specific type of code generation routine and then executing each accessed code generation routine. Further, our framework supports *generator specialization*. Generator specialization increases the flexibility and extensibility of the composable code generation framework by applying the object-oriented programming concept of polymorphism to code generation. Finally, our framework supports *cross-domain selection* that applies the aspect-oriented programming concepts of aspects and join points to code generation to enable rich code generation while maintaining cross-notation obliviousness.

2 Composable Code Generation

We have developed a new framework for generating artifacts from graphical models called *composable code generation*. Composable code generation is the result of applying certain object-oriented programming (OOP) concepts to the area of model-based code generation. The key idea is that graphical modeling entities (i.e., the nodes and arcs that comprise a model along with their internal properties) can be thought of as being analogous to objects in OOP. A natural message to send to such an object is "provide your code generator given a descriptor". A descriptor is a human-understandable description of the type of code to generate (e.g., "Java", "C++", "Documentation", and "Schedulability Analysis Input"). Under this framework, code generation becomes, not simply executing a monolithic program that is external to the model-based development tool, but an integrated process. In this process: (i) The model is traversed in some fashion. (ii) Each entity encountered is queried for its appropriate code generator of a given type, and this generator is executed. (iii) The output of each generator is concatenated to build up either the raw data that must then be formatted to correspond to the grammar of the generated language, or else the formatted output. (In this latter case, output formatting must be performed on the fly.)

In OOP, operations are specified inside of class specifications. When objects are instantiated, they may perform those operations that have been specified in their class or superclasses. In model-based development tools, instances of modeling entities have an analogous relationship to their meta-data. Therefore, it is natural to attach code generation routines (and their descriptors) to meta-data. When an entity is queried for its appropriate code generation routine, it returns the applicable routine that is attached to its meta-data. This scheme provides a base functionality for the composable code generation framework.

As an example, a simple *documentation* code generator could be constructed using only a single routine that is attached to the meta-data of all types of modeling entities. This routine has three steps: (i) print out the name of the associated entity, (ii) iterate

¹ The meta-data for each type of modeling entity defines the sets of constraints, properties, and operations for all instances of the type. The aggregation of these defines the modeling notation.

through all sub-entities to access their *documentation* code generation routines, and (iii) execute the returned routines to build up the code. Figure 1 shows the code that would be generated from such a composed code generator for the UML class, *Square*.

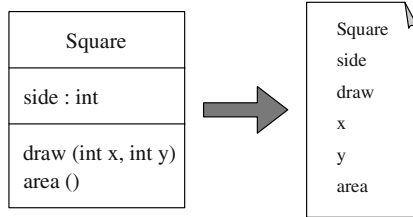


Fig. 1. A UML class and its corresponding documentation code

2.1 Advantages

Current code generation approaches can be thought of as analogous to programming with pointers in C. While extremely flexible, the program data and operations are separate. Therefore, a good deal of the software lifecycle cost is spent simply ensuring that the data is found and interpreted correctly by the operations. Our composable code generation framework essentially encapsulates code generation operations within the meta-data. Therefore, calling an entity-specific generator is similar to sending a message to an object. It is not surprising that our composable code generation framework shares many of the same software engineering advantages as OOP. For example, the close integration between graphical models and code generation routines is conducive to the development of small, modular, and easy to understand routines, making debugging and maintenance easier.

The close integration between graphical models and code generation routines also makes it easier to reason and prove qualities about generated code compared to current code generation techniques. For example, in model-based composable code generation, graphical modeling entities can be interactively examined in order to determine exactly which routines will be called during code generation. Monolithic code generation programs, on the other hand, rely on dynamically evaluated switching statements and other control primitives that make it extremely difficult to provide an analogous capability. Also, it is quite natural in model-based composable code generation to only *compose* a code generator and not to *execute* it. That is, during the code generation process, the model is traversed, and each modeling entity encountered is queried for its appropriate code generation routine. Normally, this code generation routine is immediately executed and its output is concatenated with the output obtained from other modeling entities. Alternatively, the returned routines could be written to a file. A code generator composed in this way would be structurally flat without major iterations or subroutine calls. It is much easier to analyze flat code than to analyze code containing loops and dynamic control flow statements.

Our framework also supports multi-tool cooperative code generation. When an entity is queried for its appropriate code generation routine, a predefined shell script can be returned instead of a traditional generation routine. This script can start up an external model-based development tool, load a specific file, generate tool-specific code, and format and return the results. All of this can be done transparently to the tool that drives the code generation.

2.2 Generator Specialization

In order to improve the flexibility and extensibility of our framework, we extended it to allow code generation routines to be attached, not only to the meta-data of entity types, but also to user-defined collections of modeling entities as well as to instances of modeling entities. Under this extension, it is possible that several code generation routines with the same descriptor can be associated with a single modeling entity. (For example, one could be attached to the meta-data and the other could be attached directly to the entity instance.) However, when such an entity is queried for its *appropriate* generator, a single routine must be returned from this set. Therefore, it is necessary to arbitrate between generators whenever a collision occurs. We developed a mechanism called *generator specialization* to support this functionality that is conceptually similar to polymorphism in OOP. Here, the selection among multiple applicable code generators is based upon the concept of specialization (i.e., specific routines are rated higher than general routines).

We have identified four distinct categories of generator specialization that are useful in code generation for DRE system development. These are (from most general to most specific) base, stereotype, scoped, and instance specialization.

Figure 2 is used as a running example to illustrate these concepts. It shows two models, *Software Model* and *Hardware Model*. The Software Model conforms to the abstract software modeling notation that is defined by the *Software Meta-model*. Similarly, the Hardware Model conforms to the abstract hardware modeling notation that is defined by the *Hardware Meta-model*. Within each meta-model, meta-entities define the structural and connectivity properties and constraints for different types of modeling entities. For example, the *Operation* meta-entity from the Software Meta-model defines the structural and connectivity properties for all modeling entities that are of type *operation*. The operation *m1()* of class *A* in the Software Model is an example of one of these. In 0, code generators are represented by shaded polygons that are attached to entities by dashed arrows. (Note that code generation routines and their connections to modeling entities and meta-entities need not be visible as shown in Figure 2 unless the properties of the entity are explicitly examined.)

Base code generation routines are attached to the meta-data that defines the specific type of modeling entity (as discussed in Section 2). Base code generation routines are the default routines that are used in the absence of further specialization. In Figure 2, the base code generator for all operations (i.e., those entities of type *operation*) is

shown in the Software Meta-model as represented by a shaded hexagon. If this were a Java code generator, for example, it might consist of the following.

```

$visibility $returnType
$name "(" iterateOverParameters("Java") ")" {
    $body
}

```

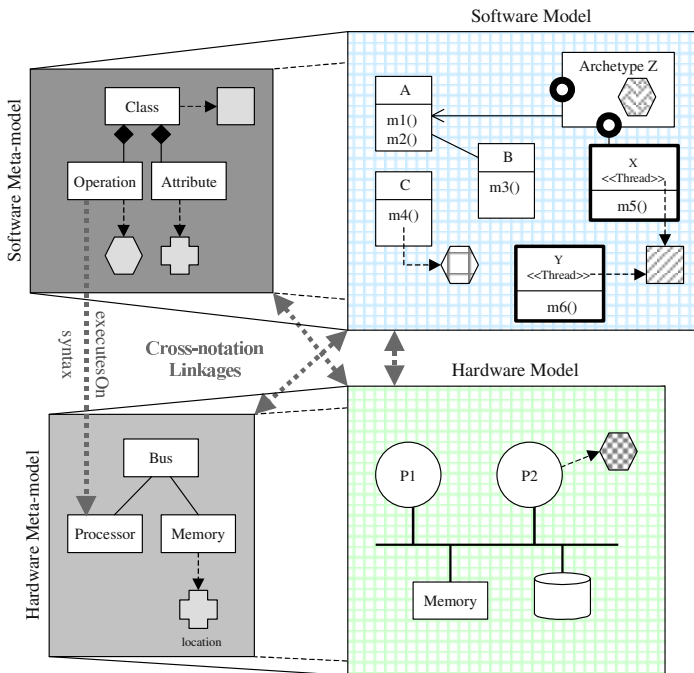


Fig. 2. A multi-model design of an embedded system along with the underlying meta-models for the two interacting modeling notations

Here, `$visibility` returns the string representation of the visibility property of the particular operation. Similarly, `$returnType`, `$name`, and `$body` return the string representations of the various entity-specific properties. The `iterateOverParameters` subroutine traverses the parameter sub-entities of the operation, queries each for its Java code generator, and then executes the returned routine. In this case, the base Java code generator for modeling entities of type *parameter* could be as follows:

```
$type $name $delineator
```

where `$type` returns the type of the parameter, `$name` returns its name, and `$delineator` returns either `,` `"` or the empty string depending on whether or not this is the final parameter.

Stereotype code generators are specialized routines that can be attached to user-defined collections of modeling entities. These are more specialized than base code generation routines, and therefore override them. For example, the *X* and *Y* stereotyped nodes in the Software Model use the specialized class code generator shown in the Software Model (represented by a shaded square) instead of the base class code generation routine shown in the Software Meta-model. In this example, the classes *X* and *Y* are stereotyped as *Threads*. In general, non-functional entities such as threads and processes are not treated the same as functional UML classes during code generation. However for DRE systems, it is often necessary to model these along with their properties and relationships in order to support the generation of many types of non-standard code (e.g., thread- and middleware-specific configuration code).

Scoped code generators are attached to specialized architectural constructs (e.g., archetypes [10]). These override base and stereotype code generation routines within their scope. For example, the archetype *Z* contains an internal code generation routine (shaded hexagon) that overrides the base operation code generation routine for all operations within its scope (i.e., those operations that are internal to the archetype).

Finally, *Instance* code generators are attached directly to modeling entity instances. These override all other code generation routines. For example, an instance code generator is shown in Figure 2 attached directly to operation *m4()* of class *C* in the Software Model.

3 Cross-Domain Selection

Cross-notation linkages as discussed in Section 1 represent the most basic of modeling concepts (i.e., a relationship between multiple modeling entities from different models). These are otherwise without syntax and semantics. However, new and complex types of linkages can be defined by building higher level syntax and semantics on top of simple cross-notation linkages.

Figure 2 provides an example. In the DRE system that is being modeled, each software component will execute on some specific hardware platform(s). Of course, the partitioning of software components has an impact on the system performance. Therefore, it is useful to model this relationship at design time. One way to do so is to define a new type of cross-notation linkage called *executesOn* and to specify additional syntax and semantics for it. In particular, such a linkage could only be legally made between an operation node and a processor node. Semantics could also be defined for *executesOn* linkages to impact code generation. For example, assume that a vendor-optimized software library exists for a subset of the processors in the model. Of course, these routines should only be called for code that is running on the specific hardware. Otherwise, the default library routines should be called. Figure 2 illustrates this example. Here, all operations that are executed on P2 (i.e., all of those that are linked to P2 via an instance of the *executesOn* linkage) should call the vendor-optimized software library as opposed to a default software library. In order to gener-

ate such code correctly, when a modeling entity (in particular a software operation) is queried for its applicable code generator, it should return the routine that is attached to P2 (shaded hexagon) in preference to other routines.

We have included support for such cross-domain selection of code generation routines in our composable code generation framework. Users can write application-specific selection routines in a functional language based on Scheme. Such a routine should return exactly one from a set of applicable routines based on the current structure and properties of the design. Conceptually, cross-domain selection is similar to aspect-oriented programming (AOP) rather than OOP. The detailed example in Section 3.1 illustrates this point. Cross-domain selection is useful for supporting the generation of diverse types of code while maintaining cross-notation obliviousness.

3.1 Example

DRE systems typically rely on COTS middleware to support communications among distributed components. Those middleware packages that do not support dynamic communications require a static list of every type of communication that could occur between distributed software components. This is used at system startup time to allocate memory and configure the middleware. Such information is often specified in a special-purpose middleware configuration file. Configuration files are useful during development as they eliminate the need for certain design-specific information to be hardcoded in the source code. Therefore, a recompilation of the system is not required every time a slight modification is considered. This encourages in-depth exploration of the design space. Also, such a capability supports early debugging, performance analysis, simulation, and verification of the design.

For similar reasons, it is useful to be able to automatically generate middleware configuration files during system development. However, generating configuration files is a non-trivial problem due to the cross-domain nature of DRE middleware. A single modeling notation (e.g., UML class diagram) is typically not sufficient to specify in a straightforward manner the information that is required for complete code generation.

Figure 3 through Figure 5 illustrate an example of how a simple middleware configuration file can be generated under the composable code generation framework with cross-domain selection. Figure 3 shows a data/control flow model of a simple embedded system. This figure contains three software components (*A*, *B*, and *C*) and two data pull arcs (*1* and *2*). The software components are periodic (i.e., they execute at regular time intervals). The period of each component is shown as a property of the component. For example, the sensor has a period of ten. At a high level, the following behavior is represented. Every ten time units, the sensor, *A*, senses the environment and writes the resulting data to memory. Every five time units, the signal-processing component, *B*, performs a remote method invocation upon the sensor component to get the new data. After this method returns, the signal-processing component processes the data. Every single time unit, the display component, *C*, per-

forms a remote method invocation upon the signal-processing component to get its processed data. Finally, the display component displays this data.

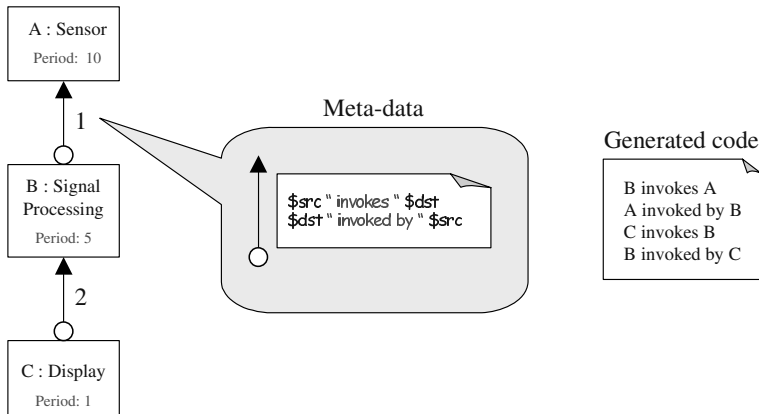


Fig. 3. A data/control flow model along with a portion of the meta-data for data pull arcs and an example of the code that would be generated from the model

Figure 3 also shows some of the meta-data that is associated with data pull arcs. Specifically, it shows the base code generator for generating a simple middleware configuration file. In order to generate a new configuration file, the top-level middleware configuration code generator simply traverses the arcs of the model in name rank order, queries each arc for its appropriate generation routine and then executes that routine. The resulting code is shown to the right of the meta-data.

This code, while correct, is not optimized for the particular hardware on which the system will run. Indeed, it cannot be, for there is no information that specifies the hardware in either the model or in the code generation routines. In order to model the embedded system more accurately - in particular, the non-functional aspects of the system - the hardware can be modeled along with a mapping of software to hardware.

Figure 4 shows a portion of the hardware architecture model for the example from Figure 3. Here, four processors are shown (1, 2, 3, and 4). Processors 1 and 2 are connected to each other via a dedicated link, while processors 2, 3, and 4 are connected to each other via a bus that is shared with other hardware resources (not shown). Figure 4 also shows a portion of the meta-data that is associated with the shared bus entity. In particular, it shows a code generation routine along with its cross-domain selection criteria. Essentially, the routine is targeted for data pull arcs whose source and destination components are mapped to processors that are connected by a shared bus and whose source component's period is less than its destination component's period. This cross-domain code generator represents an optimization strategy. That is, it may decrease the bus traffic if a proxy component is placed between two such components.

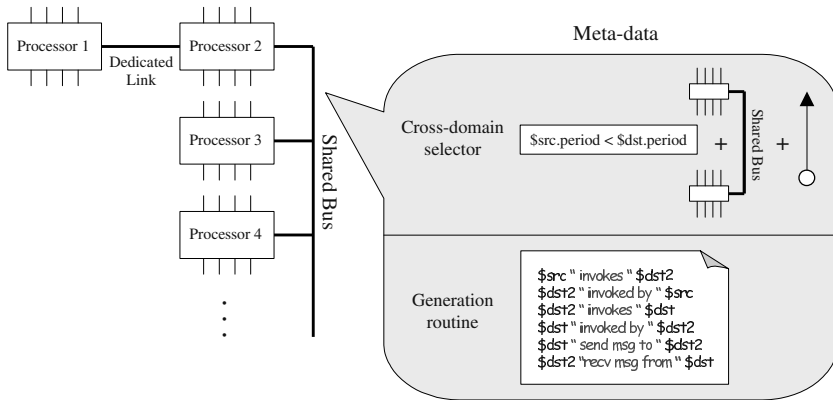


Fig. 4. A portion of the hardware architecture model for an embedded system along with a cross-domain code generator that is associated with the shared bus modeling entity

Figure 5 shows a mapping of software components to processors. Here, the sensor component is mapped to Processor 1, the signal-processing component is mapped to Processor 2, and the display component is mapped to Processor 3. After the software is mapped to the hardware as such, different code will be generated under the composable code generation framework. This is because when data pull arc 2 is queried for its appropriate generation routine, the new cross-domain selector will take effect and the code generation routine from Figure 4 will be returned. The code shown to the right in Figure 5 will be generated.

The high level description here is somewhat different than as discussed above. Again, the sensor senses and records its data every ten time units. And again, the signal-processing component performs a remote method invocation upon the sensor component every five time units and then processes the new data. Next, however, the signal-processing component sends an event to the automatically generated proxy component *B2* that is co-located with the display component. (This event is represented by the push control arc between nodes *B* and *B2* in the right of Figure 5.) Upon receiving this event, the proxy component simply performs a remote method invocation on the signal-processing component to get its new data. Every time unit, the display component, *C*, performs a local method invocation on the proxy component to get its data. This optimization effectively reduces the traffic on the shared bus by caching the signal-processing data on Processor 3.

It is shown by this example how cross-domain selection helps to support the automatic generation of proxy components. Furthermore, our framework is flexible enough to allow either the optimized or the normal code to be generated easily. In addition to its other checks, the cross-domain selector can be made dependent upon a dynamic model property. Essentially, this allows the optimization to be enabled or disabled during design time as easily as flipping a switch.

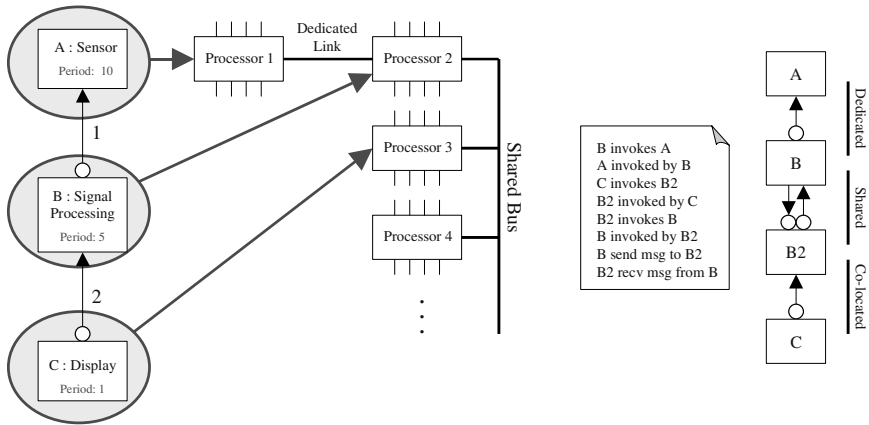


Fig. 5. A mapping of software components to hardware along with the code that would be generated utilizing the cross-domain selector in 0

4 Implementation

We have implemented our composable code generation tool utilizing a meta-modeling tool called the Domain Modeling Environment (DOME) [1]. A domain-specific model-based development tool can be defined in DOME using a graphical meta-model along with textual behavioral subroutines. The meta-model specifies classes, properties, structural constraints and visual attributes of a modeling notation, while the textual scripting language implements syntactic constraints and semantic behaviors. DOME also provides support for generating textual artifacts from models in the form of a document generation toolkit (called *MetaScribe*).

We have extended the DOME tool to support the composable code generation framework. There are four main issues here. We discuss each briefly.

- (i) *Attaching generators to modeling entities.* Code generation routines can be attached to modeling entities, meta-entities, and collections of entities by defining an additional property on general entities (be they meta-entities or entity instances). This property is a list of key-value pairs (i.e., a dictionary) in which the key is a string descriptor and the value is a file name. The named file stores the code generator.
- (ii) *Model traversal.* Model traversal for the types of structural hierarchies that are likely to exist within graphical modeling notations is a well-understood area. Efficient algorithms exist for traversing models to locate and query entities of interest. Examples of these include depth- and breadth-first traversal, attributed-based sorting, topological sorting, as well as various filtering techniques.

(iii) *Cross-notation linkages*. We have implemented a cross-notation linkage modeling notation (i.e., a modeling notation in which cross-notation linkages can be modeled) based upon the work described in [12]. The key idea here is that the interactions that exist between entities from different modeling notations represent complex systems themselves that can be graphically modeled and semantically interpreted.

(iv) *Generation routine selection*. We have implemented a default selection scheme using the textual scripting language available within DOME. This scheme is based upon the four generator specialization categories (base, stereotype, scoped, and instance). User-defined routines are also supported for cross-domain selection.

5 Experimental Results

In this section, we briefly describe results obtained under the DARPA MoBIES program [1]. Our task was to develop a multi-model design capability for an open experimental platform that is based upon a jet fighter weapon and navigation system. Our tool needed to provide automatic generation of XML code for (i) middleware configuration and (ii) import into an event-dependency analysis tool [5].

The multi-model design tool we developed is comprised of eight interacting modeling notations. These include notations for modeling hardware and software architectures, software component behaviors and interactions, component-to-process-to-processor mappings, hardware fault modes, internal state transitions of components, and internal structures of components. Our tool utilized twenty cross-notation linkages. A model of the cross-notation linkages is shown in Figure 6. We wrote thirty-five code generation routines in support of the two code generation requirements. These are quite short and modular. The average length being 51 lines, the minimum is 6 lines, and maximum is 199 lines. An example generation routine is at

<http://www.htc.honeywell.com/projects/mobies/papers/codegenTemplate.pdf>.

Throughout the project, the specifications for both code generation requirements grew and evolved. Therefore, our code generation routines had to be continually updated. Most of these changes were minor. However, midway through the project, the specification for the middleware configuration file changed significantly. We timed how long it took to update our code generators to become compliant with the new version. The entire process took less than eight hours. While we do not have data from a competing approach to which we can compare this result, we do feel that this is quite reasonable and demonstrates the agility of our scheme.

We tested our tool and its code generators on a number of multi-model designs from the DRE weapon / navigation system domain. The largest of these consisted of over forty interconnected models and hundreds of modeling entity instances. Over 4,400 and 2,800 lines of code were generated from this model for the two requirements.

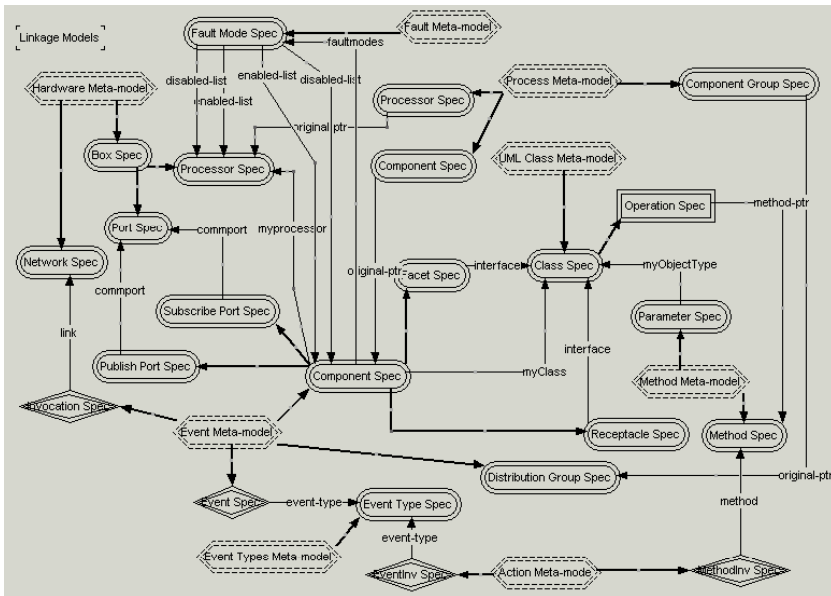


Fig. 6. The cross-notation linkage model for our multi-model design tool. Meta-models are represented as dashed hexagons. Node meta-data are represented as rounded-edged rectangles. Arc meta-data are represented as diamonds. Wide arcs indicated a *contains* relationship. Thin, labeled arcs indicate a cross-notation linkage

6 Conclusions

We have presented a new framework for composing model-based code generators that extends the state of the art in model-based code generation by applying certain OOP techniques to the area. It is reasonable then to analyze our approach in terms of object-oriented design patterns [4]. Our framework touches upon a number of common patterns at various levels, such as *Command*, *Strategy*, *Template Method*, and *Visitor*. At a high level, composable code generation applies the Template Method pattern. Here, the model-traversal algorithm plays the role of the invariant algorithmic skeleton, while the returned code generation routines play the role of the variant behavior. The polymorphic behavior of our framework can be implemented using either the Strategy or Command patterns. Also, the Visitor pattern is relevant as it provides a mechanism to add code generation operations to meta-data in a manner that is oblivious to the base modeling notation.

We have also described a mechanism for supporting the generation of cross-notation code that is based on concepts found in AOP. Figure 4 illustrates an example of this. The code generation routine shown here along with the cross-domain selection rules are analogous to join point and aspect code in AspectJ™ [13]. While this example is somewhat ad hoc, applying AOP techniques to our framework in a more formal man-

ner holds the promise of increasing coverage, while further simplifying cross-notation code generation. Another area of further research is the generation of AOP code as a target language. For example, join points and aspects could be automatically generated given cross-notation linkage models.

Apart from the experimental results presented in Section 5, we have obtained results from our composable code generation implementation in conjunction with our archetype modeling technology [10]. These results are described in a brief document: <http://www.htc.honeywell.com/projects/mobies/papers/codegenComposite.htm>. The document explains how the composite design pattern is captured as a reusable archetype and how code is generated from instances of the archetype. This document is also of interest because it includes a screen capture animation of the composable code generation process.

References

- [1] DARPA MoBIES Program. <http://dtsn.darpa.mil/ixo/programdetail.asp?progid=38>.
- [2] DOME is an open source research project available at <http://www.htc.honeywell.com/dome>.
- [3] A. Egyed and R. Balzer. *Unfriendly COTS Integration – Instrumentation and Interfaces for Improved Plugability*. In Proc. of 16th Conf. on Automated Software Engineering (ASE 2001), 2001.
- [4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995.
- [5] Z. Gu, S. Kodase, S. Wang, and K. Shin. *A Model-Based Approach to System-Level Dependency and Real-Time Analysis of Embedded Software*. In Proc. of IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS 2003), May 2003.
- [6] Kennedy Carter. *Supporting Model Driven Architecture with eExecutable UML*. Technical Report, <http://www.kc.com>, 2002.
- [7] A. Ledeczi, A. Bakay, M. Maroti, P. Volgyesi, G. Nordstrom, J. Sprinkle, and G. Karsai. *Composing Domain-specific Design Environments*. Computer, pages 44-51, November 2001.
- [8] E. Lee et al. *PTOLEMY II: Heterogeneous Concurrent Modeling and Design in Java*. <http://ptolemy.eecs.berkeley.edu>, 2002.
- [9] The MathWorks, Inc. *MATLAB User Guide*. Natick, MA 01760-1500, 1998.
- [10] D. Oglesby, K. Schloegel, D. Bhatt, and E. Engstrom. *A Pattern-based Framework to Address Abstraction, Reuse, and Cross-domain Aspects in Domain Specific Visual Languages*. In Proc. of OOPSLA 2001, 2001.
- [11] T. Quatrani. *Visual Modeling with Rational Rose and UML*. Addison-Wesley Object Technology Series, 1997.
- [12] K. Schloegel, D. Oglesby, E. Engstrom, D. Bhatt. *A New Approach to Capture Multi-model Interactions in Support of Cross-domain Analyses*, 2001.
- [13] Xerox Corporation. *The AspectJ™ Programming Guide*. <http://www.aspectj.org/>, 2002.

Code Generation for Packet Header Intrusion Analysis on the IXP1200 Network Processor

Ioannis Charitakis¹, Dionisios Pnevmatikatos¹,
Evangelos Markatos¹, and Kostas Anagnostakis²

¹ Institute of Computer Science (ICS)
Foundation of Research and Technology - Hellas (FORTH)
P.O.Box 1385, Heraklion, Crete, GR-711-10, Greece
`{haritak,pnevmati,markatos}@ics.forth.gr`

² Distributed Systems Laboratory
CIS Department, Univ. of Pennsylvania
200 S. 33rd Street, Phila, PA 19104, USA
`anagnost@dsl.cis.upenn.edu`

Abstract. We present a software architecture that enables the use of the IXP1200 network processor in packet header analysis for network intrusion detection. The proposed work consists of a simple and efficient run-time infrastructure for managing network processor resources, along with the S2I compiler, a tool that generates efficient C code from high-level, human readable, intrusion signatures. This approach facilitates the employment of the IXP1200 in network intrusion detection systems while our experimental results demonstrate that provides performance comparable to hand-crafted code.

1 Introduction

Network processor vendors have invested considerable effort in tools for cost-effective software development, however, building an application for a network processor is still a non-trivial task. To address this difficulty, recent work has demonstrated the use of component models for simplifying development (c.f. [1,8,2]). This work focuses on forwarding and routing services, exploiting application modularity in a divide and conquer approach in order to map parts of the application to network processor execution resources. The main design goal is primarily flexibility and design modularity which usually comes at the price of some performance penalty.

Network monitoring and network intrusion detection are becoming increasingly important network-embedded functions[6,5]. Network Intrusion Detection Systems improve security for organizations by monitoring in real time the traffic that crosses the border of their networks. They passively inspect traffic to determine if it matches an attack profile. The simplest and most common form of NIDS inspection is to analyze packet headers and match string patterns against the payload of packets. A popular open-source NIDS is **snort** [5], that uses signatures to describe a set of known forms of attacks. **Snort** signatures consist

```

alert    tcp 10.0.0.0/32 any -> 10.0.0.1 80      (dsize: >512;)
alert    tcp [10.0.0.1 10.0.0.2] any -> 10.0.0.2 80 (ack: >512;)
alert    tcp any any -> 10.0.0.300 20           (dsize: >512;)

```

Fig. 1. Example of **snort** signature file. Note the use of *any*, which serves as a wild-card

of three parts: the *action* (e.g. alert), the *header* (protocol + source[IP,port] + dest[IP, port]), and the *options* (ip-flags, ip-options, tcp-options, etc). Figure 1 shows a few examples of actual **snort** signatures.

The flexibility required by the dynamic nature of intrusion detection applications, along with their inherently large processing needs makes network processors an ideal implementation technology. However, these applications differ substantially from services such as packet forwarding that have been studied so far. In this paper we present the **snort** To IXP compiler (S2I), a tool to facilitate the deployment of the IXP1200 network processor in a **snort**-based NIDS. The input of the S2I compiler is a regular **snort** configuration file, which contains signatures for a collection of known intrusions (some typical signatures are shown in Figure 1). Each signature is defined in a high-level language and describes the action to be performed (e.g. alert, log, etc) when a packet satisfies a set of conditions.

S2I transforms such a set of **snort** signatures into efficient C code for the micro-engines of IXP1200. The transformation is performed using a tree-structure in order to minimize the number of required checks. The resulting code together with a general runtime environment can be compiled, optimized, and loaded on the IXP1200 using the standard tool chain. There are three main benefits from this approach. First, it offers faster execution speeds of the resulting code, which is comparable to hand-crafted code. Second it provides versatility since adding or changing the signatures involves only running tools and not hand-tuning. Finally, it offers transparent resource management.

Using cycle-accurate simulations we measure significant reduction in both the required space and execution time. Space improvements range from about 14% to 42%, with the improvement magnitude increasing with the number of signatures. In addition, execution time improves by about 20%.

The rest of this paper is organized as follows. Section 2 provides a brief overview of the target architecture, and Section 3 describes the S2I compiler. Section 4 contains an experimental analysis, Section 5 presents related work, and Section 6 concludes and discusses our plans for future work.

2 The Intel IXP1200 Network Processor

2.1 General Description

A block architecture for the IXP1200 network processor is shown in Figure 2. The IXP1200 consists of the following basic components:

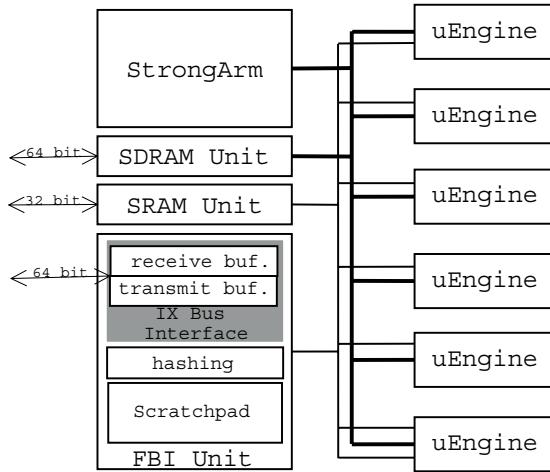


Fig. 2. Block architecture of the IXP1200 Network Processor

- A StrongARM host processor, capable of operating at 232 MHz.
- Six micro-engines operating at the same frequency as the host processor.
- An SDRAM unit communicating to external SDRAM via a 64 bit bus at 116 MHz.
- An SRAM unit communicating to external SRAM via a 32 bit bus at 116 MHz.
- The FBI unit which provides 1 KB memory of 32 bit words (the scratchpad), a hash unit and the IX bus unit. The later interfaces to network interface cards (NICs) and provides the Receive Buffer and the Transmit Buffer.

2.2 The Micro-engines

Each micro-engine (or uEngine) is a simplified RISC processor that has hardware support for four threads of execution. Special instructions allow a thread of execution to be swapped out until a certain event occurs. Such events are “data-written”, “data-read”, “signaled”, etc.

Each thread receives one fourth of the register space of the uEngine using relative register referencing, while absolute register referencing can be used for thread communication via shared registers. Each uEngine contains 128 32-bit general purpose registers, and 128 32-bit special purpose registers dedicated for data transfers (e.g. SRAM/SDRAM read-only and write-only registers, etc).

The uEngines can fetch data directly from SRAM, SDRAM, and the FBI (scratchpad, receive and transmit buffers, etc), but cannot exchange data directly between them. Instead, uEngines can exchange messages through shared memory

accesses which are expensive: for example, reading 4 bytes from the SRAM takes 22 cycles.¹

The uEngines are responsible for managing the buffers of the network interface ports e.g. monitoring the state of the buffers and initiating data transfers when necessary.

Each uEngine has 2 KB of instruction memory; programs are uploaded by the host processor by writing in a specific memory address region.

3 The S2I Compiler

The `snort-to-IXP1200` (S2I) compiler generates micro-C² code for the uEngines from a `snort` set of signatures. The generated code consists of a static and a dynamic section. The static section is a skeleton that is independent of the set of signatures and contains the fixed run-time infrastructure needed to dispatch packets for processing. The dynamic section is produced from the `snort` set of signatures and performs the actual computation to analyze packet headers and trigger the corresponding actions.

The space and performance benefits of S2I are based on the following observation. An interpretive approach where the signatures are kept in data structures in memory is expensive both in time (e.g., executed instructions and memory references) and space since for each signature, the interpreter input can be a large structure defining which fields to check, what operation to perform and against what value. A compiled approach is faster since it avoids the interpretation cost, and allows for standard compiler optimizations. The compiled approach may also result in more compact code since many of the constants can be embedded in the instructions themselves, saving space.

An essential optimization pass performed by S2I is common-subexpression elimination using an expression evaluation tree. When several signatures share the same *prefix* conditions, these conditions are evaluated only once. Organizing the signature checks in a tree saves both space (each datum is stored once) and time (each condition is evaluated once). While this possibility is available to the programmer as well, implementing the code for a large number of signatures is error prone, reduces code readability, and is very hard to adapt to a new set of signatures. S2I provides performance close to that of hand-crafted code while offering the advantage of a standard and manageable high-level input specification.

3.1 Static Section

We describe the structure of the static section of the generated code. It consists of the necessary minimal infrastructure for basic packet handling as well as an

¹ Newer models (the IXP2xxx series) support more efficient inter-uEngine communication, by chaining uEngines and by providing shared registers between neighboring uEngines.

² micro-C is an enriched version of the C language provided by Intel. It provides primitives to support the architectural advantages of the uEngines.

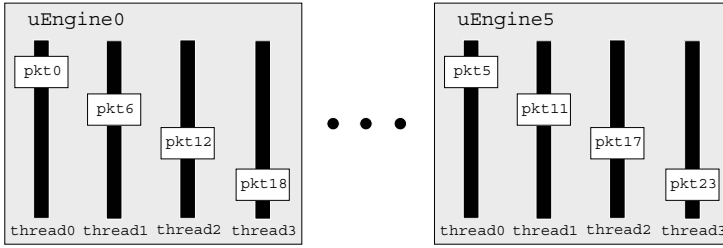


Fig. 3. Thread based scheme: Packets distributed among the threads. All threads execute the same code

algorithm for distributing the packet processing load to different units of the network processor. Currently we target 100 Mbit/s ports.

For the purpose of our particular design, each packet is processed in-full by a single uEngine. Finer-grained load distribution would require inter-uEngine transfers of data as some processing to occur in one worker and the rest to be handed off to another, due to different packets requiring different amounts of processing. Since inter-uEngine transfers are not efficiently supported by hardware, such a scheme was not considered at all for this work.

The basic approach for load distribution is therefore to assign each packet to a single worker for its entire processing. This results in a reasonably balanced system, as a busy worker cannot issue requests for more work and therefore new packets will be assigned to the least busy uEngine. This approach also minimizes accesses to shared resources as the work for each packet is isolated on a single uEngine.

Because of the multi-threaded structure of the IXP1200, a worker for a packet can be either an individual thread or an entire uEngine. We therefore consider two different methods for load distribution presented below.

The **thread-based scheme**, (shown in Figure 3) assigns the entire processing of each packet to one of the four threads of a uEngine. This has the advantage of simplicity, and yields the same code for all threads. The 2 KB of instruction memory are unified and shared by all threads.

One drawback of the thread-based scheme is that the registers of each uEngine must be equally divided among the four threads. Each thread has to fetch the headers of its packet in its local registers, for processing, consuming 14 registers: 54 bytes are needed for Ethernet, IP and TCP headers. A total of 56 registers are therefore needed for all threads, corresponding to 30% of the total number of registers that can be read. Another disadvantage is that processing of the four packets inside the uEngine is done in an interleaved manner, meaning that only one thread is active and only one fourth of these registers are actually used at any given time.

Synchronization among the threads is accomplished in two steps. First, we allow one thread from each uEngine to start issuing willingness of receiving a packet. On the second step, up to six threads (one for each uEngine) race to lock

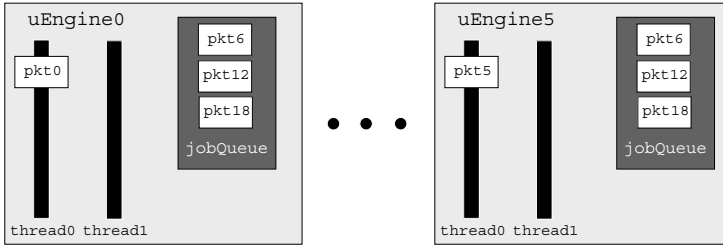


Fig. 4. uEngine based scheme: Packets distributed among the uEngines. One thread does the actual header checking, while the other one maintains the jobQueue

the input port. The winning thread receives a packet, releases the acquired lock, and commences signature checking. Meanwhile, the other threads can perform the same synchronization method in order to serve the next packet.

An alternative to the thread-based scheme is the **micro-engine-based scheme**, where an entire uEngine is allocated for serving a packet. This is illustrated in Figure 4. In this case, the threads are responsible for specific jobs of packet processing, such as moving the packet between microengine and SDRAM and performing the actual header processing.

In contrast with the thread-based scheme, the uEngine based scheme results in one packet being active per uEngine, consuming only 14 registers for packet headers. Leaving more local space available enhances the chances for the dynamic section to fit entirely within a uEngine. In other words, we want all the variables that are necessary to perform the signature checking to be mapped to registers. If there are not enough registers, some variables will have to be mapped to scratchpad or SRAM which greatly degrades performance.

Note, that in the thread-based scheme, there are four packets concurrently in each uEngine. In order to give the same processing time for each packet to the uEngine based scheme, small buffers should be kept. These buffers (jobQueues) ideally will hold pointers to three more packets, that this uEngine should process in the future. Now, the processing of these packets is done serially, rather than interleaved.

This scheme was supported further with a simpler synchronization method. Each uEngine signals the next one to start polling for arrived packets. Therefore accesses to memory (for locking/unlocking) are avoided, and the system behaves much more smoothly.

Both schemes demonstrate similar performance. However, since the uEngine based scheme requires much fewer resources, it was chosen to work with.

3.2 Dynamic Section

In this section we present the dynamic section, i.e. the generated micro-C code. This code is dynamic in the sense that it can be automatically reproduced every

time a new set of signatures is used. In this context, dynamic does not imply that the executed code is changed during run time. Certainly, it would be desirable to operate continuously and never having to interrupt packet monitoring. However, this flexibility would sacrifice performance if it was built-in the architecture of the monitoring system. On the other hand, there may be other ways to achieve both constant operation and flexibility without sacrificing performance (e.g. using redundancy). This can be subject of future work.

The dynamic section is the output of the S2I compiler for the particular **snort** input file. S2I does not yet support all **snort** features. More specifically, this version of S2I does not support payload searches. In an overview the functionality of the S2I compiler is divided in to two basic tasks. Firstly it builds a tree-like representation of the signatures in the input file and secondly it produces the corresponding micro-C code.

Building the Tree. Having a complete array of signatures, i.e. the complete input file in an internal representation, the S2I compiler starts combining the signatures in a tree structure. In this tree, each level corresponds to checking a specific field. For example at the first level we check for the protocol field, while at the second level we check for the destination port. This is depicted in Figure 5 where we show the resulting tree of the signatures presented earlier in Figure 1.

The S2I compiler initializes the tree using the first signature. Then, for each next signature, it starts combining each of the fields into the tree, following a predefined order.³ New nodes are generated if the added field of a signature checks against a value that has not been seen earlier. The algorithm that builds the tree sorts each level from the most specific values to the most general. Therefore, stricter signatures will be checked before more general.

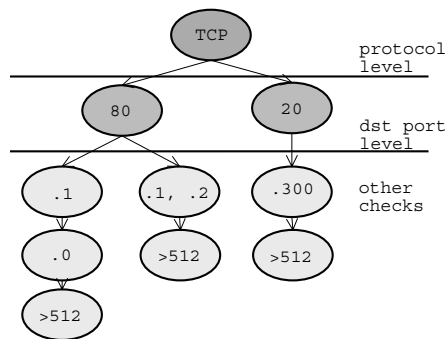


Fig. 5. Resulting tree from the signatures presented earlier in Figure 1

³ In the future, we plan to guide this process with the assistance of a profiler, in order to intelligently select which checks to perform first.

Producing the Final Code. During this phase, the S2I compiler generates the code that will be merged and compiled together with the static infrastructure presented earlier. The final object file will be loaded in the uEngines, and monitoring can be initiated. In Figure 6 we present a snapshot of some sample code produced by the S2I compiler.

```
(.....)
if (ETHPROTOCOL==0x0800 && PROTOCOL==0x6) {
    if (PORT2==0x50) {
        if (IP2==0xa000001) {
            if (IP1==0xa000000) {
                if (DSIZE>0x200) {
                    /*Action for "tcp 10.0.0.0 any -> 10.0.0.1..." */
                }
            }
        }
        ctx_swap();
        if (IP2 == 0xa000002) {
            if (IP1==0xa000001 || IP1==0xa000002) {
                if (ACK>0x200) {
                    /*Action for "tcp [10.0.0.1 10.0.0.2] any..."*/
                }
            }
        }
    }
}
(.....)
} //<<<<PORT2
if (PORT2==0x14) {
    if (IP2==0xa00012c) {
        if (DSIZE>0x200) {
            /*Action for "tcp any any -> 10.0.0.300 20..."*/
        }
    }
}
ctx_swap();
```

Fig. 6. Generated code for the tree of Figure 5

It is important to note the use of constant literals in the various checks. For example, the signature that checks for the destination port 80, will be compiled to code similar to: “if (PORT2 == 0x50)” rather than code like “if (PORT2 == ports[i])”. In this way we reduce memory accesses which are expensive and may significantly degrade performance. This optimization was discussed in [3] and was found to also work well for our particular design.

We should note that although this paper focuses on the IXP1200, the micro-C code produced by S2I can be slightly modified so as to be compiled on a general purpose processor. Moreover, it can be easily adapted for other embedded or network processors. (An i386-based implementation of a lightweight **snort**-like system is briefly analyzed in Section 4.)

For the IXP1200, the S2I compiler will also insert context swap directives in certain points of the code. Context swaps are needed to voluntarily let the current thread swap out of execution so that other threads on the same uEngine will have a chance to execute. This is done to avoid monopolizing a uEngine for

too long. If all uEngines are claimed by running threads, then the buffer of the monitored port is likely to overflow causing packet loss.

4 Evaluation

In this section we perform the evaluation of the proposed software architecture. We evaluate separately the static section and the generated dynamic code.

4.1 Evaluation of the Static Section

The evaluation of the static section was done by measuring the *headroom* [7,2] of the system: the number of cycles that can be consumed for processing each packet (plain signature checking) without causing packet loss, using minimum-sized packets.

We produced minimum sized packets arriving from one port at 100 Mbit/s. Using the uEngine-based infrastructure, each packet was received and brought locally to a uEngine. Processing on the packet was simulated by performing some initial field extractions and then running a loop checking some values against some fields. Each loop was guaranteed to perform a fixed number of calculations and to take a constant number of cycles. We measured the number of loops that can be supported without dropping packets. By varying the number of available uEngines we measured how the headroom scales. Moreover we multiplied the number of loops that can be supported by the cycles that takes each loop. The corresponding number of cycles is the available headroom. The results are shown in Figure 7 and indicate how many cycles are available in each uEngine to perform the actual signature checks. We can see that using all the uEngines of the IXP1200, we have approximately 4920 cycles available for processing of each 64 byte packet.

The results show that the headroom offered by the static section is comparable to previous estimations [7] in which the authors used 100 Mbit/s links as well.

4.2 Evaluation of the Dynamic Section

The dynamically generated code is heavily influenced by the tree structure of the field checking. In this section we evaluate the effects of using this tree structure both in space and in performance.

4.3 Evaluation of Space Requirements

Space requirements are crucial since the entire set of signatures must be loaded in the instruction memory of the uEngines in order to perform intrusion detection. Given that some space is already dedicated for the static section (approximately 476 words for the uEngine-based static scheme), the rest (1572 words) will have

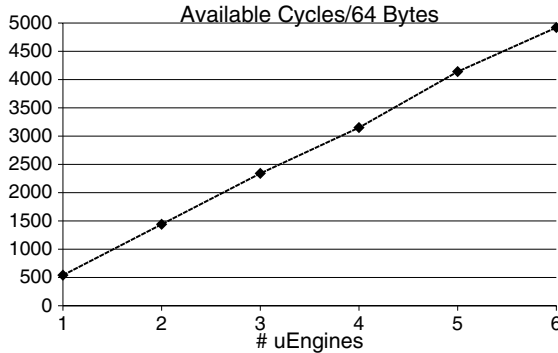


Fig. 7. Headroom in uEngine cycles (232 MHz). Simulation assumed one link at 100 Mbit/s and minimum sized packets

to be handled very efficiently. In this section we perform some simple experiments in order to measure how much space we gain by using a tree structure.

We used a modified version of the S2I compiler configured so as to produce code without using a tree structure. That is, each signature will result in one separate code-block which includes all its checks (e.g. as illustrated in Figure 8). The tree-like example of the code produced for this example was presented earlier in Figure 6.

```
(.....)
if (ETHPROTOCOL==0x0800 && PROTOCOL==0x6) {
    if (PORT2==0x50) {
        if (IP2==0xa000001) {
            if (IP1==0xa000000) {
                if (DSIZE>0x200) {
                    /* Action for "tcp 10.0.0.0 any -> 10.0...."*/
                }
            }
        }
    }
}
//alert tcp [10.0.0.1 10.0.0.2] any -> 10.0.0.2 80 (ack: >512;)
if (ETHPROTOCOL==0x0800 && PROTOCOL==0x6) {
    if (PORT2==0x50) {
        if (IP2==0xa000002) {
            if (IP1==0xa000001 || IP1==0xa000002) {
                if (ACK>0x200) {
                    /* Action for "tcp [10.0.0.1 10.0.0.2] ..."*/
                }
            }
        }
    }
}
(.....)
```

Fig. 8. Linear code produced by the S2I compiler by disabling the tree structure optimizations. Each signature is implemented in an independent code block

Using several signature input files from the **snort** distribution site, we measured the total number of instruction words that the signature checking consists of.⁴ Table 1 summarizes our findings.

Table 1. Space Savings using Tree structure

Signature File	Signatures	Plain Code inst/ions	Tree Code inst/ions	Reduction
icmp-info	79	did not fit	479	>69.00%
backdoor	44	1531	886	42.13%
web-misc	18	401	277	30.92%
virus	6	173	149	13.87%
web-cgi	4	145	120	17.24%

S2I offers size reduction (compression) for all files, with magnitude varying from 17.24% to 69%. The S2I space benefits increase as the size of the input file increases, indicating its success to combine multiple signatures in a shallow tree. At the extreme case of icmp-info signatures, S2I manages to fit all the required code in instruction memory, while with the simple approach the signatures do not fit in the uEngine memory. These results are very encouraging, since in our tests, S2I is able to perform when needed most, i.e. for large input files.

4.4 Evaluation of Execution Time

In addition to space, S2I promises also gains in performance, since traversing the tree is a very efficient way of evaluating the signatures. In order to gain intuition on the speed improvements, we contacted the following experiments in the IXP1200 Simulator.

Artificial Signatures and Artificial Traffic. We used five different signatures compiled using both the tree and without the tree. Then, we produced traffic with interleaved packets so as the signatures are matched sequentially: first packets matches first signature, second matches the second signature, etc. The fifth signature was a wild-card and therefore all packets matched it.

For this setting we measured the number of cycles spent on checking fields for the two compiled sources. In Table 2 we provide details of our findings. For each scenario, (packet matches signature 1, packet matches signature 2,...) we present the total number of cycles that were spent on performing checks. This time includes the time needed to perform an action when a match is found. (An action was simply to increment the value of an address in scratchpad).

⁴ We subtracted from the total number of instructions the size of the static section (which was 476 instructions).

Table 2. Cycles(232 MHz) spent on field checking

Scenario	Plain	Code Tree	Code Reduction
signature0+signature4	75	60	20.00%
signature1+signature4	74	62	16.22%
signature2+signature4	74	59	20.27%
signature3+signature4	74	61	17.57%
signature4 only	47	29	38.30%
Average	68.8	54.2	21.22%

As it can be seen the number of field checks is decreased by 21.2% on average. More interesting however, is that the performance gains are larger where there are fewer matches in the input (as in the "signature 4 only" case). The reason for this behavior is that if there is a match, the linear search of the simple implementation will stop quickly (for the few signatures we evaluated here). However, if the signatures do not match, the search will continue for longer. A tree structure allows the search to stop even in intermediate branches of the tree if the prefix does not match.

Using Artificial Signatures and Real Traffic. This experiment is conducted to increase our confidence in the previous evaluation and to indicate that the inputs we used are not skewed in favor of S2I. In this scenario, we conducted experiments using a small set of artificial signatures similar to the above. These signatures count packets based on protocol, source host, target host and payload size. However, unlike the previous case, we used real network traffic trace. This trace primarily consists of web traffic which was taken at ics.forth.gr, during a work day. Again we measure 20% on average reduction in the time spent on checking fields.

Using Real Signatures and Real Traffic. Finally, to get a feeling of the actual impact on real applications with real traces, we used the same trace, and the **snort** "backdoor" set of signatures. We ran this trace with the simple and the S2I tree structure, and measured total cycles spent on one packet. The results show that using the simple, sequential code, the field checking of the 44 signatures takes about 280 cycles. When compressing the field checks using the tree, the number drops to about 180 cycles, corresponding to a reduction of 35%.

Summarizing, the use of the tree is beneficial both for space and time reasons. Regarding space, we observe a minimum compression of 17.3% in instruction memory. Regarding time, we observe a significant reduction of around 20% in the time spent to apply the signatures, using some simple scenarios.

4.5 Lightweight snort for i386 Systems

The output of the dynamic section of the S2I compiler can be used as a base to program any kind of processor. In this section we present experiments with the

S2I output C code on an Intel Pentium processor. We compare the user time of executing the original **snort** and the lightweight version produced using the S2I tool.

We extracted from the default **snort** signature set all the signatures that do not require payload search. Then we used the S2I tool to produce a lightweight **snort** based on the remaining signatures. We run **snort** and lightweight **snort** over a trace taken from the NLANR archive [4]. While the user time of the original **snort** is about 12 seconds, our lightweight **snort** takes less than 5 seconds – an improvement of more than 50%.

5 Related Work

Research in tools and methodologies for network processors have focused mainly on routing-like applications and on modularity, re-usability and ease of programming.

In [8], Spalink et al. use the IXP1200 to build a software-based router. They propose a two-part architecture, which consists of a fixed infrastructure and a dynamically re-programmable part. The use of a network processor in software routers is also discussed in [2]. The authors present a tool supporting the dynamic binding of different components to form a fully-fledged router. The tool provides a basic infrastructure for controlling program flow and the data from one component to another, and a way for binding the components before uploading the code on the uEngines.

Dynamic code generation for packet filtering has been studied by Engler et al. in [3], with focus on efficient message demultiplexing in a general purpose OS. They present a tool that generates code based on a filter description language. Each filter is embodied at runtime in a filter-trie in a way that takes advantage of the known values the filter checks for.

6 Summary and Future Work

Hand coding hundreds of signatures in micro-C or assembly is a painful and error-prone task. In this paper we have proposed a software architecture and a tool for generating IXP1200 code from NIDS signatures. Using the S2I compiler, this task is being highly automated, translating a high-level signature specification into high-performance code. Therefore implementing intrusion analysis on the IXP1200 becomes a process that does not require knowledge of architecture internals and the micro-C programming language. Overall, the S2I compiler is able to produce fast and efficient code. while offering development speed and versatility.

There are several directions for future work that we are pursuing. First, we are working on tuning the S2I infrastructure. For instance, we consider improving the tree structure by adapting the field order for each sub-tree in order to minimize space, and execution profiles to reorder fields for minimizing processing time. Second, we are investigating the applicability of our design to higher-speed

ports (e.g. 1 Gbit/s on the IXP1200). Finally, we are interested in applying the same general design principles of application-specific code generation to content matching, which is of great practical interest in intrusion detection.

Acknowledgments

This work is funded by the IST project SCAMPI (IST-2001-32404) of the European Union. It is also supported by Intel through equipment donation.

References

1. Intel IXA SDK ACE programming framework developer's guide, June 2001.
<http://www.intel.com/design/network/products/npfamily/ixp1200.htm>.
2. A. Campbell, S. Chou, M. Kounavis, V. Stachtos, and J. Vicente. Netbind: A binding tool for constructing data paths in network processor-based routers. In *Proceedings of the 5th International Conference on Open Architectures and Network Programming (OPENARCH 2002)*, June 2002.
3. D. Engler and M. Kaashoek. DPF: Fast, flexible message demultiplexing using dynamic code generation. In *In Proceedings of ACM SIGCOMM'96*, pages 53–59, August 1996.
4. MRA traffic archive, September 2002.
<http://pma.nlanr.net/PMA/Sites/MRA.html>.
5. M. Roesch. Snort: Lightweight intrusion detection for networks. In *Proc. of the 1997 USENIX Systems Administration Conference (LISA)*, November 1999. (software available from <http://www.snort.org/>).
6. M. Sobirey. Intrusion detection systems.
<http://www-rnks.informatik.tu-cottbus.de/~sobirey/ids.html>.
7. T. Spalink, S. Karlin, and L. Peterson. Evaluating Network Processors in IP Forwarding. Technical report, Computer Science dep, Princeton University, Nov 15 2000.
8. T. Spalink, S. Karlin, L. Peterson, and Y. Gottlieb. Building a robust software-based router using network processors. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles*, pages 216–229, October 2001.

Retargetable Graph-Coloring Register Allocation for Irregular Architectures

Johan Runeson and Sven-Olof Nyström

Department of Information Technology
Uppsala University
{jruneson,svenolof}@csd.uu.se

Abstract. Global register allocation is one of the most important optimizations in a compiler. Since the early 80's, register allocation by graph coloring has been the dominant approach. The traditional formulation of graph-coloring register allocation implicitly assumes a single bank of non-overlapping general-purpose registers and does not handle irregular architectural features like overlapping register pairs, special purpose registers, and multiple register banks. We present a generalization of graph-coloring register allocation that can handle all such irregularities. The algorithm is parameterized on a formal target description, allowing fully automatic retargeting. We report on experiments conducted with a prototype implementation in a framework based on a commercial compiler.

1 Introduction

Embedded applications are growing larger and more complex, often reaching more than 100.000 lines of C code. To develop and maintain such an application requires a fast compiler. However, due to constraints on memory space, power consumption and other system resources, the compiler must also produce high-quality code. State-of-the-art optimization techniques from high-end RISC compilers are not always applicable, because embedded processor architectures are often irregular. Furthermore, the large number of different architectures means the compiler techniques must also be retargetable.

In this paper we focus on global register allocation, one of the most important transformations in a modern optimizing compiler [1] (page 92). For RISC-machines, Chaitin-style graph-coloring [2] is the dominant approach, as witnessed by its prominence in modern compiler construction textbooks [3,4,5]. It gives high-quality allocations, runs fast in practice, and is supported by a large body of research work (e.g. [6,7]). Unfortunately, the algorithm assumes a *regular* register architecture consisting of a single, homogenous set of general-purpose registers.

We propose a generalization of Chaitin's algorithm which allows it to be used with a wide range of *irregular* architectures, featuring for example register pairs or other clusters, and non-orthogonal constraints on the operands of certain instructions. The generalized algorithm is parameterized by an expressive formal

description of the register architecture, allowing fully automatic retargeting. It has the same time complexity as the original algorithm and is provably correct for any applicable architecture. The changes compared to the original algorithm are modest, so most existing improvements and extensions can be incorporated with little or no work.

2 Background

We assume that the register allocator is presented with low-level intermediate code, where the instructions correspond to target assembly language instructions, but where *variables* (taken from an unlimited set of names) are used instead of registers.

The goal of register allocation is to determine where to store each variable — in a particular register or in memory — in the most cost-effective way, and to rewrite the program to reflect these decisions. *Local* register allocation works in the scope of a single basic block. *Global* register allocation considers a whole function at a time.

Register allocation for a regular architecture can be formulated as a graph-coloring problem. A variable is *live* if it holds a value which may be used later in the program. Two variables which are live simultaneously are said to *interfere*, since they can not use the same register resources. Using liveness analysis, an *interference graph* can be built, where each node represents a variable, and where there is an edge between two nodes if their variables interfere. A *k-coloring* of a graph is an assignment of one of at most k colors to each node, such that no two neighbors have the same color. For a regular architecture with k registers, a k -coloring of the interference graph represents a solution to the register allocation problem, where all nodes with the same color share the same register.

Graph coloring is known to be an NP-complete problem, so heuristic techniques are used to perform register allocation in practice. Chaitin et al. [2] presented the first heuristic global register allocation algorithm based on graph coloring. Although it has a worst-case time complexity of $O(n^2)$, experiments in [6] indicate that in practice it runs in less than $O(n \log n)$ time. Due to space limitations, we can not give the full algorithm here. For the interested reader, we refer to the description by Briggs [6], or the more elaborate presentation in our technical report [8].

3 Retargetability through Parameterization

In modern retargetable compilers, *target descriptions* are often used to parameterize code generation and optimization passes in order to achieve retargetability [9,10]. We use the same approach for our register allocator. For simplicity, our target descriptions deal only with architectural features that affect register allocation. They can easily be incorporated in or derived from more extensive target descriptions.

In Chaitin's algorithm, the target is characterized only by the number of registers, k . It is assumed that the architecture is regular, i.e. that all registers are interchangeable in every situation. This assumption does not hold for irregular architectures. In our generalized algorithm, the target is characterized by an expressive *target model*, defined below, which allows features like overlapping register pairs, special purpose registers, and multiple register banks to be described. No further assumptions are made, so any architecture which can be described by a target model is applicable.

3.1 Target Models

We define a *target model* to be a tuple $\langle Regs, Conflict, Classes \rangle$, where

1. *Regs* is a set of register names,
2. *Conflict* is a symmetric and reflexive relation over the registers, and
3. *Classes* is a set of register classes, where each register class is a non-empty subset of *Regs*.

A register in *Regs* represents a fixed set of storage bits which can be accessed as a unit in some operation in the target architecture. Examples include physical registers, pairs and clusters of physical registers, and in some cases fixed memory locations which are used as registers. Note that registers may *overlap*, i.e. share bits.

Two registers (r, r') are in *Conflict* if they can not be allocated simultaneously, typically because they overlap. For example, a register pair conflicts with its component registers. The set *Regs* and the relation *Conflict* form a *conflict graph*, which describes how the register resources in the processor interact.

A register class C is included in *Classes* if there are operations which restrict a variable to be from the set C only. These restrictions are mostly imposed by the instruction set architecture, which may require, for example, that a particular operand for a particular instruction is an aligned register pair, or that the result of a particular instruction be placed in a particular register or set of registers. The run-time system may also affect the choice of register classes, by reserving certain registers for system use, or specifying that the arguments to a function are passed in particular registers.

We use register classes to enforce constraints on the operands to certain instructions. A variable which takes part in a number of operations must satisfy all the corresponding constraints, and is consequently given a class which is included in the intersection of the classes required by those operations. (Ideally, the class of the variable will equal the intersection, but this is not always possible in practice.)

As an example, consider a simple architecture with four basic registers R0–R3, which some instructions use as pairs W0 = R0:R1 and W1 = R2:R3. In the target model for this architecture, *Regs* is the set {R0, R1, R2, R3, W0, W1}. The *Conflict* relation is defined so that each register in *Regs* conflicts with itself, and the pairs conflict with their components: W0 with R0 and R1, and W1 with R2 and R3,

respectively. We define two register classes A and B , where A is $\{R0, R1, R2, R3\}$ and B is $\{W0, W1\}$. These two classes make up the set $Classes$.

The diagram in Fig. 1(a) illustrates this target model. Each box is a register, and each row gives the name and members of one register class. Furthermore, the boxes are arranged so that two registers conflict if they appear in the same column. More examples of target models can be found in Sect. 6, and in [8].

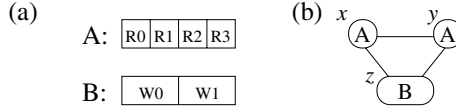


Fig. 1. A simple example: (a) target model diagram, (b) generalized interference graph

3.2 Generalized Interference Graphs

For a given target model we define a *generalized interference graph* to be a tuple $\langle N, E, class \rangle$ where N and E form an interference graph $\langle N, E \rangle$, and the function $class : N \rightarrow Classes$ maps each node to a register class. The nodes in N correspond to variables, and there is an edge in E between two nodes if their variables are simultaneously live at some point in the program.

The register class for a node constrains what registers may be assigned to that node by the allocator: We define an *assignment* for $M \subseteq N$ to be a mapping A from M to $Regs$ such that $A(n)$ is in $class(n)$ for all $n \in M$. Furthermore, we say that an assignment A for M is a *coloring* iff there are no neighboring pairs of nodes m and n in M such that $A(m)$ conflicts with $A(n)$.

Given a target model and a generalized interference graph, the register allocation problem reduces to the problem of finding a coloring for the graph.

Register allocation for regular architectures is a special case. The target model consists of a single class of k registers and an identity conflict relation. It follows that the problem of finding a coloring for a generalized interference graph is NP-hard.

Figure 1(b) shows a generalized interference graph under the target model in (a). The nodes x , y and z are annotated with register classes (A , A , and B , respectively), and from the interference edges we can see that the variables corresponding to the nodes are all live simultaneously.

4 Local Colorability

Chaitin’s graph-coloring algorithm is based on a concept which we call *local colorability*¹. In a generalized interference graph $\langle N, E, class \rangle$, a node $n \in N$ is

¹ Briggs uses the term “trivial colorability”. For an irregular architecture, determining local colorability is not always trivial.

locally colorable iff, for any assignment of registers to the neighbors of n , there exists a register r in $class(n)$ which does not conflict with any register assigned to a neighbor of n .

The coloring problem can be simplified by removing a node n which is locally colorable: given a coloring for the rest of the graph, the local colorability property guarantees that we can always find a free register to assign to n . If we can recursively simplify the graph until it is empty, then by induction it is possible to construct a coloring by assigning colors to the nodes in the reverse order from which they were removed.

4.1 Approximating Colorability

In a regular architecture with k registers, a node is locally colorable iff it has less than k neighbors in the interference graph. Chaitin's algorithm therefore removes nodes with $degree < k$.

For irregular architectures, the $degree < k$ test is not always a good indicator of local colorability. Consider the example in Fig. 1. It is easy to see that regardless of how we assign registers to y and z , there is always a free register for x . In other words, x is locally colorable, and by symmetry, the same goes for y . Now consider z . If we assign R0 to x , and R2 to y , then there is no free register for z , which is therefore not locally colorable.

All three nodes in the example have $degree = 2$, but only two of them are locally colorable. Consequently, the $degree < k$ test is not an accurate indication of local colorability in this case.

If we can not use the $degree < k$ test, what can we use instead? The definition of local colorability suggests a test based on generating and checking all possible assignments of registers to the neighbors of a node. Since there is an exponential number of possible assignments, we expect that such a test would be too expensive to use in practice.

Fortunately, the coloring algorithm does not require a precise test for local colorability. In order to guarantee that it is possible to color the nodes in the reverse order from which they were removed from the graph, it is enough if the test implies local colorability. What we need is therefore an inexpensive test which safely approximates local colorability with minimal inaccuracy.

4.2 The $\langle p, q \rangle$ Test

We propose the following approximation of the local colorability test. Given a target model as defined in Sect. 3.1, let p_B and $q_{B,C}$ be defined for all classes B and C by

$$p_B = |B|$$

$$q_{B,C} = \max_{r_C \in C} |\{r_B \in B \mid (r_B, r_C) \in \text{Conflict}\}|$$

In other words, p_B is the number of registers in the class B , and $q_{B,C}$ is the largest number of registers in B that a single register from C can conflict with.

A node n of class B in $\langle N, E, class \rangle$ is locally colorable if

$$\sum_{\substack{(n,j) \in E \\ C = class(j)}} q_{B,C} < p_B.$$

We will call this the $\langle p, q \rangle$ test.

The intuition behind the $\langle p, q \rangle$ test is as follows. To begin with there are p_B registers available for assigning to n . Each neighbor may block some of these registers. In the worst case, a neighbor from class C can block $q_{B,C}$ registers in B . If the sum of the maximum number of registers each neighbor can block is less than the number of available registers, then it is safe to say that we will be able to find a free register for n . In Sect. 4.3 we prove formally that the $\langle p, q \rangle$ test is a safe approximation of local colorability in any generalized interference graph, for any given target model.

The $\langle p, q \rangle$ test is efficient: Since p and q are fixed for a given target model, they can be pre-computed and stored in static lookup tables. This makes it possible to evaluate the $\langle p, q \rangle$ test with the same time complexity as the $degree < k$ test.

For a regular architecture with k registers, we get $p = k$ and $q = 1$, which means that the $\langle p, q \rangle$ test degenerates to the precise $degree < k$ test. Any imprecision in the $\langle p, q \rangle$ test is thus induced only by certain irregular features of the architecture.

Note that for two disjoint register classes B and C , we get $q_{B,C} = 0$. Interference edges between nodes from disjoint classes therefore do not contribute to the sum in the $\langle p, q \rangle$ test. Also, for a self-overlapping class B (e.g. a class of unaligned pairs), $q_{B,B} > 1$, since a single register from B can conflict with both itself and one or more other registers in B .

4.3 Proof of Safety

We will show for a given target model $\langle Regs, Conflict, Classes \rangle$ that in any generalized interference graph $G = \langle N, E, class \rangle$, if a node is not locally colorable, then the $\langle p, q \rangle$ test for that node is false.

Let n be a node which is not locally colorable in G . Let B be the register class of n , and J the set of neighbors of n in G . Since n is not locally colorable, there must exist an assignment A of registers to the neighbors of n , such that for all registers r_B in B , r_B conflicts with $A(j)$ for some j in J .

This allows us to express B as follows.

$$B = \bigcup_{j \in J} \{r_B \in B \mid (r_B, A(j)) \in Conflict\}$$

By definition, $p_B = |B|$, so we have

$$p_B = |B| = \left| \bigcup_{j \in J} \{r_B \in B \mid (r_B, A(j)) \in Conflict\} \right|$$

Now, the size of a union of sets is less than or equal to the sum of the sizes of the individual sets, so we can limit the size of the big union as follows.

$$p_B \leq \sum_{j \in J} |\{r_B \in B \mid (r_B, A(j)) \in \text{Conflict}\}|$$

But, for any node j , the number of registers in B in conflict with $A(j)$ can not be more than the maximum number of registers from B in conflict with any register from $\text{class}(j)$, which is exactly the definition of $q_{B,C}$.

$$p_B \leq \sum_{\substack{j \in J \\ C = \text{class}(j)}} \max_{r_C \in C} |\{r_B \in B \mid (r_B, r_C) \in \text{Conflict}\}| = \sum_{\substack{j \in J \\ C = \text{class}(j)}} q_{B,C}$$

Thus, if n is not locally colorable in G , then the $\langle p, q \rangle$ test for n is false. Conversely, if the $\langle p, q \rangle$ test is true, then n is locally colorable. This proves that the $\langle p, q \rangle$ test is a safe approximation of local colorability, for any graph in any target model.

5 The Complete Algorithm

For simplicity, we present the algorithm without coalescing and optimistic coloring. These extensions are discussed separately below.

Given a target model as in Sect. 3.1, we use the formulae in Sect. 4.2 to pre-compute p_B and $q_{B,C}$ for all classes B and C .

The algorithm is divided into four phases (Fig. 2).

1. *Build* constructs the generalized interference graph.
2. *Simplify* initializes an empty stack, and then repeatedly removes nodes from the graph which satisfy the $\langle p, q \rangle$ test. Each node which is removed is pushed on the stack.

This continues until either the graph is empty, in which case the algorithm proceeds to *Select*, or there are no more nodes in the graph which satisfy the test. In that case, *Simplify* has failed, and we go to the *Spill* phase.

3. *Select* rebuilds the graph by re-inserting the nodes in the opposite order to which *Simplify* removed them. Each time a node n is popped from the stack, it is assigned a register r from $\text{class}(n)$ such that r does not conflict with the registers assigned to any of the neighbors of n .

When *Select* finishes, it has produced a complete register allocation for the input program, and the algorithm terminates.

4. *Spill* is invoked if *Simplify* fails to remove all nodes in the graph. It picks one of the remaining nodes to spill, and inserts a load before each use of the variable, and a store after each definition. After the program is rewritten, the algorithm is restarted from the *Build* phase.

Select always finds a free register for each node, because the $\langle p, q \rangle$ test in *Simplify* guarantees that the node was locally colorable in the graph which it was removed from, and the use of a stack guarantees that it is reinserted into the same graph.

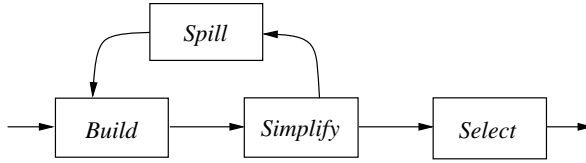


Fig. 2. Phases of the basic register allocation algorithm

In Chaitin’s original algorithm, there are no register classes. Nodes are removed in Simplify when their *degree* $< k$, and in Select registers conflict only with themselves. Other than that, the algorithms are identical.

5.1 A Simple Example

As a simple example, we run the generalized algorithm on the problem in Fig. 1. Based on the target model illustrated in (a), we compute the following parameters: $p_A = 4$, $p_B = 2$, $q_{A,A} = 1$, $q_{A,B} = 2$, $q_{B,A} = 1$, $q_{B,B} = 1$. Computing the $\langle p, q \rangle$ test for all the nodes of the graph in (b), we see that it is true for x and y , but not for z .

The fact that z is not locally colorable does not mean that it can not be colored – it just means that we should color it before some of its neighbors in order to guarantee that it will be colored. This is fine with the other two nodes: since they are locally colorable we know that we can always color them regardless of how we color z .

We pick one of the colorable nodes, x , remove it from the graph, and push it on the stack. In the resulting simplified graph, the $\langle p, q \rangle$ test is true not just for y , but for z as well. We therefore remove y and z , and proceed to the Select phase.

The first node to be popped is z . None of z ’s neighbors have been inserted in the graph yet, so we only have to worry about picking a node from the correct register class. Out of the class B , we select register **W0** for z . The next node to be popped is y . Since y interferes with z , we can not assign registers **R0** or **R1** to it, because these registers conflict with **W0**. Therefore, we select **R2** for y . Finally, we reinsert x into the graph. The only register available for x is **R3**.

5.2 Extensions

Optimistic coloring [6] is an important extension to Chaitin’s algorithm, where spilling decisions are postponed from the Simplify to the Select phase: If Simplify can find no more locally colorable nodes, one node is picked to be removed anyway and pushed on the stack optimistically. When it is popped in Select, it may be possible to color it, for example if two neighbors have been assigned the same color. If so, there is no need to spill. Nodes which are popped later and which were locally colorable when pushed are still guaranteed to find a free color. Optimistic coloring often reduces the number of spills significantly, and

can hide much of the imprecision of an approximating local colorability test [6]. It is completely orthogonal to the modifications presented here, and can (and should) be implemented just like in a regular graph coloring register allocator.

Another standard extension is *coalescing* [2], where copy-related noninterfering nodes are merged before the Simplify phase. If nodes n and n' are merged into m , then m must obey the constraints imposed on both n and n' . Therefore, it is given a register class from the intersection of the classes for n and n' . (If the intersection is empty, coalescing is not possible.)

Aggressive coalescing may sometimes cause unnecessary spills, when a node which is simple to color is merged with a node which is hard to color [6]. Therefore, *conservative coalescing* only merges two nodes if it can be guaranteed that the merged node will be locally colorable. It is straightforward to replace the $\text{degree} < k$ test with the $\langle p, q \rangle$ test to take register classes into account when doing this.

The *spill metric*, used to determine which node to pick for spilling, also deserves mention. It, too, should take register classes into account. We achieve this by picking the node with the smallest ratio

$$\text{cost}(n) / \text{benefit}(n).$$

However, rather than using $\text{degree}(n)$ as a measure of the benefit of removing that node, we define

$$\text{benefit}(n) = \sum_{\substack{(n,j) \in E \\ C = \text{class}(j)}} (q_{C,B} / p_C).$$

Dividing $q_{C,B}$ by p_C allows us to compare the benefits for neighbors of different classes.

Figure 3 shows the phases of the register allocator when all the extensions described in this section are included. (The spill metric is used in the Simplify phase to determine which node to push optimistically on the stack.) Some further extensions are discussed in [8], including an alternative local colorability test which is slower, but has higher precision.

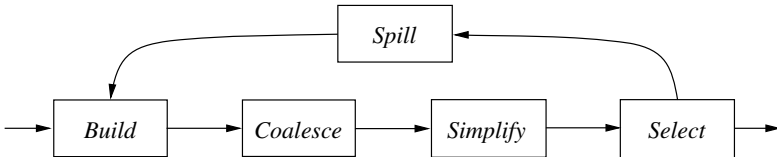


Fig. 3. Phases of the register allocation algorithm with extensions

6 Experiments

There are many factors besides register allocation which affect the quality of the code generated from a particular compiler. To make a fair comparison between different allocators, they must all be implemented in the same compiler. Often, though, there are strong dependencies between the allocator and the rest of the compiler, which could favour one allocator design unfairly over another. A good testbed for register allocation should strive to minimize such dependencies.

We have created a prototype framework for comparing different register allocators based on a commercial C/EC++ compiler from IAR Systems [11]. The framework short-circuits the existing allocator, which is closely tied to the code selection phase of the compiler. The allocator to be evaluated is inserted after the code selection phase, and presented with assembly code where the instruction operands contain virtual registers (or variables, in the terminology of this paper) annotated with register classes. The new allocator is responsible for rewriting the code with physical registers and inserting spill code, after which regular compilation resumes.

Although the compiler is retargetable², incorporation of the prototype framework requires substantial changes in the target-dependent parts of the backend. Therefore, it currently only generates code for a single target: the Thumb mode of the ARM/Thumb architecture [12]. In ARM mode, the ARM/Thumb is a RISC-like 32-bit processor with 16 registers. In Thumb mode, a compressed instruction encoding is used, with 16-bit instructions. Most instructions in Thumb mode are two-address, and can only access the first 8 registers.

6.1 Implementation

The algorithm from Sect. 5, including optimistic coloring, conservative coalescing and the spill metric from Sect. 5.2, has been implemented in the prototype framework described above. Fig. 4 illustrates the target model that we use, derived from the register classes that the framework generates for us. These classes reflect constraints imposed both by the instruction set and by the runtime system. There are classes for 32-bit and 64-bit data (in unaligned pairs), for individual 32-bit and 64-bit values (used in the calling convention), a larger class of 32-bit registers which can sometimes be used for spilling to registers, and some classes of 96 and 128-bit values used for passing structs into functions. Registers R13 and R15 are dedicated by the runtime system. Registers R8–R11 are too expensive to use profitably in Thumb mode. Table 1 shows the p and q values that we compute for the target model in Fig. 4. (The value of $q_{B,C}$ is located in the row for B and the column for C .)

We have implemented three different variants of the allocator.

1. *Full* is the full allocator described above, including the extensions from Sect. 5.2.

² Currently, IAR Systems supports over 30 different target architecture families with its suite of development tools.

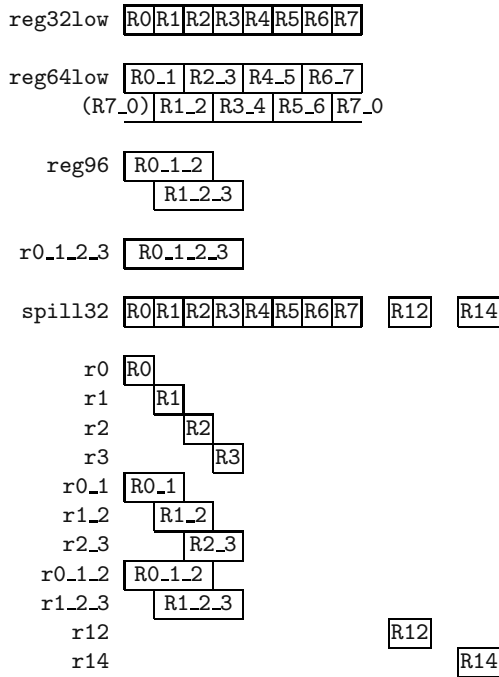


Fig. 4. Target model diagram for the Thumb architecture.

Table 1. Computed p and q values for Thumb

[illegible]

2. *Local* is the same allocator, but made to spill all variables that are live across basic block boundaries.
3. *Worst-Case* spills all variables.

The Local allocator is intended to mimic heuristic local register allocators such as used in e.g. Lcc [13]. The Worst-Case allocator represents the worst case, and gives a crude base line for comparisons.

Due to some simplifying design decisions, the prototype framework generates spill code which is less efficient than what would be acceptable in a production compiler. This exaggerates the negative effects of spilling somewhat, which should be taken into account when looking at the experimental results.

6.2 Results

Finding good benchmarks for embedded systems is hard, since typical embedded applications differ from common desktop applications in significant ways [14,15]. We have chosen to use the suites Automotive, Network and Telecomm from MiBench [14], a freely available³ collection of embedded benchmarks.

The benchmark suites were compiled with each variant of the allocator⁴. The first part of Table 2 gives the number of functions (*funcs*) in each suite, and the average number of variables per function (*vars*). The largest number of variables in any function is 1016. For each allocator, we then report the total size of the generated code (*size*), and for Full and Local the number of spilled variables (*spill*). The Full allocator is not optimized for speed, yet. Currently, the average time spent in the allocator is 1.67 seconds per function.

Table 2. Results compiling benchmark suites

<i>Suite</i>	<i>funcs</i>	<i>vars</i>	Full			Local			Worst-Case	
			<i>size</i>	<i>spill</i>	<i>cost</i>	<i>size</i>	<i>spill</i>	<i>cost</i>	<i>size</i>	<i>cost</i>
Automotive	29	113	8918	77	5232	12598	175	9452	59076	65722
Network	17	84	3260	8	690	6048	100	4501	17970	25961
Telecomm	130	118	35116	154	6020	70778	1021	51992	329102	322858
<i>Total</i>	176	114	47294	239	11942	89424	1296	65945	406148	414541

Many programs in MiBench rely on the presence of a file system for input and output. Since this was not available in our test environment we were only able to execute a few of the programs. In Table 3, we show the cycle counts ($kCycles \cdot 10^3$) from runs of three programs, one from each benchmark suite. The programs were executed in the simulator/debugger that comes with the compiler [11], using the “small” input sets. We compare the cycle counts with the accumulated spill costs for all spilled variables (*cost*). Since the spill costs are

³ See <http://www.eecs.umich.edu/mibench/>.

⁴ All files were compiled except `toast.c`, which failed because of a missing header file, and `susan.c`, which failed for unknown reasons.

weighted by loop nesting depth, spills in loops are more costly, and we expect to see some correlation with the actual run-times. We also show the accumulated spill costs for the complete benchmark suites in Table 2.

Table 3. Results running benchmark programs

<i>Program</i>	Full		Local		Worst-Case	
	<i>cost</i>	<i>kCycles</i>	<i>cost</i>	<i>kCycles</i>	<i>cost</i>	<i>kCycles</i>
Automotive/qsor	0	136729	280	142556	1990	152005
Network/dijkstra	20	154339	820	188772	7360	979790
Telecomm/CRC32	20	3416	750	12618	3210	30731

7 Related Work

Briggs’ [6] approach to handling multiple register classes (in part suggested already by [2]) is to add the physical registers to the interference graph, and make each node interfere with all registers it can not be allocated to. Edges between nodes from non-overlapping classes are removed. To handle register pairs, multiple edges are used between nodes where one is a pair. Thus, the interference graph is modified to represent both architectural and program-dependent constraints, leaving the graph-coloring algorithm unchanged.

Our approach is fundamentally different, in that we separate the constraints of the program from those of the architecture and run-time system into different structures. Instead of modifying the interference graph, we change the *interpretation* of the graph based on a separate data structure. We believe that our approach leads to a simpler and more intuitive algorithm, which avoids increasing the size of the interference graphs before simplification, and where expensive calculations relating to architectural constraints can be performed off-line.

For an architecture with aligned register pairs, the solution proposed by Briggs is equivalent to ours in terms of precision. However, Briggs gives only vague rules (“add enough edges”) for adapting the algorithm to other irregular architectures [6]. Our generalized algorithm, on the other hand, works for any architecture that can be described by a target model.

The scheme proposed by Smith and Holloway [16] is more similar to ours, in that it also leaves the interference graph (largely) unchanged. Their interpretation of the graph is based on assigning class-dependent weights to each node. Rules for assigning weights are given for a handful of common classes of irregular architectures. In contrast, our algorithm covers a much wider range of architectures without requiring classification, we give sufficient details to generate allocators automatically from target descriptions, and we prove that our local colorability test is safe for arbitrary target models.

Scholz and Eckstein [17] have recently described a new technique based on expressing global register allocation as a boolean quadratic problem, which is

solved heuristically. The range of architectures which can be handled by their technique is slightly larger than what can be represented by our target models. Practical experience with this new approach is limited, however, and it is not supported by the large body of research work that exists for Chaitin-style graph coloring.

There have been some attempts to use integer linear programming techniques to find optimal or near-optimal solutions to the global register allocation problem for irregular architectures [18,19]. These methods give allocations of very high quality, but, like other high-complexity techniques, they are much too slow to be useful for large applications.

Some people argue that longer compile times are justified for certain embedded systems with extremely high performance requirements [20]. This has prompted researchers to look into compiler techniques with worse time complexity than what is usually accepted for desk-top computing, often integrating register allocation with scheduling and/or code selection. For example, Bashford and Leupers [21] describe a backtracking algorithm with either $O(n^4)$ or exponential complexity, depending on strategy. Kessler and Bednarski [22] give an optimal algorithm for integrated code selection, register allocation and scheduling, based on dynamic programming. Still, with embedded applications reaching several 100.000 lines of C code, there is a need for fast techniques such as ours for compilers in the middle of the code-compile-test loop, or as a fall-back when more expensive techniques time out.

8 Conclusions

With our simple modifications, Chaitin-style graph-coloring register allocation can be used for irregular architectures. It is easy to incorporate well-known extensions into the generalized algorithm, allowing compiler writers to leverage the existing body of supporting research. The register allocator is parameterized on a formal target description, and we give sufficient details to allow automatic retargeting. Our plans for future work include comparisons with optimal allocations, incorporation of more extensions, and creating a free-standing implementation of the allocator to better demonstrate retargetability.

Acknowledgments

This work was conducted within the WPO project, a part of the ASTEC competence center. Johan Runeson is an industrial Ph.D. student at Uppsala University and IAR Systems. The register allocation framework used for the experiments in this paper was implemented by Daniel Widenfalk at IAR Systems. The register allocator itself was implemented by Axel Burström as a part of his Masters' thesis project. The authors wish to thank Carl von Platen for fruitful discussions and comments on drafts of this paper. We also thank the anonymous reviewers for valuable comments and suggestions for improvements.

References

1. Hennessy, J.L., Patterson, D.A.: *Computer Architecture: A Quantitative Approach*, Second Edition. Morgan Kaufmann Publishers (1996)
2. Chaitin, G.J., Auslander, M.A., Chandra, A.K., Cocke, J., Hopkins, M.E., Markstein, P.W.: Register allocation via coloring. *Computer Languages* **6** (1981) 47–57
3. Appel, A.W.: *Modern Compiler Implementation in ML*. Cambridge University Press (1998)
4. Morgan, R.: *Building an Optimizing Compiler*. Digital Press (1998)
5. Muchnick, S.S.: *Advanced Compiler Design and Implementation*. Morgan Kaufmann (1997)
6. Briggs, P.: Register allocation via graph coloring. PhD thesis, Rice University (1992)
7. George, L., Appel, A.W.: Iterated register coalescing. *TOPLAS* **18** (1996) 300–324
8. Runeson, J., Nyström, S.O.: Generalizing Chaitin’s algorithm: Graph-coloring register allocation for irregular architectures. Technical Report 021, Department of Information Technology, Uppsala University, Sweden (2002)
9. Ramsey, N., Davidson, J.W.: Machine descriptions to build tools for embedded systems. In: *LCTES*. Springer LNCS 1474 (1998) 176–188
10. Bradlee, D.G., Henry, R.R., Eggers, S.J.: The Marion system for retargetable instruction scheduling. In: *PLDI*. (1991)
11. IAR Systems: EWARM (2003) <http://www.iar.com/Products/?name=EWARM>.
12. Jagger, D., Seal, D.: *ARM Architecture Reference Manual* (2nd Edition). Addison-Wesley (2000)
13. Fraser, C.W., Hanson, D.R.: Simple register spilling in a retargetable compiler. *Software - Practice and Experience* **22** (1992) 85–99
14. Guthaus, M.R., Ringenberg, J.S., Ernst, D., Austin, T.M., Mudge, T., Brown, R.B.: MiBench: A free, commercially representative embedded benchmark suite. In: *IEEE 4th Annual Workshop on Workload Characterization*. (2001)
15. Engblom, J.: Why SpecInt95 should not be used to benchmark embedded systems tools. In: *LCTES*, ACM Press (1999)
16. Smith, M.D., Holloway, G.: Graph-coloring register allocation for architectures with irregular register resources. Unpublished manuscript, (2001)
<http://www.eecs.harvard.edu/machsuiif/publications/publications.html>.
17. Scholz, B., Eckstein, E.: Register allocation for irregular architectures. In: *LCTES-SCOPE5*, ACM Press (2002)
18. Kong, T., Wilken, K.D.: Precise register allocation for irregular register architectures. In: *Proc. Int’l Symp. on Microarchitecture*. (1998)
19. Appel, A.W., George, L.: Optimal spilling for CISC machines with few registers. In: *PLDI*. (2001)
20. Marwedel, P., Goosens, G.: *Code Generation for Embedded Processors*. Kluwer (1995)
21. Bashford, S., Leupers, R.: Phase-coupled mapping of data flow graphs to irregular data paths. In: *Design Automation for Embedded Systems*. Volume 4., Kluwer Academic Publishers (1999) 1–50
22. Kessler, C., Bednarski, A.: Optimal integrated code generation for clustered VLIW architectures. In: *LCTES*, ACM Press (2002) 102–111

Fine-Grain Register Allocation Based on a Global Spill Costs Analysis

Dae-Hwan Kim and Hyuk-Jae Lee

School of Electrical Engineering and Computer Science, P.O.Box #054
Seoul National University, San 56-1, Shilim-Dong, Kwanak-Gu, Seoul, Korea
dhtail@hanmail.net, hjlee@ee.snu.ac.kr

Abstract. A graph-coloring approach is widely used for register allocation, but its efficiency is limited because its formulation is too abstracted to use information about program context. This paper proposes a new register allocation technique that improves the efficiency by using information about the flow of variable references of a program. In the new approach, register allocation is performed at every reference of a variable in the order of the variable reference flow. For each reference, the costs of various possible register allocations are estimated by tracing a possible instruction sequence resulting from register allocations. A cost model is formulated to reduce the scope of the trace. Experimental results show that the proposed approach reduces spill code by an average of 34.3% and 17.8% in 8 benchmarks when compared to the Briggs' allocator and the interference region spilling allocator, respectively.

1 Introduction

Register allocation is an important compiler technique that determines whether a variable is to be stored in a register or in memory. The goal of register allocation is to store variables in registers as many as possible so that the number of load/store instructions can be minimized. Because the reduction of load/store instructions leads to the decrease of execution time, code size and power consumption, extensive research effort has been made to improve the efficiency of register allocation [3]-[15].

Register allocation based on graph-coloring has been the dominant approach since Chaitin first introduced the idea and Briggs improved it later [3–7, 13]. In this approach, the register allocation problem is modeled as the coloring problem of an interference graph of which each node represents a variable and an edge represents interference of variables. Any adjacent variables in the graph interfere with each other for register allocation so that they cannot share the same register. The main contribution of the graph-coloring approach is its simplicity by abstracting each variable as a single node of an interference graph. However, the simple abstraction results in the loss of information about program context and, as a result, degrades the efficiency of register allocation. This is because an edge in the interference graph only indicates that two variables interfere at some part of a program but does not specify where and how much they interfere. As a result, a register cannot be shared by two

variables throughout the program even though they interfere only at a small part of the program. To avoid the inefficiency of the graph-coloring approach, [13] proposed a fine-grain approach in which register allocation for a variable is decided not just once for an entire program, but multiple times at every reference of the variable. This approach improves the graph-coloring algorithm in the sense that it allows two variables to share the same register at some part of a program where they do not interfere although they interfere at the other part. However, it also has a drawback such that a single variable may be assigned to different registers for different references and, as a result, this register allocation often generates too many copy instructions.

In this paper, a new register allocation is proposed that combines the advantages of both the graph-coloring approach and the fine-grain approach while avoiding drawbacks of these approaches. The proposed approach attempts register allocation for every reference of a variable as the fine-grain approach. It also performs optimization to assign the same register to all references of a single variable whenever possible and desirable. With this optimization, the proposed approach can reduce the drawback of the fine-grain approach and reduce unnecessary copy instructions. To make this optimization possible, the proposed register allocation analyzes the flow of the references of each variable. Then, multiple references of a single variable are allocated not independently but in the same order as the reference flow that is likely to be the execution order of the references in a program. The allocator knows which register is assigned previously and can use the same register as previously assigned. When no register is available, the allocator preempts a register from previously assigned variable if the preemption reduces the execution cost of a program. To select the register with maximum cost reduction, the preemption cost and benefit are analyzed for all possible registers. The cost estimation often requires large computation with exponential complexity. Thus, a mathematical model for the simple estimation of an approximated cost is derived and a heuristic with a reasonable amount of computation is developed based on the model.

The rest of this paper is organized as follows. Section 2 explains the basic idea of the proposed register allocation. Section 3 presents the mathematical cost model of register spill and preemption. Section 4 discusses scratch register allocation. Section 5 analyzes the complexity of the proposed register allocation and provides experimental results. Conclusions are discussed in Section 6.

2 The Proposed Register Allocation

2.1 Motivational Example

Consider the program shown in Fig. 1 (a). Register allocation based on graph-coloring constructs the interference graph as shown in Fig. 1 (b) which shows that variables ‘a’, ‘b’, and ‘c’ interfere with each other while ‘d’ does not have interference with other variables. Assume that only two registers are available, then one variable among ‘a’,

'b', and 'c' cannot have a register. Variables 'a', 'b', and 'c' are referenced five times, four times, and three times, respectively. Thus, variable 'c' is spilled because it has the minimal spill cost (i.e., has the least number of references). As a result, three memory accesses for variable 'c' are necessary. Considering the reference order of these variables, the graph-coloring approach is not an efficient solution because variable 'c' is consecutively referenced from the fourth to the sixth statements. Thus, it is more efficient to allocate register to variable 'c' while spilling 'a' before the first access of 'c' and reloading it after the last access of 'c'. In this case, only two memory accesses are necessary which is the better result than the graph-coloring approach.

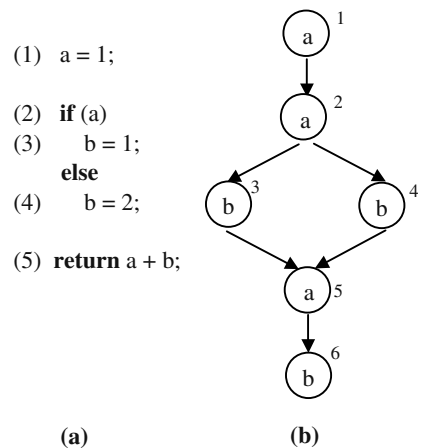
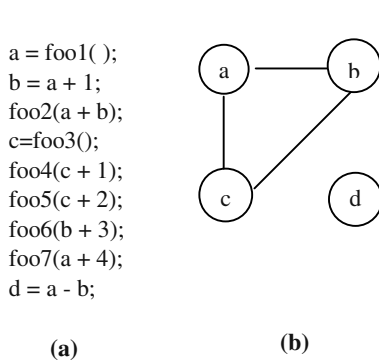


Fig. 1. Register allocation based on graph-coloring (a) example program (b) interference graph

Fig. 2. Variable reference flow graph (varef-graph) (a) example program (b) varef-graph graph

2.2 Variable Reference Flow Graph (varef-graph)

For a given program, the proposed approach constructs a varef-graph (variable reference flow graph) that is a partial order of variable references in the program. Each node of this graph represents a variable reference and an edge represents a control flow of the program, i.e., the execution order of the variable references of the program. Note that the execution is only partially-ordered because the complete control flow cannot be decided at compile-time. Fig. 2 shows an example program with the corresponding varef-graph. For illustration, the number of each statement is given in the leftmost column in the program. Each node represents a reference of a variable whose name is given inside the circle. The number to the upper right of the circle is the node number. Note that this number is different from the statement number because one statement can have multiple variable references and consequently have multiple nodes in the varef-graph. In Fig. 2 (b), the reference of variable 'a' at statement (1) is represented by node '1'. The program has two additional references of

variable ‘a’ that are represented by nodes ‘2’ and ‘5’, respectively. Variable ‘b’ is referenced three times at (3), (4), and (5) and the corresponding nodes are ‘3’, ‘4’, and ‘6’, respectively. Note that statement (5) has references of two variables ‘a’ and ‘b’ which are represented by nodes ‘5’ and ‘6’ in the graph, respectively.

An edge represents a partial execution order of the program. Statement (1) is supposed to be executed first, and the corresponding node ‘1’ is the root node. Statement (2) is supposed to be executed next, and the corresponding node ‘2’ is the successor of node ‘1’. Statements (3) and (4) are executed next to the statement (2), and therefore the corresponding nodes ‘3’ and ‘4’ are successors of node 2. Statements (3) and (4) must be executed exclusively, and therefore, there is no edge between nodes ‘3’ and ‘4’. Statements (5) and (6) are executed next in sequence, as shown in the figure.

With the order given by the varef-graph, register allocation is performed at every reference of a variable. If the register previously assigned to the variable is available, it is selected. Otherwise, any available register is selected. If no register is available, the register allocator attempts to preempt a register from another variable. Depending on which register to be preempted, the benefit of register assignment can be different (see more details on the estimation of the benefit in Section 3). Thus, the register allocator estimates the benefit and loss of preemption for all registers and selects the register with maximum benefit. If all registers have larger loss than benefit, no register is selected, and consequently, no register is assigned to the variable. The register allocation continues until all nodes in the varef-graph are visited. The visit order is a modified breadth-first order that is the same as the breadth-first order with the modification that guarantees a successor node to be always visited later than its predecessor. For those nodes that are not assigned to a register, the second stage of register allocation, called scratch register allocation, is performed. The algorithm of the second stage is the same as the first stage except a slight modification in the estimation of spill cost (see section 4 for more details).

3 Analysis of Register Allocation Benefit

The proposed register allocator visits each node of a varef-graph and decides whether to allocate a register or not. When no register is free for allocation, the allocator needs to estimate the benefit of register allocation for each register, and select the register with maximum benefit. The success of the proposed register allocation heavily depends on the precise analysis of the benefit. However, the analysis requires computation with exponential complexity. Thus, an approximated benefit is derived in the proposed register allocation with reasonable complexity. This section presents the mathematical foundation for the derivation of the benefit.

3.1 Benefit of Register Allocation

Consider register allocation for the varef-graph shown in Fig. 3 (a). Suppose that the register allocator visits node '3' while nodes '1' and '2' already receive registers 'r1' and 'r2', respectively. Assume that no registers are available for node '3' so that the register allocator decides to spill node '3'. Note that this decision may affect the register allocation for node '4' and leads to the spill of node '4'. This is because it is more beneficial to spill both nodes '3' and '4' than to spill only node '3'. Even though both nodes '3' and '4' receive a register, the allocator preempts only one register and therefore does not increase the number of load/store instructions compared to the case when only node '4' receives a register. Thus, if node '3' is spilled, node '4' is most likely to be spilled, too. Thus, the decision for node '3' is, in fact, the decision for node '4' as well.

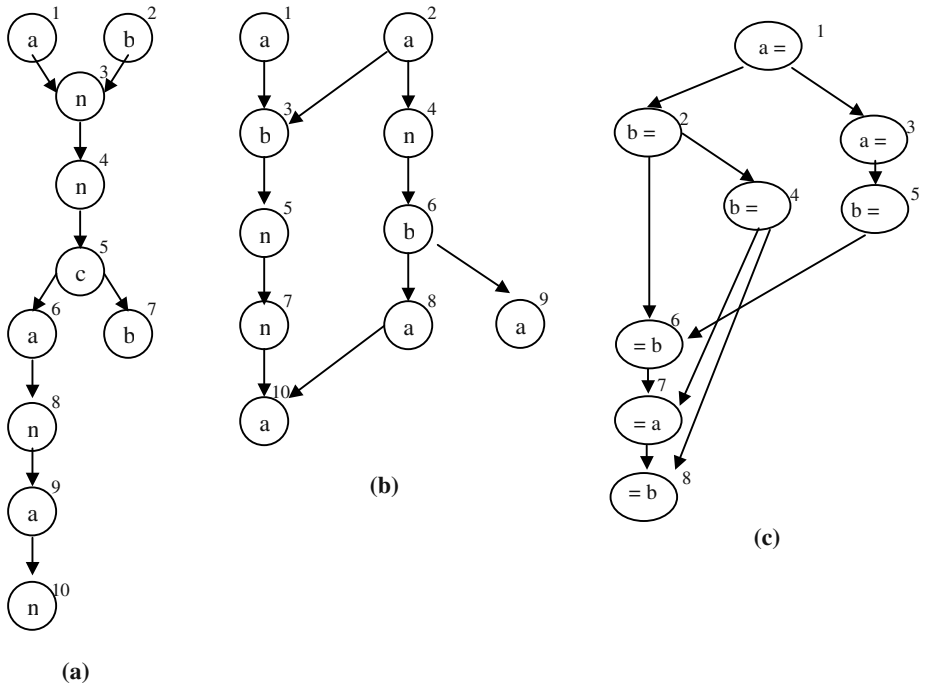


Fig. 3. Example varef-graphs

The previous example shows the register allocation for one node affects the register allocation for another node. The effect can be represented in terms of probability, $\text{ProbSpill}_{n\text{-spill}}(m)$ that denotes the probability of node 'm' to be spilled when node 'n' is decided to be spilled. Let $\text{PenaltySpill}(n)$ denote the total number of load/store instructions that are required if node 'n' is spilled. Then, $\text{PenaltySpill}(n)$ is expressed in terms of the spill probability as follows:

$$\text{PenaltySpill}(n) = \sum_m \text{ProbSpill}_{n\text{-spill}(m)} \text{cost}(m) . \quad (1)$$

where $\text{cost}(m)$ denotes the number of load/store instructions required to execute node 'm' when it is spilled.

Let $\text{PenaltyPreempt}(n,r)$ denote the number of load/store instructions when node 'n' preempts register 'r'. Let $\text{ProbSpill}_{n\text{-preempt-r}}(m)$ denote the probability of a node 'm' to be spilled when node 'n' preempts register 'r'. Then, the preemption penalty can be expressed in terms of $\text{ProbSpill}_{n\text{-preempt-r}}(m)$ as follows:

$$\text{PenaltyPreempt}(n,r) = \sum_m \text{ProbSpill}_{n\text{-preempt-r}}(m) \text{cost}(m) . \quad (2)$$

Let $\text{BenefitRegAlloc}(n,r)$ denote the benefit of the allocation of register 'r' to node 'n'. This benefit is the amount of the spill penalty subtracted by the preemption penalty:

$$\text{BenefitRegAlloc}(n,r) = \text{PenaltySpill}(n) - \text{PenaltyPreempt}(n,r) . \quad (3)$$

For efficient register allocation, the register allocator chooses the register 'r' with positive maximum $\text{BenefitRegAlloc}(n,r)$ among all available registers. If $\text{BenefitRegAlloc}(n,r)$ is negative for all registers, no register is allocated to node 'n'.

3.2 Definition of the Impact Range

Consider the derivation of $\text{PenaltySpill}(3)$ in the varef-graph of Fig. 3 (a). To derive $\text{PenaltySpill}(3)$, it is necessary to drive $\text{ProbSpill}_{3\text{-spill}}(m)$ for all 'm'. Recall that node '4' is most likely to be spilled if node '3' is spilled. Thus, $\text{ProbSpill}_{3\text{-spill}}(4) \cong 1$ is a reasonable approximation. Consider the spill probability of node '10'. This spill probability depends on the register allocation result at node '3' as well as the five nodes between node '3' and node '10'. The dependence on the other five nodes may be larger than that on node '3' because the dependence may decrease as the distance from node '10' increases. In fact, the distance from node '3' is large enough that the spill probability may hardly depend on node '3'. Thus, the spill probability of '10' may not differ whether node '3' is spilled or receives a register, i.e., $\text{ProbSpill}_{3\text{-spill}}(10) \cong \text{ProbSpill}_{3\text{-preempt-r1}}(10)$. In the derivation of $\text{BenefitRegAlloc}(3,r1) = \text{PenaltySpill}(3) - \text{PenaltyPreempt}(3,r1)$, $\text{PenaltySpill}(3)$ and $\text{PenaltyPreempt}(3,r1)$ include the terms $\text{ProbSpill}_{3\text{-spill}}(10) \text{cost}(10)$ and $\text{ProbSpill}_{3\text{-preempt-r1}}(10) \text{cost}(10)$, respectively. Since the values of these two terms are equal, they are cancelled out. Thus, these terms can be omitted in the evaluations of $\text{PenaltySpill}(3)$ and $\text{PenaltyPreempt}(3,r1)$.

Consider the effect of the register allocation for node 'n' to another node 'm'. The effect decreases as the distance between the two nodes increases. If the distance from node 'n' to 'm' is large enough, the spill probability of 'm' is independent of the register allocation for 'n'. To represent the range in which a register allocation is affected, this section defines a range called *the impact range of node 'n' for register 'r'*. In the impact range, the register allocation of node 'n' affects the spill probability of other nodes so that the spill probability depends on whether node 'n' is spilled or not. This range is denoted $\text{ImpactRange}(n,r)$ and defined as follows:

$$\text{ImpactRange}(n,r) = \{ m \mid \text{ProbSpill}_{n\text{-spill}}(m) \neq \text{ProbSpill}_{n\text{-preempt-r}}(m) \} . \quad (4)$$

When a node ‘m’ is out of the impact range of node ‘n’ for register ‘r’, $\text{ProbSpill}_{n\text{-spill}}(m)$ and $\text{ProbSpill}_{n\text{-preempt-r}}(m)$ may be the same. Thus, the derivation of $\text{PenaltySpill}(n)$ and $\text{PenaltyPreempt}(n,r)$ do not require to compute $\text{ProbSpill}_{n\text{-spill}}(m)$ and $\text{ProbSpill}_{n\text{-preempt-r}}(m)$ because they are eventually cancelled out when $\text{BenefitRegAlloc}(n,r)$ is evaluated. For the estimation of the spill and the preempt penalties, only those nodes in the impact range contribute to the estimation of spill and preempt penalties. Thus, Eq. (1) and (2) can be re-expressed as follows:

$$\text{PenaltySpill}(n,r) = \sum_{m \in \text{ImpactRange}(n,r)} \text{ProbSpill}_{n\text{-spill}}(m) \text{cost}(m), \quad (5)$$

$$\text{PenaltyPreempt}(n,r) = \sum_{m \in \text{ImpactRange}(n,r)} \text{ProbSpill}_{n\text{-preempt-r}}(m) \text{cost}(m). \quad (6)$$

Note that the spill penalty of Eq. (5) is now dependent on the preemption register ‘r’ because it depends on $\text{ImpactRange}(n,r)$.

Consider Fig. 3 (a) again. The impact range of node ‘3’ for register ‘r1’ is {3, 4, 5, 6} (the derivation of the impact range is to be explained in the next subsection). Thus, $\text{PenaltySpill}(3,r1) = \sum_{m \in \{3,4,5,6\}} \text{ProbSpill}_{3\text{-spill}}(m) \text{cost}(m)$ and $\text{PenaltyPreempt}(3,r1) = \sum_{m \in \{3,4,5,6\}} \text{ProbSpill}_{3\text{-preempt-r1}}(m) \text{cost}(m)$.

Even included in the impact range, some nodes do not contribute to $\text{BenefitRegAlloc}(n,r)$. Consider the spill probability of node ‘5’ in Fig. 3 (a). The spill cost of node ‘5’ may not be affected by the register allocation result at node ‘3’. This is because node ‘5’ references variable ‘b’ while node ‘3’ references variable ‘n’. In addition, node ‘5’ is also irrelevant of register ‘r1’ that is held by variable ‘a’. Therefore, $\text{ProbSpill}_{3\text{-spill}}(5) \equiv \text{ProbSpill}_{3\text{-preempt-r1}}(5)$ is a reasonable approximation and node ‘5’ can be omitted in the evaluation of $\text{BenefitRegAlloc}(3,r1)$.

In general, only two types of nodes mainly contribute to $\text{BenefitRegAlloc}(n,r)$. First, the node that references the same variable as node ‘n’ makes contribution. The second type is the node that references the variable that holds register ‘r’ when node ‘n’ is visited for register allocation. All other nodes do not contribute to $\text{BenefitRegAlloc}(n,r)$ because their register allocation is not affected by the register allocation results at the node ‘n’. Let $\text{var}(n)$ denote the variable that is referenced by node ‘n’. Let $\text{VarHold}(n,r)$ denote the variable that holds register ‘r’ when the register allocation is performed for node ‘n’. Let $\text{NodeHold}(n,r)$ denote the nodes that reference $\text{VarHold}(n,r)$, a predecessor of ‘n’, and no other nodes that reference $\text{VarHold}(n,r)$ exists between $\text{NodeHold}(n,r)$ and ‘n’. The impact set is defined as the subset of impact range that includes only the contributing nodes:

$$\text{ImpactSet}(n,r) = \{m \mid m \in \text{ImpactRange}(n,r), \text{ and } (\text{var}(m) = \text{var}(n) \text{ or } \text{var}(m) = \text{VarHold}(n,r))\}. \quad (7)$$

Let $\text{EffectivePenaltySpill}(n,r)$ and $\text{EffectivePenaltyPreempt}(n,r)$ denote the penalties that include only the nodes in the $\text{ImpactSet}(n,r)$. Then,

$$\text{EffectivePenaltySpill}(n,r) = \sum_{m \in \text{ImpactSet}(n,r)} \text{ProbSpill}_{n\text{-spill}}(m) \text{cost}(m). \quad (8)$$

$$\text{EffectivePenaltyPreempt}(n,r) = \sum_{m \in \text{ImpactSet}(n,r)} \text{ProbSpill}_{n\text{-preempt-r}}(m) \text{cost}(m). \quad (9)$$

Then, $\text{BenefitRegAlloc}(n,r)$ can be re-expressed in terms of the effective penalties:

$$\text{BenefitRegAlloc}(n,r) = \text{EffectivePenaltySpill}(n,r) - \text{EffectivePenaltyPreempt}(n,r). \quad (10)$$

For further simplification, the spill probability in the impact set is set to either zero or one. If a node 'n' is spilled, then all the nodes that reference the same variable have the spill probability set to one. On the other hand, if a node 'n' receives a register 'r', then all the nodes that reference the same variable has the spill probability set to zero. For nodes that use the variable that the register currently holds, the spill probability is set to one if node 'n' preempts register 'r'. On the other hand, the spill probability of the same nodes is set to zero if node 'n' does not preempt register 'r'. These probabilities are summarized as follows:

$$\text{ProbSpill}_{n\text{-spill-}r}(m) = 1 \text{ if } m \in \text{ImpactSet}(n,r) \text{ and } \text{var}(m) = \text{var}(n) . \quad (11)$$

$$\text{ProbSpill}_{n\text{-preempt-}r}(m) = 0 \text{ if } m \in \text{ImpactSet}(n,r) \text{ and } \text{var}(m) = \text{var}(n) . \quad (12)$$

$$\text{ProbSpill}_{n\text{-preempt-}r}(m) = 1 \text{ if } m \in \text{ImpactSet}(n,r) \text{ and } \text{var}(m) = \text{VarHold}(n,r) . \quad (13)$$

$$\text{ProbSpill}_{n\text{-spill-}r}(m) = 0 \text{ if } m \in \text{ImpactSet}(n,r) \text{ and } \text{var}(m) = \text{VarHold}(n,r) . \quad (14)$$

Here, the subscript 'n-spill-r' is used instead of 'n-spill' to represent that the spill probability is dependent on register 'r'. Now, the effective spill penalty is re-expressed as follows:

$$\text{EffectivePenaltySpill}(n,r) = \sum_{m \in \text{ImpactSet}(n,r)} \text{ProbSpill}_{n\text{-spill-}r}(m) \text{ cost}(m) . \quad (15)$$

Thus, the effective penalties can be expressed as follows:

$$\text{EffectivePenaltySpill}(n,r) = \sum_{m \in \text{ImpactSet}(n,r) \text{ and } \text{var}(m) = \text{var}(n)} \text{cost}(m) . \quad (16)$$

$$\text{EffectivePenaltyPreempt}(n,r) = \sum_{m \in \text{ImpactSet}(n,r) \text{ and } \text{var}(m) = \text{VarHold}(n,r)} \text{cost}(m) . \quad (17)$$

Consider $\text{ImpactSet}(3,r1)$ for the varef-graph in Fig. 3 (a). Since $\text{var}(3)=\text{var}(4)=n$ and $\text{VarHold}(3,r1)=\text{var}(6)=a$, $\text{ImpactSet}(3,r1)=\{3,4,6\}$. Since $\text{var}(3)=n$, and $\text{VarHold}(3,r1)=a$, $\text{ProbSpill}_{3\text{-spill-}r1}(3)=1$ and $\text{ProbSpill}_{3\text{-spill-}r1}(4)=1$ while $\text{ProbSpill}_{3\text{-spill-}r1}(6)=0$. Thus, $\text{EffectivePenaltySpill}(3,r1) = \sum_{m \in \{3,4,6\} \text{ and } \text{var}(m)=n} \text{cost}(m) = \text{cost}(3) + \text{cost}(4)$. On the other hand, $\text{ProbSpill}_{3\text{-preempt-}r1}(3)=0$ and $\text{ProbSpill}_{3\text{-preempt-}r1}(4)=0$ while $\text{ProbSpill}_{3\text{-preempt-}r1}(6)=1$ resulting in $\text{EffectivePenaltyPreempt}(n,r1) = \sum_{m \in \{3,4,6\} \text{ and } \text{var}(m)=a} \text{cost}(m) = \text{cost}(6)$.

3.3 Derivation of an Impact Range

By the register allocation at node 'n' for register 'r', the nodes that are most likely to be affected are all the nodes in the varef-graph between node 'n' and the nodes that reference the variable that currently holds register 'r'. Thus, the impact range is defined as these nodes. This subsection presents the mathematical representation of the impact range.

For nodes 'n₁' and 'n₂' that reference the same variable, if 'n₂' immediately succeeds 'n₁' in the graph (i.e., no other node referencing the same variable exists between 'n₁' and 'n₂'), 'n₂' is called a *next reference* of 'n₁' and 'n₁' is called a *previous reference* of 'n₂'. A node may have more than one next references or previous references. The sets of next references and previous references of a node 'n' are defined, respectively, as follows:

$$\text{NextRef}(n) = \{p \mid p \text{ is a next reference of } n\} . \quad (18)$$

$$\text{PrevRef}(n) = \{p \mid p \text{ is a previous reference of } n\} . \quad (19)$$

For a set of nodes S , the sets of next references and previous references of S are, respectively, the unions of the sets of next references and previous references of each element in S .

$$\text{NextRef}(S) = \bigcup_n \text{NextRef}(n) \text{ for all } n \in S . \quad (20)$$

$$\text{PrevRef}(S) = \bigcup_n \text{PrevRef}(n) \text{ for all } n \in S . \quad (21)$$

Let $\text{path}(n,s)$ denote all the nodes in the paths from node ' n ' to node ' s '. Note that there may exist multiple paths from node ' n ' to node ' s ' and the set $\text{path}(n,s)$ includes all the paths. For a given path (n,s) , the $\text{subpath}(n,p,s)$ is the $\text{path}(p,s) \subset \text{path}(n,s)$. Note that $\text{subpath}(n,p,s)$ is an empty set if $p \notin \text{path}(n,s)$. Subsequently, SubPathRange for a node and SubPathRange for a set are defined respectively as follows:

$$\text{SubPathRange}(n,p) = \bigcup_s \text{subpath}(n, p, s) \text{ for all } s \in \text{NextRef}(n) . \quad (22)$$

$$\text{SubPathRange}(S,p) = \bigcup_n \text{SubPathRange}(p, n) \text{ for all } n \in S . \quad (23)$$

Finally, the impact range of node ' n ' for register ' r ' is defined as the path from node ' n ' to the next references of $\text{NodeHold}(n,r)$. This path is represented as the subpath from $\text{NodeHold}(n,r)$ to their next references passing through node ' n '. Thus, the impact range is defined as:

$$\text{ImpactRange}(n,r) = \text{SubPathRange}(\text{NodeHold}(n,r),n) . \quad (24)$$

Consider the derivation of $\text{ImpactRange}(5,r1)$ in the varef-graph shown in Fig. 3 (b). Assume that both nodes '1' and '2' hold register ' $r1$ ', that is $\text{NodeHold}(5,r1)=\{1,2\}$. $\text{NextRef}(1)=\{10\}$, $\text{NextRef}(2)=\{8, 9, 10\}$, and $\text{NextRef}(\{1, 2\})=\{8,9,10\}$. Node '5' is in $\text{path}(1,10)$, thus, $\text{subpath}(1,5,10)=\{5,7,10\}$ and $\text{SubPathRange}(1,5)=\{5,7,10\}$. $\text{subpath}(2,5,8) = \{\}$ because node '5' is not in $\text{path}(2,8)$. Similarly, $\text{subpath}(2,5,9) = \{\}$. Node '5' is in $\text{path}(2,10)$. Thus, $\text{subpath}(2,5,10) = \{5,7,10\}$. Thus, $\text{SubPathRange}(2,5)=\{5,7,10\}$. Thus, $\text{SubPathRange}(\{1,2\},5) = \text{SubPathRange}(1,5) \cup \text{SubPathRange}(2,5)=\{5,7,10\}$. Thus, $\text{ImpactRange}(5,r1) = \text{SubPathRange}(\text{NodeHold}(5,r1),5) = \{5,7,10\}$.

3.4 Estimation of Spill Costs

This subsection estimates the spill cost of a node. When a node is spilled, a load/store instruction is necessary not only for the execution of the node itself but also for some other nodes that reference the same variable as the spilled node. For example, consider the varef-graph shown in Fig. 3 (c). For illustration, the assignment symbol '=' is given in the right of a variable for a definition reference, and in the left for a use reference. Assume that the allocator assigns ' $r1$ ' to a node '1' and runs out of registers at node '2'. Consider the estimation of the spill penalty of node '2' ($\text{PenaltySpill}(2,r1)$). According to the previous sections, the impact range is $\{2,4,6\}$.

Thus, $\text{EffectivePenaltySpill}(2, r1) = \text{cost}(2) + \text{cost}(4) + \text{cost}(6)$. Consider the estimation of $\text{cost}(6)$. If node '6' is spilled, a load instruction needs to be inserted for the execution of node '6'. Additional store instruction is also necessary for node '5' which also references variable 'b'. This is because node '6' loads data from memory and therefore all previous references should store the value into memory for the load by node '6'. Let $\text{NodeCost}(n)$ denote the cost of the execution of each node 'n'. Then, $\text{cost}(6)$ is the summation of $\text{NodeCost}(6)$ and $\text{NodeCost}(5)$, i.e., $\text{cost}(6) = \text{NodeCost}(6) + \text{NodeCost}(5)$. Consider the estimation of $\text{cost}(4)$. If node '4' is spilled, additional store instruction is necessary for node '4' itself as well as node '8'. This is because all the next references of a definition require to reload the value at the next uses. Thus, $\text{cost}(4) = \text{NodeCost}(4) + \text{NodeCost}(8)$.

In general, if a node 'm' is in an impact range and is a use reference, $\text{cost}(m)$ includes all the previous use references:

$$\text{cost}(m) \mid_{m:\text{use}} = \text{NodeCost}(m) + \sum_{k \in \text{PrevRef}(m), k \notin \text{impact range}} \text{NodeCost}(k) . \quad (25)$$

Note that the second term, the summation, excludes node 'k' that is inside the current impact range. This is because the $\text{NodeCost}(k)$ is added when $\text{cost}(k)$ is evaluated if 'k' is inside the current impact range. If a node 'm' is a definition reference, $\text{cost}(m)$ includes all the next use references of 'm':

$$\text{cost}(m) \mid_{m:\text{definition}} = \text{NodeCost}(m) + \sum_{k \in \text{NextRef}(m), k:\text{use}, k \notin \text{impact range}} \text{NodeCost}(k) . \quad (26)$$

From (25) and (26),

$$\text{cost}_{n\text{-spill-}r}(m) = \text{NodeCost}(m) + \sum_{k \in \text{PrevRef}(m), m:\text{use}, k \notin \text{ImpactRange}(n,r)} \text{NodeCost}(k) + \sum_{k \in \text{NextRef}(m), k:\text{use}, m:\text{definition}, k \notin \text{ImpactRange}(n,r)} \text{NodeCost}(k) . \quad (27)$$

Here, the subscript 'n-spill-r' is attached to $\text{cost}_{n\text{-spill-}r}(m)$ to represent that the cost is evaluated for the register allocation at node 'n' to be spilled for register 'r'. Now consider the evaluation of the spill cost when node 'n' preempts register 'r'. The cost is the same as Eq. (27) except the last term (the second summation) which is not necessary. This is because there is no need to insert a reload instruction at the next use of a definition when the next use is not cut by 'n'. Thus,

$$\text{cost}_{n\text{-preempt-}r}(m) = \text{NodeCost}(m) + \sum_{k \in \text{PrevRef}(m), m:\text{use}, k \notin \text{ImpactRange}(n,r)} \text{NodeCost}(k) . \quad (28)$$

Consider the evaluation of the $\text{NodeCost}(k)$. By the time the cost of a node 'k' is evaluated, its previous reference or next reference may be already visited. Thus, the $\text{NodeCost}(k)$ in (27) or (28) depends on the register allocation status of node 'k'. If node 'k' is already spilled, no additional cost is necessary. Thus, $\text{NodeCost}(k)=0$ in this case. If node 'k' is already allocated with a different register, a copy instruction is required to keep the consistency between two different registers. Thus, it is desirable to discourage this case so that the spill cost is reduced. In the other case when the node is not visited yet or allocated to the same register, the cost is simply the estimated execution time of the node. Thus, the $\text{NodeCost}(k)$ is defined as follows:

$$\begin{aligned} \text{NodeCost}(k) = 0 & \quad \text{if already visited and spilled} \\ & \text{-time}(k) \quad \text{if already visited and allocated to register } r' \neq r \\ & \text{time}(k) \quad \text{otherwise} . \end{aligned} \quad (29)$$

where $\text{time}(k)$ is the estimated execution time of each node whose formal definition is given below:

$$\text{time}(k) = \begin{cases} 2 \times 10^d & \text{if a node 'k' is a load or a store} \\ 10^d & \text{if a node 'k' is rematerialization.} \end{cases} \quad (30)$$

where d is a loop depth, and d is zero for a node not inside a loop. Note that the value of 2 is used for a load and a store because the value of 1 is, in general, used only for rematerialization as in [4].

4 Scratch Allocation

When the number of registers is not enough to hold all variables, the variable register allocation discussed in the previous sections cannot allocate a register to all variables. In addition, temporaries or intermediate values also demand registers, but they are not considered in the variable register allocation. For simplicity, both unallocated variables and temporaries are called *scratches* in this paper.

For scratch allocation, nodes corresponding to scratches are added to the varef-graph. Similarly to variable register allocation, scratch register allocation is performed by traversing the varef-graph in the modified breadth-first order. When the allocator visits a scratch node, it allocates a register to the scratch if a free register is available. Those scratches are allocated in the first step of scratch allocation. In the second step when available registers are exhausted, the allocator must preempt a register to allocate it for the scratch. Those are called *constrained* scratches. The varef-graph is re-traversed in the modified breadth-first order again in the second step and the register preemption benefit is computed for all registers. Then, the register with the maximum benefit is selected. In general, the estimation of the preemption penalty is not as easy as that for variable allocation because a preempted variable must be re-allocated at the next references. For simplicity, it is assumed that the same register is assigned to the next references when a variable is preempted for a scratch, and the preemption cost for a scratch 's' is defined similarly to variable register allocation:

$$\text{EffectivePenaltyPreempt}(s,r) = \sum_{m \in \text{ImpactSet}(s,r)} \text{cost}(m). \quad (31)$$

Now, consider the spill cost of a scratch. The meaning of the spill cost is slightly different from that for variable register allocation. If a scratch 's' preempts a register 'r', then this register can be used for the scratch 's' as well as other scratches that are in the impact range. Thus, the spill cost of a scratch 's' is the summation of the costs of all the scratches that can be allocated to the same register as 's'. For a given register 'r', not all scratches in $\text{ImpactSet}(s,r)$ can be allocated to the same register 'r' because of the overlapping of their live ranges. Thus, scratches are classified into equivalent classes such that all scratches in each equivalent class can be allocated to the same register. Then, the spill cost is the summation of the costs of nodes in the equivalent class that a scratch 's' belongs to. Let $\text{CLASS}(s)$ be the equivalent set that the scratch 's' belongs to. Then spill penalty is defined as

$$\text{EffectivePenaltySpill}(s, r) = \sum_{m \in \text{ImpactSet}(s, r), m \in \text{CLASS}(s)} \text{cost}(m). \quad (32)$$

To derive the equivalent set, a conflict graph is constructed such that the node represents each scratch and the edge represents the relationship that the corresponding two variables cannot share the same register. All the constrained scratches are colored with infinite virtual colors, and then scratches are partitioned into class according to the assigned virtual color. Although the equivalent set needs to be derived for each impact region, it is derived just once throughout a program in the proposed scratch allocation. Although this derivation is not precise, it can produce well-approximated equivalent sets.

Fig. 4 illustrates scratch register allocation. Suppose that the variable allocator assigns a register ‘r1’ to ‘a’, and ‘r2’ to ‘b’. Assume that ‘v1’, ‘v2’, ‘v3’, ‘v4’, ‘v5’, ‘v6’, and ‘v7’ are all constrained scratches. For each scratch, the equivalent class number such as C1, C2 are specified in the parenthesis in the right of the name. Assume that the scratch allocator encounters scratch ‘v1’. The $\text{ImpactSet}(v1, r1) = \{v1, v2, v3, v4, v5, 3\}$. Thus, $\text{EffectivePenaltyPreempt}(v1, r1) = \text{cost}(3) = \text{NodeCost}(3) + \text{NodeCost}(1) = 4$. Since v1, v3, and v5 are in the same equivalent class, $\text{EffectivePenaltySpill}(v1, r1) = \text{cost}(v1) + \text{cost}(v3) + \text{cost}(v5) = 6$. Thus, the preemption benefit of ‘r1’ is 2. For ‘r2’, $\text{ImpactSet}(v1, r2) = \{v1, v2, v3, v4, v5, 3, v6, v7, 4\}$. $\text{EffectivePenaltyPreempt}(v1, r2) = \text{cost}(4) = \text{NodeCost}(4) + \text{NodeCost}(2) = 22$ considering node ‘4’ is inside the loop. Since ‘v1’, ‘v3’, ‘v5’, and ‘v6’ are in the same equivalent class, $\text{EffectivePenaltySpill}(v1, r2) = 8$. Now the preemption benefit is -14. Thus ‘v1’ preempts a register ‘r1’, and v1, v3, and v5 are assigned to register ‘r1’.

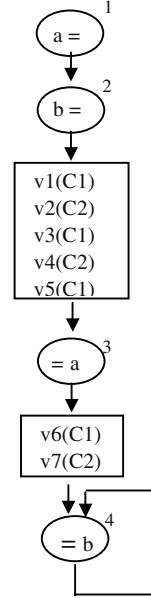


Fig. 4. Example graph for illustrating scratch allocation

5 Evaluation

5.1 Complexity Analysis

Consider the complexity of the proposed algorithm. The variable flow graph can be constructed by classical reaching definition analysis in [1], [14]. The dominant complexity is in the derivation of the impact range. The derivation of an impact range may search for all nodes in the graph and requires computation with $O(N)$ complexity where N is the number of nodes in the varef-graph. Since this computation is iterated for each register, it requires to be evaluated $O(RN)$ times, where R is the number of registers. This stage is iterated N times for each node, and N is much larger than R , the total complexity is $O(N^2)$. For the derivation of the impact range, search spaces are

localized because the next reference of a variable is generally located close to the node. Thus, the complexity may not increase as N increases in many application programs and the time complexity of the proposed approach is close to $O(N)$ in these programs.

The dominant space requirements for allocation are register context areas for each node. The required register spaces per node are $O(R)$. Due to the rapid growth of memory spaces, the space used at compile time is not an important issue in modern compilers.

5.2 Experimental Results

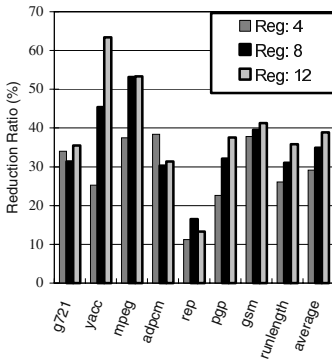


Fig. 5. The ratio of the number of spill instructions generated by the proposed approach and the Briggs' approach

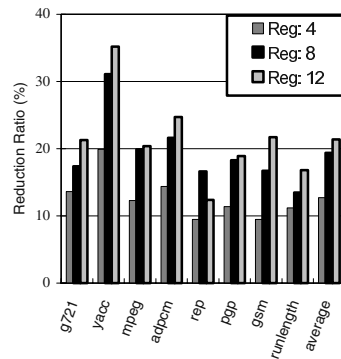


Fig. 6. The ratio of the number of spill instructions generated by the proposed approach and interference region spilling

To evaluate the efficiency, the proposed register allocation is implemented in LCC [8] (Local C Compiler) targeting ARM7TDMI processor [2]. For comparison, two more register allocators based on Briggs' algorithm [4], [5] and interference region spilling [3] are also implemented. The reason of choosing these two allocators is that the Briggs' algorithm is a widely used variation of the graph-coloring approach while the interference region spilling is one of the latest and best versions of the graph-coloring approach.

Fig. 5 shows the improvements achieved by the proposed approach. The vertical axis of the graph represents the ratio of the number of spill instructions generated by the proposed allocator and that by the Briggs' allocator. In counting the number of spill instructions, they are weighted by 10^d if the instructions are inside a loop with nesting depth d . The benchmarks are g721, yacc, adpcm, mpeg, rep, pgp, gsm, and runlength programs. The number of available registers is changed from 4, 8, to 12. With the eight benchmarks, an average of 34.3% improvement is achieved by the proposed approach over the Briggs' approach. As the number of registers increases from 4, and 8, to 12, the average improvement changes from 29.1%, and 34.9%, to 38.9%, respectively. For a small number of registers, too many spills occur even for the proposed approach, and consequently, the relative reduction ratio is small. For

every benchmark, the proposed allocator spills fewer instructions than Briggs' allocator and the reduction ratio ranges from 11.2% to 63.4%.

Fig. 6 shows the ratio of improvements achieved by the proposed approach compared to the interference region spilling. For the same benchmarks as in Fig. 5, an average of 17.8% improvements are achieved. It reduces spill instructions by 12.7%, 19.4%, and 21.4% for 4, 8, and 12 registers, respectively. It outperforms in every benchmark

Table 1. The ratio of compilation time by the proposed approach and Briggs' approach

benchmark	Number of registers		
	4	8	12
g721	1.64	1.86	1.97
yacc	1.73	2.13	2.01
mpeg	3.28	2.77	2.79
adpcm	1.29	1.49	1.62
rep	2.21	2.00	2.17
pgp	1.42	1.75	1.67
gsm	1.49	1.24	1.10
runlength	1.34	1.41	1.93

The compilation times for both the proposed approach and Briggs' approach are measured and compared in Table 1. In this table, the first column from the left represents benchmark programs, and the second, the third, and the fourth columns show the ratio of the compilation time of the proposed allocator and the Briggs' allocator when the numbers of registers are 4, 8, and 12, respectively. The ratios of the computation times vary from 1.10 to 3.28. The large increases in compilation time are due to the computation for the derivation of the impact range. Even though the proposed approach consumes much time, it is quite affordable considering the rapid growth of recent computing powers.

6 Conclusions

The proposed register allocator improves the Briggs' allocator by an average of 34.3% and the interference region spilling approach by 17.8%. This significant improvement is achieved in trade-off with the increase of computation time for analyzing the flow of all variable references. The compilation time is by an average of 1.85 times larger than that for Briggs' allocator. The time increase by the amount of 85% is not serious considering that graph-coloring allocators run fast in practice. This trade-off is in the right direction because recent dramatic increase of processor computing power may make aggressive compiler optimizations affordable.

The varef-graph used in the proposed register allocator has a large amount of information such as control flow, execution cost, and load/store identification. This

information may be used for further optimizations such as cooperation with instruction scheduling.

References

1. Aho, A.V., Sethi, R., and Ullman J.D.: *Compilers: Principles, Techniques, and Tools*. Addison-Wesley Publishing Company, Reading Mass (1986).
2. Advanced RISC Machines Ltd: *ARM Architecture Reference Manual*. Document Number: ARM DDI 0100B, Advanced RISC Machines Ltd. (ARM) (1996).
3. Bergner, P., Dahl, P., Engebretsen, D., and O'Keefe, M.: Spill code minimization via interference region spilling. *Proceedings of the ACM PLDI '97* (June 1997), 287-295.
4. Briggs, P., Cooper, K.D., and Torczon, L.: Rematerialization. *Proceedings of the ACM SIGPLAN'92 Conference on Programming Language Design and Implementation*, SIGPLAN Notices 27, 7 (June 1992), 311-321.
5. Briggs, P., Cooper, K.D., Kennedy, K., and Torczon, L.: Coloring heuristics for register allocation. *Proceedings of the ACM SIGPLAN'89 Conference on Programming Language Design and Implementation*, SIGPLAN Notices 24, 6 (June 1989), 275-284.
6. Chaitin, G.J.: Register allocation and spilling via coloring. *Proceedings of the ACM SIGPLAN '82 Symposium on Compiler Construction*, SIGPLAN Notices 17, 6 (June 1982), 98-105.
7. Chaitin, G.J., Auslander, M.A., Chandra, A.K., Cocke, J., Hopkins, M., and Markstein, P.W.: Register allocation via coloring. *Computer Languages* 6 (January 1981), 47-57.
8. Fraser, C.W., and Hanson, D.R.: *A Retargetable C Compiler: Design and Implementation*. Benjamin/Cummings, Redwood City CA (1995).
9. Farach, M., and Liberatore, V.: On local register allocation. *Proceedings of the 9th Annual ACM-SIAM Symposium on Discrete Algorithms* (1998), 564-573.
10. Goodwin, D.W., and Wilken, K.D.: Optimal and near-optimal global register allocation using 0-1 integer programming. *Software-Practice and Experience* 26, 8 (1996), 929-965.
11. Hsu, W.-C., Fischer, C. N., and Goodman, J.R.: On the minimization of loads/stores in local register allocation. *IEEE Transactions on Software Engineering* 15, 10 (October 1989), 1252-1260.
12. Kim, D.H.: Advanced compiler optimization for CalmRISC8 low-end embedded processor. *Proceedings of the 9th Int. Conference on Compiler Construction*, LNCS 1781, Springer-Verlag (March 2000), 173-188.
13. Kolte, P., and Harrold, M.J.: Load/store range analysis for global register allocation. *Proceedings of the ACM PLDI'93* (June 1993), 268-277.
14. Mushnick, S. S.: *Advanced compiler design and implementation*. Morgan Kaufmann, San Francisco CA (1997).
15. Proebsting, T. A., and Fischer, C. N.: Demand-driven register allocation. *ACM Transactions on Programming Languages and Systems* 18, 6 (November 1996), 683-710.

Unified Instruction Reordering and Algebraic Transformations for Minimum Cost Offset Assignment

Sarvani V.V.N.S and R.Govindarajan

Indian Institute of Science, Bangalore, India 560012
{sarvani,govind}@csa.iisc.ernet.in

Abstract. DSP processors have address generation units that can perform address computation in parallel with other operations. This feature reduces explicit address arithmetic instructions, often required to access locations in the stack frame, through auto-increment and decrement addressing modes, thereby decreasing the code size. Decreasing code size in embedded applications is extremely important as it directly impacts the size of on-chip program memory and hence the cost of the system. Effective utilization of auto-increment and decrement modes requires an intelligent placement of variables in the stack frame which is termed as “offset assignment”. Although a number of algorithms for efficient offset assignment have been proposed in the literature, they do not consider possible instruction reordering to reduce the number of address arithmetic instructions. In this paper, we propose an integrated approach that combines instruction reordering and algebraic transformations to reduce the number of address arithmetic instructions. The proposed approach has been implemented in the SUIF compiler framework. We conducted our experiments on a set of real programs. and compared its performance with that of Liao’s heuristic for Simple Offset Assignment (SOA), Tie-break SOA, Naive offset assignment, and Rao and Pande’s algebraic transformation approach.

1 Introduction

Embedded processors (e.g., fixed point digital signal processors, and micro controllers) are found increasingly in audio, video and communication equipments, cars, etc. While optimizing compilers have proved effective for general purpose processors, the irregular data paths and small number of registers found in embedded processors, remain a challenge to compilers [9]. The direct application of conventional code optimization methods has thus far been unable to generate code that efficiently uses the features of DSP microprocessors [9]. Thus embedded processors require not only the traditional compiler optimization techniques, but also new techniques that take advantage of the special architectural features provided by the DSP architectures. Further, the optimization goals for such processors are not just higher performance but also lower energy/power consumption.

A compile-time optimization that is important in embedded systems is code size reduction. This is because embedded processors have limited code (program) and data memory in order to keep the system cost low. Therefore, making efficient use of available program memory is very important to achieve both higher performance and cost reduction.

Many DSP processors, such as Analog Devices ADSP210x, Motorola 56K processor family, and TI TMS320C2x DSP family, have dedicated address generation units (AGUs) for parallel next-address computations through auto-increment and auto-decrement addressing modes. This feature allows address arithmetic computations to be a part of other instructions. This eliminates the need for explicit address arithmetic instructions in certain cases which, in turn, leads to code size reduction. However, in order to fully exploit this feature, an intelligent placement of automatic variables in the stack frame is necessary. The placement, or the address assignment, of these variables in the memory and their access order significantly impacts the code size. The address assignment of automatic variables, so that the number of explicit address arithmetic instructions used to access them is reduced is referred to as the “offset assignment”. The number of address arithmetic instructions required is referred to as the offset assignment cost. When the address generation unit consists of only one Address Register (AR), the offset assignment problem is called the *Simple Offset Assignment (SOA)* problem [9]. The generalization which handles any fixed number of k address registers is referred to as the *General Offset Assignment (GOA)* problem [9].

Offset assignment problem was first studied by Bartley [3] and subsequently by Liao [9]. Liao solved the simple offset assignment problem by reducing it to the maximum weight path cover problem. A generalized address assignment problem for a generic AGU model and an improved heuristic solution were discussed in [6]. The GOA problem was further generalized in [7] to include modify register (MR) and non-unit constant increment/decrement to AR. A solution method based on genetic algorithm was also proposed to solve the SOA and GOA problems. In [12], the cost of offset assignment was further reduced by exploiting commutativity and associativity properties of arithmetic expressions through algebraic transformations. All of these approaches consider a fixed instruction sequence and attempt to obtain efficient address assignment to reduce the cost. Our solution to the SOA problem considers possible instruction reordering to achieve more efficient solutions for the offset assignment problem. A somewhat similar approach is proposed in [4], although there are a few differences which will be discussed in Section 2. Further, this paper, for the first time integrates instruction reordering, and algebraic transformation, together with efficient offset assignment.

We restrict our attention in this paper to the SOA problem. We propose an efficient heuristic to reorder instructions along with possible algebraic transformation on the operands of an expression to arrive at a reduced offset assignment cost. We have implemented our method in the SUIF compiler framework [15]. We evaluate the performance of the proposed approach on a number of real bench-

mark programs taken from embedded and multimedia applications. This is in contrast to many of the earlier work which evaluate their approach using a set of randomly generated instruction sequences. Also, we compare the performance of our approach with that of Liao's SOA method [9], tie-break SOA [6] approach, and Rao and Pande's algebraic transformation approach [12]. The SOA cost (the number of address arithmetic instructions) is reduced by 8.6%, 7.4%, and 1.7%, on an average, compared to Liao's SOA, Leupers' Tie-break SOA, and Rao and Pande's heuristic methods respectively. The percentage improvement over Liao's SOA and Leupers' Tie-break SOA methods, is upto 20 –37% in certain benchmarks. The percentage improvement over Rao and Pande's method is moderate in most cases, although, in very few cases (3 benchmarks) our approach produced marginally poorer solution.

The rest of the paper is organized as follows. Section 2 deals with the necessary background and related work. In Section 3 we describe our approach to the offset assignment problem. Section 4 deals with our experimental results on a set of benchmark routines. Finally, we present concluding remarks in Section 5.

2 Background and Related Work

In this section first we describe the SOA problem. Subsequently we discuss some of the proposed approaches to solve the SOA problem.

Most DSP processors are equipped with Address Generation Units (AGUs) which are capable of performing indirect address computations in parallel with the execution of other machine instructions. The AGUs contain Address Registers (ARs) which store the effective addresses of variables in memory and can be updated by load or modify (increment or decrement by unit value) operations. For two variables i and j in a procedure, and access order in which i is accessed (immediately) before j , whether the effective address of j can be computed from the effective address of i by using the auto-increment and auto-decrement operations depends on their positions (offsets) in the stack-frame.

Simple offset Assignment is the problem of assigning offsets to the automatic variables of a procedure in the presence of a single address register. We illustrate the offset assignment problem with the help of an example adopted from [12]. Consider the instruction sequence shown in Figure 1(a). The access order of the automatic variables is shown in Figure 1(b). If the variables **a**, **b**, **c**, **d**, **e**, and **f** are placed in consecutive memory locations in the stack frame, then, e.g., access to variable **b** after an access to **a** can be accomplished using the auto-increment addressing mode, thus eliminating an address arithmetic instruction. Similarly, the first six accesses can benefit from the auto-increment addressing mode. However, to access **a** after **f** in the access sequence, requires an explicit address arithmetic instruction to set the AR. It can be seen that for the above address assignment, a total of 8 address arithmetic instructions are required. We refer to this cost, as the cost of the address assignment.

Liao proved that the Simple Offset Assignment problem is NP-complete and proposed a heuristic solution for it [9]. Liao's approach to solve the SOA problem

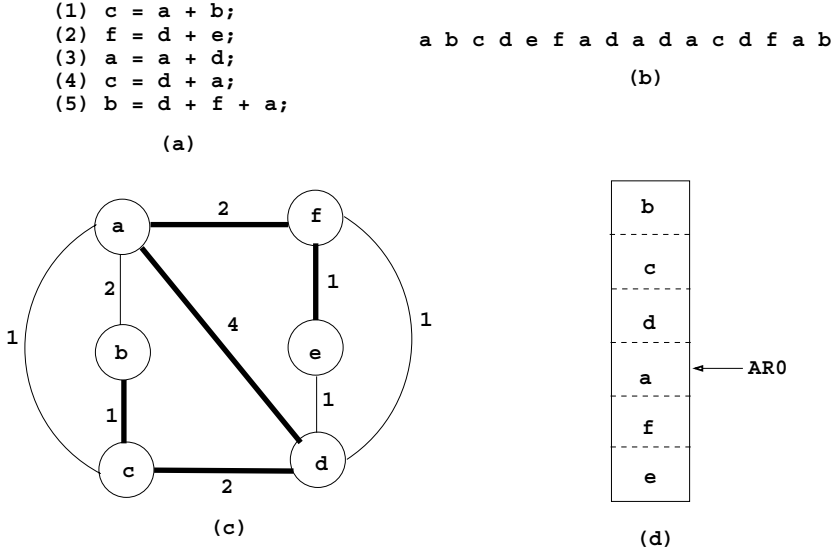


Fig. 1. Liao’s Heuristic (a) Code Sequence (b) Access Sequence (c) Access Graph (cost=5) (d) An Offset Assignment

is to formulate it as a well-defined combinatorial problem of graph covering, called *maximum weight path covering* (MWPC). From a basic block they derive an access graph [9] that gives the relative benefits of assigning each pair of variables to adjacent locations in memory. More specifically, an access graph in which each vertex corresponds to a distinct variable, and edge (v_i, v_j) with weight w exists if and only if variables i and j are adjacent to each other w times in the access sequence [9]. This graph, shown in Figure 1(c) for the example code sequence, conveys the relative benefits of assigning each pair of variables to adjacent memory locations.

An MWPC is an undirected (acyclic) path that covers all the nodes in the access graph such that the cost of the path is maximum. If (v_1, v_2) is an edge included in the MWPC, then v_1 and v_2 are assigned adjacent locations in the memory. Since v_1 and v_2 are adjacent now, the cost associated with the edge will not be incurred. Thus the edges of the graph that are not included in the MWPC contribute to the offset assignment cost.

The access graph for the instruction sequence of Figure 1(a) is shown in Figure 1(c). An MWPC in the access graph is indicated by means of thick edges. The variables connected by thin edges require explicit address arithmetic instructions, and hence contribute to the offset assignment cost. For the example assignment shown in Figure 1(d), the offset assignment cost is 5.

Leupers proposed the Tie-Break SOA heuristic [6] which assigns priority to edges with equal weights in the access graph. For an access graph $AG =$

(V, E, w) , the *Tie-Break function* $T : E \rightarrow N_0$ is defined by :

$$T(e) = \sum_{e' \in E} w(e')$$

where e' is an edge such that e and e' share a common vertex. Thus the Tie-Break function $T((v1, v2))$ is the sum of the weights of all edges that are incident on $v1$ or $v2$. For two edges e_1 and e_2 with $w(e_1) = w(e_2)$ the priority is given to the edge e_1 , exactly if $T(e_1) < T(e_2)$.

Leupers [7] formulated the offset assignment problem as an optimization problem using genetic algorithms. Atri and Ramanujam [2] propose an improvement over Liao's heuristic, by considering the maximum weight edge not included in the cover and tries to include that edge, and its effect on the cost of assignment. Rao and Pande [12] proposed a technique that applies algebraic transformations to optimize the access sequence of variables that results in fewer address arithmetic instructions. They term this problem as the Least Cost Access sequence (LCAS) problem. Their heuristic finds all the possible access sequences by applying commutative and associative transformations to each expression tree in the basic block. It then retains only those schedules having minimum number of edges. The heuristic uses Liao's access graph to find the offset assignment cost. Reordering of variables in an access sequence is restricted to accesses within a statement.

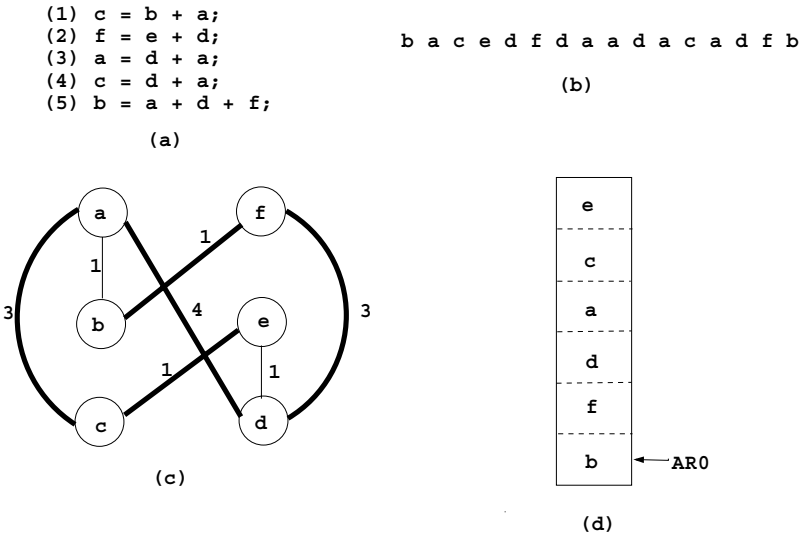


Fig. 2. Rao and Pande's Heuristic (a) Code Sequence (b) Access Sequence (c) Access Graph (cost=2) (d) An Offset Assignment

Rao and Pande's heuristic is based on the observation that reducing the number of distinct access transitions in the access sequence corresponds to an access

graph with fewer edges but possibly increased weights as compared to an access graph of the unoptimized access sequence [12]. Figure 2 shows the instruction sequence after applying algebraic transformations to the instruction sequence in Figure 1(a). For example, in instruction 3, the access order of source operands **a** and **d** are reversed so as to reduce the access transition between the last source operand (in this case **a**) and the destination operand. The access sequence in Figure 1(b) has 9 distinct access transitions while the access sequence in Figure 2(b) has only 7 distinct access transitions. This reduces the number of edges in the access graph which in turn may reduce the offset assignment cost.

Instruction reordering and offset assignment were studied together for the first time by Choi and Kim [4]. The approach proposed in this paper is somewhat similar to [4], although it was proposed independently [14]. There are two differences to these two approaches. The approach used in [4] uses a simple list-scheduling algorithm and schedules an instruction adding least cost to the access graph. Our approach uses list-scheduling internally but performs instruction scheduling exploiting data-dependences. Second, our approach integrates both instruction scheduling and algebraic transformations into a single phase, while this is performed as a separate phase after instruction scheduling in [4].

3 Our Unified Approach

In this section we motivate the unified instruction reordering and algebraic transformations for offset assignment using an example. The subsequent subsections deal with the details of the proposed solution.

3.1 Motivating Example

Consider the instruction sequence shown in Figure 3(a) which is a slightly modified sequence from the earlier example. The access sequence and the access

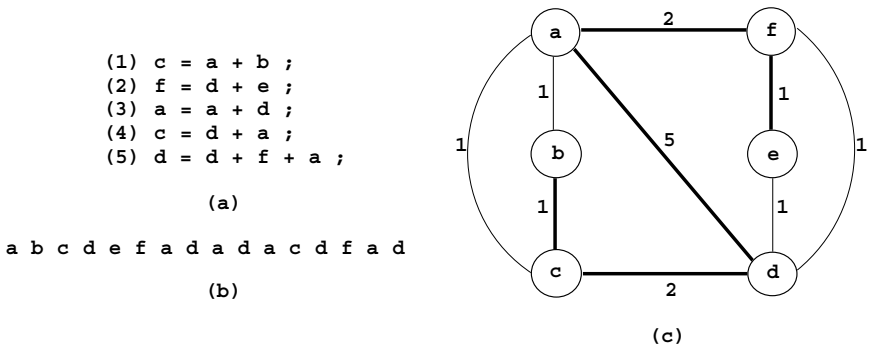


Fig. 3. Liao’s Heuristic (a) Code Sequence (b) Access Sequence (c) Access Graph (cost=4)

graphs for this are also shown in Figure 3. The maximum weight path cover is indicated by means of thick edges in the access graph. The cost of offset assignment for this access sequence is 4. It can be seen that this is the *minimum* offset assignment cost for the given access sequence. Now, if we reorder the instructions such that instructions i_3 and i_4 are scheduled ahead of instruction i_2 , and the access order of the source operands of instructions i_2 , i_3 , and i_4 reversed, then we obtain the instruction sequence shown in Figure 4(a). Note the instruction reordering performed obeys data dependences, and the commutative algebraic transformation on ‘+’ is valid.

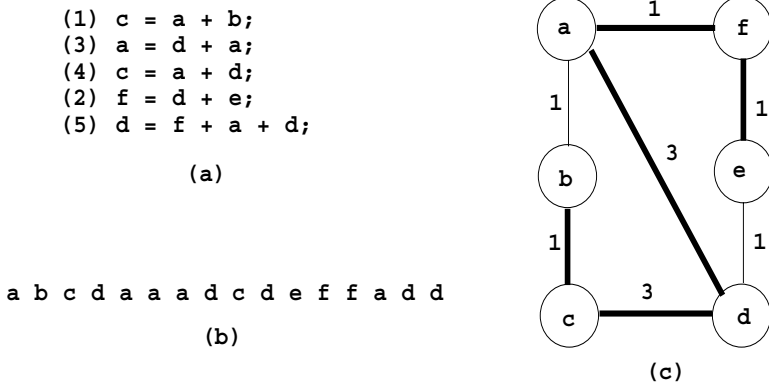


Fig. 4. Unified Heuristic (a) Code Sequence (b) Access Sequence (c) Access Graph (cost=2)

The access sequence and the access graph for the reordered instruction sequence are shown in Figure 4. As before, the maximum weight path cover is shown using thick edges. The cost of offset assignment for the modified sequence is 2, which is 50% lower than the minimum cost for the original access sequence. This shows that by reordering instructions it is possible to obtain “better” access sequences which can result in lower cost offset assignment.

3.2 Approach

It can be seen that the access graph of Figure 4, has fewer edges than the access graph of the original instruction sequence. This is an observation made by Pande and Rao in the context of algebraic transformation [12]. The same observation is also useful for instruction reordering.

Observation 1: The access sequence with fewer access transitions, i.e., having an access graph with fewer edges (but possibly with higher weights), leads to reduced offset assignment cost [12]

We make two other simple observations which lead to our unified approach.

Observation 2: When two instructions have a data dependence between them and commutativity holds on the operation involving the dependent variable, the two instructions can be scheduled as successive instructions, which we term as *Instruction Chaining*, with the dependent operand appearing as the first operand in the second instruction, to reduce the weights on the edges of the access graph.

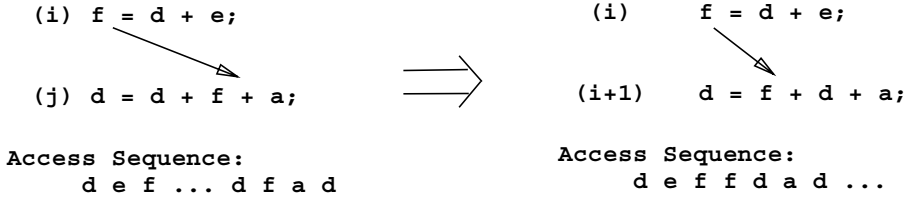


Fig. 5. Illustration of Observation 2

Figure 5 illustrates Observation 2. Instruction i is data dependent on instruction j and the dependency is on variable f . Since variable f can be commuted in j , the two instructions are *chained* and are scheduled as successive instructions, i and $(i + 1)$ as shown in Figure 5(b). With instruction chaining and (possible) operand reordering, the dependent variable appears the destination operand of i and the first source operand of $(i + 1)$. This is reflected in the access sequence as a self access transition (e.g., from f to f), which incurs zero cost in offset assignment. Thus, the resulting access sequence possibly has fewer access transitions, resulting in fewer edges in the access graph.

Observation 3: If instruction i has one of its source operand o also as its destination operand, and if the source operands can be reordered, then operand o appears as the last source operand. This reordering enables operand o to appear in succession in the access sequence, possibly reducing the number of edges in the access graph.

Figure 6 illustrates Observation 3. It can be seen from the figure that instruction $(i + 1)$ has one of its sources (operand d) same as its destination. Since, variable d can be commuted, the sources of instruction $i + 1$ can be reordered such that variable d is accessed just before the destination variable d is accessed. This is done by making the source variable d the last (right-most) source.

Note, however, that the reordering due to Observation 3 may conflict with the reordering of Observation 2, if instruction i is chained with its dependent predecessor j and the destination operands of i and j are same. this would not be possible if d were the cause of data dependence between instructions i and $i + 1$, then reordering is possible with either Observation 2 or Observation 3, but not both. In case of such a conflict, we give preference to data-dependence and chain the nodes. This preference to data-dependence is to reduce the number of

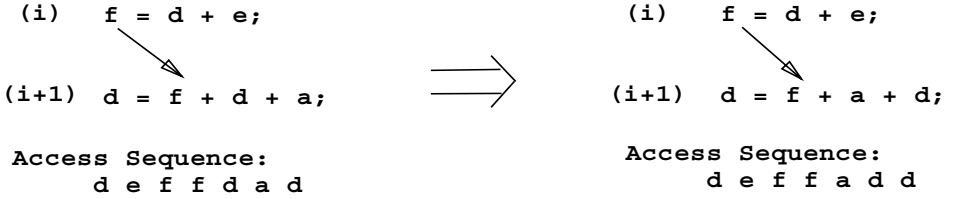


Fig. 6. Illustration of Observation 3

schedules explored as the data-dependence between two nodes fixes the schedule between the two nodes according to Observation 2.

We are now ready to describe our heuristic integrated method.

3.3 Algorithm and Methodology

Our approach proceeds by first constructing the data dependence graph (DDG) for the original instruction sequence (refer to the Algorithm shown in Figure 8). It then identifies pairs of instructions which can be chained (using Observation 2). Possible algebraic transformation (based on Observation 2 and 3) are performed on the source operands of the dependent instruction. Chaining the nodes and applying algebraic transformation are performed by the function **ChainNodesWithAlgTransformation**. The DDG for the original instruction sequence of our motivating example (refer to Figure 3) is shown in Figure 7(a). In the DDG, true dependences are shown using continuous lines and false dependences (anti- and output-dependences) are shown using dashed lines. Using

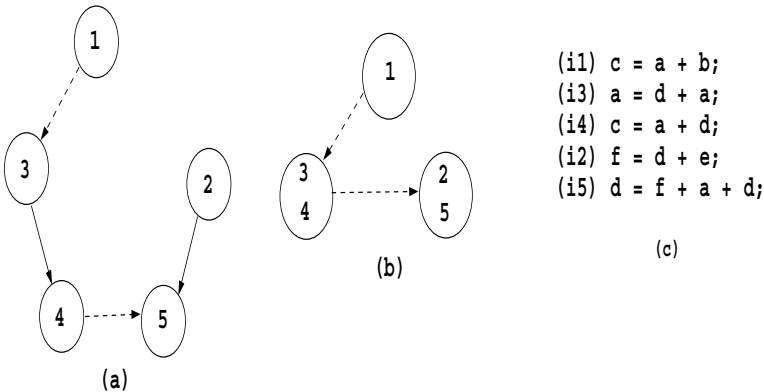


Fig. 7. Example of Unified approach (a) Data Dependence Graph (b) Data Dependence graph after chaining (c) Final Instruction schedule

Observation 2, the pairs of instructions (i2, i5) and (i3, i4) can be *chained*. The

DDG after chaining is shown in Figure 7(b). Further, algebraic transformations are applied to instructions $i3$, $i4$, and $i5$ resulting in the instruction sequence shown in Figure 7(c).

function 31. *Find Schedule*

Input : Basic Block B.

Output : Modified schedule for Basic Block B

```
{
    /* Construct Dependence Graph for the Basic Block */
    DDG = DependenceGraph (B);

    /* For nodes in DDG with single data dependent parent and no sibling,
    chain the nodes after applying possible algebraic transformations*/

    for (each node with single data dependent parent and child)
        ChainNodesWithAlgTransformation (parent, child);

    /* Initialize Ready List which has instructions with all */
    /* dependencies satisfied */
    RList = GetReadyList (DDG);
    FinalSchedules = NULL;
    FinalAccessGraphs = NULL;
    FinalAccessSeq = NULL;

    BuildPartialSchedulesIncrementally (RList, DDG, FinalSchedules,
        FinalAccessGraphs, FinalAccessSeq);

    /* Select the Schedules with least cost */
    for (each schedule S in FinalSchedules)
    {
        cost = SolveSOA (FinalAccessGraphs(S));
        if (cost < MinCost)
        {
            MinCost = cost;
            LeastCostSchedule = S;
        }
    }
    print (LeastCostSchedule);
}
```

Fig. 8. Algorithm for the Unified Approach

An instruction having more than one data-dependent parent can be chained with any of its parents. In these cases, our approach checks all possible combinations and chooses the one which may result in minimum offset assignment cost. However, since a naive approach trying all combinations of instruction chaining

is prohibitively expensive, we use an efficient heuristic to prune the search space. This heuristic, like the one used in [12], is based on the number of edges in the access graph. For this purpose, as instructions are reordered, the corresponding access sequence and access graphs are constructed incrementally in our methodology. Partial access graphs are constructed from partial access sequences for different possible schedules at each instruction level and an instruction chaining resulting in fewer edges in the access graph is chosen. Possible algebraic transformations (based on Observations 2 and 3) are applied to the reordered instruction sequence. Finally, for the possible schedules constructed by our approach, the offset assignment problem is solved using the maximum weight path cover approach [9], and the schedule that results in minimum offset assignment cost is chosen.

function 32. *BuildPartialSchedulesIncrementally*

```

Input : RList, DDG, PartialSchedules, PartialAccessGraphs, PartialAccessSeq
Output : Schedules for Basic Block B
{
    if (RList is empty)
    {
        Add PartialSchedules to FinalSchedules;
        Add PartialAccessGraphs to FinalAccessGraphs
        Add PartialAccessSeq to FinalAccessSeqs;
        return;
    }
    for (each instruction i in RList)
    {
        /* Add i to PartialSchedule after applying algebraic transformations */
        NewPartialSchedule = ConstructPartialSchedule (PartialSchedules, i);
        NewAccessGraphs = ConstructAccessGraphs (PartialAccessGraphs, i);
        NewAccessSeq = ConstructPartialAccessSequence (PartialAccessSeq, i);

        if (No. of edges in AccessGraph <= CurrentMinEdges)
        {
            /* this is a useful partial schedule */
            PartialSchedule = NewPartialSchedule;
            AccessGraphs = NewAccessGraphs;
            AccessSeq = NewAccessSeq;
            Update RList;
        }
        else
        {
            /* Discard this PartialSchedule and corresponding
               AccessGraphs and AccessSeq; */
            Goto next instruction in RList;
        }
        BuildPartialSchedulesIncrementally (RList, DDG, PartialSchedules,
                                           PartialAccessGraphs, PartialAccessSeq);
    }
}

```

Fig. 9. Algorithm for the Unified Approach (contd.)

The algorithm for our unified approach is shown in Figures 8 and 9. The function **FindSchedule** constructs the Data Dependence Graph and does chaining of nodes with a single data dependent parent and no sibling. It then con-

structs the Ready list and then calls `BuildPartialSchedulesIncrementally`. This function finds different possible ordering of the instructions in `RList`. It also considers the different possible chaining orders in case of instructions that have more than one data dependent child. The details of how the different chaining orders are considered is not shown in the algorithm for simplicity reasons.

4 Results

We have implemented the proposed heuristic in the SUIF compiler framework [15]. Our method is applied to the 3-address intermediate code (low SUIF representation). For each basic block in a procedure, we construct the DDG, and consider possible instruction chaining and algebraic transformations. Reordering of instructions is restricted to within a basic block in this paper. As we construct the reordered instruction sequence, the access graph for the basic block is constructed. The offset assignment is done for a procedure rather than a basic block. The access graph for a procedure is obtained by merging the access graphs of the individual basic blocks. That is, the access graph of a procedure includes all edges of the access graphs of its basic blocks and the weight of an edge e is the sum of the weights of the same edge e in the access graphs corresponding to different basic blocks. Since our aim is to reduce the static code size rather than the dynamic instruction count, we assumed the frequency of execution of all the basic blocks is equal. It should be noted that additional address arithmetic instructions may be added at the beginning/end of each basic block to ensure appropriate variables are accessed using the address registers. The cost reported in our experiments does not include this additional address arithmetic instructions. Although the number of instructions added may be different for different assignments, it is likely that all methods would incur more or less the same additional cost.

In addition to our method, we have implemented Liao's SolveSOA method [9], Leupers' Tie-break heuristic [6], and Rao and Pande's *Commute3-SOA* heuristic [12] in our experimental framework and compared their performance with our approach. We have also considered a *naive* offset assignment in which offsets are assigned based on the occurrence of variables in the original instruction sequence.

The benchmark routines used in our experiments are taken from real programs in DSP and multimedia applications. Benchmarks `Biquad-one-section`, `Fir`, `convolution`, `lms`, `real-update`, `fir2dim`, `dot-product` and `matrix2` are from the DSPstone benchmark suite. The routines, `doBlurConvolv`, `doPixel`, and `smoothXY`, are graphic routines from the *xv* program (Unix utility). The benchmarks, `reflect`, `erefect`, `g721-encoder` and `internal-filter`, are taken from the MediaBench benchmark suite [5]. The characteristics of the benchmarks are shown in Table 1. Column 2 reports the number of basic blocks in the benchmark routine. Columns 3 and 4 show the number of instructions in the largest basic-block and the number of scalar variables including the temporary variables.

Table 1. Benchmark Characteristics

<i>Benchmark</i>	<i>Source</i>	<i>No. of BBs</i>	<i>Largest BB Size</i>	<i>No. of scalar vars.</i>
Biquad-one-section	DSPstone	1	15	14
Fir		3	9	12
Convolution		3	6	8
lms		5	9	22
real-update		1	10	10
fir2dim		7	9	30
dot-product		3	8	13
matrix2		6	9	23
doBlurConvolve	<i>xv</i>	30	10	57
doPixel		49	19	84
smoothXY		44	19	115
reflect2	<i>Media Bench</i>	47	7	69
ereffect		59	7	85
g721-encoder		10	14	44
internal-filter		83	19	171

Table 2. Performance Results

<i>Benchmark</i>	Offset Assignment Cost				
	<i>Naive</i>	<i>Tie-Break</i>	<i>Liao's</i>	<i>Rao and Pande</i>	<i>Unified</i>
Biquad-one-section	26	16	16	11	10
Fir	25	13	13	13	11
Convolution	12	6	6	5	3
lms	42	25	26	24	23
real-update	8	5	5	5	3
fir2dim	54	41	41	31	35
dot-product	14	9	9	7	7
matrix2	44	29	30	26	26
doBlurConvolve	77	50	52	47	42
doPixel	120	91	91	89	81
smoothXY	182	131	134	123	121
reflect2	91	61	63	61	58
ereffect	121	80	82	79	76
g721-encoder	39	25	25	20	24
internal-filter	502	354	354	340	346
<i>Total</i>	1349	936	947	881	866

In Table 2 we summarize the performance of different offset assignment approaches. Columns 2 – 5 report the offset assignment costs for the different approaches, namely, the Naive approach, Tie-break SOA [6], Liao's SOA [9], Rao and Pande's heuristic [12], and our unified approach. Note that the cost reported here is the number of address arithmetic instructions (static instruction count).

It can be seen from Table 2 that our unified approach has decreased the offset assignment cost considerably for most of the benchmarks compared to *Tie-Break* and *Liao's* heuristics. Our unified approach reduces the offset assignment cost in for some of the benchmarks (e.g., **Biquad-one-section**, **Convolution**, and **doBlurCovolv**) by as much as 20–37% over *Liao's* method or over the *Tie-Break* heuristic. On the average, our approach results in 35.8% improvement over Naive offset assignment, 7.4% over the *Tie-Break* approach, and 8.6% over *Liao's* method. In some of the benchmarks (e.g., **internal-filter** and **g721-encoder**), the improvement due to instruction reordering in our unified approach is less as there were not many data dependences to be exploited. For some of the benchmarks, the unified approach reduced the offset assignment cost between 20–37% over *Liao's* heuristic and *Tie-Break* heuristic.

Compared to Rao and Pande's approach, our unified approach gives only a marginal improvement (1.7% on the average). The marginal improvement is seen in 10 out of 15 benchmarks. However, in 3 benchmarks (**fir2dim**, **g721encoder**, and **internal-filter**) our unified approach gives poorer results compared to Rao and Pande's approach. The reason for this is that in these applications, as mentioned earlier, there were only a few data dependences that could be exploited. On the average our approach shows 35.8% improvement over Naive offset assignment, 7.4% over *Tie-Break* approach, 1.7% over Rao and Pande's heuristic and 8.6% over *Liao's* Solve-SOA heuristic.

5 Conclusions

In this paper, we propose a unified approach to instruction reordering and algebraic transformations to minimize the number of address arithmetic instructions. We have implemented our approach in the SUIF compiler framework and reported performance results for a set of real benchmarks and multimedia applications. We have compared our approach with three existing approaches, viz., *Liao's* method [9], Leupers' *Tie-break* method [7], and Rao and Pande's algebraic transformation method [12]. Our unified approach results in considerable improvement over the first two methods, and marginal improvement over the third one. Since our approach considers many possible instruction ordering, its execution time to compute the minimum cost schedule may be considerable especially for large basic blocks. In this aspect our approach is similar to Rao and Pande's method, which also considers all possible operand ordering for a given instruction sequence. However, for smaller basic blocks the proposed heuristic can obtain the minimum cost schedule fairly quickly.

References

1. A. V. Aho, R. Sethi, and J. D. Ullman. *Compilers Principles, Techniques and Tools*. Addison Wesley, Reading, MA, 1988.
2. S. Atri, J. Ramanujam, and M. Kandemir. Improving offset assignment for embedded processors. In *Proc. of the Workshop on Languages and Compilers for High Performance Computing (LCPC 2000)*, Yorktown Heights, NY, Aug. 2000.

3. D.Bartley. Optimizing stack frame accesses for processors with restricted addressing modes. In *Software Practice and Experience*, 22(2):101-110, Feb. 1992.
4. Y. Choi and T. Kim. Address assignment combined with scheduling in DSP code generation. In *Proc. of the Design Automation Conference*, New Orleans, LA, June 2002.
5. C. Lee, M. Potkonjak, and W.H. Mangione-Smith. MediaBench: A tool for evaluating and synthesizing multimedia and communication systems. in *Proc. of the 30th Ann. Intl. Symp. on Microarchitecture (MICRO-30)*, Raleigh, NC, 1997.
6. R. Leupers and P. Marwedel. Algorithms for address assignment in DSP code generation. In *Intl. Conf. on Computer Aided Design*, San Jose, CA, Nov. 1996.
7. R. Leupers and F. David. A Uniform Optimization technique for Offset Assignment. In *Proc. the 11th International Symposium on System Synthesis*, 1998.
8. R. Leupers. Code generation for embedded processors. In *Proc. of the 13th Intl. Symp. on System Synthesis*, Sep. 2000.
9. S. Liao, S. Devadas, K. Keutzer, S. Tjiang, and A. Wang. Storage assignment to decrease code size. In *Proc. of 1995 ACM SIGPLAN Conference on Programming Language Design and Implementation*, La Jolla, CA, June 1995.
10. S. Liao. *Code Generation and Optimization for Embedded Digital Signal Processors*. Ph.D thesis, Department of EECS, Massachusetts Institute of Technology, Cambridge, MA, January 1996.
11. S.S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kauffman Publishers, San Francisco, 1997.
12. A. Rao and S. Pande. Storage assignment to generate compact and efficient code on embedded DSPs. In *Proc. 1999 ACM SIGPLAN Conference on Programming Language Design and Implementation*, Montreal, Canada, June 1998.
13. A. Rao. Compiler optimizations for storage assignment on embedded DSPs. Master's Thesis, Dept. of ECECS, Univ. of Cincinnati, OH, Oct. 1998.
14. Sarvani V.V.N.S and R. Govindarajan. Unified instruction reordering and algebraic transformations for minimum cost offset assignment. In *Student poster session, Programming Languages Design and Implementation, Berlin, Germany, June 2002*.
15. *Stanford University Intermediate Format*. <http://suif.stanford.edu>
16. A. Sudarshanam and S. Malik. Memory bank and register allocation in software synthesis of ASIPs. In *Proc. of 1997 ACM/IEEE Design Automation Conference on Computer-Aided Design*, pages 388-392, San Jose, CA, Nov. 1997.
17. A. Sudarsanam, S. Liao, and S. Devadas. Analysis and evaluation of address arithmetic capabilities in custom DSP architectures. In *Proc. of the 1997 ACM/IEEE Design Automation Conference*, pages 297-292, Anaheim, CA, June 1997.

Improving Offset Assignment through Simultaneous Variable Coalescing

Desiree Ottoni¹, Guilherme Ottoni², Guido Araujo¹, and Rainer Leupers³

¹ IC-UNICAMP - Brazil

² Princeton University, Department of Computer Science - USA

³ Aachen University of Technology, Integrated Signal Processing Systems - Germany

Abstract. Efficient address code optimization is a central problem in code generation for processors with restricted addressing modes, like Digital Signal Processors (DSPs). This paper proposes a new heuristic to solve the Simple Offset Assignment (SOA) problem, the problem of allocating scalar variables to memory so as to minimize addressing code. This new approach, called Coalescing SOA (CSOA), performs variable memory slot coalescing simultaneously to offset assignment computation. Experimental results, based on compiling MediaBench benchmark programs with LANCE compiler, reveal a very significant improvement over the previous solutions to SOA. In fact, CSOA produces, on average, 37.3% fewer update instructions when comparing with the prior solution that perform memory slot coalescing before applying SOA, and 66.2% fewer update instructions when comparing with the best traditional SOA solution.

1 Introduction

The growth of the DSP market and the increasing demand for new and complex applications running on these processors have brought a strong interest to compilers capable of generating efficient DSP code. However, as DSPs have very irregular architectures, traditional compiling techniques designed for general-purpose processors [1, 21] are not capable of generating efficient code for DSPs [14]. As a result, new techniques tailored for these processors have been proposed and intensively studied. Due to their instruction size and performance constraints, DSPs traditionally have no offset addressing mode, containing only indirect addressing, and a few general-purpose registers. In addition, DSPs have specialized *Address Generation Units (AGU)*, that provide address computation in parallel to datapath computation. AGUs perform *auto-increment (decrement)* in address registers (AR) by some fixed values¹. For different values, the program is required to have an explicit *update instruction* (prior to the memory access) that uses datapath resources to compute the memory address. Therefore, in order to produce efficient code for such DSPs, it is important to use auto-increment (decrement) addressing modes effectively.

¹ Generally, the values of auto-increment (decrement) are one, but in some architectures these values can sometimes be larger.

The optimization that tries to maximize the use of instructions with auto-increment (decrement) for local scalar variables is called *Offset Assignment* (OA). This optimization finds a stack layout for these variables in such a manner that auto-increment (decrement) addressing modes are used whenever possible. The variation of the OA problem when there is only one address register and auto-increment (decrement) by 1 is called *Simple Offset Assignment (SOA)* [20] and is the focus of this paper.

In this paper we describe a new approach to the SOA optimization problem, called *Coalescing SOA (CSOA)*. It uses liveness information [1, 21] to simultaneously coalesce variable memory slots while solving SOA optimization. The interference graph [21] is used to identify which pairs of variables can be coalesced. Only variables that do not interfere² can be coalesced during CSOA. We show that variable coalescing can lead to a large improvement in code quality (66.2% fewer update instructions) when comparing to the best algorithm in OffsetStone [15, 22], and 37.3% fewer update instructions when comparing with the other coalescing approach described in [25]. This result dismisses the first assumptions to this problem, as in Liao [19], that seemed to indicate the opposite. Moreover, CSOA reduces both the code and the data segment, resulting in 92.3% of the number of memory slots when comparing with the results obtained through SOA-Liao.

The remainder of this paper is organized as follows. Section 2 exhibits an example that illustrates how the use of coalescing can affect the number of update instructions. Section 3 lists the previous work on SOA. Section 4 describes our technique (CSOA) and Section 5 shows the time-complexity of our method. Section 6 shows a small example that demonstrates the workings of the algorithm. In addition, Section 7 evaluates the results of CSOA, while Section 8 summarizes the main results.

2 Motivation

This section shows an example that illustrates how coalescing variables can decrease the number of update instructions. Consider that only a single address register is available in the processor, which can be auto-incremented (decremented) only by one.

Figure 1(a) shows a fragment of C code with the liveness information annotated at each program point. Figures 1(b) and (c) show two possible memory layouts for the variables, and the sequence in which the variables are accessed in memory. The arrows in Figures 1(b) and (c) indicate that an explicit address calculation instruction (i.e. update instruction) is required to make the address register point to the next variable, because the distance between the variables is greater than one. The layout showed in Figure 1(c) has one slot that is shared between two variables (*b* and *g*) which do not interfere at runtime. By sharing these variables, one less update instruction is required in the program. Clearly,

² Two variables interfere when they are simultaneously live.

coalescing variables increases the closeness between the variables on the stack, thus reducing the number of update instructions.

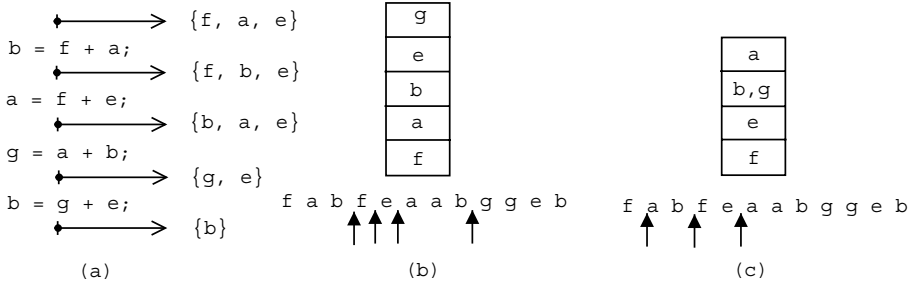


Fig. 1. (a) A fragment of C code. (b) Memory layout with one slot per variable. (c) Memory layout with more than one variable per slot

3 Related Work

The Simple Offset Assignment (SOA) problem was first studied by Bartley [5]. Later, Liao et al [20] showed that the graph problem *Maximum Weight Path Cover* (MWPC) (known to be NP-Complete) can be reduced to SOA, thus proving that SOA is NP-Hard. After that, a large number of heuristic techniques have been proposed for SOA [20, 17, 4, 15, 24], making it one of the most studied problems in code generation for DSPs.

Liao et al [20] used a heuristic to solve SOA based on the *Kruskal Minimum Spanning Tree* algorithm [12]. Given a basic block, Liao et al [20] call *access sequence* the sequence used by the program to access variables during execution time. For example, in instruction $a = b \text{ op } c$, the access sequence is bca . Based on the access sequence, Liao et al define an weighted graph $G(V, E)$, called *access graph*, where V is the set of variables in the basic block, and E is the set of edges. An edge $e = (u, v)$, with weight $w(e)$, indicates that there are $w(e)$ consecutive accesses to variables u and v (or v and u) in the access sequence. If two variables u and v are never accessed consecutively, then $(u, v) \notin E$. Once the access graph is constructed, Liao's algorithm tries to find a set of maximum weighted paths, called *assignment* that define the variable layout in memory. The *cost* of an assignment is the addition of the weights of all edges between variables in non-adjacent memory positions, as only auto-increment (decrement) by one is available. To illustrate these concepts consider Figure 2. Figure 2(a) shows a fragment of C code. Figure 2(b) shows the corresponding access sequence, and Figure 2(c) its associated access graph. Liao's heuristic is a greedy algorithm that, at each step, chooses the edge with the greatest weight, taking care not to select an edge that will stay with degree greater than 2 neither a edge that

can form a cycle with the already selected edges. By using this heuristic, the assignment selected in the access graph of Figure 2(c) would be **fecadgb**, as highlighted in that figure. This choice results in an offset cost of four, i.e. four update instructions are required, corresponding to the non-highlighted edges.

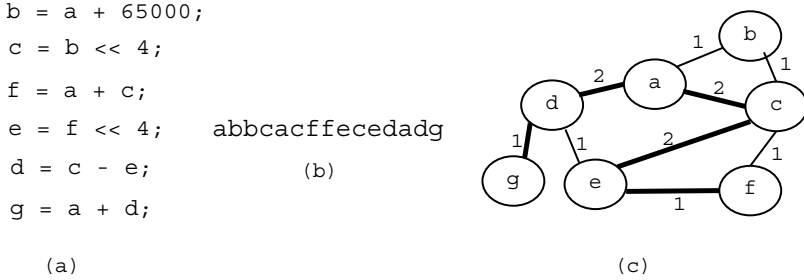


Fig. 2. (a) A fragment of C code. (b) The access sequence of this fragment. (c) The corresponding access graph

Sudarsanam et al [25] performed graph coloring to coalesce variables before SOA, but their goal was to reduce memory utilization, and they have not shown that this would improve the offset cost. In section 7 the results of this heuristic are showed and compared with the results of CSOA.

Leupers and Marwedel [18] proposed an extension to Liao’s heuristic, called tie-break, that decides what edge to choose when there are edges with the same weight. Rao and Pande [24] described a technique that considers the order of the accesses. This technique optimizes the access sequence through algebraic transformations in the expression tree. In [17], Leupers and David proposed a genetic algorithm to solve SOA. Instead of using the access sequence, they computed the offset assignment directly by a simulation of a natural evolution process.

Many generalizations of the SOA problem have been studied. One important generalization is the *General Offset Assignment (GOA)* problem [20, 18, 17], that is the offset assignment problem when more than one address register is available. In addition, generalizations that exploit the use of modify registers [26, 18, 17], auto-increment (decrement) ranges [25, 11], instruction scheduling coupled with offset assignment [6] and procedure-level offset assignment [9] were also studied.

Another problem related to SOA is the problem known as *Array Reference Allocation (ARA)*, which optimizes the access to array variables instead of scalar variables. This problem was originally studied by Araujo et al in [3], and later extended by other researchers [16, 23, 2, 7].

4 Coalescing Simple Offset Assignment

This section describes an optimization for offset assignment that is based on variable liveness information. Our approach, called *Coalescing Simple Offset Assignment (CSOA)*, receives as input the access sequence and the interference graph of the variables. Its output is an offset assignment for the variables in memory.

Our technique is an extension to most of the previous heuristics that solve SOA [5, 20, 18, 4, 15]. For the purpose of testing CSOA, we use the algorithm proposed by Liao et al [20] with the tie-break heuristic in [18] to decide between edges with the same weight. Liao et al try to form a maximum path in the access graph, sorting the edges of the access graph in decreasing order of their weights. After that, their algorithm iterates until all vertices are inserted onto the path or no other edge is available. At each step of the iteration, Liao et al choose the *valid edge*, (i.e. one not already selected, that does not cause a cycle, and does not increase the degree of a vertex on the path to more than two) with maximum weight.

Algorithm 1 presents pseudo-code for CSOA. At each iteration step, instead of always choosing an edge, as in typical SOA solutions, it considers another alternative: coalescing two vertices. Specifically, we do one of two operations: (a) coalesce two vertices u and v in the access graph, if they do not interfere; (b) pick a valid edge of maximum weight from the sorted list of edges, L in the Algorithm 1, as in Liao's approach.

In Algorithm 1, function *FindCandidatePair* tries to find the two candidates for coalescing. This function returns a quadruple $(coal, u, v, csave)$, where $coal$ is a flag that is set if there are two vertices u and v for coalescing, and $csave$ is the number of update instructions that are saved if u and v are coalesced.

In order to find the two candidates for coalescing, function *FindCandidatePair* (line (7)) searches among all possible combinations of two vertices u and v , in the interference graph, considering only the vertices that satisfy the following conditions:

1. $(u, v) \notin$ the interference graph;
2. Coalescing u and v does not create a cycle, considering only the selected edges;
3. Coalescing u and v , does not cause the coalesced vertex to have degree greater than two, considering only the selected edges.

Then, it picks, among all pairs of vertices that satisfy the above conditions, the pair u and v whose coalescing results in the highest $csave$.

To calculate $csave$, function *FindCandidatePair* computes the following statements, where $Adj_{sel}(y)$ is the set of vertices adjacent to y , considering only the already selected edges:

1. $\forall x \in (Adj_{sel}(u) - Adj_{sel}(v))$, add $w(x, v)$ to $csave$;
2. $\forall x \in (Adj_{sel}(v) - Adj_{sel}(u))$, add $w(x, u)$ to $csave$;
3. Add the weight of the edge between u and v , $w(u, v)$, to $csave$, if the edge was not selected yet.

Algorithm 1 Coalescing-Based SOAInput: the access sequence L_{AS} ,the interference graph $G_I(V_I, E_I)$.

Output: the offset assignment.

```

(1)  $G_A(V_A, E_A) \leftarrow \text{BuildAccessGraph}(L_{AS});$ 
(2)  $L = \text{sorted list of the } E_A;$ 
(3)  $coal \leftarrow \text{false};$ 
(4)  $sel \leftarrow \text{false};$ 
(5) repeat
(6)    $rebuild \leftarrow \text{false};$ 
(7)    $(coal, u, v, csave) \leftarrow \text{FindCandidatePair}(G_I, u, v);$ 
(8)    $sel \leftarrow \text{FindEdgeValidNotSel}(L, e);$ 
(9)   if  $(coal \ \&\& \ sel)$ 
(10)    if  $(csave \geq w(e))$ 
(11)      $rebuild \leftarrow \text{true};$ 
(12)    else
(13)     mark  $e$  as selected;
(14)  else
(15)    if  $(coal)$ 
(16)      $rebuild \leftarrow \text{true};$ 
(17)    else if  $(sel)$ 
(18)     mark  $e$  as selected;
(19)  if  $(rebuild)$ 
(20)     $\text{RebuildAccessGraph}(G_A, u, v);$ 
(21)     $\text{RebuildInterferenceGraph}(G_I, u, v);$ 
(22)     $\text{RebuildL}(L);$ 
(23) until  $(!(coal \ || \ sel))$ 
(24) return  $\text{BuildOffset}(G_A);$ 

```

For the sake of clarity, consider Figure 3. According to the statements above and Figure 3, the value of $csave$ when u and v are coalesced is the weight of edge (x, v) (since x is adjacent to u , edge (x, u) is selected, and edge (x, v) is not selected) plus the weight of the non-selected edge (u, v) . The value of $csave$ becomes 6, 4 from edge (u, v) and 2 from edge (x, v) .

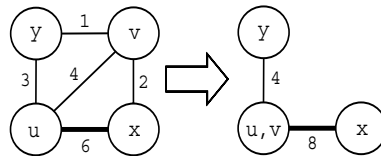


Fig. 3. (a) One access graph. (b) The access graph after coalescing variables u and v

After that, in line (8) of the Algorithm 1, function *FindEdgeValidNotSel* searches for the valid edge e with maximum weight $w(e)$ in the sorted list of edges L , and if it exists, flag *sel* is set.

Finally, if both *coal* and *sel* are true (line (9)), Algorithm 1 chooses (line (10)) the one that makes the best reduction in the number of update instructions.

When two vertices u and v are coalesced, parts of the access and the interference graphs need to be rebuilt in order to reflect the operation. This is performed in lines (19)-(22) of Algorithm 1. In the new access graph, all the old adjacencies of u and v must be redirected to the coalesced vertex (uv) . In the new interference graph, the coalesced vertex must interfere with all vertices that were adjacent to either u or v in the old interference graph.

Algorithm 1 uses function *RebuildL*, in line (22), to reconstruct the sorted list of edges (i.e. L) from the new access graph. Algorithm 1 ends when there are no more valid edges that can be chosen and no more vertices to coalesce. This condition is tested by using flags *sel* and *coal* in line (23) of Algorithm 1.

5 Complexity Analysis of CSOA

In this section, it is analyzed the time-complexity of CSOA in the worst case. In this analysis, consider m as the length of the access sequence, and n the number of variables considered for CSOA. In Algorithm 1, the complexity of *BuildAccessGraph* is $O(m + n^2)$. The sorting operation in line (2) takes $O(n^2 \log n)$. After this, the *repeat-until* loop can be executed at most $2(n - 1)$ times (at most $n - 1$ edges are selected and $n - 1$ coalescing operations are performed). The *repeat-until* loop is dominated by the *RebuildL* function, which is $O(n^2 \log n)$. So, this loop has complexity $O(n^3 \log n)$. Finally, the *BuildOffset* function takes $O(n^2)$ time. Therefore, CSOA has time-complexity $O(m + n^3 \log n)$.

It is worth bringing to attention that this is a worst case analysis, and that in practice one can expect a better runtime for CSOA.

6 Example of Coalescing SOA

To better illustrate CSOA, consider the code fragment of Figure 4(a). Each program point in the code shows the set of live variables (assuming that only g is live at the exit of the fragment). When Algorithm 1 is applied to this example, it receives as input the interference graph shown in Figure 4(b) and the access sequence (Figure 4(c)).

As the algorithm proceeds, it produces at each iteration the access graphs shown in Figures 4(d)-(j), after which it reaches the final memory assignment. The edges selected during the assignments are highlighted. The final memory layout is shown in Figure 4(k).

Although not illustrated, the reader should remember that, whenever two vertices in the access graph are coalesced, these vertices are also coalesced in the interference graph. In the first iteration (Figure 4(d)), edge (a, c) is selected,

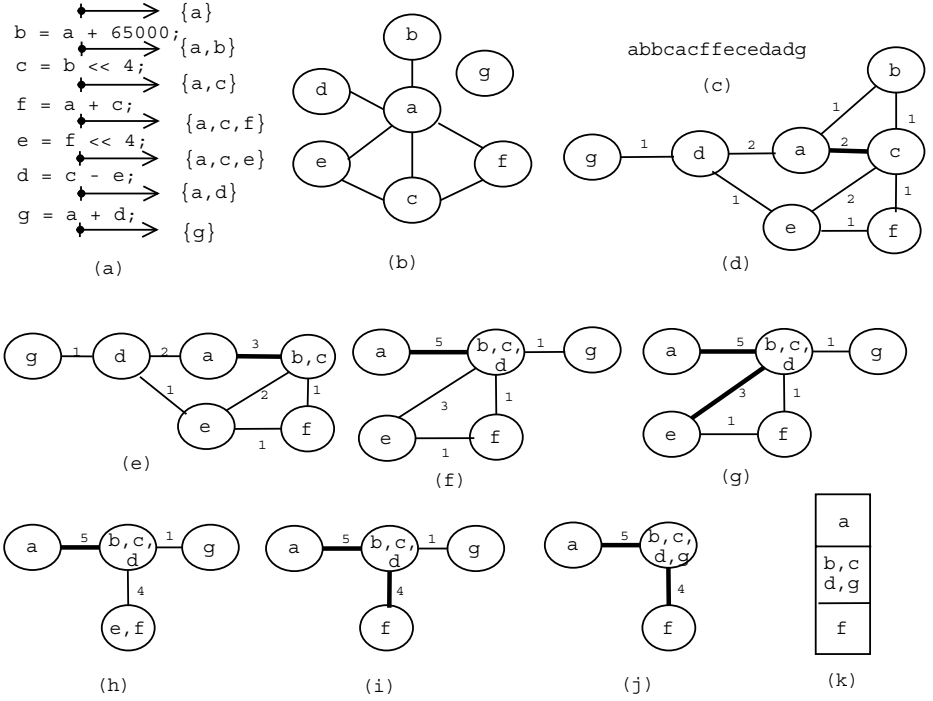


Fig. 4. (a) A fragment of C code with liveness information at each point. (b) The interference graph of the variables. (c) The access sequence of this fragment. (d)-(j) The access graphs resulting after each iteration of the algorithm. (k) The memory layout. Selected edges are highlighted

as no pair of vertices can be coalesced to produce saving as high as 2. In the next iteration, the best choice is to coalesce vertices b and c , given that this operation results in a saving of 2 (corresponding to the edges (a, b) and (b, c)). The new vertex (bc) becomes adjacent to the vertices that were adjacent to b or c in the previous access graph, that is, a , e and f . Notice that the weight of the edge between a and (bc) becomes 3, the summation of the weights of edges (a, b) and (a, c) in the previous graph. The algorithm proceeds, choosing between coalescing two vertices or selecting an edge, until no more operations are possible, thus resulting in Figure 4(j).

The final cost of applying CSOA to this example is zero, as all edges in the final access graph are selected. Notice that this example is the same as the one in Section 3, for which Liao's algorithm produces a final cost of four.

7 Experimental Results

In this section, we compare CSOA with four other approaches to SOA. We use the MediaBench benchmark [13] to evaluate the five heuristics.

We implemented our approach using OffsetStone [15, 22], a toolset used to test and evaluate OA algorithms. All benchmark programs were compiled with the Lance [8] compiler front-end, which translates the C source code into three-address code intermediate representation. The code in this intermediate representation was then optimized through a combination of the following optimizations: constant folding, constant propagation, jump optimization, loop invariant code motion, induction variable elimination, global common subexpression elimination, dead code elimination and copy propagation. Access sequences were then extracted from each basic block, and basic block access graphs merged on a function basis. The live ranges of the variables were calculated doing liveness analysis [1] in the intermediate representation after the optimizations described above.

In Table 1, we compare CSOA with four other approaches. We measured the percentage of the number of update instructions inserted by each method, with respect to the number of update instructions inserted by SOA-Liao, the algorithm described in [20]. The four other methods used in the comparison are: SOA-TB, the heuristic described in [18]; SOA-GA, the heuristic described in [17]; SOA-INC-TB [15], the combination of two SOA algorithms, SOA-incremental [4] and SOA-TB [18], and SOA-Color, the optimization described in [25]. The SOA-Color algorithm constructs the interference graph based on the live ranges, and then uses Kempe’s [10] coloring heuristic to coalesce variables that do not interfere. After this, it is applied SOA-Liao heuristic in the coalesced variables.

Table 1. Offset costs relative to Liao’s algorithm cost

Benchmarks	TB	GA	INC-TB	SOA-Color	CSOA
adpcm	89.1%	89.1%	89.1%	55.8%	45.6%
epic	96.8%	96.6%	96.6%	74.3%	50.2%
g721	96.2%	96.2%	96.2%	50.6%	27.9%
gsm	96.3%	96.3%	96.3%	26.6%	19.4%
jpeg	96.9%	96.7%	96.7%	52.6%	32.2%
mpeg2	97.3%	97.1%	97.2%	60.2%	34.3%
pegwit	91.1%	90.7%	90.7%	75.2%	38.8%
pgp	94.9%	94.8%	94.8%	55.0%	32.2%
rasta	98.6%	98.5%	98.5%	33.2%	21.1%
Average	95.2%	95.1%	95.1%	51.2%	32.1%

Notice from Table 1 that CSOA reduces, on average, the number of update instructions to 32.1% of the SOA-Liao cost. This is a significant improvement over the previous algorithms. The best of the other algorithms (SOA-Color) reduced the offset cost, on average, to 51.2% of the SOA-Liao cost. So, the difference between SOA-Color and CSOA, in relation of SOA-Liao, is 19%. This means that, in relation to SOA-Color, CSOA produces 37.3% fewer update instructions than this technique. We believe that this exceptional improvement is due to the fact that CSOA does not coalesce variables indiscriminately, but tries

to make adjacent in memory variables that have many consecutive accesses. This increases the closeness between variables that are accessed consecutively. CSOA, in opposition to other techniques that naively coalesce variable slots [25], wisely takes advantage of coalescing to reduce both the SOA cost and the memory requirement. This is achieved by simultaneously performing variable coalescing while solving SOA.

Table 2 lists, for each benchmark, the following measurements for SOA-Color and CSOA relative to SOA-Liao’s algorithm: the percentage of program code size, the percentage of data memory size, and the percentage of total memory size (code plus data). For SOA-Liao’s method Table 2 shows the number of memory words used with code, data and code plus data, which is represented by C+D in Table 2. In the percentage of data memory size, only statically allocated variables were considered. To estimate the number of instructions, each three-address IR instruction of the benchmarks was considered to be stored in one memory word. This way, the real effect of the two techniques in the memory size can be better analyzed, since both memory, data and code, are reduced. The data area is reduced by coalescing, and the code by minimizing the number of update instructions.

Table 2. Number of memory words of code, data and code plus data, using SOA-Liao algorithm, and percentage of memory savings relative to Liao’s algorithm when using SOA-Color and CSOA

Bench.	SOA-Liao			SOA-Color			CSOA		
	Code	Data	C+D	Code	Data	C+D	Code	Data	C+D
adpcm	601	29038	29639	89.9%	99.4%	99.2%	87.5%	99.5%	99.3%
epic	14541	137936	152477	92.0%	97.3%	96.8%	84.6%	97.8%	96.6%
g721	2786	2266	5052	90.7%	55.9%	75.1%	86.4%	61.9%	75.4%
gsm	13963	15882	29845	94.3%	71.8%	82.3%	93.7%	76.3%	84.4%
jpeg	65630	53748	119378	94.9%	78.3%	87.4%	92.7%	83.4%	88.5%
mpeg2	31354	148106	179460	92.8%	94.7%	94.4%	88.0%	95.9%	94.6%
pegwit	14685	85217	99902	97.4%	95.6%	95.9%	93.6%	96.9%	96.4%
pgp	618243	148204	766447	99.6%	94.4%	98.6%	99.4%	95.6%	98.7%
rasta	18691	4346642	4365333	84.4%	99.9%	99.8%	81.6%	99.9%	99.9%
Average	17002.0	73550.8	119257.4	92.8%	86.1%	91.8%	89.6%	88.3%	92.3%

From Table 2, one can observe that SOA-Color reduces the size of the memory used to store variables to 86.1%, when comparing to the other five methods [20, 18, 17, 15, 4] that do not perform coalescing, while our method reduces to 88.3%. On the other hand, our method reduces the size of the code memory to 89.6%, when comparing with the size of the memory code resulting of SOA-Liao, while SOA-Color reduces to 92.8%. Considering both memory segments, our method reduces memory to 92.3% and SOA-Color to 91.8% when comparing with total size of memory resulting from SOA-Liao. Though CSOA results in 0.5% more

memory area than SOA-Color, it produces 37.3% fewer update instructions, thus resulting in a better performance.

Finally, Table 3 shows the number of temporary variables (among those considered for SOA) in each program.³ Observe through these numbers that, on average, 64.1% of the variables are temporaries. Memory stored temporaries are very common in DSP architectures, given their reduced number of general-purpose registers. Thus, temporary allocation plays an important role in the final code performance, reinforcing our perception that there are many opportunities for CSOA to coalesce variables in DSP code, as shown by the experimental results.

Table 3. Percentage of temporary variables, considering as temporary a variable that is alive only in one basic block

Benchmarks	%Temporaries
adpcm	59.6%
epic	48.1%
g721	80.7%
gsm	86.6%
jpeg	65.2%
mpeg2	65.6%
pegwit	72.1%
pgp	67.5%
rasta	43.6%
Average	64.1%

Another result measured is that in 43.9% of the instances of all benchmarks, CSOA resulted in zero cost. So, for at least this percentage of the instances CSOA resulted in the optimal cost, and we believe this percentage to be significantly higher in fact, as many of the other instances may have an optimal cost greater than zero.

8 Conclusions and Future Work

In this paper we proposed a heuristic to solve the Simple Offset Assignment (SOA) problem based on coalescing memory variable slots. The experimental results show that our method (CSOA) eliminates, on average, 37.3% of the update instructions when comparing with the SOA-Color. Another important side effect of our technique is the reduction in the size of the memory layout to 92.3% when comparing with the SOA-Liao approach. The large presence of temporaries in DSP programs and the increased closeness resulting from the coalescing technique seem to explain well these exceptional numbers.

³ We consider here as temporaries those variables whose liveness are restricted to a single basic block.

In this paper, we only addressed the SOA problem. We are currently investigating the use of coalescing to partition the access graph in the case of the General Offset Assignment (GOA) problem.

Acknowledgments

This work was partially supported by FAPESP (2000/15083-9), and by fellowship grant FAPESP (01/12762-5). We also thank the reviewers for their comments.

References

- [1] A. V. Aho, R. Sethi, and J. D. Ullman. *Addressing Modes for Fast and Optimal Code Generation*. 1987.
- [2] Guido Araujo, Guilherme Ottoni, and Marcelo Cintra. Global array reference allocation. *ACM Trans. on Design Automation of Electronic Systems*, 7(2):336–357, April 2002.
- [3] Guido Araujo, Ashok Sudarsanam, and Sharad Malik. Instruction set design and optimizations for address computation in DSP architectures. In *Proc. of the 9th. ACM/IEEE International Symposium on System Synthesis*, pages 102–107, November 1996.
- [4] Sunil Atri, J. Ramanujam, and Mahmut Kandemir. Improving offset assignment for embedded processors. *Lecture Notes in Computer Science*, 2017, 2001.
- [5] David H. Bartley. Optimizing stack frame accesses for processors with restricted addressing modes. *Software - Practice and Experience*, 22(2):101–110, 1992.
- [6] Yoonseo Choi and Taewhan Kim. Address assignment combined with scheduling in DSP code generation. In *Proc. of the 39th Design Automation Conference, DAC 2002*, 2002.
- [7] Marcelo Cintra and Guido Araujo. Array reference allocation using ssaform and live range growth. In *Proc. of the ACM SIGPLAN 2000 LCTES*, pages 26–33, June 2000.
- [8] LANCE Retargetable C compiler.
<http://ls12-www.cs.uni-dortmund.de/lance/>.
- [9] Erik Eckstein and Andreas Krall. Minimizing cost of local variables access for DSP-processors. In *Proc. of the ACM SIGPLAN 1999 LCTES*, 1999.
- [10] A. Kempe. On the geographical problem of four colors. *Amer. J. Math*, 2, 1879.
- [11] Nakaba Kogure, Nobuhiko Sugino, and Akinori Nishihara. Memory address allocation method with ± 2 update operations in indirect addressing. In *European Conference on Circuit Theory and Design (ECCTD)*, 1997.
- [12] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7:48–50, 1956.
- [13] Chunho Lee, Miodrag Potkonjak, William H., and Mangione-Smith. Mediabench: A tool for evaluating and synthesizing multimedia and communications systems. In *Proc. of the 30th Annual International Symposium on Microarchitecture (Micro 30)*, December 1997.
- [14] Rainer Leupers. Code generation for embedded processors. In *International System Synthesis Symposium*, 2000.

- [15] Rainer Leupers. Offset assignment showdown: Evaluation of DSP address code optimization algorithms. In *Proceedings of the 12th International Conference on Compiler Construction*, April 2003.
- [16] Rainer Leupers, Anupam Basu, and Peter Marwedel. Optimized array index computation in DSP programs. In *Proc. of the Asia South Pacific Design Automation Conference (ASP-DAC)*. IEEE, February 1998.
- [17] Rainer Leupers and Fabian David. A uniform optimization technique for offset assignment problems. In *Proc. of the International Symposium on System Synthesis (ISSS)*, pages 3–8, 1998.
- [18] Rainer Leupers and Peter Marwedel. Algorithms for address assignment in DSP code generation. In *International Conference on Computer-Aided Design (ICCAD)*, pages 109–112, 1996.
- [19] Stan Liao. *Code Generation and Optimization for Embedded Digital Signal Processors*. Ph.D thesis, Massachusetts Institute of Technology, 1996.
- [20] Stan Liao, Srinivas Devadas, Kurt Keutzer, Steven Tjiang, and Albert Wang. Storage assignment to decrease code size. *ACM Transactions on Programming Languages and Systems*, 18(3):235–253, May 1996.
- [21] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [22] OffsetStone. <http://www.address-code-optimization.org>.
- [23] Guilherme Ottoni, Sandro Rigo, Guido Araujo, Subramanian Rajagopalan, and Sharad Malik. Optimal live range merge for address register allocation in embedded programs. In *Proceedings of the 10th International Conference on Compiler Construction, CC 2001, LNCS 2027*, pages 274–288. Springer, April 2001.
- [24] Amit Rao and Santosh Pande. Storage assignment optimizations to generate compact and efficient code on embedded DSPs. In *SIGPLAN Conference on Programming Language Design and Implementation*, pages 128–138, 1999.
- [25] Ashok Sudarsanam, Stan Liao, and Srinivas Devadas. Analysis and evaluation of address arithmetic capabilities in custom DSP architectures. In *Design Automation Conference*, pages 287–292, 1997.
- [26] Bernhard Wess and Martin Gotschlich. Optimal DSP memory layout generations a quadratic assignment problem. In *Int. Symp. on Circuits and Systems (ISCAS)*, 1997.

Transformation of Meta-information by Abstract Co-interpretation^{*}

Raimund Kirner and Peter Puschner

Institut für Technische Informatik
Technische Universität Wien
Treitlstraße 3/182/1, A-1040 Wien, Austria,
{raimund,peter}@vmars.tuwien.ac.at

Abstract. In this paper we present an approximation method based on abstract interpretation to transform meta-information in parallel with the transformation of concrete data. The meta-information is assumed to describe further properties of the specific data. The construction of a correct transformation function for the meta-information can be quite complicated in case of complex data transformations or data structures. A special approximation method is presented that works with data abstraction.

Performing worst-case execution time (WCET) analysis for optimized code is described as a concrete example for the application of this approach. A transformation framework is constructed to correctly update the flow information in case of code transformations.

1 Introduction

The parallel transformation of data and attached meta-information requires to construct a transformation function for the meta-information that maintains the semantics of the meta-information. The construction of such a function can be quite difficult. For example, if the data domain to be transformed represents programs or functions, there can be a sensible dependency between data and meta-information.

Abstract interpretation [1,2] is a useful technique to reduce the complexity in constructing safe approximations of given interpretations. The classic application of abstract interpretation is the formalization of an approximation correspondence between the concrete semantics and an abstract semantics for a program written in a given programming language. However, it has been already shown that abstract interpretation can be also applied to more general application areas. For example, abstract interpretation has been modeled for program transformers [3].

^{*} This work has been supported by the IST research project “High-Confidence Architecture for Distributed Control Applications (NEXT TTA)” under contract IST-2001-32111.

In this paper we use abstract interpretation in a special way to work on a tuple consisting of data and meta-information. The method we call *abstract co-interpretation* is intended for the development of appropriate update functions for the meta-information in case of a given data transformation.

The application of this technique is demonstrated for the support of WCET analysis by an optimizing compiler. In this context, data values are programs to be transformed by the compiler and meta-information is additional flow information required for the calculation of the WCET. As reported in [4], the development of an accurate flow information transformation function can be quite complex, even for simple transformations like branch optimization. The concept presented in this paper forms the foundation to construct such transformations of flow information for WCET analysis systematically.

This article is structured as follows: Section 2 briefly introduces the basic concepts of abstract interpretation. Section 3 introduces abstract co-interpretation as a framework to correctly transform meta-information in parallel to data transformation. A concrete example to demonstrate the application of abstract co-interpretation is given in section 4. Section 5 gives a conclusion to the article.

2 Basic Concepts of Abstract Interpretation

Abstract interpretation formalizes the correspondence between two semantics of a program with different approximation levels. That means, abstract interpretation allows to construct a safe approximation for a given concrete program semantics. The classical abstract interpretation framework was introduced in [1] as a tuple $\langle \langle \mathcal{D}, \sqsubseteq \rangle, F \rangle$ where $\langle \mathcal{D}, \sqsubseteq \rangle$ was assumed to be a complete lattice. This definition was generalized later (e.g., [3]) to require $\langle \mathcal{D}, \sqsubseteq \rangle$ only to be a partial ordered set (poset), with optional stronger properties.

The approximation of a given program semantics is called abstract semantics, while the original semantics is called concrete semantics. The concrete semantics is assumed to work on a domain $\mathcal{D}$ that is a poset $\langle \mathcal{D}, \sqsubseteq \rangle$ ordered by the approximation ordering $\sqsubseteq$. The relation $\sqsubseteq$ formalizes the loss of information, e.g., by using an interval to describe the possible value of a variable. The abstract semantics also works on a poset $\langle \tilde{\mathcal{D}}, \tilde{\sqsubseteq} \rangle$.

The correspondence between the concrete and the abstract semantics is given by a pair of maps: $\alpha : \mathcal{D} \rightarrow \tilde{\mathcal{D}}$ is the abstraction, and $\gamma : \tilde{\mathcal{D}} \rightarrow \mathcal{D}$ is the concretization. We have a sound approximation, if for all abstract representations $\tilde{d} \in \tilde{\mathcal{D}}$ of a concrete value $d \in \mathcal{D}$ it follows that $d \sqsubseteq \tilde{d}$. Further, if any element $d \in \mathcal{D}$ has a unique best approximation $\tilde{d} \in \tilde{\mathcal{D}}$ given by $\tilde{d} = \alpha(d)$, then the pair (α, γ) is a Galois connection, written as $\langle \langle \mathcal{D}, \sqsubseteq \rangle, \alpha, \gamma, \langle \tilde{\mathcal{D}}, \tilde{\sqsubseteq} \rangle \rangle$. The benefit of constructing a Galois connection is to have a safe approximating mapping between concrete and abstract domain: $\forall d \in \mathcal{D} \wedge \forall \tilde{d} \in \tilde{\mathcal{D}} : \alpha(\gamma(\tilde{d})) \sqsubseteq \tilde{d} \wedge d \sqsubseteq \gamma(\alpha(d))$. A Galois insertion is a Galois connection where γ is injective and a Galois isomorphism is a Galois connection where both α and γ are injective. A Galois isomorphism allows to map data in both directions between concrete and abstract domain without loss of information.

The semantic transition function $\tilde{F} : \tilde{\mathfrak{D}} \rightarrow \tilde{\mathfrak{D}}$ of the abstract semantics can be constructed from the concrete semantic transition function $F : \mathfrak{D} \rightarrow \mathfrak{D}$ as follows: $\tilde{F} = \alpha \circ F \circ \gamma$. The abstract semantics is said to be a safe γ -approximation if it fulfills: $\forall \tilde{d} \in \tilde{\mathfrak{D}} : F(\gamma(\tilde{d})) \sqsubseteq \gamma(\tilde{F}(\tilde{d}))$. Having a safe γ -approximation allows to show the correctness of the approximation by proving correctness of a single execution of each semantic transition.

3 Correct Transformation of Data and Meta-information

The transformation method described in this section applies to meta-information $d_2 \in \mathfrak{D}_2$ that describes additional properties of data $d_1 \in \mathfrak{D}_1$. In case that the data is transformed by the transition function $F_1 : \mathfrak{D}_1 \rightarrow \mathfrak{D}_1$, a transition function $F_2 : \mathfrak{D}_2 \rightarrow \mathfrak{D}_2$ has to be constructed that transforms the meta-information so that after the transformation it still describes valid data properties.

We assume that the transition function F_2 for the meta-information $\mathfrak{D}_2$ can be directly obtained from the operations performed by the data transformation function F_1 , denoted as $F_2 = \text{impl}(F_1/\mathfrak{D}_2)$. The direct construction of a correct meta-information transformation function F_2 can become infeasible complicated in case that the operations performed by F_1 are quite complex or the binding between the data and the meta-information is very sensible. A sensible binding between data and meta-information can arise in the case that the data domain is quite complex, for example a data item itself represents functions like $\mathfrak{D}_1 : \mathcal{A} \rightarrow \mathcal{B}$ (with a corresponding semantic transition function $F_1 : (\mathcal{A} \rightarrow \mathcal{B}) \rightarrow (\mathcal{A} \rightarrow \mathcal{B})$).

To reduce the complexity of constructing a correct transformation function for the meta-information, we abstract from the concrete data to a representation that is more close to that of the meta-information. Inducing an adequate transition function for the abstracted data it becomes possible to calculate a transformation function for the meta-information.

First of all we have to define the concrete domains $\langle \mathfrak{D}_1, \sqsubseteq_1 \rangle$ and $\langle \mathfrak{D}_2, \sqsubseteq_2 \rangle$ for the data and its meta-information. At next, based on these concrete domains we construct suitable abstract domains $\langle \tilde{\mathfrak{D}}_1, \sqsubseteq_1 \rangle$ and $\langle \tilde{\mathfrak{D}}_2, \sqsubseteq_2 \rangle$. To avoid unnecessary accuracy reduction due to the approximation, it is intended to construct the structure of $\langle \tilde{\mathfrak{D}}_2, \sqsubseteq_2 \rangle$ as close as possible to that of $\langle \mathfrak{D}_2, \sqsubseteq_2 \rangle$. Based on the concrete and abstract domains we construct two Galois connections $\langle \langle \mathfrak{D}_1, \sqsubseteq_1 \rangle, \alpha_1, \gamma_1, \langle \tilde{\mathfrak{D}}_1, \sqsubseteq_1 \rangle \rangle$ and $\langle \langle \mathfrak{D}_2, \sqsubseteq_2 \rangle, \alpha_2, \gamma_2, \langle \tilde{\mathfrak{D}}_2, \sqsubseteq_2 \rangle \rangle$ (as motivated above, it is intended to design the Galois connection $\langle \langle \mathfrak{D}_2, \sqsubseteq_2 \rangle, \alpha_2, \gamma_2, \langle \tilde{\mathfrak{D}}_2, \sqsubseteq_2 \rangle \rangle$ to be a Galois isomorphism).

Based on these two Galois connections, the so-called *independent attribute method* as described in [10] can be applied to construct a Galois connection for the combined domains $\mathfrak{D} : \mathfrak{D}_1 \times \mathfrak{D}_2$ and $\tilde{\mathfrak{D}} : \tilde{\mathfrak{D}}_1 \times \tilde{\mathfrak{D}}_2$. The pair of maps for the resulting Galois connection $\langle \langle \mathfrak{D}, \sqsubseteq \rangle, \alpha, \gamma, \langle \tilde{\mathfrak{D}}, \sqsubseteq \rangle \rangle$ is defined as $\alpha = \alpha_1 \times \alpha_2$ and $\gamma = \gamma_1 \times \gamma_2$. The relation between the resulting concrete interpretation $\langle \langle \mathfrak{D}, \sqsubseteq \rangle, F \rangle$ and the abstract interpretation $\langle \langle \tilde{\mathfrak{D}}, \sqsubseteq \rangle, F \rangle$ with $F = F_1 \times F_2$ and $\tilde{F} = \tilde{F}_1 \times \tilde{F}_2$ is shown in figure 1. A Galois connection that has been designed using the *independent attribute method* allows to perform separate abstraction

$$\begin{array}{ccc}
\langle d_1, d_2 \rangle & \xrightarrow{F} & \langle d'_1, d'_2 \rangle \\
\mathcal{S}[\![\langle d_1, d_2 \rangle]\!] & & \mathcal{S}[\![\langle d'_1, d'_2 \rangle]\!] \\
\alpha_1 \downarrow \uparrow \gamma_1 & & \alpha_1 \downarrow \uparrow \gamma_1 \\
& \alpha_2 \downarrow \uparrow \gamma_2 & \\
\langle \tilde{d}_1, \tilde{d}_2 \rangle & \xrightarrow{\tilde{F}} & \langle \tilde{d}'_1, \tilde{d}'_2 \rangle \\
\tilde{\mathcal{S}}[\![\langle \tilde{d}_1, \tilde{d}_2 \rangle]\!] & & \tilde{\mathcal{S}}[\![\langle \tilde{d}'_1, \tilde{d}'_2 \rangle]\!]
\end{array}$$

Fig. 1. Transformation of Data and Meta-information

and concretizations of its components. Therefore, the construction of a sound abstract transition function $\tilde{F}$ can be done by fulfilling equ. 1.

$$\begin{aligned}
\forall \langle \tilde{d}_1, \tilde{d}_2 \rangle \in \tilde{\mathfrak{D}}_1 \times \tilde{\mathfrak{D}}_2 : & F_1(\gamma_1(\tilde{d}_1)) \sqsubseteq_1 \gamma_1(\tilde{F}_1(\tilde{d}_1)) \wedge \\
& F_2(\gamma_2(\tilde{d}_2)) \sqsubseteq_2 \gamma_2(\tilde{F}_2(\tilde{d}_2))
\end{aligned} \tag{1}$$

In figure 1, the semantics of the concrete domain $\langle \mathfrak{D}_1 \times \mathfrak{D}_2, \sqsubseteq \rangle$ and the abstract domain $\langle \tilde{\mathfrak{D}}_1 \times \tilde{\mathfrak{D}}_2, \sqsubseteq \rangle$ are denoted by $\mathcal{S}[\![\langle d_1, d_2 \rangle]\!]$ and $\tilde{\mathcal{S}}[\![\langle \tilde{d}_1, \tilde{d}_2 \rangle]\!]$. $\mathcal{S}[\![\langle d_1, d_2 \rangle]\!]$ represents the *extended semantics* (def. 1) for the concrete data . Analogous, $\tilde{\mathcal{S}}[\![\langle \tilde{d}_1, \tilde{d}_2 \rangle]\!]$ is the corresponding abstract extended semantics. The extended semantics can be seen as a metric to verify whether the meta-information attached to a data value is correct.

Definition 1. (Extended Semantics $\mathcal{S}[\![\langle d_1, d_2 \rangle]\!]$) represents the semantics of data $d_1 \in \mathfrak{D}_1$ under consideration of the meta-information $d_2 \in \mathfrak{D}_2$. $\mathcal{S}[\![\langle d_1, d_2 \rangle]\!]$ is the standard semantics $\mathcal{S}[d]$ with the additional constraint that d_1 fulfills the properties described by d_2 . If d_1 cannot fulfill these properties, then d_1 are invalid data in respect of the given meta-information d_2 .

Definition 2. (Abstract Co-interpretation)

Assumptions:

- $\langle \langle \mathfrak{D}_1, \sqsubseteq_1 \rangle, \alpha_1, \gamma_1, \langle \tilde{\mathfrak{D}}_1, \sqsubseteq_1 \rangle \rangle$ and $\langle \langle \mathfrak{D}_2, \sqsubseteq_2 \rangle, \alpha_2, \gamma_2, \langle \tilde{\mathfrak{D}}_2, \sqsubseteq_2 \rangle \rangle$ are two Galois connections with independent attributes and the Galois connection $\langle \langle \mathfrak{D}_1 \times \mathfrak{D}_2, \sqsubseteq \rangle, \alpha_1 \times \alpha_2, \gamma_1 \times \gamma_2, \langle \tilde{\mathfrak{D}}_1 \times \tilde{\mathfrak{D}}_2, \sqsubseteq \rangle \rangle$ has been constructed based on the independent attribute method [10].
- $\langle \langle \tilde{\mathfrak{D}}_1, \sqsubseteq_1 \rangle, \tilde{F}_1 \rangle$ is a safe γ_1 -approximation of $\langle \langle \mathfrak{D}_1, \sqsubseteq_1 \rangle, F_1 \rangle$ and $\langle \langle \tilde{\mathfrak{D}}_1 \times \tilde{\mathfrak{D}}_2, \sqsubseteq \rangle, \tilde{F}_1 \times \tilde{F}_2 \rangle$ is a safe $\gamma_1 \times \gamma_2$ -approximation of $\langle \langle \mathfrak{D}_1 \times \mathfrak{D}_2, \sqsubseteq \rangle, F_1 \times F_2 \rangle$.

A definition of a function $\tilde{F}_2 : \tilde{\mathfrak{D}}_2 \rightarrow \tilde{\mathfrak{D}}_2$ that can be implied from the transformation performed by $\tilde{F}_1 : \tilde{\mathfrak{D}}_1 \rightarrow \tilde{\mathfrak{D}}_1$ is denoted as $\tilde{F}_2 = \text{impl}(\tilde{F}_1/\tilde{\mathfrak{D}}_2)$. If the

implied function $\tilde{F}_2 = \text{impl}(\tilde{F}_1/\tilde{\mathcal{D}}_2)$ fulfills the following condition

$$\forall \langle \tilde{d}_1, \tilde{d}_2 \rangle \in \tilde{\mathcal{D}}_1 \times \tilde{\mathcal{D}}_2 : F_2(\gamma_2(\tilde{d}_2)) \sqsubseteq_2 \gamma_2(\tilde{F}_2(\tilde{d}_2))$$

then $\langle \langle \mathcal{D}_1 \times \mathcal{D}_2, \sqsubseteq \rangle, F_1 \times \gamma_2 \circ \text{impl}(\tilde{F}_1/\tilde{\mathcal{D}}_2) \circ \alpha_2 \rangle$ is a safe approximation of $\langle \langle \mathcal{D}_1 \times \mathcal{D}_2, \sqsubseteq \rangle, F_1 \times F_2 \rangle$.

The approximation $\langle \langle \mathcal{D}_1 \times \mathcal{D}_2, \sqsubseteq \rangle, F_1 \times \gamma_2 \circ \text{impl}(\tilde{F}_1/\tilde{\mathcal{D}}_2) \circ \alpha_2 \rangle$ is called an *abstract co-interpretation*.

Analogous to the concrete transition function F_2 , the abstract transition $\tilde{F}_2$ for the meta-information can be directly calculated from the operations performed by the abstract data transformation function $\tilde{F}_1$, denoted as

$$\tilde{F}_2 = \text{impl}(\tilde{F}_2/\tilde{F}_1)$$

Based on this implication of $\tilde{F}_2$ we use a novel interpretation method – called *abstract co-interpretation* (as described in def. 2) – to calculate a correct approximation for F_2 . The novel aspect of *abstract co-interpretation* is that one component ($\mathcal{D}_1$) of the composed domain is interpreted both for the concrete and the abstract domain to reduce the complexity of calculating a safe approximation of the function F_2 . This approximation based on *abstract co-interpretation* is safe if it fulfills equ. 2. The resulting approximating interpretation is written as $\langle \langle \mathcal{D}_1 \times \mathcal{D}_2, \sqsubseteq \rangle, F_1 \times \gamma_2 \circ \text{impl}(\tilde{F}_1/\tilde{\mathcal{D}}_2) \circ \alpha_2 \rangle$.

$$\forall \langle d_1, d_2 \rangle \in \mathcal{D}_1 \times \mathcal{D}_2 : \langle F_1(d_1), F_2(d_2) \rangle \sqsubseteq \langle F_1(d_1), \gamma_2(\tilde{F}_2(\alpha_2(d_2))) \rangle \quad (2)$$

To summarize, *abstract co-interpretation* can be used for data with attached meta-information describing additional data properties to simplify the calculation of a correct meta-information transformation function for a given data transformation function. Using an abstraction of the concrete data close to the representation level of the meta-information, the implication of the transfer function for the meta-information is simplified, since only the essential properties of the data transformation function are considered.

4 Example: WCET Analysis Support in Optimizing Compilers

This section describes an example for the application of *abstract co-interpretation*, the integration of support for worst-case execution time (WCET) analysis into an optimizing compiler.

4.1 Introduction to WCET Analysis

The knowledge of the WCET is mandatory to guarantee the timeliness of hard real-time systems. An overview about research in WCET analysis can be found

in [11]. This section only presents some aspects of WCET analysis that will help to demonstrate the approach of *abstract co-interpretation*.

To calculate the WCET of a program, additional information about the possible control flow of the code is necessary. This flow information is often called *flow facts*. Due to undecidability it is not possible to automatically extract all flow facts that are necessary to calculate the WCET from the program. Additional information given by the user is required. It is preferable to provide such annotations at the source code level [8]. For most accurate results it is required to perform WCET at the object code level. It is necessary to transform the flow facts in parallel to any code transformations performed by a compiler [7]. A framework to fully support code transformations for WCET analysis is described in [6].

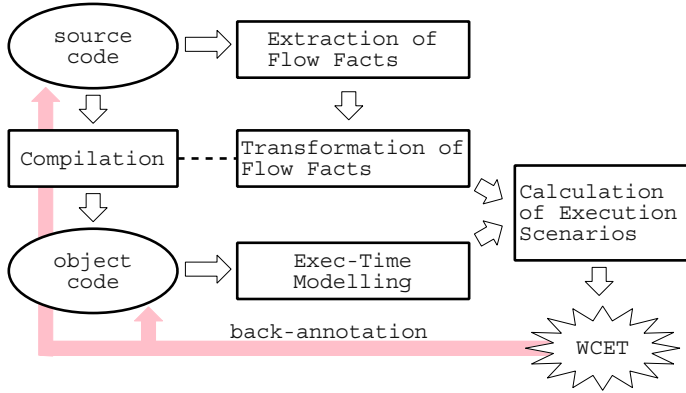


Fig. 2. Generic WCET Analysis Framework

The context of the flow facts within a generic WCET analysis framework is shown in figure 2. The flow facts have to be transformed in parallel to any code transformations performed by the compiler. Afterwards, methods like integer linear programming can be used to calculate the WCET [12]. The program $\mathcal{P}$ to be transformed by the compiler corresponds to the concrete data $d \in \mathcal{D}$ of the previous section. The flow facts $\mathcal{ff}$ are the meta-information attached to the data. The flow facts $\mathcal{ff}$ describe a closure for the possible control flow paths (*CFP*) of a program $\mathcal{P}$. The possible *CFP* of a program $\mathcal{P}$ is denoted as $CFP_{opt}(\mathcal{P})$, the closure described by the flow facts $\mathcal{ff}$ is denoted as $CFP_{\mathcal{ff}}(\mathcal{P})$.

4.2 Correct Transformation of Flow Facts

$\mathcal{P} \in \mathbb{P}$ represents the program to be transformed by the transformation function $F_{t1} : \mathbb{P} \rightarrow \mathbb{P}$. To enable the calculation of a WCET bound for $\mathcal{P}$, additional flow facts $\mathcal{ff} \in \mathbb{F}$ are assigned to $\mathcal{P}$. The flow facts of $\mathbb{F}$ form a domain $\langle \mathbb{F}, \sqsubseteq_2 \rangle$ where $\sqsubseteq_2$ is defined as $\mathcal{ff} \sqsubseteq_2 \mathcal{ff}' \Leftrightarrow (\mathcal{ff}' \text{ is a less restrictive subset of } \mathcal{ff})$. The exact

$$\begin{array}{ccccccc}
\langle \mathcal{P}_s, \epsilon \rangle & \xrightarrow{a} & \langle \mathcal{P}_s, \text{ff}_a \rangle & = a[\![\mathcal{P}_s]\!] & \xrightarrow{F_s} & \langle \mathcal{P}_i, \text{ff} \rangle & \xrightarrow{F_t} & \langle \mathcal{P}_t, \text{ff}_t \rangle \\
\mathcal{S}[\![\langle \mathcal{P}_s, \epsilon \rangle]\!] & & \mathcal{S}[\![\langle \mathcal{P}_s, \text{ff}_a \rangle]\!] & & & \mathcal{S}[\![\langle \mathcal{P}_i, \text{ff} \rangle]\!] & & \mathcal{S}[\![\langle \mathcal{P}_t, \text{ff}_t \rangle]\!] \\
\downarrow \alpha_o & & \downarrow \alpha_o & & & \downarrow \alpha_o & & \downarrow \alpha_o \\
\alpha_o(\langle \mathcal{P}_s, \epsilon \rangle) & = & \alpha_o(\langle \mathcal{P}_s, \text{ff}_a \rangle) & = & \alpha_o(\langle \mathcal{P}_i, \text{ff} \rangle) & = & \alpha_o(\langle \mathcal{P}_t, \text{ff}_t \rangle)
\end{array}$$

Fig. 3. Observational Correctness of Transformation

definition of $\sqsubseteq_2$ depends on the concrete type of supported flow facts. Intuitively spoken, for each element $f' \in \text{ff}'$ there exists an element $f \in \text{ff}$ where f' is less restrictive than f .

To describe the correct $\mathbb{F}$ transformation, $\mathbb{P}$ and $\mathbb{F}$ will be grouped together to form the domain $\langle \mathfrak{D}, \sqsubseteq \rangle$ with $\mathfrak{D} : \mathbb{P} \times \mathbb{F}$, having a combined transformation function $F_t = F_{t1} \times F_{t2}$. The relation $\sqsubseteq$ is defined as

$$\forall \langle \mathcal{P}, \text{ff} \rangle, \langle \mathcal{P}', \text{ff}' \rangle \in \mathfrak{D} : \langle \mathcal{P}, \text{ff} \rangle \sqsubseteq \langle \mathcal{P}', \text{ff}' \rangle \Leftrightarrow (\mathcal{P} = \mathcal{P}') \wedge (\text{ff} \sqsubseteq_2 \text{ff}')$$

It remains to construct a correct $\mathbb{F}$ transformation function $F_{t2} : \mathbb{F} \rightarrow \mathbb{F}$ to complete the definition of $F_t : \mathfrak{D} \rightarrow \mathfrak{D}$.

The correctness of the transformation is proven by showing observational equivalence [3]: an abstraction function α_o is used to extract the relevant properties for correctness. An example prepared for our needs is given in figure 3 for the transformation of flow facts in parallel to the code transformation. As already mentioned, the calculation of flow facts cannot be complete. Therefore, certain flow facts ff_a are given manually by the user (denoted by the operation a). Further flow information ff_{impl} is extracted by semantic code analysis denoted by the operation F_s . The resulting flow information is denoted $\text{ff} = \text{ff}_a \cup \text{ff}_{impl}$. Finally, the operation

$$F_t = F_{t1} \times F_{t2}$$

represents the code optimization performed by the compiler and the flow facts transformation performed in parallel. The correctness condition shown in figure 3 requires that the observational abstraction α_o has an unchanged semantics $\mathcal{S}[\![\langle \mathcal{P}, \text{ff} \rangle]\!]$ for both code annotation and transformation (def. 3).

$\mathcal{P}_i$ is the program which has been annotated with flow facts. $\mathcal{P}_i$ is transformed by the compiler into $\mathcal{P}_t$. ff has to be transformed into ff_t in parallel with the transformation of $\mathcal{P}_i$. Conventional WCET analysis tools will use $\mathcal{P}_t$ and ff_t as input to calculate the WCET.

Definition 3. (Extended Program Semantics $\mathcal{S}[\![\langle \mathcal{P}, \text{ff} \rangle]\!]$) represents the semantics of program $\mathcal{P}$ under consideration of the flow facts ff . The CFP described by ff for a program $\mathcal{P}$ is denoted as $\text{CFP}_{\text{ff}}(\mathcal{P})$. $\mathcal{S}[\![\langle \mathcal{P}, \text{ff} \rangle]\!]$ is the stan-

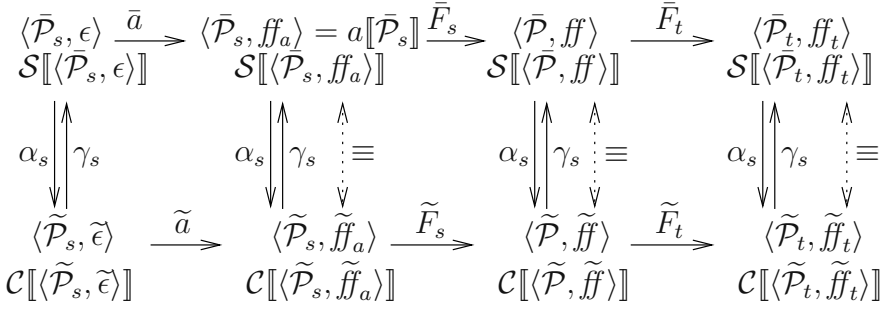


Fig. 4. Transformation of Flow Facts

standard program semantics $\mathcal{S}[\mathcal{P}]$ with the additional constraint that the possible $CFP_{opt}(\mathcal{P})$ of $\mathcal{P}$ is a subset of $CFP_{ff}(\mathcal{P})$. If the $CFP_{opt}(\mathcal{P})$ of $\mathcal{P}$ is not a subset of $CFP_{ff}(\mathcal{P})$ then $\mathcal{P}$ is an invalid program in respect of the given flow facts ff .

Definition 4. (Observational Correctness of $F : \mathbb{P} \times \mathbb{F} \rightarrow \mathbb{P} \times \mathbb{F}$) A transformation $F : \mathbb{P} \times \mathbb{F} \rightarrow \mathbb{P} \times \mathbb{F}$ is defined to be correct for a given input tuple $\langle \mathcal{P}, ff \rangle \in \mathbb{P} \times \mathbb{F}$, iff the standard program semantics $\mathcal{S}[\mathcal{P}]$ is not changed by the transformation F and $\langle \mathcal{P}, ff \rangle$ as well as $F(\langle \mathcal{P}, ff \rangle)$ are valid programs with respect to their extended program semantics (def. 3).

A formal definition of observational correctness for F is:

$$\begin{aligned}
\langle \mathcal{P}', ff' \rangle = F(\langle \mathcal{P}, ff \rangle) \wedge CFP_{opt}(\mathcal{P}) \subseteq CFP_{ff}(\mathcal{P}) \wedge \\
\mathcal{S}[\mathcal{P}] = \mathcal{S}[\mathcal{P}'] \wedge CFP_{opt}(\mathcal{P}') \subseteq CFP_{ff'}(\mathcal{P}')
\end{aligned}$$

To conclude, the correctness of the example given in figure 3 requires that the observational correctness (def. 4) holds for the transformations a , F_s , and F_t . The transformation $F_{t2} : \mathbb{F} \rightarrow \mathbb{F}$ has to be defined correctly so that the observational correctness of $F_t = F_{t1} \times F_{t2}$ is guaranteed.

4.3 Construction of a Flow Facts Transformation Framework

Based on the code annotation and transformation shown in figure 3 we perform an abstract interpretation with control-flow path abstraction to correctly transform the flow facts in parallel to the code transformation F_{t1} . The extraction of flow facts ff_{impl} from the source code is not topic of our work. There exists work like [5] tackling this problem.

The concept of our method based on the theory of abstract interpretation to construct a correct ff transformation function $F_{t2} : \mathbb{F} \rightarrow \mathbb{F}$ is shown in figure 4. The flow facts ff describe a closure for the possible CFP of a program $\mathcal{P}$. An abstract interpretation that operates on the structure of a program $\mathcal{P}$ is

appropriate to induce a correct update function of the flow facts. The meaning of $\mathcal{S}[\langle \bar{\mathcal{P}}, \bar{ff} \rangle]$ and $\mathcal{C}[\langle \tilde{\mathcal{P}}, \tilde{ff} \rangle]$ is explained after the construction of the concrete and abstract domains.

Construction of Concrete and Abstract Domains

The abstract interpretation operating on the program structure requires to abstract from the concrete program transformation. The abstraction can be done independently for the $\mathbb{P}$ and $\mathbb{F}$ attributes. We therefore use the *independent attribute method* to construct a Galois connection out of two separate Galois connections.

The construction of the Galois connection for the flow facts is trivial. Since the flow facts are already at a representation level that describes the control flow of a program, their abstraction can be constructed by a Galois isomorphism: $\langle \langle \mathbb{F}, \sqsubseteq_2 \rangle, \alpha_\sqsubseteq, \gamma_\sqsubseteq, \langle \tilde{\mathbb{F}}, \tilde{\sqsubseteq}_2 \rangle \rangle$. The advantage of a Galois isomorphism compared to a Galois connection is that data is converted between abstract and concrete domain without loss of information.

The construction of the Galois connection to abstract the program representation $\mathbb{P}$ requires more considerations. The five steps described in [10] to construct a Galois connection are:

1. *Construction of a concrete domain $\langle \bar{\mathbb{P}}, \bar{\sqsubseteq}_1 \rangle$* : It is intended to use a program abstraction based on the structure of a program $\mathcal{P} \in \mathbb{P}$. The program structure will contain information like control-flow and loop scopes. Constructing an appropriate partial order for a simple concrete domain like $\langle \mathbb{P}, \sqsubseteq_1 \rangle$ is not possible because the concretization from a code structure cannot be mapped to a single program $\mathcal{P} \in \mathbb{F}$ due to information loss by the abstraction. The solution is to lift the programs $\mathcal{P} \in \mathbb{P}$ to sets of programs $\bar{\mathcal{P}} \in \bar{\mathbb{P}}$ with $\bar{\mathbb{P}} : \wp(\mathbb{P})$ and the additional restriction that all programs $\mathcal{P} \in \bar{\mathcal{P}}$ have the same code structure, denoted by $\forall \mathcal{P}_1 \in \bar{\mathcal{P}}_1, \forall \mathcal{P}_2 \in \bar{\mathcal{P}}_2 : (\bar{\mathcal{P}}_1 = \bar{\mathcal{P}}_2) \rightarrow (struct(\mathcal{P}_1) = struct(\mathcal{P}_2))$. The partial order $\langle \bar{\mathbb{P}}, \bar{\sqsubseteq}_1 \rangle$ can be now defined as:

$$\forall \bar{\mathcal{P}}_1, \bar{\mathcal{P}}_2 \in \bar{\mathbb{P}} : \bar{\mathcal{P}}_1 \bar{\sqsubseteq}_1 \bar{\mathcal{P}}_2 \Leftrightarrow \bar{\mathcal{P}}_1 \subseteq \bar{\mathcal{P}}_2$$

2. *Construction of the corresponding abstract domain $\langle \tilde{\mathbb{P}}, \tilde{\sqsubseteq}_1 \rangle$* : The abstract program domain $\langle \tilde{\mathbb{P}}, \tilde{\sqsubseteq}_1 \rangle$ is designed to represent the unique code structure of a program set $\bar{\mathcal{P}} \in \bar{\mathbb{P}}$ which is calculated by the function $struct : \mathbb{P} \rightarrow \tilde{\mathbb{P}}$. The domain $\langle \tilde{\mathbb{P}}, \tilde{\sqsubseteq}_1 \rangle$ is a “flat poset”: $\forall \tilde{\mathcal{P}}_1, \tilde{\mathcal{P}}_2 \in \tilde{\mathbb{P}} : \tilde{\mathcal{P}}_1 \tilde{\sqsubseteq}_1 \tilde{\mathcal{P}}_2 \Leftrightarrow \tilde{\mathcal{P}}_1 = \tilde{\mathcal{P}}_2$.
3. *Correctness relation $\mathcal{R}_s$* : The correctness relation $\mathcal{R}_s : \bar{\mathbb{P}} \times \tilde{\mathbb{P}} \rightarrow \{true, false\}$ is defined as $\bar{\mathcal{P}} \mathcal{R}_s \tilde{\mathcal{P}} \Leftrightarrow (\forall \mathcal{P} \in \bar{\mathcal{P}} : struct(\mathcal{P}) \tilde{\sqsubseteq}_1 \tilde{\mathcal{P}})$. Since each $\mathcal{P} \in \bar{\mathcal{P}}$ has the same program structure $struct(\mathcal{P})$, the resulting representation function $\beta_s : \bar{\mathbb{P}} \rightarrow \tilde{\mathbb{P}}$ is calculated as follows: $\forall \bar{\mathcal{P}} \in \bar{\mathbb{P}} : (\bar{\mathcal{P}} \in \bar{\mathbb{P}}) \Rightarrow (\beta_s(\bar{\mathcal{P}}) = struct(\mathcal{P}))$.
4. *Check for the existence of a best approximation*: Because the domain $\langle \tilde{\mathbb{P}}, \tilde{\sqsubseteq}_1 \rangle$ is designed as a “flat poset” (there exists a unique abstract property that represents a concrete property), it directly follows that $\forall \bar{\mathcal{P}} \in \bar{\mathbb{P}}, \forall \tilde{\mathcal{P}} \in \tilde{\mathbb{P}}, \exists \tilde{\mathcal{P}}_1 \in \tilde{\mathbb{P}} : \bar{\mathcal{P}} \mathcal{R}_s \tilde{\mathcal{P}}_1 \wedge (\bar{\mathcal{P}} \mathcal{R}_s \tilde{\mathcal{P}} \Rightarrow \tilde{\mathcal{P}}_1 \tilde{\sqsubseteq} \tilde{\mathcal{P}})$.

5. *Calculation of the abstraction function α_s and the concretization function γ_s* : The abstraction function $\alpha_s : \bar{\mathbb{P}} \rightarrow \bar{\mathbb{P}}$ is calculated as follows: $\forall \bar{\mathcal{P}} \in \bar{\mathbb{P}} : \alpha_s(\bar{\mathcal{P}}) = \beta_s(\bar{\mathcal{P}})$. The concretization function $\gamma_s : \bar{\mathbb{P}} \rightarrow \bar{\mathbb{P}}$ calculates the set of all programs that match the given program structure: $\gamma_s(\bar{\mathcal{P}}) = \sqcup \{ \tilde{\mathcal{P}} \in \bar{\mathbb{P}} \mid \beta(\tilde{\mathcal{P}}) \sqsubseteq_1 \bar{\mathcal{P}} \}$. It is important to note that $\gamma_s(\bar{\mathcal{P}})$ cannot be calculated in practice, since it results in a set of infinite programs. However, the calculation of $\gamma_s(\bar{\mathcal{P}})$ is not required as we use *abstract co-interpretation* (def. 2) to induce $\bar{F}_{t2}$.

It is interesting to note that the above defined Galois connection $\langle \langle \bar{\mathbb{P}}, \sqsubseteq_1 \rangle, \alpha_s, \gamma_s, \langle \tilde{\mathbb{P}}, \sqsubseteq_1 \rangle \rangle$ also forms a Galois insertion. The Galois insertion $\langle \langle \bar{\mathbb{P}}, \sqsubseteq_1 \rangle, \alpha_s, \gamma_s, \langle \tilde{\mathbb{P}}, \sqsubseteq_1 \rangle \rangle$ and the Galois isomorphism $\langle \langle \mathbb{F}, \sqsubseteq_2 \rangle, \alpha_{\equiv}, \gamma_{\equiv}, \langle \tilde{\mathbb{F}}, \sqsubseteq_2 \rangle \rangle$ are combined using the *independent attribute method* to construct the new Galois insertion $\langle \langle \bar{\mathbb{P}} \times \mathbb{F}, \sqsubseteq \rangle, \alpha_s \times \alpha_{\equiv}, \gamma_s \times \gamma_{\equiv}, \langle \tilde{\mathbb{P}} \times \tilde{\mathbb{F}}, \sqsubseteq \rangle \rangle$.

The Semantics of the Concrete and Abstract Domains

In figure 4, the semantics of the concrete domain $\langle \bar{\mathbb{P}} \times \mathbb{F}, \sqsubseteq \rangle$ and the abstract domain $\langle \tilde{\mathbb{P}} \times \tilde{\mathbb{F}}, \sqsubseteq \rangle$ are denoted by $\mathcal{S}[\langle \bar{\mathcal{P}}, \tilde{ff} \rangle]$ and $\mathcal{C}[\langle \tilde{\mathcal{P}}, \tilde{ff} \rangle]$.

$\mathcal{S}[\langle \bar{\mathcal{P}}, \tilde{ff} \rangle]$ represents the *extended program semantics* (def. 3) for all programs $\mathcal{P} \in \bar{\mathcal{P}}$. Since we use a special interpretation – which we call *abstract co-interpretation* – we do not need to calculate the concretization function $\gamma_s : \tilde{\mathbb{P}} \rightarrow \bar{\mathbb{P}}$. As a consequence, the program set $\bar{\mathcal{P}}$ of $\langle \bar{\mathcal{P}}, \tilde{ff} \rangle$ contains only a single program which has to be valid in terms of the *extended program semantics*.

The abstract semantics $\mathcal{C}[\langle \tilde{\mathcal{P}}, \tilde{ff} \rangle]$ describes $CFP_{\tilde{ff}}(\mathcal{P})$, a closure for the possible control flow paths $CFP_{opt}(\mathcal{P})$ during the execution of a program $\mathcal{P} \in \bar{\mathcal{P}}$. The code structure information of $\langle \tilde{\mathcal{P}}, \tilde{ff} \rangle$ may contain for example the control-flow graph (CFG) and information about loop scopes.

Construction of a Safe Approximation to Calculate F_t

Based on the concrete domain $\langle \bar{\mathbb{P}}, \sqsubseteq_1 \rangle$ and the program transformation function $F_{t1} : \bar{\mathbb{P}} \rightarrow \bar{\mathbb{P}}$ we can construct an interpretation $\langle \langle \bar{\mathbb{P}}, \sqsubseteq_1 \rangle, \bar{F}_{t1} \rangle$ using the following transition function:

$$\forall \bar{\mathcal{P}} \in \bar{\mathbb{P}} : \bar{F}_{t1}(\bar{\mathcal{P}}) = \{ F_{t1}(\mathcal{P}) \mid (\mathcal{P} \in \bar{\mathcal{P}}) \wedge \text{defined}(F_{t1}(\mathcal{P})) \}$$

The constraint $\text{defined}(F_{t1}(\mathcal{P}))$ is given since F_{t1} is not a total function over all programs $\mathcal{P}$ of the set $\bar{\mathcal{P}}$ of programs with the same code structure. If $F_{t1}(\mathcal{P})$ is defined, then F_{t1} is also defined for the result of $F_{t1}(\mathcal{P})$. Therefore, the use of $\text{defined}()$ is only necessary for formal completeness regarding the Galois insertion since by applying *abstract co-interpretation* we never use the concretization function γ_s for the calculation of F_t .

The concrete interpretation $\langle \langle \bar{\mathbb{P}} \times \mathbb{F}, \sqsubseteq \rangle, \bar{F}_t \rangle$ of the concrete transformation of programs with attached flow facts has the following transition function $\bar{F}_t : \bar{\mathbb{P}} \times \mathbb{F} \rightarrow \bar{\mathbb{P}} \times \mathbb{F}$:

$$\bar{F}_t = \bar{F}_{t1} \times F_{t2}$$

To calculate F_{t2} we construct $\langle \langle \tilde{\mathbb{P}} \times \tilde{\mathbb{F}}, \sqsubseteq \rangle, \tilde{F}_t \rangle$ with the transition function $\tilde{F}_t : \tilde{\mathbb{P}} \times \tilde{\mathbb{F}} \rightarrow \tilde{\mathbb{P}} \times \tilde{\mathbb{F}}$:

$$\tilde{F}_t = \tilde{F}_{t1} \times \tilde{F}_{t2}$$

as a safe $\gamma_s \times \gamma_{\equiv}$ – *approximation* of $\langle \langle \tilde{\mathbb{P}} \times \tilde{\mathbb{F}}, \sqsubseteq \rangle, \tilde{F}_t \rangle$. The construction of a sound operation $\tilde{F}_t$ is done by fulfilling equ. 3. $\tilde{F}_{t1}$ is the abstraction of $\bar{F}_{t1}$ by transforming a program’s code structure.

$$\begin{aligned} \forall \langle \tilde{\mathcal{P}}, \tilde{f} \rangle \in \tilde{\mathbb{P}} \times \tilde{\mathbb{F}} : \bar{F}_{t1}(\gamma_s(\tilde{\mathcal{P}})) \sqsubseteq_1 \gamma_s(\tilde{F}_{t1}(\tilde{\mathcal{P}})) \wedge \\ F_{t2}(\gamma_{\equiv}(\tilde{f})) \sqsubseteq_2 \gamma_{\equiv}(\tilde{F}_{t2}(\tilde{f})) \end{aligned} \quad (3)$$

The flow facts transformation function $\tilde{F}_{t2}$ can be directly calculated from $\tilde{F}_{t1}$. $\tilde{F}_{t1}$ describes the structural program transformation including semantic information about the transformation describing the update of the program’s control flow. An example for such a control-flow update information is the information known by the compiler for the update of the iteration bound of the modified loop when performing the code transformation *loop unrolling* [9]. The information about the structural program transformation of $\tilde{F}_{t1}$ is sufficient to describe the transformation of the flow facts done by $\tilde{F}_{t2}$.

Since $\langle \langle \mathbb{F}, \sqsubseteq_2 \rangle, \alpha_{\equiv}, \gamma_{\equiv}, \langle \tilde{\mathbb{F}}, \sqsubseteq_2 \rangle \rangle$ is a Galois isomorphism (i.e., $\tilde{f} = \gamma_{\equiv}(\alpha_{\equiv}(\tilde{f}))$ and $\tilde{f} = \alpha_{\equiv}(\gamma_{\equiv}(\tilde{f}))$) we can use *abstract co-interpretation* (as defined in def. 2) to construct $\langle \langle \tilde{\mathbb{P}} \times \tilde{\mathbb{F}}, \sqsubseteq \rangle, \tilde{F}_{t1} \times \gamma_{\equiv} \circ \text{impl}(\tilde{F}_{t1}/\tilde{\mathbb{F}}) \circ \alpha_{\equiv} \rangle$ as an approximation of $\langle \langle \tilde{\mathbb{P}} \times \tilde{\mathbb{F}}, \sqsubseteq \rangle, \tilde{F}_t \rangle$. This approximation is safe, because equ. 4 follows from the definition of the *abstract co-interpretation*. Further, as $\langle \langle \mathbb{F}, \sqsubseteq_2 \rangle, \alpha_{\equiv}, \gamma_{\equiv}, \langle \tilde{\mathbb{F}}, \sqsubseteq_2 \rangle \rangle$ is a Galois isomorphism it follows that even equ. 5 holds and therefore that this approximation is also precise: $F_{t2} = \gamma_{\equiv} \circ \text{impl}(\tilde{F}_{t1}/\tilde{\mathbb{F}}) \circ \alpha_{\equiv}$.

$$\forall \langle \tilde{\mathcal{P}}, \tilde{f} \rangle \in \tilde{\mathbb{P}} \times \tilde{\mathbb{F}} : \langle \bar{F}_{t1}(\tilde{\mathcal{P}}), F_{t2}(\tilde{f}) \rangle \sqsubseteq \langle \bar{F}_{t1}(\tilde{\mathcal{P}}), \gamma_{\equiv}(\tilde{F}_{t2}(\alpha_{\equiv}(\tilde{f}))) \rangle \quad (4)$$

$$\forall \langle \tilde{\mathcal{P}}, \tilde{f} \rangle \in \tilde{\mathbb{P}} \times \tilde{\mathbb{F}} : \langle \bar{F}_{t1}(\tilde{\mathcal{P}}), F_{t2}(\tilde{f}) \rangle = \langle \bar{F}_{t1}(\tilde{\mathcal{P}}), \gamma_{\equiv}(\tilde{F}_{t2}(\alpha_{\equiv}(\tilde{f}))) \rangle \quad (5)$$

This section has shown an application for *abstract co-interpretation* to transform meta-information by $F_{t2} : \mathbb{F} \rightarrow \mathbb{F}$ in parallel to the transformation of a program $\mathcal{P} \in \mathbb{P}$. The transformations that have to be performed by F_{t2} are discussed in more detail in [8,6].

In general, program transformation performed by a compiler is a complex domain. Especially, supporting WCET analysis for code optimizations by compilers as listed in [9] is a complicated task. By abstracting code transformations to their impact on the code structure, it is possible to construct a correct transformation mechanism for the flow facts.

4.4 A Concrete Flow Facts Transformation Framework

The generic steps to construct a flow facts transformation framework by using *abstract co-interpretation* are described in section 4.3.

This section briefly describes a concrete flow facts transformation framework. A more detailed description of this framework is given in [6]. First, the flow facts that are utilized by the concrete WCET calculation method are described. Afterwards, the basic components for a $\tilde{ff}$ transformation function $\tilde{F}_{t_2}$ are introduced and a code transformation example shows their usage.

Description of Flow Facts

Flow facts describe the possible control-flow path of a program. The diversity of applicable types of flow facts depends on the concrete WCET calculation method. We use the implicit path enumeration technique (IPET) which is described by Puschner and Schedl in [12]. To calculate the WCET by IPET the structure of a program's CFG is translated into a set of graph flow constraints. The WCET is the maximized sum of the execution time of each node multiplied by its iteration frequency. To calculate the WCET it is necessary to search for a maximized solution over the iteration frequency variables that still fulfills the graph flow constraints. The WCET bound is calculated by a standard constraint solver.

To calculate the WCET, beside the structural graph flow constraints, additional information about the iteration bound of each loop is necessary. We represent the iteration bound of a loop by the tuple $L_x \langle l_0, u_0 \rangle$ where L_x is a unique loop identifier and l_0, u_0 are the lower respective upper iteration bounds of the loop. These loop bounds can be translated into further graph flow constraints of the IPET.

The consideration of (in)feasible paths improves the accuracy of the calculated WCET bound. IPET also allows to specify arbitrary constraints to describe (in)feasible paths. Such constraints are (un)equations of sums of *restriction terms*. A *restriction term* is a tuple $\langle n_0 \cdot mN_iN_j[t_0] \rangle$ where $mN_iN_j[t_0]$ represents a variable for the iteration count of the control-flow edge $\langle N_i, N_j \rangle$. $mN_iN_j[t_0]$ is also called a *marker binding* to the control-flow edge $\langle N_i, N_j \rangle$ with key t_0 . The key t_0 is used to distinguish multiple control-flow edges between the same nodes. The constant n_0 specifies the relative execution count of this control flow edge compared to the edges in the other restriction terms of the constraint.

Specification of Induced Flow Facts Transformation

The flow facts described above can be derived by semantic code analysis or given explicitly by code annotations. The flow facts have to be updated in parallel to every code transformation performed by the compiler that changes the control flow of the code.

To perform the update of the flow facts we developed three basic $\tilde{ff}$ transitions which are described in detail in [6]. Complex code transformations are modeled by grouping these transitions. All transitions within a group are applied in parallel. The following lists these three basic $\tilde{ff}$ transitions:

Update of marker bindings ($\xrightarrow{M}$): The induced update of *marker bindings* is given by a transition sequence of the following form:

$$mN_iN_j[t] \xrightarrow{M} \{mN_kN_l[t_1], mN_mN_n[t_2], \dots\}$$

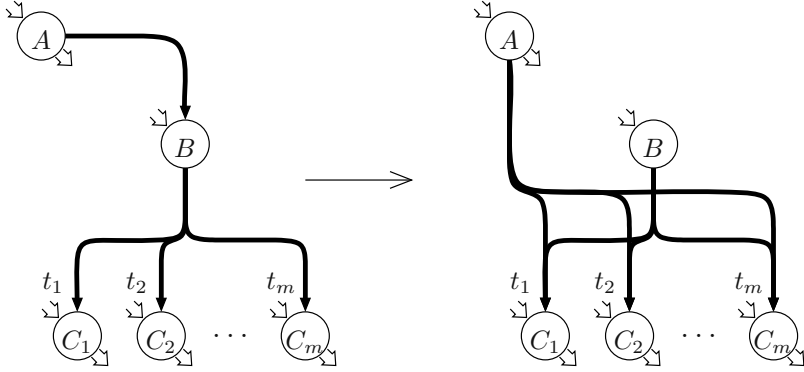


Fig. 5. CFG Transformation on Branch Optimization

The semantics of this transition is to remove the marker binding $mN_iN_j[t]$ and instead create the marker bindings $\{mN_kN_l[t_1], mN_mN_n[t_2], \dots\}$.

Update of restrictions ($\xrightarrow{R}$): The induced update of *restriction terms* is given by a transition sequence of the following form:

$$\langle n_0 \cdot mN_iN_j[t] \rangle \xrightarrow{R} \{ \langle n_1 \cdot mN_kN_l[t_1] \rangle, \langle n_2 \cdot mN_mN_n[t_2] \rangle, \dots \}$$

The semantics of this transition is to replace the term $\langle n \cdot mN_iN_j[t] \rangle$ in the left and right side of all restrictions by the list of terms $\{ \langle n_1 \cdot mN_kN_l[t_1] \rangle, \langle n_2 \cdot mN_mN_n[t_2] \rangle, \dots \}$.

Update of Loop Flow Facts ($\xrightarrow{L}$): The induced update of loop flow facts is given by a transition sequence of the following form:

$$L_x \langle l_0, u_0 \rangle \xrightarrow{L} \{ L_y \langle l_1, u_1 \rangle, L_z \langle l_2, u_2 \rangle, \dots \}$$

The semantics of this transition is to remove the old loop information $L_x \langle l_0, u_0 \rangle$ and instead create the new loop information $\{ L_y \langle l_1, u_1 \rangle, L_z \langle l_2, u_2 \rangle, \dots \}$.

Besides these three transitions only additional operations for creating new restrictions and for grouping the transitions are required to complete the $\tilde{ff}$ transformation framework.

Modeling a Concrete Code Optimization

This subsection gives an example for inducing $\tilde{F}_{t_2}$ in the case that F_{t_1} performs the code transformation *branch optimization*.

The CFG transformation for branch optimization is shown in figure 5. The abstract program transformation function $\tilde{F}_{t_1}$ describes the structural CFG

transformation together with the flow distribution caused by the specific code optimization.

Using $\tilde{F}_{t1}$, the following set of transitions is induced for the flow facts transformation function $\tilde{F}_{t1}$:

$$\begin{array}{l}
mBC_1[t_1] \xrightarrow{M} mBC_1[t_1], mAC_1[t_1] \\
\vdots \quad \quad \quad \vdots \\
mBC_m[t_m] \xrightarrow{M} mBC_m[t_m], mAC_m[t_m] \\
mAB[b] \xrightarrow{M} mAC_1[t_1], mAC_2[t_2], \dots, mAC_m[t_m] \\
\langle n \cdot mBC_1[t_1] \rangle \xrightarrow{R} \langle n \cdot mBC_1[t_1] - n \cdot mAC_1[t_1] \rangle \\
\langle n \cdot mBC_2[t_2] \rangle \xrightarrow{R} \langle n \cdot mBC_2[t_2] - n \cdot mAC_2[t_2] \rangle \\
\vdots \quad \quad \quad \vdots \\
\langle n \cdot mBC_m[t_m] \rangle \xrightarrow{R} \langle n \cdot mBC_m[t_m] - n \cdot mAC_m[t_m] \rangle \\
\langle n \cdot mAB[b] \rangle \xrightarrow{R} \langle n \cdot mAC_1[t_1] + n \cdot mAC_2[t_2] + \dots + n \cdot mAC_m[t_m] \rangle
\end{array}$$

For inducing $\tilde{F}_{t2}$ only the structural CFG transformations and the flow distributions caused by *branch optimization* are exploited. Other code transformation details are not relevant, therefore it is possible to use a single $\tilde{ff}$ transformation function for all possible combinations of conditional/unconditional branches. Such a simplification by using *abstract co-interpretation* to construct correct flow facts transformations is helpful for all code transformations.

5 Summary and Conclusion

Abstract interpretation is a universal formalism applicable to various interpretation scenarios. It allows to construct safe approximations by just examining local interpretation steps.

In this paper we presented a special application for abstract interpretation. We restricted the domain of the interpretation to consist of data and attached additional meta-information. The challenge was to construct a correct update of the meta-information for a given data transformation function. We introduced the notion of *extended semantics* to refer to valid data with respect of its attached meta-information. A further specialization we made was the assumption that the transformation function for the meta-information can be calculated from the data transformation function. The developed interpretation method has been named *abstract co-interpretation*, because it performs for the data transformation both, concrete and abstract interpretation. This is done to simplify the calculation of a suitable transformation for the meta-information.

Abstract co-interpretation is suitable to various applications where meta-information has to be transformed in parallel to data. However, often problems are

relatively simple, so that this approach may not be necessary. But in the given example of flow facts transformation for WCET analysis *abstract co-interpretation* significantly reduces the overall complexity by dividing the construction of flow facts update function into two phases with reduced complexity.

This approximation method has been developed to add support for WCET analysis into an optimizing compiler. The construction of an adequate update of flow information has been simplified significantly by abstraction of the performed program transformations. This paper presents the formal foundation for the flow facts transformation framework described in [6].

References

1. Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Conference Record of the Fourth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 238–252, Los Angeles, California, 1977. ACM Press, New York, NY.
2. Patrick Cousot and Radhia Cousot. Systematic design of program analysis frameworks. In *Conference Record of the 6th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 269–282, San Antonio, Texas, 1979.
3. Patrick Cousot and Radhia Cousot. Systematic design of program transformation frameworks by abstract interpretation. In *Conference Record of the Twentyninth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 178–190, Portland, Oregon, Jan. 2002. ACM Press, New York.
4. Jakob Engblom, Andreas Ermedahl, and Peter Altenbernd. Facilitating Worst-Case Execution Time Analysis for Optimized Code. In *Proc. 10th Euromicro Real-Time Workshop*, Berlin, Germany, June 1998.
5. Jan Gustafsson. *Analysing Execution-Time of Object-Oriented Programs Using Abstract Interpretation*. PhD thesis, Uppsala University, Uppsala, Sweden, May 2000.
6. Raimund Kirner. *Extending Optimising Compilation to Support Worst-Case Execution Time Analysis*. PhD thesis, Technische Universität Wien, Treitlstr. 3/3/182-1, 1040 Vienna, Austria, May 2003.
7. Raimund Kirner and Peter Puschner. Transformation of Path Information for WCET Analysis during Compilation. In *Proc. 13th IEEE Euromicro Conference on Real-Time Systems*, pages 29–36, Delft, The Netherlands, June 2001. Technical University of Delft.
8. Raimund Kirner and Peter Puschner. Timing analysis of optimised code. In *Proc. 8th IEEE International Workshop on Object-oriented Real-time Dependable Systems (WORDS 2003)*, Guadalajara, Mexico, January 2003.
9. Steven S. Muchnick. *Advanced Compiler Design & Implementation*. Morgan Kaufmann Publishers, Inc., 1997. ISBN 1-55860-320-4.
10. Flemming Nielson, Hanne R. Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, 1999. ISBN: 3-540-65410-0.
11. Peter Puschner and Alan Burns. A Review of Worst-Case Execution-Time Analysis. *Journal of Real-Time Systems*, 18(2/3):115–128, May 2000.
12. Peter Puschner and Anton V. Schedl. Computing Maximum Task Execution Times – A Graph-Based Approach. *The Journal of Real-Time Systems*, 13:67–91, 1997.

Performance Analysis for Identification of (Sub-)Task-Level Parallelism in Java

Richard Stahl*, Robert Paško, Luc Rijnders, Diederik Verkest**,
Serge Vernalde, Rudy Lauwereins***, and Francky Catthoor†

IMEC vzw, Kapeldreef 75, B-3001 Leuven, Belgium
`richard.stahl@imec.be`

Abstract. In the era of future embedded systems the designer is confronted with multiple processors both for performance and energy reasons. Exploiting (sub-)task-level parallelism is crucial when targeting those multi-processor systems, because ILP on itself is not sufficient. The challenge is to build compiler tools which automatically explore potential (sub-)task parallelism in the programs, and allow designer to optimise it for the underlying architecture. To achieve this goal we are building a transformation framework which employs task-level analysis and code transformations to extract the parallelism from sequential object-oriented programs.

Parallel performance analysis is one of the crucial techniques for estimation of the transformation effects and their optimisation. We have implemented support for performance analysis and profiling of Java programs. The toolkit comprises automated instrumentation, parallel profiling and post-processing analysis. We demonstrate its usability on three realistic applications.

1 Introduction

The future embedded systems confront the designer with multi-processor architectures which have performance and energy-consumption constraints. Those constraints bring new challenges to extraction of the parallelism and optimal mapping to the underlying processing units. For multi-processor systems, in particular, two inseparable challenges exist. First, the parallel tasks have to be identified and extracted. Second, in the optimal case, a very good match should exist between the tasks and the architecture resources, and this both for the "node functionality" and especially for their dependencies. Any mismatch at critical parts of the application will result in performance loss, an decrease of the resource utilisation and a reduced energy efficiency of the whole system.

From the designer point of view, in general three prospective approaches exist to solve those challenges: manual, automated and tool supported. In the first

* Also PhD student at Katholieke Universiteit Leuven.

** Also professor at Katholieke Universiteit Leuven and Vrije Universiteit Brussel.

*** Also professor at Katholieke Universiteit Leuven.

† Also professor at Katholieke Universiteit Leuven.

case, the designer manually translates the sequential or partly parallel program into an optimised parallel one with respect to the system constraints. This approach can lead to the most optimal solution yet it requires a considerable effort and the solution is dedicated to a specific problem. In the second case, the designer has a fully automated tool which is the ultimate goal of all the research in parallelising compilers. It leads to the easiest solution for the designer yet usually not the optimal one. The third approach enables the designer to use a number of analysis and transformation tools. He or she can interactively transform and optimise the program for the platform so the manual effort can be considerably reduced. We consider the development of those tools as an important intermediate step towards more automated parallelism extraction and optimisation for embedded systems. Even when fully automated tools would exist, then they will still be complemented with interactive tools in an industrial environment.

We propose a transformation framework for extraction and optimisation of task and subtask-level parallelism from sequential object-oriented programs with respect to architectural and energy-consumption constraints. Such sequential OO programs are becoming the most common form of code that is produced for embedded multi-media applications today (in C++ or Java) and it is not expected that this will change soon again in an industrial context. So a pure data-flow programming style or other paradigms which make the parallelism extraction and analysis much simpler, are not an option for the mid term. As a basis for our framework we have implemented parallel profiling and performance analysis for Java programs. The tools help designers to understand the behaviour of the sequential or (partly) parallel program, to find the bottlenecks in execution, and to provide the designer with concise interpretation of the analysis results. It automatically instruments the code with respect to designer input constraints, profiles the program, and interprets the profiling information. Thus, it can considerably improve program understandability. This information can be used to steer the process of extraction and optimisation of parallelism.

We believe that this approach is also applicable for performance analysis and optimisation of programs for embedded processors with support for multi-threading and hyper-threading paradigms. Both homogeneous and heterogeneous multi-processor targets can benefit from it. For the homogeneous case, also data parallelisation is clearly crucial, but that is complementary to the focus of this paper.

The remainder of this paper is organised as follows. Section 2 describes different approaches to task-level parallelism extraction and gives concise overview of related work. It gives an overview of our approach and its distinguishing features with respect to the related work. Section 3 describes the parallel performance analysis approach we implemented; Section 4 presents the experimental evaluation of our tools, and finally, Section 5 gives concluding remarks.

2 Parallelism Extraction and Related Work

The challenge in extraction of potential parallelism from sequential OO programs is first in the identification of the program segments which can run con-

currently and second in finding of an optimal mapping of the parallel tasks for given embedded platform. In automatic parallelism extraction most of the research has focused on scientific applications written in Fortran or C. Paraphrase-II [1], Paradigm [2], Promis [3] and pTask [4] are the most advanced (task-level) parallelisation frameworks. Their automatic task-level parallelism extraction is based on control and data-dependence analysis. It is best suited for large-scale array-processing scientific applications. However, current applications, written in object-oriented programming languages, are more irregular and often pointer-based. Those features are not supported in the tools mentioned. Moreover the tools do not take into account the energy-consumption constraints of embedded systems.

zJava [5], a follow-up project for pTask, extracts task-level parallelism also from pointer-based programs written in Java. However, it assigns every method a separate thread so the final tasks are very irregular with respect to the execution time. The task synchronisation and management is fully done by the run-time system. Thus, it can result in unoptimal task management and high energy consumption of the platform.

When targeting energy-aware multi-processor platforms we need to minimise the run-time task-management overhead while trading off the performance and energy of parallel execution. Thus, we propose a transformation framework for semi-automatic extraction and optimisation of task-level parallelism from object-oriented Java programs. The framework is based on extensive profiling and performance analysis and subsequent program transformation. Program slicing [6,7,22] is the basic method for introducing parallel code. Instrumentation then translates the original program into a new parallel form. Nevertheless, we first need to find the dominant parts of the program so we propose to use parallel profiling and performance analysis tools.

De facto, the standard profiling tool for UNIX systems is the GNU Profiler - GProf [11]. However, it uses statistical sampling which result in information loss, and it does not distinguish between the busy waiting and productive work. Hollingsworth and Miller [12] compare different performance analysis tools for parallel systems, e.g., critical-path analysis [13], Quartz NPT [14]. We have adopted their critical-path analysis for the postprocessing phase of our work.

In the object-oriented domain, Shende et al. [16] have presented a profiling tool which supports C++ source code linking with the profile information. From this work we have borrowed the concept of selective profiling.

Java Virtual Machine Profiler Interface (JVMPI) [17] is the standard profiler interface for Java yet it provides only limited functionality. On the other hand the JVMPI is used in a number of commercial tools [18,19,20].

Kazi et al. [21] have implemented the Javiz - low-overhead profiler for client/-server distributed Java applications. They do not use JVMPI yet they have adapted the JVM implementation to gather all necessary information. We use a similar approach to their run-time building of an execution tree. This simplifies the post-processing analysis.

Sevitsky et al. [22] present a framework for performance analysis of Java programs. Their focus is on the post-processing phase of the analysis. They have

defined a concept of execution slices which allows selective analysis of the profile data.

All the above mentioned tools can produce parallel profile information only when used on real multi-processor architectures. However, for the single-processor case the profile does not reflect any parallelism and its run-time effects. Thus, in that case the parallel performance analysis is almost impossible. That is a severe limitation because designers like to analyse and optimise their code first on a host station, which is not the final target and that is usually single processor. In addition, for hyper-threading on a single processor the subtask parallelism analysis and extraction is also crucial as a preprocessing phase.

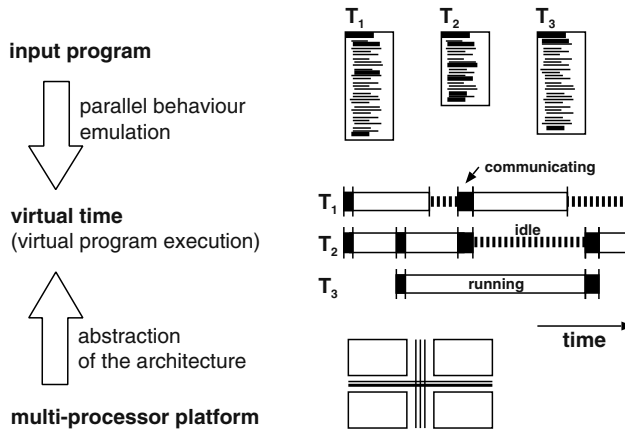


Fig. 1. Concept of virtual time. The virtual time allows emulation of the parallel program execution, communication and idleness of its tasks. Moreover, it can abstract characteristic features of the underlying platform

Proposed Parallel Performance Analysis. The performance analysis tool, we propose, is based on a concept of virtual time (Figure 1). The virtual time is used to emulate parallel execution of program tasks while the program is actually executed on the underlying platform, which does not need to be the final target platform. It allows to reason about the parallelism and communication effects between the tasks with respect to the physical parallelism of the target platform. Thus, it allows to perform quantitative analysis of a certain parallel program and later interpret those data.

The virtual time is used to abstract specific architectural features of the target platform. By means of virtual time we define three platform optimisation criteria - task-creation overhead, balanced task granularity and communication overhead (Figure 2).

- Task-creation overhead is represented as the ratio between the task creation interval and task execution interval. The task-creation part has to be negligible compared to the task execution. We abstract it with concept of minimal task granularity, i.e., minimal task (Figure 2-a).
- Balanced task execution reduces idleness of concurrent communicating tasks (Figure 2-b). For example, assume that two tasks are executing in parallel, and they eventually synchronise. If the tasks complete at very different moments, one of them spends most of its time waiting for the synchronisation. This idleness can strongly degrade the overall performance.
- Communication overhead represents the amount of time a task spends in transferring data (Figure 2-c). This time has to be small compared to tasks execution. It has to be analysed and minimised because it also determines the overall performance of the parallel program.

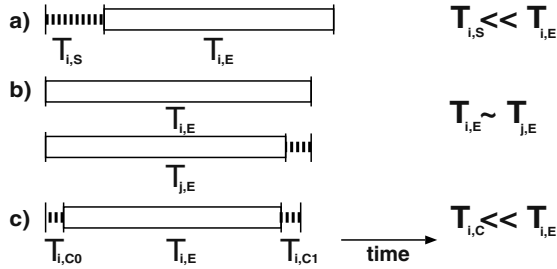


Fig. 2. Optimisation criteria: a) minimal task - minimal execution time for which the task-creation overhead is negligible, b) balanced task - balanced execution time of two parallel tasks reduces task idleness, c) maximal communication - maximal allowed communication time with respect to task execution time

Therefore, in the performance analysis we focus on producing representative profiling information and their interpretation with respect to the above defined optimisation criteria.

3 Java Parallel Performance Analysis

To provide the designer with realistic figures on performance of the program we have implemented the parallel performance analysis for concurrent Java programs (Figure 3). The tools work as follows. Firstly, the program is automatically extended with profiling code based on designer's input constraints. Secondly, the parallel program execution is emulated and profiled. Lastly, the profiling information is analysed and indication on performance bottlenecks are reported.

To reduce the amount of information generated by profiling we use a selective profiling technique. It reduces run-time overhead of the profiling and it also helps the designer to concentrate on a particular problem. The designer defines parts

of the program which have to be profiled. He or she also indicates the profiling mode, i.e., how the profiling is performed and what results are to be reported.

To ease the usability of our tool we have implemented instrumentation support. The instrumentation replaces standard Java threads and synchronisation primitives with profiler-specific equivalents. This simplifies the profiler's implementation, which results in its lower run-time overhead.

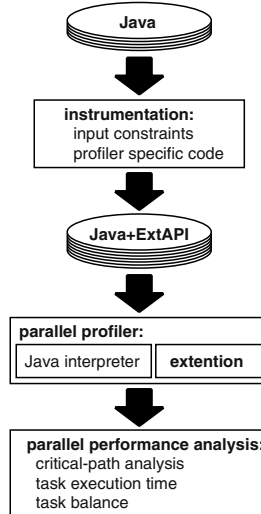


Fig. 3. Performance analysis. Firstly, the instrumentation tool adapts the original program with respect to input constraints and profiler. The parallel profiler interprets the program and gathers program parallel profile. The report generated by the profiler is later processed by the performance analysis algorithm to indicate performance bottlenecks

We have defined a Java Extension API (ExtAPI) as an interface to the instrumentation. ExtAPI reflects all the designer input options and the profiler-specific synchronisation primitives. It also reifies¹ parts of the profiler interface to designer disposal. Therefore an expert designer can manually define the profiling level for different parts of the program. Moreover, the profiler behaviour can be adapted to a particular situation at run-time.

As mentioned above, we have based the profiler on the concept of virtual time or virtual program execution (Figure 1), which we have implemented as an extension to the Java byte-code interpreter. The extension basically consists of an arbitrary number of timers and counters.

¹ Reification is the process by which a designer program or any aspect of a programming language, which were implicit in the translated program and the run-time system, are brought to the fore using a representation expressed in the language itself and made available to the program, which can inspect them as ordinary data. [23]

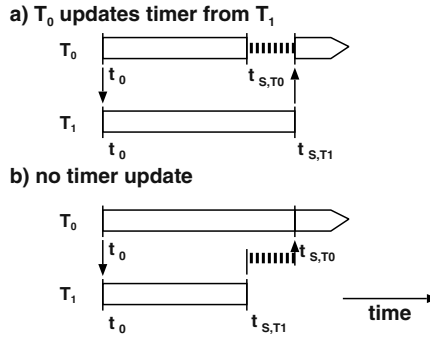


Fig. 4. Executing multithreaded Java program with virtual time. The thread local timer is updated when threads synchronise. The local timer is updated a) if the time elapsed in blocked thread is shorter than time elapsed in the blocking thread otherwise b) there is no need to update timers

The virtual-time timers are assigned to every profiled task of the program. They gather the information on its execution and synchronisation as the task proceeds in time (Figure 4). A timer of a blocked task is updated with the correct time at the synchronisation points. Thus the timer either updates its value from the blocking task if the blocking task has executed longer than the blocked one (Figure 4-a) or it keep its value (Figure 4-b). This way the virtual time propagates through the executing tasks and the parallel profile information is independent of the real task execution sequence.

The parallel profile information is later processed by a critical-path analysis algorithm in the post-processing phase of performance analysis. The algorithm reports on the program critical path, critical methods, task granularity and idleness. This information is essential for further exploitation of task-level parallelism.

3.1 Java Byte-Code Instrumentation

The main goal of instrumentation is to selectively transform the program with respect to designer constraints. The instrumentation consists of two phases: first, insertion of the profiling code based on designer constraints, and second transformation of Java synchronisation primitives into the profiler-specific ones.

Reflecting Designer's Options in the Program. In this phase the instrumenter reflects the designer-set constraints during the adaptation of the original program. The tool supports the following set of input constraints/options: (non-)cumulative profiling, selective profiling, two modes of report generation. Depending on the option specified, the instrumenter inserts the appropriate code into the original program.

Cumulative vs. non-cumulative option specifies how method timers update when a particular method (caller) calls another method (call-site) (Figure 5). In the cumulative scenario the caller's timer proceeds in counting even for the call-site's execution time (Figure 5-a). Therefore, it cumulates their total execution time. In the non-cumulative scenario the timers are stopped till the execution returns to the actual method (Figure 5-b). This feature allows the designer to find the critical parts of the program by distinguishing between the actual time spend in the method and the time spent in its call-sites.

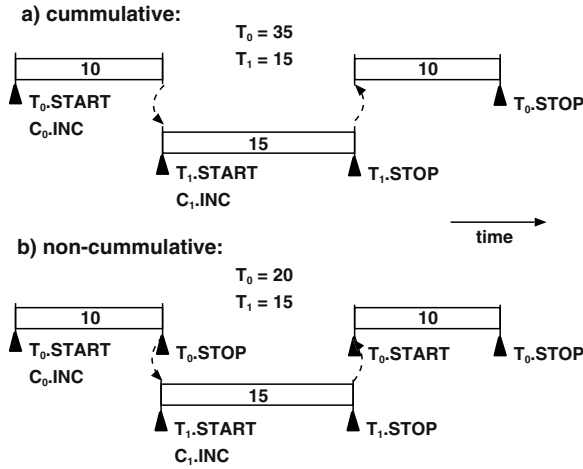


Fig. 5. Cumulative option a) does not pause the timer of the caller method when entering the call site while non-cumulative option b) pauses the method timer on any call from this method and starts on any return to it ($T(i) = T(i,0) + T(i,1)$)

The selective profiling consists of two modes which can be freely combined for different parts of the program. In the first mode the instrumenter adapts only listed methods of specific classes. In the second mode, the instrumenter inserts the code in the subgraph of the static call graph starting at the specified method. Selective profiling allows to put a narrow focus and to precisely analyse the local parts once the global profile is available. In our case the selection is made as a pre-processing step so it reduces the run-time overhead of the actual profiling.

The profile reporting code is also directly inserted into the original program. During the execution, it specifies when the report has to be generated and how detailed the report shall be. Two levels of detail are included: concise and complete. The complete report provides detailed information on method timers, counter, task execution, synchronisation and idleness.

Transforming Standard Java Synchronisation Primitives. In the second phase the instrumenter transforms the standard Java synchronisation primitives into new ones. The instrumentation is based on pattern matching where the original code fragments are interchanged with profiler-specific ones. The synchronisation primitives used are profiler-specific binary semaphores. The main purpose of the semaphores is to correctly propagate task's local time from one task to another (Section 3.3).

The binary semaphore has two states: locked and unlocked, and two atomic operations: *sema.V()* and *sema.P()*. *sema.V()* is a non-blocking operation which unlocks the semaphore, i.e., task executing this operation can immediately proceed in execution. *sema.P()* probes the semaphore. If a task executes this operation and the semaphore is locked, the task is blocked. The task unblocks after the semaphore is unlocked and it automatically locks it again.

We unify three Java features of concurrency (thread operations, object locking and synchronised statements) into equivalent code based on the semaphores (Figure 6). This way we make the features explicit to the profiler.

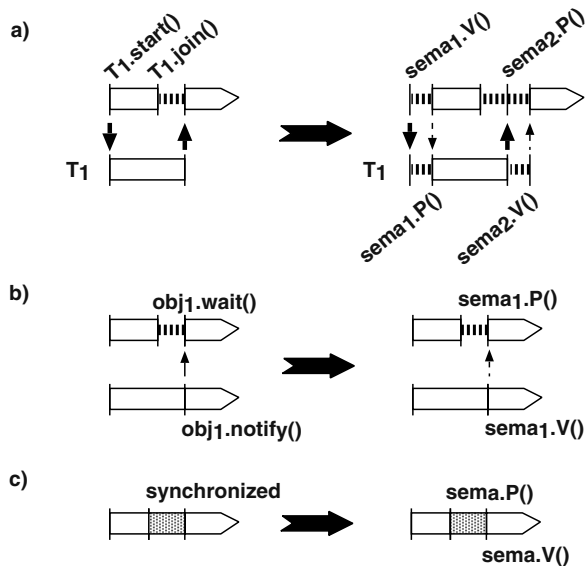


Fig. 6. Unified synchronisation patterns. Java thread operations a) and synchronisation primitives b), c) are replaced by profiler-specific patterns based on binary semaphore (sema). The semaphores propagate the virtual time through the program execution

- *Thread.start()* and *Thread.join()* methods are extended with semaphore synchronisation scheme (Figure 6-a). This allows proper propagation of the virtual time at the beginning and end of any thread.

- *Object.wait()* method is replaced by *sema.P()*. *Object.notify()* and *Object.notifyAll()* methods are replaced by *sema.V()*.
- *synchronised* statement is replaced with a *sema.P()* at the beginning of the synchronised block and *sema.V()* at its end. The atomic access to the shared resource is preserved and the timing information is correctly propagated.

The implementation of the instrumenter is based on the SOOT optimisation framework [9].

3.2 ExtAPI - Interface to the Profiler

All the above described features are made available to the designer at Java source-code level via Java Extension API (ExtAPI). This interface allows to change the profiling configuration, behaviour of the program and reported information at run-time. Using it, the designer can control program timing, insert general purpose timers and/or counters and generate partial profile information at any point of execution.

The ExtAPI is implemented partially in Java yet the major part of it is implemented inside the interpreter via Java Native Interface [8]. The ExtAPI is summarised in Table 1.

Table 1. ExtAPI features

synchronisation	
<i>sema</i> (state <i>s</i>)	create semaphore with initial state
<i>sema.P()</i>	probe if the semaphore is set, blocking
<i>sema.V()</i>	set the semaphore, non-blocking
reification - counters/timers	
<i>cnt</i> (int count)	create counter with initial count
<i>cnt.set</i>	set counter value
<i>cnt.rst</i>	reset counter value
<i>cnt.inc</i>	increment counter with a number [1]
<i>cnt.dec</i>	decrement counter with a number [1]
<i>cnt.get</i>	read counter value
reification - profiler	
<i>prf.getStat</i>	get information on this thread execution
<i>prf.getAllStat</i>	get information on all threads execution
<i>prf.skipExtAPI</i>	stop timers if executing ExtAPI code
<i>prf.skipJavaAPI</i>	stop timers if executing Java API code

3.3 Parallel Java Profiler

The parallel profiler is the essential element of the performance analysis tool. It actually implements the virtual-time concept. First of all, it must guarantee correct propagation of the simulated time during the program execution. It also

has to adapt its behaviour according to the designer constraints reflected in the program code.

The profiler uses a modified Java interpreter to actually execute the profiled program. It assigns each task/thread of the program a separate timer and an unique identification number to correctly propagate the virtual-time information of the simulated parallel execution. This way it also ensure that the time information is independent on context switching in the underlying system. Thus, the parallel behaviour of the program is properly simulated while the program is actually executed in the underlying interpreter.

The profiler executes the program till it identifies any of the ExtAPI features. In that case it performs an adequate action (Table 1): synchronisation, operations on counters and timers, and operations on profiler itself. While the operations on counters, timers and profiler adjust the process of profiling, the synchronisation actually propagates the correct virtual-time information. As mentioned above, all synchronisation in the program is unified into operations on binary semaphores from the ExtAPI.

The profiler handles the time propagation as follows (Figure 4). Let us assume there are two cooperating threads where one is the parent thread (T_0) and the other is its child thread (T_1). They have started their concurrent execution in time t_0 and they synchronise in the future at synchronisation point S at time t_{S,T_0} and t_{S,T_1} respectively. As mentioned above, the actual synchronisation is performed via binary semaphore operations, i.e., waiting thread T_0 executes probing operation *sema.P()* while blocking thread T_1 executes setting operation *sema.V()* to eventually release T_0 . There are two possible scenarios. If $t_{S,T_0} < t_{S,T_1}$ the virtual time has to be propagated from T_1 to T_0 , i.e., T_1 sets the semaphore timer to t_{S,T_1} . This value updates the T_0 local timer, i.e., the virtual time is propagated between the two threads. If $t_{S,T_0} > t_{S,T_1}$ then T_0 has executed longer than T_1 and there is no need to update its local timer. This way the virtual time is propagated via operations on semaphores and incremented by the execution of byte-code instructions of a particular thread in the Java interpreter.

To obtain more realistic figures for the simulated processor, the profiler associates the byte-code instructions with corresponding time budgets. We have performed profiling of the execution of the instructions on our platform (computer with Linux operating system and Pentium processor). The time budget varies from 1 processor cycles (simple instructions) to 27244 cycles (multianewarray - byte-code instruction allocating a multidimensional array). The typical number is about 8 cycles². Those data vary for different processors yet this way we can obtain execution profiles also for other platforms, e.g., using a Java processor or other interpreter implementations instead of the interpreter running on Pentium processor.

We have used interpreter of the Kaffe Virtual Machine [10] to implement the profiler. We have observed small performance degradation of the profiler (less than 5%) compared to the Kaffe interpreter.

² Those numbers are obtained after eliminating number of processor cycles needed for accessing processor-cycle counter which is approximately 92 cycles

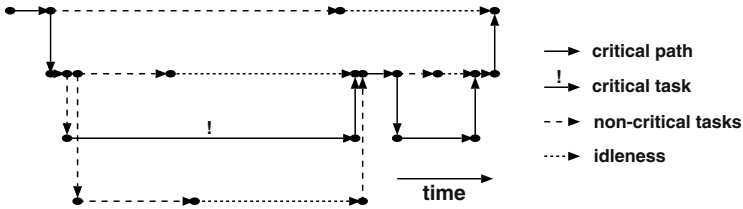


Fig. 7. Critical path analysis identifies the critical path in the program execution and identifies the critical method where reduction in execution time would have the strongest influence on overall time reduction

3.4 Critical-Path Analysis as a Parallel Performance Metric

Critical path analysis (CPA) is one of parallel performance analysis algorithms [13,15]. The CPA algorithm reports on the critical path in the program execution, critical threads and their methods as well as thread idleness. The CPA algorithm is applied as a post-processing phase of the profiling.

The CPA algorithm uses a program activity graph as an input. The program activity graph (PAG) is defined as the graph of a single program trace [15], where nodes in the PAG are events in program execution and arcs represent the ordering of events within the process or communication dependencies between the processes. Each arc is labelled with the amount of CPU time between the events.

In our case the graph is explicitly present in the profiler report. The nodes are reduced to synchronisation points, i.e., operations on semaphores. The arcs within a thread are labelled with execution time between two consecutive synchronisations (Figure 7). The detailed report includes additional arcs representing all method invocations in a particular thread.

4 Experimental Results

In experiments accomplished so far we have focused on a proof of the concept for the proposed OO parallelism extraction technique. We have used the performance analysis to identify the dominant parts of three realistic programs and evaluate the performance of their transformed parallelised versions. Because task-level parallelism only has a realistic effect on complex realistic programs and not on artificial small benchmarks, we have only focused on the former class. Because of the large effort needed to set up the experiments we have restricted ourselves to the three representative cases from different domains.

For the evaluation we have used an MPEG video player [24], a 3D application [25] and a Java compiler [26]. Using the performance analysis tool, we have analysed the original programs and manually transformed them into parallel versions. We have profiled and analysed the parallel versions to obtain the feedback on efficiency of our transformations. The results shown in Table 2

Table 2. Performance analysis results

program	speed-up	# threads	idleness [%]	instrum.time [s]
MPEG	2.3	5	20	30
3D v1	4.1	8	23	31
3D v2	4.6	18	36	31
javac v1	1.1	7	0	210
	1.2	12	0	
javac v2	1.4	21	25	210
	1.9	32	34	
javac v3	1.8	21	21	210
	2.3	32	32	

report on the speed-up compared to the original program, the number of concurrent threads, the idleness of all threads with respect to the execution time and the instrumenter execution time.

An example of instrumentation results based on the designer input options is shown in Figure 8 and an example of the generated profiling report is provided in Figure 9.

We draw some conclusions after the performance analysis: The MPEG player is data-dominated and better suited for exploiting loop-level parallelism. The implementation consist of 115 methods and the parallel version uses 5 threads. The performance improvement is dependent on heavy communication between the threads. The communication overhead was estimated by a dedicated message passing extension to the profiler.

The 3D application has dynamic behaviour which strongly influences the performance. The profiler has identified texture processing and scene rebuilding as the dominant parts of the program. The communication overhead profiling was manually introduced into the code according to the size of the shared objects. The two versions of 3D applications represent two mappings of the tasks/threads to the target platform. In both versions the program consists of 18 parallel tasks and 403 methods. In the first version, we have mapped few tasks to the same processor, i.e., we have assigned a few small tasks to the same processor - thread timer of the profiler. This mapping results in task idleness of 23% and requires 8 parallel processors. In the second version, all tasks run in parallel which results in even higher idleness of the tasks but the highest speed-up. However, the resource utilisation decreases: we need another 10 processors to improve the speed-up from 4.1 to 4.6 and then the total idleness increases to 36 % (Table 2).

The Java compiler is the largest of the three applications. Its complexity is also visible from the instrumentation time needed (Table 2). The analysis has shown that the compiler implementation is dominated by shared objects and recursive method calls, which considerably complicates the process of parallelism extraction. We have extracted and evaluated 3 parallel versions of the program. The first version uses threads for the code-generation phase, i.e., there is no further need to synchronise the threads and the total idle time is zero. The total

original program:	instrumented program:
<pre>public class Scene { public void rebuild() { synchronize(this) { hashtable.add(3DObject); } return; } }</pre>	<pre>public class Scene { public void rebuild() { Meta.startCnt(0); // starts method timer Meta.incCnt(1); // increments method call counter bsema.P(); // synchronized begin Meta.stopCnt(0); // non-cumulative option hashtable.add(3DObject); // Meta.startCnt(0); // non-cumulative option } bsema.V(); synchronized end Meta.stopCnt(0); // stops the method timer return; } }</pre>

Fig. 8. An example of code instrumentation. Counter 0 is timer for method rebuild(). Counter 1 is the method-call counter. The non-cumulative option is reflected in the code by stopping counter 0 when executing hashtable.add(). Semaphores reflect the synchronised statement in the program

```
/* program start
#JVM thread@tid(15284637) demo.main()
#JVM thread@tid(17283921) Texture.run()

// synchronisation before starting new thread
#JVM bsema(0).P() tid 17283921 Texture.process() 5000 0
#JVM bsema(0).V() tid 15284637 demo.main() 5000

// synchronisation: Texture.process() executed for 2000 !
#JVM bsema(0).P() tid 17283921 Texture.process() 8000 1000
#JVM bsema(0).V() tid 15284637 demo.main() 8000

// synchronisation at the end of Texture.done()
#JVM bsema(2).V() tid 17283921 Texture.done() 10000
#JVM bsema(2).P() tid 15284637 demo.main() 10000 -500
// program end

// profiling information:
#JVM thread 'timers:
tid 15284637 time 12000 waited -500
tid 17283921 time 5000 waited 1000
```

Fig. 9. An example of a profiler report. The calls to binary semaphores determine the arcs in the Program-Activity Graph. The data reported are thread id, method name, actual timer value at synchronisation point and idleness time of the thread

observed speed-up is small due to small portion of the parallel part compared to the whole execution time (Amdahl’s law). The second version uses multiple threads for dominant part of the execution. The overall speed-up is lower compared to the 3D application because of very unbalanced thread execution times. This reflects the large differences in size and complexity of the compiled classes and methods. The third version is a combination of the previous two. As the two phases are non-overlapping in their execution, the computational resources can be reused and the total thread idleness is reduced. We have evaluated the three

versions on the compilation of different java source codes resulting in different idleness, speed-up and total number of threads. The lower and upper bound of the execution results are shown in the Table 2.

We have not been able to directly compare our results to the results obtained using zJava [5]. However, as mentioned in Section 2, the tool assigns to every method a separate task and resolves task dependences. The run-time system uses this information to control task granularity and assignment to resources. From our experimental results we can conclude that run-time task-management overhead can be considerably increased if handling a large number of small tasks, e.g., the 3D application has 403 methods. Moreover, the presence of many small tasks results in an increase of potential tasks idleness (low utilisation), little improvement on overall performance and higher requirement on resources.

The performance analysis tool, we implemented, allows to exploit and analyse the performance issues for any target platform at compile time which can considerably reduce the complexity of the run-time task management.

5 Conclusions and Future Work

We have introduced the performance analysis part of a transformation framework for extraction of task-level parallelism from sequential object-oriented programs. The main difference of our approach compared to related work is in the concept of virtual time, which allows to emulate behaviour of the parallel program with respect to the architectural constraints of the target platform. Moreover, the emulation is independent of the underlying system. We have implemented the concept as an extension to a Java interpreter. To increase efficiency of the profiling, we have adopted a selective profiling technique and have implemented automatic instrumentation of the Java byte-code.

We have demonstrated our performance analysis technique on three realistic test vehicles. We have also shown how different mappings to a particular target platform influence the overall parallel performance. We have demonstrated the potential of our technique for exploration and analysis of the parallel program performance on target multi-processor platforms.

In the future, we would like to extend our tools with automated support for communication analysis and experiment with different configurations of the profiler for different processor architectures including power models.

References

1. Girkar, M., Polychronopoulos, C.D.: Automatic Extraction of Functional Parallelism from Ordinary Programs, *IEEE Trans. on Parallel and Distributed Systems* (1992)
2. Baneerjee, P., et.al.: The Paradigm compiler for Distributed-Memory Multicomputers, *IEEE Trans. on Computer* (1995)
3. Saito, H., et.al.: The Design of the PROMIS Compiler, *Proceedings of the International Conference on Compiler Construction* (1999)

4. Huynh, S.: Exploiting Task-level Parallelism Automatically Using pTask, Master Thesis at University of Toronto (1996)
5. Chan, B., Abdelrahman, T.S.: Run-time support for the automatic parallelization of Java programs, Proc. of Int. Conf. on Parallel and Distributed Computing and Systems (2001)
6. Weiser, M.: Program slicing, Proceedings of 5th International Conference on Software Engineering (1981)
7. Hatchiff, J., et al.: A Formal Study of Slicing for Multi-threaded Programs with JVM Concurrency Primitives, Static Analysis Symposium (1999)
8. Liang, S.: The JavaTM Native Interface: Programmer's Guide and Specification, Addison Wesley Longman Inc. (1999)
9. Vallee-Rai, R., Hendren, L., Sundaresan, V., Lam, P., Gagnon, E., Co, P.: Soot - A Java Optimization Framework, Proc. of CASCON (1999)
10. Kaffe Virtual Machine: <http://www.kaffe.org>
11. Fenlason, J., Stallman, R.: GNU gprof - The GNU Profiler
<http://www.gnu.org/manual/gprof-2.9.1/gprof.html>
12. Hollingsworth, J.K., Miller, B.P.: Parallel Program Performance Metrics: A Comparison and Validation, Supercomputing (1992) 4-13
13. Miller, B.P., Clark, M., Hollingsworth, J.K., Kierstead, S., Lim, S., Torzewski, T.: IPS-2: The Second Generation of a Parallel Program Measurement System, IEEE Transactions on Parallel and Distributed Systems, Vol.1, N.2 (1990) 206-217
14. Anderson, T.E., Lazowska, E.D.: Quartz: A Tool for Tuning Parallel Program Performance, Performance Evaluation Review, Special Issue, ACM SIGMETRICS, Vol.18, No.1 (1990) 115-125
15. Hollingsworth, J.K.: Critical Path Profiling of Message Passing and Shared-Memory Programs, IEEE Transactions on Parallel and Distributed Systems, Vol.9, No.10 (1998) 1029-1040
16. Shende, S., Malony, A.D., Cuny, J., Lindlan, K., Beckman, P., Karmesin, S.: Portable Profiling and Tracing for Parallel Scientific Applications using C++, Proceedings of ACM SIGMETRICS Symposium on Parallel and Distributed Tools (1998)
17. Java Virtual Machine Profiler Interface,
<http://java.sun.com/products/jdk/1.2/docs/guide/jvmpi/jvmpi.html>
18. Borland Optimizeit Suite 5 <http://www.borland.com/optimizeit/>
19. Rational Quantify
http://www.rational.com/products/quantify_unix/index.jsp
20. Vtune Enterprise Analyzer, Intel Inc.
<http://developer.intel.com/software/products/vtune/vte/%5Fjava10/>
21. Kazi, I.H., et al.: Javiz: A Client/Server Java Profiling Tool, IBM Systems Journal, Vol.39, N.1 (2000)
22. Sevitsky, G., De Pauw, W., Konuru, R.: An Information Exploration Tool for Performance Analysis of Java Programs, TOOLS Europe (2001)
23. Malenfant, J., Jacques, M., Demers, F.N.: A tutorial on behavioral reflection and its implementation, Proceedings of the Reflection'96 Conference (1996) 1-20
24. Anders, J.: MPEG-1 player in Java
http://rnvs.informatik.tu-chemnitz.de/~jan/MPEG/MPEG_Play.html
25. Walser, P.: IDX 3D engine,
<http://www2.active.ch/~proxima>
26. Java Compiler, <http://java.sun.com/j2se/1.3/>

Towards Superinstructions for Java Interpreters

Kevin Casey¹, David Gregg¹, M. Anton Ertl², and Andrew Nisbet¹

¹ Department of Computer Science, Trinity College, Dublin 2, Ireland
`{Kevin.Casey,David.Gregg,Andy.Nisbet}@cs.tcd.ie`

² Institut für Computersprachen, TU Wien, A-1040 Wien, Austria
`anton@complang.tuwien.ac.at`

Abstract. The Java Virtual Machine (JVM) is usually implemented by an interpreter or just-in-time (JIT) compiler. JITs provide the best performance, but interpreters have a number of advantages that make them attractive, especially for embedded systems. These advantages include simplicity, portability and lower memory requirements. Instruction dispatch is responsible for most of the running time of efficient interpreters, especially on pipelined processors. Superinstructions are an important optimisation to reduce the number of instruction dispatches. A superinstruction is a new Java instruction which performs the work of a common sequence of instructions. In this paper we describe work in progress on the design and implementation of a system of superinstructions for an efficient Java interpreter for connected devices and embedded systems. We describe our basic interpreter, the interpreter generator we use to automatically create optimised source code for superinstructions, and discuss Java specific issues relating to superinstructions. Our initial experimental results show that superinstructions can give large speedups on the SPECjvm98 benchmark suite.

1 Motivation

The Java Virtual Machine (JVM) is usually implemented by an interpreter or just-in-time (JIT) compiler. JITs provide the best performance, but interpreters have a number of advantages that make them attractive, especially for embedded systems. First, interpreters require much less memory than JITs, both for the interpreter itself and the Java bytecode. For example, Hoogerbrugge et al. [13] found that a bytecode representation of a program could be up to five times smaller than the corresponding machine code. Many embedded systems have small memories giving interpreters a decisive advantage.

A second important advantage of interpreters is that they can be constructed to be trivially portable to new architectures, assuming that a C compiler for the new architecture already exists. In contrast, it can take many months to port the back end of a JIT compiler. Portability means that the Java interpreter can be rapidly moved to a new architecture, reducing time to market. There are also significant advantages in different target versions of the interpreter being compiled from the same source code. The various ports are likely to be more reliable, since the same piece of source code is being run and tested on many

different architectures. A single version of the source code is also significantly cheaper to maintain. There are other parts of the JVM that are more difficult to port (such as the Java Native Interface for calling machine code functions), but many embedded JVMs, such as Sun's KVM [19] for mobile devices, have limited support for these unportable features.

A third advantage of interpreters is that they are significantly smaller and simpler than JIT compilers. Simplicity makes them more reliable, quicker to construct and easier to maintain. When building a JIT compiler one must not only debug the code for the compiler, but must often also debug the code *generated* by the compiler. This is not an issue for interpreters. A final smaller advantage of interpreters is that they do not necessarily have to compile the bytecode into another format before execution. Sun's Hotspot mixed mode compiler/interpreter JVM takes advantage of this by only compiling code that has been shown to be frequently executed. The compilation overhead for rarely used code is often greater than the time needed to execute that code on an interpreter. A similar strategy is used by Transmeta for their Crusoe processor which emulates the x86 instruction set through a combination of interpreting and binary translation.

A weakness of using interpreters is that they run most code much slower than JITs. Even very efficient interpreters are typically about ten times slower than a JIT compiler [13]. The goal of our work is to narrow that gap, by applying speed optimisations to Java interpreters. One such optimisation is the use of superinstructions. Certain sequences of VM instructions (such as `ALOAD_0 GETFIELD`) occur frequently in Java bytecode. A superinstruction is a new instruction that behaves in the same way as a sequence of simple Java instructions. By replacing such sequences with the corresponding superinstruction, the work of several instructions can be performed, but with the interpreter overhead of only a single VM instruction.

Superinstructions have been used for many years to optimise interpreters. Traditionally, the addition of superinstructions to an interpreter made it much less maintainable, because they increased the size of the source code. We use an interpreter generator to automatically generate source code for superinstructions, based on a specification of the component instructions. Our generator system automatically optimises the source code for superinstructions to avoid unnecessary loads and stores by keeping intermediate values in registers, and by combining stack pointer updates.

This paper describes the design and implementation of a system of superinstructions for an optimised Java interpreter. Preliminary experimental results show that superinstructions can greatly increase the speed of a portable Java interpreter, allowing it to significantly outperform commercial Java interpreters hand-coded in assembly language.

2 Superinstructions

A superinstruction is a new virtual machine instruction that consists of a sequence of several existing VM instructions. There are several advantages in

combining instructions in this way. First, it reduces the number of instruction dispatches required to perform a certain sequence of instructions. This is important since instruction dispatch is usually the most time consuming part of executing and instruction¹. Secondly, it allows us to optimise the interpreter source code. For example, our interpreter generator automatically reuses values across VM instructions without reloading them, eliminates cancelling stack pointer updates, and performs other small stack optimisations when generating C code from the instruction definition. Thirdly, combining the source code for instructions together exposes a larger “window” of code to the C compiler, which allows greater opportunities for optimisation.

We use the interpreter generator `vmgen` [7] to allow us to generate superinstructions using profiling information. `vmgen` takes in an instruction definition, and outputs an interpreter in C which implements the definition. The interpreter generator translates the stack specification of the instruction definition into pushes and pops of the stack, adds code to invoke following instructions, and makes it easy to apply optimizations to all virtual machine instructions, without modifying the code for each separately.

Figure 1 shows the instruction definition for the JVM instruction `ILOAD` (load integer local variable). The `#` symbol in the definition means that it takes an immediate value from the VM instruction stream. Note that we need to update the instruction pointer by two positions, since the VM instruction consists of the `ILOAD` opcode followed by an immediate operand containing the number of the local variable to load onto the stack.

```
ILOAD ( #iIndex -- iResult ) 0x21
{
    iResult = locals[iIndex];
}
```

Fig. 1. Definition of `ILOAD` VM instruction

By adding `ILOAD-IADD` to the list of superinstructions for our code copying compiler, `vmgen` will produce the source code in figure 2, which is generated automatically from the instruction definitions of `ILOAD` and `IADD`.

There are a number of notable features about this code. First, all used stack items are loaded from memory into local variables at the start of the code. The different VM instructions within the superinstruction communicate by reading from and assigning to these local variables.

Presuming that the C compiler is able to allocate these local variables to registers, this will greatly reduce the amount of memory traffic from accessing the VM stack. `IADD` alone requires two loads and one store to access the stack, and

¹ Instruction dispatch is expensive on modern architectures because it involves a difficult-to-predict indirect branch. In the case of threaded code interpreters, superinstructions not only reduce the number of dispatches, but also make the remaining branches more easily predictable using a branch target buffer (BTB) [6].

ILOAD requires one store. In contrast, the superinstruction ILOAD-IADD requires only one load and one store access to the stack to perform the same work. Thus stack memory traffic is reduced by 50%.

```

START_ILOAD_IADD: /* start label */
{
int sp0; /* synthetic names */
int sp1;
int ip1; /* synthetic name for item in VM instruction stream */
ip1 = *(ip+1); /* fetch immediate value */
sp0 = *(sp);
{ /* ILOAD */
    int iIndex; /* declare stack item */
    int iResult;
    /* fetch stack item to local variable */
    iIndex = ip1;
    { /* user provided C code */
        iResult = locals[iIndex];
    }
    sp1 = iResult; /* store stack result */
}
{ /* IADD */
    int iValue1; /* declare stack items */
    int iValue2;
    int iResult;
    iValue1 = sp1; /* fetch stack items to */
    iValue2 = sp0; /* ...local variables */
    { /* user provided C code */
        iResult = iValue1 + iValue2;
    }
    sp0 = iResult; /* store stack result */
}
*(sp) = sp0;
ip += 3; /* update VM ip */
}
NEXT; /* indirect goto */

```

Fig. 2. Simplified Vmgen output for ILOAD-IADD superinstruction

Another notable feature of the code in figure 2 is that there is no stack pointer update. ILOAD increases the size of the stack by one, and IADD reduces its size by one. Vmgen detects that the two stack pointer updates are redundant, and eliminates them. In addition, there is only one instruction pointer update.

3 Design Issues

3.1 Which Sequences?

The main determinant of the usefulness of superinstructions is whether the sequences we choose to make into superinstructions account for a large proportion of the running time of the programs that run on the interpreter. The set of superinstructions must be chosen when the interpreter is constructed, most likely at a time when one doesn't know which programs will be run on the interpreter. Thus, one must somehow guess which superinstructions are likely to be useful for a set of programs that one has never seen.

The most common way to make guesses at the behaviour of unseen programs is to measure the behaviour of a set of standard benchmarks programs, and hope that these benchmarks resemble the real programs. A question remains, however, as to how the benchmarks should be measured to identify useful superinstructions. Gregg and Waldron [12] tested a wide range of strategies for choosing superinstructions for Forth programs. They found, perhaps surprisingly, that the best strategy was to simply choose those sequences that appear most frequently in the static code. We use this strategy for the main experiments in this paper.

One complication in a Java interpreter is that the JVM comes with a large library of classes that are used internally by the JVM and by running programs. Approximately 33% of the executed bytecode instructions in the SPECjvm98 benchmark suite [18] are in library rather than program methods [21]. This library code is available at the time the interpreter is built, so there is potential for choosing superinstructions specifically for commonly used library code.

3.2 Parsing

The use of superinstructions is in many respects the same problem as dictionary-based text compression [2]. Dictionary-based compression attempts to find common sequences of symbols in the text, and replaces them with references to a single copy of the sequence. Thus, when designing a superinstruction system, we can draw on a large body of theory and experience on text compression.

Parsing is the process of modifying the original sequence of instructions by replacing some subsequences with superinstructions. The simplest strategy is known as *greedy parsing*, where at each VM instruction we search for the longest superinstruction that will match the code from that point.

For example, consider the basic block in figure 3. Assume that we have two superinstructions available: `ILOAD-ILOAD` and `ILOAD-IADD-ISTORE`. Following a greedy strategy, we would find the longest sequence that matches a superinstruction from the start of the basic block. Thus, we would replace the first two instructions with the superinstruction `ILOAD-ILOAD`, and reduce the number of dispatches needed to execute this code by one. The main advantage of greedy parsing is that it is very fast — an important factor in an optimisation that we apply to a Java method at run time, the first time that it is invoked. Greedy parsing is also simple to implement and requires little memory.

```

ILOAD 4 ; load local 4
ILOAD 5 ; load local 5
IADD    ; integer add
ISTORE 6 ; store TOS to local 6
ILOAD 6 ; load local 6
IFEQ 7   ; branch by 7 if TOS == 0

```

Fig. 3. Example basic block

The weakness of greedy parsing becomes apparent when we consider whether a better parse of the code in figure 3 is possible. Clearly, it would be better to replace the second, third and fourth instructions with the superinstruction `ILOAD-IADD-ISTORE`. This would reduce the number of dispatches by two. To be sure of always finding the best possible parse, an optimal parsing algorithm must be used. Fortunately, optimal parsing can be solved using dynamic programming [2], so efficient algorithms are available. However, our preliminary experiments show that even fast implementations are measurably slower than greedy parsing. Furthermore, these preliminary experiments show optimal parsing reducing the number of instruction dispatches by less than 5%.

Our current implementation uses a simple version of greedy parsing. In the bytecode translator, we always keep a buffer of the most recently generated threaded code instruction in the basic block. When we generate the next instruction, we check whether it can be combined with the one in the buffer. If it can, then the instruction in the buffer is replaced with the corresponding combined superinstruction. If not, the instruction in the buffer is written to the code area for that method, and it is replaced in the buffer by the just generated instruction. This strategy is simple to implement, requires little memory, and makes the check for replacement with superinstructions extremely fast.

One weakness of this strategy, however, is that for a long superinstruction to be usable, all prefixes of the instruction must also be valid superinstructions. For example, if we have the superinstruction `ILOAD-ILOAD-IADD-ISTORE`, then we must also have the superinstructions `ILOAD-ILOAD` and `ILOAD-ILOAD-IADD`. In practice, this is not a problem, since we usually select superinstructions based on the frequency of sequences in real programs, and by definition subsequences have a frequency at least equal to that of the longer sequence. However, in future implementations we intend to relax this restriction to allow us to exploit more complicated superinstruction selection strategies.

3.3 Quick Instructions

Several Java bytecode instructions must perform various class initialisations on the first time that they are executed. On subsequent executions no initialisations are necessary. A common way to implement this functionality is with “quick” instructions. The first time a given instruction of this type is executed, it performs the necessary initialisations, and then replaces itself in the instruction stream

with a corresponding quick instruction, which does not do these initialisations. On subsequent executions of this code, the quick instruction is executed.

Quick instructions are vital to the performance of most Java interpreters, since the check for class initialisation is expensive, and because they are among the most commonly executed instructions. For example, in the SPECjvm98 benchmarks `GETFIELD` and `PUTFIELD` account for about one sixth of all executed instructions, and run very slowly unless converted to quick versions [21]. Eller [3] found that adding quick instructions to the Kaffe interpreter could speed it up by almost a factor of three.

A problem with quick instructions is that they make it difficult to replace sequences of instructions with superinstructions. No instruction that will be replaced with another instruction at run time can be placed in a superinstruction, since that would involve replacing the entire superinstruction. Furthermore, some instructions, such as `LDC` (load constant from constant pool) and `INVOKEVIRTUAL` become different superinstructions depending on the value of their inline arguments, or the type of class or method they belong to.

An additional complication when dealing with non-quick instructions is race conditions. Due to the threaded nature of the Java interpreter, during quickening it is quite possible for two threads to almost simultaneously access a non-quick instruction triggering a potential race condition. Such race conditions are avoided in the current implementation of `cvm` by using mutually exclusive locks, but adding support to allow quickened instructions to become part of a superinstruction after translation could lead to race conditions.

Our current implementation does not allow any “quickable” instructions to participate in superinstructions. However, we are experimenting with a wide range of strategies to change this. Perhaps our most promising is to simply add an extra routine to the quickening process to reparse the basic block once the original instruction has been replaced. This approach is greatly simplified by leaving gaps for removed instructions in the code, as is outlined in the next subsection.

3.4 Across Basic Blocks

Superinstructions are normally only applied to instructions within basic blocks. However, with relatively small modifications, it is possible to extend superinstructions across basic block boundaries in two specific situations. First, we consider control flow joins. A join is a point in the program with incoming control flow from two or more different places. Usually one of those places is simply the proceeding basic block, and control falls through to the join without any branching. In these cases, the falling-through code is simply a straight-line sequence of instructions. However, it is not normally safe to allow a superinstruction to be formed across the join, because it would not then be clear where the other incoming control-flow paths should branch to.

The solution we use is to create superinstructions, but not to remove the gaps that are created by eliminating the original instructions. In fact, we leave the original instructions in these gaps. Figure 4 shows an example of, where we

	ILOAD		ILOAD
	4		4
	ILOAD		ILOAD-IADD
	5		5
join:	IADD	join:	IADD
	ISTORE		ISTORE
	6		6

Fig. 4. Original code (left) and same code with ILOAD-IADD superinstruction (right)

have replaced the sequence ILOAD, IADD with the superinstruction ILOAD-IADD. We actually replace the ILOAD instruction with ILOAD-IADD, but leave the IADD instruction where it is. When we fall-through from the first basic block to the second, we execute ILOAD-IADD, which performs its normal work and then skips over the IADD instruction. On the other hand when we branch to the second basic block from elsewhere, we branch to the IADD instruction which executes and continues as normal. This scheme allows us to form superinstructions across fall-through joins.

We believe that this scheme is particularly valuable for while loops. The standard *javac* code generation strategy appears to be to place the loop test at the end of the loop, and on the first iteration to jump directly to this test. Unfortunately, the result is that there is a control flow join just before the loop test that would normally hinder optimisation. We believe we have successfully overcome this problem.

```
IFNULL ( #aTarget aRef -- ) 0xc6
{
    if ( aRef == NULL ) {
        SET_IP(aTarget);
        TAIL;
    }
}
```

Fig. 5. Definition of a branch VM instruction

A second opportunity for cross-basic block superinstructions is with the fall-through direction of VM conditional branches. Currently, superinstructions are not allowed to extend across branches. However, *vmgen* already provides a facility for specifying a taken branch. Figure 5 shows the instruction definition for a branch instruction. Inside the *if* statement the *vmgen* keyword *TAIL* is used to specify that a copy of the dispatch code that normally appears at the end of the instruction should be placed here. We believe that with some modifications to *vmgen*, the same facility can be used to create superinstructions that extend

across untaken branches, with the necessary code for the taken path generated using the TAIL mechanism.

4 Experimental Evaluation

The primary purpose of the work presented here was to evaluate the effect of adding superinstructions to the JVM. By adding superinstructions, we reduced the number of stack updates and also eliminated branch target mispredictions for instructions within the superinstruction. As a result, we expected to see significant improvements as more and more superinstructions were added to our JVM (subject to some limitations). It was also strongly suspected that the method used to select which superinstructions to add would have a substantial effect on the superinstructed JVM. The benchmarks selected for evaluating the effect of changes to the JVM were taken from the SPECjvm98 suite.

In order to obtain a JVM with support for superinstructions it was necessary to modify Sun Microsystem's CVM for embedded processors. Apart from converting the JVM to work with dynamically threaded code, the bulk of the work was in porting the main interpreter loop to Vmgen in a satisfactory manner to allow for superinstructions. Once the interpreter loop had been ported to vmgen, the selection of candidate superinstructions and the actual inclusion of superinstructions in the JVM became a relatively straightforward process due to the nature of Vmgen.

To select superinstructions to add to the JVM, two contrasting approaches were taken. In the first approach, all benchmarks were run and all sequences of bytecode (and their subsequences) encountered for the first time were recorded. When all benchmarks were completed, a histogram of these sequences was built up. From this histogram the most common statically appearing sequences of bytecodes were selected.

The second approach was more aggressive from an optimization point of view. In this approach we ran each benchmark separately and for each benchmark recorded all sequences of bytecodes encountered during the execution of the benchmark (i.e. not just the first time they are encountered). Thus the same sequence of superinstructions could be recorded several times, for example if they occurred within the body of a loop. Then, for each individual benchmark a histogram of the most commonly encountered superinstructions was generated. Then, to optimize for a particular benchmark, the histogram for that particular benchmark was used to select the most commonly executed (dynamically appearing) sequences.

Generating superinstructions based on static frequency may appear to be over-simplistic, but as an initial method of selecting superinstructions, it does seem more realistic than the dynamic approach. One of the main reasons is that for the static approach we attempted to optimize the JVM for all benchmarks at once. With the dynamic approach, the JVM was optimized separately for each benchmark before running that benchmark. Despite the artificial nature of the dynamic approach, it does give us a standard by which to measure the performance of the static approach.

When selecting superinstructions from the histogram in either approach, some superinstructions are not permitted. For example superinstructions containing “quickable” (see section 3.3) instructions are dispensed with, as there is currently no facility for dealing with them in our modified JVM.

In our modified JVM, translation takes place when a method has been called for the first time, but before any bytecodes in that method have been executed. At this point in time only immutable opcodes can be included in superinstructions, since superinstructions themselves are immutable. One possible workaround would be to try to quicken all instructions in the method and then try to translate the code to superinstructed code. However this approach would inevitably lead to the quickening of code that may never be run, and also may force static initializers to be run before they are supposed to be. Another approach would be to allow superinstructions to be added dynamically as instructions get quickened in the usual way.

The modified version of CVM used for these tests was compiled under GCC 2.96. Optimization flags “-O4 fomit-frame-pointer” were used. The “-fno-gcse” flag was used additionally to compile the file containing the main interpreter loop. This is used to disable global common subexpression elimination, which can interact badly with GNU C labels as values, which are used by our interpreter for efficient instruction dispatch [5]. The hardware used to run the benchmarks was based on a Pentium IV 1.6 Ghz with 1GB of memory.

Each benchmark was run with no superinstructions to establish a reference time. Then the benchmarks were run on versions of CVM compiled with 8, 16, 32, 64, 128, 256, 512 and 1024 superinstructions. The results were then graphed as a speedup over the time it took each benchmark to complete with no superinstructions. All SPEC benchmarks were run using the largest (size 100) input sets.

The static results are shown in figure 6. All results are averages of 5 runs of the benchmark under the same conditions. In this figure we see a small improvement for most benchmarks, even at 8 superinstructions. Two benchmarks with 8 superinstructions perform worse, however. One possible reason for the lack of improvement in *compress* and *mtrt* is that the 8 superinstructions added simply do not occur frequently, if at all, in these benchmarks. One explanation for the reduction in performance (albeit less than 1%) could be the overhead of scanning through code at translation time to see if superinstructions can be formed. Other possible reasons are discussed below.

As superinstructions are added, there is a general trend upwards in performance which is what we would expect. Benchmarks *mpegaudio*, *compress* and to a lesser degree *db*, all spend much of their time in a small number of methods [21]. It seems most likely that some superinstructions are being introduced into these commonly used methods, giving the significant performance boost. The benchmark that gets greatest benefit from superinstructions is *mpegaudio*, with a maximum speedup of about 1.56. It is interesting to note that this benefit is not substantial until 256 superinstructions are introduced.

It is not always the case that the addition of extra superinstructions improves performance. A temporary drop-off in performance can be seen in all

benchmarks at some stage, the most spectacular being when moving from 32 superinstructions to 64 superinstructions in both jack and jess. There are a number of possible explanations for these drop-offs. One possibility is that the register allocation mechanism in gcc is breaking down for superinstructions added at these points. Another is that superinstructions are causing conflict misses in the instruction cache or branch predictor. Finally, the process of scanning through a method to find possible superinstructions is slowed by the addition of extra superinstructions to the JVM.

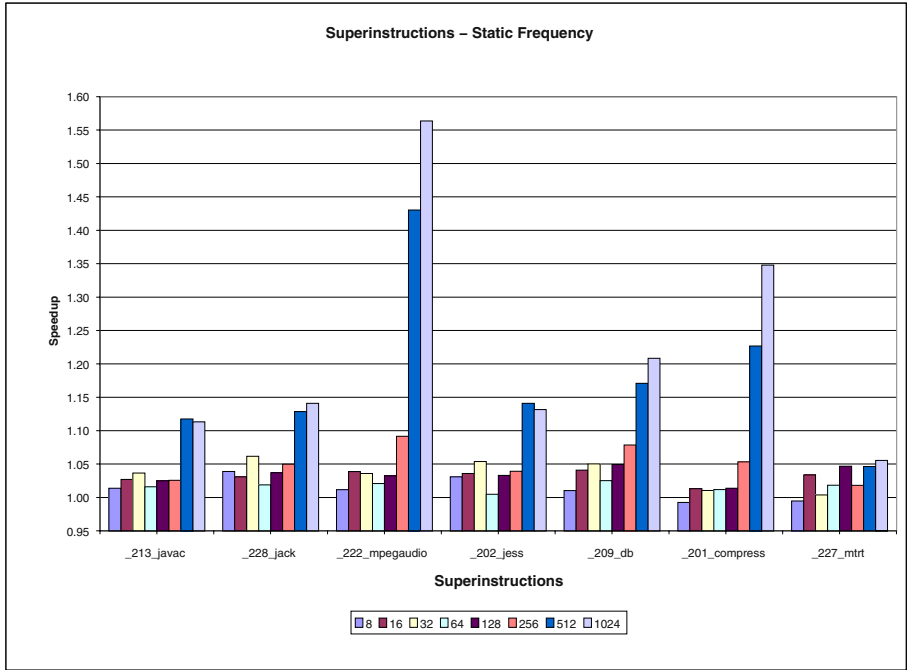


Fig. 6. Running times of the benchmarks with varying numbers of superinstructions. Superinstructions are chosen on the basis of **static** frequency of sequences across all SPECjvm98 programs

In figure 7 the performance of CVM with superinstructions based on dynamic frequency for this particular program can be seen. Performance is much better, but this is expected since CVM is optimized for each benchmark separately. This time the maximum speedup is 1.90 (*mpegaudio*). As before, the benchmarks that register the greatest improvements are those that spend much of their execution time in a limited set of methods. It can be surmised that a substantial number of superinstructions are being created in these methods.

At certain stages, the JVMs based on dynamically selected superinstructions suffer from the same drop-off in performance seen in figure 6. This time *javac*

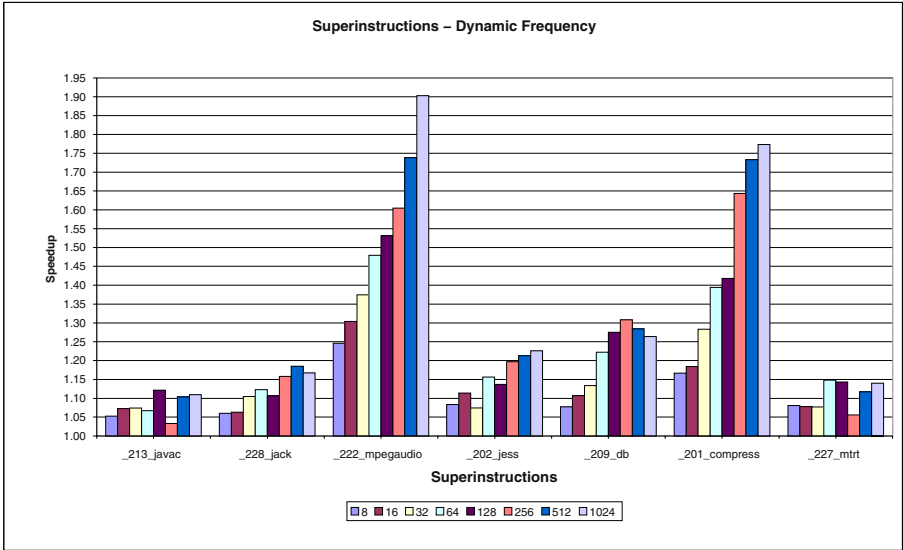


Fig. 7. Running times of the benchmarks with varying numbers of superinstructions. Superinstructions chosen are the most frequent dynamically executed sequences based on a training run of the same program

and *mtrt* both suffer a degradation in performance when moving from the 128 superinstruction JVM to a 256 superinstruction JVM.

Table 1 shows the absolute running times of the SPECjvm98 benchmarks on three different JVMs. The first is our base interpreter with no superinstructions. We also show running times for Sun’s HotSpot mixed-mode interpreter and JIT compiler, and for HotSpot using only the interpreter. Overall, the Hotspot interpreter is on average 20.4% faster than the our interpreter.

Table 1. Comparison of running time of our base interpreter (without superinstructions) with the Sun HotSpot Client VM Interpreter, and mixed mode interpreter—JIT compiler on the SPECjvm98 benchmark programs

Benchmark	Our Base Interp.	Hotspot Interp.	Hotspot Mixed-mode
javac	55.79	44.38	10.16
jack	33.48	27.68	5.19
mpeg	150.08	139.65	9.55
jess	48.14	34.38	4.35
db	116.63	86.27	26.6
compress	170.01	153.19	18.9
mtrt	52.41	43.56	6.06

There are two main reasons for this. Firstly, Hotspot has a much faster run time system than CVM. This can be seen especially strongly in the *db* benchmark, which runs 34% faster on Hotspot. The Hotspot run time system is large and sophisticated, and would not be suitable for an embedded system. Furthermore, much effort has been put into tuning the Hotspot run time system as it is more widely used than CVM. The second reason that Hotspot outperforms our version of CVM is that the Hotspot interpreter is faster than our interpreter. Its dynamically-generated, highly-tuned assembly language interpreter is able to execute bytecodes more quickly than our portable interpreter written in C. The difference in speeds of the interpreter cores can be seen by examining the benchmarks that spend most of their time in the interpreter core: *compress* is 9.1% faster and *mpeg* is 5.2% faster on the Hotspot interpreter. Finally, the mixed-mode compiler- interpreter is very much faster than either our interpreter or the Hotspot interpreter. Where speed is more important than memory use, portability, and maintainability, a JIT compiler is the correct solution.

5 Related Work

Some recent important developments in interpreters include the following. Stack caching [4] is a general technique for storing the topmost elements of the stack in registers. Ertl and Gregg [5] showed that interpreters (especially those using switch dispatch) spend most of their time in branch mispredictions on modern desktop architectures. Interpreter software pipelining [13] is a valuable technique for architectures with delayed branches (e.g. Philips Trimedia) or prepare to branch instructions (e.g. PowerPC), which makes the target of the dispatch branch available earlier by moving much of the dispatch code into the previous VM instruction. Costa [17] discusses various smaller optimizations.

The Sable VM [9] is an interpreter-based research JVM. This interpreter uses a run-time code generation system [15], not dissimilar from a just-in-time compiler. Sable uses a novel system of *preparation sequences* [10,8] to deal with bytecode instructions that perform initialisations the first time they are executed, which make code generation difficult. We believe that the same procedure could also be used to allow such instructions be part of superinstructions.

Venugopal et al. [20] present an embedded JVM system, which uses *semantically enriched code* (sEc). The sEc technique generates a custom JVM for each application. In addition, aggressive optimizations are applied to the program to allow it to make the best use of the custom JVM features. This tight coupling of the program and the interpreter allows large speedups. The weaknesses of this approach are that the code to be run must be available at the time the JVM is created, and that the JVM is no longer general purpose.

Combining operations using an interpreter generator system was previously explored in the context of superoperators [16]. A superoperator is pattern of more than one operator in a tree representation of an expression. Superoperators chosen for a particular program allowed speedups of about a factor of two in an interpreter using switch dispatch. Switch dispatch is so expensive that almost anything that reduces the number of dispatches is worthwhile.

Gregg et al. [11] and Ertl et al. [7] presented a prototype interpreter based on the Cacao research JVM [14]. This interpreter was built using Vmgen and used the facility for generating superinstructions. With large numbers of superinstructions, reductions in running time of the order of one third were possible. Unfortunately, the system was rather unstable and could run only a handful of programs. It also did not support a number of language features such as multithreading and correct initialisation of classes. In contrast, the interpreter described in this paper is a full, stable version that fully supports the standard and runs all programs that we have tried.

6 Conclusion

We have described a system of superinstructions for a portable, efficient Java interpreter. Our interpreter generator automatically creates source code for superinstructions from instruction definitions. Stack access code is optimised to reuse the topmost stack items between the component instructions in a superinstruction. This can significantly reduce stack traffic. Furthermore, our interpreter generator optimises stack pointer updates by combining and possibly eliminating them across component instructions. Our interpreter generator also provides a profiling system to identify common sequences of instructions. Experimental results show that significant speedups of up to 90% are possible with large numbers of appropriate superinstructions, due to reduction in dispatches and optimised superinstruction code.

Although our superinstruction system is stable and gives speedups in most configurations, considerable work remains for the future. The most important future development will be a scheme to allow “quickable” instructions to participate in superinstructions. Many of the most frequently executed Java instructions such as field access (16.4% of executed instructions in SPECjvm98 [21]) and method invokes (5.7%) are “quickable”. We believe that allowing these instructions to participate in superinstructions will greatly increase the running speed of our interpreter. We also plan work in the area of better heuristics for choosing superinstruction, better parsing algorithms, and superinstructions across basic block boundaries.

References

1. J.R. Bell. Threaded code. *Commun. ACM*, 16(6):370–372, 1973.
2. T. Bell, J. Cleary, and I. Witten. *Text Compression*. Prentice Hall, 1990.
3. H. Eller. Threaded code and quick instructions for kaffe.
<http://www.complang.tuwien.ac.at/java/kaffe-threaded/>.
4. M.A. Ertl. Stack caching for interpreters. In *SIGPLAN '95 Conference on Programming Language Design and Implementation*, pages 315–327, 1995.
5. M.A. Ertl and D. Gregg. The behaviour of efficient virtual machine interpreters on modern architectures. In *Euro-Par 2001*, pages 403–412. Springer LNCS 2150, 2001.

6. M.A. Ertl and D. Gregg. Optimizing indirect branch prediction accuracy in virtual machine interpreters. In *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation (PLDI 2003)*, San Diego, California, June 2003. ACM. to appear.
7. M.A. Ertl, D. Gregg, A. Krall, and B. Paysan. *vmgen* — A generator of efficient virtual machine interpreters. *Software—Practice and Experience*, 32(3):265–294, 2002.
8. E. Gagnon. *A Portable Research Framework for the Execution of Java Bytecode*. PhD thesis, McGill University, December 2002.
9. E. Gagnon and L. Hendren. SableVM: A research framework for the efficient execution of Java bytecode. In *First USENIX Java Virtual Machine Research and Technology Symposium*, Monterey, California, April 2001.
10. E. Gagnon and L. Hendren. Effective inline-threaded interpretation of java bytecode using preparation sequences. In *Proceedings of the 12th International Conference on Compiler Construction*, LNCS 2622, pages 170–184, April 2003.
11. D. Gregg, A. Ertl, and A. Krall. Implementation of an efficient Java interpreter. In *Proceedings of the 9th High Performance Computing and Networking Conference*, LNCS 2110, pages 613–620, Amsterdam, The Netherlands, June 2001.
12. D. Gregg and J. Waldron. Primitive sequences in general purpose forth programs. In *18th Euroforth Conference*, pages 24–32, Vienna, Austria, September 2002.
13. J. Hoogerbrugge, L. Augusteijn, J. Trum, and R. van de Wiel. A code compression system based on pipelined interpreters. *Software—Practice and Experience*, 29(11):1005–1023, Sept. 1999.
14. A. Krall and R. Grafl. CACAO — a 64 bit JavaVM just-in-time compiler. In G. C. Fox and W. Li, editors, *PPoPP’97 Workshop on Java for Science and Engineering Computation*, Las Vegas, June 1997. ACM.
15. I. Piumarta and F. Ricciardi. Optimizing direct threaded code by selective inlining. In *SIGPLAN’98 Conference on Programming Language Design and Implementation*, pages 291–300, 1998.
16. T. A. Proebsting. Optimizing an ANSI C interpreter with superoperators. In *Principles of Programming Languages (POPL’95)*, pages 322–332, 1995.
17. V. Santos Costa. Optimising bytecode emulation for Prolog. In *LNCS 1702, Proceedings of PPDP’99*, pages 261–267. Springer-Verlag, September 1999.
18. SPEC. SPEC releases SPEC JVM98, first industry-standard benchmark for measuring Java virtual machine performance. Press Release, August 19 1998. <http://www.specbench.org/osg/jvm98/press.html>.
19. Sun Microsystems Inc. *Java 2 Platform Micro Edition (J2ME) Technology for Creating Mobile Devices*, May 2000.
20. K.S. Venugopal, G. Manjunath, and V. Krishnan. sEc: A portable interpreter optimizing technique for embedded java virtual machine. In *Second USENIX Java Virtual Machine Research and Technology Symposium*, San Francisco, California, August 2002.
21. J. Waldron. Dynamic bytecode usage by object oriented java programs. In *Proceedings of the Technology of Object-Oriented Languages and Systems 29th International Conference and Exhibition*, Nancy, France, June 7-10 1999.

Partitioning for DSP Software Synthesis

Ming-Yung Ko and Shuvra S. Bhattacharyya

Electrical and Computer Engineering Department, and

Institute for Advanced Computer Studies

University of Maryland, College Park, Maryland 20742, USA

Abstract. Many modern DSP processors have the ability to access multiple memory banks in parallel. Efficient compiler techniques are needed to maximize such parallel memory operations to enhance performance. On the other hand, stringent memory capacity is also an important requirement to meet, and this complicates our ability to lay out data for parallel accesses. We examine these problems, data partitioning and minimization, jointly in the context of software synthesis from dataflow representations of DSP algorithms. Moreover, we exploit specific characteristics in such dataflow representations to streamline the data partitioning process. Based on these observations on practical dataflow-based DSP benchmarks, we develop simple, efficient partitioning algorithms that come very close to optimal solutions. Our experimental results show 19.4% average improvement over traditional coloring strategies with much higher efficiency than ILP-based optimal partitioning computation. This is especially useful during design space exploration, when many candidate synthesis solutions are being evaluated iteratively.

1 Introduction

Limited memory space is an important issue in design space exploration for embedded software. An efficient strategy is necessary to fully utilize stringent storage resources. In modern DSP processors, the memory minimization problem must often be considered in conjunction with the availability of parallel memory banks, and the need to place certain groups (usually pairs) of storage blocks (program variables or arrays) into distinct banks. This paper develops techniques to perform joint data partitioning and minimization in the context of software synthesis from *Synchronous Dataflow (SDF)* specifications of DSP applications [10]. SDF is a high-level, domain specific programming model for DSP that is widely used in commercial DSP design tools (e.g., see [4][5]). We report on insights on program structure obtained from analysis of numerous practical SDF benchmark applications, and apply these insights to develop an efficient data partitioning algorithm that frequently achieves optimum results.

The assignment techniques that we develop consider variable-sized storage blocks as well as placement constraints for simultaneous bank accesses across pairs

of blocks. These constraints derive from the feature of simultaneous multiple memory bank accesses provided in many modern DSP processors, such as the Motorola DSP56000, NEC μ PD77016, and Analog Devices ADSP2100. These models all have dual, homogenous parallel memory banks. Memory allocation techniques that consider this architectural characteristic can employ more parallelism and therefore speed up execution. The issue is one of performing strategic *data partitioning* across the parallel memory banks to map simultaneously-accessible storage blocks into distinct memory banks. Such data partitioning has been researched for scalar variables and register allocation [7][8][18]. However, the impact of array size is not investigated in those papers. Furthermore, data partitioning has not been explored in conjunction with SDF-based software synthesis. The main contribution of this paper is in the development of novel data partitioning techniques for heterogeneous-sized storage blocks in the synthesis of software from SDF representations.

In this paper, we assume that the potential parallelism in data accesses is specified by a high level language, e.g., C. Programmers of the SDF *actor* (dataflow graph vertex) library provide possible and necessary parallel accesses in the form of language directives or pseudocode. Then the optimum bank assignment is left to software synthesis. Because of the early specifications, users can not foresee the parallelism that will be created by compiler optimization techniques, like code compaction and selection. It is neither our intention to explore such low level parallelism. From the benchmarks collected (in the form of undirected graphs), a certain structural pattern is found. The observations help in the analysis on practical applications and motivates a specialized, simple, and fast heuristic algorithm.

To describe DSP applications, dataflow models are quite often used. An application is divided into modules with data passing between modules. Modules receive input data and output results after processing. Data for module communication flows through and is stored in buffers. In dataflow semantics, buffers are allocated for every flow. In Section 4, it is demonstrated that the nature of buffers helps in optimizing parallel memory operations. SDF [5] for multirate applications is especially suitable for buffer analysis and is referenced in our discussion.

The paper is organized as follows. A brief survey of related work is in section 2. Detailed and formal descriptions of the problem are given in section 3. Some interesting observations on SDF benchmarks are presented in section 4. A specialized as well as a general case algorithm are provided in section 5. In section 6 are the experimental results and our conclusion.

2 Related Work

Due to performance concerns, embedded systems often provide heterogeneous data paths. These systems are generally composed of specialized registers, multiple memory modules, and address generators. The heterogeneity opens new research problems in compiler optimization.

One such problem is memory bank assignment. One early article of relevance on this topic is [15]. This work presents a naive alternating assignment approach. In [17], interference graphs are derived by analyzing possible dual memory accesses in

high level code. Interference edges are also associated with integer weights that are identical to the loop nesting depths of memory operations. The rationale behind the weight definition is that memory loads/stores within inner loops are called more frequently. The objective is to evaluate a maximum edge cut such that the induced node sets are accessed in parallel most often. A greedy heuristic is used due to the intractability of the maximum edge cut problem [9]. A similar problem is described in [11] though with an *Integer Linear Programming (ILP)* strategy employed instead.

Register allocation is often jointly discussed with bank assignment. These two problems lack orthogonality, and are usually closely related. In [18], a constraint graph is built after symbolic code compaction. Variables and registers are represented by graph nodes. Graph edges specify constraints according to the target architecture's data path as well as some optimization criteria. Nodes are then labelled under the constraints to reach lowest labelling cost. Because of the high intractability of the problem, a simulated annealing approach is used to compute solutions. In [8], an evolutionary strategy is combined with tree techniques and list scheduling to jointly optimize memory bank assignment and register allocation. The evolutionary hybrid is promising due to linear order complexity. Unlike phase-coupling strategies, a de-coupling approach is recently suggested in [7]. Conventional graph coloring is employed in this work along with maximum spanning tree computation.

While the algorithms described above are effective in parallel memory operations, array size is not considered. For systems with heterogeneous memory modules, the issue of variable size is important when facing storage capacity limitations. Generally, the optimization objective aims at promoting execution performance. Memory assignment is done according to features (e.g., capacity and access speed) of each module to determine a best running status [1]. Configurability of banks is examined in [12] to achieve an optimum working configuration. Furthermore, trade-offs between on-chip and off-chip memory data partitioning are researched in [14]. Though memory space occupation is investigated in those papers, parallel operations are not considered. The goal is to leverage overall execution speed-up by exploiting each module's advantage.

A similar topic, termed memory bank disambiguation, can be found in the field of multiple processor systems. The task is to determine which bank a memory reference is accessing at compile-time. One example is the compiler technique for the RAW architecture from MIT [3]. The architecture of RAW is a two-dimensional mesh of tiles and each tile is composed of a processor and a memory bank. Because of the capability of fast static communication between tiles, fine-grained parallelism and quick inter-bank memory accesses can be accomplished. Memory bank disambiguation is rendered in compile time to support static memory parallelism as much as possible. Since each memory bank is with a processor, concurrent execution is assumed. Program segments as well as data layout are distributed in the disambiguation process. In other words, the design of RAW targets scalable processor level parallelism, which contrasts to our work of instruction level parallelism intrinsically.

In the data and memory management literature, manipulation of arrays is generally at a high level. Source analysis or transformation techniques are applied well before assembly code translation. Some examples are the heterogeneous memory discussion in [1][12]. For general discussions regarding space, such as storage estimation, sharing of

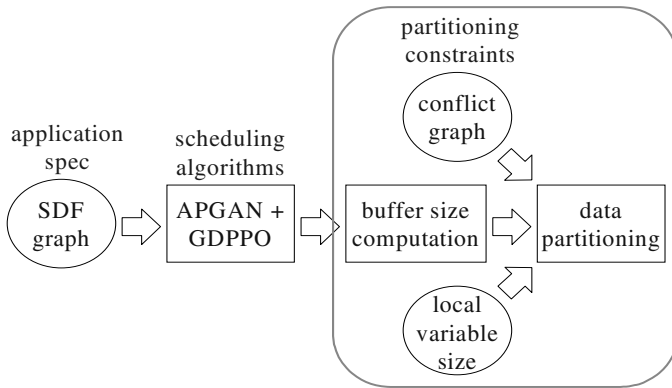


Fig.1. Overview of SDF-based software synthesis

physical locations, lifetime analysis, and variable dependencies, arrays are examined in high level code quite often [13]. This fact demonstrates the efficacy to explore arrays at the high level language level, which we explore in this paper as well.

3 Problem Formulation

Given a set of variables along with the size, we would like to calculate an optimum bank assignment. It is assumed that there are two homogeneous memory banks of equal capacity. This assumption is practical and similar architectures can be found in products such as the Motorola DSP56000, NEC μ PD77016, and Analog Devices ADSP2100. Each bank can be independently accessed in parallel. Such parallelism for memories enhances execution performance. The problem then is to compute a bank assignment with maximum simultaneous memory accesses and minimum capacity requirement.

To demonstrate an overview of our work, an SDF-based software synthesis process is drawn in Figure . First, applications are modeled by SDF graphs, which are effective at representing multirate signal processing systems. Scheduling algorithms are then employed to calculate a proper actor execution order. The order has significant impact on actor communication buffer sizes and makes scheduling a non-trivial task. For scheduler selection, APGAN and GDPPO are proven to reach certain lower bounds on buffer size if they are achievable [5]. Possible simultaneous memory accesses, partitioning constraints in the figure, together with actor communication buffer sizes and local state variable sizes in actors are then passed as inputs to data partitioning. Our focus in this paper is on the rounded rectangle part in [].

One important consideration is that scalar variables are not targeted in this research. Mostly, they are translated to registers or immediate values. Compilers generally do so to promote execution performance. Memory cost is primarily due to arrays or consecutive data. As we described earlier, therefore, scalar variables and registers are often managed together. Since we are addressing data partitioning at the

system design level, consecutive-data variables at a higher level in the compilation process are our major concern in this work.

The description above can be formalized in terms of graph theory. First, we build an undirected graph, called a *conflict graph* (e.g., see [7][11] for elaboration), $G = (V, E)$, where V and E are sets of nodes and edges respectively. Variables are represented by nodes and potential parallel accesses by edges. There is an integer weight $w(v)$ associated with every node $v \in V$. The value of a weight is equal to the size of the corresponding variable.

The problem of bank assignment, with two banks, is to find a disjoint bi-partition of nodes, P and Q , with each associated to one bank. The subset of edges with end nodes falling in different partitions is called an *edge cut*. Edge cut χ is formally defined as $\chi = \{e \in E | (v' \in P) \wedge (v'' \in Q)\}$ where v' and v'' are endpoints of edge e . Since a partition implies a collection of variables assigned to one bank, elements of the edge cut are the parallel accesses that can be carried out. Conversely, parallel accesses are not permissible for edges that do not fall in the edge cut. We should note that edges in the conflict graph represent possible parallelism in the application, and are not always achievable in any solution. Therefore, the objective is to maximize the cardinality of χ .

The other goal is to find minimum capacity requirement. Because of homogeneous size in both banks, we aim at storage balancing as well. That is, the capacity requirement is exactly the largest space occupation of either bank. Let $C(P)$ denote the total space cost of bank P . It is defined as

$$C(P) = \sum_{\forall v \in P} w(v).$$

Cost $C(Q)$ is defined in the same way. The objective is to reduce the capacity requirement M under the constraints of $C(P) \leq M$ and $C(Q) \leq M$. In summary, we have two objectives to optimize the partitioning problem:

$$\min(M) \text{ and } \max |\chi|. \quad (1)$$

Though there are two goals, priority is given to aximum? in decision making. When there are contradictions between the objectives, a solution with maximum parallelism is chosen. In the following, we work on parallelism exploration first and then on examination of capacity. Alternatively, parallelism can be viewed as a constraint to fit. This is the view taken in the ILP approach proposed later.

Variables can be categorized as two types. One is actor communication buffers and the other is state variables local to actors. Buffers are for message passing in dataflow models and management over them is important for multirate applications. SDF offers several advantages in buffer management. One example is space minimization under single appearance scheduling constraint. As mentioned earlier, the APGAN and GDP-PO algorithms in [5] are proven to reach a lower bound on memory requirements under certain conditions. However, buffer size is not our primary focus in this work though we do apply APGAN and GDPPO as part of the scheduling phase. The other type, state variables, is local and private to individual actors. State variables act as internal temporary variables or parameters in implementation and are not parts of dataflow expression. In this paper, however, variables are not distinguished by types. Types are merely mentioned to explain the source of variables in dataflow programs.

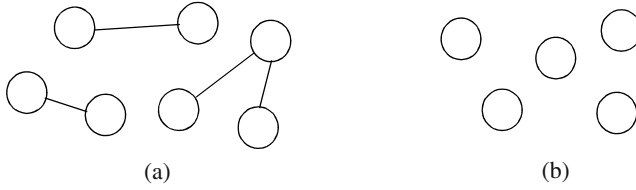


Fig. 2. Features of conflict graph connected components extracted from real applications. (a) short chains (b) trivial components, single nodes without edges

4 Observations on Benchmarks

We have found that benchmarks, in the form of conflict graphs, derived from several applications (provided in section 6) have sparse connections. For example, a convolution actor involves only two arrays in simultaneous accesses. Other variables to maintain temporary values, local states, loop control, etc. are not apparently beneficial, though no harm is inflicted either, if they are accessed in parallel.

Connected components (abbreviated as *CGCC*, *Conflict Graph Connected Component*) of benchmarks also tend to be acyclic and bipartite. We say a graph is *bipartite* if the node set can be partitioned into two sets such that all edges are with end nodes falling in distinct node partitions. This is good news to graph partitioning. Most of them have merely two nodes with a connecting edge. For those a bit more complicated, short chains account for the major structure. There are also many trivial CGCCs containing one node each and no edges. Typical topologies of CGCCs are illustrated in [] and an example is in []. Variable *signalIn* in [] is an input buffer of the actor and its size is to be decided by schedulers. Variables like *hamming* and *window* are arrays internal to the actor. For each iteration of the loop, *signalIn* and *hamming* are fetched to complete the multiplication and qualify for parallel accesses.

The characteristic of loose connectivity appears to high level relationships among consecutive-data variables. Though we did not investigate characteristics of the connectivity in the scalar case, it is believed that the connectivity is much more complicated than what we observe for arrays. In [7], though, the authors mention that the whole graph may not be connected and multiple connected components exist, and a heuristic approach is adopted to cope with complex topologies of the connected components. The topologies derived in [18] should be even more intricate because

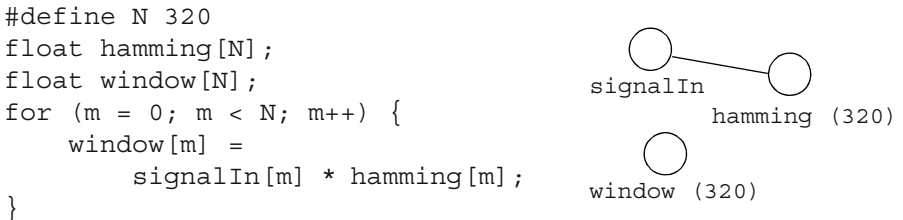


Fig. 3. A conflict graph example of an actor that windows input signals

more factors are considered. Readers are reminded here once again that only arrays are focused on at high level in our context of combined memory minimization and data partitioning.

Another contribution to loose connectivity lies in the nature of coarse-grain dataflow graphs. Actors of dataflow graphs communicate with each other only through communication buffers represented by edges. State variables internal to an actor are inaccessible and invisible to that of other actors. This feature forces modularity of dataflow implementation and causes numerous CGCCs. Moreover, except for communication buffer purposes, any global variables are disallowed. This prevents their occurrences in arbitrary numbers of routines and hence reduces conflicts across actors. Furthermore, based on our observations, communication buffers contribute to conflicts mostly in read accesses. In other words, buffer writing is usually not found in parallel with other memory accesses. The phenomenon is natural in single assignment semantics.

In [3], to facilitate memory bank disambiguation, information about aliased memory references is required. To determine aliases, pointer analysis is performed. The analysis results are then represented by a directional bipartite graph. The graph nodes could be memory reference operations or physical memory locations. The edges are directed from operations to locations to indicate dependencies. The graph is partitioned into connected components, called *Alias Equivalence Classes (AEC)*, where any alias reference can only occur in a particular class. AECs are assigned to RAW tiles so that tasks are done independently without any inter-tile communication. [] is given to illustrate the concept of AECs. For the sample C code in (a), variable *b* is aliased by *x*. Memory locations and referencing code are expressed by a directional bipartite graph in (b). Parenthesized integers next to variables are memory location numbers (or addresses) and *b* and *x* are aliased to each other with identical location number 2. The connected component in (b) is the corresponding AEC of (a).

A relationship exists between AEC and CGCC, keeping in mind that conflict edges indicate concurrent accesses to two arrays. All program instructions issuing accesses to either array are grouped to an identical alias equivalence class. Therefore, both arrays can be found exclusively in that class. In other words, the node set of a CGCC can appear only in a certain single AEC instead of multiple ones. Take [] as an example. The node set in (c) can be found only in the node set of (b). For an application, therefore, the number of CGCCs is greater than or equal to that of AECs. The relationship between

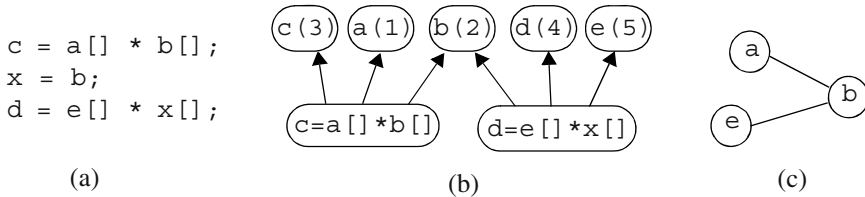


Fig. 4. Example of the relationship between AECs and CGCCs. (a) sample C code, (b) AEC, (c) CGCC

AEC and CGCC makes it promising in the automatic derivation of conflict graphs. This is an interesting topic for further work.

It is found in [3] that practical applications have several AECs. According to the relationship revealed in the previous paragraph, the number of CGCCs is bigger. If the modularity of dataflow semantics is considered, the number is even bigger. The fact of multiple AECs backs our discovery of numerous CGCCs and loose connectivity. However, the counts of AEC are not related to the simple topology, as demonstrated in [], of CGCC. Due to the feasibility of reducing CGCC from AEC, we believe that the graph structure of CGCC is much simpler than that of AEC.

5 Algorithms

In this section, three algorithms are discussed. The first one is a 0/1 ILP approach, where all ILP variables are restricted to values 0 or 1. The second one is a coloring method, which is a typical strategy from the relevant literature. The third one is a greedy algorithm that is motivated by our observations on the structure of practical, SDF-based conflict graphs.

5.1 ILP

In this subsection, a 0/1 ILP strategy [2] is proposed to solve benchmarks with bipartite structure. Constraint equations are made for the bipartite requirement. If the conflict graph is not bipartite, it is rejected as failure. Fortunately, most benchmarks are bipartite according to our observations. On the other hand, the objective $\min(M)$ in equation (1) is translated to minimizing space cost difference, $\min|C(Q) - C(P)|$, due to ILP restriction on single optimization equation. For each array u , there is an associated bank assignment b_u to be decided and $b_u \in \{0, 1\}$. Values of b_u denote banks, say B_P and B_Q respectively. A constant integer z_u denotes the size of array u . Memory parallelism constraints $b_u + b_y = 1$ are imposed if arrays u and y are to be accessed simultaneously and these constraints also act as the bipartite requirement. The constraints guarantee that distinct banks are assigned to the variables. Let us denote D as the capacity exceeding amount of bank B_Q beyond B_P . That is,

$$D = \sum_{\forall y \in B_Q} z_y - \sum_{\forall u \in B_P} z_u.$$

This equation can be further decomposed as follows.

$$\begin{aligned} D &= \sum_{\forall y \in B_Q} z_y \cdot 1 + \sum_{\forall u \in B_P} z_u \cdot (0 - 1) \\ &= \sum_{\forall y \in B_Q} z_y b_y + \sum_{\forall u \in B_P} z_u (b_u - 1) \\ &= \sum_{\forall y} z_y b_y + \sum_{\forall u} z_u (b_u - 1) \\ &= \sum_{\forall u} (z_u b_u + z_u (b_u - 1)) \end{aligned}$$

$$= \sum_{\forall u} z_u(2b_u - 1)$$

Finally, we end up with

$$D = 2 \sum_{\forall u} z_u b_u - \sum_{\forall u} z_u.$$

Since the goal is to minimize the absolute value of D , one more constraint $D \geq 0$ is also required.

5.2 2-Coloring and Weighted Set Partitioning

A traditional coloring approach is partially applicable for our data partitioning problem in equation [1]. If colors represent banks, a bank assignment is achieved while the coloring is done. Though minimum coloring is an NP-hard problem, it becomes polynomially solvable for the case of two colors [9]. However, using a two-coloring approach, only the problem of simultaneous memory access is handled. Balancing of memory space costs is left unaddressed.

To cover space cost balancing, it is necessary to incorporate an additional algorithm. Among those integer set or weighted set problems that are similar to balancing costs, *weighted set partitioning* is chosen in our discussion because it searches for a solution with exact balancing costs. Weighted set partitioning states that: given a finite set A and a size $s(a) \in \mathbb{Z}^+$ for each $a \in A$, is there a subset $A' \subseteq A$ such that

$$\sum_{a \in A'} s(a) = \sum_{a \in A - A'} s(a) \quad ?$$

This problem is NP-hard [9]. If conflicts are ignored, balancing space costs can be reduced from weighted set partitioning and therefore balancing space costs with conflicts considered is NP-hard as well (see Appendix A).

5.3 SPF — A Greedy Strategy

In this section, we develop a low-complexity heuristic called *SPF (Smallest Partition First)* for the heterogeneous-size data partitioning problem. Although 0/1 ILP calculates exact solutions, its complexity is non-polynomial, and therefore its use is problematic within intensive design space exploration loops, and for very large applications, it may become infeasible altogether. Coloring and weighted set partitioning each compute partial results. In addition, the efficacy of coloring is on heavily connected graphs. With the observations of loose connectivity in practice, coloring does not offer much contribution. A combination of coloring and weighted set partitioning would be interesting and is left as future work. In this article, the heuristic of SPF is proposed instead that is tailored to the restricted nature of SDF-based conflict graphs. The results and performance will be compared to that of 0/1 ILP and coloring in the next section.

A pseudocode specification of the SPF greedy heuristic is provided in Figure . Connected components or nodes with large weights are assigned first to the bank with least space usage. Variables of smaller size are gradually filled to narrow the space cost gap between banks. The assignment is also interleaved to maximize memory parallelism. Note that the algorithm is able to handle an arbitrary number of memory banks, and is

```

procedure SPFDataPartitioning
  input: a conflict graph  $G = (V, E)$  with integer node
  weights  $W(V)$  and an integer constant  $K$  representing
  the number of banks.
  output: partitions of nodes  $B[1...K]$ .
  set an array  $B[1...K]$  of  $K$  node sets (banks).
  set  $C$  to connected components of  $G$ .
  sort  $C$  in decreasing order on total node weights.
  for each connected component  $c \in C$ 
    get the node  $v \in c$  with largest weight.
    call  $\text{AlternateAssignment}(v)$ .
  end for
  output array  $B[1...K]$ .

```

```

procedure AlternateAssignment
  input: a node  $v$ .
  set a boolean variable assigned to false.
  sort  $B$  in increasing order on total node weights.
  for each node set  $B[i]$ 
    if no  $u \in B[i]$  such that  $u$  is a neighbor of  $v$ 
      add  $v$  to  $B[i]$ .
      assigned  $\leftarrow$  true.
      quit the for loop.
    end if
  end for
  if assigned = false
    add  $v$  to the smallest set,  $B[1]$ .
  end if
  call  $\text{ProcessNeighborsOf}(v)$ .

```

```

procedure ProcessNeighborsOf
  input: a node  $v$ .
  for each neighbor  $b$  of  $v$ 
    if node  $b$  has not been processed
      call  $\text{AlternateAssignment}(b)$ .
    end if
  end for

```

Fig. 5. Our data partitioning algorithm (SPF) for consecutive-data variables

applicable to non-bipartite graphs. Thus, it provides solutions to any input application with arbitrary bank count.

The SPF algorithm achieves a low computational complexity solution. In the pseudocode specification, the procedure *AlternateAssignment* performs the major function of data partitioning and is called exactly once for every node in a recursive style through *ProcessNeighborsOf*. First, the bank array $B[1 \dots K]$ is sorted in *AlternateAssignment* according to present storage usage. After that, internal edges linked to the input node are examined for every bank, keeping in mind that only cut edges are desired. The last step is querying the assignment of neighbor nodes and a recursive call. Therefore, the complexity of *AlternateAssignment* is $O(K \log K + KN + N)$, where N denotes the largest node degree in the conflict graph. Though the practical complexity of N can be $O(1)$ according to our observations, the worst case is of $O(|V|)$ complexity. In our assumption, K is a constant provided by the system. For the whole program execution, all calls to *AlternateAssignment* contribute $O(|V|^2)$ in worst case and $O(|V|)$ in practice. The remaining computations in SPF include strongly connected component decomposition, sorting connected components by total node weights, and building neighbor node lists. Their complexities are $O(|V| + |E|)$ [19], $O(|C| \log |C|)$, and $O(|E|)$, respectively ($|C|$ denotes the number of connected components in the conflict graph.). In summary, the overall computational complexity is $O(|V|^2)$ in worst case and practically $O(\max(|V| + |E|, |C| \log |C|))$ for several real applications.

6 Experimental Results

Our experiments are performed for all three algorithms: ILP, 2-coloring, and our SPF algorithm. Since all conflict graphs from our benchmarks are bipartite, every edge falls in the edge cut and memory parallelism is maximized by all three algorithms. Therefore, only the capacity requirement is chosen as our comparison criteria. Improvement is evaluated for SPF over 2-coloring, a classical bank assignment strategy. Performance of SPF is also compared to that of ILP to give an idea of the effectiveness of SPF. For ILP computation, we use the solver OPBDP which is an implementation based on the theories of [2].

To decide bank assignment for coloring, the first node of a connected component is always fixed to the first bank and the remaining nodes are typically assigned in an alternate way because of the commonly-found bipartite graph structure. The order in which the algorithm traverses the nodes of a graph is highly implementation dependent and the result depends on this order. Thus, some results may become better while others may become worse if another ordering is tried. However, the average improvement of SPF is still believed to be high since numerous applications have been considered in the experiments with our implementation.

A summary of the results is given in Table . The first column lists all the benchmarks that were used in our experiments. The second and third columns provide the number of variables and parallel accesses, respectively. Since the benchmarks are in the format of conflict graphs, those two columns represent node and edge counts, too. The fourth to sixth columns give the bank capacity requirement for each of the three algo-

rithms. Capacity reduction for SPF over 2-coloring is placed in the last column as an improvement measure.

Table 1. Summary of the experimental results

	variable counts	conflict counts	coloring	SPF	ILP	improve- ment(%)
analytic	9	3	756	448	448	40.7
bpsk10	22	8	140	90	90	35.7
bpsk20	22	7	240	156	156	35.0
bpsk50	22	8	300	228	228	24.0
bpsk100	22	8	500	404	404	19.2
cep	14	2	1602	1025	1025	36.0
cd2dat	15	7	1459	1343	1343	8.0
dat2cd	10	5	412	412	412	0.0
discWavelet	92	56	1000	999	999	0.1
filterBankNU	15	10	196	165	164	15.8
filterBankNU2	52	27	854	658	658	23.0
filterBankPR	92	56	974	851	851	12.6
filterBankSub	54	32	572	509	509	11.0
qpsk10	31	16	173	146	146	15.6
qpsk20	31	14	361	277	277	23.3
qpsk50	31	16	453	426	426	6.0
qpsk100	31	16	803	776	776	3.4
satellite	26	9	1048	771	771	26.4
telephone	11	2	1633	1105	1105	32.3
Average						19.4

Most of the benchmarks are extracted from real applications in the Ptolemy environment [6]. Ptolemy is a design environment for heterogeneous systems and many examples of real applications are also included. A brief description of all the benchmarks follows. Two of them are rate converters, *cd2dat* and *dat2cd*, between CD and DAT devices. Filter bank examples are *filterBankNU*, *filterBankNU2*, *filterBankPR*, and *filterBankSub*. The first two are two-channel non-uniform filter banks with dif-

ferent depths. The third one is an eight-channel perfect reconstruction filter bank, while the last one is for four-channel subband speech coding with APCM. Modems of BPSK and QPSK are *bpsk10-100* and *qpsk10-100* with various intervals. A telephone channel simulation is represented by *telephone*. Filter stabilization using cepstrum is in *cep*. An analytic filter with sample rate conversion is *analytic*. A satellite receiver abstraction, *satellite*, is obtained from [16]. Because *satellite* is just an abstraction without implementation details, reasonable synthetic conflicts are added according to our benchmark observations.

Table demonstrates the performance of SPF. Not only does it generate less capacity requirement than the classical coloring method does, but also the results are almost equal to the optimality evaluated by ILP. The polynomial computational complexity (see subsection 5.3) is also lower than the exponential complexity of ILP. In our ILP experiments on a 1GHz Pentium III machine, most of the benchmarks finish within a few seconds. However, *discWavelet* and *filterBankPR* spend several hours to complete. In contrast, SPF finishes in less than ten seconds for all cases. In summary, SPF is effective both in the results and the computation time.

7 Conclusion

Bank assignment for arrays has great impact both on parallel memory accesses and memory capacity. Traditional bi-partitioning or two coloring strategies for scalar variables cannot be well adapted to applications with arrays. The variety of array sizes complicates memory management especially for typical embedded systems with stringent storage capacity. We propose an effective approach to jointly optimize memory parallelism and capacity when synthesizing software from dataflow graphs.

Surprisingly but reasonably, high level analysis presents a distinctive type of graph topology for real applications. Graph connections are sparse and connected components are in the form of chains, bipartite connected components, or trivial singletons.

Some possible future works follow. Our SPF algorithm generates results quite close to optimality. We are curious about the efficacy to graphs with arbitrary topology. Sparse connections found in dataflow models also arouses our interests in the applicability to procedural languages like C. Integration of high and low level optimization is also promising. An integrated optimization scheme involving arrays, scalar variables, and registers is a particularly useful target for further study. Automating conflict information through alias equivalence class calculation is a possible future work as well. Another potential work is to reduce storage requirements further by sharing physical space among variables whose lifetimes do not overlap.

Acknowledgement

This research was supported by the Semiconductor Research Corporation (2001-HJ-905).

References

- [1] O. Avissar, R. Barua, and D. Stewart. Heterogeneous Memory Management for Embedded Systems. CASES 2001, pp. 34-43, Atlanta, November 2001.
- [2] Peter Barth. *Logic-Based 0-1 Constraint Programming*. Kluwer Academic Publishers. 1996.
- [3] R. Barua, W. Lee, S. Amarasinghe, and A. Agarwal. Compiler Support for scalable and Efficient Memory Systems. *IEEE Transactions on Computers*, 50(11):1234-1247, November 2001.
- [4] S.S. Bhattacharyya, R. Leupers, and P. Marwedel. Software synthesis and code generation for DSP. *IEEE Transactions on Circuits and Systems - II: Analog and Digital Signal Processing*, 47(9):849-875, September 2000.
- [5] S.S. Bhattacharyya, P.K. Murthy, and E. A. Lee. *Software Synthesis from Dataflow Graphs*. Kluwer Academic Publishers. 1996.
- [6] J.T. Buck, S. Ha, E.A. Lee, and D.G. Messerschmitt. Ptolemy: A Framework for Simulating and Prototyping Heterogeneous Systems. *International Journal of Computer Simulation*, 4:155-182, April 1994.
- [7] J. Cho, Y. Paek, and D. Whalley. Efficient Register and Memory Assignment for Non-orthogonal Architectures via Graph coloring and MST Algorithms. LCTES 2002-SCOPES 2002, pp. 130-138, Berlin, June 2002.
- [8] S. Frohlich and B. Wess. Integrated Approach to Optimized Code Generation for Heterogeneous-Register Architectures with Multiple Data-Memory Banks. *Proceedings of IEEE 14th Annual ASIC/SOC Conference*, pp. 122-126, Arlington, September 2001.
- [9] M.R. Garey and D.S. Johnson. *Computers and Intractability*. W. H. Freeman. 1979.
- [10] E.A. Lee and D.G. Messerschmitt. Synchronous dataflow. *Proceedings of the IEEE*, 75(9):1235-1245, September 1987.
- [11] R. Leupers and D. Kotte. Variable Partitioning for Dual Memory Bank DSPs. ICASSP, Salt Lake City, May 2001.
- [12] P.R. Panda. Memory Bank customization and Assignment in Behavioral Synthesis. *Proceedings of IEEE/ACM International Conference on Computer-Aided Design*, pp. 477-481, San Jose, November 1999.
- [13] P.R. Panda, F. Catthoor, N.D. Dutt, et. al. Data and Memory Optimization Techniques for Embedded Systems. *ACM Transactions on Design Automation for Electronic Systems*, 6(2):149-206, April 2001.
- [14] P.R. Panda, N.D. Dutt, and A. Nicolau. On-Chip vs. Off-Chip Memory: The Data Partitioning Problem in Embedded Processor-Based Systems. *ACM Transactions on Design Automation of Electronic Systems*, 5(3):682-704, July 2000.
- [15] D.B. Powell, E.A. Lee, and W.C. Newman. Direct Synthesis of Optimized DSP Assembly Code from Signal Flow Block Diagrams. ICASSP'92, 5:23-26, March 1992.
- [16] S. Ritz, M. Willems, and H. Meyr. Scheduling for Optimum Data Memory Compaction in Block Diagram Oriented Software Synthesis. ICASSP'95, pp. 2651-2654, May 1995.
- [17] M.A.R. Saghir, P. Chow, and C.G. Lee. Exploiting Dual Data-Memory Banks in Digital Signal Processors. *Proceedings of the 7th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 234-243, October 1996.
- [18] A. Sudarsanam and S. Malik. Simultaneous Reference Allocation in Code Generation for Dual Data Memory Bank ASIPs. *ACM Transactions on Design Automation of Electronic Systems*, 5(2):242-264, April 2000.
- [19] R.E. Tarjan. Depth first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146-160, 1972.

Appendix A: NP-Hardness Proof

In this section, we establish the NP-hardness of the data partitioning problem addressed in this paper. As described earlier, data partitioning involves both bi-partitioning a graph and balancing of node weights. In other words, it is a combination of graph 2-coloring and weighted set partitioning, where the second problem is NP-hard. Therefore, for simplicity, we only prove that balancing node weights is NP-hard. Equivalently, we establish NP-hardness for the special case of data partitioning instances that have no conflicts.

The problem of space balancing is defined in section [3] and the objective is to minimize the capacity requirement M . The decision version of the optimization problem is to check whether both $C(P) \leq M$ and $C(Q) \leq M$ hold for a given constant integer M . In the following paragraphs, we demonstrate the NP-hardness reduction from a known NP-hard problem, weighted set partitioning.

Weighted set partitioning states that: given a finite set A and a size $s(a) \in \mathbb{Z}^+$ for each $a \in A$, is there a subset $A' \subseteq A$ such that

$$\sum_{a \in A'} s(a) = \sum_{a \in A - A'} s(a) \quad ? \quad (2)$$

The decision version of our space balancing problem can be rewritten as: Given a set of arrays U , the associated size $z(u) \in \mathbb{Z}^+$ for every $u \in U$, and a constant integer $M > 0$, is there a subset $U' \subseteq U$ such that

$$\sum_{u \in U'} z(u) \leq M \text{ and } \sum_{u \in U - U'} z(u) \leq M \quad ? \quad (3)$$

Now given an instance (A, s) of weighted set partitioning, we derive an instance of space balancing by first setting

$$M = \left\lfloor \frac{\sum_{a \in A} s(a)}{2} \right\rfloor. \quad (4)$$

Then, for every element $a \in A$, we can have a corresponding array u and U is the set of all u . Moreover, $z(u) = s(a)$ for each corresponding pair of u and a . If a subset A' exists to satisfy equation (2), the corresponding U' also makes equation (3) true. If a subset of arrays U' exists for equation (3), the corresponding A' also makes (2) true because

$$\sum_{a \in A'} s(a) + \sum_{a \in A - A'} s(a) = \sum_{a \in A} s(a),$$

where

$$\sum_{a \in A'} s(a) \leq \left\lfloor \frac{\sum_{a \in A} s(a)}{2} \right\rfloor \text{ and } \sum_{a \in A - A'} s(a) \leq \left\lfloor \frac{\sum_{a \in A} s(a)}{2} \right\rfloor.$$

The above arguments justify the necessary and sufficient conditions of the reduction from equation (2) to (3).

Efficient Variable Allocation to Dual Memory Banks of DSPs

Viera Sipkova

CD-Lab Compilation Techniques for Embedded Processors
Institut für Computersprachen, Technische Universität Wien
Argentinierstraße 8, A-1040 Vienna, Austria
Tel.: (+43-1)-58801-58520
`sipka@complang.tuwien.ac.at`

Abstract. To improve the overall performance, many of the modern advanced digital signal processors (DSPs) are equipped with on-chip multiple data memory banks which can be accessed in parallel in one instruction. In order to effectively exploit this architectural feature, the compiler must partition program variables between the memory banks appropriately – two parallel memory accesses always must take place on different memory banks. There is some research work that addresses this issue, however, most of this has been proposed as a post-pass (machine dependent) optimization. We attempt to resolve this problem by applying an algorithm which operates on the high-level intermediate representation, independent of the target machine. The partitioning scheme is based on the concepts of the interference graph which is constructed utilizing the control flow, data flow, and alias information. Partitioning of the interference graph is modeled as a Max Cut problem. The variable partitioning algorithm has been designed as an optional optimization phase integrated in the C compiler for a digital signal processor. This paper describes our efforts. The experimental results demonstrate that our partitioning algorithm finds a fairly good assignment of variables to memory banks. For small kernels from the DSPstone benchmark suite the performance is improved from 10% to 20%, for FFT filters by about 10%.

1 Introduction

To improve the effective bandwidth and memory access speed, recently, designers of embedded systems prefer the on-chip memory over the use of the external memory or more complicated hardware mechanisms. They have developed special architectural features to access multiple data memories in parallel, provided that referenced variables have been allocated to different memory banks. Furthermore, the instruction set may encode parallel accesses in a single instruction word, which improves the code density and reduces the code size. Examples of processors which support such memory architecture include the Motorola DSP56000, Analog Devices ADSP2106x, NEC μ PD77016, etc. In this research we will be using the experimental digital signal processor xDSPcore [1].

Unfortunately, the current compiler technology is generally unable to deliver high-quality code for DSPs whose architectures are extremely irregular. High-level C data types and language constructs are not easily mapped into dedicated DSP machine instructions. The reason is a lack of suitable optimization techniques. Much of the research for optimizing compilers has been done for general-purpose microprocessors and has focused on traditional machine-independent optimizations. Producing a high-performance code for DSPs requires adequate support for each specialized architectural feature.

The goal of this paper is to present the new optimization technique which attempts to maximize the benefit of dual data-memory bank DSPs. In order to make an efficient use of the bandwidth increase offered by dual memory banks (often denoted by X and Y), the C program variables have to be partitioned appropriately between X and Y.

```
int a[100], b[100];
int dot_product(void)
{
    int dot = 0;
    for (i = 0; i < 100; i++)
        dot += a[i] * b[i];
    return dot;
}
```

Fig. 1. Dot Product (C code)

Multi-memory bank architectures have been proved to be effective for many operations commonly found in embedded applications. For instance, in a dot product operation shown in Fig. 1 arrays **a** and **b** must be placed in different memory banks for allowing simultaneous access. The corresponding assembly code looks as outlined in Fig. 2. The notation **||** denotes that the combined operations should be executed in parallel. The instruction (3) performs both *loads*.

To solve the problem of memory assignment several approaches are possible at different stages of compilation flow. Our partitioning technique has been designed as a separate optimization module of the C compiler for the xDSPcore. It operates on the high-level intermediate representation, so it is not dependent on the target-machine. The result of the partitioning is the intermediate representation annotated with the X/Y bank assignment information for all variables. This can be utilized later in the subsequent code generation phase. The main scheme of our approach is similar to that proposed in [2]. It is modeled by a graph which tries to reflect all the potential parallelisms between the variables and also provides a weight metric for different parallel access demands. The partitioning itself is solved as the combinatorial optimization problem *Max Cut*, which is known as NP-complete. To find a near optimal partitioning we have implemented several partitioning algorithms, exact and also approximating.

```

(1)  movcl  g_b,R1    ||  movc   0,D0
(2)  movcl  g_a,R0    ||  bkrep  100,LBL
(3)  ld     (R0)+,D2  ||  ld     (R1)+,D1
(4)  nop
(5)  mul    D2,D1,A1
(6)  nop
(7)  add    D0,D2,D0
(8)  LBL:
(9)  ret

```

Fig. 2. Dot Product (assembly language)

The structure of the paper is organized as follows. In Section 2 a brief summary of the previous work is presented. In Section 3 the partitioning strategy is described. Section 4 provides our experimental results, and finally, Section 5 presents conclusions and future plans.

2 Related Work

The earliest work on this problem was presented by **Powell, Lee, and Newman** [3]. Here, the assignment of program variables to the X/Y memory banks occurs on the meta-assembly code, after the scheduling and register allocation phase. Variables are assigned to X and Y in an alternating fashion, according to their access sequence in the program code, without any analysis.

In the work of **Saghir, Chow, and Lee** [4,5] a variable partitioning technique for a hypothetical VLIW DSP architecture is presented. They describe two algorithms: compaction-based data partitioning, and partial data duplication. Both are performed as the post-pass phase operating only on basic blocks. The central data structure is an *interference graph*, whose nodes are partitioned into two sets heuristically, by searching for the minimum-cost partitioning.

In the approach of **Sudarsanam and Malik** [6,7] the memory bank allocation and register allocation take place in a single phase, after a pre-compaction step of the input program producing the symbolic assembly code. The algorithm is based on *graph labeling*, the objective of which is to find an optimal labeling of a constraint graph representing conditions on the register and memory bank allocation. The simulated annealing is used to find a good labeling.

In the work of **Leupers and Kotte** [2] the variable partitioning is performed as a separate optimization phase after the initial run of the backend used only to determine the exact set of memory accesses. The variable partitioning is modeled as Integer Linear Programming based on the *interference graph*.

The most recent papers concerning the problem of the memory banks assignment are probably [8,9,10].

Cho, Paek, and Whalley [8] presented a work where they study the memory and register allocation for non-orthogonal architectures. Memory bank as-

signment is done after the code compaction phase. For partitioning they use a heuristic that chooses the *maximum spanning tree* of the *simultaneous reference graph*. Then X memory is assigned in even depth and Y memory in odd depth in this tree.

Zhuang, Pande, and Greenland [9] proposed a post-register allocation solution which attempts to maximally combine loads and stores to generate parallel load/store instructions after code is generated. They introduce the *motion schedule graph*, which is partitioned applying the two-coloring algorithm.

The work of Zhuge, Xiao, and Sha [10] describes two algorithms: variable partitioning and scheduling with variable re-partition. The idea here is to reveal the true picture of potentially parallel memory accesses that can really occur in scheduling. The problem is modeled by the *variable independence graph* refined by a *mobility window* used by eliminating these edges that are impossible to be scheduled in the same control step. To partition the graph into multiple disjoint sets a greedy strategy is used.

In all previous work some kind of graphs have been used which are partitioned applying different optimization methods. However, all (except of [2]) have been proposed as a post-pass backend phase operating on the assembly code. This has a benefit that all memory accesses can be captured, however, generally, it can not be performed separately without any impact on the register allocation and scheduling.

In our approach the algorithm operates on high level intermediate representation. To find any potential parallelism between memory accesses information from all the sophisticated program analysis are possible to be utilized. Our framework is global (intra-procedural) and is not just limited to basic blocks. Memory accesses of the entire program are handled and relations between them are analyzed at once, so no contrary demands on assigning a certain variable to either X or Y can arise. Surely, it is not always possible to recognize all memory accesses, however, as will be reported later in this paper, our performance results are quite encouraging.

3 Partitioning Scheme

The C compiler which our variable partitioner has been integrated into, accepts a C-source code that is translated through the frontend into the tree-like high-level intermediate representation (HIR). The root of the HIR is the *unit* which contains a list of *functions*, *global variables*, *externals* and *types*. Every function contains a *list of function parameters*, *local variables*, and *basic blocks* consisting of a *sequence of statements*. The HIR is optimized applying the standard machine-independent transformation. Furthermore, the frontend provides also some abstract structures of the program, such as *call graph*, *control flow graph*, *dominator tree*, *SSA-form*, which are bases for the advanced analysis framework. The HIR is taken as input for the partitioner which may be invoked at any point after the compiler frontend and before the backend. For illustration, the HIR of the dot product code introduced in Fig. 1 is outlined in Fig. 3.

```

( 1)  IrBlock bb1
( 2)    IrAssign
( 3)      IrAddress (IrLocal tmp_b)
( 4)      IrConvert
( 5)*      IrAddress (IrGlobal b)
( 6)    IrAssign
( 7)      IrAddress (IrLocal tmp_dot)
( 8)      IrConstant 0
( 9)    IrAssign
(10)      IrAddress (IrLocal tmp_a)
(11)      IrConvert
(12)*      IrAddress (IrGlobal a)
(13)    IrLoopStart
(14)      IrConstant 100
(15)      IrAddress (IrBlock bb2)
(16)  IrBlock bb2
(17)    IrAssign
(18)      IrAddress (IrLocal tmp_dot)
(19)      IrAdd
(20)        IrRead
(21)          IrAddress (IrLocal tmp_dot)
(22)        IrMult
(23)          IrRead
(24)            IrRead
(25)*          IrAddress (IrLocal tmp_a)
(26)            IrRead
(27)            IrRead
(28)*          IrAddress (IrLocal tmp_b)
(29)    IrAssign
(30)      IrAddress (IrLocal tmp_b)
(31)      IrAdd
(32)        IrRead
(33)          IrAddress (IrLocal tmp_b)
(34)          IrConstant 1
(35)    IrAssign
(36)      IrAddress (IrLocal tmp_a)
(37)      IrAdd
(38)        IrRead
(39)          IrAddress (IrLocal tmp_a)
(40)          IrConstant 1
(41)    IrLoopEnd
(42)      IrAddress (IrBlock bb2)
(43)      IrAddress (IrBlock bb3)
(44)  IrBlock bb3
(45)    IrReturnValue
(46)      IrRead
(47)        IrAddress (IrLocal tmp_dot)
(48)      returnReg

```

Fig. 3. Dot Product (HIR code)

In our approach we focus on the set of global variables and static local variables. Local variables and parameters of a function are processed later in the code generation phase. They are allocated either in registers, or in the stack which is part of one particular memory bank. These temporaries are handled by the scheduler so that the memory conflicts are avoided. Array variables are treated as monolithic entities that are allocated to a single memory bank. To determine the optimal memory bank assignment for given variables, references over all functions in the program need to be observed at the same time.

The partitioning algorithm is based on the concepts of the *interference graph*, where each memory access is represented by one vertex. An edge between two vertices indicates that they may be accessed in parallel, and that the corresponding variables should be stored in separate memory banks. The goal is to partition the interference graph in such a way that the potential parallelism is maximized. The partitioning process consists of two separate components: the first constructs the interference graph, the second partitions the interference graph.

3.1 Construction of the Interference Graph

Definition 3.1 *The interference graph is defined as an edge-weighted undirected graph $G = (V, E)$, where each vertex $v \in V$ represents a memory access, and an edge $e = (v, u) \in E$ connecting a pair of vertices v and u , indicates that there is no dependence between them. With each edge $e = (v, u) \in E$ a nonnegative weight $W(e)$ is associated which represents the extent of independence between v and u .*

The interference graph is constructed for the whole program. The set of vertices is generated by traversing the HIR of the program (all functions, basic blocks and statements) and looking for objects **IrAddress** which point to global variables (see Fig. 3). Local variables (**tmp_a**, **tmp_b**, and **tmp_dot**) will be allocated in registers. For each memory access found one interference vertex is created. The **IrAddress** can represent one or more memory accesses, dependent on how many **IrRead** operators are preceding to it. **IrRead** denotes the read of the value at the address which is specified by the following address expression. Multiple consecutively **IrRead** operators substitute the multilevel indirect addressing, and to determine all global variables associated, the *alias analysis* is required. Currently, we utilize only information from the SSA (static single assignment) form, so not all memory accesses can be caught. The percentage of not-resolved variable references is strongly dependent on the structure of the source program. In our example there were recognized two memory accesses to **a** – (12), (25), and two memory accesses to **b** – (5), (28). Accesses (25) and (28) were identified through the double **IrRead** operator.

The interference vertex, besides the memory address itself, encapsulates also all information about its enclosing context (owner statement, owner block, def/use attribute, etc.), which serves as a framework for determining graph edges.

Generating the set of edges E on the set of vertices V is equivalent to the identifying all pairs of memory accesses that can be combined together for parallel execution. To accomplish this problem, at first, we construct the intra-procedural control dependence and data dependence graphs which define the relationship between the basic blocks and also between the statements within each function. There will be an edge $e = (v, u)$ between vertices $v, u \in V$ if and only if the statements (or expressions) enclosing the v and u , respectively, are not control-dependent and also not data-dependent. We suppose that memory accesses occurring in different functions or in different basic blocks can not be scheduled for parallel processing, so, no edge is generated between them.

According to the context in which the memory accesses are included a weight W is assigned to each edge $e = (v, u) \in E$ which is defined :

$$W(e) = EF \times DW(e)$$

where EF represents the *execution frequency* of the enclosing basic block, and DW represents the *distance weight* of the edge.

$$DW(e) = \begin{cases} 2 & \text{if } v \text{ and } u \text{ are contained in expressions of the same statement} \\ 1 & \text{if } v \text{ and } u \text{ are contained in different statements} \end{cases}$$

We chose this simple weight as a heuristic measure, it can be seen as the rate of the probability that the connected vertices will be scheduled into the same instruction.

Once the interference graph has been constructed, each vertex subset $\{v_1, \dots, v_k\} \subseteq V$ representing accesses to the same variable, is merged into a single vertex v , and all edges containing $v_1, \dots, v_k$ are redirected to the new vertex v . The weight of an edge $e = (v, u)$ is modified to

$$W(e) = \text{Max}(W(e_i)) \times k$$

where $e_i = (v_i, u)$, for $i = 1, \dots, k$. So, the size of the graph (number of vertices) is equal to the number of global variables accessed.

3.2 Partitioning of the Interference Graph

The best partitioning of the interference graph $G = (V, E)$ is achieved if the set of vertices V can be divided into two disjoint sets $S \subseteq V$ and $\bar{S} = V - S$, such that the sum of the weights of all edges that connect a vertex $v \in S$ to a vertex $u \in \bar{S}$ is maximal. Variables corresponding to vertices from S are assigned to X memory bank, and variables corresponding to vertices from $\bar{S}$ are assigned to Y memory bank.

Theoretically, in this case the highest number of parallel memory accesses can be obtained. Practically, however, the performance gain is affected by the fact, how the scheduler actually realizes the calculated parallelism.

This partitioning task can be formulated as the combinatorial optimization problem *Max Cut*. The cut $Cut(S, \bar{S})$ is defined as the set of edges that have one

endpoint in S and the other endpoint in $\bar{S}$. The Max Cut consists in finding a subset of vertices S such that the weight of $Cut(S, \bar{S})$ given by

$$\sum_{e \in Cut(S, \bar{S})} W(e)$$

is maximized.

Let $V = \{v_1, v_2, \dots, v_n\}$ be the set of vertices of $G = (V, E)$; we use i for an vertex v_i , and w_{ij} for the weight of an edge $(v_i, v_j) \in E$ (for $e = (v_i, v_j) \notin E$ we set $w_{ij} = 0$). When introducing *cut vectors* $x \in \{-1, 1\}^n$ with $x_i = 1$ for $v_i \in S$, and $x_i = -1$ for $v_i \in \bar{S}$, then the algebraic formulation for Max Cut can be written as follows:

$$\begin{aligned} & \text{maximize} && \frac{1}{2} \sum_{1 \leq i < j \leq n} w_{ij}(1 - x_i x_j) \\ & \text{subject to} && x_i \in \{-1, 1\}, i = 1, \dots, n. \end{aligned} \tag{1}$$

The key property of the formulation (1) is that $(1 - x_i x_j)/2$ can take only two values - either 0 or 1, which allows to model the appearance of an edge in a cut within the objective function. For any feasible solution $x = (x_1, \dots, x_n)$, the set $S = \{v_i \in V : x_i = 1\}$ defines the cut $Cut(S, \bar{S})$ which has the weight equal to the objective value at x .

The first feasible solution of this NP-complete problem was proposed in 1976 by Sahni and Gonzales [11], they presented an approximation algorithm with the performance guarantee $0.5 \times$ optimal value. Since then for a nearly twenty years no significant progress has been made in improving this performance guarantee. Only in 1994 Goemans and Williamson [12,13] proposed a randomized algorithm based on the semidefinite programming which always delivers a solution of value at least $0.87856 \times$ the optimal value. There exists several extensions of the Goemans and Williamson technique. For example, Frieze and Jerrum [14] designed an algorithm for the *Max k -Cut*, where $k \geq 2$, which can be applicable to an arbitrary number of memory banks.

3.3 Implementation of Partitioning

Provided that the number of vertices is small (less than twenty), the Max Cut is possible to be solved exactly still in reasonable time. Otherwise, approximating techniques are applied. To find a near optimal partitioning we have implemented several approximating algorithms, simple and also more sophisticated. This which yields the best solution is chosen for the partitioning. Algorithms are described in the following.

Exact Algorithm

This algorithm computes the Max Cut exactly. It generates recursive all possible cut vectors and calculates its cut. The cut vector having the maximal cut value is chosen as the solution. It can happen that there exists more than one solution - several different cut vectors with the equal maximal cut value. In this case to select the best one must be experimentally examined.

Greedy Algorithm

This approximating algorithm represents the iterative approach which utilizes the property of the Max Cut problem that the value of any local optimum is not too far from the value of the total optima. Implementation is based on the scheme described in [15]. The algorithm begins with a naive initial approximation to the solution – all vertices of G are placed into the set S , with the set $\bar{S}$ being empty. Then the method repeatedly iterates over all vertices in order to find a vertex whose relocation to other set could increase the cut. The algorithm is running until it reaches a fix point where each pass produces no further increase of the cut. Algorithm runs in the polynomial time $O(n \times m)$, where n is the number of vertices, and m is the number of edges. It delivers a solution of value at least $0.5 \times$ the optimal value.

Semidefinite Programming Relaxation

This approximating algorithm was provided by Goemans and Williamson [12,13]. It is a simple and elegant technique that randomly rounds the solution to a nonlinear semidefinite programming relaxation. The algorithm always delivers a solution of value at least $0.87856 \times$ the optimal value.

Let $\mathcal{R}^n$ denote the space of real n -dimensional column vectors. The *unit scalars* x_i of (1) can be viewed as vectors of *unit norm* belonging to $\mathcal{R}^n$; or more precisely, to the n -dimensional unit sphere $\mathcal{S}_n = \{y \in \mathcal{R}^n : \|y\| = y^T y = 1\}$. Associating scalars x_i with the unit vectors $y_i \in \mathcal{S}_n$, for $i = 1, \dots, n$, the products $x_i x_j \in \{-1, 1\}$ may be relaxed to $y_i^T y_j \in \langle -1, 1 \rangle$.

Then after some mathematical manipulations, (1) can be formulated as a relaxation to a semidefinite program (for more details see [12,13]):

$$\begin{aligned} & \text{maximize} && C \bullet Y \\ & \text{subject to} && \text{diag}(Y) = e \\ & && Y \succeq 0 \end{aligned} \tag{2}$$

Given a feasible solution Y of (2), the set of unit vectors $y_j, j = 1, \dots, n$, can be obtained by the Cholesky factorization $Y = Z^T Z$, where columns of the matrix Z correspond exactly to the vectors $y_1, \dots, y_n$.

Using the geometric interpretation, a solution $(y_1, \dots, y_n)$ consists of n points on the surface of the unit sphere $\mathcal{S}_n$, each representing a vertex of the graph, and the product $y_i^T y_j$ is the cosine of the angle enclosed by these vectors.

Goemans and Williamson proposed the following randomized algorithm for generating cuts : construct a random hyperplane through the origin of $\mathcal{S}_n$ and group all vectors on the same side of this hyperplane together. The hyperplane can be constructed by choosing a random vector r uniformly distributed on the unit sphere $\mathcal{S}_n$: $\mathcal{H}(r) = \{y \in \mathcal{R}^n : r^T y = 0\}$. Partitioning of the vertex set V into $(S, \bar{S})$ is formed by assigning all vertices $v_i \in V$ to S whose corresponding vectors y_i have positive inner product with r :

$$\begin{aligned} S &= \{v_i \in V : y_i^T r \geq 0\} \\ \bar{S} &= \{v_i \in V : y_i^T r < 0\} \end{aligned}$$

This semidefinite relaxation has been implemented using the SDPA solver developed by Fujisawa, Kojima, Nakata, and Yamashita [16]. For the Cholesky factorization and randomizing the solution the LAPACK-library is utilized.

Semidefinite Rank-2 Relaxation

For experimental reasons we have implemented also this algorithm which was developed by Burer, Monteiro, and Zhang [17]. It represents the specialized version of the Goemans–Williamson randomized technique with the same performance guarantee. Algorithm was implemented utilizing the Fortran 90 software package CIRCUT [18] which was rewritten into C++ object.

4 Experimental Results

Our partitioning technique was empirically evaluated on the simulator of the experimental digital signal processor xDSPcore [1]. We did experiments with various small kernels from the DSPstone benchmark suite [19], and some applications. The metrics which the performance is measured in is the number of cycles executed, and the number of memory conflicts appeared. A memory conflict occurs if two accesses to the same memory bank are scheduled in one instruction; in this case an extra (stalling) cycle is generated by a special hardware mechanism.

In order to demonstrate the effectiveness of our partitioning algorithm, for each kernel several variants were compiled, executed, and evaluated. In the first version variables are assigned explicitly only to one memory bank. In the second version variables are not assigned before linking phase; here an optimistic algorithm by scheduling is applied and the linker tries to resolve the variable allocation. For these two cases the partitioner was disabled. In the third version variables are assigned to memory banks by means of the partitioner. These three cases are referred to as *X-Allocating*, *Scheduling*, and *Partitioning*, respectively.

Table 1 lists the performance results obtained for some selected DSPstone kernels. Each kernel contains some loops with operations on two or three global arrays. According to the information about the memory assignment the code generator schedules the operations into instructions, so, for our three examined cases the target code may look differently. For each variant the first column shows the total number of cycles executed (memory conflicts are not included), the second and third columns show the number of accesses to X and Y memory bank, and the fourth column shows the number of memory conflicts. We can see that in the first version, where all variables are allocated to X memory bank, the number of memory conflicts is equal to zero only in this case when memory accesses are scheduled in separate instructions, that is, the number of executed cycles is increased. In the optimistic version variables are tried to be allocated to both memory banks, however, the results are not better than in the first case. It is evident that the best performance gain is achieved by the version with partitioning. For these small kernels, all implemented algorithms, exact and also approximating, yield the identical partitioning result which seems to

Table 1. DSPstone Kernels

Kernel	X-Allocating				Scheduling				Partitioning			
	Cycl.	X	Y	Confl.	Cycl.	X	Y	Confl.	Cycl.	X	Y	Confl.
dot_product	625	200	0	0	525	200	0	100	525	100	100	0
convolution	625	200	0	0	525	134	66	34	525	100	100	0
matrix_mult_1	5368	2100	0	1000	5368	2100	0	1000	5368	1000	1100	0
matrix_mult_2	5014	2010	0	900	4993	2010	0	900	4993	1100	910	0
mat1x3	85	24	0	0	76	24	0	9	76	9	15	0
lms	219	95	0	0	219	12	83	16	188	48	47	0
fir2dim	963	304	0	144	963	304	0	144	963	144	160	0
biquad_n_sections	71	38	0	12	84	38	0	16	66	21	17	0
Total Number of Cycles												
dot_product	625				625				525 (84.0%)			
convolution	625				559				525 (84.0%)			
matrix_mult_1	6368				6368				5368 (84.3%)			
matrix_mult_2	5914				5893				4993 (84.4%)			
mat1x3	85				85				76 (89.4%)			
lms	219				235				188 (85.8%)			
fir2dim	1107				1107				963 (87.0%)			
biquad_n_sections	83				100				66 (79.5%)			

Table 2. FFT Filters

Kernel	X-Allocating				Scheduling				Alternate Alloc.			
	Cycl.	X	Y	Confl.	Cycl.	X	Y	Confl.	Cycl.	X	Y	Confl.
fft256_1	194681	110252	0	17052	204178	113248	0	20195	195560	60888	49364	12831
fft256_2	162341	91046	0	11053	168927	91291	0	17661	146322	48479	39920	8823
	Partitioning											
	Execution Frequency						No Frequency					
	Cycl.	X	Y	Confl.		Cycl.	X	Y	Confl.			
fft256_1	194261	24110	86142	12572 (74%)		194261	24110	86142	12572 (74%)			
fft256_2	145977	44909	42518	8785 (79%)		152171	29957	61936	7427 (67%)			
	Total Number of Cycles											
	X-Allocating			Scheduling			Alternate Alloc.			Partitioning		
	211733			224373			208391			206833 (97%)		
fft256_2	173394			186588			155145			154762 (89%)		

be quite ideal. The pure memory access cycles are decreased by about 50%, and the improvement of total number of cycles ranges from 10% to 20%. In real applications, however, this would not be true.

Table 2 presents performance results of code which contains a fixed-point implementation of 256-point complex Fast Fourier Transform (FFT) and the inverse FFT. It is based on Radix-2 decimation in frequency domain algorithm on a block of complex numbers. Two versions of the FFT code have been examined. In `fft256_1` the real and imaginary values of the complex data are stored in one array in interleaved format (real followed by imaginary). The `fft256_2` represents a slightly modified code; in order to avoid the successive memory accesses to the same array, the real and imaginary values of the complex data are stored in two separate arrays. In both versions all global arrays are referenced through the subscripts, not through the pointers, so, all accesses could be found and resolved without any complicated alias analysis.

Additionally to the *X-Allocating*, *Scheduling*, and *Partitioning* strategies we measured also the approach where the vertices of the interference graph are partitioned in the alternate way starting with X-memory, it is referred to as *Alternate Alloc*. By partitioning we experimented with several heuristics. In Table 2 results from two instances are reported : in the first the edges are weighted by the execution frequency of basic blocks as defined in Section 3.1; while in the second, the edges are weighted without using any frequency estimates (*EF* is supposed to have the value one).

For the `fft256_1` code the size of the interference graph is equal to 10, and surprisingly, the partitioning algorithm yields only one solution regardless of the execution frequency is used or not. Wenn comparing the *X-Allocating* with the *Partitioning* the number of memory conflicts is decreased by 27%, however, the total number of cycles is approximately the same.

For the `fft256_2` code the size of the interference graph is equal to 13. In this case better results can be achieved because the butterfly FFT-algorithm operates now on two arrays (real and imaginary) instead of on one array. The partitioning algorithm without the execution frequency used yields three solutions which give the equal results. The algorithm using the execution frequency yields twelve solutions giving several different results, the best one is introduced in the table. Also for this version the execution frequency does not improve significantly the quality of the results. Wenn comparing the *X-Allocating* and *Partitioning* strategies, the number of memory conflicts is decreased by 33%, and the total number of cycles by about 10%.

The *Alternate Allocating* approach for both codes shows the comparable results as the *Partitioning* strategy. This is due to the character of the FFT-algorithm.

It is worth to say, that for each observed benchmark approximating algorithms give the identical solution as the exact algorithm. So, which algorithm is preferred has not a great impact on the partitioning result. To obtain a real performance improvement, the most significant is to provide the correct information for partitioning. A good graph model should reflect all the potentially parallel memory accesses that may actually occur in scheduling.

5 Conclusion

In this paper we have presented an algorithm which attempts to maximize the benefit of dual data memory banks. The algorithm is based on partitioning the interference graph whose nodes represent variables and edges represent potential parallel accesses to pairs of variables. The interference graph is constructed utilizing the control flow, data flow, and alias information. For partitioning itself, formulated as *Max Cut* problem, we have implemented several methods. All of them work very well and fast. The important contribution of our approach is that the algorithm operates on the high-level intermediate representation, independent of the target machine. Our framework is global and is not just limited to basic blocks. Both scalar and array variables of the entire program are handled at once, so no contrary demands on assigning a certain variable to either X or Y can arise.

The experimental results demonstrate that our method finds a quite satisfying memory assignment. On small kernels we were able to reduce the number of memory cycles by 50%, and the total number of cycles by 10%–20%. For FFT filters the number of memory conflicts is decreased by 30%, and the total number of cycles by 10%.

In the future we plan to work on the refinement of the interference graph. We would like to make experiments with several new heuristics including runtime profiling information, and evaluate the method on real bigger applications. We also plan to explore the memory partitioning for DSP architectures which are equipped with interleaved memory banks where the interleaving factor can be any number, not only two.

Acknowledgments

I would like to acknowledge the *Christian Doppler Forschungsgesellschaft* and *Infineon* for funding this research. I would also like to thank *Andreas Krall* for his valuable comments on this paper and *Ulrich Hirnschrott* for his help by compiling and simulating the kernels.

References

1. C. Panis, G. Laure, W. Lazian, A. Krall, H. Grünbacher, J. Nurmi: *DSPxPlore – Design Space Exploration for a Configurable DSP Core*. In: Proceedings of the GSPx, Dallas, Texas, USA (2003)
2. R. Leupers and D. Kotte: *Variable Partitioning for Dual Memory Bank DSPs*. In: Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ASSP). Volume 2. (2001) 1121–1124
3. D.B. Powell, E.A. Lee, and W.C. Newman: *Direct Synthesis of Optimized DSP Assembly Code from Signal Flow Block Diagrams*. In: Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ASSP). Volume 5. (1992) 553–556
4. M.A.R. Saghir, P. Chow, and C.G. Lee: *Automatic Data Partitioning for HLL DSP Compilers*. In: Proceedings of the 6th International Conference on Signal Processing Applications and Technology. (1995) I-866–871

5. M.A.R. Saghir, P. Chow, and C.G. Lee: *Exploiting Dual Data-Memory Banks in Digital Signal Processor*. In: ACM SIGOPS Operating Systems Review, Proceedings of the 7th International Conference on Architectural Support for Programming Languages and Operating Systems. Volume 30(5). (1996) 234–243
6. A. Sudarsanam and S. Malik: *Memory Bank and Register Allocation in Software Synthesis for ASIPs*. In: Proceedings of the IEEE/ACM International Conference on Computer Aided Design. (1995) 388–392
7. A. Sudarsanam and S. Malik: *Simultaneous Reference Allocation in Code Generation for Dual Data Memory Bank ASIPs*. Journal of the ACM Transactions on Automation of Electronic Systems (TODAES) **5** (2000) 242–264
8. J. Cho, Y. Paek, and D. Whalley: *Efficient Register and Memory Assignment for Non-orthogonal Architectures via Graph Coloring and MST Algorithm*. In: Proceedings of the International Conference on the LCTES and SCOPES, Berlin, Germany (2002)
9. X. Zhuang, S. Pande, and J.S. Greenland: *A Framework for Parallelizing Load/Stores on Embedded Processors*. In: Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT), Virginia (2002)
10. Q. Zhuge, B. Xiao, and E.H.-M. Sha: *Variable Partitioning and Scheduling of Multiple Memory Architectures for DSP*. In: Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS). (2002)
11. S. Sahni and T. Gonzales: *P-complete Approximation Problems*. Journal of the ACM **23** (1976) 555–565
12. M.X. Goemans and D.P. Williamson: *0.878-Approximation Algorithms for MAX CUT and MAX 2SAT*. In: Proceedings of the 26th Annual ACM Symposium on Theory of Computing. (1994) 422–431
13. M.X. Goemans and D.P. Williamson: *Improved Approximation Algorithms for MAX CUT and Satisfiability Problems Using Semidefinite Programming*. Journal of the ACM **42** (1995) 1115–1145
14. A. Frieze and M. Jerrum: *Improved Approximation Algorithms for Max k-Cut and Max Bisection*. Algorithmica **18** (1997) 61–77
15. Hromkovic, J.: *Algorithmics for Hard Problems*. Springer-Verlag, Berlin (2001)
16. K. Fujisawa, M. Kojima, K. Nakata, and M. Yamashita: *SDPA (Semidefinite Programming Algorithm)*, vers. 4.10, Research Report on Mathematical and Computing Sciences, Tokyo Institute of Technology, Japan. (1998)
17. S. Burer, R.D.C. Monteiro, and Y. Zhang: *Rank-two Relaxation Heuristics for Max-Cut and Other Binary Quadratic Programs*. SIAM Journal on Optimization **12** (2001) 503–521
18. S. Burer, R.D.C. Monteiro, and Y. Zhang: *CirCut vers. 1.0612, Fortran 90 Package for Finding Approximate Solutions of Certain Binary Quadratic Programs* (2000)
19. V. Zivojnovic, J.M. Velarde, C. Schager, and H. Meyr: *DSPstone – A DSP oriented Benchmarking Methodology*. In: Proceedings of the 6th International Conference on Signal Processing Applications and Technology. (1994)

Cache Behavior Modeling of Codes with Data-Dependent Conditionals^{*}

Diego Andrade, Basilio B. Fraguera, and Ramón Doallo

Computer Architecture Group

Universidade da Coruña

Dept. de Electrónica e Sistemas

Facultade de Informática

Campus de Elviña, 15071 A Coruña, Spain

{dcanosa,basilio,doallo}@udc.es

Abstract. The increasing gap between the speed of the processor and the memory makes the role played by the memory hierarchy essential in the system performance. There are several methods for studying this behavior. Trace-driven simulation has been the most widely used by now. Nevertheless, analytical modeling requires shorter computing times and provides more information. In the last years a series of fast and reliable strategies for the modeling of set-associative caches with LRU replacement policy has been presented. However, none of them has considered the modeling of codes with data-dependent conditionals. In this article we present the extension of one of them in this sense.

1 Introduction

The memory hierarchy plays an essential role in bridging the increasing gap between the processor and the memory speed. The optimal usage of the memory hierarchy is specially important in real-time systems and systems that require low power and energy consumption. This way, although the research in this area has traditionally focused on the optimization of codes executed in computers, we consider that its relevance is even greater in the field of the embedded systems. Programmers use many methods in order to improve the performance of the memory hierarchy during the execution of their codes. Unfortunately, the only tool available for a long time to study this behavior has been trace-driven simulation [1]. The main drawback of this method is the long computing time it requires. Some architectures implement built-in hardware counters [2], but their availability is limited to certain architectures. In addition, in both cases either the code or a simulation needs to be executed in order to obtain data on the memory hierarchy performance, and neither of them explains the observed behavior. Analytical models are faster than the previous methods and give us much

^{*} This work has been supported in part by the Spanish Ministry of Science and Technology under contract TIC2001-3694-C02-02.

more information. Many models of this kind have been proposed in the bibliography [3,4,5]. The main drawbacks of these models are the lack of modularity and the fact they can only model a limited set of program structures.

The model we propose in this paper is an extension of the probabilistic model introduced in [3]. That work proposes a very modular model, what makes it easily extensible. We have extended the set of code constructions it supports with data-dependent conditionals, a program structure that no previous work in this area has modeled. As a first step, we only consider conditions that follow an uniform distribution, but we regard this extension very interesting as a first step towards the study of whole real programs.

The model proposed in [3] builds automatically equations, referred as Probabilistic Miss Equations (PMEs), that estimate the number of misses that a given code generates. This method models the behavior of set-associate caches with LRU replacement policy. It is applicable to perfectly nested loops and non-perfectly nested loops with one loop per nesting level. It allows several references per data structure and loops controlled by other loops. Loop nests with several loops per level can also be analysed by this model, although certain conditions need to be fulfilled in order to obtain accurate estimations.

This paper describes the extension of this model in order to consider codes with data-dependent conditionals that follow an uniform distribution. Sect. 2 presents the main concepts in which our model is based. Then, Sect. 3 introduces the area vector concept, which is used by our model to represent the impact of a series of accesses to a data structure on the cache. The strategy to build formulas that estimate the number of cache misses in codes containing data-dependent conditionals is explained in Sect. 4, which is followed by a validation using a simple code and trace-driven simulations in Sect. 5. Sect. 6 is a brief review of the related works. Finally, Sect. 7 is devoted to the conclusions and future work.

2 Modeling Concepts

We consider a cache with a size of C_s words, a line size of L_s words, an associativity degree k , where we refer as word to the size of the elements of our data structures. There are two situations that can generate a miss in the access to a line. The first one is the first access to this line, which is known as an intrinsic miss. Each one of the remaining accesses will result in a miss if k or more different lines accessed since the last reference to that line are mapped to the same cache set. These misses are known as interference misses. This way, the probability an access results in an interference miss is equal to the probability that k or more lines have been mapped to cache set of the accessed line since the previous access to the line took place.

The misses generated by a reference can be estimated by means of a formula that includes the number of different lines it accesses (intrinsic misses), the number of line reuses it generates, and the interference probability for such accesses (interference misses). The calculation of this probability involves estimating the memory region accessed between each two consecutive accesses to the same line,

and the mapping of this region on the cache. The miss probability will be equal to the ratio of sets that receive k or more different lines.

```

DO I0=1, N0
  DO I1=1, N1
    ...
    DO IZ=1, NZ
      A(fA1(IA1), ..., fAdA(IAdA))
      ...
      IF B(fB1(IB1), ..., fBdB(IBdB))
        C(fC1(IC1), ..., fCdC(ICdC))
        ...
      END DO
    ...
  END DO
...
END DO

```

Fig. 1. Nested loops with data-dependent conditions

Figure 1 shows a nest of normalized loops that contains references inside data-dependent conditionals. This is the type of structures we consider in our extension. Our model considers references whose indexes are affine functions of the type $f_{A1}(I_{A1}) = \alpha_{A1}I_{A1} + \delta_{A1}$. The references can be found in any nesting level, not just in the innermost one. The number of iterations of every loop must be known at compile time and must be the same in every execution of the loop. The reuse among different references to the same data structure can be analyzed using our model only if those references are uniformly generated [6], that is, they only differ in one or more of the added δ constants. This is by far the most common situation in scientific codes. Uniformly generated references are typically found in the same scope in a given nest, as they use the same variables for their indexing. Thus, as a simplification, when there are references to the same data structure in different scopes of the same nest, their potential reuse is not considered. Still, if the references are found in different nests (which may share outer level loops), reuse is estimated following a conservative approach.

As for the conditional structures, in this work we consider conditions whose verification follows an uniform distribution, as stated in the introduction. This means that in every evaluation of the condition there is a constant probability p that it is fulfilled.

3 Area Vectors

Miss probabilities are calculated using area vectors. These vectors represent the impact on the cache of the accesses to one or several data structures. Given a

data structure V , $S_V = S_{V_0}, S_{V_1}, \dots, S_{V_k}$ is the area vector associated with the access to V during a given period of the program execution. The i -th element, $i > 0$, of this vector represents the ratio of sets that have received $k - i$ lines from the structure. As for S_{V_0} , it is the ratio of sets that have received k or more lines.

The two most common access patterns found in the kind of codes we intend to model are the sequential access and the access described as “access to n groups of t elements separated by a constant stride d ”. The representation and calculation of the impact on the cache of these and other access patterns by means of area vectors has been solved in [3].

3.1 Area Vectors Addition

It is very common that references to more than one data structure take place between two accesses to the same line of a data structure. This implies that a mechanism is needed to add the area vectors associated with these structures in order to calculate the global area vector.

Given two area vectors $S_U = (S_{U_0}, S_{U_1}, \dots, S_{U_k})$ and $S_V = (S_{V_0}, S_{V_1}, \dots, S_{V_k})$, the addition of them, $S_U \cup S_V$, is defined as

$$\begin{aligned} (S_U \cup S_V)_0 &= \sum_{j=0}^K \left(S_{U_j} \sum_{i=0}^{K-j} S_{V_i} \right) \\ (S_U \cup S_V)_i &= \sum_{j=i}^K S_{U_j} S_{V_{(K+i-j)}} \quad 0 < i \leq K. \end{aligned} \quad (1)$$

This method is based in the addition of independent probabilities, which means that it does not take into account the relative positions of the data structures in memory. If such positions are known, the overlapping coefficients of the footprints associated with the accesses to these structures on the cache can be estimated. The accuracy of the addition may be improved by using them to scale or weight the area vectors [3]. This way, area vectors corresponding to data structures that overlap more in the cache with the data structure affected by the reference that is being analyzed, get higher weights than those ones that overlap less or even do not overlap. The latter case would in fact turn the corresponding area vector S into an empty one ($S_i = 0, 0 \leq i < k$ and $S_k = 1$).

4 Probabilistic Miss Equations

Our method generates a *Probabilistic Miss Equation* (PME) for each reference in each nesting level. Let $F_i(R, S(\text{RegInput}), p)$ be the PME that estimates the number of misses generated by reference R in nesting level i . It is a function of $S(\text{RegInput})$, the area vector associated to the region that has been accessed since the last access to a given line of the data structure that R references. If the reference is inside a conditional sentence whose condition follows an uniform distribution, p is the probability that the condition is true. The probability of the conditionals can be obtained either by several means : profiling, input data analysis, or previous knowledge of the application field.

The loops are examined from the innermost one to the outermost one in order to calculate the number of misses generated by each reference. In each level a formula is generated depending on whether the variable associated to the current loop indexes or not any of the references found in the condition(s) of the conditional sentence. If the loop variable is not used in the indexes of any of these variables, then a *Condition Independent Reference Formula* (CIRF) is applied. Otherwise, a *Condition Dependent Reference Formula* (CDRF) is built.

4.1 Condition Independent Reference Formulas

This kind of formulas has already been described in [3]. It assumes that, if the analyzed reference reuses a given line in the current loop, the last access to that line took place in the previous iteration of the considered loop. The reuse in the loop may take place either because of temporal reuse (the loop variable does not index the reference) or spatial reuse (the loop variable indexes the reference and its stride is smaller than the line size). Let N_i be the number of iterations in the loop of the nesting level i , and L_{Ri} be the number of iterations in which there is no possible reuse for the lines referenced by R , then we can define $F_i(R, S(\text{RegInput}), p)$ as

$$F_i(R, S(\text{RegInput}), p) = L_{Ri} F_{i+1}(R, S(\text{RegInput}), p) + (N_i - L_{Ri}) F_{i+1}(R, S(\text{Reg}(A, i, 1)), p), \quad (2)$$

where $\text{Reg}(A, i, j)$ stands for the memory region accessed during j iterations of the loop in the nesting level i that can interfere with data structure A . $S(\text{Reg}(A, i, j))$ represents the area vector associated to that region.

The formula reflects the fact that for the L_{Ri} iterations in which there can be no reuse in this loop, the miss probability depends on the accesses and reference patterns in the outer loops. In the remaining iterations, this probability is calculated as a function of the accessed regions during the portion of program executed between those reuses, this is, during one iteration of loop i .

The indexes of the reference R are affine functions of the variables of the loops that enclose it. As a result, R follows a constant stride S_{Ri} along the iterations of loop i . This value is calculated as $S_{Ri} = \alpha_{A_j} d_{A_j}$, where j is the dimension whose index depends on I_i , the variable of the loop; α_{A_j} is the scalar that multiplies the loop variable in the affine function, and d_{A_j} is the size of the j -th dimension. If I_i does not index reference R , then $S_{Ri} = 0$. This way, L_{Ri} can be calculated as,

$$L_{Ri} = 1 + \left\lfloor \frac{N_i - 1}{\max\{L_s/S_{Ri}, 1\}} \right\rfloor. \quad (3)$$

The formula calculates the number of accesses of R that can not exploit either spatial or temporal locality, which is equivalent to estimating the number of different lines that are accessed during N_i iterations with stride S_{Ri} .

4.2 Condition Dependent Reference Formulas

The second kind of formulas is applied when I_i , the variable associated to the current loop, is used in the indexes of the references found in the condition of a conditional sentence that controls the execution of the reference R whose behavior we are analyzing. In this case, the last access of R to a given line may have happened an indeterminate number of iterations ago, depending on the probability p that the condition is fulfilled and thus R is executed.

Weighted Reuse. When a reference is located inside a data-dependent conditional sentence whose outcome changes for the different iterations of a given loop, it is not possible to estimate accurately the number of iterations of the loop between two accesses to the same line by the reference. The reason is that accesses only take place with a given probability. Thus, a probabilistic approach must be followed to estimate this value, which is the reuse distance in the loop. This way, the probability that the last access has happened 1,2... iterations ago must be weighted. We define the weighted reuse for the j -th consecutive access to a given line during the execution of the loop in nesting level i , $WR(p_i, \text{RegInput}, i, j, p)$ with this purpose. In this expression, p_i stands for the probability the line is accessed by the considered reference during one iteration of the loop, and RegInput , stands for the region accessed since the last reference to the line when the loop execution begins, just as in the previous formulas. The weighted reuse is calculated as

$$\begin{aligned} WR(p_i, S(\text{RegInput}), i, j, p) \\ = (1 - p_i)^{j-1} F_{i+1}(R, S(\text{RegInput}) \cup S(\text{Reg}(A, i, j - 1)), p) \\ + \sum_{k=1}^{j-1} p_i (1 - p_i)^{k-1} F_{i+1}(R, S(\text{Reg}(A, i, k - 1)), p) . \quad (4) \end{aligned}$$

The first term considers the case that the line has not been accessed during any of the previous $j - 1$ iterations. In this case, the RegInput region that could generate interference with the new access to the line when the execution of the loop begins must be added to the regions accessed during these $j - 1$ previous iterations of the loop in order to estimate the complete interference region. The second term weights the probability that the last access took place in each of the $j - 1$ previous iterations of the considered loop.

Given a loop with n iterations, we define the total weighted reuse in its n iterations, $TWR(p_i, S(\text{RegInput}), i, n, p)$, as¹ the addition of the weighted reuse for every one of them:

$$TWR(p_i, S(\text{RegInput}), i, n, p) = \sum_{j=1}^n WR(p_i, S(\text{RegInput}), i, j, p) . \quad (5)$$

¹ If n is not an integer value, it is estimated as $TWR(p_i, S(\text{RegInput}), i, n, p) = (n - \lfloor n \rfloor) TWR(p_i, S(\text{RegInput}), i, \lceil n \rceil, p) + (1 - (n - \lfloor n \rfloor)) TWR(p_i, S(\text{RegInput}), i, \lfloor n \rfloor, p)$

Line Access Probability. The fact that every access takes place only with probability p complicates the calculation of the probability that a given line is accessed during each iteration of the considered loop. This probability depends not only on the access pattern to the line in this nesting level, but also in the inner ones. This way, access probabilities are calculated starting in the innermost loop and analyzing the nest outwards, just as the PMEs.

In the CIRF formula we had defined L_{Ri} as the number of loop iterations where there is no possible reuse. Now we define G_{Ri} as the number of iterations that can potentially reuse the lines accessed in those L_{Ri} iterations. The product of both terms must be equal to the number of iterations of the loop, thus $G_{Ri} = N_i/L_{Ri}$. We represent the probability that a line is accessed during one iteration of the loop in nesting level i as p_i . If the loop variable for the level $i + 1$ is not used in the indexes of the references found in the condition, then $p_i = p_{i+1}$. Otherwise, $p_i = 1 - (1 - p_{i+1})^{G_{Ri+1}}$. In the innermost loop $p_i = p$.

Formulation. Once the previous concepts have been established, the final formula that estimates the number of misses of a conditional dependent reference R (CDRF) in nesting level i is,

$$F_i(R, S(\text{RegInput}), p) = L_{Ri} TWR(p_i, S(\text{RegInput}), i, G_{Ri}, p) . \quad (6)$$

4.3 Calculation of the Number of Misses

In the innermost level that contains the reference R , $F_{i+1}(R, S(\text{RegInput}), p)$, the number of misses caused by the reference in the immediately inner level is $S_0(\text{RegInput})$, this is, the first element in the area vector associated to the region RegInput . If the reference is inside a conditional sentence, this value is multiplied by p , as the reference only happens with probability p .

Once the formulas for the outermost level are calculated, the number of misses is estimated as $F_0(R, S(\text{RegInput}_{\text{total}}), p)$, where $\text{RegInput}_{\text{total}}$ is the total region, this is, the region that covers the whole cache. The miss probability associated with this region is one.

5 Model Validation

We have validated our model by applying it manually to the simple codes shown in Figs. 2 and 3. They are, respectively, a synthetic kernel and an optimized matrix product. These codes consist of a nest of loops that contain references inside a conditional sentence. We are using FORTRAN in the examples we model, but there is no problem in modeling codes with other languages. The analytical model only depends on the access patterns, not on the language that generates them.

A tool to apply automatically our modeling strategy is currently under construction.

```

DO I = 1,M
  X = A(I)
  DO J = 1,N
    Y = B(J)
    IF (B(J).GT.K) THEN
      C(J) = X+Y
    ENDIF
  ENDDO
ENDDO

```

Fig. 2. Synthetic kernel code

```

DO I=1,M
  DO J=1,P
    T=0
    DO K=1,N
      IF (A(I,K).NEQ.0) THEN
        T=T+A(I,K)*B(K,J)
      ENDIF
    ENDDO
    C(I,J)=C(I,J)+T
  ENDDO
ENDDO

```

Fig. 3. Optimized matrix product

5.1 Synthetic Kernel Modeling

Without loss of generality, we assume a compiler that maps scalar variables to registers and which tries to reuse the memory values recently read in processor registers. Under these conditions, the code in Fig. 2 contains three references to memory. If the compiler followed a different policy to generate the code, we would just model the access pattern generated by the references it produces. The model in [3] can estimate the behavior of the references $A(I)$ and $B(J)$, which take place in every iteration of their enclosing loops. Notice that the second access to $B(J)$ would reuse the value which was previously loaded in order to check the condition found in the code. This way, $C(J)$ is the only access to memory that takes place under the control of a conditional, which has an uniform probability p of being fulfilled, and thus we will focus our explanation on the modeling of its behavior.

The modeling begins in the innermost loop, in level 1. This loop variable indexes the reference involved in the condition, so the CDRF is to be used. Let $S_{R1} = 1$, $L_{R1} = 1 + \lfloor (N-1)/L_s \rfloor$, $G_{R1} \simeq L_s$, $p_1 = p$, then we obtain the following formula,

$$F_1(R, S(\text{RegInput}), p) = (1 + \lfloor (N-1)/L_s \rfloor) TWR(p, S(\text{RegInput}), 1, L_s, p) . \quad (7)$$

As this loop is in the innermost level, $F_2(R, \text{RegInput}, p) = pS_0(\text{RegInput})$. The calculation of TWR (5) from WR (4) requires to estimate the memory regions accessed during i iterations of this loop that may generate interference with $\mathcal{C}$, the data structure affected by the reference we are analyzing:

$$S(\text{Reg}(C, 1, i)) = S_s(i) \cup S_{\text{sauto}}(i) . \quad (8)$$

The first term corresponds to the sequential access to i consecutive elements of $\mathcal{B}$, and the second term stands for the autointerference produced by the access to i consecutive elements of $\mathcal{C}$. The autointerference is the interference that the accesses to a given data structure may generate on other accesses to that same structure. It is calculated in a slightly different way to that of cross interferences, which are the interferences due to the accesses to other data structures. The reason is that accesses to a given line do not generate interferences on that very same line, but they can of course generate interference with other lines of the same data structure.

In the next outer level, level 0, the loop index does not index the reference used in the conditional, thus the CIRF is applied. Its formulation for $L_{R0} = 1$ is,

$$\begin{aligned} F_0(R, S(\text{RegInput}), p) &= F_1(R, S(\text{RegInput}), p) \\ &+ (M - 1) F_1(R, S(\text{Reg}(C, 0, 1)), p) . \end{aligned} \quad (9)$$

In this case $\text{Reg}(C, 0, 1)$, the region accessed during one iteration of the in loop level 0 that may affect data structure $\mathcal{C}$ in the cache is

$$S(\text{Reg}(C, 0, 1)) = S_s(1) \cup S_s(N) \cup S_{\text{sauto}}(N) . \quad (10)$$

The first term is associated to one element in $\mathcal{A}$ and the second one stands for the access to N consecutive element of $\mathcal{B}$. Finally, the third term corresponds to the autointerference produced by the access to N consecutive elements of $\mathcal{C}$.

As we have reached the outermost level, the number of misses generated by the reference may be estimated as $F_0(R, S(\text{RegInput}_{\text{total}}), p)$, where $\text{RegInput}_{\text{total}}$ is the region that covers all the cache and so $S_0(\text{RegInput}_{\text{total}}) = 1$.

5.2 Optimized Product Modeling

The second code used in the validation is shown in Fig. 3. This kernel multiplies a matrix with a uniform distribution of zero entries by another matrix B . As an optimization, when the element of $\mathcal{A}$ to be used in the current product is 0, the operation is not performed. This way two arithmetic operations and one data load are avoided.

This code comprises three different references. Considering the assumptions described in the previous example, the behavior of references $\mathcal{C}(\mathcal{I}, \mathcal{J})$ and $\mathcal{A}(\mathcal{I}, \mathcal{K})$ could be modeled following [3]. Thus we will devote our explanation to the analysis of $\mathcal{B}(\mathcal{K}, \mathcal{J})$.

In the innermost level, level 2, the loop variable indexes the reference of the condition, so the CDRF formula must be applied. As $S_{R2} = 1$, $L_{R2} = 1 + \lfloor (N - 1)/L_s \rfloor$, $G_{R2} \simeq L_s$ and $p_2 = p$, then the formulation is

$$F_2(R, S(\text{RegInput}), p) = (1 + \lfloor (N - 1)/L_s \rfloor) TWR(p, S(\text{RegInput}), 2, L_s, p) . \quad (11)$$

This loop is in the innermost level. Thus, $F_3(R, \text{RegInput}, p) = pS_0(\text{RegInput})$. In this case the calculation of WR (4) requires

$$S(\text{Reg}(B, 2, i)) = S_{l_{\text{auto}}}(i, p_{\text{line}}) \cup S_r(i, 1, M) . \quad (12)$$

The first term represents the autointerference of **B**, which is due to the access to i consecutive elements with a uniform probability of access per cache line of **B** of p_{line} . The second term corresponds to the access to i elements of **A** that belong to different columns, each column having a size of M elements. In general, $S_r(g, s, d)$ calculates the area vector associated to the access to g groups of size s separated by d elements.

In the next level, level 1, the loop variable indexes the reference in the condition, so the CIRF formula is to be applied. Let $L_{R1} = P$, the formulation is

$$F_1(R, S(\text{RegInput}), p) = PF_2(R, S(\text{RegInput}), p) . \quad (13)$$

Also $p_1 = 1 - (1 - p)^{L_s}$. In the outermost level the loop variable indexes the reference of the condition. As a result, the CDRF formula is to be applied again. Being $S_{R0} = 0$, $L_{R0} = 1$, $G_{R0} = M$ and $p_0 = p_1$, the formulation is

$$F_0(R, S(\text{RegInput}), p) = TWR(p_0, S(\text{RegInput}), 0, M, p) . \quad (14)$$

We need to know the value of the accessed regions $\text{Reg}(B, 0, i)$ to compute WR :

$$S(\text{Reg}(B, 0, i)) = S_{l_{\text{auto}}}(P * N, p_{\text{line}}) \cup S_r(N, i, M) \cup S_r(P, i, M) . \quad (15)$$

The first term is associated to the autointerference of **B**, which is the access to $P * N$ consecutive elements with an uniform probability of access to each line of p_{line} . The second term represents the access to i consecutive elements from each one of the N columns of matrix **A**, which have a size of M elements each. The third term represents the access to i consecutive elements from each one of the P columns in **C**, which also have size M .

5.3 Validation Results

Our validation is based on the comparison of the predictions of the model with the results of trace-driven simulations. Different cache configurations, problem sizes and probabilities for the conditionals were used in our experiments.

Table 1 and Table 2 show the validations results for the codes in Figs. 2 and 3, respectively. The first three columns contain the problem size as well as

Table 1. Validation data for the code in Fig. 2 for several cache configurations and different problem sizes and condition probabilities

M	N	p	C_s	L_s	K	Δ_{MR}	Δ_{NM}	σ	$T_{simulation}$	$T_{execution}$	$T_{modeling}$
50000	47500	0.2	65536	16	2	0.372	5.067	5.515	141	60	0.005
50000	47500	0.6	65536	16	8	0.001	0.021	0	262	74	0.004
50000	47500	0.2	8192	32	4	0.004	0.094	0	138	50	0.005
50000	47500	0.4	16384	8	2	0.015	0.086	0	182	68	0.005
50000	47500	0.8	16384	8	2	0.001	0.012	0	255	67	0.004
22000	14500	0.4	32768	16	4	0.001	7.010	7.375	28	7	0.003
22000	14500	0.2	16384	8	4	0.239	1.260	0.144	21	6	0.005
22000	14500	0.9	16384	8	16	0.005	0.041	0	50	7	0.003
22000	14500	0.4	8192	8	1	0.067	0.381	0	65	7	0.004
22000	14500	0.4	8192	32	2	0.007	0.165	0	22	8	0.004
22000	14500	0.7	8192	32	8	0.007	0.206	0	31	7	0.004
18000	22000	0.2	32768	16	2	0.574	8.051	8.326	23	7	0.004
18000	22000	0.6	32768	16	4	0.341	4.489	3.963	40	10	0.005
18000	22000	0.1	16384	8	2	0.076	0.431	0.383	22	6	0.004
18000	22000	0.8	16384	8	8	0	0	0	52	8	0.004
18000	22000	0.3	4096	32	4	0.141	0.417	0	95	8	0.004
14500	19500	0.7	65536	8	8	0	0.032	0	32	7	0.005
14500	19500	0.2	16384	4	2	0.252	0.790	0.766	20	5	0.005
14500	19500	0.3	8192	4	1	0.124	0.366	0	20	6	0.004
14500	19500	0.8	8192	4	4	0.009	0.032	0	43	6	0.004
1750	1750	0.4	8192	4	8	0	0.108	0	1	1	0.003
1750	1750	0.7	8192	8	4	0	0.230	0	0	0	0.003
950	1150	0.4	1024	4	8	0.349	1.046	0	0	0	0.001
950	1150	0.2	4096	8	16	0	0.389	0	0	0	0.001
850	1200	0.6	1024	16	4	0	0	0	0	0	0

the probability p that the condition is fulfilled. Then the cache configuration is given by C_s , the cache size, L_s , the line size, and the degree of associativity of the cache, K . The sizes are measured in elements of the arrays used in the codes.

Two metrics have been used in order to study the accuracy of the model. One of them, Δ_{MR} , is based on the miss rate (MR). It stands for the absolute value of the difference between the predicted and the measured miss rate. We also use Δ_{NM} , which expresses the error in the prediction of the number of misses as a percentage of the number of misses measured by the trace-driven simulation. The tables also include σ , the typical deviation of the number of misses measured expressed as a percentage of the average number of misses measured. For every combination of a cache configuration and a data input, 25 different simulations have been made, using different base addresses for the data structures in each of those simulations. The usage of the overlapping coefficients helps adapt the model prediction to the variability of the cache behavior that is due to the different relative positions of the data structures.

Table 2. Validation data for the code in Fig. 3 for several cache configurations and different problem sizes and condition probabilities

M	N	P	p	C_s	L_s	K	Δ_{MR}	Δ_{NM}	σ	$T_{simulation}$	$T_{execution}$	$T_{modeling}$
1700	1600	1250	0.2	32768	16	2	0.010	0.039	0.037	331	162	0.035
1700	1600	1250	0.4	16384	32	2	0	0	0	372	197	0.041
1700	1600	1250	0.6	16384	32	16	0	0	0	1047	210	0.214
1700	1600	1250	0.2	8192	8	4	0.017	0.018	0	342	162	0.023
1700	1600	1250	0.8	8192	8	4	0	0	0	715	199	0.023
1000	850	900	0.3	8192	8	4	0.007	0.038	0.019	83	40	0.016
1000	850	900	0.8	4096	4	8	0.033	0.047	0	206	45	0.017
1000	850	900	0.2	4096	4	1	0.068	0.085	0.015	64	36	0.013
1000	850	900	0.3	4096	8	1	0.054	0.074	0.004	70	40	0.013
1000	850	900	0.7	4096	8	2	0.017	0.026	0	122	46	0.014
900	850	900	0.1	65536	8	1	0.065	0.525	0.006	48	29	0.020
900	850	900	0.9	65536	8	8	0.015	0.233	0	107	38	0.024
900	850	900	0.2	16384	32	2	0.055	0.064	0	63	32	0.026
900	850	900	0.8	16384	32	2	0.036	0.064	0	104	39	0.024
750	750	1000	0.4	32768	4	2	0.040	0.260	0	56	31	0.019
750	750	1000	0.2	16384	8	4	0.114	0.633	0.568	57	26	0.016
750	750	1000	0.4	8192	16	1	0.147	0.210	0.005	56	32	0.020
750	750	1000	0.8	8192	16	16	0.064	0.109	0	180	32	0.054
200	250	150	0.8	16384	4	2	0.114	0.810	0.020	1	0	0
200	250	150	0.3	4096	4	8	0.113	0.759	0	1	0	0
200	250	150	0.8	2048	4	2	0.348	1.257	0.468	1	0	0
200	250	150	0.1	1024	8	4	0.139	0.143	0	1	0	0.01
100	350	90	0.8	4096	4	8	0.077	0.547	0	1	0	0
100	350	90	0.8	1024	4	8	0.077	0.111	0	1	0	0.01
100	350	90	0.4	2048	8	4	0.141	0.176	0	0	0	0.01

The model provides a good estimation of the caches behavior, as the tables show. The prediction error is smaller or almost equal than the typical deviation introduced by the own variability of the number of misses of the code. We can observe that when the cache works well, this is, when it is large enough to hold the program data structures, the typical deviation is much greater and so the error in the prediction is greater too, although it is smaller or similar than the typical deviation.

Although the model only considers one cache at a time, it is straightforward to predict the behavior of a whole memory hierarchy using it. The model can be applied separately to each level of the hierarchy and then combine these partial results to obtain a good prediction of the behavior of the whole memory system. Some of our experiments in [3] prove this.

The three last columns in Tables 1 and 2 show the simulation, source code execution, and modeling times (in seconds) measured in a 800 MHz Pentium III system for our two example codes, respectively. As we see, modeling times are several orders of magnitude shorter than trace-driven simulation and even

execution times. The modelling time does not include the time required to build the formulas for the example codes. This will be made automatically by the tool we are currently developing. According to our experience in [3], the overhead of such tool is negligible.

6 Related Work

There are a number of previous works that also try to study and improve the behavior of the memory hierarchy by means of analytical models based on the structure of the code. Among those works we find [7], which is restricted to the modeling of direct-mapped caches and that lacks an automatic implementation. Later, [8] and [9] overcame some of these limitation. Cache Miss Equations (CMEs) are constructed in [8], which are lineal systems of Diophantine equations, where each solution corresponds to a potential miss cache. One of its main limitations is its high computational cost. The computing times required by [9] are much shorter, and similar to those ones of our model, however, the error is larger than that of our model. Both models in [8] and [9] share the limitation that they are only suitable for regular access patterns found in perfectly nested loops, and they do not take into account the possible reuses in structures that have been accessed in previous loops. This is a very important subject, as most misses in numerical codes are inter-nest [10], which implies that optimizations should consider several nests.

More recently, [4] and [5] allow the analysis of not perfectly nested loops and consider the reuse between loops in different nests. The former is based on Presburger formulas and provides very accurate estimations for small kernels but it can only handle modest levels of associativity (for example its validation only considers degrees of associativity one and two), and it is very time-consuming, in fact, running a simulation is much faster than solving the equations this model generates, which reduces its applicability. As for the latter, it is based on the extension of [11] in order to quantify the reuse, and it applies the CMEs of [8] in order to estimate the number of misses. The time it requires to solve the CMEs is reduced considerably by applying statistical techniques that allow to provide a prediction within a confidence interval. This model can analyze complete programs, imposing the conditions that the accesses follow regular patterns and that the codes do not contain input data dependent constructions, neither in the loop conditions nor in the conditional sentences. The model precision is similar to that of ours in most of the cases, however its computing times are longer.

Unlike our model, all these approaches require knowing the base addresses of the data structures. This restricts their scope of application, as these addresses are not available in many situations (physically-addressed caches, dynamically allocated data structures, ...). Besides, none of them can model codes with data dependent conditions. Indeed, it is the probabilistic nature of our model what allows us to consider this broad scope of codes.

7 Conclusions and Future Work

An extension to the model in [3] has been presented which allows the analysis of codes with data dependent conditional sentences whose conditions follow an uniform distribution. No other model in the bibliography can estimate the cache behavior of this kind of codes. A validation using simple codes has proved the model estimation to be very accurate despite the very short time required to compute it.

Our model is very suitable to guide the optimization process of a compiler and to help programmers understand the behavior of their codes. In the field of embedded systems the study of the behavior of the memory hierarchy is not only relevant in order to reduce the execution times and the energy and power consumption, but also to calculate the WCET (*Worst Case Execution Time*). Studying the application of our model to this latter usage is part of our future work.

We are currently working in an automatic implementation that applies our model automatically and transparently to the programmer on a great variety of codes. We plan to use the Polaris [12] compiler framework as platform for this purpose, although the model can be coupled with any other front-end and used to model codes written in any programming language. We believe that the probabilistic model proposed here is suitable for the modeling of other kinds of codes, like those that contain irregular access patterns due to the use of indirects and pointers. These codes have been largely ignored in all the previous bibliography despite being common in scientific and engineering applications. Some problems related to the modeling of this kind of codes that we anticipate include getting the distribution of the accesses, mapping the real distribution to one that can be modeled. The more irregular the distribution is, the bigger the mathematical complexity of the associated model so we should try to minimize the corresponding modeling time. All of these problems are to be solved first manually. Then a way to characterize each step and develop a method to apply it automatically is to be found.

References

1. Uhlig, R., Mudge, T.: Trace-Driven Memory Simulation: A Survey. *ACM Computing Surveys* **29** (1997) 128–170
2. Ammons, G., Ball, T., Larus, J.R.: Exploiting Hardware Performance Counters with Flow and Context Sensitive Profiling. In: *SIGPLAN Conference on Programming Language Design and Implementation*. (1997) 85–96
3. Fraguera, B.B., Doallo, R., Zapata, E.L.: Probabilistic Miss Equations: Evaluating Memory Hierarchy Performance. *IEEE Transactions on Computers* **52** (2003) 321–336
4. Chatterjee, S., Parker, E., Hanlon, P., Lebeck, A.: Exact Analysis of the Cache Behavior of Nested Loops. In: *Proc. of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation (PLDI 2001)*. (2001) 286–297

5. Vera, X., Xue, J.: Let's Study Whole-Program Behaviour Analytically. In: Proc. of the 8th Int'l Symposium on High-Performance Computer Architecture (HPCA8). (2002) 175–186
6. Gannon, D., Jalby, W., Gallivan, K.: Strategies for Cache and Local Memory Management by Global Program Transformation. *Journal of Parallel and Distributed Computing* **5** (1988) 587–616
7. Temam, O., Fricker, C., Jalby, W.: Cache Interference Phenomena. In: Proc. Sigmetrics Conference on Measurement and Modeling of Computer Systems, ACM Press (1994) 261–271
8. Ghosh, S., Martonosi, M., Malik, S.: Cache Miss Equations: A Compiler Framework for Analyzing and Tuning Memory Behavior. *ACM Transactions on Programming Languages and Systems* **21** (1999) 702–745
9. Harper, J.S., Kerbyson, D.J., Nudd, G.R.: Analytical Modeling of Set-Associative Cache Behavior. *IEEE Transactions on Computers* **48** (1999) 1009–1024
10. McKinley, K.S., Temam, O.: Quantifying Loop Nest Locality Using SPEC'95 and the Perfect Benchmarks. *ACM Transactions on Computer Systems* **17** (1999) 288–336
11. Wolf, M.E., Lam, M.S.: A Data Locality Optimizing Algorithm. In: Proc. of the ACM SIGPLAN'91 Conference on Programming Language Design and Implementation. (1991) 30–44
12. Blume, W., Doallo, R., Eigenmann, R., Grout, J., Hoefflinger, J., Lawrence, T., Lee, J., Padua, D., Paek, Y., Pottenger, B., Rauchwerger, L., Tu, P.: Parallel Programming with Polaris. *IEEE Computer* **29** (1996) 78–82

FICO: A Fast Instruction Cache Optimizer

Marco Garatti

STMicroelectronics
marco.garatti@st.com

Abstract. This paper shows the results obtained by FICO, a tool aimed at reducing instruction cache conflict misses. FICO reorders functions without requiring any program execution to gather profiling information. The control flow graph annotated with estimated execution frequencies is the actual input of the algorithm.

The tool has been implemented as a post linking phase in a newly developed state-of-the-art commercial-quality compiler codesigned by STMicroelectronics and Hewlett-Packard for their embedded processor family LX. Experimental results show that FICO can provide a speed-up of about 8% on embedded applications.

1 Introduction

Caches and complex memory hierarchies have been used for a long time with the goal of reducing the gap between processors and memories speed. But the problem keeps becoming more important as it is pointed out by [9], because processors and memories speed grow at very different steps. The problem can be tackled in different ways, using software, hardware or mixed approach. A wide overview of techniques that can be used to improve cache efficiencies is presented in [9]. When a datum is not found in cache a *cache miss* is generated and the hardware is forced to fetch the information from the next level of the memory hierarchy. The goal of cache optimizations is to decrease the frequency of misses or reducing the penalty of the miss. To better characterize and understand caches behavior, misses can be classified according to the following definition [9]:

- *Compulsory.* The very first access to a block *cannot be* in the cache, so the block must be brought into the cache. These are also called *cold start misses* or *first reference misses*
- *Capacity.* If the cache cannot contain all the blocks needed during execution of a program, capacity misses will occur because of blocks being discarded and later retrieved
- *Conflict.* If the block placement strategy is set associative or direct mapped, conflict misses (in addition to compulsory and capacity misses) will occur because a block can be discarded and later retrieved if too many blocks map to its set. These are also called *collision misses* or *interference misses*.

Typical behaviors of these three types of misses in standard applications are given in [9]. A compiler can improve instruction caches effectiveness in two ways:

- decreasing the code size to decrease the effect of capacity misses
- improving the code layout to decrease the effect of conflict misses

FICO is a static software technique that decreases conflict misses in direct mapped caches by recomputing an effective functions layout *without* increasing the global size.

FICO is not radically new compared to other techniques that reduce conflict misses, but it mixes aspects from different algorithms and introduces new heuristics. Moreover it has been implemented into a commercial-quality compiler that is also flexible enough to be used in compiler-research. This compiler targets the LX architecture ([1]). LX is a scalable and customizable VLIW processor technology platform designed by Hewlett-Packard and STMicroelectronics that allows variations in instruction issue width, the number and capabilities of structures, and the processor instruction set. A first implementation within this architecture platform is the ST210, a 250-MHz VLIW developed by STMicroelectronics.

The paper is organized as follow: Section 2 gives an overview of existing techniques for instruction cache misses reduction. Section 3 presents FICO and Section 4 shows experimental results. Eventually Section 5 presents conclusions.

2 Overview of Previous Code Layout Work

Since the impact of cache misses is very important, a lot of work has been done in this area. This brief overview is focused on techniques based on code reordering.

The easiest I-cache optimization is based on the reorder of functions according to their calling relationship. This approach is based on the premise that functions and their callers are likely to be temporally close to each other and hence should be placed so that they do not interfere spatially. The reorder can be guided by profiling information if available. An implementation of this approach is presented in [8].

McFarling developed an intraprocedural approach to compute an optimized program layout [7]. The algorithm constructs a control flow graph with basic blocks, functions and loop nodes. It then tries to partition the graph, concentrating on the loop nodes, so that the height of each partitioned tree is less than the size of the cache. If this is the case, then all the nodes inside the tree can be trivially mapped since they will not interfere with each other in the cache. If this is not the case, then some nodes in the mapping might conflict with others in the cache.

Hwu and Chang use inlining, basic blocks reordering and functions reordering to improve instruction cache performance [6]. This algorithm builds a call graph with weighted call edges produced by profiling. For the procedures reordering, their algorithm processes the call graph depth first, mapping the procedures to address space in depth first order. Their depth-first traversal is guided by the edge weights determined by the profile, where a heavier edge is traversed (laid out) before an infrequently executed one.

Pettis and Hansen also described a number of techniques for improving code layout that include: basic blocks reordering, procedure splitting, and procedure

reordering [10]. The reordering starts with the heaviest executed call edge in the program call graph. The two nodes connected by the heaviest edge will be placed next to each other in the final link order. This is achieved by merging the two nodes into a *chain*. The remaining edges entering and exiting the chain are coalesced. This algorithm uses a *closest-is-best* heuristic, trying to place as close as possible functions that might conflict. This criterion is employed whenever multiple choices during the placement are encountered.

Hashemi, Kaeli, and Calder present an algorithm for functions reordering to reduce the first generation conflicts (mapping conflicts between a parent and a child) [5]. Their algorithm differs from previous ones with respect to three main aspects:

- the call graph is preprocessed and pruned according to its profiling weights. The set of functions is partitioned into *popular* and *unpopular* elements. A function is popular if it is often a caller or a callee
- cache is more accurately modeled. The conflict of two functions is verified by a precise model (that takes into account the actual cache implementation). Instead of considering just functions proximity, the model is used to decide where a function must be placed
- empty spaces are used. When a function is placed it can be inserted at a given distance from its predecessors/successors. This is done whenever this position allows the function not to conflict with its caller-callee. Empty blocks can be later filled with *unpopular* functions.

The algorithm uses colors to track cache lines assigned to a procedure and to check the cost of a placement. Experiments show that this technique is more precise and more effective than Pettis and Hansen approach.

Most of the approaches presented above can work with both real and estimated profiling information.

There are also methods that rely more heavily on profiling information gathered by executing the program. There are two different reasons why this kind of information can be preferable and more effective:

- it is more precise. This is evident if we consider a single run, but as [2] points out, profiling information is usually valid for a wide set of inputs
- it can capture the temporal behavior. Plain profiling information can tell us, for example, that function A calls B 100 times and C 85 times. But the temporal distribution is unknown. Suppose the two functions are invoked within the same loop, one after the other. In this case their conflict is highly undesirable. If the two functions are invoked from different loops, then their conflict can be irrelevant. This observation is one of the key in [4].

Some approaches, like [4], use temporal information obtained from profiling to minimize the number of conflicts. These methods are more powerful than the ones based on the call graph because they have more information. The price is that an execution with significant input data is required.

3 FICO

FICO is a *static* (no program execution required) approach to optimize instruction cache conflict misses. Even if profiling information could lead to a more powerful optimizer, it has been discarded because programmers are usually reluctant to use it (even in embedded systems). The optimization reorders functions in an executable file at link time. The only requirement is that the compiler must generate some extra information about each function. In particular:

- each function call must be annotated with the *local* execution frequency (or an estimation of it). The frequency is local because it says how many times the *callee* is called per *caller* invocation
- each function call has also a number specifying the innermost loop it belongs to. If the call is not part of any loop, then this value is set to -1.

This information is statically generated by the compiler and stored in the object file in an *ad hoc* section. The instruction cache optimization process can be split in 6 different steps:

1. the global call graph is computed. The program call graph is computed by a linear scan of the instructions in the binary program. Each time a call is found then a new node (if it did not exist) is created and profiling information is fetched from the appropriate section in the binary file. If a function has multiple call points to the same target, then they are merged and form a single arc in the call graph
2. the global call graph is pruned by removing recursive calls. Functions belonging to a recursive cycle in the call graph are given an extra fixed increase of their execution frequency. Cycles are detected with the algorithm presented in [11] and are broken by deleting one of the edge, in particular the algorithm tries to identify the one that goes backward in the call chain. After this phase the call graph is a DAG. This property is required by the next step that otherwise would be unable to compute global frequencies
3. global frequencies are computed. This task is achieved by propagating the local frequencies. For each node N but *main* its global frequency is:

$$Global(N) = \sum_{F \in predecessor(N)} Global(F) \cdot L(F, N) \quad \text{and} \quad Global(main) = 1$$

4. functions reordering method is chosen. The algorithm can compute the functions order using two different algorithms. One fast and less effective is named *Temperature Order Computation*, while the other is named *Coil placement*. If the call graph has a number of edges that exceeds a given threshold, then the first algorithm is used. This heuristic can be overridden by the user that can specify proper flags on the command line

5. the new functions layout is computed
6. the actual placement is performed. This task is performed using the features provided by the BFD library ([3]). The BFD library allows the analysis and manipulation of binary executables.

The following sections describe the functions reordering methods.

3.1 Temperature Order Computation

This algorithm is very simple. Functions are placed in the order of their global execution frequency. All the most executed functions are placed close to each other. In this case the call graph is not really taken into account, just the execution frequencies are. The idea behind this approach is that *hot* functions (most executed ones) conflict less if they are placed close to each other. This method is very imprecise because it does not model the cache and it does not take into account the call graph. But experimental results show that in general it improves the quality of the functions placement, providing a speed-up.

3.2 Coil Placement

This is the heart of FICO, because it contains the actual and more sophisticated algorithm for computing the functions order. The algorithm takes the pruned call graph annotated with estimated execution frequencies as input. In particular each node has the number of times it is executed ($G(F)$) (entered, as it represents a function) and each edge has the number of times it is traversed per single function invocation ($L(F_1, F_2)$). The goal of the algorithm is to decrease conflict misses by computing a new functions layout that does not increase the total program size.

The first action performed is the computation, for each node in the graph, of its *interesting* neighbors. For a given function F the set of its interesting neighbors ($IN(F)$) is the set of functions that should not conflict with F . The idea is that $IN(F)$ contains the functions that, being close to F in the call graph, should not conflict with it. Of course, in general, this set is composed by all the functions that, at run-time, can cause a conflict miss with F . Since this information is not available to any static technique, an heuristic approach is used to compute an approximation of $IN(F)$. Functions that are close to F in the call graph are potential conflicting elements. This is a generalization of the first level conflict concept used in other approaches (e.g. [5]). Instead of just considering the parent-child relationship, the algorithm considers more relatives. $IN(F)$ can include children, grandchildren, parents, grandparents, cousins and so on. Big $IN(F)$ s increase the precision but slow down the placement, so there's an heuristics to tune the size of the sets based on the call graph size. Figure 2 shows an example of call graph with the neighbors of F_4 in gray.

The size of neighborhoods is a critical parameter of the algorithm, because it is the primary mean to trade effectiveness with time needed to perform the optimization. Experiments showed that in many cases non first level conflicts can

be the main source of conflicts. Therefore the interesting neighbor sets should be as big as possible. On the other hand in big programs, where the number of functions can be in the order of thousands, big neighbor sets make the algorithm too slow. The algorithm used in these experiments has adaptive neighbor sets, where the size of these sets is computed according to the size of the call graph.

For each node N we can define the set of interesting neighbors as the union of three subsets:

- parenthood. Nodes reachable from N following the parent relationship. Starting from N the algorithm follows all the parents whose distance from N is less or equal to n_1 (algorithm parameter)
- childhood. Nodes reachable from N following the child relationship. The maximum distance is specified by n_2
- brotherhood. Nodes reachable from N following the brother relationship. Instead of stopping at the children level, the algorithm considers also cousins and their children with a maximum distance n_3 (a distance of 1 corresponds to nodes having at least one common father)

Figure 1 shows the settings used in the experiments. These values were chosen because they are a good trade off between precision and speed.

Number of edges	n1	n2	n3
(3000,...]	1	1	1
(2000,3000]	1	3	2
(1000,2000]	3	5	4
(0,1000]	4	6	5

Fig. 1. Interesting neighbor sets size

Each neighbor N of a node F has a cost associated that expresses the penalty given by F and N conflicting. Let F_1 and F_2 be two functions that in the layout being computed are going to conflict, then $W(F_1, F_2)$ represents this cost. An initial version (refined later) of the formulas to compute these costs is:

- let F_a be in the parenthood of F_b and let $P = F_a F_1 F_2 \dots F_n F_b$ be one valid path in the call graph. The cost of F_a, F_b conflicting (along P) is:

$$W_p(P) = G(F_a) \cdot L(F_a, F_1) \cdot \min \left[1, L(F_n, F_b) \cdot \prod_{i=1}^{n-1} L(F_i, F_{i+1}) \right]$$

For example in figure 2 $W_p(F_3, F_4)$ is 40 while $W_p(F_1, F_4)$ is 10. F_3 and F_4 can conflict 40 times, while F_1 and F_4 can conflict only 10 times. The idea behind the formula can be explained on the example. If F_1 and F_4 conflict,

then the maximum number of times they will conflict is given by how many times F_1 calls F_3 . Even if F_3 calls many times F_4 the number of conflicts between F_1 and F_4 is bounded by 10. The only exception is if $L(F_3, F_4)$ is smaller than 1. In this case the number of conflicts can be less than 10, and this is the reason why the formula contains the *min*

- let F_a be in the childhood of F_b and let $P = F_b F_1 F_2 \dots F_n F_a$ be a valid path in the call graph. The cost of F_a, F_b conflicting (along P) is:

$$W_c(P) = G(F_b) \cdot L(F_a, F_1) \cdot \min \left[1, L(F_n, F_b) \cdot \prod_{i=1}^{n-1} L(F_i, F_{i+1}) \right]$$

In the example $W_c(F_4, F_6)$ is 120, because F_6 is called 120 times by F_4 . But the same number of possible conflicts holds for the pair $F_4 F_7$, even if the edge from F_6 to F_7 weights 2.

- let F_a be in the brotherhood of F_b and F_c be the only common father of F_a and F_b . The cost of F_a, F_b conflicting is:

$$W_b(F_b, F_c, F_a) = G(F_c) \cdot \min [L(P_c, P_a), L(P_c, P_b)]$$

The algorithm actually also considers nephew. Let F_d be a son of F_b , then $W_b(F_a, F_c, F_b, F_d) = W_b(F_a, F_c) * \min[1, L(F_b, F_d)]$. The rule can easily be generalized to any nephew at any distance. In the example $W_b(F_4, F_5)$ is 40, because the two functions can conflict at most 40 times. If two brothers have more than one common father, then all the father contributions must be taken into account.

These basic rules to compute the cost of two conflicting functions are actually refined using two other heuristics:

- each cost has a weight that decreases as the distance between the two functions increases. The coefficient can be computed as: $C = K^{d-1}$ where K is a constant experimentally computed, while d is the distance between the two nodes. The distance measures the number of nodes that must be traversed to reach the target starting from the source. In figure 2 $d(F_1, F_3)$ is 1, $d(F_1, F_4)$ is 2 and so on. $d(F_4, F_5)$ is 2. $d(F_4, F_6)$ is 3 in the brotherhood and 1 in both the parenthood and childhood.
- the cost associated to brothers can be further corrected. Each function call has an attribute that specifies the loop *id* the call belongs to. If the function call is not in a loop, then this value is set to -1. If two brothers are called in different loops, then their conflict is less important than if they are called within the same loop. Figure 3 shows an example. In function $f1$ a conflict

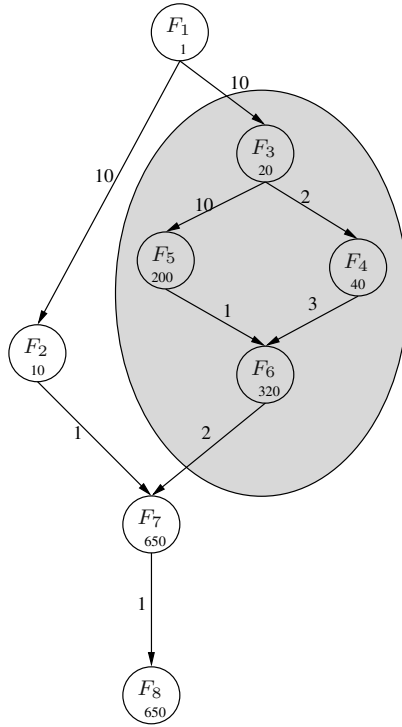


Fig. 2. Section of an annotated call graph. The number on the edge represents the local frequency, while the one inside the node is the global. This is just a section of a call graph, and this why, for example, the frequency of F_3 is not 10. Grey shaded nodes represent the interesting neighbors of F_4

between f_{oo} and bar can be generated at each loop iteration (for each invocation of f_1). In f_2 the two functions f_{oo} and bar can conflict one time for each invocation of f_2 , therefore in this case $W(f_{oo}, bar)$ must be lower than the previous one. The weight is scaled by a constant factor named L_c .

Table 4 shows some weights for the partial call graph of figure 2 with K set to 0.6 and L_c set to 0.8 (assuming F_2 and F_3 are invoked within different loops).

After the computation of interesting neighbors has been performed, the algorithm can determine the actual functions layout. Functions are placed in memory in the order of the new layout. Memory is modeled as a sorted list of blocks, where each block can represent:

- a function that has already been placed. Blocks of this type have the following attributes:
 - function name and size
 - an offset that represents the starting position of the function in memory

```
void f1(int n)
{
    for(i=0;i<n;i++) {
        foo();
        ...
        bar();
        ...
    }
}

void f2(int n)
{
    for(i=0;i<n;i++) {
        foo();
        ...
    }
    for(i=0;i<n/2;i++) {
        bar();
        ...
    }
}
```

Fig. 3. Example of loop heuristics criterion

Parenthood	Childhood	Brotherhood
$W_p(F_3, F_1) = 10$	$W_c(F_1, F_3) = 10$	$W_b(F_2, F_1, F_3) = 8$
$W_p(F_2, F_1) = 10$	$W_c(F_1, F_2) = 10$	$W_b(F_4, F_3, F_5) = 40$
$W_p(F_5, F_3) = 200$	$W_c(F_3, F_5) = 200$	$W_b(F_4, F_3, F_1, F_2) = 6$
$W_p(F_4, F_3) = 40$	$W_c(F_3, F_4) = 40$	
$W_p(F_4, F_1) = 6$	$W_c(F_1, F_4) = 6$	

Fig. 4. Examples of interesting neighbors cost

– an empty block that may be used to place new functions. An empty block, that lays between two functions or at the memory boundaries, has a *maximum* size. The actual size of an empty block is zero (we want to be able to eliminate them at the end), but in case of need it can be extended up to the maximum size. Empty blocks can be thought as coils with a maximum extension.

The step of functions placement is performed with a greedy approach. First of all the call graph edges are sorted according to their execution frequency. Then each edge (source and target functions) is placed in memory.

At this point the algorithm begins to place functions. Call graph edges have already been sorted according to their execution frequency. At the beginning least important functions are chosen and placed. For each edge, the caller-callee are placed close to each other (unless one of the two has already been placed). Figure 5(a) shows an example. Each pair of functions is divided by a *coil*, that is an empty block where functions can be placed later. Each coil has a maximum extension (maximum size of the empty block) that is set to infinite at this point.

After this initial step the algorithm begins the placement of the most frequently executed edges. The first one is taken and the caller-callee functions are placed close to each other. The algorithm adds these two functions to the mem-

ory map and wraps them with two *empty blocks* (or more properly, two coils) around.

The next most executed edge is then chosen. Let F_1 be the caller and F_2 the callee, and let's suppose that none of the two has already been placed. The algorithm scans the memory layout checking all the coils big enough to accommodate F_1, F_2 , to determine the best one for placing them. For each coil that is candidate to host the two functions, a placement cost is computed. This cost is proportional to the number of neighbors of F_1 and F_2 that conflict if this coil is chosen as destination. In the example (figure 2) the second most executed edge is $F_6 \rightarrow F_7$. In the example function F_7 has already been placed, so the algorithm must decide only the placement for F_6 . Both the coils are suitable for it. A placement cost is computed using the following formula:

$$C(F_a, x) = \sum_{F_b \in \text{neighbors}(F_a)} W(F_a, F_b) * \text{conflicts}(F_a, F_b, x)$$

where *conflicts* is a function that returns 1 iff F_a conflict with F_b when F_a is placed at the empty block x . Otherwise the function returns 0. This function depends on the instruction cache that is in use and must be kept synchronized with the hardware architecture. This is the only part of the overall algorithm that is *target dependent*. The empty block with the minimum placement cost is chosen. It is possible, during the placement, that an edge is placed and none of its target and source interesting neighbors has been placed yet. In this case the placement can be performed in any available coil, and the two functions can be wrapped by two empty blocks. In this case new empty blocks can be created in the middle of an existing layout. Figure 5(b) shows a possible situation where 4 functions have already been placed. Let $F_4 \rightarrow F_5$ be the next edge to be placed, ϵ_3 the most convenient coil (where F_5 will be finally placed) and F_1 the only interesting neighbor for F_5 . Let also ϵ_3 be a position that makes F_5 and F_1 non conflicting. For this to hold, the two functions must be kept at a maximum distance, otherwise if ϵ_2 is stretched too much they can get in conflict. This is how the maximum size for the empty blocks is computed. Each time a function is placed, all the empty blocks that lay between the function and any of its interesting neighbors are resized. The precise cache model is necessary in this step to compute the exact *conflict* relationship.

3.3 Restrictions

Like any other approach based on the call graph, FICO cannot produce good results if the calling sequences cannot be properly reconstructed with a static analysis. There are basically two situations where this happens:

- indirect calls. FICO works at the executable program level, so complex analysis (such as pointer analysis) is hard to perform to discover the behavior of pointers used to call functions

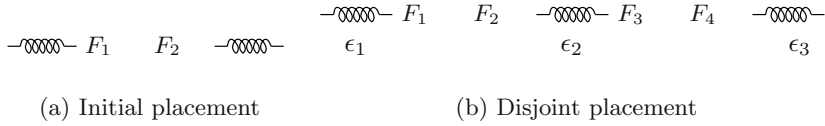


Fig. 5. Example of placements

- calls to the operating system. System calls are usually implemented via traps and not using explicit call instructions. The function to be called is also usually expressed by a value in a register. These mechanisms make impossible, for a static analysis, to determine the call graph.

When at least one of the previous condition holds, FICO has incomplete information and therefore can lead to negative results. The first case can be overcome if the user provides the missing information annotating the source code at the indirect calls. The annotation could include the functions that can be invoked and the probability of each one.

System calls are more difficult to handle. Even if the user could provide information on which function is going to be called, FICO would not have any further information (such as function size and their local call graph). Moreover FICO could not handle these functions as regular functions anyway, because it cannot decide their placement position. The only thing FICO could do is to start from an initial map where these functions position is set and it could not change it. In the embedded domain this problem may or may not be so relevant. Many embedded applications have computational kernels where systems calls are not performed. System calls are usually performed at the beginning and at the end of the real work to read and write data. But this time is usually not dominant, so that a degradation in performance in these parts of code can be acceptable.

4 Experimental Results

The effectiveness of FICO has been measured running a fairly large of benchmarks on LX, a VLIW processor designed by HP and STMicroelectronics [1]. The suite of benchmarks is composed of:

- *Multimedia* benchmarks. Typical of the VLIW targeted market, they include audio, still image and video compression and decompression, cryptography and printing. Most of these benchmarks are proprietary, however they are reported in figure 6 with a concise description.
- *go* taken from SpecInt95 suite. *go* has been chosen because pretty big and because it does not contain indirect calls. The benchmark has been changed

so that system calls are not in the critical loop (printing to the *stdout* is performed only at the end).

benchmark	description	benchmark	description
<i>adpcm</i>	ADPCM audio encoder/decoder	<i>copymark</i>	Color copier pipeline
<i>crypto</i>	ECC, RSA and DES algorithms	<i>csc</i>	color-space conversion
<i>mp2audio</i>	MPEG-1 layer 2 encoder	<i>mp2vloop</i>	MPEG-2 video loop
<i>mp2avswitch</i>	MPEG-2 system layer de-multiplexor	<i>mpeg2</i>	MPEG-2 decoder
<i>mp4dec</i>	MPEG-4 decoder	<i>opendivx</i>	open-source MPEG-4 decoder
<i>tjpeg</i>	JPEG-like encoder/decoder		

Fig. 6. Multimedia benchmarks

The configuration used in this experiments has a direct mapped instruction cache of 32K, with 64 bytes lines. Miss delays have typical values for a complex embedded system where memory must be accessed through a shared bus.

Figure 7 shows the impact of instruction cache on the overall performance (data cache assumed to be perfect, without miss). Misses have been decomposed into the three main components. The left side bars have been obtained without using FICO (the right side bars have been obtained with FICO). In the first case the functions layout is determined by the linking order of the files composing the benchmark. Instruction cache produces an average slow down of 22%, and on average the 12% (more than 50% of the overall slow down) is due to conflict misses. This means that, at most, FICO can decrease the global impact of instruction cache to 10%. When FICO is used, the slow down due to instruction cache is now 7% (it was 12%).

Finally figure 8 shows the speed-up, for each benchmark, that FICO can achieve.

It is interesting to see what happens in *mp2avswitch* and *tjpeg*. These two benchmarks have, in the critical loops, system calls. In *mp2avswitch* the new layout computed by the algorithm is worst than the original one because the call graph is incomplete. Also in *tjpeg* the call graph is incomplete, but the placement is, by chance, better. These two examples have been included to show that in the presence of system calls in the critical loops, the behavior becomes random.

Another interesting benchmark is *mpeg2*, where (although positive) the speed-up achieved by FICO is not close to the maximum. The reason is that there are two functions that are pretty far in the call graph, but they are the main source of conflicts. The heuristics of giving higher conflicting cost to functions that are close is generally a good choice, but in some cases it is not the optimal choice.

All the experiments have used to coil placement. On the same set of benchmarks the temperature placement causes an improvement of about 1% compared

to not using any instruction cache optimizer. This is significantly smaller than what is achieved by FICO. This result might seem poor, but the usage of estimated profiling information makes this approach less effective.

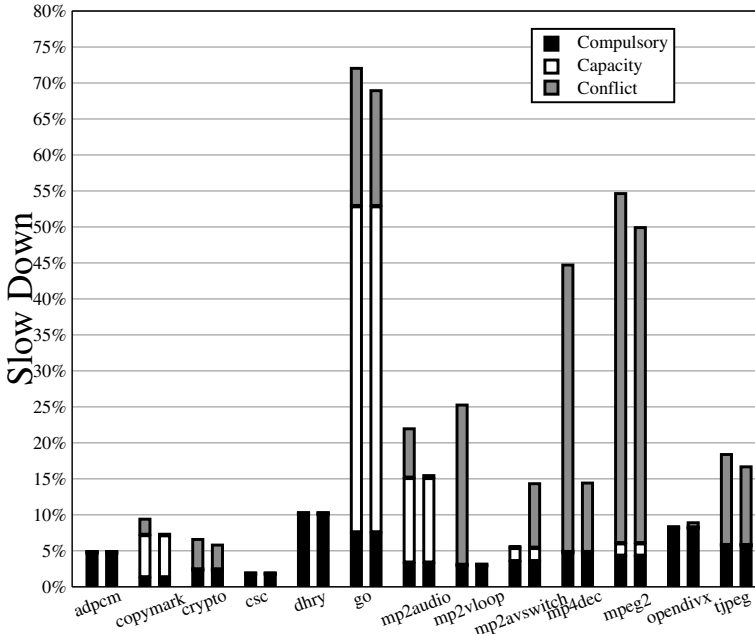


Fig. 7. Slow down given by Instruction Cache

5 Conclusions

The idea of recomputing the program code layout to reduce the number of conflict misses is certainly not novel. As reported in the introduction, a lot of work has already been done in this direction. This work gives two contributions:

- the presented algorithm is a refinement over existing work
- the tool has been tested in a newly developed state-of-the-art commercial quality compiler that targets the LX architecture.

Hwu and Chang [6] traverse the call graph with a DFS guided by edges execution frequency. This is similar to the greedy placement performed by the *coil placement*. The assumption here is that profiling information is very reliable, otherwise we can have important and close functions placed far in the final layout (thus possibly conflicting). Estimated profiling information, although pretty good, cannot guarantee a high degree of precision, therefore the approach can be

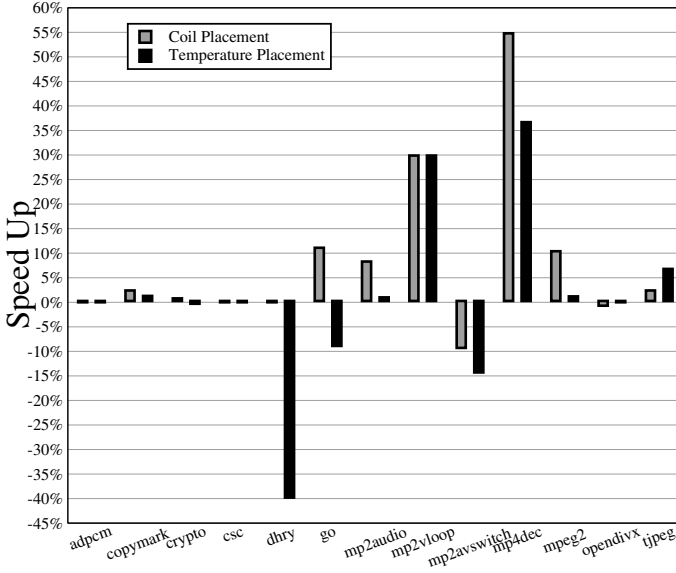


Fig. 8. Speed Up provided by FICO

not so effective if used with this kind of profiling information. Hwu and Chang also works at the basic block level and perform other code transformations, while FICO just recomputes the code layout.

Pettis and Hansen [10] also begins with the most executed edge and place its source and target close to each other. The *closest is best* heuristic is used to place functions. There are some important differences with FICO. First of all FICO has the concept of coils, empty blocks where functions can be placed. The idea is to place as close as possible functions that may conflict, but instead of having a monolithic chain, the memory layout can have blocks where functions may be placed. Since the placement is performed with a greedy strategy it is important to not decide too much at the beginning. Therefore the algorithm decides which functions must not conflict with others, but leaves the possibility to change the layout without affecting previous decisions. Another difference is that FICO at first places non critical functions, so that the memory layout has many holes that can be used later to separate functions that need to be distant. FICO also has a precise cache model to determine exactly if two functions conflict or not, while [10] does not have it. Last difference is that FICO tries to minimize a wide set of conflicts, not only first level, but it considers a parenthood that includes parents, children, brothers and so on.

Hashemi and others [5] presented an approach that is similar to FICO. They use empty blocks as FICO, but they place them to separate functions that would conflict otherwise. These empty blocks may be filled with *unpopular* functions later, but this could end in having holes in the final layout, therefore the opti-

mized program could be bigger than the initial one. This situation cannot happen in FICO. Another important difference is that FICO does not consider only first level conflicts, but it consider a wider neighborhood.

5.1 Future Work

Heuristics can be further refined. For example the detection of unpopular functions is performed using a given threshold. An adaptive threshold could be determined analyzing the call graph frequencies. It would also be interesting to investigate FICO impact on different cache architectures (e.g. associative) and the cache model could be extended to model associative caches.

Acknowledgments

I want to thank all the STMicroelectronics compiler team for the help and support they gave me. In particular Erven Rohou and Roberto Costa. A special thank to Giuseppe Desoli for his wise hints and the tool to perform the actual code reordering.

References

1. Faraboschi, P., Brown, G., Fisher, J.A., Desoli, G., and Homewood, F. Lx: a technology platform for customizable VLIW embedded processing. In *The 27th Annual Int. Symp. on Computer architecture 2000* (2000), ACM Press.
2. Fisher, J., and Freudenberger, S. Predicting conditional branch directions from previous runs of a program. In *Proc. of the Fifth Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS-V)* (1992).
3. Foundation, F.S. Lib bfd, the binary file descriptor library. (<http://www.gnu.org/manual/bfd-2.9.1/bfd.html>).
4. Gloy, N., and Smith, M. D. Procedure placement using temporal-ordering information. *ACM Transactions on Programming Languages and Systems* 21, 5 (1999).
5. Hashemi, A.H., Kalamatianos, J., Calder, B., Kaeli, D., Khalafi, A., and Meleis, W. Cache line coloring using real and estimated profiles.
6. Hwu, W., and Chang, P. Achieving high instruction cache performance with an optimizing compiler. In *Proc. of the 16th Int. Symp. on Computer Architecture* (1989).
7. McFarling, S. Program optimization for instruction caches. In *Proc. of the Third Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS-III)* (1989).
8. Muchnick, S.S. *Advanced compiler design and implementation*. Morgan Kaufmann Publishers, 1997.
9. Patterson, D.A., and Hennessy, J.L. *Computer Architecture: A Quantitative Approach (2nd edition)*. Morgan Kaufmann, San Mateo, CA, 1996.
10. Pettis, K., and Hansen, R. Profile guided code positioning. In *Proc. of the ACM SIGPLAN '90 Conf. on Programming Language Design and Implementation* (1990).
11. T.H. Cormen, L.E. Leiserson, R.R., and Steim, C. *Introduction to Algorithms (2nd edition)*. MIT Press, 2001.

Author Index

- Ahn, Minwook 151
Anagnostakis, Kostas 226
Andrade, Diego 373
Araujo, Guido 285
Ascheid, Gerd 167

Bhatt, Devesh 211
Bhattacharyya, Shuvra S. 344
Braun, Gunnar 167

Casey, Kevin 329
Cattthor, Francky 101, 313
Charitakis, Ioannis 226
Cheung, Warren 17
Corporaal, Henk 2
Cytron, Ron 117

Davidson, Jack W. 33
Decker, Björn 81
Deconinck, Geert 101
Dehnert, James C. 1
Doallo, Ramón 373

Eckstein, Erik 49
Engstrom, Eric 211
Ertl, M. Anton 329
Evans, William 17

Fraguela, Basilio B. 373

Garatti, Marco 388
Gorvindarajan, R. 270
Gregg, David 329

Hiller, Martin 182
Himpe, Stefaan 101
Hiser, Jason 33
Hohenauer, Manuel 167

Imai, Masaharu 66
Ishikawa, Hiroo 198

Jhumka, Arshad 182

Kästner, Daniel 81
Kavi, Krishna 117
Kim, Dae-Hwan 255
Kirner, Raimund 298
Ko, Ming-Yung 344
Kobayashi, Shinsuke 66
König, Oliver 49

Lauwereins, Rudy 313
Lee, Hyuk-Jae 255
Lee, Jaejin 33
Lee, Sheayun 33
Lee, Soonho 151
Leupers, Rainer 167, 285

Markatos, Evangelos 226
Mesman, Bart 2
Meyr, Heinrich 167
Min, Sang Lyul 33
Moses, Jeremy 17

Nakajima, Tatsuo 198
Nie, Xiaoning 167
Nisbet, Andrew 329
Nyström, Sven-Olof 240

Oglesby, David 211
Ottoni, Desiree 285
Ottoni, Guilherme 285

Paek, Yunheung 151
Paško, Robert 313
Pnevmatikatos, Dionisios 226
Puschner, Peter 298

Rijnders, Luc 313
Runeson, Johan 240

Sakanushi, Keishi 66
Sarvani, V.V.N.S. 270
Schloegel, Kirk 211
Scholz, Bernhard 49
Sipkova, Viera 359
Song, Litong 117
Stahl, Richard 313
Suri, Neeraj 182

Takeuchi, Yoshinori 66
Tanaka, Hiroaki 66

Uh, Gang-Ryung 133

Verkest, Diederik 313
Vernalde, Serge 313

Wahlen, Oliver 167

Zhao, Qin 2